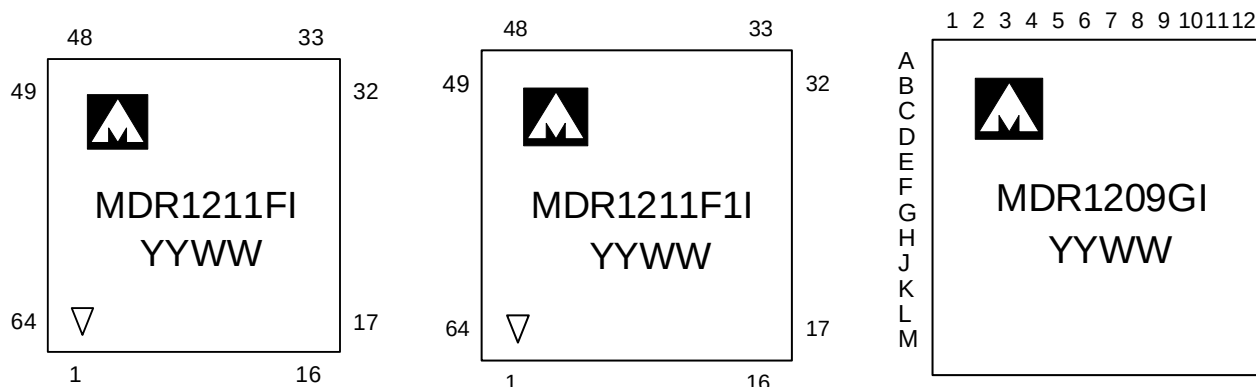




**Микросхема 32-разрядного однокристального
микро-ЭВМ с памятью Flash-типа
K1986BE92FI, K1986BE92F1I, K1986BE94GI**



YY – год выпуска

WW – неделя выпуска

Тип корпуса:

- Микросхемы K1986BE92FI поставляются в 64-выводном пластмассовом корпусе QFN64 9×9×0,75 (0,5);
- Микросхемы K1986BE92F1I поставляются в 64-выводном пластмассовом корпусе QFN64 12×12×1,6 (0,5);
- Микросхемы K1986BE94GI поставляются в 144-выводном пластмассовом корпусе BGA144 16×16×1,6 (1,27)A.

Масса микросхем:

- K1986BE92FI – не более 0,25 г;
- K1986BE92F1I – не более 0,5 г;
- K1986BE94GI – не более 1,1 г.

Уровень чувствительности к влаге MSL:

- K1986BE92FI – 2a;
- K1986BE92F1I – 5a;
- K1986BE94GI – 2a.

Температурный диапазон: от минус 40 до плюс 85 °С.

Важно:

1 Микросхемы чувствительны к влажности. Порядок обращения должен соответствовать требованиям, приведенным в ТСКЯ.430106.004Д12.

2 Спецификация действительна совместно с документом K1986BE92FI, K1986BE92F1I, K1986BE94GI Errata Notice.

Основные характеристики микросхем

Ядро:

– ARM 32-битное RISC-ядро Cortex™-M3 ревизии 2.0, тактовая частота до 80 МГц, производительность 1,25 DMIPS/МГц (Dhrystone 2.1) при нулевой задержке памяти;

- блок аппаратной защиты памяти MPU;
- умножение за один цикл, аппаратная реализация деления.

Память:

– встроенная энергонезависимая Flash-память программ размером 128 Кбайт;

– встроенное ОЗУ размером 32 Кбайт;

– контроллер внешней шины с поддержкой микросхем памяти СОЗУ, ПЗУ, NAND Flash.

Питание и тактовая частота:

– внешнее питание $2,2 \div 3,6$ В;

– встроенный регулируемый стабилизатор напряжения на 1,8 В для питания ядра;

- встроенные схемы контроля питания;
- встроенный домен с батарейным питанием;
- встроенные подстраиваемые RC-генераторы 8 МГц и 40 кГц;
- внешние кварцевые резонаторы на $2 \div 16$ МГц и 32 кГц;
- встроенный умножитель тактовой частоты PLL для ядра;
- встроенный умножитель тактовой частоты PLL для USB.

Режим пониженного энергопотребления:

– режимы Sleep, Deep Sleep и Standby;

– батарейный домен с часами реального времени и регистрами аварийного сохранения.

Аналоговые модули:

- два 12-разрядных АЦП (до 16 каналов);
- температурный датчик;
- двухканальный 12-разрядный ЦАП;
- встроенный компаратор.

Периферия:

– контроллер DMA с функциями передачи Периферия-Память, Память-Память;

– два контроллера CAN интерфейса;

– контроллер USB интерфейса с функциями работы Device и Host;

– контроллеры интерфейсов UART, SPI, I2C;

– три 16-разрядных таймер-счетчика с функциями ШИМ и регистрации событий;

– до 96 пользовательских линий ввода-вывода.

Отладочные интерфейсы:

- последовательные интерфейсы SWD и JTAG.

Значение теплового сопротивления кристалл-корпус:

- K1986BE92FI – не более 3,3 °С/Вт.

Содержание

1	Введение	15
2	Основные характеристики	16
3	Структурная блок-схема микросхемы	17
4	Условные графические изображения	19
5	Описание выводов	21
6	Диаграмма расположения выводов в планарных корпусах	53
7	Указания по применению и эксплуатации	54
8	Система питания	56
8.1	Схема сброса при включении и выключении основного питания	58
9	Организация памяти	60
9.1	Секция CODE	62
9.1.1	Область BOOT ROM	62
9.1.2	Область EEPROM PROGRAM	62
9.1.3	Область EXTERNAL BUS	62
9.2	Секция DATA	62
9.2.1	Область Internal SRAM (Data)	62
9.2.2	Область BIT BAND REGION TO SRAM (Data)	62
9.2.3	Область EXTERNAL BUS	63
9.3	Секция PERIPHERAL	63
9.3.1	Область PERIPHERAL (Data)	63
9.3.2	Область BIT BAND REGION TO PERIPHERAL (Data)	63
9.3.3	Область EXTERNAL BUS	63
9.4	Секция EXTERNAL RAM	63
9.4.1	Область EXTERNAL BUS	63
9.5	Секция SYSTEM	63
9.6	Блок BUS MATRIX	63
9.7	Память BOOT ROM	64
9.8	Память EEPROM	64
9.9	Память SRAM	64
9.10	Регионы памяти, типы и атрибуты	64
9.10.1	Normal	64
9.10.2	Device	64
9.10.3	Strongly-ordered	65
9.10.4	Shareable	65
9.10.5	Execute Never (XN)	65
9.11	Последовательность обращений к памяти	65
9.12	Поведение обращений к памяти	66
9.12.1	Дополнительные условия доступа к совместно используемой памяти	67
9.12.2	Программное упорядочение обращений к памяти	67
9.12.3	Bit-band регионы	69
9.13	Примитивы синхронизации	70
9.13.1	Команда Load-Exclusive	71
9.13.2	Команда Store-Exclusive	71
9.13.3	Указания по программированию примитивов синхронизации	72
9.14	Базовые адреса процессора	72
10	Загрузочное ПЗУ и режимы работы микросхемы	74

10.1	Микроконтроллер в режиме отладки.....	76
10.2	Микропроцессор в режиме отладки.....	76
10.3	UART-загрузчик без отладки	77
10.3.1	Параметры связи по UART.....	77
10.3.2	Протокол обмена по UART	77
10.3.3	Синхронизация с внешним устройством	78
10.3.4	Команда CMD_SYNC	78
10.3.5	Команда CMD_CR.....	78
10.3.6	Команда CMD_BAUD.....	79
10.3.7	Команда CMD_LOAD.....	79
10.3.8	Команда CMD_VFY	80
10.3.9	Команда CMD_RUN.....	80
10.3.10	Прием параметров команды	81
10.3.11	Сообщения об ошибках	81
10.4	Тестовый режим работы.....	81
11	Контроллер Flash-памяти программ MDR_EEPROM.....	83
11.1	Работа Flash-памяти программ в обычном режиме.....	83
11.2	Работа Flash-памяти программ в режиме программирования	84
11.2.1	Стирание всей памяти	85
11.2.2	Стирание банка памяти одной страницы размером 4 Кбайт	85
11.2.3	Запись 32-битного слова в память	86
11.2.4	Чтение 32-битного слова из памяти	87
11.3	Производственная информация	88
11.4	Регистры управления контроллера Flash-памяти программ	89
11.4.1	MDR_EEPROM->CMD	89
11.4.2	MDR_EEPROM->ADR.....	90
11.4.3	MDR_EEPROM->DI.....	91
11.4.4	MDR_EEPROM->DO	91
11.4.5	MDR_EEPROM->KEY	92
12	Процессорное ядро	93
12.1	Программная модель	94
12.2	Стек	95
12.3	Регистры ядра.....	96
12.3.1	Регистры общего назначения R0-R12	97
12.3.2	Указатель стека SP R13.....	97
12.3.3	Регистр связи LR R14.....	97
12.3.4	Счетчик команд PC R15.....	98
12.3.5	Программный регистр состояния PSR	98
12.3.6	Программный регистр состояния приложения APSR	98
12.3.7	Программный регистр состояния прерываний IPSR	99
12.3.8	Программный регистр состояния выполнения EPSR	100
12.3.9	Регистр маски исключений Exception mask.....	101
12.3.10	Регистр маски приоритетов Priority Mask.....	101
12.3.11	Регистр маски сбоев Fault Mask.....	102
12.3.12	Регистр базового приоритета маски Base Priority Mask	102
12.3.13	Регистр управления CONTROL	103
12.4	Исключения и прерывания	103
13	Система команд.....	105
13.1	Встроенные функции.....	108

13.2	Описание инструкций	109
13.2.1	Операнды.....	109
13.2.2	Ограничения на использование PC и SP.....	110
13.2.3	Формат второго операнда.....	110
13.2.4	Операции сдвига.....	111
13.2.5	Выравнивание адресов.....	114
13.2.6	Адресация относительно счетчика команд PC.....	114
13.2.7	Условное исполнение.....	115
13.2.8	Выбор размера кода инструкции	117
13.3	Команды доступа к памяти.....	118
13.3.1	ADR.....	118
13.3.2	LDR и STR, непосредственно заданное смещение	119
13.3.3	LDR и STR, смещение задано в регистре	121
13.3.4	LDR and STR, непривилегированный доступ.....	123
13.3.5	LDR, адресация относительно счетчика команд PC.....	124
13.3.6	LDM и STM.....	125
13.3.7	PUSH и POP	127
13.3.8	LDREX и STREX.....	128
13.3.9	CLREX	130
13.4	Инструкции обработки данных	130
13.4.1	ADD, ADC, SUB, SBC и RSB	131
13.4.2	AND, ORR, EOR, BIC и ORN	133
13.4.3	ASR, LSL, LSR, ROR и RRX	134
13.4.4	CLZ.....	136
13.4.5	CMP и CMN	136
13.4.6	MOV и MVN	137
13.4.7	MOVT	139
13.4.8	REV, REV16, REVSH и RBIT.....	139
13.4.9	TST и TEQ	140
13.5	Инструкции умножения и деления	141
13.5.1	MUL, MLA и MLS	142
13.5.2	UMULL, UMLAL, SMULL и SMLAL	143
13.5.3	SDIV и UDIV	144
13.6	Инструкции преобразования данных с насыщением	144
13.6.1	SSAT и USAT.....	144
13.7	Команды работы с битовыми полями.....	146
13.7.1	BFC и BFI	146
13.7.2	SBFX и UBFX	147
13.7.3	SXT и UXT	148
13.8	Инструкции передачи управления	149
13.8.1	B, BL, BX и BLX.....	149
13.8.2	CBZ и CBNZ.....	151
13.8.3	IT	152
13.8.4	TBB и TBH	154
13.9	Прочие инструкции	155
13.9.1	CPS	155
13.9.2	DMB	156
13.9.3	DSB	156
13.9.4	ISB.....	157

13.9.5	MRS.....	157
13.9.6	MSR.....	158
13.9.7	NOP	159
13.9.8	SEV.....	159
13.9.9	SVC	160
13.9.10	WFE.....	160
13.9.11	WFI.....	161
14	Системный таймер SysTick.....	162
14.1	Описание регистров системного таймера SysTick	162
14.1.1	SysTick->CTRL	162
14.1.2	SysTick->LOAD	163
14.1.3	SysTick->VAL	163
14.1.4	SysTick->CAL	164
14.2	Советы и особенности при применении системного таймера	164
15	Модуль защиты памяти MPU	165
15.1	Описание регистров MPU	166
15.1.1	MPU->TYPE.....	166
15.1.2	MPU->CTRL.....	167
15.1.3	MPU->RNR.....	168
15.1.4	MPU->RBAR	169
15.1.5	MPU->RASR	170
15.1.6	Атрибуты разрешения доступа MPU.....	171
15.1.7	Несоответствие MP	172
15.1.8	Обновление MPU региона	173
15.2	Советы и особенности применения MPU.....	175
15.2.1	Конфигурация MPU для микросхемы	175
16	Сигналы тактовой частоты MDR_RST_CLK.....	177
16.1	Встроенный RC-генератор HSI	178
16.2	Встроенный RC-генератор LSI.....	178
16.3	Внешний генератор HSE	178
16.4	Внешний генератор LSE	178
16.5	Встроенный блок умножения системной тактовой частоты	179
16.6	Встроенный блок умножения USB тактовой частоты	179
16.7	Описание регистров блока контроллера тактовой частоты	179
16.7.1	MDR_RST_CLK->CLOCK_STATUS	180
16.7.2	MDR_RST_CLK->PLL_CONTROL	181
16.7.3	MDR_RST_CLK->HS_CONTROL	181
16.7.4	MDR_RST_CLK->CPU_CLOCK.....	182
16.7.5	MDR_RST_CLK->USB_CLOCK.....	183
16.7.6	MDR_RST_CLK->ADC_MCO_CLOCK.....	183
16.7.7	MDR_RST_CLK->RTC_CLOCK.....	184
16.7.8	MDR_RST_CLK->PER_CLOCK	185
16.7.9	MDR_RST_CLK->CAN_CLOCK.....	186
16.7.10	MDR_RST_CLK->TIM_CLOCK	187
16.7.11	MDR_RST_CLK->UART_CLOCK.....	187
16.7.12	MDR_RST_CLK->SSP_CLOCK.....	188
17	Батарейный домен и часы реального времени MDR_BKP	190
17.1	Часы реального времени	190
17.2	Регистры аварийного сохранения	191

17.3	Описание регистров блока батарейного домена.....	191
17.3.1	MDR_BKP->REG_[0D...00]	192
17.3.2	MDR_BKP->REG_0E	193
17.3.3	MDR_BKP->REG_0F	194
17.3.4	MDR_BKP->RTC_CNT	196
17.3.5	MDR_BKP->RTC_DIV	196
17.3.6	MDR_BKP->RTC_PRL	197
17.3.7	MDR_BKP->RTC_ALRM	197
17.3.8	MDR_BKP->RTC_CS	198
18	Порты ввода-вывода MDR_PORTx	199
18.1	Описание регистров портов ввода-вывода.....	202
18.1.1	MDR_PORTx->RXTX	203
18.1.2	MDR_PORTx->OE	203
18.1.3	MDR_PORTx->FUNC.....	203
18.1.4	MDR_PORTx->ANALOG	204
18.1.5	MDR_PORTx->PULL	204
18.1.6	MDR_PORTx->PD	204
18.1.7	MDR_PORTx->PWR.....	205
18.1.8	MDR_PORTx->GFEN	205
19	Детектор напряжения питания MDR_POWER.....	206
19.1.1	MDR_POWER->PVDCS	206
20	Внешняя системная шина MDR_EBC	209
20.1	Работа с внешними статическими ОЗУ, ПЗУ и периферийными устройствами	209
20.1.1	Обмен по внешней системной шине при задании длительности через WS_ACTIVE, WS_SETUP, WS_HOLD	212
20.2	Работа с внешней NAND Flash-памятью.....	212
20.3	Описание регистров блока контроллера внешней системной шины	216
20.3.1	MDR_EBC->CONTROL.....	217
20.3.2	MDR_EBC->NAND_CYCLES.....	220
20.3.3	MDR_EBC->REGx_CNTRL	220
21	Контроллер интерфейса MDR_USB	222
21.1	Инициализация контроллера при включении.....	222
21.2	Задание параметров шины USB и события подключения/отключения	222
21.3	Задание адреса и инициализация конечных точек	223
21.4	Транзакция IN (USB Device).....	223
21.5	Транзакция SETUP/OUT (USB Device)	225
21.6	Транзакция SETUP/OUT (USB Host)	227
21.7	Транзакция IN (USB Host)	227
21.8	Отправка SOF пакетов и отсчет времени (USB Host)	228
21.9	Описание регистров управления контроллером USB интерфейса	229
21.9.1	MDR_USB->HSCR	231
21.9.2	Регистры HOST режима.....	232
21.9.3	USB Slave (Device)	240
22	Контроллер интерфейса MDR_CAN.....	248
22.1	Режимы работы	249
22.2	Типы пакетов сообщений.....	250
22.3	Структура пакета данных (Data Frame)	251
22.3.1	Начало пакета (Start of frame).....	252

22.3.2	Поле арбитража (Arbitration field).....	252
22.3.3	Поле управления (Control field)	253
22.3.4	Поле данных (Data field)	253
22.3.5	Поле CRC (CRC field)	254
22.3.6	Поле подтверждения (ACK field).....	254
22.3.7	Конец пакета (End of frame)	254
22.3.8	Структура пакета удаленного запроса данных (Remote frame)	254
22.3.9	Арбитраж на шине.....	254
22.4	Инициализация.....	256
22.5	Передача сообщений	256
22.6	Передача сообщений по Remote Transmit Request (RTR).....	257
22.7	Прием сообщений	257
22.8	Автоматическая фильтрация принимаемых сообщений	257
22.9	Перезапись принятых сообщений.....	258
22.10	Задание скорости передачи и момента семплирования.....	258
22.11	Синхронизация.....	259
22.12	Обработка ошибок	260
22.13	Прерывания	263
22.14	Описание регистров контроллера CAN.....	264
22.14.1	MDR_CANx->CONTROL	266
22.14.2	MDR_CANx->STATUS.....	266
22.14.3	MDR_CANx->BITTMNG.....	268
22.14.4	MDR_CANx->INT_EN	269
22.14.5	MDR_CANx->OVER	270
22.14.6	MDR_CANx->BUF_CON[x]	270
22.14.7	MDR_CANx->INT_RX.....	271
22.14.8	MDR_CANx->RX	272
22.14.9	MDR_CANx->INT_TX.....	272
22.14.10	MDR_CANx->TX.....	272
22.14.11	MDR_CANx->RXID MDR_CANx->TXID MDR_CANx->CAN_BUF[x].ID MDR_CANx->CAN_BUF_FILTER[x].MASK MDR_CANx->CAN_BUF_FILTER[x].FILTER.....	273
22.14.12	MDR_CANx->RXDLC MDR_CANx->TXDLC MDR_CANx->CAN_BUF[x].DLC	273
22.14.13	MDR_CANx->RXDATA MDR_CANx->TXDATA MDR_CANx->CAN_BUF[x].DATA	274
22.14.14	MDR_CANx->RXDATAH MDR_CANx->TXDATAH MDR_CANx->CAN_BUF[x].DATAH	275
23	Таймеры общего назначения MDR_TIMERx	276
23.1	Основные характеристики	276
23.1.1	Структурная схема	276
23.2	Базовый блок таймера	277
23.2.1	Инициализация тактирования таймера	278
23.2.2	Инициализация основного счетчика таймера.....	278
23.2.3	Режимы счета	279
23.2.4	Тактовая частота F _{DTS}	281
23.3	Источники событий для счета	282
23.3.1	Внутренний тактовый сигнал (TIM_CLKd).....	283

23.3.2	Событие в другом таймере (CNT==ARR).....	284
23.3.3	Внешний тактовый сигнал, «Режим 1»: событие переднего фронта на входе канала CНуi	285
23.3.4	Внешний тактовый сигнал, «Режим 2»: событие переднего или заднего фронта на входе ETR.....	287
23.4	Режим захвата	288
23.5	Режим ШИМ.....	290
23.5.1	Генератор опорного сигнала REF	290
23.5.2	Генератор «мертвой зоны»	292
23.5.3	Выходные блоки	293
23.6	Блок цифрового фильтра.....	293
23.7	Флаги состояний, прерывания и запросы DMA	295
23.7.1	Флаги состояний.....	296
23.7.2	Прерывания.....	296
23.7.3	Запросы DMA	296
23.8	Примеры	296
23.8.1	Обычный счетчик	296
23.8.2	Режим захвата	297
23.8.3	Режим ШИМ	298
23.9	Описание регистров блока таймера	299
23.9.1	MDR_TIMERx->CNT	301
23.9.2	MDR_TIMERx->PSG.....	301
23.9.3	MDR_TIMERx->ARR.....	302
23.9.4	MDR_TIMERx->CNTRL.....	303
23.9.5	MDR_TIMERx->CCRy	304
23.9.6	MDR_TIMERx->CCRy1	304
23.9.7	MDR_TIMERx->CHy_CNTRL	305
23.9.8	MDR_TIMERx->CHy_CNTRL1	307
23.9.9	MDR_TIMERx->CHy_CNTRL2	308
23.9.10	MDR_TIMERx->CHy_DTG	309
23.9.11	MDR_TIMERx->BRKETR_CNTRL	309
23.9.12	MDR_TIMERx->STATUS.....	310
23.9.13	MDR_TIMERx->IE	312
23.9.14	MDR_TIMERx->DMA_RE	313
24	Контроллер MDR_ADC	315
24.1	Преобразование внешнего канала.....	316
24.2	Последовательное преобразование нескольких каналов.....	316
24.3	Преобразование с контролем границ.....	317
24.4	Внутренний источник опорного напряжения	317
24.5	Датчик температуры.....	318
24.6	Синхронный запуск двух АЦП.....	318
24.7	Время заряда внутренней емкости	319
24.8	Описание регистров блока контроллера АЦП.....	320
24.8.1	MDR_ADC->ADC1_CFG.....	321
24.8.2	MDR_ADC->ADC2_CFG.....	323
24.8.3	MDR_ADC->ADCx_H_LEVEL	325
24.8.4	MDR_ADC->ADCx_L_LEVEL	325
24.8.5	MDR_ADC->ADCx_RESULT	325
24.8.6	MDR_ADC->ADCx_STATUS	326

24.8.7	MDR_ADC->ADCx_CHSEL	326
25	Контроллер MDR_DAC	328
25.1	Описание регистров блока контроллера ЦАП.....	328
25.1.1	MDR_DAC->CFG	328
25.1.2	MDR_DAC->DAC1_DATA	329
25.1.3	MDR_DAC->DAC2_DATA	329
26	Контроллер схемы компаратора MDR_COMP	331
26.1	Сравнение внешних сигналов.....	331
26.2	Сравнение сигнала с внутренним источником опорного напряжения	332
26.3	Сравнение внешних сигналов с внутренней шкалой напряжений	332
26.4	Формирование внутренней шкалы напряжений.....	332
26.5	Описание регистров блока контроллера компаратора.....	334
26.5.1	MDR_COMP->CFG	334
26.5.2	MDR_COMP->RESULT.....	335
26.5.3	MDR_COMP->RESULT_LATCH.....	335
27	Контроллер интерфейса MDR_I2C	336
27.1	Конфигурация контроллера I2C	336
27.2	Конфигурация системы.....	337
27.3	Протокол I2C	337
27.4	Сигнал START	338
27.5	Передача адреса	338
27.6	Передача данных.....	339
27.7	Сигнал STOP	339
27.8	Описание регистров контроллера I2C	339
27.8.1	MDR_I2C->PRL	340
27.8.2	MDR_I2C->PRH.....	340
27.8.3	MDR_I2C->CTR.....	340
27.8.4	MDR_I2C->RXD	341
27.8.5	MDR_I2C->STA	341
27.8.6	MDR_I2C->TXD	342
27.8.7	MDR_I2C->CMD	342
28	Контроллер MDR_SSP	344
28.1	Основные характеристики модуля SSP	344
28.2	Программируемые параметры.....	344
28.3	Характеристики интерфейса SPI.....	346
28.4	Характеристики интерфейса Microwire.....	346
28.5	Характеристики интерфейса SSI.....	346
28.6	Общий обзор модуля SSP	346
28.6.1	Блок формирования тактового сигнала.....	347
28.6.2	Буфер FIFO передатчика.....	347
28.6.3	Буфер FIFO приемника	347
28.6.4	Блок приема и передачи данных.....	347
28.6.5	Блок формирования прерываний	348
28.6.6	Интерфейс прямого доступа к памяти	348
28.6.7	Конфигурирование приемопередатчика	348
28.6.8	Разрешение работы приемопередатчика.....	349
28.6.9	Соотношения между тактовыми сигналами	349
28.6.10	Программирование регистра управления CR0.....	350
28.6.11	Программирование регистра управления CR1	350

28.6.12	Формирование тактового сигнала обмена данными.....	351
28.6.13	Формат информационного кадра	351
28.6.14	Формат синхронного обмена SSI фирмы Texas Instruments	352
28.6.15	Формат синхронного обмена SPI фирмы Motorola	353
28.6.16	Формат синхронного обмена Microwire фирмы National Semiconductor	357
28.6.17	Примеры конфигурации модуля в ведущем и ведомом режимах	360
28.6.18	Интерфейс прямого доступа к памяти	362
28.7	Программное управление модулем.....	364
28.7.1	Общая информация	364
28.7.2	Описание регистров контроллера SSP	364
28.8	Прерывания	370
28.8.1	SSPRXINTR.....	371
28.8.2	SSPTXINTR.....	371
28.8.3	SSPRORINTR.....	371
28.8.4	SSPRTINTR.....	371
28.8.5	SSPINTR	371
29	Контроллер MDR_UART.....	372
29.1	Основные сведения.....	372
29.1.1	Основные характеристики модуля UART.....	372
29.1.2	Программируемые параметры	373
29.1.3	Отличия от контроллера UART 16C650.....	373
29.2	Функциональные возможности	374
29.3	Описание функционирования блока UART.....	375
29.3.1	Генератор тактового сигнала приемопередатчика.....	375
29.3.2	Буфер FIFO передатчика.....	376
29.3.3	Буфер FIFO приемника	376
29.3.4	Блок передатчика.....	376
29.3.5	Блок приемника	376
29.3.6	Блок формирования прерываний	376
29.3.7	Интерфейс прямого доступа к памяти	377
29.3.8	Блок и регистры синхронизации.....	377
29.4	Описание функционирования ИК кодека IrDA SIR.....	377
29.4.1	Кодер ИК передатчика	377
29.4.2	Декодер ИК приемника.....	378
29.5	Описание работы UART.....	379
29.5.1	Сброс модуля	379
29.5.2	Тактовые сигналы.....	379
29.5.3	Работа универсального асинхронного приемопередатчика.....	379
29.5.4	Коэффициент деления частоты	379
29.5.5	Передача и прием данных.....	380
29.5.6	Биты ошибки	381
29.5.7	Бит переполнения буфера.....	381
29.5.8	Запрет буфера FIFO	382
29.5.9	Работа кодека ИК-обмена данными IrDA SIR.....	382
29.5.10	Модуляция данных IrDA	384
29.6	Линии управления модемом	384
29.6.1	Аппаратное управление потоком данных.....	384
29.6.2	Управление потоком данных по линии RTS	385

29.6.3	Управление потоком данных по линии CTS	386
29.7	Интерфейс прямого доступа к памяти	386
29.8	Прерывания	388
29.8.1	UARTMSINTR	389
29.8.2	UARTRXINTR	389
29.8.3	UARTTXINTR.....	389
29.8.4	UARTRTINTR.....	390
29.8.5	UARTEINTR	390
29.8.6	UARTINTR.....	390
29.9	Программное управление модулем.....	391
29.9.1	Общая информация	391
29.9.2	MDR_UARTx->DR.....	392
29.9.3	MDR_UARTx->RSR_ECR	393
29.9.4	MDR_UARTx->FR.....	394
29.9.5	MDR_UARTx->ILPR.....	395
29.9.6	MDR_UARTx->IBRD.....	396
29.9.7	MDR_UARTx->FBRD	396
29.9.8	MDR_UARTx->LCR_H.....	398
29.9.9	MDR_UARTx->CR	400
29.9.10	MDR_UARTx->IFLS	402
29.9.11	MDR_UARTx->IMSC.....	403
29.9.12	MDR_UARTx->RIS	404
29.9.13	MDR_UARTx->MIS	405
29.9.14	MDR_UARTx->ICR.....	406
29.9.15	MDR_UARTx->DMACR.....	406
30	Контроллер прямого доступа в память MDR_DMA	408
30.1	Основные свойства контроллера DMA	408
30.2	Термины и определения	408
30.3	Функциональное описание	409
30.3.1	Распределение каналов DMA	410
30.3.2	Блок, подключенный к шине APB	410
30.3.3	Блок, подключенный к шине AHB	411
30.3.4	Управляющий блок DMA	411
30.3.5	Типы передач	411
30.3.6	Разрядность передач данных	411
30.3.7	Управление защитой данных	412
30.3.8	Инкремент адреса	412
30.4	Управление DMA.....	413
30.4.1	Правила обмена данными	413
30.4.2	Диаграммы работы контроллера DMA	415
30.4.3	Правила арбитража DMA	421
30.4.4	Приоритет.....	421
30.4.5	Типы циклов DMA	423
30.4.6	Индикация ошибок.....	434
30.5	Структура управляющих данных канала	435
30.6	Описание регистров контроллера DMA	445
30.6.1	MDR_DMA->STATUS.....	447
30.6.2	MDR_DMA->CFG	448
30.6.3	MDR_DMA->CTRL_BASE_PTR	449

30.6.4	MDR_DMA->ALT_CTRL_BASE_PTR	449
30.6.5	MDR_DMA->WAITONREQ_STATUS.....	450
30.6.6	MDR_DMA->CHNL_SW_REQUEST	450
30.6.7	MDR_DMA->CHNL_USEBURST_SET.....	451
30.6.8	MDR_DMA->CHNL_USEBURST_CLR.....	452
30.6.9	MDR_DMA->CHNL_REQ_MASK_SET	452
30.6.10	MDR_DMA->CHNL_REQ_MASK_CLR.....	453
30.6.11	MDR_DMA->CHNL_ENABLE_SET	454
30.6.12	MDR_DMA->CHNL_ENABLE_CLR.....	454
30.6.13	MDR_DMA->CHNL_PRI_ALT_SET	455
30.6.14	MDR_DMA->CHNL_PRI_ALT_CLR	456
30.6.15	MDR_DMA->CHNL_PRIORITY_SET.....	457
30.6.16	MDR_DMA->CHNL_PRIORITY_CLR.....	458
30.6.17	MDR_DMA->ERR_CLR.....	459
31	Прерывания и исключения	460
31.1	Типы исключений	460
31.1.1	RESET	460
31.1.2	NON MASKABLE INTERRUPT (NMI).....	460
31.1.3	Hard Fault	460
31.1.4	Memory Management fault	460
31.1.5	Bus Fault.....	461
31.1.6	Usage Fault.....	461
31.1.7	SVCall	461
31.1.8	PendSV	461
31.1.9	SysTick	461
31.2	Прерывания (IRQ).....	461
31.3	Обработчики исключений.....	462
31.4	Таблица векторов	463
31.5	Приоритеты исключений	463
31.5.1	Группировка приоритетов прерываний	464
31.6	Вход в обработчик и выход из обработчика	464
31.6.1	Приоритетное прерывание	464
31.6.2	Возврат.....	464
31.6.3	Передача управления без восстановления контекста (tail-chaining)	465
31.6.4	Запоздавшее исключение (late-arriving exception)	465
31.6.5	Вход в процедуру обработки исключения	465
31.6.6	Возврат из обработчика исключения	466
31.7	Обработка отказов	467
31.7.1	Типы отказов	468
31.7.2	Эскалация отказов и тяжелые отказы.....	469
31.7.3	Регистры состояния и адреса отказа.....	470
31.7.4	Блокировка	470
31.8	Управление электропитанием	470
31.8.1	Переход в режим пониженного энергопотребления.....	471
31.8.2	Ожидание прерывания	471
31.8.3	Ожидание события	471
31.8.4	Переход в режим ожидания по выходу из обработчика исключения (режим Sleep)	471
31.8.5	Выход из состояния ожидания.....	471

31.8.6	Рекомендации по программированию режима энергопотребления	472
32	Контроллер прерываний NVIC	474
32.1	Логика работы прерываний контроллера NVIC	475
32.2	Упрощенный доступ к регистрам контроллера прерываний	478
32.2.1	NVIC->ISER[x]	479
32.2.2	NVIC->ICER[x]	479
32.2.3	NVIC->ISPR[x]	480
32.2.4	NVIC->ICPR[x]	480
32.2.5	NVIC->IABR[x]	481
32.2.6	NVIC->IPR[x]	481
32.2.7	NVIC->STIR	482
32.3	Прерывания, срабатывающие по уровню сигнала	482
32.4	Аппаратное и программное управление прерываниями	482
32.5	Рекомендации по работе с контроллером прерываний	483
33	Блок управления системой	485
33.1	Упрощенный доступ к регистрам блока управления системой	486
33.1.1	InterruptType->ACTLR	486
33.1.2	SCB->CPUID	487
33.1.3	SCB->ICSR	487
33.1.4	SCB->VTOR	489
33.1.5	SCB->AIRCRR	489
33.1.6	SCB->SCR	491
33.1.7	SCB->CCR	491
33.1.8	SCB->SHP[x]	494
33.1.9	SCB->SHCSR	495
33.1.10	SCB->CFSR	497
33.1.11	SCB->HFSR	501
33.1.12	SCB->MMFAR	502
33.1.13	SCB->BFAR	503
33.1.14	Рекомендации по программированию блока управления системой	503
34	Сторожевые таймеры	504
34.1	Блок сторожевого таймера MDR_IWDG	504
34.1.1	Описание регистров блока сторожевых таймеров IWDG	505
34.2	Блок сторожевого таймера MDR_WWDG	509
34.2.1	Описание регистров блока сторожевых таймеров WWDG	509
35	Электрические параметры микросхемы	511
36	Предельно допустимые характеристики микросхемы	513
37	Справочные данные	515
38	Типовые зависимости	519
39	Габаритный чертеж микросхемы	520
40	Информация для заказа	522

1 Введение

Микросхемы интегральные K1986BE92FI, K1986BE92FII, K1986BE94GI (далее микросхемы), построенные на базе высокопроизводительного процессорного RISC-ядра ARM Cortex-M3, содержат встроенную 128 Кбайт Flash-память программ и 32 Кбайт ОЗУ. Микросхемы работают на тактовой частоте до 80 МГц. Периферия микросхемы включает контроллер USB-интерфейса со встроенным аналоговым приемопередатчиком со скоростями передачи 12 Мбит/с (Full Speed) и 1,5 Мбит/с (Low Speed), стандартные интерфейсы UART, SPI и I2C, контроллер внешней системной шины, что позволяет работать с внешними микросхемами статического ОЗУ и ПЗУ, NAND Flash-памятью и другими внешними устройствами. Микросхемы содержат три 16-разрядных таймера с четырьмя каналами схем захвата и ШИМ с функциями формирования «мертвой зоны» и аппаратной блокировки, а также системный 24-разрядный таймер и два сторожевых таймера. Кроме того, в состав микросхем входят: два 12-разрядных высокоскоростных (до 0,5 Мвыборок/с) АЦП с возможностью оцифровки информации от 16 внешних каналов и от встроенных датчиков температуры и опорного напряжения; два 12-разрядных ЦАП; встроенный компаратор с тремя входами и внутренней шкалой напряжений.

Встроенные RC-генераторы HSI (8 МГц) и LSI (40 кГц) и внешние генераторы HSE (2...16 МГц) и LSE (32 кГц) и две схемы умножения тактовой частоты PLL для ядра и USB-интерфейса позволяют гибко настраивать скорость работы микросхем.

Архитектура системы памяти за счет матрицы системных шин позволяет минимизировать возможные конфликты при работе системы и повысить общую производительность. Контроллер DMA позволяет ускорить обмен информацией между ОЗУ и периферией без участия процессорного ядра.

Встроенный регулятор, предназначенный для формирования питания внутренней цифровой части, формирует напряжение 1,8 В и не требует дополнительных внешних элементов. Таким образом, для работы микросхемы достаточно одного внешнего напряжения питания в диапазоне от 2,2 до 3,6 В. Также в микросхемах реализован батарейный домен, работающий от внешней батареи, который предназначен для обеспечения функций часов реального времени и сохранения некоторого объема данных при отсутствии основного питания. Встроенные детекторы напряжения питания могут отслеживать уровень внешнего основного питания, уровень напряжения питания на батарее. Аппаратные схемы сброса при просадке питания позволяют исключить сбойную работу микросхемы при выходе уровня напряжения питания за допустимые пределы.

2 Основные характеристики

В зависимости от корпуса, в котором выпускается микросхема, изменяются функциональные возможности, но при этом объем памяти программ и ОЗУ остается одинаковым.

Таблица 1 – Основные характеристики микросхем

Характеристика	K1986BE92F1, K1986BE92F1I	K1986BE94GI
Корпус	64 вывода	144 вывода
Ядро	ARM Cortex-M3	
ПЗУ	128 Кбайт Flash	
ОЗУ	32 Кбайт	
Питание	2,2...3,6 В	
Частота	80 МГц	
USER IO	43	96
USB	Device и Host FS (до 12 Мбит/с) встроенный PHY	
UART	2	2
CAN	2	2
SPI	2	2
I2C	1	1
2 × 12-разрядных АЦП	8 каналов	16 каналов
ЦАП 12 разрядов	1	2
Компаратор	2 входа	3 входа
Внешняя шина	8 разрядов	32 разряда

3 Структурная блок-схема микросхемы

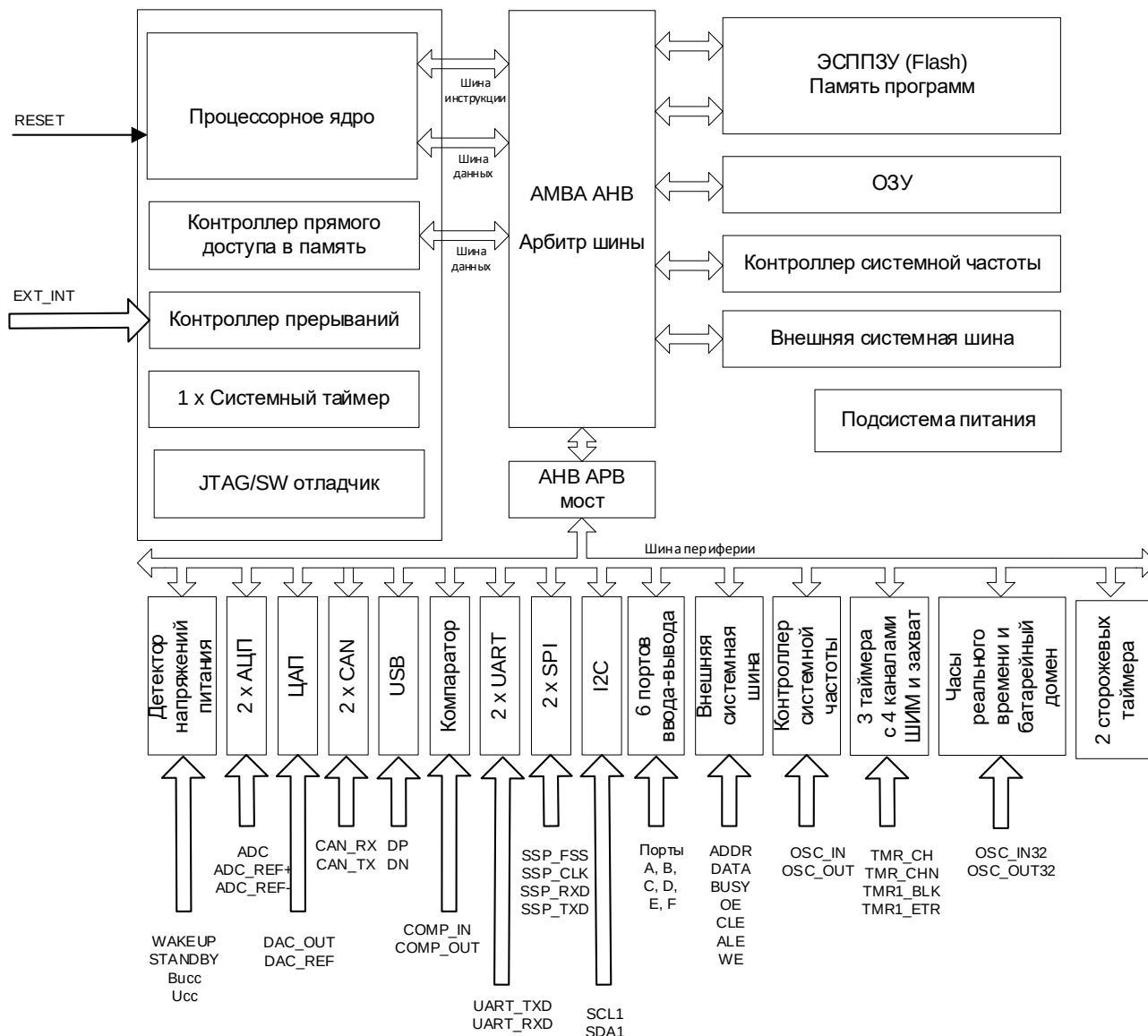


Рисунок 1 – Структурная блок-схема микросхем K1986BE92FI, K1986BE92F1I

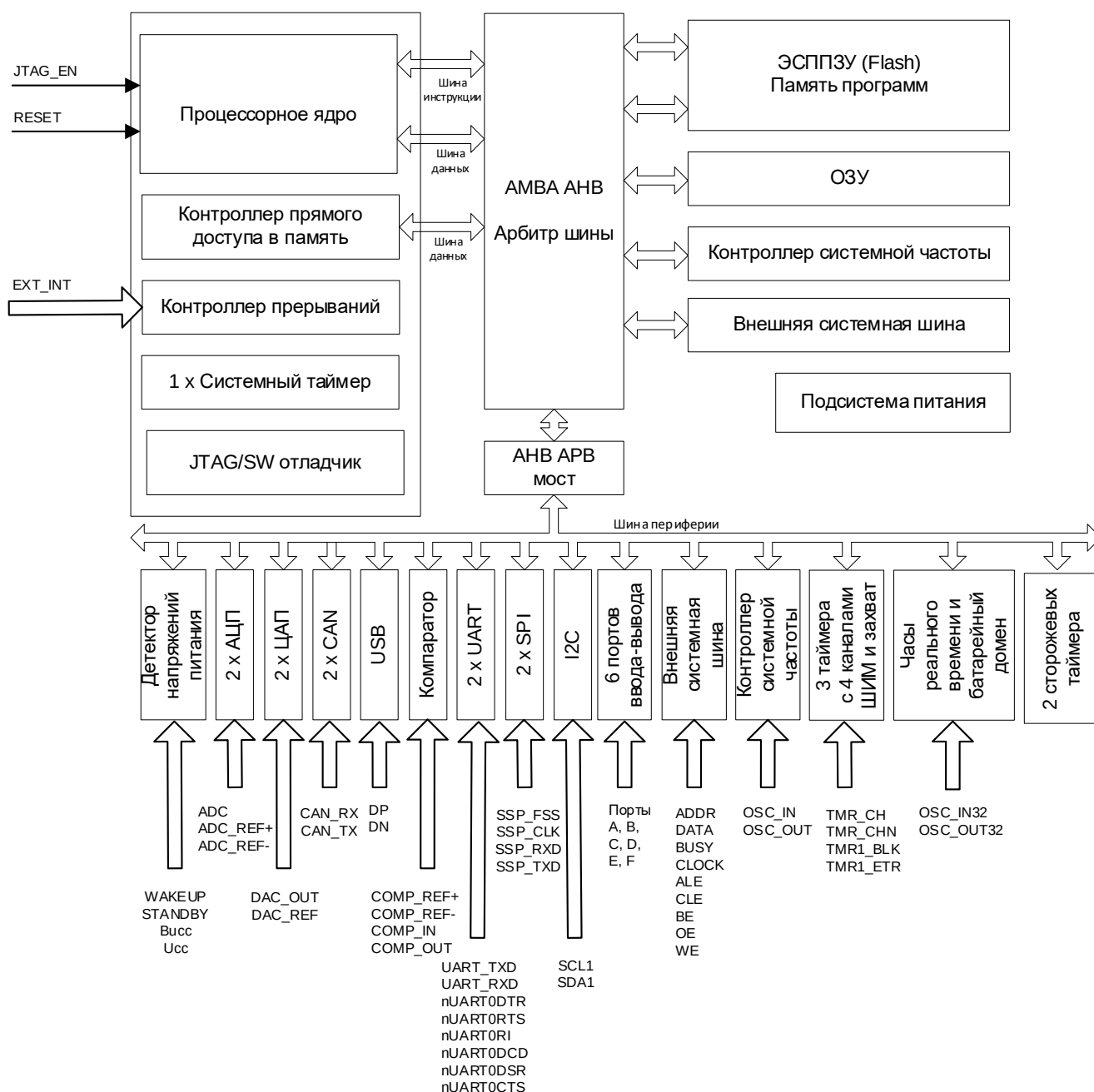


Рисунок 2 – Структурная блок-схема микросхем K1986BE94GI

4 Условные графические изображения

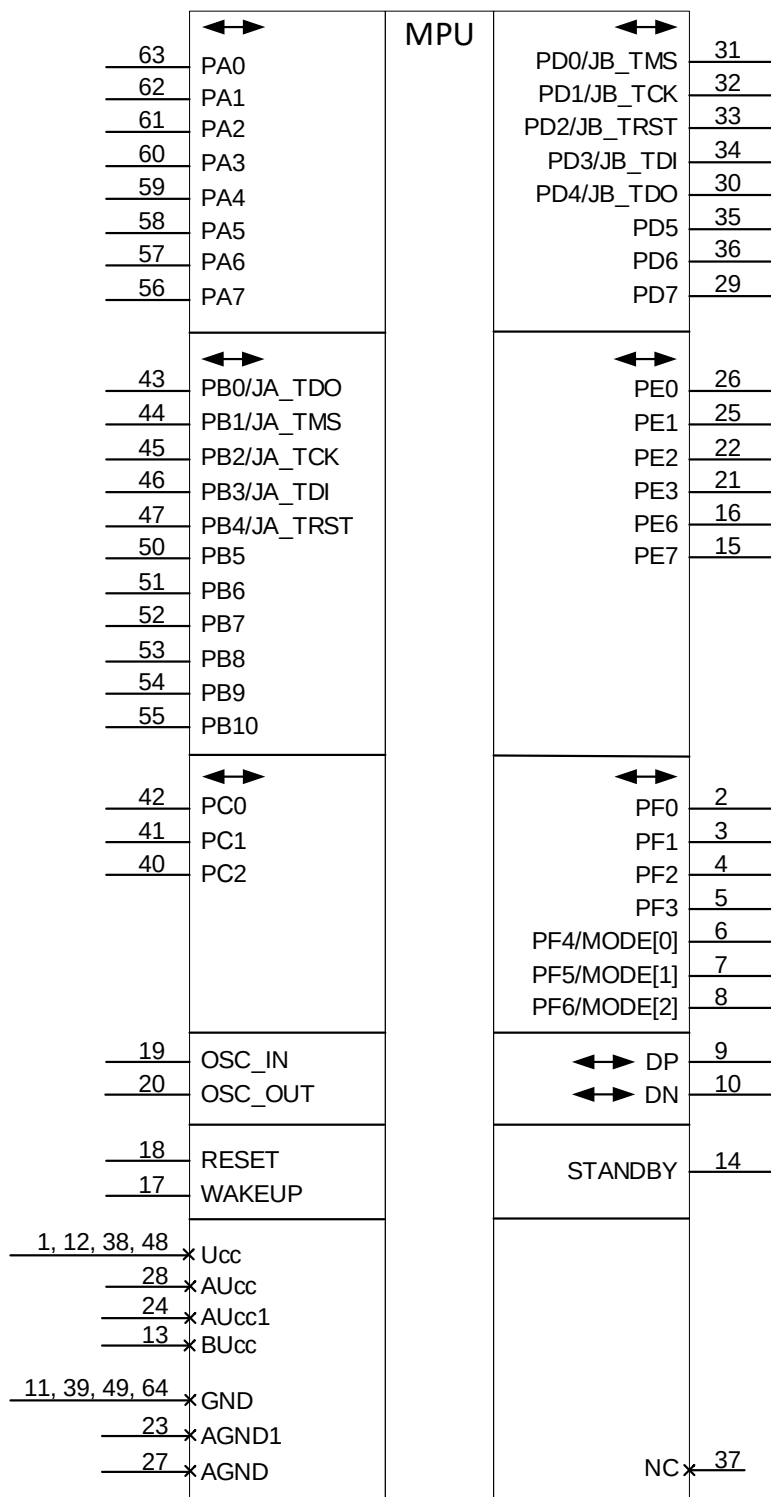


Рисунок 3 – Условное графическое изображение микросхем K1986BE92FI, K1986BE92FII

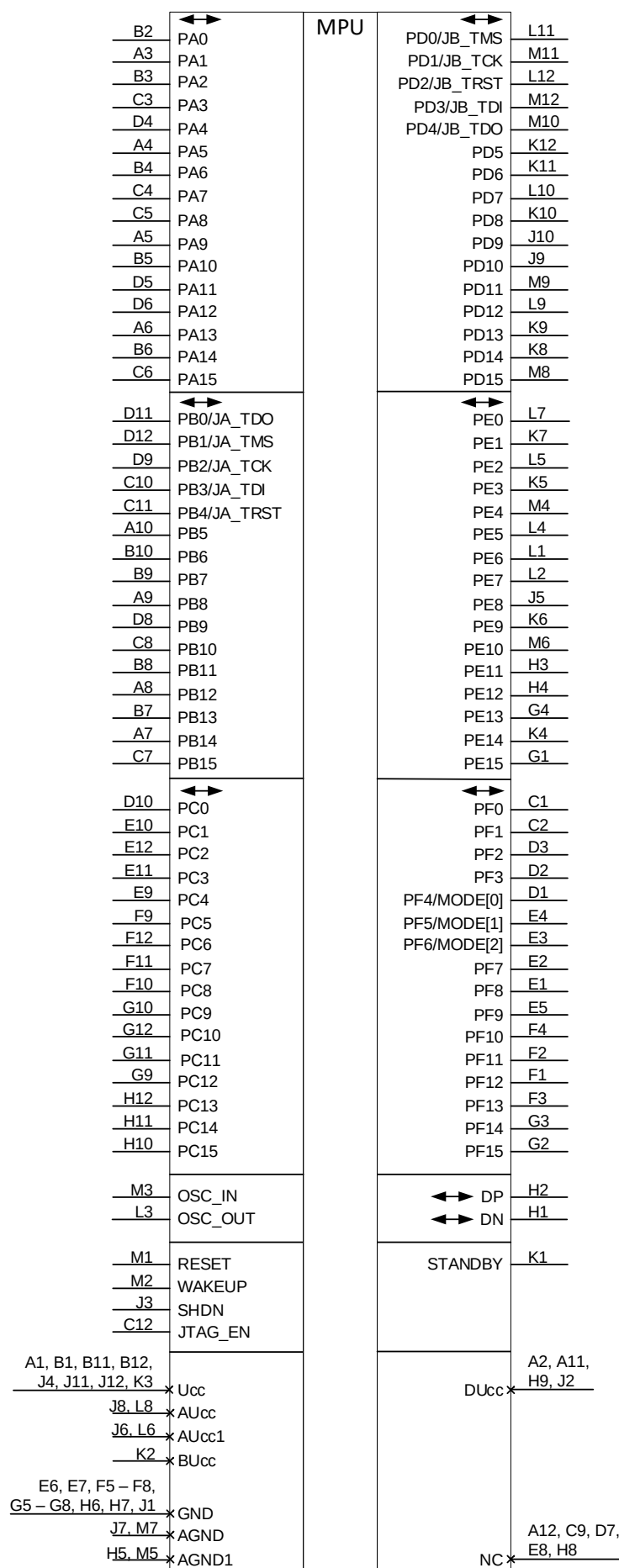


Рисунок 4 – Условное графическое изображение микросхем K1986BE94GI

5 Описание выводов

Таблица 2 – Описание выводов микросхем K1986BE92FI, K1986BE92FII

Номер вывода	Обозначение вывода	Функциональное назначение вывода
1	Ucc	Питание (2,2 – 3,6) В
2	PF0	Вход/выход 0 порта F
3	PF1	Вход/выход 1 порта F
4	PF2	Вход/выход 2 порта F
5	PF3	Вход/выход 3 порта F
6	PF4/MODE[0]	Вход/выход 4 порта F/Режим работы микроконтроллера
7	PF5/MODE[1]	Вход/выход 5 порта F/Режим работы микроконтроллера
8	PF6/MODE[2]	Вход/выход 6 порта F/Режим работы микроконтроллера
9	DP	Шина USB D+
10	DN	Шина USB D-
11	GND	Общий
12	Ucc	Питание (2,2 – 3,6) В
13	BUcc	Питание батарейного домена (1,8 – 3,6) В
14	STANDBY	Флаг режима Standby
15	PE7	Вход/выход 7 порта E
16	PE6	Вход/выход 6 порта E
17	WAKEUP	Сигнал внешнего выхода из режима Standby
18	RESET	Сигнал внешнего сброса
19	OSC_IN	Вход генератора HSE
20	OSC_OUT	Выход генератора HSE
21	PE3	Вход/выход 3 порта E
22	PE2	Вход/выход 2 порта E
23	AGND1	Общий
24	AUcc1	Питание аналоговое, АЦП, ЦАП, компаратор (должно совпадать с Ucc)
25	PE1	Вход/выход 1 порта E
26	PE0	Вход/выход 0 порта E
27	AGND	Общий
28	AUcc	Питание аналоговое, АЦП, ЦАП, компаратор (должно совпадать с Ucc)
29	PD7	Вход/выход 7 порта D
30	PD4/JB_TDO	Вход/выход 4 порта D/Выход тестовых данных JTAG интерфейса
31	PD0/JB_TMS	Вход/выход 0 порта D/Выбор режима тестирования JTAG интерфейса
32	PD1/JB_TCK	Вход/выход 1 порта D/Тестовое тактирование JTAG интерфейса
33	PD2/JB_TRST	Вход/выход 2 порта D/Тестовый сброс JTAG интерфейса
34	PD3/JB_TDI	Вход/выход 3 порта D/Вход тестовых данных JTAG интерфейса
35	PD5	Вход/выход 5 порта D
36	PD6	Вход/выход 6 порта D

Номер вывода	Обозначение вывода	Функциональное назначение вывода
37	NC	Не подключать
38	Ucc	Питание (2,2 – 3,6) В
39	GND	Общий
40	PC2	Вход/выход 2 порта С
41	PC1	Вход/выход 1 порта С
42	PC0	Вход/выход 0 порта С
43	PB0/JA_TDO	Вход/выход 0 порта В/Выход тестовых данных JTAG интерфейса
44	PB1/JA_TMS	Вход/выход 1 порта В/Выбор режима тестирования JTAG интерфейса
45	PB2/JA_TCK	Вход/выход 2 порта В/Тестовое тактирование JTAG интерфейса
46	PB3/JA_TDI	Вход/выход 3 порта В/Вход тестовых данных JTAG интерфейса
47	PB4/JA_TRST	Вход/выход 4 порта В/Тестовый сброс JTAG интерфейса
48	Ucc	Питание (2,2 – 3,6) В
49	GND	Общий
50	PB5	Вход/выход 5 порта В
51	PB6	Вход/выход 6 порта В
52	PB7	Вход/выход 7 порта В
53	PB8	Вход/выход 8 порта В
54	PB9	Вход/выход 9 порта В
55	PB10	Вход/выход 10 порта В
56	PA7	Вход/выход 7 порта А
57	PA6	Вход/выход 6 порта А
58	PA5	Вход/выход 5 порта А
59	PA4	Вход/выход 4 порта А
60	PA3	Вход/выход 3 порта А
61	PA2	Вход/выход 2 порта А
62	PA1	Вход/выход 1 порта А
63	PA0	Вход/выход 0 порта А
64	GND	Общий

Таблица 3 – Назначение выводов микросхем K1986BE92FI, K1986BE92FII по блокам

Номер вывода	Обозначение вывода	Назначение и функции вывода		
		Тип функции вывода	Обозначение функции вывода	Функциональное назначение вывода
Порт А				
63	РА0	Вход/выход 0 порта А		
		Аналоговая функция	—	—
		Основная функция	DATA0	Разряд 0 шины данных DATA[31:0] EXT_BUS
		Альтернативная функция	EXT_INT1	Вход внешнего прерывания
		Переопределенная функция	—	—
62	РА1	Вход/выход 1 порта А		
		Аналоговая функция	—	—
		Основная функция	DATA1	Разряд 1 шины данных DATA[31:0] EXT_BUS
		Альтернативная функция	TMR1_CH1	Вход/выход канала 1 таймера 1
		Переопределенная функция	TMR2_CH1	Вход/выход канала 1 таймера 2
61	РА2	Вход/выход 2 порта А		
		Аналоговая функция	—	—
		Основная функция	DATA2	Разряд 2 шины данных DATA[31:0] EXT_BUS
		Альтернативная функция	TMR1_CH1N	Инверсный выход канала 1 таймера 1
		Переопределенная функция	TMR2_CH1N	Инверсный выход канала 1 таймера 2
60	РА3	Вход/выход 3 порта А		
		Аналоговая функция	—	—
		Основная функция	DATA3	Разряд 3 шины данных DATA[31:0] EXT_BUS
		Альтернативная функция	TMR1_CH2	Вход/выход канала 2 таймера 1
		Переопределенная функция	TMR2_CH2	Вход/выход канала 2 таймера 2
59	РА4	Вход/выход 4 порта А		
		Аналоговая функция	—	—
		Основная функция	DATA4	Разряд 4 шины данных DATA[31:0] EXT_BUS
		Альтернативная функция	TMR1_CH2N	Инверсный выход канала 2 таймера 1
		Переопределенная функция	TMR2_CH2N	Инверсный выход канала 2 таймера 2
58	РА5	Вход/выход 5 порта А		
		Аналоговая функция	—	—
		Основная функция	DATA5	Разряд 5 шины данных DATA[31:0] EXT_BUS

Номер вывода	Обозначение вывода	Назначение и функции вывода		
		Тип функции вывода	Обозначение функции вывода	Функциональное назначение вывода
		Альтернативная функция	TMR1_CH3	Вход/выход канала 3 таймера 1
		Переопределенная функция	TMR2_CH3	Вход/выход канала 3 таймера 2
57	PA6	Вход/выход 6 порта A		
		Аналоговая функция	–	–
		Основная функция	DATA6	Разряд 6 шины данных DATA[31:0] EXT_BUS
		Альтернативная функция	CAN1_TX	Выходные данные интерфейса CAN1
		Переопределенная функция	UART1_RXD	Входные данные UART1
56	PA7	Вход/выход 7 порта A		
		Аналоговая функция	–	–
		Основная функция	DATA7	Разряд 7 шины данных DATA[31:0] EXT_BUS
		Альтернативная функция	CAN1_RX	Входные данные интерфейса CAN1
		Переопределенная функция	UART1_TXD	Выходные данные UART1
Порт B				
43	PB0/ JA_TDO	Вход/выход 0 порта B/ Выход тестовых данных JTAG-интерфейса		
		Аналоговая функция	–	–
		Основная функция	DATA16	Разряд 16 шины данных DATA[31:0] EXT_BUS
		Альтернативная функция	TMR3_CH1	Вход/выход канала 1 таймера 3
		Переопределенная функция	UART1_TXD	Выходные данные UART1
44	PB1/ JA_TMS	Вход/выход 1 порта B/ Выбор режима тестирования JTAG интерфейса		
		Аналоговая функция	–	–
		Основная функция	DATA17	Разряд 17 шины данных DATA[31:0] EXT_BUS
		Альтернативная функция	TMR3_CH1N	Инверсный выход канала 1 таймера 3
		Переопределенная функция	UART2_RXD	Входные данные UART2
45	PB2/ JA_TCK	Вход/выход 2 порта B/ Тестовое тактирование JTAG интерфейса		
		Аналоговая функция	–	–
		Основная функция	DATA18	Разряд 18 шины данных DATA[31:0] EXT_BUS
		Альтернативная функция	TMR3_CH2	Вход/выход канала 2 таймера 3
		Переопределенная функция	CAN1_TX	Выходные данные интерфейса CAN1

Номер вывода	Обозначение вывода	Назначение и функции вывода		
		Тип функции вывода	Обозначение функции вывода	Функциональное назначение вывода
46	PB3/ JA_TDI	Вход/выход 3 порта В/ Вход тестовых данных JTAG интерфейса		
		Аналоговая функция	—	—
		Основная функция	DATA19	Разряд 19 шины данных DATA[31:0] EXT_BUS
		Альтернативная функция	TMR3_CH2N	Инверсный выход канала 2 таймера 3
		Переопределенная функция	CAN1_RX	Входные данные интерфейса CAN1
47	PB4/ JA_TRST	Вход/выход 4 порта В/ Тестовый сброс JTAG интерфейса		
		Аналоговая функция	—	—
		Основная функция	DATA20	Разряд 20 шины данных DATA[31:0] EXT_BUS
		Альтернативная функция	TMR3_BLK	Асинхронный вход внешнего события таймера 3
		Переопределенная функция	TMR3_ETR	Синхронный вход внешнего события таймера 3
50	PB5	Вход/выход 5 порта В		
		Аналоговая функция	—	—
		Основная функция	DATA21	Разряд 21 шины данных DATA[31:0] EXT_BUS
		Альтернативная функция	UART1_TXD	Выходные данные UART1
		Переопределенная функция	TMR3_CH3	Вход/выход канала 3 таймера 3
51	PB6	Вход/выход 6 порта В		
		Аналоговая функция	—	—
		Основная функция	DATA22	Разряд 22 шины данных
		Альтернативная функция	UART1_RXD	Входные данные UART1
		Переопределенная функция	TMR3_CH3N	Инверсный выход канала 3 таймера 3
52	PB7	Вход/выход 7 порта В		
		Аналоговая функция	—	—
		Основная функция	DATA23	Разряд 23 шины данных
		Альтернативная функция	nSIROUT1	Выходные данные IRDA SIR ART1
		Переопределенная функция	TMR3_CH4	Вход/выход канала 4 таймера 3
53	PB8	Вход/выход 8 порта В		
		Аналоговая функция	—	—
		Основная функция	DATA24	Разряд 24 шины данных
		Альтернативная функция	COMP_OUT	Выходные данные компаратора
		Переопределенная функция	TMR3_CH4N	Инверсный выход канала 4 таймера 3

Номер вывода	Обозначение вывода	Назначение и функции вывода		
		Тип функции вывода	Обозначение функции вывода	Функциональное назначение вывода
54	PB9	Вход/выход 9 порта В		
		Аналоговая функция	—	—
		Основная функция	DATA25	Разряд 25 шины данных DATA[31:0] EXT_BUS
		Альтернативная функция	nSIRIN1	Входные данные IRDA SIR UART1
		Переопределенная функция	EXT_INT4	Вход внешнего прерывания
55	PB10	Вход/выход 10 порта В		
		Аналоговая функция	—	—
		Основная функция	DATA26	Разряд 26 шины данных DATA[31:0] EXT_BUS
		Альтернативная функция	EXT_INT2	Вход внешнего прерывания
		Переопределенная функция	nSIROUT1	Выходные данные IRDA SIR UART1
Порт С				
42	PC0	Вход/выход 0 порта С		
		Аналоговая функция	—	—
		Основная функция	—	—
		Альтернативная функция	SCL1	Выход синхросигнала I2C
		Переопределенная функция	SSP2_FSS	Вход/выход разрешения SPI 2
41	PC1	Вход/выход 1 порта С		
		Аналоговая функция	—	—
		Основная функция	OE	Выход сигнала разрешения выдачи данных EXT_BUS
		Альтернативная функция	SDA1	Вход/выход данных I2C
		Переопределенная функция	SSP2_CLK	Вход/выход синхросигнала SPI2
40	PC2	Вход/выход 2 порта С		
		Аналоговая функция	—	—
		Основная функция	WE	Выход сигнала разрешения записи данных EXT_BUS
		Альтернативная функция	TMR3_CH1	Вход/выход канала 1 таймера 3
		Переопределенная функция	SSP2_RXD	Входные данные SPI2

Номер вывода	Обозначение вывода	Назначение и функции вывода		
		Тип функции вывода	Обозначение функции вывода	Функциональное назначение вывода
Порт D				
31	PD0/ JB_TMS	Вход/выход 0 порта D/ Выбор режима тестирования JTAG интерфейса		
		Аналоговая функция	ADC0_REF+	Положительное опорное напряжение / Входной канал 0 АЦП
		Основная функция	TMR1_CH1N	Инверсный выход канала 1 таймера 1
		Альтернативная функция	UART2_RXD	Входные данные UART2
		Переопределенная функция	TMR3_CH1	Вход/выход канала 1 таймера 3
32	PD1/ JB_TCK	Вход/выход 1 порта D/ Тестовое тактирование JTAG интерфейса		
		Аналоговая функция	ADC1_REF-	Отрицательное опорное напряжение / Входной канал 1 АЦП
		Основная функция	TMR1_CH1	Вход/выход канала 1 таймера 1
		Альтернативная функция	UART2_TXD	Выходные данные UART2
		Переопределенная функция	TMR3_CH1N	Инверсный выход канала 1 таймера 3
33	PD2/ JB_TRST	Вход/выход 2 порта D/ Тестовый сброс JTAG интерфейса		
		Аналоговая функция	ADC2	Входной канал 2 АЦП
		Основная функция	BUSY1	Вход сигнала занятости внешнего устройства EXT_BUS
		Альтернативная функция	SSP2_RXD	Входные данные SPI2
		Переопределенная функция	TMR3_CH2	Вход/выход канала 2 таймера 3
34	PD3/ JB_TDI	Вход/выход 3 порта D/ Вход тестовых данных JTAG интерфейса		
		Аналоговая функция	ADC3	Входной канал 3 АЦП
		Основная функция	—	—
		Альтернативная функция	SSP2_FSS	Вход/выход разрешения SPI2
		Переопределенная функция	TMR3_CH2N	Инверсный выход канала 1 таймера 3
30	PD4/ JB_TDO	Вход/выход 4 порта D/ Выход тестовых данных JTAG-интерфейса		
		Аналоговая функция	ADC4	Входной канал 4 АЦП
		Основная функция	TMR1_ETR	Вход внешнего события таймера 1
		Альтернативная функция	nSIROUT2	Выходные данные IRDA SIR UART2
		Переопределенная функция	TMR3_BLK	Вход внешнего события таймера 3

Номер вывода	Обозначение вывода	Назначение и функции вывода		
		Тип функции вывода	Обозначение функции вывода	Функциональное назначение вывода
35	PD5	Вход/выход 5 порта D		
		Аналоговая функция	ADC5	Входной канал 5 АЦП
		Основная функция	CLE	Выход разрешения защёлкивания команды EXT_BUS
		Альтернативная функция	SSP2_CLK	Вход/выход синхросигнала SPI2
		Переопределенная функция	TMR2_ETR	Вход внешнего события таймера 2
36	PD6	Вход/выход 6 порта D		
		Аналоговая функция	ADC6	Входной канал 6 АЦП
		Основная функция	ALE	Выход разрешения защёлкивания адреса EXT_BUS
		Альтернативная функция	SSP2_TXD	Выходные данные SPI2
		Переопределенная функция	TMR2_BLK	Вход внешнего события таймера 2
29	PD7	Вход/выход 7 порта D		
		Аналоговая функция	ADC7	Входной канал 7 АЦП
		Основная функция	TMR1_BLK	Вход внешнего события таймера 1
		Альтернативная функция	nSIRIN2	Входные данные IRDA SIR UART2
		Переопределенная функция	UART1_RXD	Входные данные UART1
Порт E				
26	PE0	Вход/выход 0 порта E		
		Аналоговая функция	DAC2_OUT	Аналоговый выход ЦАП2
		Основная функция	ADDR16	Разряд 16 шины адреса
		Альтернативная функция	TMR2_CH1	Вход/выход канала 1 таймера 2
		Переопределенная функция	CAN1_RX	Входные данные интерфейса CAN1
25	PE1	Вход/выход 1 порта E		
		Аналоговая функция	DAC2_REF	Опорное напряжение ЦАП2
		Основная функция	ADDR17	Разряд 17 шины адреса ADDR [31:0] EXT_BUS
		Альтернативная функция	TMR2_CH1N	Инверсный выход канала 1 таймера 2
		Переопределенная функция	CAN1_TX	Выходные данные интерфейса CAN1
22	PE2	Вход/выход 2 порта E		
		Аналоговая функция	COMP_IN1	Вход 1 компаратора
		Основная функция	ADDR18	Разряд 18 шины адреса ADDR [31:0] EXT_BUS
		Альтернативная функция	TMR2_CH3	Вход/выход канала 3 таймера 2
		Переопределенная функция	TMR3_CH1	Вход/выход канала 1 таймера 3

Номер вывода	Обозначение вывода	Назначение и функции вывода		
		Тип функции вывода	Обозначение функции вывода	Функциональное назначение вывода
21	PE3	Вход/выход 3 порта E		
		Аналоговая функция	COMP_IN2	Вход 2 компаратора
		Основная функция	ADDR19	Разряд 19 шины адреса
		Альтернативная функция	TMR2_CH3N	Инверсный выход канала 3 таймера 2
		Переопределенная функция	TMR3_CH1N	Инверсный выход канала 1 таймера 3
16	PE6	Вход/выход 6 порта E		
		Аналоговая функция	OSC_IN32	Вход осциллятора 32 кГц
		Основная функция	ADDR22	Разряд 22 шины адреса ADDR [31:0] EXT_BUS
		Альтернативная функция	CAN2_RX	Входные данные интерфейса CAN2
		Переопределенная функция	TMR3_CH3	Вход/выход канала 3 таймера 3
15	PE7	Вход/выход 7 порта E		
		Аналоговая функция	OSC_OUT32	Выход осциллятора 32 кГц
		Основная функция	ADDR23	Разряд 23 шины адреса ADDR [31:0] EXT_BUS
		Альтернативная функция	CAN2_TX	Выходные данные интерфейса CAN2
		Переопределенная функция	TMR3_CH3N	Инверсный выход канала 3 таймера 3
Порт F				
2	PF0	Вход/выход 0 порта F		
		Аналоговая функция	—	—
		Основная функция	ADDR0	Разряд 0 шины адреса ADDR [31:0] EXT_BUS
		Альтернативная функция	SSP1_TXD	Выходные данные SPI1
		Переопределенная функция	UART2_RXD	Входные данные UART2
3	PF1	Вход/выход 1 порта F		
		Аналоговая функция	—	—
		Основная функция	ADDR1	Разряд 1 шины адреса ADDR [31:0] EXT_BUS
		Альтернативная функция	SSP1_CLK	Вход/выход синхросигнала SPI1
		Переопределенная функция	UART2_TXD	Выходные данные UART2
4	PF2	Вход/выход 2 порта F		
		Аналоговая функция	—	—
		Основная функция	ADDR2	Разряд 2 шины адреса ADDR [31:0] EXT_BUS
		Альтернативная функция	SSP1_FSS	Вход/выход разрешения SPI1
		Переопределенная функция	CAN2_RX	Входные данные интерфейса CAN2

Номер вывода	Обозначение вывода	Назначение и функции вывода		
		Тип функции вывода	Обозначение функции вывода	Функциональное назначение вывода
5	PF3	Вход/выход 3 порта F		
		Аналоговая функция	—	
		Основная функция	ADDR3	Разряд 3 шины адреса ADDR [31:0] EXT_BUS
		Альтернативная функция	SSP1_RXD	Входные данные SPI1
		Переопределенная функция	CAN2_TX	Выходные данные интерфейса CAN2
6	PF4/ MODE[0]	Вход/выход 4 порта F/ Режим работы микроконтроллера		
		Аналоговая функция	—	—
		Основная функция	ADDR4	Разряд 4 шины адреса ADDR [31:0] EXT_BUS
		Альтернативная функция	—	—
		Переопределенная функция	—	—
7	PF5/ MODE[1]	Вход/выход 5 порта F/ Режим работы микроконтроллера		
		Аналоговая функция	—	—
		Основная функция	ADDR5	Разряд 5 шины адреса ADDR [31:0] EXT_BUS
		Альтернативная функция	—	—
		Переопределенная функция	—	—
8	PF6/ MODE[2]	Вход/выход 6 порта F/ Режим работы микроконтроллера		
		Аналоговая функция	—	—
		Основная функция	ADDR6	Разряд 6 шины адреса ADDR [31:0] EXT_BUS
		Альтернативная функция	TMR1_CH1	Вход/выход канала 1 таймера 1
		Переопределенная функция	—	—
Системное управление				
18	RESET	Сигнал внешнего сброса		
17	WAKEUP	Сигнал внешнего выхода из режима Standby		
14	STANDBY	Флаг режима Standby		
19	OSC_IN	Вход генератора HSE		
20	OSC_OUT	Выход генератора HSE		
Интерфейс USB				
9	DP	Шина USB D+		
10	DN	Шина USB D-		

Номер вывода	Обозначение вывода	Назначение и функции вывода		
		Тип функции вывода	Обозначение функции вывода	Функциональное назначение вывода
Питание				
1, 12, 38, 48	Ucc	Питание (2,2 – 3,6) В		
28	AUcc	Питание аналоговое, АЦП, ЦАП, компаратор (должно совпадать с Ucc)		
24	AUcc1	Питание аналоговое, АЦП, ЦАП, компаратор (должно совпадать с Ucc)		
13	BUcc	Питание батарейного домена (1,8 – 3,6) В		
11, 39, 49, 64	GND	Общий		
27	AGND	Общий		
23	AGND1	Общий		
Не используются				
37	NC	Не используются. Рекомендуется не подсоединять или подключить к шине Общий		

Таблица 4 – Описание выводов микросхем K1986BE94GI

Номер вывода	Обозначение вывода	Функциональное назначение вывода
A1	Ucc	Питание (2,2 – 3,6) В
A2	DUcc	Тестовый выход внутреннего питания 1,8 В. Рекомендуется не подсоединять
A3	PA1	Вход/выход 1 порта А
A4	PA5	Вход/выход 5 порта А
A5	PA9	Вход/выход 9 порта А
A6	PA13	Вход/выход 13 порта А
A7	PB14	Вход/выход 14 порта В
A8	PB12	Вход/выход 12 порта В
A9	PB8	Вход/выход 8 порта В
A10	PB5	Вход/выход 5 порта В
A11	DUcc	Тестовый выход внутреннего питания 1,8 В. Рекомендуется не подсоединять
A12	NC	Не используется. Рекомендуется не подсоединять или подключить к шине общий
B1	Ucc	Питание (2,2 – 3,6) В
B2	PA0	Вход/выход 0 порта А
B3	PA2	Вход/выход 2 порта А
B4	PA6	Вход/выход 6 порта А
B5	PA10	Вход/выход 10 порта А
B6	PA14	Вход/выход 14 порта А
B7	PB13	Вход/выход 13 порта В
B8	PB11	Вход/выход 11 порта В
B9	PB7	Вход/выход 7 порта В
B10	PB6	Вход/выход 6 порта В
B11	Ucc	Питание (2,2 – 3,6) В
B12	Ucc	Питание (2,2 – 3,6) В
C1	PF0	Вход/выход 0 порта F
C2	PF1	Вход/выход 1 порта F
C3	PA3	Вход/выход 3 порта А
C4	PA7	Вход/выход 7 порта А
C5	PA8	Вход/выход 8 порта А
C6	PA15	Вход/выход 15 порта А
C7	PB15	Вход/выход 15 порта В
C8	PB10	Вход/выход 10 порта В
C9	NC	Не используется. Рекомендуется не подсоединять или подключить к шине общий
C10	PB3/JA_TDI	Вход/выход 3 порта В/Вход тестовых данных JTAG интерфейса
C11	PB4/JA_TRST	Вход/выход 4 порта В/Тестовый сброс JTAG интерфейса
C12	JTAG_EN	Тестовый вход включения/отключения ТАР-контроллера. Рекомендуется не подсоединять или подключить к шине общий

Номер вывода	Обозначение вывода	Функциональное назначение вывода
D1	PF4/MODE[0]	Вход/выход 4 порта F/Режим работы микроконтроллера
D2	PF3	Вход/выход 3 порта F
D3	PF2	Вход/выход 2 порта F
D4	PA4	Вход/выход 4 порта A
D5	PA11	Вход/выход 11 порта A
D6	PA12	Вход/выход 12 порта A
D7	NC	Не используется. Рекомендуется не подсоединять или подключить к шине общий
D8	PB9	Вход/выход 9 порта B
D9	PB2/JA_TCK	Вход/выход 2 порта B/Тестовое тактирование JTAG интерфейса
D10	PC0	Вход/выход 0 порта C
D11	PB0/JA_TDO	Вход/выход 0 порта B/Выход тестовых данных JTAG интерфейса
D12	PB1/JA_TMS	Вход/выход 1 порта B/Выбор режима тестирования JTAG интерфейса
E1	PF8	Вход/выход 8 порта F
E2	PF7	Вход/выход 7 порта F
E3	PF6/MODE[2]	Вход/выход 6 порта F/Режим работы микроконтроллера
E4	PF5/MODE[1]	Вход/выход 5 порта F/Режим работы микроконтроллера
E5	PF9	Вход/выход 9 порта F
E6, E7	GND	Общий
E8	NC	Не используется. Рекомендуется не подсоединять или подключить к шине общий
E9	PC4	Вход/выход 4 порта C
E10	PC1	Вход/выход 1 порта C
E11	PC3	Вход/выход 3 порта C
E12	PC2	Вход/выход 2 порта C
F1	PF12	Вход/выход 12 порта F
F2	PF11	Вход/выход 11 порта F
F3	PF13	Вход/выход 13 порта F
F4	PF10	Вход/выход 10 порта F
F5 – F8	GND	Общий
F9	PC5	Вход/выход 5 порта C
F10	PC8	Вход/выход 8 порта C
F11	PC7	Вход/выход 7 порта C
F12	PC6	Вход/выход 6 порта C
G1	PE15	Вход/выход 15 порта E
G2	PF15	Вход/выход 15 порта F
G3	PF14	Вход/выход 14 порта F
G4	PE13	Вход/выход 13 порта E
G5 – G8	GND	Общий
G9	PC12	Вход/выход 12 порта C

Номер вывода	Обозначение вывода	Функциональное назначение вывода
G10	PC9	Вход/выход 9 порта C
G11	PC11	Вход/выход 11 порта C
G12	PC10	Вход/выход 10 порта C
H1	DN	Шина USB D-
H2	DP	Шина USB D+
H3	PE11	Вход/выход 11 порта E
H4	PE12	Вход/выход 12 порта E
H5	AGND1	Общий
H6, H7	GND	Общий
H8	NC	Не используется. Рекомендуется не подсоединять или подключить к шине общий
H9	DUcc	Тестовый вывод внутреннего питания 1,8 В. Рекомендуется не подсоединять
H10	PC15	Вход/выход 15 порта C
H11	PC14	Вход/выход 14 порта C
H12	PC13	Вход/выход 13 порта C
J1	GND	Общий
J2	DUcc	Тестовый выход внутреннего питания 1,8 В. Рекомендуется не подсоединять
J3	SHDN	Тестовый вход включения/отключения внутреннего регулятора LDO. Рекомендуется не подсоединять или подключить к шине общий
J4	Ucc	Питание (2,2 – 3,6) В
J5	PE8	Вход/выход 8 порта E
J6	AUcc1	Питание аналоговое, АЦП, ЦАП, компаратор (должно совпадать с Ucc)
J7	AGND	Общий
J8	AUcc	Питание аналоговое, АЦП, ЦАП, компаратор (должно совпадать с Ucc)
J9	PD10	Вход/выход 10 порта D
J10	PD9	Вход/выход 9 порта D
J11, J12	Ucc	Питание (2,2 – 3,6) В
K1	STANDBY	Флаг режима Standby
K2	BUcc	Питание батарейного домена (1,8 – 3,6) В
K3	Ucc	Питание (2,2 – 3,6) В
K4	PE14	Вход/выход 14 порта E
K5	PE3	Вход/выход 3 порта E
K6	PE9	Вход/выход 9 порта E
K7	PE1	Вход/выход 1 порта E
K8	PD14	Вход/выход 14 порта D
K9	PD13	Вход/выход 13 порта D
K10	PD8	Вход/выход 8 порта D
K11	PD6	Вход/выход 6 порта D

Номер вывода	Обозначение вывода	Функциональное назначение вывода
K12	PD5	Вход/выход 5 порта D
L1	PE6	Вход/выход 6 порта E
L2	PE7	Вход/выход 7 порта E
L3	OSC_OUT	Выход генератора HSE
L4	PE5	Вход/выход 5 порта E
L5	PE2	Вход/выход 2 порта E
L6	AUcc1	Питание аналоговое, АЦП, ЦАП, компаратор (должно совпадать с U _{CC})
L7	PE0	Вход/выход 0 порта E
L8	AUcc	Питание аналоговое, АЦП, ЦАП, компаратор (должно совпадать с U _{CC})
L9	PD12	Вход/выход 12 порта D
L10	PD7	Вход/выход 7 порта D
L11	PD0/JB_TMS	Вход/выход 0 порта D/Выбор режима тестирования JTAG интерфейса
L12	PD2/JB_TRST	Вход/выход 2 порта D/Тестовый сброс JTAG интерфейса
M1	RESET	Сигнал внешнего сброса
M2	WAKEUP	Сигнал внешнего выхода из режима Standby
M3	OSC_IN	Вход генератора HSE
M4	PE4	Вход/выход 4 порта E
M5	AGND1	Общий
M6	PE10	Вход/выход 10 порта E
M7	AGND	Общий
M8	PD15	Вход/выход 15 порта D
M9	PD11	Вход/выход 11 порта D
M10	PD4/JB_TDO	Вход/выход 4 порта D/Выход тестовых данных JTAG-интерфейса
M11	PD1/JB_TCK	Вход/выход 1 порта D/Тестовое тактирование JTAG интерфейса
M12	PD3/JB_TDI	Вход/выход 3 порта D/Вход тестовых данных JTAG интерфейса

Таблица 5 – Назначение выводов микросхем K1986BE94GI по блокам

Номер вывода	Обозначение вывода	Назначение и функции вывода		
		Тип функции вывода	Обозначение функции вывода	Функциональное назначение вывода
Порт А				
B2	PA0	Вход/выход 0 порта А		
		Аналоговая функция	—	—
		Основная функция	DATA0	Разряд 0 шины данных DATA[31:0] EXT_BUS
		Альтернативная функция	EXT_INT1	Вход внешнего прерывания
		Переопределенная функция	—	—
A3	PA1	Вход/выход 1 порта А		
		Аналоговая функция	—	—
		Основная функция	DATA1	Разряд 1 шины данных DATA[31:0] EXT_BUS
		Альтернативная функция	TMR1_CH1	Вход/выход канала 1 таймера 1
		Переопределенная функция	TMR2_CH1	Вход/выход канала 1 таймера 2
B3	PA2	Вход/выход 2 порта А		
		Аналоговая функция	—	—
		Основная функция	DATA2	Разряд 2 шины данных DATA[31:0] EXT_BUS
		Альтернативная функция	TMR1_CH1N	Инверсный выход канала 1 таймера 1
		Переопределенная функция	TMR2_CH1N	Инверсный выход канала 1 таймера 2
C3	PA3	Вход/выход 3 порта А		
		Аналоговая функция	—	—
		Основная функция	DATA3	Разряд 3 шины данных DATA[31:0] EXT_BUS
		Альтернативная функция	TMR1_CH2	Вход/выход канала 2 таймера 1
		Переопределенная функция	TMR2_CH2	Вход/выход канала 2 таймера 2
D4	PA4	Вход/выход 4 порта А		
		Аналоговая функция	—	—
		Основная функция	DATA4	Разряд 4 шины данных DATA[31:0] EXT_BUS
		Альтернативная функция	TMR1_CH2N	Инверсный выход канала 2 таймера 1
		Переопределенная функция	TMR2_CH2N	Инверсный выход канала 2 таймера 2
A4	PA5	Вход/выход 5 порта А		
		Аналоговая функция	—	—
		Основная функция	DATA5	Разряд 5 шины данных DATA[31:0] EXT_BUS
		Альтернативная функция	TMR1_CH3	Вход/выход канала 3 таймера 1
		Переопределенная функция	TMR2_CH3	Вход/выход канала 3 таймера 2

Номер вывода	Обозначение вывода	Назначение и функции вывода		
		Тип функции вывода	Обозначение функции вывода	Функциональное назначение вывода
B4	PA6	Вход/выход 6 порта A		
		Аналоговая функция	—	—
		Основная функция	DATA6	Разряд 6 шины данных DATA[31:0] EXT_BUS
		Альтернативная функция	CAN1_TX	Выходные данные интерфейса CAN1
		Переопределенная функция	UART1_RX D	Входные данные UART1
C4	PA7	Вход/выход 7 порта A		
		Аналоговая функция	—	—
		Основная функция	DATA7	Разряд 7 шины данных DATA[31:0] EXT_BUS
		Альтернативная функция	CAN1_RX	Входные данные интерфейса CAN1
		Переопределенная функция	UART1_TX D	Выходные данные UART1
C5	PA8	Вход/выход 8 порта A		
		Аналоговая функция	—	—
		Основная функция	DATA8	Разряд 8 шины данных DATA[31:0] EXT_BUS
		Альтернативная функция	TMR1_CH3N	Инверсный выход канала 3 таймера 1
		Переопределенная функция	TMR2_CH3N	Инверсный выход канала 3 таймера 2
A5	PA9	Вход/выход 9 порта A		
		Аналоговая функция	—	—
		Основная функция	DATA9	Разряд 9 шины данных DATA[31:0] EXT_BUS
		Альтернативная функция	TMR1_CH4	Вход/выход канала 4 таймера 1
		Переопределенная функция	TMR2_CH4	Вход/выход канала 4 таймера 2
B5	PA10	Вход/выход 10 порта A		
		Аналоговая функция	—	—
		Основная функция	DATA10	Разряд 10 шины данных DATA[31:0] EXT_BUS
		Альтернативная функция	nUART1DTR	Выход готовности для передачи данных UART1
		Переопределенная функция	TMR2_CH4N	Инверсный выход канала 4 таймера 2
D5	PA11	Вход/выход 11 порта A		
		Аналоговая функция	—	—
		Основная функция	DATA11	Разряд 11 шины данных DATA[31:0] EXT_BUS
		Альтернативная функция	nUART1RTS	Выход запроса на передачу данных UART1
		Переопределенная функция	TMR2_BLK	Вход внешнего события таймера 2

Номер вывода	Обозначение вывода	Назначение и функции вывода		
		Тип функции вывода	Обозначение функции вывода	Функциональное назначение вывода
D6	PA12	Вход/выход 12 порта A		
		Аналоговая функция	—	—
		Основная функция	DATA12	Разряд 12 шины данных DATA[31:0] EXT_BUS
		Альтернативная функция	nUART1RI	Вход сигнала вызова от источника
		Переопределенная функция	TMR2_ETR	Вход внешнего события таймера 2
A6	PA13	Вход/выход 13 порта A		
		Аналоговая функция	—	—
		Основная функция	DATA13	Разряд 13 шины данных DATA[31:0] EXT_BUS
		Альтернативная функция	nUART1DCD	Вход информационного сигнала от источника
		Переопределенная функция	TMR1_CH4N	Инверсный выход канала 4 таймера 1
B6	PA14	Вход/выход 14 порта A		
		Аналоговая функция	—	—
		Основная функция	DATA14	Разряд 14 шины данных DATA[31:0] EXT_BUS
		Альтернативная функция	nUART1DSR	Вход готовности для передачи данных источником
		Переопределенная функция	TMR1_BLK	Вход внешнего события таймера 1
C6	PA15	Вход/выход 15 порта A		
		Аналоговая функция	—	—
		Основная функция	DATA15	Разряд 15 шины данных DATA[31:0] EXT_BUS
		Альтернативная функция	nUART1CTS	Вход готовности для приема данных источником
		Переопределенная функция	TMR1_ETR	Вход внешнего события таймера 1
Порт B				
D11	PB0/JA_TDO	Вход/выход 0 порта B/ Выход тестовых данных JTAG-интерфейса		
		Аналоговая функция	—	—
		Основная функция	DATA16	Разряд 16 шины данных DATA[31:0] EXT_BUS
		Альтернативная функция	TMR3_CH1	Вход/выход канала 1 таймера 3
		Переопределенная функция	UART1_TXD	Выходные данные UART1

Номер вывода	Обозначение вывода	Назначение и функции вывода		
		Тип функции вывода	Обозначение функции вывода	Функциональное назначение вывода
D12	PB1/ JA_TMS	Вход/выход 1 порта В/ Выбор режима тестирования JTAG интерфейса		
		Аналоговая функция	—	—
		Основная функция	DATA17	Разряд 17 шины данных DATA[31:0] EXT_BUS
		Альтернативная функция	TMR3_CH1N	Инверсный выход канала 1 таймера 3
		Переопределенная функция	UART2_RXD	Входные данные UART2
D9	PB2/ JA_TCK	Вход/выход 2 порта В/ Тестовое тактирование JTAG интерфейса		
		Аналоговая функция	—	—
		Основная функция	DATA18	Разряд 18 шины данных DATA[31:0] EXT_BUS
		Альтернативная функция	TMR3_CH2	Вход/выход канала 2 таймера 3
		Переопределенная функция	CAN1_TX	Выходные данные интерфейса CAN1
C10	PB3/ JA_TDI	Вход/выход 3 порта В/ Вход тестовых данных JTAG интерфейса		
		Аналоговая функция	—	—
		Основная функция	DATA19	Разряд 19 шины данных DATA[31:0] EXT_BUS
		Альтернативная функция	TMR3_CH2N	Инверсный выход канала 2 таймера 3
		Переопределенная функция	CAN1_RX	Входные данные интерфейса CAN1
C11	PB4/ JA_TRST	Вход/выход 4 порта В/ Тестовый сброс JTAG интерфейса		
		Аналоговая функция	—	—
		Основная функция	DATA20	Разряд 20 шины данных DATA[31:0] EXT_BUS
		Альтернативная функция	TMR3_BLK	Асинхронный вход внешнего события таймера 3
		Переопределенная функция	TMR3_ETR	Синхронный вход внешнего события таймера 3
A10	PB5	Вход/выход 5 порта В		
		Аналоговая функция	—	—
		Основная функция	DATA21	Разряд 21 шины данных DATA[31:0] EXT_BUS
		Альтернативная функция	UART1_TX D	Выходные данные UART1
		Переопределенная функция	TMR3_CH3	Вход/выход канала 3 таймера 3
B10	PB6	Вход/выход 6 порта В		
		Аналоговая функция	—	—
		Основная функция	DATA22	Разряд 22 шины данных
		Альтернативная функция	UART1_RX D	Входные данные UART1
		Переопределенная функция	TMR3_CH3N	Инверсный выход канала 3 таймера 3

Номер вывода	Обозначение вывода	Назначение и функции вывода		
		Тип функции вывода	Обозначение функции вывода	Функциональное назначение вывода
B9	PB7	Вход/выход 7 порта В		
		Аналоговая функция	—	—
		Основная функция	DATA23	Разряд 23 шины данных
		Альтернативная функция	nSIROUT1	Выходные данные IRDA SIR ART1
A9	PB8	Переопределенная функция	TMR3_CH4	Вход/выход канала 4 таймера 3
		Вход/выход 8 порта В		
		Аналоговая функция	—	—
		Основная функция	DATA24	Разряд 24 шины данных
D8	PB9	Альтернативная функция	COMP_OUT	Выходные данные компаратора
		Переопределенная функция	TMR3_CH4N	Инверсный выход канала 4 таймера 3
		Вход/выход 9 порта В		
		Аналоговая функция	—	—
C8	PB10	Основная функция	DATA25	Разряд 25 шины данных DATA[31:0] EXT_BUS
		Альтернативная функция	nSIRIN1	Входные данные IRDA SIR UART1
		Переопределенная функция	EXT_INT4	Вход внешнего прерывания
		Вход/выход 10 порта В		
B8	PB11	Аналоговая функция	—	—
		Основная функция	DATA26	Разряд 26 шины данных DATA[31:0] EXT_BUS
		Альтернативная функция	EXT_INT2	Вход внешнего прерывания
		Переопределенная функция	nSIROUT1	Выходные данные IRDA SIR UART1
A8	PB12	Вход/выход 11 порта В		
		Аналоговая функция	—	—
		Основная функция	DATA27	Разряд 27 шины данных DATA[31:0] EXT_BUS
		Альтернативная функция	EXT_INT1	Вход внешнего прерывания
B7	PB13	Переопределенная функция	COMP_OUT	Выходные данные компаратора
		Вход/выход 12 порта В		
		Аналоговая функция	—	—
		Основная функция	DATA28	Разряд 28 шины данных DATA[31:0] EXT_BUS
		Альтернативная функция	SSP1_FSS	Вход/выход разрешения SPI1
		Переопределенная функция	SSP2_FSS	Вход/выход разрешения SPI2
		Вход/выход 13 порта В		
		Аналоговая функция	—	—
		Основная функция	DATA29	Разряд 29 шины данных DATA[31:0] EXT_BUS
		Альтернативная функция	SSP1_CLK	Вход/выход синхросигнала SPI1
		Переопределенная функция	SSP2_CLK	Вход/выход синхросигнала SPI2

Номер вывода	Обозначение вывода	Назначение и функции вывода		
		Тип функции вывода	Обозначение функции вывода	Функциональное назначение вывода
A7	PB14	Вход/выход 14 порта В		
		Аналоговая функция	—	—
		Основная функция	DATA30	Разряд 30 шины данных DATA[31:0] EXT_BUS
		Альтернативная функция	SSP1_RXD	Входные данные SPI1
		Переопределенная функция	SSP2_RXD	Входные данные SPI2
C7	PB15	Вход/выход 15 порта В		
		Аналоговая функция	—	—
		Основная функция	DATA31	Разряд 31 шины данных DATA[31:0] EXT_BUS
		Альтернативная функция	SSP1_TXD	Выходные данные SPI1
		Переопределенная функция	SSP2_TXD	Выходные данные SPI2
Порт С				
D10	PC0	Вход/выход 0 порта С		
		Аналоговая функция	—	—
		Основная функция	—	—
		Альтернативная функция	SCL1	Выход синхросигнала I2C
		Переопределенная функция	SSP2_FSS	Вход/выход разрешения SPI 2
E10	PC1	Вход/выход 1 порта С		
		Аналоговая функция	—	—
		Основная функция	OE	Выход сигнала разрешения выдачи данных EXT_BUS
		Альтернативная функция	SDA1	Вход/выход данных I2C
		Переопределенная функция	SSP2_CLK	Вход/выход синхросигнала SPI2
E12	PC2	Вход/выход 2 порта С		
		Аналоговая функция	—	—
		Основная функция	WE	Выход сигнала разрешения записи данных EXT_BUS
		Альтернативная функция	TMR3_CH1	Вход/выход канала 1 таймера 3
		Переопределенная функция	SSP2_RXD	Входные данные SPI2
E11	PC3	Вход/выход 3 порта С		
		Аналоговая функция	—	—
		Основная функция	BE0	Выход разрешения байта 0 в 32 разрядном слове шины данных EXT_BUS
		Альтернативная функция	TMR3_CH1N	Инверсный выход канала 1 таймера 3
		Переопределенная функция	SSP2_TXD	Выходные данные SPI2

Номер вывода	Обозначение вывода	Назначение и функции вывода		
		Тип функции вывода	Обозначение функции вывода	Функциональное назначение вывода
E9	PC4	Вход/выход 4 порта C		
		Аналоговая функция	—	—
		Основная функция	BE1	Выход разрешения байта 1 в 32 разрядном слове шины данных EXT_BUS
		Альтернативная функция	TMR3_CH2	Вход/выход канала 2 таймера 3
		Переопределенная функция	TMR1_CH1	Вход/выход канала 1 таймера 1
F9	PC5	Вход/выход 5 порта C		
		Аналоговая функция	—	—
		Основная функция	BE2	Выход разрешения байта 2 в 32 разрядном слове шины данных EXT_BUS
		Альтернативная функция	TMR3_CH2N	Инверсный выход канала 2 таймера 3
		Переопределенная функция	TMR1_CH1N	Инверсный выход канала 1 таймера 1
F12	PC6	Вход/выход 6 порта C		
		Аналоговая функция	—	—
		Основная функция	BE3	Выход разрешения байта 3 в 32 разрядном слове шины данных EXT_BUS
		Альтернативная функция	TMR3_CH3	Вход/выход канала 3 таймера 3
		Переопределенная функция	TMR1_CH2	Вход/выход канала 2 таймера 1
F11	PC7	Вход/выход 7 порта C		
		Аналоговая функция	—	—
		Основная функция	CLOCK	Выход синхросигнала EXT_BUS
		Альтернативная функция	TMR3_CH3N	Инверсный выход канала 3 таймера 3
		Переопределенная функция	TMR1_CH2N	Инверсный выход канала 2 таймера 1
F10	PC8	Вход/выход 8 порта C		
		Аналоговая функция	—	—
		Основная функция	CAN1_TX	Выходные данные интерфейса CAN1
		Альтернативная функция	TMR3_CH4	Вход/выход канала 4 таймера 3
		Переопределенная функция	TMR1_CH3	Вход/выход канала 3 таймера 1
G10	PC9	Вход/выход 9 порта C		
		Аналоговая функция	—	—
		Основная функция	CAN1_RX	Входные данные интерфейса CAN1
		Альтернативная функция	TMR3_CH4N	Инверсный выход канала 4 таймера 3
		Переопределенная функция	TMR1_CH3N	Инверсный выход канала 3 таймера 1

Номер вывода	Обозначение вывода	Назначение и функции вывода		
		Тип функции вывода	Обозначение функции вывода	Функциональное назначение вывода
G12	PC10	Вход/выход 10 порта C		
		Аналоговая функция	—	—
		Основная функция	—	—
		Альтернативная функция	TMR3_ETR	Вход внешнего события таймера 3
		Переопределенная функция	TMR1_CH4	Вход/выход канала 4 таймера 1
G11	PC11	Вход/выход 11 порта C		
		Аналоговая функция	—	—
		Основная функция	—	—
		Альтернативная функция	TMR3_BLK	Вход внешнего события таймера 3
		Переопределенная функция	TMR1_CH4N	Инверсный выход канала 4 таймера 1
G9	PC12	Вход/выход 12 порта C		
		Аналоговая функция	—	—
		Основная функция	—	—
		Альтернативная функция	EXT_INT2	Вход внешнего прерывания
		Переопределенная функция	TMR1_ETR	Вход внешнего события таймера 1
H12	PC13	Вход/выход 13 порта C		
		Аналоговая функция	—	—
		Основная функция	—	—
		Альтернативная функция	EXT_INT4	Вход внешнего прерывания
		Переопределенная функция	TMR1_BLK	Вход внешнего события таймера 1
H11	PC14	Вход/выход 14 порта C		
		Аналоговая функция	—	—
		Основная функция	—	—
		Альтернативная функция	SSP2_FSS	Вход/выход разрешения SPI2
		Переопределенная функция	CAN2_RX	Входные данные интерфейса CAN2
H10	PC15	Вход/выход 15 порта C		
		Аналоговая функция	—	—
		Основная функция	—	—
		Альтернативная функция	SSP2_RXD	Входные данные SPI2
		Переопределенная функция	CAN2_TX	Выходные данные интерфейса CAN2
Порт D				
L11	PD0/ JB_TMS	Вход/выход 0 порта D/ Выбор режима тестирования JTAG интерфейса		
		Аналоговая функция	ADC0_REF+	Положительное опорное напряжение / Входной канал 0 АЦП
		Основная функция	TMR1_CH1N	Инверсный выход канала 1 таймера 1
		Альтернативная функция	UART2_RXD	Входные данные UART2
		Переопределенная функция	TMR3_CH1	Вход/выход канала 1 таймера 3

Номер вывода	Обозначение вывода	Назначение и функции вывода		
		Тип функции вывода	Обозначение функции вывода	Функциональное назначение вывода
M11	PD1/ JB_TCK	Вход/выход 1 порта D/ Тестовое тактирование JTAG интерфейса		
		Аналоговая функция	ADC1_REF-	Отрицательное опорное напряжение / Входной канал 1 АЦП
		Основная функция	TMR1_CH1	Вход/выход канала 1 таймера 1
		Альтернативная функция	UART2_TXD	Выходные данные UART2
		Переопределенная функция	TMR3_CH1N	Инверсный выход канала 1 таймера 3
L12	PD2/ JB_TRST	Вход/выход 2 порта D/ Тестовый сброс JTAG интерфейса		
		Аналоговая функция	ADC2	Входной канал 2 АЦП
		Основная функция	BUSY1	Вход сигнала занятости внешнего устройства EXT_BUS
		Альтернативная функция	SSP2_RXD	Входные данные SPI2
		Переопределенная функция	TMR3_CH2	Вход/выход канала 2 таймера 3
M12	PD3/ JB_TDI	Вход/выход 3 порта D/ Вход тестовых данных JTAG интерфейса		
		Аналоговая функция	ADC3	Входной канал 3 АЦП
		Основная функция	—	—
		Альтернативная функция	SSP2_FSS	Вход/выход разрешения SPI2
		Переопределенная функция	TMR3_CH2N	Инверсный выход канала 1 таймера 3
M10	PD4/ JB_TDO	Вход/выход 4 порта D/ Выход тестовых данных JTAG-интерфейса		
		Аналоговая функция	ADC4	Входной канал 4 АЦП
		Основная функция	TMR1_ETR	Вход внешнего события таймера 1
		Альтернативная функция	nSIROUT2	Выходные данные IRDA SIR UART2
		Переопределенная функция	TMR3_BLK	Вход внешнего события таймера 3
K12	PD5	Вход/выход 5 порта D		
		Аналоговая функция	ADC5	Входной канал 5 АЦП
		Основная функция	CLE	Выход разрешения защёлкивания команды EXT_BUS
		Альтернативная функция	SSP2_CLK	Вход/выход синхросигнала SPI2
		Переопределенная функция	TMR2_ETR	Вход внешнего события таймера 2
K11	PD6	Вход/выход 6 порта D		
		Аналоговая функция	ADC6	Входной канал 6 АЦП
		Основная функция	ALE	Выход разрешения защёлкивания адреса EXT_BUS
		Альтернативная функция	SSP2_TXD	Выходные данные SPI2
		Переопределенная функция	TMR2_BLK	Вход внешнего события таймера 2

Номер вывода	Обозначение вывода	Назначение и функции вывода		
		Тип функции вывода	Обозначение функции вывода	Функциональное назначение вывода
L10	PD7	Вход/выход 7 порта D		
		Аналоговая функция	ADC7	Входной канал 7 АЦП
		Основная функция	TMR1_BLK	Вход внешнего события таймера 1
		Альтернативная функция	nSIRIN2	Входные данные IRDA SIR UART2
		Переопределенная функция	UART1_RXD	Входные данные UART1
K10	PD8	Вход/выход 8 порта D		
		Аналоговая функция	ADC8	Входной канал 8 АЦП
		Основная функция	TMR1_CH4N	Инверсный выход канала 4 таймера 1
		Альтернативная функция	TMR2_CH1	Вход/выход канала 1 таймера 2
		Переопределенная функция	UART1_TXD	Выходные данные UART1
J10	PD9	Вход/выход 9 порта D		
		Аналоговая функция	ADC9	Входной канал 9 АЦП
		Основная функция	CAN2_TX	Выходные данные интерфейса CAN2
		Альтернативная функция	TMR2_CH1N	Инверсный выход канала 1 таймера 2
		Переопределенная функция	SSP1_FSS	Вход/выход разрешения SPI1
J9	PD10	Вход/выход 10 порта D		
		Аналоговая функция	ADC10	Входной канал 10 АЦП
		Основная функция	TMR1_CH2	Вход/выход канала 2 таймера 1
		Альтернативная функция	TMR2_CH2	Вход/выход канала 2 таймера 2
		Переопределенная функция	SSP1_CLK	Вход/выход синхросигнала SPI1
M9	PD11	Вход/выход 11 порта D		
		Аналоговая функция	ADC11	Входной канал 11 АЦП
		Основная функция	TMR1_CH2N	Инверсный выход канала 2 таймера 1
		Альтернативная функция	TMR2_CH2N	Инверсный выход канала 2 таймера 2
		Переопределенная функция	SSP1_RXD	Входные данные SPI1
L9	PD12	Вход/выход 12 порта D		
		Аналоговая функция	ADC12	Входной канал 12 АЦП
		Основная функция	TMR1_CH3	Вход/выход канала 3 таймера 1
		Альтернативная функция	TMR2_CH3	Вход/выход канала 3 таймера 2
		Переопределенная функция	SSP1_TXD	Выходные данные SPI1
K9	PD13	Вход/выход 13 порта D		
		Аналоговая функция	ADC13	Входной канал 13 АЦП
		Основная функция	TMR1_CH3N	Инверсный выход канала 3 таймера 1
		Альтернативная функция	TMR2_CH3N	Инверсный выход канала 3 таймера 2
		Переопределенная функция	CAN1_TX	Выходные данные интерфейса CAN1

Номер вывода	Обозначение вывода	Назначение и функции вывода		
		Тип функции вывода	Обозначение функции вывода	Функциональное назначение вывода
K8	PD14	Вход/выход 14 порта D		
		Аналоговая функция	ADC14	Входной канал 14 АЦП
		Основная функция	TMR1_CH4	Вход/выход канала 4 таймера 1
		Альтернативная функция	TMR2_CH4	Вход/выход канала 4 таймера 2
		Переопределенная функция	CAN1_RX	Входные данные интерфейса CAN1
M8	PD15	Вход/выход 15 порта D		
		Аналоговая функция	ADC15	Входной канал 15 АЦП
		Основная функция	CAN2_RX	Входные данные интерфейса CAN2
		Альтернативная функция	BUSY2	Вход сигнала занятости внешнего устройства EXT_BUS
		Переопределенная функция	EXT_INT3	Вход внешнего прерывания
Порт E				
L7	PE0	Вход/выход 0 порта E		
		Аналоговая функция	DAC2_OUT	Аналоговый выход ЦАП2
		Основная функция	ADDR16	Разряд 16 шины адреса
		Альтернативная функция	TMR2_CH1	Вход/выход канала 1 таймера 2
		Переопределенная функция	CAN1_RX	Входные данные интерфейса CAN1
K7	PE1	Вход/выход 1 порта E		
		Аналоговая функция	DAC2_REF	Опорное напряжение ЦАП 2
		Основная функция	ADDR17	Разряд 17 шины адреса ADDR [31:0] EXT_BUS
		Альтернативная функция	TMR2_CH1N	Инверсный выход канала 1 таймера 2
		Переопределенная функция	CAN1_TX	Выходные данные интерфейса CAN1
L5	PE2	Вход/выход 2 порта E		
		Аналоговая функция	COMP_IN1	Вход 1 компаратора
		Основная функция	ADDR18	Разряд 18 шины адреса ADDR [31:0] EXT_BUS
		Альтернативная функция	TMR2_CH3	Вход/выход канала 3 таймера 2
		Переопределенная функция	TMR3_CH1	Вход/выход канала 1 таймера 3
K5	PE3	Вход/выход 3 порта E		
		Аналоговая функция	COMP_IN2	Вход 2 компаратора
		Основная функция	ADDR19	Разряд 19 шины адреса
		Альтернативная функция	TMR2_CH3N	Инверсный выход канала 3 таймера 2
		Переопределенная функция	TMR3_CH1N	Инверсный выход канала 1 таймера 3

Номер вывода	Обозначение вывода	Назначение и функции вывода		
		Тип функции вывода	Обозначение функции вывода	Функциональное назначение вывода
M4	PE4	Вход/выход 4 порта E		
		Аналоговая функция	COMP_REF+	Положительное опорное напряжение компаратора
		Основная функция	ADDR20	Разряд 20 шины адреса ADDR [31:0] EXT_BUS
		Альтернативная функция	TMR2_CH4N	Инверсный выход канала 4 таймера 2
		Переопределенная функция	TMR3_CH2	Вход/выход канала 2 таймера 3
L4	PE5	Вход/выход 5 порта E		
		Аналоговая функция	COMP_REF-	Отрицательное опорное напряжение компаратора
		Основная функция	ADDR21	Разряд 21 шины адреса ADDR [31:0] EXT_BUS
		Альтернативная функция	TMR2_BLK	Вход внешнего события таймера 2
		Переопределенная функция	TMR3_CH2N	Инверсный выход канала 2 таймера 3
L1	PE6	Вход/выход 6 порта E		
		Аналоговая функция	OSC_IN32	Вход осциллятора 32 кГц
		Основная функция	ADDR22	Разряд 22 шины адреса ADDR [31:0] EXT_BUS
		Альтернативная функция	CAN2_RX	Входные данные интерфейса CAN2
		Переопределенная функция	TMR3_CH3	Вход/выход канала 3 таймера 3
L2	PE7	Вход/выход 7 порта E		
		Аналоговая функция	OSC_OUT32	Выход осциллятора 32 кГц
		Основная функция	ADDR23	Разряд 23 шины адреса ADDR [31:0] EXT_BUS
		Альтернативная функция	CAN2_TX	Выходные данные интерфейса CAN2
		Переопределенная функция	TMR3_CH3N	Инверсный выход канала 3 таймера 3
J5	PE8	Вход/выход 8 порта E		
		Аналоговая функция	COMP_IN3	Вход 3 компаратора
		Основная функция	ADDR24	Разряд 24 шины адреса ADDR [31:0] EXT_BUS
		Альтернативная функция	TMR2_CH4	Вход/выход канала 4 таймера 2
		Переопределенная функция	TMR3_CH4	Вход/выход канала 4 таймера 3
K6	PE9	Вход/выход 9 порта E		
		Аналоговая функция	DAC1_OUT	Аналоговый выход ЦАП1
		Основная функция	ADDR25	Разряд 25 шины адреса ADDR [31:0] EXT_BUS
		Альтернативная функция	TMR2_CH2	Вход/выход канала 2 таймера 2
		Переопределенная функция	TMR3_CH4N	Инверсный выход канала 4 таймера 3

Номер вывода	Обозначение вывода	Назначение и функции вывода		
		Тип функции вывода	Обозначение функции вывода	Функциональное назначение вывода
M6	PE10	Вход/выход 10 порта E		
		Аналоговая функция	DAC1_REF	Опорное напряжение ЦАП1
		Основная функция	ADDR26	Разряд 26 шины адреса ADDR [31:0] EXT_BUS
		Альтернативная функция	TMR2_CH2N	Инверсный выход канала 2 таймера 2
		Переопределенная функция	TMR3_ETR	Вход внешнего события таймера 3
H3	PE11	Вход/выход 11 порта E		
		Аналоговая функция	—	—
		Основная функция	ADDR27	Разряд 27 шины адреса ADDR [31:0] EXT_BUS
		Альтернативная функция	nSIRIN1	Входные данные IRDA SIR UART1
		Переопределенная функция	TMR3_BLK	Вход внешнего события таймера 3
H4	PE12	Вход/выход 12 порта E		
		Аналоговая функция	—	—
		Основная функция	ADDR28	Разряд 28 шины адреса ADDR [31:0] EXT_BUS
		Альтернативная функция	SSP1_RXD	Входные данные SPI1
		Переопределенная функция	UART1_RXD	Входные данные UART1
G4	PE13	Вход/выход 13 порта E		
		Аналоговая функция	—	—
		Основная функция	ADDR29	Разряд 29 шины адреса ADDR [31:0] EXT_BUS
		Альтернативная функция	SSP1_FSS	Вход/выход разрешения SPI1
		Переопределенная функция	UART1_TXD	Выходные данные UART1
K4	PE14	Вход/выход 14 порта E		
		Аналоговая функция	—	—
		Основная функция	ADDR30	Разряд 30 шины адреса ADDR [31:0] EXT_BUS
		Альтернативная функция	TMR2_ETR	Вход внешнего события таймера 2
		Переопределенная функция	SCL1	Выход синхросигнала I2C
G1	PE15	Вход/выход 15 порта E		
		Аналоговая функция	—	—
		Основная функция	ADDR31	Разряд 31 шины адреса
		Альтернативная функция	EXT_INT3	Вход внешнего прерывания
		Переопределенная функция	SDA1	Вход/выход данных I2C

Номер вывода	Обозначение вывода	Назначение и функции вывода		
		Тип функции вывода	Обозначение функции вывода	Функциональное назначение вывода
Порт F				
C1	PF0	Вход/выход 0 порта F		
		Аналоговая функция	—	—
		Основная функция	ADDR0	Разряд 0 шины адреса ADDR [31:0] EXT_BUS
		Альтернативная функция	SSP1_TXD	Выходные данные SPI1
		Переопределенная функция	UART2_RXD	Входные данные UART2
C2	PF1	Вход/выход 1 порта F		
		Аналоговая функция	—	—
		Основная функция	ADDR1	Разряд 1 шины адреса ADDR [31:0] EXT_BUS
		Альтернативная функция	SSP1_CLK	Вход/выход синхросигнала SPI1
		Переопределенная функция	UART2_TXD	Выходные данные UART2
D3	PF2	Вход/выход 2 порта F		
		Аналоговая функция	—	—
		Основная функция	ADDR2	Разряд 2 шины адреса ADDR [31:0] EXT_BUS
		Альтернативная функция	SSP1_FSS	Вход/выход разрешения SPI1
		Переопределенная функция	CAN2_RX	Входные данные интерфейса CAN2
D2	PF3	Вход/выход 3 порта F		
		Аналоговая функция	—	—
		Основная функция	ADDR3	Разряд 3 шины адреса ADDR [31:0] EXT_BUS
		Альтернативная функция	SSP1_RXD	Входные данные SPI1
		Переопределенная функция	CAN2_TX	Выходные данные интерфейса CAN2
D1	PF4/ MODE[0]	Вход/выход 4 порта F/ Режим работы микроконтроллера		
		Аналоговая функция	—	—
		Основная функция	ADDR4	Разряд 4 шины адреса ADDR [31:0] EXT_BUS
		Альтернативная функция	—	—
		Переопределенная функция	—	—
E4	PF5/ MODE[1]	Вход/выход 5 порта F/ Режим работы микроконтроллера		
		Аналоговая функция	—	—
		Основная функция	ADDR5	Разряд 5 шины адреса ADDR [31:0] EXT_BUS
		Альтернативная функция	—	—
		Переопределенная функция	—	—

Номер вывода	Обозначение вывода	Назначение и функции вывода		
		Тип функции вывода	Обозначение функции вывода	Функциональное назначение вывода
E3	PF6/ MODE[2]	Вход/выход 6 порта F/ Режим работы микроконтроллера		
		Аналоговая функция	—	—
		Основная функция	ADDR6	Разряд 6 шины адреса ADDR [31:0] EXT_BUS
		Альтернативная функция	TMR1_CH1	Вход/выход канала 1 таймера 1
		Переопределенная функция	—	—
E2	PF7	Вход/выход 7 порта F		
		Аналоговая функция	—	—
		Основная функция	ADDR7	Разряд 7 шины адреса ADDR [31:0] EXT_BUS
		Альтернативная функция	TMR1_CH1N	Инверсный выход канала 1 таймера 1
		Переопределенная функция	TMR3_CH1	Вход/выход канала 1 таймера 3
E1	PF8	Вход/выход 8 порта F		
		Аналоговая функция	—	—
		Основная функция	ADDR8	Разряд 8 шины адреса ADDR [31:0] EXT_BUS
		Альтернативная функция	TMR1_CH2	Вход/выход канала 2 таймера 1
		Переопределенная функция	TMR3_CH1N	Инверсный выход канала 1 таймера 3
E5	PF9	Вход/выход 9 порта F		
		Аналоговая функция	—	—
		Основная функция	ADDR9	Разряд 9 шины адреса ADDR [31:0] EXT_BUS
		Альтернативная функция	TMR1_CH2N	Инверсный выход канала 2 таймера 1
		Переопределенная функция	TMR3_CH2	Вход/выход канала 2 таймера 3
F4	PF10	Вход/выход 10 порта F		
		Аналоговая функция	—	—
		Основная функция	ADDR10	Разряд 10 шины адреса ADDR [31:0] EXT_BUS
		Альтернативная функция	TMR1_CH3	Вход/выход канала 3 таймера 1
		Переопределенная функция	TMR3_CH2N	Инверсный выход канала 3 таймера 2
F2	PF11	Вход/выход 11 порта F		
		Аналоговая функция	—	—
		Основная функция	ADDR11	Разряд 11 шины адреса ADDR [31:0] EXT_BUS
		Альтернативная функция	TMR1_CH3N	Инверсный выход канала 3 таймера 1
		Переопределенная функция	TMR3_ETR	Вход внешнего события таймера 3

Номер вывода	Обозначение вывода	Назначение и функции вывода		
		Тип функции вывода	Обозначение функции вывода	Функциональное назначение вывода
F1	PF12	Вход/выход 12 порта F		
		Аналоговая функция	—	—
		Основная функция	ADDR12	Разряд 12 шины адреса ADDR [31:0] EXT_BUS
		Альтернативная функция	TMR1_CH4	Вход/выход канала 4 таймера 1
		Переопределенная функция	SSP2_FSS	Вход/выход разрешения SPI2
F3	PF13	Вход/выход 13 порта F		
		Аналоговая функция	—	—
		Основная функция	ADDR13	Разряд 13 шины адреса ADDR [31:0] EXT_BUS
		Альтернативная функция	TMR1_CH4N	Инверсный выход канала 4 таймера 1
		Переопределенная функция	SSP2_CLK	Вход/выход синхросигнала SPI2
G3	PF14	Вход/выход 14 порта F		
		Аналоговая функция	—	—
		Основная функция	ADDR14	Разряд 14 шины адреса ADDR [31:0] EXT_BUS
		Альтернативная функция	TMR1_ETR	Вход внешнего события таймера 1
		Переопределенная функция	SSP2_RXD	Входные данные SPI2
		Тип функции вывода	Обозначение функции вывода	Функциональное назначение вывода
G2	PF15	Вход/выход 15 порта F		
		Аналоговая функция	—	—
		Основная функция	ADDR15	Разряд 15 шины адреса ADDR [31:0] EXT_BUS
		Альтернативная функция	TMR1_BLK	Вход внешнего события таймера 1
		Переопределенная функция	SSP2_TXD	Выходные данные SPI2
Системное управление				
M1	RESET	Сигнал внешнего сброса		
M2	WAKEUP	Сигнал внешнего выхода из режима Standby		
K1	STANDBY	Флаг режима Standby		
M3	OSC_IN	Вход генератора HSE		
L3	OSC_OUT	Выход генератора HSE		
USB интерфейс				
H2	DP	Шина USB D+		
H1	DN	Шина USB D-		

Номер вывода	Обозначение вывода	Назначение и функции вывода		
		Тип функции вывода	Обозначение функции вывода	Функциональное назначение вывода
Питание				
A1, B1, B11, B12, J4, J11, J12, K3	Ucc	Питание (2,2 – 3,6) В		
J8, L8	AUcc	Питание аналоговое, АЦП, ЦАП, компаратор (должно совпадать с Ucc)		
J6, L6	AUcc1	Питание аналоговое, АЦП, ЦАП, компаратор (должно совпадать с Ucc)		
K2	BUcc	Питание батарейного домена (1,8 – 3,6) В		
E6, E7, F5 – F8, G5 – G8, H6, H7, J1	GND	Общий		
J7, M7	AGND	Общий		
H5, M5	AGND1	Общий		
Выводы для тестирования				
A2, A11, H9, J2	DUcc	Тестовый выход внутреннего питания 1,8 В. Рекомендуется не подсоединять		
J3	SHDN	Тестовый вход включения/отключения внутреннего регулятора LDO. Рекомендуется не подсоединять или подключить к шине Общий		
C12	JTAG_EN	Тестовый вход включения/отключения ТАР-контроллера. Рекомендуется не подсоединять или подключить к шине Общий		
Не используются				
A12, C9, D7, E8, H8	NC	Не используются. Рекомендуется не подсоединять или подключить к шине Общий		

6 Диаграмма расположения выводов в планарных корпусах

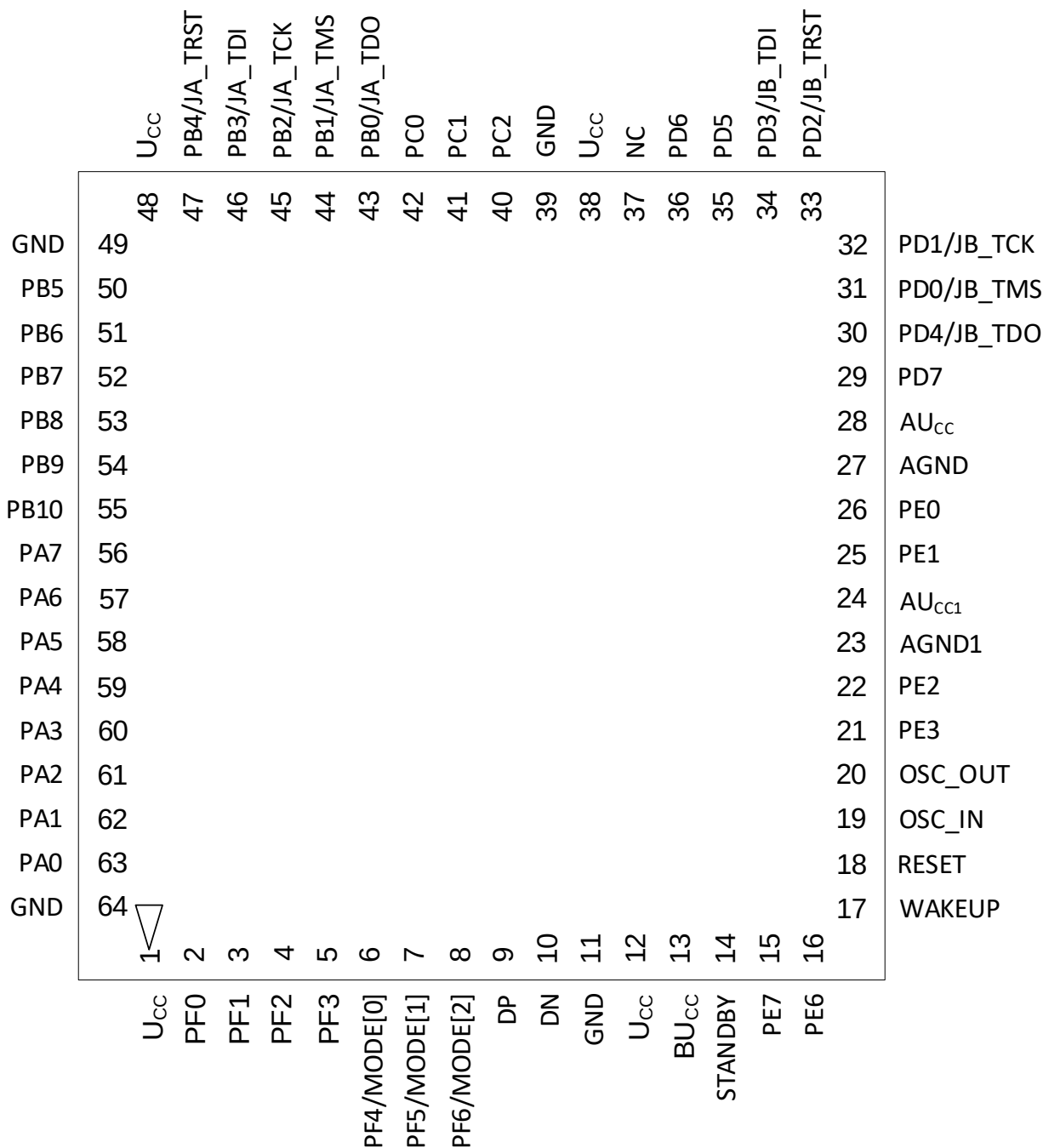


Рисунок 5 – Диаграмма расположения выводов микросхем K1986BE92FI, K1986BE92FII

7 Указания по применению и эксплуатации

Указания по режимам и условиям монтажа микросхем согласно ТСКЯ.430106.004Д12.

Выводы микросхем обеспечивают способность их к пайке при температуре 220 °С.

Микросхемы K1986BE92FI выдерживают воздействие тепла, возникающего при температуре пайки (250 + 5) °С в течение не более (10 ± 1) с.

Микросхемы K1986BE92FII, K1986BE94GI выдерживают воздействие тепла, возникающего при температуре пайки (220 + 5) °С в течение не более (10 ± 1) с.

Между выводами питания и общий устанавливается фильтрующая емкость не менее 0,1 мкФ.

Для задания режимов MODE при запуске микросхем должны использоваться резисторы (номинал не более 10 кОм) доопределения до «0» или питания, подключаемые к следующим выводам:

- 6, 7, 8 микросхем K1986BE92FI, K1986BE92FII;
- D1, E3, E4 микросхем K1986BE94GI.

В зависимости от рабочей частоты микроконтроллера для правильной работы регулятора LDO должны быть настроены поля SelectRI[2:0] и LOW[2:0] регистра REG_0E блока батарейного домена (при переходе на более высокую рабочую частоту настройка полей выполняется до перехода, при переходе с высокой частоты на низкую настройка полей выполняется после смены частоты).

Для обеспечения корректной работы микроконтроллера значения Trim[2:0], HSI_TRIM[5:0] и LSI_TRIM[4:0] должны быть считаны из информационной памяти и записаны в регистры REG_0E и REG_0F.

При использовании выводов порта В и порта D, задействованных для отладочных интерфейсов, а также при некорректной настройке частоты тактирования ядра микроконтроллера, нарушается работа отладочного интерфейса. В первом случае микроконтроллер не определяется в среде программирования, во втором – микроконтроллер может определяться в среде программирования, но не осуществляется переход в режим отладки и, как следствие, не выполняются операции стирания и программирования микросхемы.

В микросхемах K1986BE94GI не допускается подведение каких-либо сигналов, кроме сигнала общий, к выводам A12, C9, C12, D7, E8, H8, J3.

В микросхемах K1986BE92FI, K1986BE92FII не допускается подведение каких-либо сигналов, кроме сигнала общий, к выводу 37.

Неиспользуемые пользовательские выводы PA – PF, RESET, WAKEUP в цифровом режиме должны быть доопределены до допустимых логических уровней напряжения (высокого или низкого) с помощью внутренних программируемых резисторов доопределения или через внешние резисторы номиналом (1 – 100) кОм, или должны быть переведены в аналоговый режим.

Выводы A2, A11, H9, J2 микросхем K1986BE94GI рекомендуется оставлять свободными.

Металлизация обратной стороны корпуса микросхем K1986BE92FI электрически изолирована от выводов микросхемы. Рекомендуется не подсоединять или подключить к шине общий.

Запрещается подведение каких-либо электрических сигналов (в том числе шин питания, общий) к выводам микросхем, не используемым согласно таблицам 2 – 5.

Порядок подачи и снятия напряжение питания и входных сигналов на микросхемы:

- подача (включение микросхемы) – общий, питание батарейного домена U_{CCB} , напряжение питания U_{CC} , входные сигналы или одновременно;
- снятие (выключение микросхемы) – в обратном порядке или одновременно.

8 Система питания

Микросхемы имеют несколько типов выводов питания:

U_{CC} – основное питание микросхемы включает питание пользовательских выводов, встроенного регулятора напряжения, USB PHY и генераторов. Входное напряжение должно быть в пределах от 2,2 до 3,6 В. Если используется интерфейс USB, то входное напряжение должно быть в пределах от 3,0 до 3,6 В. Если используется АЦП или ЦАП, то входное напряжение должно быть в пределах от 2,4 до 3,6 В;

DU_{CC} – питание внутренней цифровой части, памяти ОЗУ и Flash-памяти. Это питание формируется внутренним регулятором напряжения из U_{CC}. В нормальном режиме работы этот вывод должен остаться неподсоединенным. В некоторых корпусах данные выводы отсутствуют. Напряжение на выводе DU_{CC} должно быть в пределах от 1,62 до 1,98 В;

BU_{CC} – питание батарейного домена используется при отсутствии основного питания U_{CC} для питания батарейного домена и LSE генератора. Переключение с основного питания на батарейное происходит автоматически при снижении уровня U_{CC} ниже 2,0 В. Переключение с батарейного питания на основное происходит автоматически спустя примерно 4 мс после превышения уровнем U_{CC} порога в 2,0 В. Входное напряжение должно быть в пределах от 1,8 до 3,6 В. Если в системе не требуется батарейное питание, то вывод BU_{CC} должен быть объединен с U_{CC};

BDU_{CC} – результирующие напряжения после выбора между BU_{CC} и U_{CC} при питании батарейного домена. В нормальном режиме этот вывод должен остаться неподсоединенным. В некоторых корпусах данные выводы отсутствуют;

AU_{CC} – питание аналоговых блоков АЦП, ЦАП и Компаратора выведено на отдельные выводы для уменьшения помех, создаваемых работой других блоков. На данные выводы рекомендуется подавать напряжение из того же источника, что и U_{CC}, но при этом на печатной плате должны быть применены меры по снижению помех. Допускается использование отдельного источника для питания аналоговых блоков, при этом его выходное напряжение не должно отличаться от U_{CC} более чем на $\pm 0,2$ В. Для корректной работы АЦП входное напряжение должно быть в пределах от 2,4 до 3,6 В. Если входное напряжение будет в пределах от 2,2 до 2,4 В, то корректная работа АЦП не гарантируется;

AU_{CC1} – питание аналоговых блоков и схем PLL выведено на отдельные выводы для уменьшения помех, создаваемых работой других блоков. На данные выводы должно подаваться напряжение из того же источника, что и U_{CC}, но при этом на печатной плате должны быть применены меры по снижению помех;

GND – основные выводы «Общий» питания;

AGND – выводы «Общий» аналогового питания AU_{CC}. Данные выводы должны соединяться с GND, но при этом на печатной плате должны быть применены меры по снижению помех;

AGND1 – выводы «Общий» аналогового питания AU_{CC1}. Данные выводы должны соединяться с GND, но при этом на печатной плате должны быть применены меры по снижению помех.

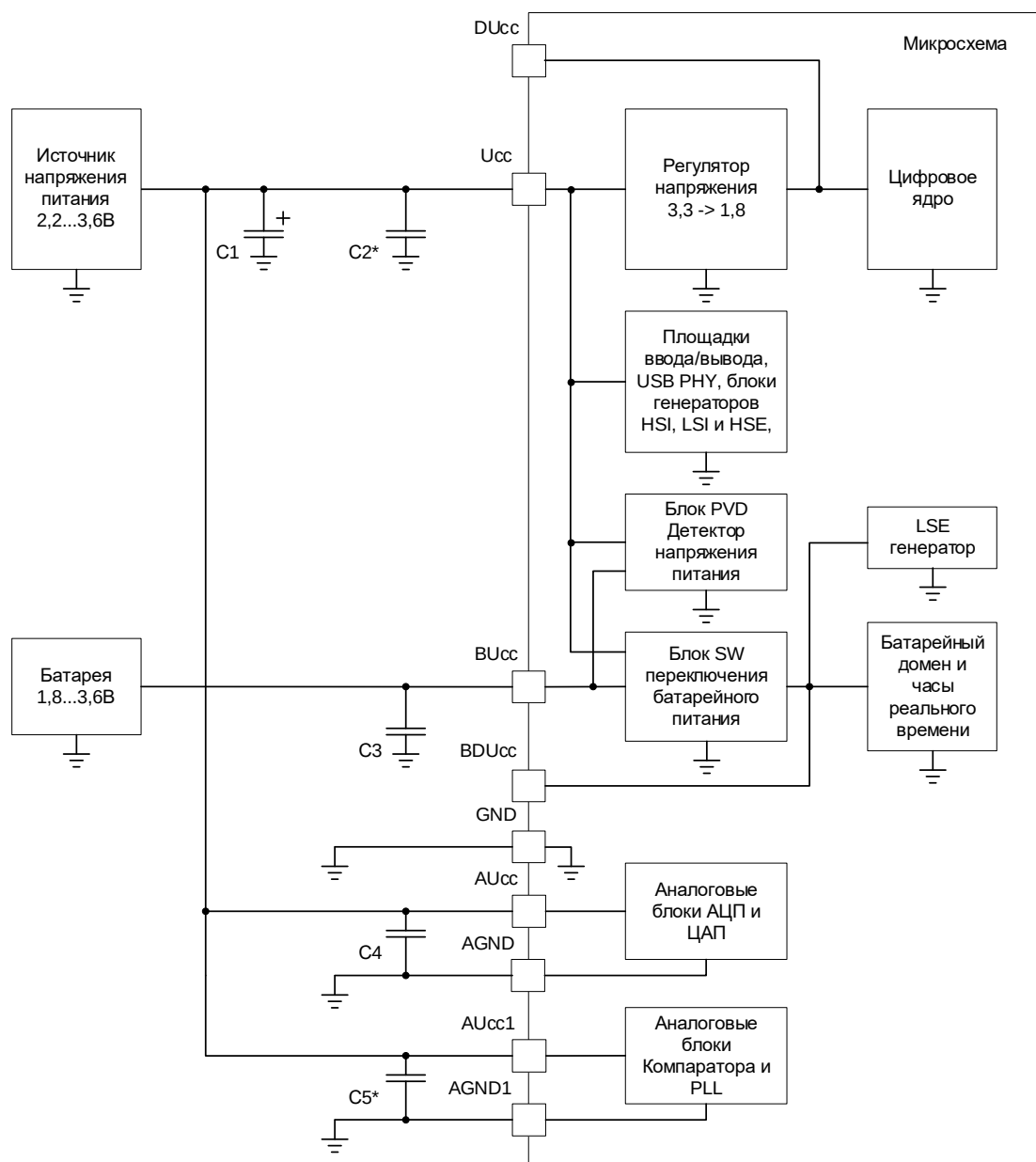


Рисунок 6 – Структурная блок-схема подачи питания

Примечания

1 Конденсаторы должны быть установлены у каждого вывода питания;

2 Конденсатор C1 = 22 мкФ, C2 = C3 = C4 = C5 = 0,1 мкФ;

3 Если не используется батарейное питание, то вывод BU_{CC} должен быть объединен с U_{CC};

4 Если используется интерфейс USB, то напряжение питания U_{CC} должно быть в пределах от 3,0 до 3,6 В;

5 Если используется АЦП или ЦАП, то напряжение питания U_{CC} (AU_{CC} и AU_{CC1}) должно быть в пределах от 2,4 до 3,6 В.

6 Допускается использование отдельного источника для питания аналоговых блоков, при этом его выходное напряжение не должно отличаться от U_{CC} более чем на ± 0,2 В.

Микросхема имеет несколько режимов энергопотребления, подробнее см. раздел 16 «Сигналы тактовой частоты» и подраздел 31.8 «Управление электропитанием». Микросхема имеет встроенный детектор напряжения питания, подробнее см. раздел 19 «Детектор напряжения питания».

8.1 Схема сброса при включении и выключении основного питания

При включении питания вырабатывается внутренний сигнал сброса POR для цифровой части, питание U_{CC} нарастает и, пока оно не превысило уровень $\sim 2,1$ В, сигнал сброса POR удерживается в низком уровне; после превышения данного уровня сигнал POR удерживается еще на протяжении ~ 6 мс для того, чтобы гарантировано установилось напряжение питания, после чего сигнал POR снимается, и схема может начать работать.

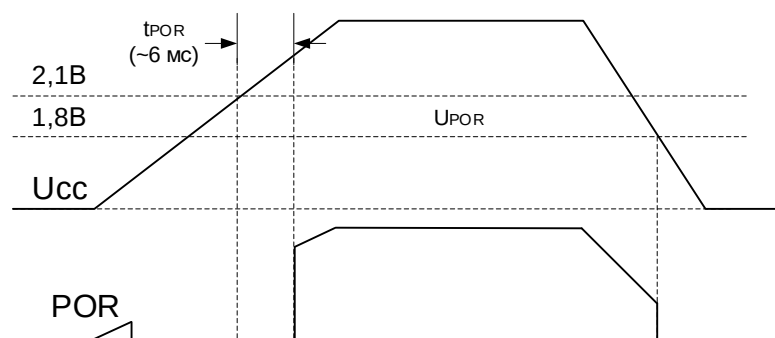


Рисунок 7 – Сигнал сброса при включении и выключении основного напряжения питания

При снижении напряжения питания U_{CC} ниже уровня $\sim 1,8$ В сигнал POR вырабатывается без задержки.

Сигнал POR также служит для переключения питания батарейного домена между BU_{CC} и U_{CC} .

При включении основного напряжения питания U_{CC} автоматически включается встроенный регулятор напряжения для формирования напряжения DU_{CC} питания цифрового ядра. В ходе работы микросхемы встроенный регулятор может быть отключен. Подробнее см. в подразделе 31.8 «Управление электропитанием».

Микросхема также может быть установлена в начальное состояние внешним сигналом сброса RESET, внутренними сигналами сброса сторожевых таймеров или программным сбросом. При этом сигнал сброса формируется специальной схемой сброса, содержащий фильтр «иголок» по сигналу сброса и одновибратор для увеличения длительности сигнала сброса.

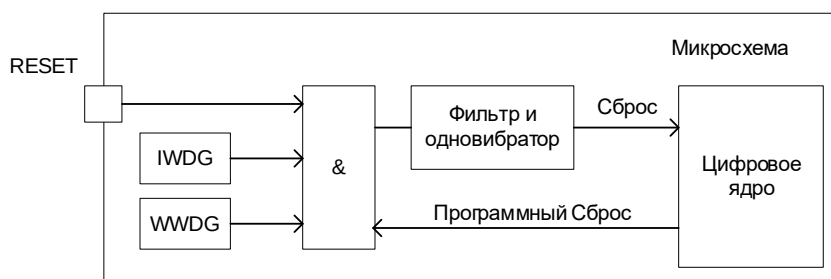


Рисунок 8 – Структурная блок-схема сброса

При приходе импульсов сброса длительностью менее 10 нс эти импульсы отфильтровываются и не приводят к сбросу процессора. Если длительность импульса больше 200 нс, вырабатывается сигнал сброса. При этом длительность сформированного сигнала сброса будет не менее 20 мкс.

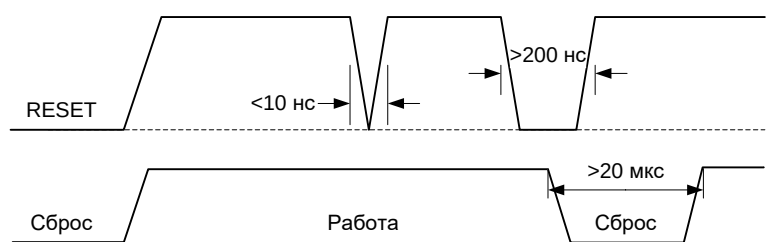


Рисунок 9 – Формирование сигнала сброса

9 Организация памяти

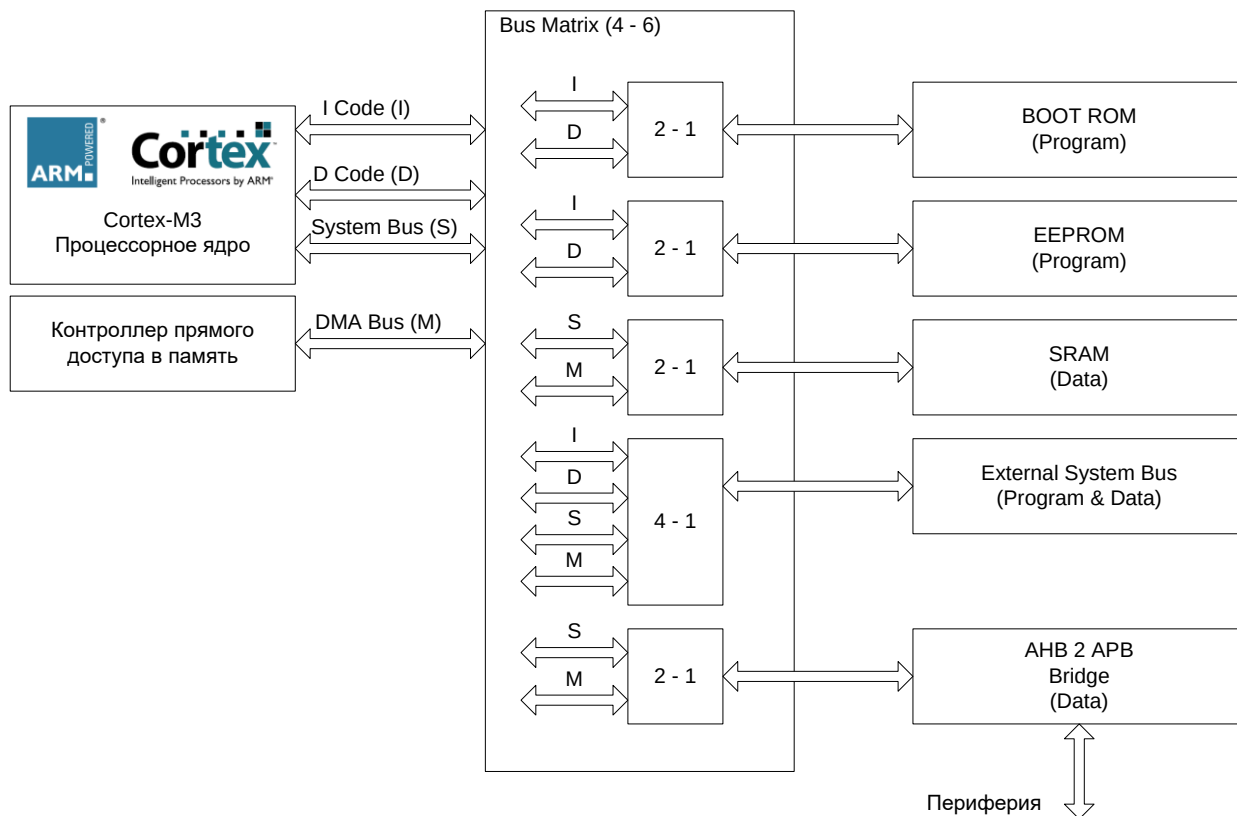


Рисунок 10 – Структурная схема организации памяти

Процессорное ядро имеет три системных шины:

I Code – шина выборки инструкций;

D Code – шина выборки данных, расположенных в коде программы;

S Bus – шина выборки данных, расположенных в области ОЗУ.

Также в микросхеме реализован контроллер прямого доступа в память (DMA), который осуществляет выборку через шину DMA Bus.

Все адресное пространство микросхемы едино и имеет максимальный объем 4 Гбайт. В данное адресное пространство отображаются различные модули памяти и периферии.

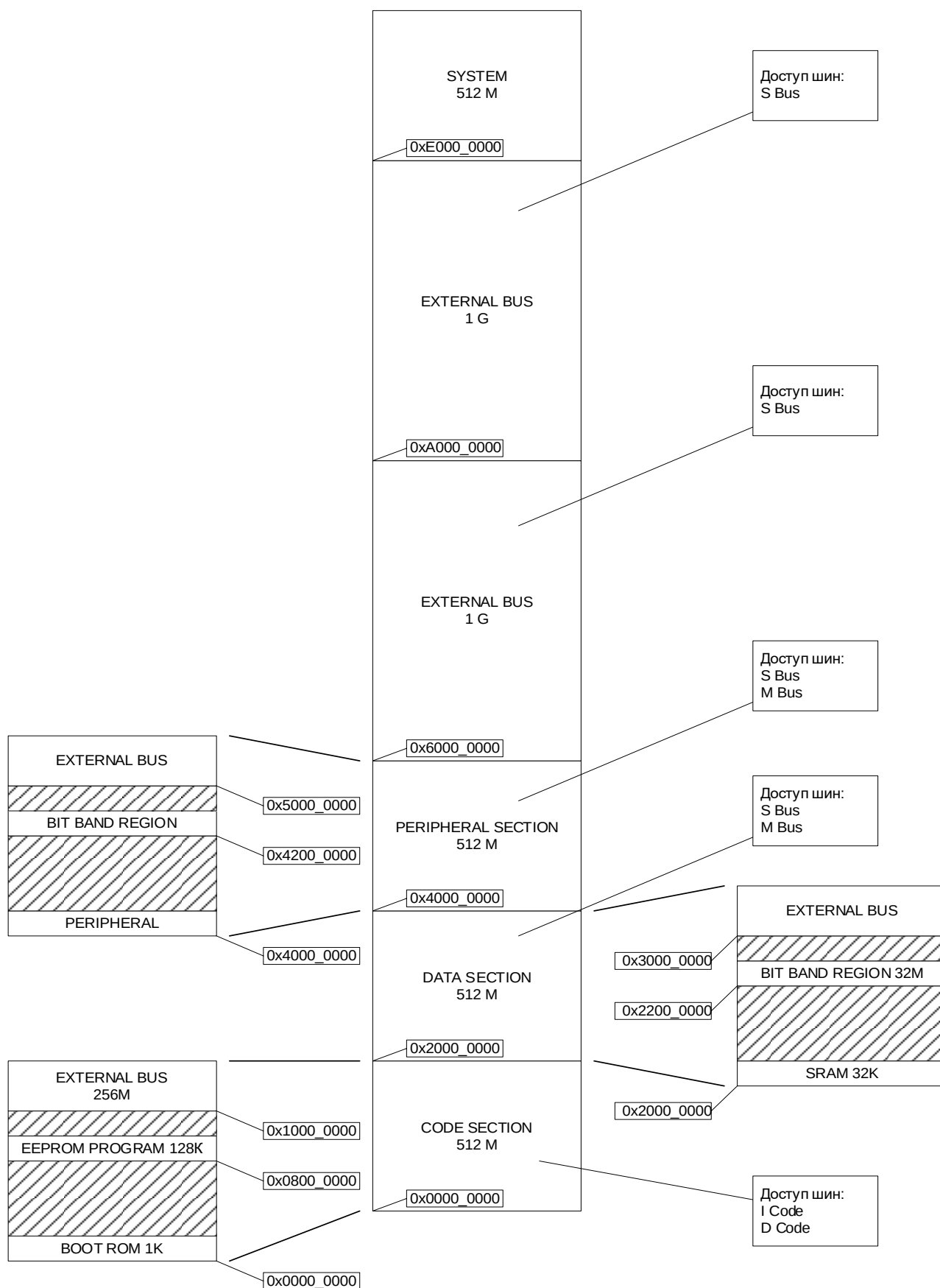


Рисунок 11 – Карта распределения основных областей памяти

9.1 Секция CODE

9.1.1 Область BOOT ROM

Предназначена для хранения программы запуска микросхемы; в ходе выполнения этой программы определяется режим запуска основной программы или переход в режим программирования микросхемы.

9.1.2 Область EEPROM PROGRAM

Основная область энергонезависимой памяти программы, доступной для перепрограммирования пользователем. Память предназначена для хранения основной рабочей программы.

9.1.3 Область EXTERNAL BUS

Область отображения внешней системной шины в адресное пространство области программы. Предназначена для хранения кода программ во внешних микросхемах памяти, подсоединенных к внешней системной шине.

9.2 Секция DATA

9.2.1 Область Internal SRAM (Data)

Основная область ОЗУ, предназначенная для хранения данных программы. В данной области также располагаются стек (stack) и «куча» (heap) программы. Адресные диапазоны стека и «кучи» задаются пользователем при написании программы.

9.2.2 Область BIT BAND REGION TO SRAM (Data)

Виртуальная область памяти данных, предназначенная для осуществления побитного доступа к области Internal SRAM. Работа с BIT BAND REGION позволяет осуществлять операции «Установка бита» и «Сброс Бита» одной инструкцией «Чтение-Модификация-Запись».

Запись слова в область alias-региона имеет тот же эффект, что и операция чтение-модификация-запись для целевого бита в области bit-band региона. Бит [0] записываемого значения в область alias-региона определяет значение, которое будет записано в целевой бит в области bit-band региона. Запись значения с установленным битом [0] записывает «1» в целевой бит bit-band региона, а запись значения с очищенным битом [0] записывает «0» в целевой бит bit-band региона. Биты [31:1] записываемого слова в область alias-региона не влияют на целевой бит bit-band региона. Запись 0x01 имеет тот же эффект, что и запись 0xFF. Запись 0x00 имеет тот же эффект, что и запись 0xFE. При чтении слова из области alias-региона будет возвращено значение 0x01 или 0x00. Биты [31:1] равны нулю. Значение 0x01 указывает, что целевой бит в области bit-band региона установлен. Значение 0x00 указывает, что целевой бит в области bit-band региона сброшен.

9.2.3 Область EXTERNAL BUS

Область отображения внешней системной шины в адресное пространство области данных. Предназначена для хранения данных во внешних микросхемах памяти, подсоединенных к внешней системной шине.

9.3 Секция PERIPHERAL

9.3.1 Область PERIPHERAL (Data)

Область отображения регистров периферии в общее адресное пространство памяти.

9.3.2 Область BIT BAND REGION TO PERIPHERAL (Data)

Виртуальная область памяти данных, предназначенная для осуществления побитного доступа к области PERIPHERAL. Работа с BIT BAND REGION позволяет осуществлять операции «Установка бита» и «Сброс Бита» одной инструкцией «Чтение-Модификация-Запись».

9.3.3 Область EXTERNAL BUS

Область отображения внешней системной шины в адресное пространство области периферии. Предназначена для хранения данных во внешних микросхемах памяти или для работы с периферийными устройствами, подсоединенными к внешней системной шине.

9.4 Секция EXTERNAL RAM

9.4.1 Область EXTERNAL BUS

Область отображения внешней системной шины в адресное пространство области внешней памяти и периферии. Эта секция предназначена для хранения данных во внешних микросхемах памяти или для работы с периферийными устройствами, подсоединенными к внешней системной шине.

9.5 Секция SYSTEM

Предназначена для отображения системных регистров ядра и системной периферии.

9.6 Блок BUS MATRIX

Блок BUS MATRIX предназначен для переключения системных шин I Code, D Code, System Bus и DMA Bus между различными областями памяти. Переключение производится автоматически на основании адреса запроса каждой конкретной шины. Если адреса запросов не пересекаются, то они могут быть выполнены одновременно. Если адреса запросов пересекаются, то они выполняются в порядке приоритета. Приоритеты обращений заданы аппаратно. Наивысшим приоритетом обладает запрос по шине SystemBus, затем следует запрос D Code, затем I Code и наименьшим приоритетом обладает запрос DMA Bus. Если два запроса пришли одновременно, то выполняется

запрос с большим приоритетом. Запрос с меньшим приоритетом задерживается до окончания выполнения запроса с большим приоритетом. При переключении между шинами возникает дополнительная задержка в один цикл. Если запросы идут непосредственно друг за другом, то дополнительных задержек не возникает.

9.7 Память BOOT ROM

Память области BOOT ROM реализована в виде MASK ROM, с занесением информации одним из технологических слоев при изготовлении кристалла микросхемы. Скорость доступа к памяти BOOT ROM – один цикл системной частоты.

9.8 Память EEPROM

Память области EEPROM реализована в виде перепрограммируемой энергонезависимой памяти. Скорость доступа к памяти EEPROM – порядка 40 нс. При работе микросхемы на скорости до 100 МГц скорость доступа к памяти может составлять до 5 циклов системной частоты. При последовательной выборке за счет упреждающего чтения задержка может быть сокращена до одного цикла системной частоты. Более подробная информация о работе контроллера EEPROM памяти программ представлена в разделе 11 «Контроллер Flash-памяти программ».

9.9 Память SRAM

Память области SRAM реализована в виде блока статической памяти. Скорость доступа к памяти SRAM – один цикл системной частоты.

9.10 Регионы памяти, типы и атрибуты

Карта памяти и модуль защиты памяти (MPU) разбивают все адресное пространство на регионы. Каждый регион имеет определенный тип памяти, а некоторые регионы имеют дополнительные атрибуты. Тип памяти и атрибуты определяют поведение системы при доступе к этим регионам. Подробнее см. раздел 15 «Модуль защиты памяти».

По отношению к порядку выполнения обращений к памяти различаются следующие типы памяти:

- Normal;
- Device;
- Strongly-ordered («Строго упорядоченная»).

9.10.1 Normal

Процессор может переопределить последовательность обращений для большей эффективности или чтобы выполнить опережающее чтение.

9.10.2 Device

Процессор сохраняет последовательность обращений по отношению к другим обращениям к памяти типов Device или Strongly-ordered.

9.10.3 Strongly-ordered

Процессор сохраняет последовательность обращений по отношению ко всем другим обращениям.

Различие в требованиях к памяти типов Device и Strongly-Ordered состоит в том, что система памяти может буферизировать запись в память типа Device, но буферизация записи в память типа Strongly-ordered не допускается.

Дополнительные атрибуты памяти:

- Shareable («Допускающая совместное использование»);
- Execute Never или XN («Не выполнять»).

9.10.4 Shareable

Для регионов с атрибутом Shareable система памяти обеспечивает синхронизацию между различными устройствами управления передачей данных по шине при наличии нескольких таких устройств, например, процессор и контроллер DMA.

Память типа Strongly-ordered всегда имеет атрибут Shareable.

Если несколько устройств управления передачей данных по шине могут обращаться к региону, не обладающему атрибутом Shareable, непротиворечивость данных между такими устройствами должна быть гарантирована программным обеспечением.

9.10.5 Execute Never (XN)

Атрибут указывает на запрет обращения к командам (инструкциям). Любая попытка извлечь инструкцию из XN региона приведет к исключению типа «Memory Management Fault».

9.11 Последовательность обращений к памяти

Для большинства обращений к памяти, инициируемых явно командами обращения к памяти, если только это не затрагивает поведения командной последовательности, система памяти не гарантирует, что порядок, в котором выполняются эти обращения, совпадает с порядком, в котором соответствующие команды следуют в программе.

Обычно, если правильное выполнение программы требует, чтобы два обращения к памяти произошли в порядке, заданном программой, то между командами обращения к памяти в программе должна быть вставлена инструкция барьерной синхронизации (memory barrier instruction), см. подраздел 9.12.2 «Программное упорядочение обращений к памяти».

Впрочем, система памяти гарантирует некоторый порядок доступа в регионы памяти Device и Strongly-ordered. Для двух команд обращения к памяти, A1 и A2, если A1 следует перед A2 в коде программы, последовательность обращений к памяти, вызванных этими двумя командами, будет такой, как показывает таблица 6.

Таблица 6 – Последовательность обращений инструкций к памяти

A1	A2			
	Normal	Device		Strongly-ordered
		«non-shareable»	shareable	
Normal	–	–	–	–
Device, «non-shareable»	–	<	–	<
Device, shareable	–	–	<	<
Strongly-ordered	–	<	<	<
Примечания 1 Обозначения в таблице: « – » – система памяти не гарантирует последовательность выполнения обращений; « < » – обращение к памяти вследствие инструкции A1 всегда будет происходить перед обращением вследствие инструкции A2. 2 Типы памяти Normal, Device, Strongly-ordered и атрибут Shareable см. подраздел 9.10 «Регионы памяти, типы и атрибуты»; «non-shareable» обозначает регион, не обладающий атрибутом Shareable				

9.12 Поведение обращений к памяти

Особенности доступа к разным областям памяти показывает таблица 7.

Таблица 7 – Поведение обращений к памяти

Адресный диапазон	Секция памяти	Тип памяти	XN	Описание
0x00000000-0x1FFFFFFF	Code	Normal	-	Область памяти для кода программы, данные также могут храниться здесь.
0x20000000-0x3FFFFFFF	SRAM	Normal	-	Область памяти для данных. Код программы также может располагаться здесь. Эта секция содержит области bit-band доступа
0x40000000-0x5FFFFFFF	Peripheral	Device	XN	Эта секция содержит области bit-band доступа
0x60000000-0x9FFFFFFF	External RAM	Normal	-	Область памяти для данных и кода
0xA0000000-0xDFFFFFFF	External Device	Device	XN	Область памяти для внешних устройств
0xE0000000-0xE00FFFFF	Private Peripheral Bus	Strongly-ordered	XN	Эта секция содержит регистры NVIC, системный таймер и регистры блока управления ядра
0xE0100000-0xFFFFFFFF	Резерв	Device	XN	Зарезервировано

Секции Code, SRAM и External RAM могут содержать код программы. Однако рекомендуется для программы использовать секцию Code, так как процессор имеет отдельные шины доступа к этой секции, что позволяет одновременно выполнять выборку инструкций и данных.

Блок MPU может изменить описанное здесь стандартное поведение при доступе в память. Подробности см. раздел 15 «Модуль защиты памяти».

9.12.1 Дополнительные условия доступа к совместно используемой памяти

Если система содержит совместно используемую память, следует учитывать для некоторых регионов дополнительные условия доступа, и для некоторых регионов их собственное разбиение, как это показывает таблица 8.

Таблица 8 – Дополнительные условия совместного использования памяти

Адресный диапазон	Секция памяти	Тип памяти	Возможность совместного использования	
0x00000000-0x1FFFFFFF	Code	Normal	-	
0x20000000-0x3FFFFFFF	SRAM	Normal	-	
0x40000000-0x5FFFFFFF	Peripheral	Device	-	
0x60000000-0x7FFFFFFF	External RAM	Normal	-	WBWA
0x80000000-0x9FFFFFFF				WT
0xA0000000-0xBFFFFFFF	External device	Device	Shareable	
0xC0000000-0xDFFFFFFF			«non-shareable»	
0xE0000000-0xE00FFFFF	Private peripheral bus	Strongly-ordered	Shareable	
0xE0100000-0xFFFFFFFF	Vendor-specific device	Device	-	
Примечание – Типы памяти Normal, Device, Strongly-ordered и атрибут Shareable см. подраздел 9.10 «Регионы памяти, типы и атрибуты»; «non-shareable» обозначает регион, не обладающий атрибутом Shareable				

9.12.2 Программное упорядочение обращений к памяти

Последовательность расположения инструкций (команд) в потоке программы не всегда гарантирует последовательность соответствующих обращений к памяти; это происходит потому, что:

- процессор может переменить порядок следования некоторых обращений для увеличения производительности при условии, что при этом не изменяется общее поведение программы;
- процессор имеет несколько интерфейсов для обращений к памяти;
- память или устройства могут иметь различные скорости доступа;
- для некоторых обращений к памяти имеет место буферизация или упреждающее выполнение.

Этот раздел описывает, как гарантировать при необходимости корректную последовательность обращений к памяти. Если порядок обращений к памяти критичен, программный код должен содержать инструкции барьерной синхронизации, чтобы

обеспечить нужный порядок. Процессор предлагает следующие инструкции (команды) барьерной синхронизации:

- **DMB** – инструкция Data Memory Barrier (DMB) позволяет быть уверенным, что все выполняемые обращения к памяти будут завершены до следующего обращения к памяти. См. пункт 13.9.2 «DMB»;

- **DSB** – инструкция Data Synchronization Barrier (DSB) позволяет быть уверенным, что все выполняемые обращения к памяти будут завершены до начала выполнения следующей команды (инструкции). См. пункт 13.9.3 «DSB»;

- **ISB** – инструкция Instruction Synchronization Barrier (ISB) позволяет быть уверенным, что эффект от выполнения всех завершенных обращений к памяти будет распространяться на последующие команды. См. пункт 13.9.4 «ISB».

Инструкции барьерной синхронизации используются, например, в таких случаях:

- Программирование MPU:

- используйте инструкцию DSB для уверенности в том, что изменение настроек MPU будет иметь эффект немедленно вслед за изменением контекста;

- используйте инструкцию ISB для уверенности в том, что изменение настроек MPU будет иметь эффект немедленно после программирования новых MPU регионов, если конфигурационный код MPU вызывается через переход или вызов функции. Если конфигурационный код MPU вызывается через механизм исключений (прерывания), то инструкция ISB не требуется;

- Таблица векторов прерывания. Если программа изменяет таблицу векторов прерывания и затем разрешает прерывания, то перед разрешением должна быть поставлена инструкция DMB. Это гарантирует, что в случае прерывания процессор уйдет на обработчик по новому адресу таблицы;

- Самомодифицируемый код. Если программа содержит самомодифицируемый код, используйте инструкцию ISB сразу после модификации кода программы. Это гарантирует, что после этого будет выполняться уже модифицированный код;

- Переключение карты памяти. Если система содержит механизм переключения карты памяти, то используйте инструкцию DSB после переключения карты памяти в программе. Это гарантирует, что дальнейшее выполнение инструкций будет идти с новой картой памяти;

- Динамическое изменение приоритетов исключений. Когда приоритеты исключений изменяются во время обработки исключения, используйте инструкцию DSB после изменения. Это гарантирует, что изменение произойдет при завершении инструкции DSB;

- Использование семафоров в системе с несколькими устройствами управления передачей данных по шине. Если система содержит несколько таких устройств управления, например, другой процессор, то оба процессора должны использовать инструкции DMB после каждой инструкции работы с семафорами. Это гарантирует, что другой мастер будет видеть обращения к памяти в той последовательности, в которой они выполняются.

Обращения к памяти типа Strongly-ordered, например, к системному блоку управления ядра (NVIC, System Timer и так далее) не требуют использовать инструкции DMB.

9.12.3 Bit-band регионы

Механизм bit-band отображает каждый бит bit-band региона в слово в bit-band alias регионе. Bit-band регион занимает младший 1 Мбайт области SRAM и области периферийных устройств (Peripheral). Им соответствуют области по 32 Мбайта bit-band alias, как это указано в таблице 9.

Таблица 9 – Описание bit-band регионов

Адресный диапазон	Регион памяти	Доступ к инструкциям и данным
0x2000_0000-0x200F_FFFF	SRAM bit-band	Доступ к словам
0x2200_0000-0x23FF_FFFF	SRAM bit-band alias	Доступ к битам в области SRAM bit-band через доступ к словам в области SRAM bit-band alias
0x4000_0000-0x400F_FFFF	Peripheral bit-band	Доступ к словам
0x4200_0000-0x43FF_FFFF	Peripheral bit-band alias	Доступ к битам в области Peripheral bit-band через доступ к словам в области Peripheral bit-band alias

Формулы показывают, как регион bit-band alias отображается в bit-band region

$$bit_word_offset = (byte_offset \cdot 32) + (bit_number \cdot 4); \quad (1)$$

$$bit_word_addr = bit_band_base + bit_word_offset, \quad (2)$$

где bit_word_offset – позиция необходимого бита в bit-band регионе;

bit_word_addr – адрес слова в bit-band alias регионе, отображающего необходимый бит в bit-band регионе;

bit_band_base – начальный адрес bit-band alias региона;

byte_offset – номер байта с необходимым битом в bit-band регионе;

bit_number – номер необходимого бита в байте.

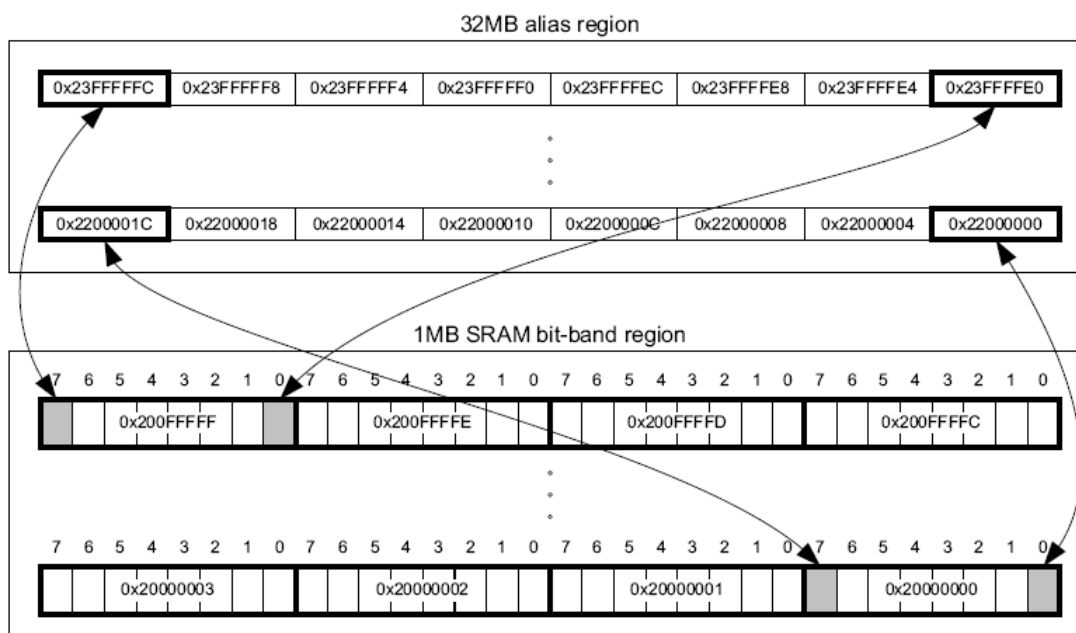


Рисунок 12 – Схема отображения региона bit-band alias в регионе bit-band region

Запись в слово в alias регионе обновляет бит в bit-band регионе.

Bit[0] записываемого слова будет определять значение устанавливаемого бита.

Bit[31:1] слова в bit-band alias регионе не имеют значения для бита в bit-band регионе. Запись 0x01 имеет тот же эффект, что и запись 0xFF, а запись 0x00 имеет тот же эффект, что и запись 0x0E.

Процессор имеет little-endian организацию расположения байтов, т.е. байт с меньшей значимостью хранится по меньшему адресу, например:

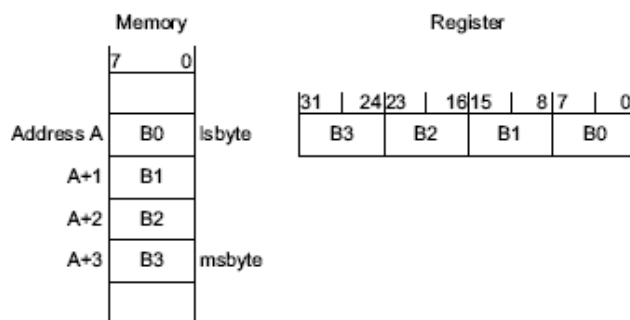


Рисунок 13 – Организация расположения байтов в 32-битной памяти

9.13 Прimitives синхронизации

Система команд Cortex-M3 включает в себя несколько парных *примитивов синхронизации*. Это позволяет реализовать неблокирующий механизм, который поток или процесс могут использовать для эксклюзивного доступа в память. Программное обеспечение может использовать их для гарантированного выполнения последовательности read-modify-write или реализации механизма семафоров.

В каждую пару команд (инструкций) примитива синхронизаций входят:

- команда Load-Exclusive;
- команда Store-Exclusive.

9.13.1 Команда Load-Exclusive

Используется, чтобы прочесть значение из некоторого адреса памяти; запрашивает эксклюзивный доступ к этому адресу.

9.13.2 Команда Store-Exclusive

Используется для попытки записи в тот же самый адрес памяти; возвращает бит статуса. Этот бит принимает значения:

- 0 – если поток или процесс получил эксклюзивный доступ к памяти и запись выполнена;
- 1 – если поток или процесс не получил эксклюзивного доступа в память и запись не выполнена.

Команды Load Exclusive и Store-Exclusive образуют следующие пары:

- LDREX и STREX – работа с словами;
- LDREXH и STREXH – работа с полусловами;
- LDREXB и STREXB – работа с байтами.

Программное обеспечение должно использовать инструкцию Load Exclusive с соответствующей инструкций Store-Exclusive.

Чтобы гарантировать чтение-модификацию-запись по какому-либо адресу памяти, программа должна:

- инструкцией Load-Exclusive считать значение из памяти;
- изменить значение;
- инструкцией Store-Exclusive попытаться записать новое значение обратно в память; проверить возвращаемый статусный бит. Если этот бит – 0, то процедура «чтение-модификация-запись» выполнена успешно. Если этот бит – 1, то запись не была выполнена. Это означает, что значение, считанное первоначально, возможно устарело, и программа должна повторить цикл «чтение-модификация-запись».

Программное обеспечение может использовать примитивы синхронизации для реализации семафоров, как это описано ниже:

- использовать команду Load-Exclusive для чтения из адреса семафора, чтобы определить, свободен ли семафор;
- если семафор свободен, использовать Store-Exclusive для записи в семафор требуемого значения (признака захвата);
- если возвращаемый статусный бит указывает на успешное выполнение инструкции Store-Exclusive, то это означает, что семафор захвачен. Если же команда Store-Exclusive не была выполнена, то это означает, что другой процесс мог захватить семафор ранее.

Ядро Cortex-M3 имеет монитор эксклюзивных доступов, который отмечает, что процессор выполнил команду Load-Exclusive. Если процессор является частью многопроцессорной системы, то система также отмечает на глобальном уровне адреса памяти, для которых имел место эксклюзивный доступ со стороны каждого процессора.

Процессор удаляет свою отметку об эксклюзивном доступе в тех случаях, когда:

- процессор выполняет команду CLREX;
- процессор выполняет команду Store-Exclusive, при этом не имеет значения, была ли запись успешна;

была ли запись успешна;

– происходит исключение. Это означает, что процессор может разрешить конфликт между семафорами в различных потоках.

В многопроцессорной реализации:

- выполнение инструкции CLREX удаляет только локальную отметку об эксклюзивном доступе для данного процессора;
- выполнение инструкции Store-Exclusive или же обработка исключения удаляют локальную и все глобальные отметки об эксклюзивном доступе для данного процессора.

Подробнее см. описание инструкций в пунктах 13.3.8 «LDREX и STREX» и 13.3.9 «CLREX».

9.13.3 Указания по программированию примитивов синхронизации

ANSI C не может создавать непосредственно инструкции эксклюзивного доступа. Некоторые C компиляторы предлагают встроенные функции для создания этих инструкций:

Таблица 10 – Встроенные функции для создания инструкций эксклюзивного доступа

Инструкции	Функции
LDREX, LDREXH, LDREXB	unsigned int __ldrex(volatile void *ptr)
STREX, STREXH, STREXB	int __strex(unsigned int val, volatile void *ptr)
CLREX	void __clrex(void)

Получаемые инструкции эксклюзивного доступа зависят от типа данных указателя, передаваемого встроенной функции. Например, следующий код создаст инструкцию LDREXB:

`__ldrex((volatile char *) 0xFF).`

9.14 Базовые адреса процессора

Таблица 11 – Базовые адреса процессора

Адрес	Размер	Блок	Примечание
Память программ			
0x0000_0000	1 Кбайт	BOOT ROM	Загрузочная программа
0x0800_0000	128 Кбайт	EEPROM	Область Flash-памяти программ с пользовательской программой
0x1000_0000	256 Мбайт	EXTERNAL BUS	Область доступа к внешней системной шине
Память данных			
0x2000_0000	32Кбайт	SYSTEM RAM	Область внутреннего ОЗУ
0x2200_0000	1 Мбайт	SYSTEM RAM Bit Band Region	Область битового доступа внутреннего ОЗУ
0x3000_0000	256 Мбайт	EXTERNAL BUS	Область доступа к внешней системной шине

Адрес	Размер	Блок		Примечание
Периферия				
0x4000_0000	1536 байт	0	CAN1	Регистры контроллера интерфейса CAN1
0x4000_8000	1536 байт	1	CAN2	Регистры контроллера интерфейса CAN2
0x4001_0000	904 байт	2	USB	Регистры контроллера интерфейса USB
0x4001_8000	20 байт	3	EEPROM_CNTRL	Регистры контроллера Flash-памяти программ
0x4002_0000	48 байт	4	RST_CLK	Регистры контроллера сигналов тактовой частоты
0x4002_8000	80 байт	5	DMA	Регистры контроллера прямого доступа в память
0x4003_0000	72 байт	6	UART1	Регистры контроллера интерфейса UART1
0x4003_8000	72 байт	7	UART2	Регистры контроллера интерфейса UART2
0x4004_0000	36 байт	8	SSP1	Регистры контроллера интерфейса SSP1
0x4004_8000	-	9	-	
0x4005_0000	28 байт	10	I2C1	Регистры контроллера интерфейса I2C1
0x4005_8000	4 байт	11	POWER	Регистры детектора напряжения питания
0x4006_0000	12 байт	12	WWDG	Регистры контроллера сторожевого таймера WWDG
0x4006_8000	16 байт	13	IWDG	Регистры контроллера сторожевого таймера IWDG
0x4007_0000	128 байт	14	TIMER1	Регистры управления Таймер 1
0x4007_8000	128 байт	15	TIMER2	Регистры управления Таймер 2
0x4008_0000	128 байт	16	TIMER3	Регистры управления Таймер 3
0x4008_8000	48 байт	17	ADC	Регистры управления АЦП
0x4009_0000	12 байт	18	DAC	Регистры управления ЦАП
0x4009_8000	12 байт	19	COMP	Регистры управления Компаратора
0x400A_0000	36 байт	20	SSP2	Регистры контроллера интерфейса SSP2
0x400A_8000	32 байт	21	PORTA	Регистры управления порта А
0x400B_0000	32 байт	22	PORTB	Регистры управления порта В
0x400B_8000	32 байт	23	PORTC	Регистры управления порта С
0x400C_0000	32 байт	24	PORTD	Регистры управления порта D
0x400C_8000	32 байт	25	PORTE	Регистры управления порта E
0x400D_0000	-	26	-	
0x400D_8000	84 байт	27	BKP	Регистры доступа и управления батарейным доменом
0x400E_0000	-	28	-	
0x400E_8000	32 байт	29	PORTF	Регистры управления порта F
0x400F_0000	88 байт	30	EXT_BUS_CNTRL	Область доступа к внешней системной шине
0x400F_8000	-	31	-	
0x4200_0000	32 Мбайт	PERIPHERAL Bit Band Region		Область битового доступа к регистрам периферии
0x5000_0000	256 Мбайт	EXTERNAL BUS		Область доступа к внешней системной шине
Внешняя системная шина				
0x6000_0000	1 Гбайт	EXTERNAL BUS		Область доступа к внешней системной шине
0xA000_0000	1 Гбайт	EXTERNAL BUS		Область доступа к внешней системной шине
SYSTEM REGION				
0xE000_0000	256 Мбайт			Системные регистры процессора ARM Cortex-M3

10 Загрузочное ПЗУ и режимы работы микросхемы

После включения питания и снятия внутренних (POR) и внешних (RESET) сигналов сброса, микросхема начинает выполнять программу из загрузочной области ПЗУ BOOT ROM. В загрузочной программе микросхема определяет, в каком из режимов она будет функционировать, и переходит в этот режим. Режим функционирования определяется внешними выводами MODE[2:0] (PF[6:4]), при этом перед опросом состояния этих выводов, для них включается внутренняя подтяжка к шине «Общий» (встроенные резисторы подтяжки к шине «Общий» имеют сопротивление ~50 кОм). Также устанавливается бит FPOR в регистре BKP_REG_0E, который может быть сброшен только при отключении основного питания U_{CC}. После перезапуска микросхемы уровни на выводах MODE[2:0] не влияют на режим функционирования микросхемы, если установлен бит FPOR. В пользовательской программе выводы PF[6:4] могут использоваться пользователем.

Таблица 12 – Режимы первоначального запуска микросхемы

MODE[2:0]	Режим	Стартовый адрес / таблица векторов прерываний	Описание
000	Микроконтроллер в режиме отладки	0x0800_0000	Микросхема начинает выполнять программу из внутренней Flash-памяти программ. При этом установлен отладочный интерфейс JTAG_B
001	Микроконтроллер в режиме отладки	0x0800_0000	Микросхема начинает выполнять программу из внутренней Flash-памяти программ. При этом разрешается работа отладочного интерфейса JTAG_A
010-011	Микропроцессор в режиме отладки	0x1000_0000	Микросхема конфигурирует внешнюю системную шину в режим работы ROM с Wait_States = 0xF и начинает выполнять программу из внешней памяти, установленной на внешней системной шине. При этом разрешается работа отладочного интерфейса JTAG_B. Примечание – Для микросхем K1986BE92FI, K1986BE92FII выполнение программы из внешней памяти не доступно
100	Зарезервировано	-	-
101	UART-загрузчик без отладки	Определяется пользователем	Микросхема через интерфейс UART2 на выводах PD[1:0] получает код программы в ОЗУ для исполнения. При этом отладочный интерфейс JTAG/SW заблокирован
110	UART-загрузчик без отладки	Определяется пользователем	Микросхема через интерфейс UART2 на выводах PF[1:0] получает код программы в ОЗУ для исполнения. При этом отладочный интерфейс JTAG/SW заблокирован
111	Тестовый режим работы	-	Микросхема переводится в тестовый режим работы, к выводам JTAG_A подключается тестовый ТАР-контроллер

При работе в режиме отладки разрешается работа отладочного интерфейса JTAG/SW. При этом к микросхеме может быть подключен JTAG/SW адаптер, с помощью которого программные средства разработки позволяют работать с микросхемой в режиме отладки. Линии JTAG должны быть подтянуты к питанию сопротивлениями не менее 10 кОм с учетом, чтобы эти подтяжки не влияли на работу системы.

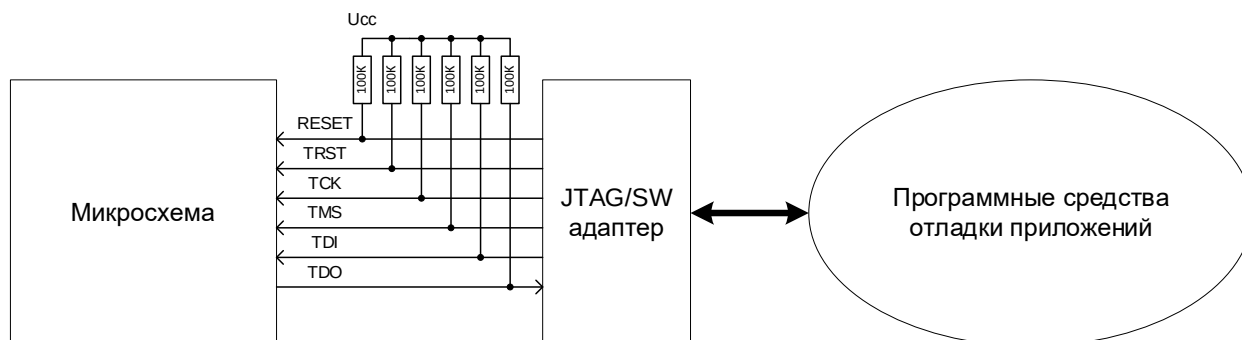


Рисунок 14 – Схема работы в режиме отладки

В режиме отладки можно:

- стирать, записывать, считывать внутреннюю Flash-память программ;
- считывать и записывать содержимое ОЗУ, периферии;
- выполнять программу в пошаговом режиме;
- запускать программу в нормальном режиме;
- останавливать программу по точкам останова;
- просматривать переменные выполняемой программы;
- проводить трассировку хода выполнения программного обеспечения.

В зависимости от режима работы выводы интерфейса JTAG/SW переопределяются на различные выводы микросхемы, как это показано в таблице 13.

Таблица 13 – Переопределение выводов интерфейса JTAG/SW

Вывод JTAG/SW	Вывод микросхемы	Описание
JTAG_A		
TRST	PB4/JA_TRST	В качестве выводов интерфейса используются выводы порта B, совмещенные с выводами данных внешней системной шины, выводами таймера 3, выводами UART1, UART2 и CAN1, использование которых при отладке запрещено
TCK	PB2/JA_TCK	
TMS	PB1/JA_TMS	
TDI	PB3/JA_TDI	
TDO	PB0/JA_TDO	
JTAG_B		
TRST	PD2/JB_TRST	В качестве выводов интерфейса используются выводы порта D, совмещенные с каналами АЦП, выводами каналов Таймера 1 и 3, UART2 и SSP2, использование которых при отладке запрещено
TCK	PD1/JB_TCK	
TMS	PD0/JB_TMS	
TDI	PD3/JB_TDI	
TDO	PD4/JB_TDO	

10.1 Микроконтроллер в режиме отладки

После определения данного режима работы (MODE[2:0] = 000 или 001) загрузочная программа выполняет следующие действия:

- разрешает работу отладочного интерфейса JTAG (регистр REG_0E):
 - при MODE[2:0] = 000 разрешается работа интерфейса JTAG_B;
 - при MODE[2:0] = 001 разрешается работа интерфейса JTAG_A;
- устанавливает адрес таблицы векторов в значение 0x0800_0000 (регистр VTOR);
- устанавливает указатель стека в значение, считанное из ячейки с адресом 0x0800_0000 (таблица векторов, поле «Initial SP value»);
- осуществляет безусловный переход по адресу, считанному из ячейки с адресом 0x0800_0004 (таблица векторов, поле «Reset»).

10.2 Микропроцессор в режиме отладки

Для микросхем K1986BE92FI, K1986BE92FII в данном режиме работы выполнение программы из внешней памяти не доступно.

После определения данного режима работы (MODE[2:0] = 010 или 011) загрузочная программа выполняет следующие действия:

- разрешает работу отладочного интерфейса JTAG_B (регистр REG_0E);
- конфигурирует набор выводов для работы с внешней памятью:
 - шина данных D[31:0] ({PB[15:0], PA[15:0]});
 - шина адреса A[31:0] ({PE[15:0], PF[15:0]});
 - сигнал чтения данных OE (PC[1]);
 - сигнал записи данных WE (PC[2]);
 - сигналы выборки байтов BE[3:0] (PC[6:3]);
 - сигнал синхронизации CLOCK (PC[7]);
- конфигурирует контроллер внешней системной шины MDR_EBC:
 - режим работы – ROM (только чтение);
 - разрядность шины данных – 32 бита;
 - длительность фаз обращения WAIT_STATE[3:0] = 0xF;
 - полярность сигнала CLOCK – положительная;
- устанавливает адрес таблицы векторов в значение 0x1000_0000 (регистр VTOR);
- устанавливает указатель стека в значение, считанное из ячейки с адресом 0x1000_0000 (таблица векторов, поле «Initial SP value»);
- осуществляет безусловный переход по адресу, считанному из ячейки с адресом 0x1000_0004 (таблица векторов, поле «Reset»).

10.3 UART-загрузчик без отладки

В режиме UART-загрузчика (MODE[2:0] = 101 или 110) используют один и тот же периферийный модуль UART2, один и тот же протокол обмена, но различные порты ввода/вывода, как это показывает таблица 14.

Таблица 14 – Используемые порты ввода/вывода UART-загрузчиком

Режим MODE[2:0]	TX	RX
101	PD[1]	PD[0]
110	PF[1]	PF[0]

Данные режимы работы предоставляют достаточный набор операций, необходимых для записи в ОЗУ какой-либо программы (в частности, программатора Flash-памяти), ее верификации и запуска на выполнение. Кроме того, существует возможность задания внешним устройством скорости обмена. Помимо доступа к ОЗУ, может быть осуществлен доступ и к другим адресным диапазонам (EEPROM, ROM, Периферия).

В качестве источника тактовой частоты UART2 используется внутренний RC-генератор HSI с частотой 8 МГц. Так как имеется разброс значений частоты HSI, то требуется этап подбора значения делителя частоты UART2 для синхронизации с внешним устройством.

После определения данного режима работы загрузочная программа запрещает работу отладочных интерфейсов JTAG_A и JTAG_B.

10.3.1 Параметры связи по UART

Для связи по UART выбраны следующие параметры канала связи:

- начальная скорость – 9600 Бод;
- количество бит данных – 8;
- четность – нет;
- количество Stop бит – 1;
- загрузчик не использует FIFO UART2;
- загрузчик всегда выступает в качестве Slave, а внешнее устройство, подающее команды – в качестве Master;
- данные передаются младшим битом вперед.

10.3.2 Протокол обмена по UART

После синхронизации с внешним устройством, подающим команды (Master), загрузчик переходит в диспетчер команд.

Таким образом, внешнему устройству доступны следующие команды:

Таблица 15 – Команды UART-загрузчика

Команда	Код	ASCII Символ	Описание
CMD_SYNC	0x00		Пустая команда. Загрузчик ее принимает, но ничего по ней не делает
CMD_CR	0x0D		Выдача приглашения Master-y

Команда	Код	ASCII Символ	Описание
CMD_BAUD	0x42	'B'	Установка скорости обмена
CMD_LOAD	0x4C	'L'	Загрузка массива байт
CMD_VFY	0x59	'Y'	Выдача массива байт
CMD_RUN	0x52	'R'	Запуск программы на выполнение

10.3.3 Синхронизация с внешним устройством

Начальные условия.

На этапе синхронизации с внешним устройством (Master) вывод Rx используется как вход. Master постоянно посылает в канал синхросимвол – 0. Загрузчик подстраивает свою скорость таким образом, чтобы минимизировать ошибки обмена. Как только Загрузчик настроил скорость, он переходит в диспетчер команд и выдает приглашение (три байта 0x0D (перевод строки), 0x0A (возврат каретки), 0x3E ('>'),) Master-у.

Master завершает выдачу синхросимволов и теперь может подавать команды согласно протоколу обмена.

10.3.4 Команда CMD_SYNC

Пустая команда.

Загрузчик (Slave) ее принимает, но ничего по ней не делает. Код команды соответствует символу синхронизации.

Таблица 16 – Команда CMD_SYNC

Код команды	CMD_SYNC = 0x00
ASCII символ, соответствующий коду команды	нет
Количество параметров команды	0
Формат команды:	
Master: Выдает код команды CMD_SYNC	Slave: если команда принята с ошибками, то выдает сообщение об ошибке 0x45 ('E') и завершает обработку текущей команды

10.3.5 Команда CMD_CR

Выдача приглашения Master-у.

Таблица 17 – Команда CMD_CR

Код команды	CMD_CR = 0x0D
ASCII символ, соответствующий коду команды	нет
Количество параметров команды	0
Формат команды:	
Master: Выдает код команды CMD_CR	Slave: если команда принята с ошибками, то выдает сообщение об ошибке 0x45 ('E') и завершает обработку текущей команды. Выдает код команды CMD_CR. Выдает код 0x0A. Выдает код 0x3E (ASCII символ '>')

10.3.6 Команда CMD_BAUD

Установка скорости обмена.

Таблица 18 – Команда CMD_BAUD

Код команды	CMD_BAUD = 0x42
ASCII символ, соответствующий коду команды	'B'
Количество параметров команды	1
Параметр	Новое значение скорости обмена [Бод]
Формат команды:	
Master: Выдает код команды CMD_BAUD	Slave: если команда принята с ошибками, то выдает сообщение об ошибке 0x45 ('E') и завершает обработку текущей команды
Master: Выдает параметр	Slave: если параметр принят с ошибками, то выдает сообщение об ошибке 0x45 ('E') и завершает обработку текущей команды. Выдает код команды CMD_BAUD. Устанавливает новое значение скорости обмена во время отправки кода команды CMD_BAUD

10.3.7 Команда CMD_LOAD

Загрузка массива байт в память микросхемы.

Таблица 19 – Команда CMD_LOAD

Код команды	CMD_LOAD = 0x4C
ASCII символ, соответствующий коду команды	'L'
Количество параметров команды	2
Параметр 1.	Адрес памяти приемника данных
Параметр 2.	Размер массива в байтах
Формат команды:	
Master: Выдает код команды CMD_LOAD	Slave: если команда принята с ошибками, то выдает сообщение об ошибке 0x45 ('E') и завершает обработку текущей команды
Master: Выдает параметр 1	Slave: если хотя бы один из параметров принят с ошибками, то выдает сообщение об ошибке 0x45 ('E') и завершает обработку текущей команды
Master: Выдает параметр 2	Slave: если хотя бы один из параметров принят с ошибками, то выдает сообщение об ошибке 0x45 ('E') и завершает обработку текущей команды. Выдает код команды CMD_LOAD
Master: Выдает массив байт младшим байтом вперед	Slave: принимает массив байт. Если хотя бы один байт принят с ошибками, то выдает сообщение об ошибке и завершает обработку текущей команды, не дожидаясь окончания принятия всего массива. По окончании принятия массива выдает код ответа REPLY_OK = 0x4B ('K')

10.3.8 Команда CMD_VFY

Выдача массива байт из памяти микросхемы.

Таблица 20 – Команда CMD_VFY

Код команды	CMD_VFY = 0x59
ASCII символ, соответствующий коду команды	'Y'
Количество параметров команды	2
Параметр 1	Адрес памяти источника данных
Параметр 2	Размер массива в байтах
Формат команды:	
Master: Выдает код команды CMD_VFY	Slave: если команда принята с ошибками, выдает сообщение об ошибке 0x45 ('E') и завершает обработку текущей команды
Master: Выдает параметр 1	Slave: если хотя бы один из параметров принят с ошибками, выдает сообщение об ошибке 0x45 ('E') и завершает обработку текущей команды
Master: Выдает параметр 2	Slave: если хотя бы один из параметров принят с ошибками, то выдает сообщение об ошибке 0x45 ('E') и завершает обработку текущей команды. Выдает код команды CMD_VFY. Выдает массив байт младшим байтом вперед. По окончании передачи массива выдает код ответа REPLY_OK = 0x4B ('K')

10.3.9 Команда CMD_RUN

Запуск программы на выполнение.

Таблица 21 – Команда CMD_RUN

Код команды	CMD_RUN = 0x52
ASCII символ, соответствующий коду команды	'R'
Количество параметров команды	1
Параметр	Адрес таблицы векторов загруженной программы
Формат команды:	
Master: Выдает код команды CMD_RUN	Slave: если команда принята с ошибками, то выдает сообщение об ошибке 0x45 ('E') и завершает обработку текущей команды
Master: Выдает параметр	Slave: если параметр принят с ошибками, то выдает сообщение об ошибке и завершает обработку текущей команды. Выдает код команды CMD_RUN. Устанавливает значение MSP и PC согласно таблице векторов (NVIC не перепрограммируется) и, таким образом, Slave завершает свое выполнение

10.3.10 Прием параметров команды

Параметры команд – это четырехбайтные числа.

Параметры передаются младшим байтом вперед.

В качестве значения параметра запрещено использовать число 0xFFFFFFFF.

Если при приеме параметра обнаружена аппаратная ошибка (UART установил в «1» какой-либо из флагов ошибки), то прием параметров не прекращается.

Анализ всех видов ошибок, связанных с передачей параметров, загрузчик производит только после принятия всех параметров команды.

10.3.11 Сообщения об ошибках

Сообщения об ошибках – это символ 0x45 ('E').

После выдачи сообщения об ошибке загрузчик переходит в режим ожидания следующей команды, поэтому Master после получения такого сообщения должен прекратить передачу байтов, относящихся к текущей команде.

После принятия сообщения об ошибке Master должен подавать команду CMD_CR до тех пор, пока не получит корректный ответ, соответствующий этой команде.

10.3.11.1 Ошибка ERR_CHN

Аппаратная ошибка UART.

Код ошибки 0x69 ('i').

Выдается, если UART установил в 'I' один из аппаратных флагов ошибки при приеме очередного байта.

Примечание – Не реализовано в загрузчике.

10.3.11.2 Ошибка ERR_CMD

Принята неизвестная команда.

Код ошибки 0x63 ('c').

Выдается диспетчером команд, если принят неизвестный код команды.

Примечание – Не реализовано в загрузчике.

10.3.11.3 Ошибка ERR_BAUD

Принята неизвестная команда.

Код ошибки 0x62 ('b').

Выдается диспетчером команд, если по принятому от Master-а значению скорости обмена невозможно вычислить корректное значение делителя частоты UART.

Примечание – Не реализовано в загрузчике.

10.4 Тестовый режим работы

Микросхема переводится в тестовый режим:

- при установке высокого уровня на выводе JTAG_EN (только для микросхем K1986BE94GI);
- при определении загрузочной программой режима MODE[2:0] = 111;
- при установке в единицу бит JTAG A и JTAG B в регистре REG_0E блока батарейного домена.

При переходе в тестовый режим работа микросхемы в функциональном режиме останавливается.

В тестовом режиме к выводам JTAG_A подключается тестовый TAP-контроллер. Регистр инструкций (IR) тестового TAP-контроллера имеет длину 8 бит. Загрузка инструкций в регистр IR переводит микросхему в тестовые режимы, указанные в таблице 22.

Таблица 22 – Инструкции тестового TAP-контроллера

IR[7:0]	Название инструкции	Описание
0x00	EXTEST	Переводит микросхему в режим внешнего граничного тестирования. В качестве регистра данных выбирается регистр граничного сканирования
0x01	SAMPLE	Позволяет считать регистр граничного сканирования, который выбирается в качестве регистра данных
0x02	IDCODE	В качестве регистра данных выбирается регистр ID, который имеет значение 0x8D200FDF
0x06	PRELOAD	Позволяет загрузить регистр граничного сканирования, который выбирается в качестве регистра данных
0xFF	BYPASS	В качестве регистра данных выбирается 1-битный регистр. Используется для обеспечения последовательного пути минимальной длины между выводами TDI и TDO

11 Контроллер Flash-памяти программ MDR_EEPROM

Микросхема содержит встроенную Flash-память программ с объемом 128 Кбайт основной памяти программ и 4 Кбайта информационной памяти.

В обычном режиме (бит CON = 0, регистр EEPROM_CMD) доступна основная память программ через системные шины I Code и D code для выборки инструкций и данных кода программы.

В режиме программирования (бит CON = 1, регистр EEPROM_CMD) основная и информационная память доступны как периферийные устройства и могут быть использованы для нужд разработчика приложения. В режиме программирования программный код должен выполняться из области системной шины или ОЗУ. Выполнение программного кода из Flash-памяти программ в режиме программирования невозможно.

11.1 Работа Flash-памяти программ в обычном режиме

Скорость доступа во Flash-память ограничена и составляет порядка 40 нс, в результате выдача новых значений из Flash-памяти может происходить с частотой не более 25 МГц. Для того чтобы процессорное ядро могло получать новые инструкции на больших частотах, в микросхеме реализуется Flash-память с физической организацией 8К на 128 разрядов (четыре сектора 8К на 32 разряда). Данные из различных секторов чередуются в линейном адресном пространстве. Таким образом, за 40 нс из Flash-памяти извлекается 16 байтов, в которых может быть закодировано от четырех до восьми последовательных инструкций процессора. И пока ядро выполняет эти инструкции, из памяти извлекается следующая порция данных. Таким образом, тактовая частота может превышать частоту извлечения данных из памяти в несколько раз при линейном выполнении программы.

При возникновении переходов в выполнении программы, когда из памяти программ не выбраны нужные инструкции, возникает пауза в несколько тактов процессора для того, чтобы данные успели считаться из Flash. Число тактов паузы зависит от тактовой частоты процессора; так при работе с частотой ниже 25 МГц пауза не требуется, поскольку Flash-память успевает выдать новые данные за один такт, при частоте от 25 до 50 МГц требуется один такт паузы, и так далее. Число тактов паузы задается в регистре EEPROM_CMD битами Delay[2:0]. В таблице 23 приведены характеристики необходимой паузы для работы Flash-памяти программ.

Таблица 23 – Дополнительная пауза для работы Flash-памяти

Delay[2:0]	Такты паузы	Тактовая частота, МГц	Примечание
0x00	0	до 25	
0x01	1	до 50	
0x02	2	до 75	
0x03	3	до 100	Работа микросхемы с частотой более 80 МГц не гарантируется
0x04	4	до 125	Установлено по умолчанию после сброса
0x05	5	до 150	
0x06	6	до 175	
0x07	7	до 200	

Число тактов паузы устанавливается до момента повышения тактовой частоты или после снижения тактовой частоты.

11.2 Работа Flash-памяти программ в режиме программирования

В режиме программирования Flash-память программ не может выдавать инструкции и данные процессору, поэтому перевод памяти в режим программирования (установка бита CON = 1) возможен только программой, исполняемой из памяти, установленной на внешней системной шине, или ОЗУ. Перед переводом памяти в режим программирования необходимо в регистр EEPROM_KEY записать комбинацию 0x8AAA5551.

В режиме программирования возможны следующие операции как с основной (бит IFREN = 0, регистр EEPROM_CMD), так и с информационной (бит IFREN = 1) памятью:

- стирание всей памяти;
- стирание страницы памяти размером 4 Кбайт;
- запись 32-битного слова в память;
- чтение 32-битного слова из памяти.

Нельзя повторять циклы стирания – записи и стирания – стирания одной ячейки памяти с периодом менее 4 мс.

Страница 31	0x0801_FFFC ...	0x0801_FFF8 ...	0x0801_FFF4 ...	0x0801_FFF0 ...
256 x 128 4K x 8	0x0801_F00C	0x0801_F008	0x0801_F004	0x0801_F000
	...	...	...	...
Страница 1	0x0800_1FFC ...	0x0800_1FF8 ...	0x0800_1FF4 ...	0x0800_1FF0 ...
256 x 128 4K x 8	0x0800_100C	0x0800_1008	0x0800_1004	0x0800_1000
	...	...	...	...
Страница 0	0x0800_0FFC ...	0x0800_0FF8 ...	0x0800_0FF4 ...	0x0800_0FF0 ...
256 x 128 4K x 8	0x0800_001C 0x0800_000C	0x0800_0018 0x0800_0008	0x0800_0014 0x0800_0004	0x0800_0010 0x0800_0000
	Sector_D	Sector_C	Sector_B	Sector_A
	256 x 32 1K x 8	256 x 32 1K x 8	256 x 32 1K x 8	256 x 32 1K x 8

Основная память (IFREN=0)

Страница 0	0x0800_0FFC ...	0x0800_0FF8 ...	0x0800_0FF4 ...	0x0800_0FF0 ...
256 x 128 4K x 8	0x0800_001C 0x0800_000C	0x0800_0018 0x0800_0008	0x0800_0014 0x0800_0004	0x0800_0010 0x0800_0000
	Sector_D	Sector_C	Sector_B	Sector_A
	256 x 32 1K x 8	256 x 32 1K x 8	256 x 32 1K x 8	256 x 32 1K x 8

Информационная память (IFREN=1)

Рисунок 15 – Структура памяти Flash

11.2.1 Стирание всей памяти

Стирание всей памяти выполняется в четыре этапа:

- 1 этап – стирание Sector_A для всей памяти;
- 2 этап – стирание Sector_B для всей памяти;
- 3 этап – стирание Sector_C для всей памяти;
- 4 этап – стирание Sector_D для всей памяти.

Стирание одного сектора памяти возможно только в режиме программирования. Для стирания одного сектора памяти надо установить необходимое значение в бит IFREN (1 – для основной и информационной памяти и 0 – для основной памяти), и номер сектора EEPROM_ADR[3:2] (00 – Sector_A, 01 – Sector_B, 10 – Sector_C и 11 – Sector_D) затем установить биты XE, MAS1 и ERASE в «1», и спустя время t_{nvs} (не менее 5 мкс) установить бит NVSTR в «1». Полное стирание памяти длится время t_{me} (от 20 до 40 мс). Спустя это время необходимо очистить бит ERASE, и спустя время t_{nvhl} (не менее 100 мкс) очистить биты XE, MAS1 и NVSTR. Последующие операции с памятью можно выполнять спустя время t_{rcv} (не менее 1 мкс). Временная диаграмма стирания памяти представлена далее (см. рисунок 16). При стирании информационной области, автоматически стирается и основная. Для стирания всей памяти необходимо осуществить стирание всех секторов памяти.

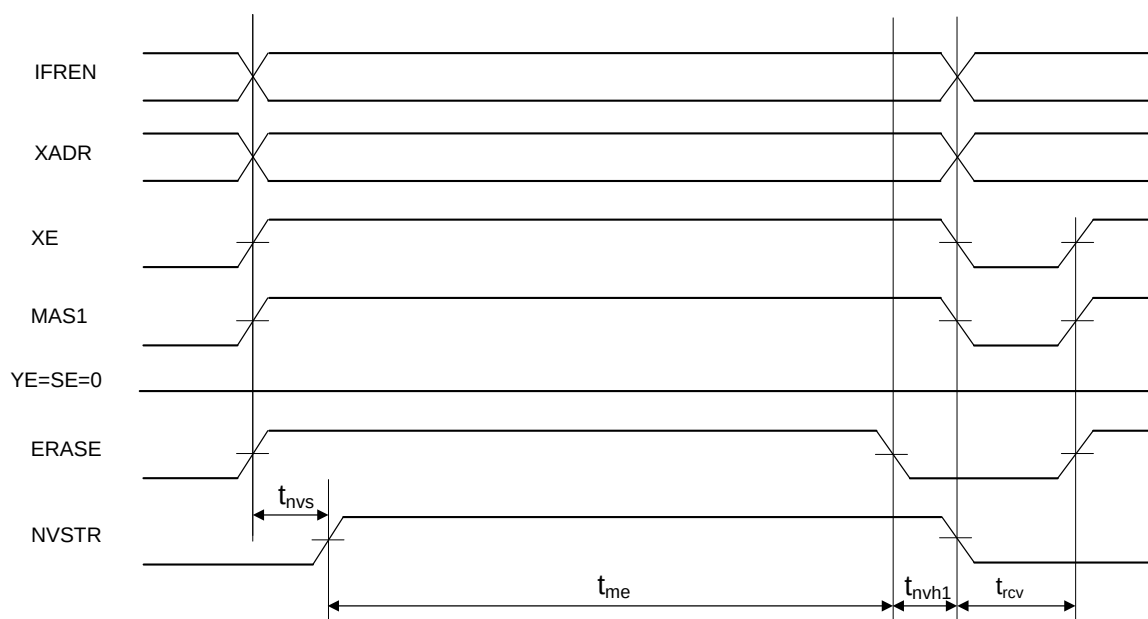


Рисунок 16 – Временная диаграмма стирания памяти

11.2.2 Стирание банка памяти одной страницы размером 4 Кбайт

Стирание одной страницы выполняется в четыре этапа:

- 1 этап – стирание Sector_A для одной страницы;
- 2 этап – стирание Sector_B для одной страницы;
- 3 этап – стирание Sector_C для одной страницы;
- 4 этап – стирание Sector_D для одной страницы.

Стирание одного сектора страницы памяти возможно только в режиме программирования. Для стирания страницы памяти надо установить необходимое

значение в бит IFREN (1 – для информационной памяти и 0 – для основной памяти), затем установить адрес стираемой страницы в регистре EEPROM_ADR[16:12] и номер сектора EEPROM_ADR[3:2] (00 – Sector_A, 01 – Sector_B, 10 – Sector_C и 11 – Sector_D) и установить биты XE и ERASE в «1», и спустя время t_{nvs} (не менее 5 мкс) установить бит NVSTR в «1». Стирание страницы памяти длится время t_{erase} (от 20 до 40 мс). Спустя это время необходимо очистить бит ERASE, и спустя время t_{nvh} (не менее 5 мкс) очистить биты XE и NVSTR. Последующие операции с памятью можно выполнять спустя время t_{rcv} (не менее 1 мкс). Временная диаграмма стирания страницы памяти представлена далее (см. рисунок 17). Для стирания всей страницы необходимо осуществить стирание всех секторов страницы.

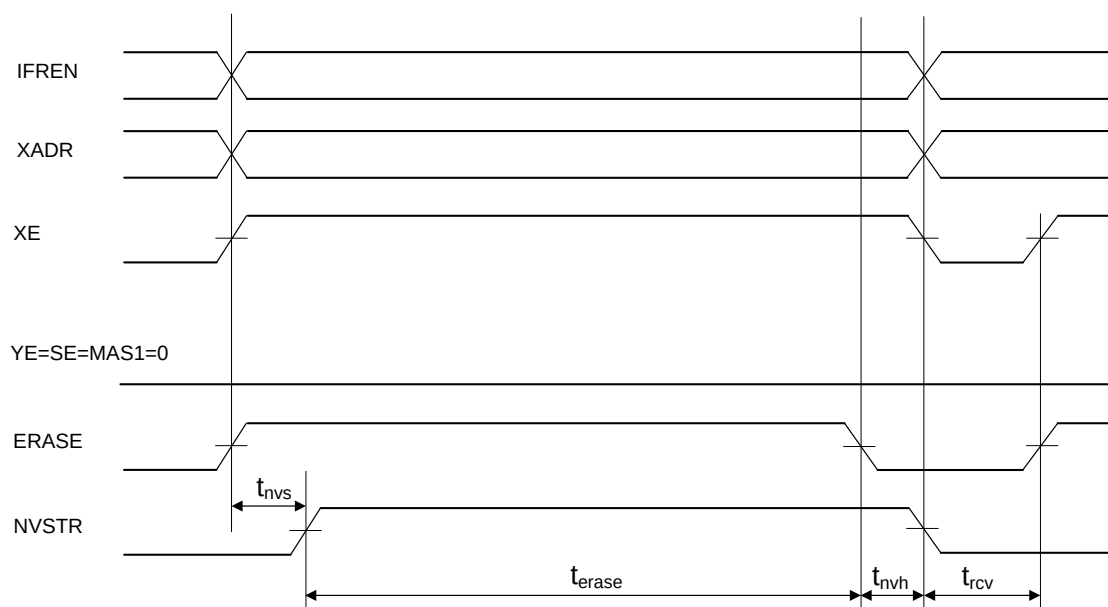


Рисунок 17 – Временная диаграмма стирания банка памяти

11.2.3 Запись 32-битного слова в память

Запись в память возможна только в режиме программирования. Для записи в память надо установить необходимое значение в бит IFREN (1 – для информационной памяти и 0 – для основной памяти), затем установить адрес, по которому производится запись, в регистре EEPROM_ADR, в регистр EEPROM_DI поместить записываемое в память слово и установить биты XE и PROG в «1», и спустя время t_{nvs} (не менее 5 мкс) установить бит NVSTR в «1». Спустя время t_{pgs} (не менее 10 мкс) установить бит YE в «1». Запись в память длится время t_{prog} (от 20 до 40 мкс). Спустя это время необходимо очистить бит YE, и через t_{adh} (не менее 20 нс) установить новый адрес (в пределах того же сектора и той же страницы) и значение для записи в другую ячейку памяти; затем через t_{ads} (не менее 20 нс) установить YE в «1» и записать следующее слово.

Примечания

- 1 Перед программированием ячейки необходимо выполнить ее стирание.
- 2 В одном цикле допускается запись в пределах одного сектора одной страницы.

Если запись больше не требуется, то спустя время t_{pgh} (не менее 20 нс) после очистки бита YE необходимо очистить бит PROG и спустя время t_{nvh} (не менее 5 мкс)

очистить биты XE и NVSTR. Последующие операции с памятью можно выполнять спустя время t_{rcv} (не менее 1 мкс).

Временная диаграмма записи памяти представлена ниже (см. рисунок 18).

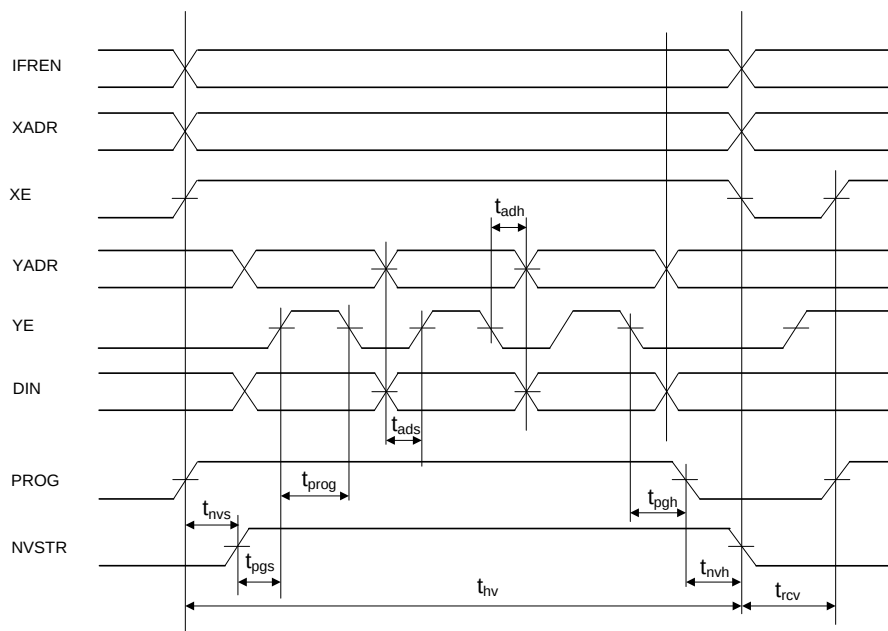


Рисунок 18 – Временная диаграмма записи памяти

11.2.4 Чтение 32-битного слова из памяти

В обычном режиме работы для чтения доступна только основная память, необходимо просто считать требуемый адрес памяти.

В режиме программирования для чтения доступна и основная, и информационная память. Для чтения из памяти надо установить необходимое значение в бит IFREN (1 – для информационной памяти и 0 – для основной памяти), затем установить адрес, из которого необходимо считать данные, в регистре EEPROM_ADR и установить биты XE, YE и SE в «1», и спустя время t_{xa} (не менее 30 нс) из регистра EEPROM_DO можно считать данные. Если необходимо считать следующее слово, то в регистр EEPROM_ADR следует записать новый адрес и, спустя время t_{xa} (не менее 30 нс), из регистра EEPROM_DO можно считать следующие данные. Если чтение больше не требуется, то можно очистить все биты управления. Временная диаграмма чтения памяти в режиме программирования представлена далее (см. рисунок 19).

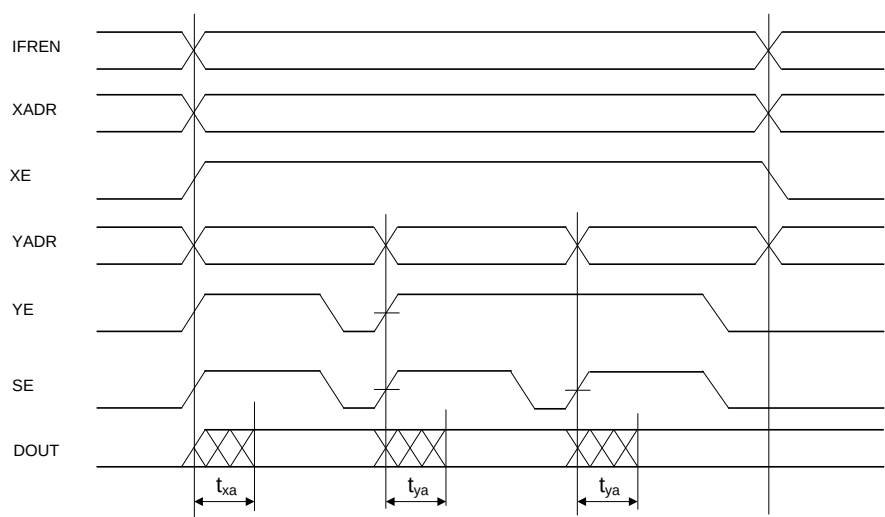


Рисунок 19 – Временная диаграмма чтения памяти

11.3 Производственная информация

В ячейках информационной памяти flash (Sector_A) по адресам, приведенным в таблице 24, записана производственная информация, которая содержит идентификационные номера и коэффициенты подстройки блоков.

Таблица 24 – Описание области производственной информации

Адрес	Название	Описание
0x08000FF0 0x08000FE0 0x08000FD0 0x08000FC0 0x08000FB0	UNIQ_ID[159:0]	Уникальный идентификационный номер
0x08000FA0	LDO_TRIM[2:0]	Коэффициент подстройки опорного напряжения LDO, 1,8 В
0x08000F90	HSI_TRIM[5:0]	Коэффициент подстройки частоты генератора HSI, 8 МГц
0x08000F80	LSI_TRIM[4:0]	Коэффициент подстройки частоты генератора LSI, 40 кГц

Для уникальной идентификации микросхемы необходимо использовать все поля UNIQ_ID.

Для обеспечения корректной работы микросхемы значения коэффициентов подстройки LDO_TRIM[2:0], HSI_TRIM[5:0] и LSI_TRIM[4:0] должны быть считаны из информационной памяти и записаны в регистры REG_0E и REG_0F.

11.4 Регистры управления контроллера Flash-памяти программ

В таблице 25 приведен перечень регистров управления контроллера Flash-памяти программ.

Таблица 25 – Регистры управления контроллера Flash-памяти программ

Базовый адрес	Название	Описание
0x4001_8000	MDR_EEPROM	Регистры контроллера Flash-памяти программ
Смещение		
0x00	CMD	Регистр команды
0x04	ADR	Регистр адреса
0x08	DI	Регистр данных на запись
0x0C	DO	Регистр данных считанных
0x10	KEY	Регистр ключа

Далее каждый из регистров управления контроллера рассмотрен отдельно.

Далее по тексту для описания доступа к битам регистров использованы следующие обозначения:

R/W – бит доступен на чтение и запись;

RO – бит доступен только на чтение;

U – бит физически не реализован или зарезервирован.

11.4.1 MDR_EEPROM->CMD

Таблица 26 – Регистр команды EEPROM_CMD

Номер	31...14	13	12	11	10
Доступ	U	R/W	R/W	R/W	R/W
Сброс	0	0	0	0	0
		NVSTR	PROG	MAS1	ERASE

Номер	9	8	7	6	5...3	2, 1	0
Доступ	R/W	R/W	R/W	R/W	R/W	U	R/W
Сброс	0	0	0	0	100	0	0
	IFREN	SE	YE	XE	Delay[2:0]		CON

Таблица 27 – Описание бит регистра EEPROM_CMD

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...14	-	Зарезервировано
13	NVSTR	Операции записи или стирания: 0 – при чтении; 1 – при записи или стирании
12	PROG	Записать данные по ADR[16:2] из регистра EEPROM_DI: 0 – нет записи; 1 – есть запись

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
11	MAS1	При ERASE =1: 0 – стирается страница с адресом ADR[16:12] в секторе с адресом ADR[3:2]; 1 – стирается весь сектор с адресом ADR[3:2]
10	ERASE	0 – нет стирания; 1 – стирание
9	IFREN	Работа с блоком информации: 0 – основная память; 1 – информационный блок
8	SE	Усилитель считывания: 0 – не включен; 1 – включен
7	YE	Выдача адреса ADR[8:2]: 0 – не разрешено; 1 – разрешено
6	XE	Выдача адреса ADR[16:9]: 0 – не разрешено; 1 – разрешено
5...3	Delay[2:0]	Задержка памяти программ при чтении в циклах (в рабочем режиме): 000 – 0 цикл 001 – 1 цикл 111 – 7 циклов
2, 1	-	Зарезервировано
0	CON	Переключение контроллера памяти EEPROM на регистровое управление, не может производиться при исполнении программы из области EEPROM: 0 – управление EEPROM от ядра, рабочий режим; 1 – управление от регистров, режим программирования

11.4.2 MDR_EEPROM->ADR

Таблица 28 – Регистр адреса EEPROM_ADR

Номер	31...0
Доступ	R/W
Сброс	0
	ADR [31:0]

Таблица 29 – Описание бит регистра адреса EEPROM_ADR

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...0	ADR[31:0]	Адрес обращения в память: ADR[1:0] – не имеет значения, минимально адресуемая ячейка 32 бита

11.4.3 MDR_EEPROM->DI

Таблица 30 – Регистр записываемых данных EEPROM_DI

Номер	31...0
Доступ	R/W
Сброс	0
	DATA [31:0]

Таблица 31 – Описание бит регистра записываемых данных EEPROM_DI

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...0	DATA[31:0]	Данные для записи в EEPROM

11.4.4 MDR_EEPROM->DO

Таблица 32 – Регистр считываемых данных EEPROM_DO

Номер	31...0
Доступ	R/W
Сброс	0
	DATA [31:0]

Таблица 33 – Описание бит регистра считываемых данных EEPROM_DO

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...0	DATA[31:0]	Данные, считанные из EEPROM

11.4.5 MDR_EEPROM->KEY

Таблица 34 – Регистр ключа EEPROM_KEY

Номер	31...0
Доступ	R/W
Сброс	0
	KEY [31:0]

Таблица 35 – Описание бит регистра ключа EEPROM_KEY

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...0	KEY[31:0]	Ключ для разрешения доступа к Flash-памяти через регистровый доступ. Перед переводом памяти в режим программирования необходимо в регистр EEPROM_KEY записать комбинацию 0x8AAA5551

12 Процессорное ядро

Процессорное ядро Cortex-M3 с периферией ядра является высокопроизводительным 32-разрядным процессором, разработанным для микроконтроллерных систем, и обладает следующими преимуществами:

- высокая производительность скомбинирована с быстрой обработкой прерываний;
- расширенная система отладки с точками останова и трассировкой;
- эффективное процессорное ядро для работы системы и памяти;
- сверхнизкое потребление с встроенными режимами Sleep;
- защищенная система с интегрированными блоком защиты памяти MPU.

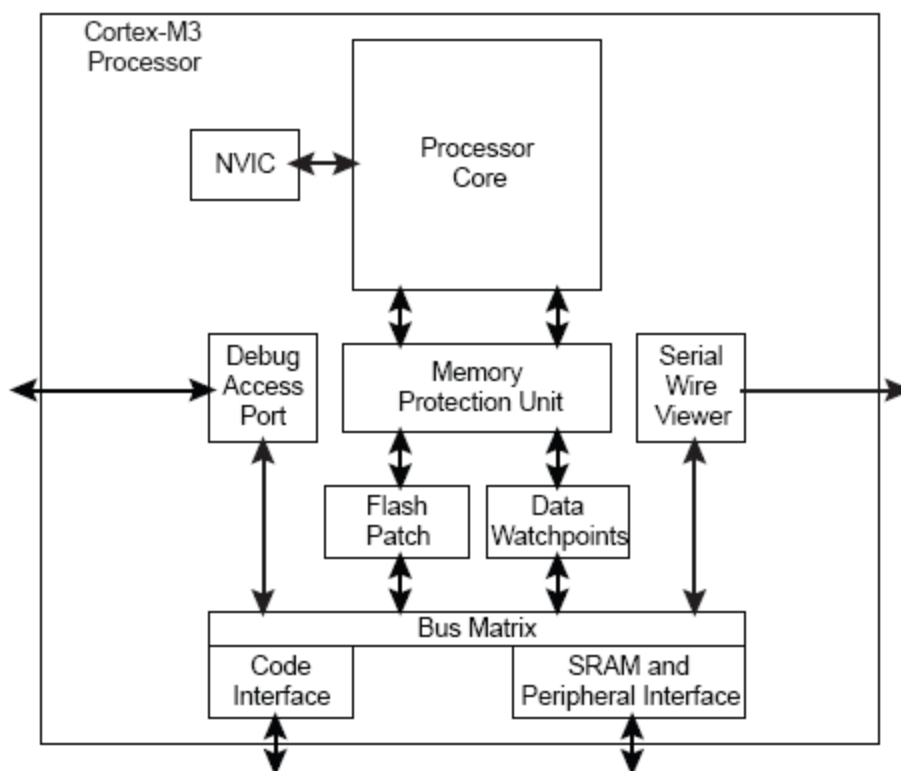


Рисунок 20 – Структурная блок-схема процессорного ядра Cortex-M3

Процессор Cortex-M3 построен на высокопроизводительном ядре с трехстадийным конвейером и Гарвардской архитектурой, что делает его идеальным для микроконтроллерных приложений. Процессор предлагает превосходную энергоэффективность, эффективный набор инструкций, оптимальный дизайн аппаратных средств, включающих одноктактную инструкцию умножения 32x32 и аппаратное деление. Процессор содержит интегрированный контроллер прерываний и встроенные средства отладки. Процессор реализует набор инструкций Thumb2, обеспечивающих высокую плотность кода и более экономное использование памяти программ. Процессор Cortex-M3 обеспечивает производительность 32-битных архитектур с размером кода, сравнимым с 8- и 16-битными микроконтроллерами.

Процессор содержит контроллер прерываний NVIC, обеспечивающий высокоскоростную обработку прерываний. NVIC обеспечивает до восьми уровней приоритетов прерываний. Интеграция контроллера прерываний в ядро позволяет

реализовать быстрое исполнение обработчиков прерываний (interrupt service routines – ISR), эффективно снижающее задержку обработки прерываний. Это обеспечивается аппаратным сохранением в стеке регистров, выполняемым одной инструкцией множественной записи и считывания памяти. Реализация обработчиков прерываний не требует их описания на ассемблере и позволяет удалить из обработчика код по перегрузке контекста. Оптимизация сцепления концов обработчиков позволяет снизить затраты при переключении с одного обработчика на другой.

Оптимизированный для пониженного энергопотребления контроллер NVIC обеспечивает режимы Sleep и Deep Sleep, которые позволяют быстро снизить потребление.

Ядро Cortex-M3 обеспечивает большую скорость и низкую задержку обращения в память. Также поддерживаются невыровненные обращения и битовые манипуляции с областью ОЗУ и регистрами периферии.

Ядро Cortex-M3 содержит блок защиты памяти (MPU) который обеспечивает граничное управление памятью, позволяющий приложениям реализовывать различные уровни привилегий безопасности, разделяя код, данные и стеки для различных задач, что требуется для критичных к сбоям решений.

Ядро Cortex-M3 реализует аппаратную поддержку функций отладки. Отладка позволяет отображать состояние системы и памяти через стандартный JTAG разъем или двухпроводной интерфейс SWD.

Для трассировки в ядре реализован модуль ITM, отслеживающий точки просмотра данных и сообщения профилирования.

Периферийными блоками ядра являются:

- контроллер прерываний NVIC – реализует высокоскоростную обработку прерываний;
- блок системного управления SBC – программный интерфейс процессора, реализует отображение информации о системной реализации, а также управление системой, включая конфигурирование, управление и отображение событий в системе;
- системный таймер SysTick – 24-битный счетчик, считающий вниз, используется операционными системами реального времени для подсчета тактов или как обычный счетчик;
- блок защиты памяти MPU – используется для повышения надежности системы путем задания различных атрибутов для регионов памяти. Поддерживает до восьми различных регионов и один опциональный предопределенный регион.

12.1 Программная модель

Процессор может функционировать в режимах:

- Thread – используется для исполнения приложений, процессор находится в этом режиме сразу после сброса;
- Handler – используется для обработки исключений. После обработки исключения процессор переходит в режим Thread.

Уровни привилегий при исполнении программ:

- Unprivileged – непривилегированное программное обеспечение:
- имеет ограниченный доступ к инструкциям MSR и MRS, и не может использовать инструкцию CPS;
- не имеет доступа к системному таймеру, NVIC и блоку системного управления;
- может иметь пониженный уровень доступа к памяти или периферии.
- Privileged – привилегированное программное обеспечение. Имеет полный доступ ко всем инструкциям и ресурсам.

В режиме Thread регистр CONTROL определяет уровень исполнения программы unprivileged или privileged. Подробнее см. в описании регистра CONTROL. В режиме Handler программное обеспечение всегда выполняется на уровне privileged.

Только привилегированное программное обеспечение может писать в регистр CONTROL для изменения уровня исполнения программы в режиме Thread. Непривилегированное программное обеспечение может использовать инструкцию SVC для выполнения supervisor call для передачи управления привилегированной программе.

12.2 Стек

Процессор использует нисходящий стек. Это означает, что указатель стека обозначает последний сохраненный в стеке элемент в стековой области памяти. Когда процессор записывает новый элемент в стек, сначала декрементируется указатель и затем записывается новый элемент в память. Процессор реализует два стека – main и process с независимыми указателями стеков, подробнее смотрите указатели стека.

В Thread режиме регистр CONTROL определяет, какой стек используется – main или process, подробнее в описании CONTROL регистра. В Handler режиме процессор всегда использует main стек.

Таблица 36 – Режимы работы процессора при выполнении программы

Режим процессора	Использование	Уровни привилегии для программного обеспечения	Используемый стек
<i>Thread</i>	Выполнение приложений	<i>Privileged</i> или <i>Unprivileged</i> ⁽¹⁾	<i>Main</i> или <i>Process</i> *
<i>Handler</i>	Обработка исключений	Всегда <i>Privileged</i>	<i>Main</i> стек

* Подробнее см. описание в пункт 12.3.13 «Регистр управления CONTROL».

12.3 Регистры ядра

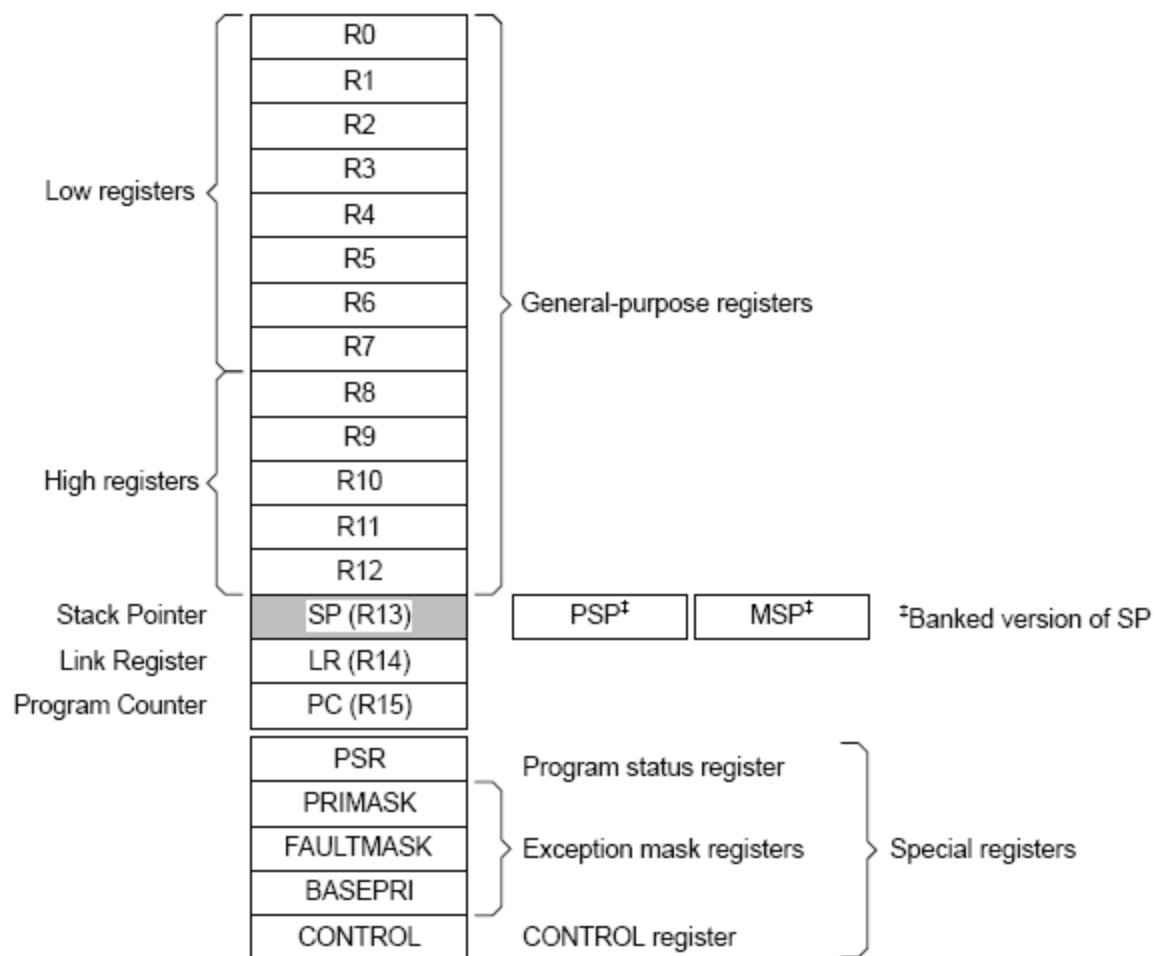


Рисунок 21 – Регистры ядра

Таблица 37 – Сводная таблица регистров ядра

Название	Тип ¹⁾	Требуемый уровень привилегий	Значение после сброса	Описание
R0-R12	RW	Оба ²⁾	Неизвестно	Регистры общего назначения
MSP	RW	Privileged	См. описание	Указатель стека <i>main</i> Stack Pointer
PSP	RW	Оба ²⁾	Неизвестно	Указатель стека <i>process</i> Stack Pointer
LR	RW	Оба ²⁾	0xFFFFFFFF	Регистр связи Link Register
PC	RW	Оба ²⁾	См. описание	Счетчик команд Program Counter
PSR	RW	Privileged	0x01000000	Программный регистр состояния Program Status Register
ASPR	RW	Оба ²⁾	0x00000000	Программный регистр состояния приложения Application Program Status Register
IPSR	RO	Privileged	0x00000000	Программный регистр состояния прерываний Interrupt Program Status Register
ESPR	RO	Privileged	0x01000000	Программный регистр состояния выполнения Execution Program Status Register
PRIMASK	RW	Privileged	0x00000000	Регистр маски приоритетов Priority Mask Register
FAULTMASK	RW	Privileged	0x00000000	Регистр маски сбоев Fault Mask Register
BASEPRI	RW	Privileged	0x00000000	Регистр базового приоритета маски Base Priority Mask Register
CONTROL	RW	Privileged	0x00000000	Регистр Управления CONTROL Register
¹⁾ Определяет режим доступа при исполнении программы в режимах <i>thread</i> и <i>handler</i> . В режиме отладки может отличаться. ²⁾ Регистр доступен при исполнении программы с обоими уровнями привилегий				

12.3.1 Регистры общего назначения R0-R12

R0-R12 – это 32-разрядные регистры для данных при выполнении операций.

12.3.2 Указатель стека SP R13

Stack Pointer Register (SP) – это регистр R13. В Thread режиме бит 1 регистра CONTROL обозначает, какой указатель стека используется:

- 0 – Main Stack Pointer (MSP). Сразу после сброса;
- 1 – Process Stack Pointer (PSP).

При сбросе в MSP устанавливается 0x00000000.

12.3.3 Регистр связи LR R14

Link Register – это регистр R14. Регистр используется для сохранения информации об адресе возврата при уходе на обработку прерываний, вызовах функций и обработке исключений. При сбросе устанавливается в 0xFFFFFFFF.

12.3.4 Счетчик команд PC R15

Program Counter – это регистр R15. Он содержит адрес текущей инструкции. Бит 0 всегда 0, так как все инструкции выровнены на полуслово. При сбросе процессор считывает в этот регистр вектор сброса, который расположен по адресу 0x00000004.

12.3.5 Программный регистр состояния PSR

Регистр Program Status Register (PSR) объединяет регистры:

- Application Program Status Register (APSR);
- Interrupt Program Status Register (IPSR);
- Execution Program Status Register (EPSR).

Эти регистры разделяют различные битовые поля в 32-разрядном PSR. Описание регистров приведено ниже. Доступ к этим регистрам может быть как индивидуальный, так и комбинированный к двум или всем трем разом, с использованием имен регистров в качестве аргументов инструкций MSR или MRS.

Например:

- читать все регистры, используя PSR с инструкцией MRS;
- записать только в APSR, используя APSR с инструкцией MSR.

Таблица 38 – Комбинация PSR и их атрибуты

Регистр	Тип	Комбинация
PSR	RW ^{1, 2)}	APSR, EPSR и IPSR
IEPSR	RO	EPSR и IPSR
IAPSR	RW ¹⁾	APSR и IPSR
EAPSR	RW ²⁾	APSR и EPSR
¹⁾ Игнорируется запись в IPSR биты. ²⁾ При чтении EPSR бит читаются нули, и запись в них игнорируется		

Подробнее описание инструкций см. в пунктах 13.9.5 «MRS» и 13.9.6 «MSR».

12.3.6 Программный регистр состояния приложения APSR

Регистр APSR содержит текущие флаги состояния выполнения предыдущей инструкции.

Таблица 39 – Регистр APSR

Номер	31	30	29	28	27	26...0
Доступ	R/W	R/W	R/W	R/W	R/W	
Сброс	0	0	0	0	0	
	N	Z	C	V	Q	-

Таблица 40 – Описание бит регистра APSR

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31	N	Negative: 0 – результат операции положительный или нулевой, либо «больше чем или равно»; 1 – результат операции отрицательный, либо «меньше чем».
30	Z	Zero: 0 – результат операции не нулевой; 1 – результат операции нулевой.
29	C	Carry: 0 – при суммировании не было переноса, либо при вычитании не было заема; 1 – при суммировании был перенос, либо при вычитании был заем.
28	V	Overflow: 0 – в результате операции не было переполнения; 1 – в результате операции было переполнение.
27	Q	Saturation: 0 – обозначает, что не было накопления с момента сброса либо с момента установки бита в ноль; 1 – обозначает, что в результате выполнения инструкций SSAT и USAT было накопление. Флаг сбрасывается в ноль программно инструкцией MRS.
26...0	-	Зарезервировано

12.3.7 Программный регистр состояния прерываний IPSR

Регистр IPSR содержит номер типа исключения для текущего обработчика прерывания.

Таблица 41 – Регистр IPSR

Номер	31...9	8...0
Доступ	-	RO
Сброс	-	0
	-	ISR_NUMBER

Таблица 42 – Описание бит регистра IPSR

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...9	-	Зарезервировано
8...0	ISR_NUMBER	Номер текущего исключения: 0 – режим Thread; 1 – зарезервировано; 2 – NMI; 3 – Hard Fault; 4 – Memory Management Fault; 5 – Bus Fault; 6 – Usage Fault; 7...10 – зарезервировано; 11 – SVCcall;

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
		12 – зарезервировано для отладки; 13 – PendSV; 15 – SysTick; 16 – IRQ0; ... 48 – IRQ31. Подробнее см. раздел 31 «Прерывания и исключения»

12.3.8 Программный регистр состояния выполнения EPSR

Регистр EPSR содержит бит состояния Thumb инструкции, и биты состояния выполнения для инструкций:

- If-Then (IT) блок инструкций;
- Interruptible-Continuable Instruction (ICI) поле для прерываемых инструкций

множественного сохранения и считывания

Таблица 43 – Регистр EPSR

Номер	31...27	26, 25	24	23...16	15...10	9...0
Доступ		RO	RO		RO	
Сброс		0	1		0	
	-	ICI/IT	T	-	ICI/IT	-

Таблица 44 – Описание бит регистра EPSR

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...27	-	Зарезервировано
26, 25	ICI/IT	ICI: Биты Interruptible-Continuable Instruction IT: Обозначает выполнение инструкции IT
24	T	Всегда 1
23...16	-	Зарезервировано
15...10	ICI/IT	ICI: Биты Interruptible-Continuable Instruction IT: Обозначает выполнение инструкции IT
9...0	-	Зарезервировано

Попытка читать EPSR напрямую приложением, используя MSR инструкцию, всегда возвращает ноль. Попытка записать EPSR, используя MSR, напрямую приложением игнорируется. Обработчик сбойной ситуации может считать значение EPSR, сохранив его в стеке для отображения операции вызвавшей сбой. Подробнее раздел вход и выход в исключения.

12.3.8.1 Interruptible-Continuable Instruction

Когда происходит прерывание при выполнении инструкций LDM или STM, то процессор:

- временно останавливает операцию множественного чтения или записи;
- сохраняет номер следующий регистр операнда в множественной операции в битах EPSR[15:12].

После обработки прерывания процессор:

- возвращает номер регистра для сохранения из бит EPSR[15:12];
- возобновляет выполнение операции множественного чтения или записи.

Когда EPSR содержит состояние выполнения ICI инструкции, то биты [26:25] и [11:10] содержат нули.

12.3.8.2 If-Then блок инструкций

Блок If-Then содержит до четырех инструкций, следующих за 16-битной инструкцией IT. Каждая инструкция в этом блоке условная. Условие для инструкций может быть одним либо обратным для некоторых из них. Подробнее см. пункт 13.8.3 «IT».

12.3.9 Регистр маски исключений Exception mask

Регистр маски исключений запрещает обработку исключений процессором. Запрещение исключений может потребоваться в критичных по времени задачах.

Для доступа к регистру маски исключений используются инструкции MSR и MRS, или инструкция CPS для изменения значения PRIMASK и FAULTMASK. Подробнее см. в пунктах 13.9.1 «CPS», 13.9.5 «MRS» и 13.9.6 «MSR».

12.3.10 Регистр маски приоритетов Priority Mask

Регистр PRIMASK предотвращает активацию всех исключений с конфигурируемым приоритетом.

Таблица 45 – Регистр PRIMASK

Номер	31...1	0
Доступ	U	R/W
Сброс	0	0
	-	PRIMASK

Таблица 46 – Описание бит регистра PRIMASK

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...1	-	Зарезервировано
0	PRIMASK	0 – не влияет. 1 – предотвращает активацию всех исключений с конфигурируемым приоритетом

12.3.11 Регистр маски сбоев Fault Mask

Регистр FAULTMASK предотвращает активацию всех исключений.

Таблица 47 – Регистр FAULTMASK

Номер	31...1	0
Доступ	U	R/W
Сброс	0	0
	-	FAULTMASK

Таблица 48 – Описание бит регистра FAULTMASK

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...1	-	Зарезервировано
0	FAULTMASK	0 – не влияет. 1 – предотвращает активацию всех исключений

Процессор устанавливает бит FAULTMASK в «0» при выходе из всех обработчиков за исключением NMI обработчика.

12.3.12 Регистр базового приоритета маски Base Priority Mask

Регистр BASEPRI задает минимальный приоритет процессу обработки исключений. Когда BASEPRI установлен в ненулевое значение, это приводит к предотвращению активации всех исключений с таким же или более низким уровнем приоритета как значение в BASEPRI. Подробнее смотрите сводную таблицу регистров ядра для данных атрибутов. Значение бит представлено ниже.

Таблица 49 – Регистр BASEPRI

Номер	31...8	7...0
Доступ	U	R/W
Сброс	0	0
	-	BASEPRI

Таблица 50 – Описание бит регистра BASEPRI

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...8	-	Зарезервировано
7...0	BASEPRI	0 – не влияет. Ненулевое – задает базовый приоритет для процесса обработки исключений

Процессор не обрабатывает какие-либо исключения со значением приоритета, большим или равным BASEPRI.

Это поле подобно полю приоритета в регистре приоритетов прерываний. Процессор анализирует только биты [7:5] этого поля, биты [4:0] читаются как нули и игнорируются при записи. Для более полной информации см. таблицу 28. Помните, что большее значение в поле приоритета соответствует меньшему приоритету обработчика.

12.3.13 Регистр управления CONTROL

Регистр CONTROL задает текущий стек и уровень привилегий выполняемой процессором программы в Thread режиме. Смотрите описание регистров процессорного ядра для атрибутов. Описание бит приведено ниже.

Таблица 51 – Регистр CONTROL

Номер	31...2	1	0
Доступ	U	R/W	R/W
Сброс	0	0	0
	-	Active Stack Pointer	Thread Mode Privilege Level

Таблица 52 – Описание бит регистра CONTROL

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...2	-	Зарезервировано
1	Active Stack Pointer	0 – MSP текущий указатель стека 1 – PSP текущий указатель стека
0	Thread Mode Privilege Level	0 – привилегирован 1 – непривилегирован

Handler режим всегда использует указатель стека MSP, таким образом процессор игнорирует прямые записи в бит Active Stack Pointer регистра CONTROL в handler-режиме. Вход и выход в обработчик исключений обновляют регистр CONTROL.

При работе с операционными средами рекомендуется, чтобы потоки, запущенные в Thread-режиме, использовали стек PSP, а ядро и обработчики исключений использовали стек MSP.

По определению, режим Thread использует MSP. Для переключения указателя стека в Thread режиме на PSP используется MSR инструкция для установки бита Active Stack Pointer в «1», смотрите описание инструкции MSR

Когда изменяется указатель стека, программное обеспечение должно использовать ISB инструкцию немедленно за MSR инструкцией. Это позволяет быть уверенным, что инструкции, следующие за выполнением ISB, будут использовать новый указатель стека. Смотрите описание инструкции ISB.

12.4 Исключения и прерывания

Процессорное ядро ARM Cortex-M3 поддерживает прерывания и системные исключения. Процессор и контроллер прерываний NVIC устанавливают приоритеты и обрабатывают все исключения. Исключение изменяет нормальный поток выполнения программы. Процессор использует handler режим для обработки всех исключений кроме сброса. Смотрите вход в исключение и выход из исключения для большей информации.

Регистры NVIC управляют обработкой прерываний. Подробнее см. раздел 32 «Контроллер прерываний NVIC».

Типы данных

Процессор поддерживает следующие типы данных:

- 32-бита words;
- 16-бит halfwords;
- 8-бит bytes.

Процессор поддерживает 64-битную инструкцию передачи.

Процессор манипулирует всеми данными в little-endian режиме. Доступ в память инструкций и Private Peripheral Bus (PPB) всегда в little-endian режиме. Подробнее см. подраздел 9.10 «Регионы памяти, типы и атрибуты».

13 Система команд

В процессоре реализована версия системы команд Thumb. Поддерживаемые команды приведены в таблице 53.

В таблице используются следующие обозначения:

- в угловых скобках < > записываются альтернативные формы представления операндов;
- в фигурных скобках {} указываются необязательные операнды;
- информация в столбце «операнды» может быть неполной;
- второй операнд Op2 может быть либо регистром, либо константой;
- большинство команд могут содержать суффикс кода условного выполнения.

Более подробная информация приведена в описаниях команд.

Таблица 53 – Система команд процессора Cortex-M3

Мнемокод команды	Операнды	Краткое описание	Флаги	Прим.
ADC, ADCS	{Rd,} Rn, Op2	Сложение с переносом	N,Z,C,V	
ADD, ADDS	{Rd,} Rn, Op2	Сложение	N,Z,C,V	
ADD, ADDW	{Rd,} Rn, #imm12	Сложение	N,Z,C,V	
ADR	Rd, label	Загрузка адреса, заданного относительно счетчика команд	-	
AND, ANDS	{Rd,} Rn, Op2	Логическое И	N,Z,C	
ASR, ASRS	Rd, Rm, <Rs#n>	Арифметический сдвиг вправо	N,Z,C	
B	label	Переход	-	
BFC	Rd, #lsb, #width	Сброс элемента битового поля	-	
BFI	Rd,Rn,#lsb,#width	Запись заданного значения битового поля	-	
BIC, BICS	{Rd,} Rn, Op 2	Сброс бит по маске	N,Z,C	
BKPT	#imm	Точка останова	-	
BL	label	Переход со связью	-	
BLX	Rm	Косвенный переход со связью	-	
BX	Rm	Косвенный переход	-	
CBNZ	Rn, label	Сравнение с нулем и переход по неравенству	-	
CBZ	Rn, label	Сравнение с нулем и переход по равенству	-	
CLREX	-	Сброс эксклюзивного доступа	-	
CLZ	Rd, Rm	Определить количество ведущих нулей	-	
CMN, CMNS	Rn, Op2	Сравнить с противоположным знаком	N,Z,C,V	
CMP, CMPS	Rn, Op2	Сравнить	N,Z,C,V	
CPSID	iflags	Изменить состояние процессора, запретить прерывания	-	
CPSIE	iflags	Изменить состояние процессора, разрешить прерывания	-	
DMB	-	Барьер синхронизации доступа к памяти данных	-	

Мнемокод команды	Операнды	Краткое описание	Флаги	Прим.
DSB	-	Барьер синхронизации доступа к памяти данных	-	
EOR, EORS	{Rd,} Rn, Op2	Исключающее ИЛИ	N,Z,C	
ISB	-	Барьер синхронизации доступа к инструкциям	-	
IT	-	Начало блока условно исполняемых инструкций	-	
LDM	Rn{!}, reglist	Загрузка множества регистров, инкремент после доступа	-	
LDMDB, LDMEA	Rn{!}, reglist	Загрузка множества регистров, декремент перед доступом	-	
LDMFD, LDMIA	Rn{!}, reglist	Загрузка множества регистров, инкремент после доступа	-	
LDR	Rt, [Rn, #offset]	Загрузка слова в регистр	-	
LDRB, LDRBT	Rt, [Rn, #offset]	Загрузка байта в регистр	-	
LDRD	Rt, Rt2, [Rn, #offset]	Загрузка двойного слова в пару регистров	-	
LDREX	Rt, [Rn, #offset]	Эксклюзивное чтение регистра	-	
LDREXB	Rt, [Rn]	Эксклюзивное чтение регистра, байт	-	
LDREXH	Rt, [Rn]	Эксклюзивное чтение регистра, полуслово	-	
LDRH, LDRHT	Rt, [Rn, #offset]	Загрузка полуслова в регистр	-	
LDRSB, LDRSBT	Rt, [Rn, #offset]	Загрузка в регистр байта со знаком	-	
LDRSH, LDRSHT	Rt, [Rn, #offset]	Загрузка в регистр полуслова со знаком	-	
LDRT	Rt, [Rn, #offset]	Загрузка в регистр слова	-	
LSL, LSLs	Rd, Rm, <Rs#n>	Логический сдвиг влево	N,Z,C	
LSR, LSRS	Rd, Rm, <Rs#n>	Логический сдвиг вправо	N,Z,C	
MLA	Rd, Rn, Rm, Ra	Умножение и сложение, 32-битный результат	-	
MLS	Rd, Rn, Rm, Ra	Умножение и вычитание, 32-битный результат	-	
MOV, MOVs	Rd, Op2	Загрузка	N,Z,C	
MOVT	Rd, #imm16	Загрузка в старшее полуслово	-	
MOVW, MOV	Rd, #imm16	Загрузка 16-битной константы	N,Z,C	
MRS	Rd, spec_reg	Считать специальный регистр в регистр общего назначения	-	
MSR	spec_reg, Rm	Записать регистр общего назначения в специальный регистр	N,Z,C,V	
MUL, MULs	{Rd,} Rn, Rm	Умножение, 32-разрядный результат	N,Z	
MVN, MVNs	Rd, Op2	Загрузка инверсного значения	N,Z,C	
NOP	-	Нет операции	-	
ORN, ORNs	{Rd,} Rn, Op2	Логическое ИЛИ-НЕ	N,Z,C	
ORR, ORRs	{Rd,} Rn, Op2	Логическое ИЛИ	N,Z,C	
POP	reglist	Извлечь регистры из стека	-	

Мнемокод команды	Операнды	Краткое описание	Флаги	Прим.
PUSH	reglist	Занести регистры в стек	-	
RBIT	Rd, Rn	Изменить на обратный порядок бит в слове	-	
REV	Rd, Rn	Изменить на обратный порядок байтов в слове	-	
REV16	Rd, Rn	Изменить на обратный порядок байтов в полусловах	-	
REVSH	Rd, Rn	Изменить на обратный порядок байт в младшем полуслове, произвести распространение знакового бита в старшее полуслово	-	
ROR, RORS	Rd, Rm, <Rs n>	Циклический сдвиг вправо	N,Z,C	
RRX, RRXS	Rd, Rm	циклический сдвиг вправо на один бит с учетом переноса	N,Z,C	
RSB, RSBS	{Rd,} Rn, Op2	Вычитание с противоположным порядком аргументов	N,Z,C,V	
SBC, SBCS	{Rd,} Rn, Op2	Вычитание с учетом переноса	N,Z,C,V	
SBFX	Rd, Rn, #lsb, #width	Чтение значения битового поля, интерпретируемого как число со знаком	-	
SDIV	{Rd,} Rn, Rm	Деление чисел со знаком	-	
SEV	-	Установить признак события	-	
SMLAL	RdLo, RdHi, Rn, Rm	Умножение чисел со знаком с накоплением, 64-битный результат	-	
SMULL	RdLo, RdHi, Rn, Rm	Умножение чисел со знаком, 64-битный результат	-	
SSAT	Rd,#n,Rm{,shift#s}	Преобразование 32-разрядного числа в n-разрядное со знаком, с насыщением	Q	
STM	Rn{!}, reglist	Сохранение множества регистров, инкремент после доступа	-	
STMDB, STMEA	Rn{!}, reglist	Сохранение множества регистров, декремент перед доступом	-	
STMFD, STMIA	Rn{!}, reglist	Сохранение множества регистров, инкремент после доступа	-	
STR	Rt, [Rn, #offset]	Сохранение регистра	-	
STRB, STRBT	Rt, [Rn, #offset]	Сохранение регистра, байт	-	
STRD	Rt, Rt2, [Rn, #offset]	Сохранение пары регистров, двойное слово	-	
STREX	Rd, Rt, [Rn, #offset]	Эксклюзивная запись регистра	-	
STREXB	Rd, Rt, [Rn]	Эксклюзивная запись регистра, байт	-	
STREXH	Rd, Rt, [Rn]	Эксклюзивная запись регистра, полуслово	-	
STRH, STRHT	Rt, [Rn, #offset]	Сохранение регистра, полуслово	-	
STRT	Rt, [Rn, #offset]	Сохранение регистра, слово	-	
SUB, SUBS	{Rd,} Rn, Op2	Вычитание	N,Z,C,V	
SUB, SUBW	{Rd,} Rn, #imm12	Вычитание	N,Z,C,V	
SVC	#imm	Вызов супервизора	-	

Мнемокод команды	Операнды	Краткое описание	Флаги	Прим.
SXTB	{Rd,}Rm{,ROR#n}	Преобразовать байт со знаком в слово	-	
SXTH	{Rd,}Rm{,ROR#n}	Преобразовать полуслово со знаком в слово	-	
TBB	[Rn, Rm]	Табличный переход по индексу, смещения – байты	-	
TBH	[Rn, Rm, LSL #1]	Табличный переход по индексу, смещения – полуслова	-	
TEQ	Rn, Op2	Проверка равенства	N,Z,C	
TST	Rn, Op2	Проверка значения бит по маске	N,Z,C	
UBFX	Rd, Rn, #lsb, #width	Чтение значения битового поля, интерпретируемого как число без знака	-	
UDIV	{Rd,} Rn, Rm	Деление чисел без знака	-	
UMLAL	RdLo, RdHi, Rn, Rm	Умножение чисел без знака с накоплением, 64-битный результат	-	
UMULL	RdLo, RdHi, Rn, Rm	Умножение чисел без знака, 64-битный результат	-	
USAT	Rd,#n,Rm{,shift#s}	Преобразование 32-разрядного числа в n-разрядное без знака, с насыщением	Q	
UXTB	{Rd,}Rm{,ROR#n}	Преобразовать байт без знака в слово	-	
UXTH	{Rd,}Rm{,ROR#n}	Преобразовать полуслово без знака в слово	-	
WFE	-	Ожидать событие	-	
WFI	-	Ожидать прерывание	-	

13.1 Встроенные функции

Стандарт ANSI языка C не обеспечивает непосредственного доступа к некоторым инструкциям процессора Cortex-M3. В данном разделе описаны встроенные (intrinsic) функции, которые указывают компилятору на необходимость генерации соответствующих инструкций. В случае если используемый компилятор не поддерживает ту или иную встроенную функцию, рекомендуется включить в текст программы ассемблерную вставку с необходимой инструкцией.

В CMSIS предусмотрены следующие встроенные функции, расширяющие возможности стандарта ANSI C.

Таблица 54 – Встроенные функции CMSIS, позволяющие генерировать некоторые инструкции процессора Cortex-M3

Мнемокод команды процессора	Описание встроенной функции
CPSIE I	void __enable_irq(void)
CPSID I	void __disable_irq(void)
CPSIE F	void __enable_fault_irq(void)
CPSID F	void __disable_fault_irq(void)
ISB	void __ISB(void)

Мнемокод команды процессора	Описание встроенной функции
DSB	void __DSB(void)
DMB	void __DMB(void)
REV	uint32_t __REV(uint32_t int value)
REV16	uint32_t __REV16(uint32_t int value)
REVSH	uint32_t __REVSH(uint32_t int value)
RBIT	uint32_t __RBIT(uint32_t int value)
SEV	void __SEV(void)
WFE	void __WFE(void)
WFI	void __WFI(void)

Кроме того, CMSIS также обеспечивает возможность чтения и записи специальных регистров процессора, доступных с помощью команд MRS и MSR.

Таблица 55 – Встроенные функции CMSIS для доступа к специальным регистрам процессора

Наименование специального регистра	Режим доступа	Описание встроенной функции
PRIMASK	Чтение	uint32_t __get_PRIMASK (void)
	Запись	void __set_PRIMASK (uint32_t value)
FAULTMASK	Чтение	uint32_t __get_FAULTMASK (void)
	Запись	void __set_FAULTMASK (uint32_t value)
BASEPRI	Чтение	uint32_t __get_BASEPRI (void)
	Запись	void __set_BASEPRI (uint32_t value)
CONTROL	Чтение	uint32_t __get_CONTROL (void)
	Запись	void __set_CONTROL (uint32_t value)
MSP	Чтение	uint32_t __get_MSP (void)
	Запись	void __set_MSP (uint32_t TopOfMainStack)
PSP	Чтение	uint32_t __get_PSP (void)
	Запись	void __set_PSP (uint32_t TopOfProcStack)

13.2 Описание инструкций

В разделе представлена подробная информация об инструкциях процессора:

- операнды;
- ограничения на использование счетчика команд PC и указателя стека SP;
- формат второго операнда;
- операции сдвига;
- выравнивание адресов;
- выражения с участием счетчика команд;
- условное исполнение;
- выбор размера кода инструкции.

13.2.1 Операнды

В качестве операнда инструкции может выступать регистр, константа, либо другой параметр, специфичный для конкретной команды. Процессор применяет инструкцию к операндам и, как правило, сохраняет результат в регистре-получателе. В

случае если формат команды предусматривает спецификацию регистра-получателя, он, как правило, указывается непосредственно перед операндами.

Операнды в некоторых инструкциях допускают гибкий формат представления, то есть могут быть как регистром, так и константой. Подробнее см. пункт 13.2.3 «Формат второго операнда».

13.2.2 Ограничения на использование PC и SP

Многие инструкции не позволяют использовать регистры счетчика команд (PC) и указателя стека (SP) в качестве регистра-получателя. Подробная информация содержится в описании конкретных инструкций.

Бит [0] адреса, загружаемого в PC с помощью одной из команд BX, BLX, LDM, LDR или POP должен быть равен 1, так как этот бит указывает на требуемый набор команд, а процессор Cortex-M3 поддерживает только инструкции из набора Thumb.

13.2.3 Формат второго операнда

Большинство команд обработки данных поддерживает гибкий формат задания второго операнда. Далее в описании синтаксиса инструкций процессора такой операнд будет обозначаться как Operand2. При этом в качестве операнда может выступать:

- константа;
- регистр с необязательным параметром сдвига.

13.2.3.1 Константа

Данный тип второго операнда задается в формате
#constant

где constant может быть:

- любой константой, которая может быть получена путем сдвига восьмиразрядного числа влево на любое количество разрядов в пределах 32-разрядного слова;
- любая константа в виде 0x00XY00XY;
- любая константа в виде 0xXY00XY00;
- любая константа в виде 0xXYXYXYXY.

Во всех вышеописанных случаях X и Y представляют шестнадцатеричные цифры.

Кроме того, в небольшом количестве инструкций constant может принимать более широкий диапазон значений. Подробности изложены в описании соответствующих инструкций.

При использовании константного операнда Operand2 в командах MOVS, MVNS, ANDS, ORRS, ORNS, EORS, BICS, TEQ и TST в случае, если константа больше 255 и может быть получена путем сдвига восьмиразрядного числа, значение бита [31] константы влияет на значение флага переноса. Для всех остальных значений Operand2 изменения флага переноса не происходит.

Замена инструкций

В случае если пользователь указывает константу, не удовлетворяющую требованиям, приведенным в подпункте 13.2.3.1 «Константа», ассемблер может

сгенерировать код с использованием другой инструкции, обеспечивающей необходимую функциональность.

Например, команда CMP Rd, #0xFFFFFFFFE может быть преобразована в эквивалентную команду CMN Rd, #0x2.

13.2.3.2 Регистр с необязательным параметром сдвига

В данном случае операнд Operand2 задается в форме

Rm {, shift}

где Rm – регистр, содержащий данные для второго операнда инструкции;

shift – необязательный параметр, определяющий сдвиг данных регистра Rm. Он может принимать одно из следующих значений:

- ASR #n – арифметический сдвиг вправо на n бит, $1 \leq n \leq 32$;
- LSL #n – логический сдвиг влево на n бит, $1 \leq n \leq 31$;
- LSR #n – логический сдвиг вправо на n бит, $1 \leq n \leq 32$;
- ROR #n – циклический сдвиг вправо на n бит, $1 \leq n \leq 31$;
- RRX – циклический сдвиг вправо на один бит, с учетом переноса.

Случай, когда сдвиг не указан, эквивалентен заданию сдвига LSL #0. При этом в качестве операнда используется непосредственно значение регистра Rm без каких-либо дополнительных преобразований.

При указании параметра сдвига в качестве операнда используется преобразованное соответствующим образом 32-разрядное значение регистра Rm, однако содержимое самого регистра Rm не меняется.

Использование операнда со сдвигом в некоторых инструкциях влияет на значение флага переноса. Более подробно действие операций сдвига и их влияние на флаг переноса рассмотрено в пункте 13.2.4 «Операции сдвига».

13.2.4 Операции сдвига

Операции сдвига переносят значение бит содержимого регистра влево или вправо на заданное количество позиций – длина сдвига. Сдвиг может выполняться:

- непосредственно с помощью инструкций ASR, LSR, LSL, ROR и RRX, при этом результат сдвига заносится в регистр-получатель;
- во время вычисления значения второго операнда Operand2 команд, при этом результат сдвига используется как один из операндов инструкции.

Допустимая длина сдвига зависит от типа сдвига и инструкции, в которой он был применен. В случае если этот параметр равен 0, фактически сдвиг не производится. Операции сдвига регистра влияют на значение флага переноса, за исключением случая, когда длина сдвига равна 0. Различные варианты сдвига и их влияние на флаг переноса описаны в следующем подразделе (Rm – сдвигаемый регистр, n – длина сдвига).

13.2.4.1 ASR

Арифметический сдвиг вправо на n бит переносит крайние слева 32-n бит регистра Rm вправо на n позиций, то есть на место крайних справа 32-n. Бит [31]

исходного значения регистра записывается в n крайних слева бит результата (см. рисунок 22).

Операцию $ASR \# n$ можно использовать для деления значения регистра Rm на 2^n , с округлением результата в меньшую сторону (в направлении минус бесконечности).

При использовании инструкции $ASRS$, а также в случае, если сдвиг $ASR \# n$ используется при вычислении второго операнда команд $MOVS$, $MVNS$, $ANDS$, $ORRS$, $ORNS$, $EORS$, $BICS$, TEQ или TST , флаг переноса принимает значение последнего бита, вытесненного в результате операции сдвига, то есть бита $[n-1]$ регистра Rm .

В случае если $n \geq 32$, все биты результата устанавливаются в значение бита $[31]$ регистра Rm . Если при этом операция влияет на флаг переноса, то значение этого флага устанавливается равным значению бита $[31]$ регистра Rm .

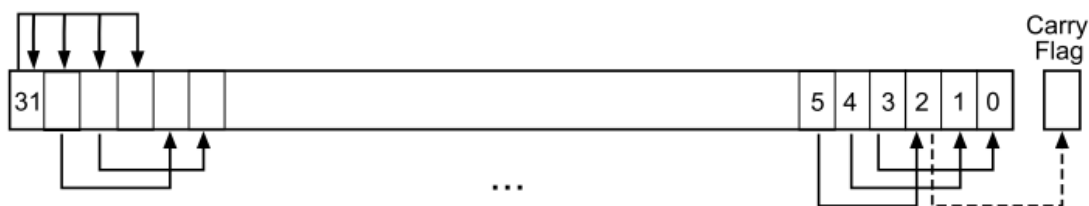


Рисунок 22 – Инструкция $ASR \# 3$

13.2.4.2 LSR

Логический сдвиг вправо на n бит переносит крайние слева $32-n$ бит регистра Rm вправо на n позиций, то есть на место крайних справа $32-n$. При этом в n крайних слева бит результата записывается 0. См. рисунок 23.

Операцию $LSR \# n$ можно использовать для деления значения регистра Rm на 2^n в случае, если значение интерпретируется как целое число без знака.

При использовании инструкции $LSRS$, а также в случае, если сдвиг $LSR \# n$ используется при вычислении второго операнда команд $MOVS$, $MVNS$, $ANDS$, $ORRS$, $ORNS$, $EORS$, $BICS$, TEQ или TST , флаг переноса принимает значение последнего бита, вытесненного в результате операции сдвига, то есть бита $[n-1]$ регистра Rm .

В случае если $n \geq 32$, все биты результата устанавливаются в 0. Если $n \geq 33$ и операция влияет на флаг переноса, то значение этого флага устанавливается равным 0.

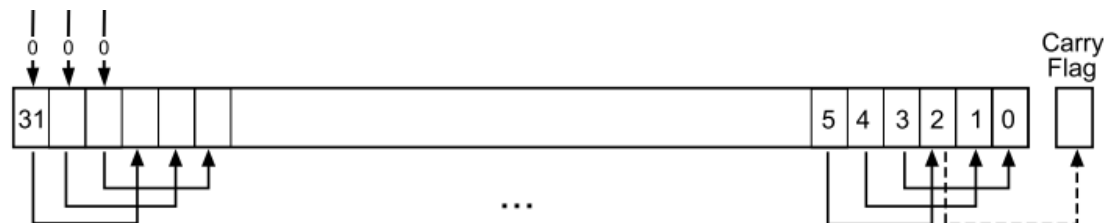


Рисунок 23 – Инструкция $LSR \# 3$

13.2.4.3 LSL

Логический сдвиг влево на n бит переносит крайние справа $32-n$ бит регистра Rm влево на n позиций, то есть на место крайних слева $32-n$. При этом в n крайних слева бит результата записывается 0. Смотрите рисунок 24.

Операцию $LSL \# n$ можно использовать для умножения значения регистра Rm на 2^n в случае, если значение интерпретируется как целое число без знака, либо как целое

число со знаком, записанное в дополнительном коде. Переполнение при выполнении умножения не диагностируется.

При использовании инструкции LSLS, а также в случае, если сдвиг LSL #n используется при вычислении второго операнда команд MOVNS, MVNS, ANDS, ORRS, ORNS, EORS, BICS, TEQ или TST, флаг переноса принимает значение последнего бита, вытесненного в результате операции сдвига, то есть бита [32-n] регистра Rm. Инструкция LSL #0 не влияет на значение флага переноса.

В случае если $n \geq 32$, все биты результата устанавливаются в 0. Если $n \geq 33$ и операция влияет на флаг переноса, то значение этого флага устанавливается равным 0.

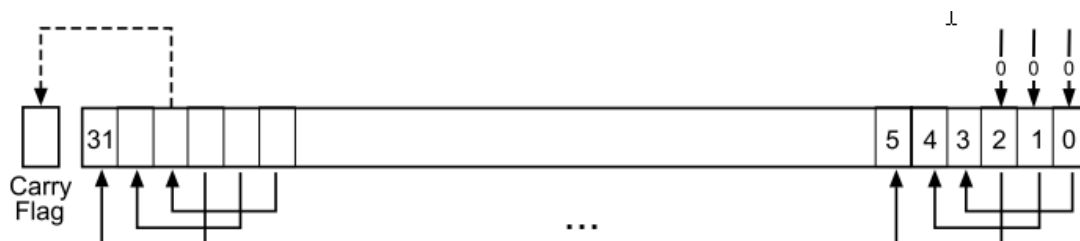


Рисунок 24 – Инструкция LSL # 3

13.2.4.4 ROR

Циклический сдвиг вправо на n бит переносит крайние слева 32-n бит регистра Rm вправо на n позиций, то есть на место крайних справа 32-n. При этом n крайних справа разрядов регистра переносятся в n крайних слева разрядов результата (см. рисунок 25).

При использовании инструкции RORS, а также в случае, если сдвиг ROR #n используется при вычислении второго операнда команд MOVNS, MVNS, ANDS, ORRS, ORNS, EORS, BICS, TEQ или TST, флаг переноса принимает значение последнего сдвинутого бита, то есть бита [n-1] регистра Rm.

В случае если $n = 32$, результат совпадает с исходным значением регистра. Если $n = 32$ и операция влияет на флаг переноса, то значение этого флага устанавливается равным биту [31] регистра Rm.

Операция циклического сдвига ROR с параметром, большим 32, эквивалентна циклическому сдвигу с параметром $n-32$.

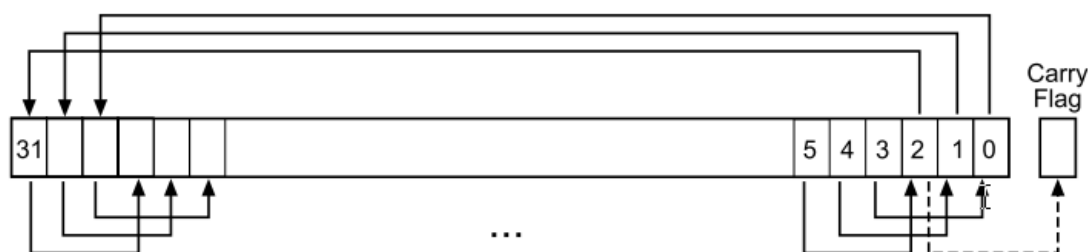


Рисунок 25 – Инструкция ROR # 3

13.2.4.5 RRX

Циклический сдвиг вправо на один бит с переносом переносит разряды регистра Rm вправо на одну позицию, при этом в позицию [31] результата записывается значение флага переноса (см. рисунок 26).

При использовании инструкции RRXS, а также в случае, если сдвиг RRX #n используется при вычислении второго операнда команд MOVS, MVNS, ANDS, ORRS, ORNS, EORS, BICS, TEQ или TST, флаг переноса принимает значение бита [0] регистра Rm.

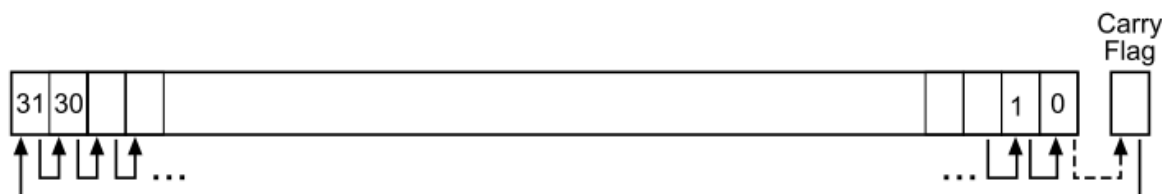


Рисунок 26 – Инструкция RRX

13.2.5 Выравнивание адресов

Под доступом по выровненным адресам понимаются операции, в которых чтение и запись слов, двойных слов и более длинных последовательностей слов осуществляется по адресам, выровненным по границе слова, а доступ к полусловам осуществляется по адресам, выровненным по границе полуслова. Чтение и запись байтов гарантированно являются выровненными.

Процессор Cortex-M3 поддерживает доступ по невыровненным адресам только для следующих инструкций:

- LDR, LDRT;
- LDRH, LDRHT;
- LDRSH, LDRSHT;
- STR, STRT;
- STRH, STRHT.

Все остальные инструкции при попытке доступа по невыровненному адресу генерируют исключение (usage fault). Подробнее см. подраздел 31.7 «Обработка отказов».

Невыровненный доступ к данным, как правило, осуществляется медленнее, чем выровненный. Кроме того, некоторые области адресного пространства могут не поддерживать доступ по невыровненному адресу. В связи с этим ARM рекомендует программистам обеспечивать необходимое выравнивание данных. Для того чтобы избежать случаев, в которых невыровненный доступ осуществляется непреднамеренно, следует установить в 1 бит UNALIGN_TRP регистра конфигурации и управления CCR, что приведет тогда к формированию процессором исключительной ситуации (см. пункт 33.1.7 «SCB->CCR»).

13.2.6 Адресация относительно счетчика команд PC

В системе команд Cortex-M3 предусмотрена адресация команды или области данных в виде суммы значения счетчика команд PC плюс/минус численное смещение. Смещение вычисляется ассемблером автоматически, исходя из адреса метки и текущего адреса. В случае, если смещение слишком велико, диагностируется ошибка.

Для инструкций B, BL, CBNZ и CBZ текущий адрес определяется как адрес этой инструкции плюс 4 байта;

Для всех остальных инструкций текущий адрес определяется как адрес инструкции плюс 4 байта, при этом бит [1] результата должен быть установлен в 0 для обеспечения выравнивания адреса по границе слова.

Ассемблер может поддерживать расширенные варианты синтаксиса для адресации относительно PC, например, «метка плюс/минус число» или выражения типа [PC, #number].

13.2.7 Условное исполнение

Большая часть команд обработки данных способна изменять значения флагов в регистре состояния прикладной программы (APSR) в зависимости от результата выполнения (см. пункт 12.3.6 «Программный регистр состояния приложения APSR»).

Некоторые команды влияют на все флаги, некоторые только на часть. В случае если инструкция не меняет значение данного флага, сохраняется его старое значение. Более подробно влияние на флаги рассмотрено в описании конкретных инструкций.

Возможность исполнения или неисполнения инструкции в зависимости от значения флагов, сформированных ранее, может быть достигнута либо за счет использования условных переходов, либо путем добавления суффикса условия исполнения к инструкции. В таблице 56 представлен список суффиксов, которые можно добавить к инструкции для того, чтобы сделать ее условной.

При наличии одного из указанных суффиксов процессор проверяет значение флагов на соответствие заданному условию. Если условие не выполняется, то инструкция:

- не исполняется;
- не записывает значение операции в регистр-получатель;
- не влияет на флаги;
- не генерирует исключений.

Условные инструкции, за исключением условных переходов, должны располагаться внутри блока условно исполняемых инструкций (далее по тексту - IT-блок). Ассемблеры некоторых поставщиков могут самостоятельно вставлять инструкцию IT в случае, если программист разместит условную инструкцию за пределами IT-блока.

Для сравнения регистра с нулем и условного перехода по результату рекомендуется использовать команды CBZ и CBNZ.

Ниже в разделе рассматриваются:

- флаги условий;
- суффиксы условного исполнения.

13.2.7.1 Флаги условий

Регистр состояния прикладной программы APSR содержит следующие флаги:

- N=1 в случае, если результат операции меньше нуля, в противном случае – 0.
- Z=1 в случае, если результат равен нулю, в противном случае – 0.
- C=1 в случае, если при выполнении операции возник перенос, в противном случае – 0.

– $V=1$ в случае, если при выполнении операции возникло переполнение, в противном случае – 0.

Более подробно регистр APSR рассмотрен в пункте 12.3.5 «Программный регистр состояния PSR».

Перенос возникает в следующих случаях:

- результат сложения оказался больше или равен 232;
- результат вычитания больше или равен нулю;
- в результате работы внутренней логики процессора при операциях загрузки данных и логических операций.

Переполнение возникает в случае, если результат сложения, вычитания или сравнения больше или равен 2^{31} , либо меньше -2^{31} .

Большая часть инструкций меняют значение флагов только в случае, если у них указан суффикс S. Подробную информацию см. в описании конкретных команд.

13.2.7.2 Суффиксы условного исполнения

В мнемокодах команд, допускающих условное исполнение, предусмотрена возможность указания необязательного кода условия. В описании синтаксиса это обозначается как {cond}. Исполнению условной инструкции должна предшествовать инструкция IT.

Если код условия указан, инструкция выполняется только при удовлетворении соответствующему условию флагов регистра APSR. Используемые коды представлены в таблице 56. Там же указаны соответствующие логические выражения для значений флагов.

Условные команды рекомендуется использовать для снижения количества ветвлений в программе.

Таблица 56 – Суффиксы условного исполнения

Суффикс	Флаги	Значение
EQ	$Z = 1$	Равенство
NE	$Z = 0$	Неравенство
CS или HS	$C = 1$	Больше или равно, беззнаковое сравнение
CC или LO	$C = 0$	Меньше, беззнаковое сравнение
MI	$N = 1$	Меньше нуля
PL	$N = 0$	Больше или равно нулю
VS	$V = 1$	Переполнение
VC	$V = 0$	Нет переполнения
HI	$C = 1 \text{ and } Z = 0$	Больше, беззнаковое сравнение
LS	$C = 0 \text{ or } Z = 1$	Меньше или равно, беззнаковое сравнение
GE	$N = V$	Больше или равно, знаковое сравнение
LT	$N \neq V$	Меньше, знаковое сравнение
GT	$Z = 0 \text{ and } N = V$	Больше, знаковое сравнение
LE	$Z = 1 \text{ and } N \neq V$	Меньше или равно, знаковое сравнение
AL	1	Безусловное исполнение

13.2.7.3 Пример. Вычисление абсолютного значения

В данном примере проиллюстрировано использование условных инструкций для вычисления абсолютного значения числа: $R0 = \text{ABS}(R1)$.

MOVS R0, R1	– R0 = R1, установка флагов
IT MI	– IT инструкция для условия отрицательного результата
RSBMI R0, R1, #0	– если результат был меньше нуля, присвоить R0 = – R1

13.2.7.4 *Пример. Сравнение и изменение значения регистра*

В данном примере условные инструкции используются во фрагменте кода, изменяющего значение регистра R4 в случае, если число со знаком в регистр R0 больше числа в R1 и R2 больше R3.

CMP R0, R1 – Сравнение R0 и R1, установка флагов
ITT GT – IT инструкция для двух условий «больше»
CMPGT R2, R3 – если R0>R1, сравнить R2 и R3, установить флаги
MOVGT R4, R5 – если условие «больше» все еще выполняется,
присвоить R4 = R5

13.2.8 Выбор размера кода инструкции

Многие команды процессора Cortex-M3 могут быть представлены как 16-разрядными, так и 32-разрядными кодами инструкции. Во многих случаях выбор формата зависит от операндов и регистра-получателя результата.

Нередко существует возможность принудительно задать размер инструкции с помощью суффикса размера команды. Суффикс `.W` задает кодирование команды в 32-битном формате, суффикс `.N` – в 16-битном формате.

В случае если суффикс размера указан, а ассемблер не в состоянии сгенерировать код команды соответствующего размера, диагностируется ошибка.

Для того чтобы принудительно задать размер кода инструкции, необходимо поместить соответствующий суффикс непосредственно после мнемокода команды и кода условного выполнения, если он указан. В примере, приведенном ниже, представлены инструкции с заданным кодом размера.

BCS.W label – формирует 32-битный код команды, даже для коротких переходов
 ADDS.W R0, R0, R1 – формирует 32-битный код команды, хотя данная операция
 может быть закодирована 16 битами

13.3 Команды доступа к памяти

Обобщенные данные о командах доступа к памяти представлены в таблице 57.

Таблица 57 – Команды доступа к памяти

Мнемокод	Краткое описание	Прим.
ADR	Загрузка адреса, заданного относительно счетчика команд	
CLREX	Сброс эксклюзивного доступа	
LDM{mode}	Загрузка множества регистров	
LDR{type}	Загрузка регистра, непосредственно указанное смещение	
LDR{type}	Загрузка регистра, смещение в регистре	
LDR{type}T	Загрузка регистра с непривилегированным доступом	
LDR	Загрузка регистра по относительному адресу	
LDREX{type}	Эксклюзивное чтение регистра	
POP	Извлечение регистров из стека	
PUSH	Загрузка регистров в стек	
STM{mode}	Сохранение множества регистров	
STR{type}	Сохранение регистра, непосредственно указанное смещение	
STR{type}	Сохранение регистра, смещение в регистре	
STR{type}T	Сохранение регистра с непривилегированным доступом	
STREX{type}	Эксклюзивная запись регистра	

13.3.1 ADR

Загрузка адреса, заданного относительно счетчика команд.

13.3.1.1 Синтаксис

ADR{cond} Rd, label

Где:

cond – необязательный код условия, см. пункт 13.2.7 «Условное исполнение».

Rd – регистр-получатель.

label – относительный адрес, см. пункт 13.2.6 «Адресация относительно счетчика команд PC».

13.3.1.2 Описание

Инструкция ADR вычисляет адрес доступа к памяти путем сложения текущего значения счетчика команд PC и непосредственно заданного смещения, после чего записывает результат в регистр-получатель.

Благодаря использованию относительной адресации, код команды не зависит от ее размещения в физической памяти.

При формировании с помощью команды ADR адреса перехода для команд BX или BLX программисту необходимо убедиться, что бит [0] формируемого адреса установлен в 1.

Значения смещения относительно PC должны находиться в пределах -4095...+4095. Для того чтобы использовать максимально широкий диапазон относительных адресов, а также чтобы иметь возможность генерировать адреса, не

выровненные по границе слова, может потребоваться задать суффикс .W (см. пункт 13.2.8 «Выбор размера кода инструкции»).

13.3.1.3 Ограничения

В качестве регистра Rd нельзя использовать указатель стека SP и счетчик команд PC.

13.3.1.4 Флаги

Данная инструкция не влияет на состояние флагов.

13.3.1.5 Примеры

ADR R1, TextMessage – Загрузить адрес позиции, указанный меткой TextMessage, в регистр R1.

13.3.2 LDR и STR, непосредственно заданное смещение

Загрузка или сохранение регистра в режиме адресации со смещением, адресации с пре-индексированием или адресации с пост-индексированием.

13.3.2.1 Синтаксис

op{type}{cond} Rt, [Rn {, #offset}] – адресация со смещением

op{type}{cond} Rt, [Rn, #offset]! – преиндексирование

op{type}{cond} Rt, [Rn], #offset – пост-индексирование

opD{cond} Rt, Rt2, [Rn {, #offset}] – адресация со смещением, двойное слово

opD{cond} Rt, Rt2, [Rn, #offset]! – преиндексирование, двойное слово

opD{cond} Rt, Rt2, [Rn], #offset – пост-индексирование, двойное слово

где op – один из кодов операций:

- LDR – загрузить регистр;
- STR – сохранить регистр.

type – один из суффиксов размера данных:

- B – байт без знака, при загрузке старшие байты устанавливаются в 0;
- SB – байт со знаком, при загрузке происходит распространение знакового бита

в старшие байты (только LDR);

- H – беззнаковое полуслово, при загрузке старшие байты устанавливаются в 0;

– SH – полуслово со знаком, при загрузке происходит распространение знакового бита в старшие байты (только LDR);

- без суффикса – 32-разрядное слово.

cond – необязательный код условия, см. пункт 13.2.7 «Условное исполнение».

Rt – регистр, в который должна производиться загрузка или значение которого должно быть сохранено.

Rn – регистр, содержащий базовый адрес памяти.

offset – смещение относительно базового адреса Rn. В случае если смещение не указано, оно подразумевается равным нулю.

Rt2 – дополнительный регистр, предназначенный для двухсловных операций чтения или записи.

13.3.2.2 Описание

LDR – загружает один или два регистра значением из памяти.

STR – сохраняет значение одного или двух регистров в память.

Инструкции с непосредственно заданным смещением могут функционировать в одном из следующих режимов адресации:

Адресация со смещением

Значение смещения добавляется к или вычитается из содержимого регистра Rn. Результат используется в качестве адреса чтения или записи. Значение регистра Rn остается неизменным.

Синтаксис задания данного режима:

[Rn, #offset].

Адресация с пре-индексированием

Значение смещения добавляется к или вычитается из содержимого регистра Rn. Результат используется в качестве адреса чтения или записи, а также записывается обратно в регистр Rn.

Синтаксис задания данного режима:

[Rn, #offset]!.

Адресация с пост-индексированием

Содержимое регистра Rn используется в качестве адреса чтения или записи. Значение смещения добавляется к или вычитается из содержимого регистра Rn, после чего записывается обратно в регистр Rn.

Синтаксис задания данного режима:

[Rn], #offset .

Загружаемое или сохраняемое значение может быть байтом, полусловом, словом или двойным словом. Байты и полуслова могут интерпретироваться как числа со знаком или без знака – см. пункт 13.2.5 «Выравнивание адресов».

В таблице 58 приведены диапазоны значений смещения для различных форм адресации.

Таблица 58 – Диапазон значений смещения

Тип инструкции	Смещение	Преиндексирование	Пост-индексирование
Слово, полуслово, байт	от -255 до 4095	от -255 до 255	от -255 до 255
Двойное слово	Значения, кратные 4, в диапазоне от -1020 до 1020		

13.3.2.3 Ограничения

Для команд загрузки регистров:

– использовать в качестве Rt регистры PC и SP можно только в командах загрузки слова;

- при загрузке двойных слов регистры Rt и Rt2 не должны совпадать;
- в режимах адресации с пре- и пост-индексированием регистр Rn не должен совпадать с регистрами Rt или Rt2.

В случае если в команде загрузки слова в качестве регистра Rt используется счетчик команд PC:

- бит [0] загружаемого значения должен быть равен 1;
- передача управления происходит по адресу, соответствующему значению бита [0] в 0;
- если инструкция является условной, то она должна быть последней инструкцией в IT-блоке.

Для команд сохранения регистров:

- использовать в качестве Rt регистры SP можно только в командах записи слова;
- в качестве регистров Rt и Rn нельзя использовать счетчик команд PC;
- в режимах адресации с пре- и пост-индексированием регистр Rn не должен совпадать с регистрами Rt или Rt2.

13.3.2.4 Флаги

Данная инструкция не влияет на состояние флагов.

13.3.2.5 Примеры

- LDR R8, [R10] – Загрузка регистра R8 из ячейки по адресу, содержащемуся в R10
- LDRNE R2, [R5, #960]! – Условная загрузка R2 из слова памяти, расположенного на 960 байт выше адреса в регистре R5, увеличение регистра R5 на 960
- STR R2, [R9, #const-struct] – const-struct - выражение с постоянным значением, лежащим в диапазоне 0-4095
- STRH R3, [R4], #4 – Записать содержимое R3, интерпретируемое как полуслово, по адресу, содержащемуся в R4, после чего увеличить R4 на 4
- LDRD R8, R9, [R3, #0x20] – Загрузить R8 словом данных, расположенным на 32 байта выше адреса в R3, загрузить R9 словом данных, расположенным на 36 байта выше адреса в R3
- STRD R0, R1, [R8], #-16 – Сохранить R0 по адресу, содержащемуся в R8, сохранить R1 по адресу, расположенному на 4 байта выше адреса в R8, уменьшить значение R8 на 16

13.3.3 LDR и STR, смещение задано в регистре

Загрузка или сохранение регистра в режиме адресации со смещением, заданным в регистре.

13.3.3.1 Синтаксис

`op{type}{cond} Rt, [Rn, Rm {, LSL #n}]`

где `op` – один из кодов операций:

- `LDR` загрузить регистр.
- `STR` сохранить регистр.

`type` – один из суффиксов размера данных:

- `B` – байт без знака, при загрузке старшие байты устанавливаются в 0.
- `SB` – байт со знаком, при загрузке происходит распространение знакового бита в старшие байты (только `LDR`).
- `H` – беззнаковое полуслово, при загрузке старшие байты устанавливаются в 0.
- `SH` – полуслово со знаком, при загрузке происходит распространение знакового бита в старшие байты (только `LDR`).
- без суффикса – 32-разрядное слово.

`cond` – необязательный код условия, см. “Условное исполнение”.

`Rt` – регистр, в который должна производиться загрузка или значение которого должно быть сохранено.

`Rn` – регистр, содержащий базовый адрес памяти.

`Rm` – регистр, содержащий смещение относительно базового адреса.

`LSL #n` – необязательный параметр сдвига, в диапазоне от 0 до 3.

13.3.3.2 Описание

`LDR` – загружает регистра значением из памяти.

`STR` – сохраняет значение регистра в памяти.

Адрес области памяти, в которую будет производиться обращение, вычисляется на основании значения базового адреса в регистре `Rn` и смещения. Смещение определяется значением регистра `Rm` и параметром сдвига влево значения этого регистра.

Считываемое или записываемое значение может иметь размер байта, полуслова или слова. При загрузке данных из памяти байты и полуслова могут интерпретироваться либо как числа со знаком, либо как беззнаковые - см. “Выравнивание адресов”.

13.3.3.3 Ограничения

Для данных команд:

- `Rn` не может быть счетчиком команд `PC`;
- `Rm` не может быть `SP` или `PC`;
- использовать в качестве `Rt` регистр `SP` можно только в командах чтения и записи слова;
- использовать в качестве `Rt` регистр `PC` можно только в командах чтения слова;

В случае, если в команде загрузки слова в качестве регистра `Rt` используется счетчик команд `PC`:

- бит [0] загружаемого значения должен быть равен 1, передача управления при этом осуществляется по выровненному по границе полуслова адресу;

– если инструкция является условной, то она должна быть последней инструкцией в IT-блоке.

13.3.3.4 Флаги

Данная инструкция не влияет на состояние флагов.

13.3.3.5 Примеры

STR R0, [R5, R1] – Записать значение R0 по адресу, равному сумме R5 и R1

LDRSB R0, [R5, R1, LSL #1] – Считать байт по адресу, равному сумме R5 и R1, умноженному на два, распространить значение знакового бита на старшие значащие байты слова, загрузить результат в регистр R0

STR R0, [R1, R2, LSL #2] – Сохранить значение регистра R0 по адресу, равному $R1 + 4 \cdot R2$.

13.3.4 LDR and STR, непривилегированный доступ

Загрузка или сохранение регистра в режиме непривилегированного доступа.

13.3.4.1 Синтаксис

`ор{type}T{cond} Rt, [Rn {, #offset}]`

где ор – один из кодов операций:

- LDR загрузить регистр.
- STR сохранить регистр.

type – один из суффиксов размера данных:

- B – байт без знака, при загрузке старшие байты устанавливаются в 0.
- SB – байт со знаком, при загрузке происходит распространение знакового бита в старшие байты (только LDR).

– H – беззнаковое полуслово, при загрузке старшие байты устанавливаются в 0.

– SH – полуслово со знаком, при загрузке происходит распространение знакового бита в старшие байты (только LDR).

– без суффикса – 32-разрядное слово.

cond – необязательный код условия, см. пункт 13.2.7 «Условное исполнение».

Rt – регистр, в который должна производиться загрузка или значение которого должно быть сохранено.

Rn – регистр, содержащий базовый адрес памяти.

offset – смещение относительно базового адреса Rn в диапазоне от 0 до 255.

В случае если смещение не указано, оно подразумевается равным 0.

13.3.4.2 Описание

Описываемые инструкции загрузки и сохранения данных функционируют также, как и инструкции доступа к памяти с непосредственно задаваемым смещением, см. пункт 13.3.2 «LDR и STR, непосредственно заданное смещение».

Отличие состоит в том, что рассматриваемые в данном разделе команды обеспечивают исключительно непривилегированный доступ, даже в случае, если они исполняются в привилегированном приложении.

При использовании в непривилегированном приложении какие-либо отличия от команд нормального доступа к памяти с непосредственно задаваемым смещением отсутствуют.

13.3.4.3 Ограничения

В данных инструкциях:

- Rn не может быть счетчиком команд PC;
- Rt не может быть SP или PC.

13.3.4.4 Флаги

Данная инструкция не влияет на состояние флагов.

13.3.4.5 Примеры

STRBTEQ R4, [R7] – Условная запись младшего значащего байта в регистр R4 по адресу, хранящемуся в R7, с непривилегированным доступом;

LDRHT R2, [R2, #8] – Загрузка полуслова из памяти по адресу, равному сумме регистра R2 и 8 в регистр R2, с непривилегированным доступом.

13.3.5 LDR, адресация относительно счетчика команд PC

Загрузка регистра из памяти.

13.3.5.1 Синтаксис

LDR{type}{cond} Rt, label

LDRD{cond} Rt, Rt2, label ; Load two words

где type – один из суффиксов размера данных:

- B – байт без знака, при загрузке старшие байты устанавливаются в 0.
- SB – байт со знаком, при загрузке происходит распространение знакового бита в старшие байты (только для LDR).
- H – беззнаковое полуслово, при загрузке старшие байты устанавливаются в 0.
- SH – полуслово со знаком, при загрузке происходит распространение знакового бита в старшие байты (только для LDR).
- без суффикса – 32-разрядное слово.

cond – необязательный код условия, см. пункт 13.2.7 «Условное исполнение».

Rt – регистр, в который должна производиться загрузка.

Rt2 – второй регистр, в который должна производиться загрузка.

label – относительный адрес, см. пункт 13.2.6 «Адресация относительно счетчика команд PC».

13.3.5.2 Описание

LDR – загружает регистр значением из памяти с адресом относительно счетчика команд PC, заданным в виде метки.

Считываемое значение может иметь размер байта, полуслова или слова. При загрузке данных из памяти байты и полуслова могут интерпретироваться либо как числа со знаком, либо как беззнаковые (см. пункт 13.2.5 «Выравнивание адресов»).

Метка должна располагаться на ограниченном расстоянии от текущей инструкции. Таблица 59 показывает возможные значения смещений между меткой данных и текущим значением счетчика команд. Для использования больших смещений, возможно, придется указать суффикс .W размера команды (см. пункт 13.2.8 «Выбор размера кода инструкции»).

Таблица 59 – Диапазон значений смещения

Тип инструкции	Диапазон значений смещения
Слово, полуслово со знаком или без знака, байт со знаком или без знака	от -4095 до 4095
Двойное слово	от -1020 до 1020

13.3.5.3 Ограничения

В данной инструкции:

- использовать в качестве Rt регистры PC или SP можно только в командах чтения слова;
- нельзя использовать в качестве Rt2 регистры PC и SP;
- при загрузке двойных слов регистры Rt и Rt2 не должны совпадать.

В случае если в команде загрузки слова в качестве регистра Rt используется счетчик команд PC:

- бит [0] загружаемого значения должен быть равен 1, передача управления при этом осуществляется по выровненному по границе полуслова адресу;
- если инструкция является условной, то она должна быть последней инструкцией в IT-блоке.

13.3.5.4 Флаги

Данная инструкция не влияет на состояние флагов.

13.3.5.5 Примеры

LDR R0, LookUpTable – Загрузить R0 словом данных по адресу с меткой LookUpTable;

LDRSB R7, localdata – Загрузить байт данных по адресу с меткой localdata, распространить значение знакового бита в старшие байты слова данных, сохранить результат в R7.

13.3.6 LDM и STM

Загрузка или сохранение множества регистров.

13.3.6.1 Синтаксис

op{addr_mode}{cond} Rn{!}, reglist

где op – один из кодов операций:

- LDM загрузить множество регистров.
- STM сохранить множество регистров.
- `addr_mode` – один из режимов адресации:
- IA – с увеличением адреса после каждого доступа. Этот режим используется по умолчанию.

- DB – с уменьшением адреса перед каждым доступом.

`cond` – необязательный код условия, см. пункт 13.2.7 «Условное исполнение».

`Rn` – регистр, содержащий базовый адрес памяти.

! – необязательный суффикс обратной записи значения базового регистра. В случае если он присутствует в команде, последний адрес, по которому осуществлялся доступ, будет записан обратно в регистр `Rn`.

`reglist` – заключенный в фигурные скобки список из одного или нескольких регистров, которые должны быть записаны или считаны. В списке можно указывать диапазон номеров регистров. Начальный и конечный регистр диапазона разделены знаком "-". Элементы списка (отдельные регистры или диапазоны) разделяются запятыми.

Мнемокоды LDM и LDMFD являются синонимами LDMIA. Наименование LDMFD обусловлено использованием данной команды для организации извлечения данных из стека, растущего вниз, с указателем на последний загруженный элемент (Full Descending stack).

Мнемокод LDMEA является синонимом LDMDB, его наименование обусловлено использованием данной команды для организации извлечения данных из стека, растущего вверх, с указателем на последнюю свободную ячейку (Empty Ascending stack).

Мнемокоды STM и STMEA являются синонимами STMIA. Наименование STMEA обусловлено использованием данной команды для организации записи данных из стека, растущего вверх, с указателем на последнюю свободную ячейку.

Мнемокод STMFD является синонимом STMDB, его наименование обусловлено использованием данной команды для организации извлечения данных из стека, растущего вниз, с указателем на последний загруженный элемент.

13.3.6.2 Описание

Инструкции LDM загружают регистры, перечисленные в списке `reglist`, значениями слов данных из памяти с базовым адресом, указанным в регистре `Rn`.

Инструкции STM записывают слова данных, содержащиеся в регистрах, перечисленных в списке `reglist`, в память с базовым адресом, указанным в регистре `Rn`.

Команды LDM, LDMIA, LDMFD, STM, STMIA и STMEA для доступа используют адреса памяти в интервале от `Rn` до $Rn + 4 \cdot (n - 1)$, где n - количество регистров в списке `reglist`. Доступ осуществляется в порядке увеличения номера регистра, при этом регистр с наименьшим номером соответствует наименьшему адресу памяти, а регистр с наибольшим номером - наибольшему адресу. В случае если в команде указан суффикс «!», значение $Rn + 4 \cdot (n - 1)$ записывается обратно в регистр `Rn`.

Команды LDMDB, LDMEA, STMDB и STMFD для доступа используют адреса памяти в интервале от `Rn` до $Rn - 4 \cdot (n - 1)$, где n - количество регистров в списке `reglist`.

Доступ осуществляется в порядке уменьшения номера регистра, при этом регистр с наибольшим номером соответствует наибольшему адресу памяти, а регистр с наименьшим номером - наименьшему адресу. В случае если в команде указан суффикс «!», значение $R_n - 4 \cdot (n - 1)$ записывается обратно в регистр R_n .

Инструкции PUSH и POP могут быть выражены через инструкции LDM и STM. Подробнее см. в пункте 13.3.7 «PUSH и POP».

13.3.6.3 Ограничения

В описываемых в разделе командах:

- в качестве регистра R_n нельзя использовать счетчик команд PC;
- список регистров reglist не может содержать указатель стека SP;
- в любой инструкции STM в списке регистров reglist нельзя указывать PC;
- в любой инструкции LDM в reglist нельзя указывать одновременно PC и LR;
- список reglist не может содержать R_n в случае, если указан суффикс «!».

В случае если инструкция LDM содержит в списке reglist счетчик команд PC:

- бит [0] загружаемого значения должен быть равен 1, передача управления при этом осуществляется по выровненному по границе полуслова адресу;
- если инструкция является условной, то она должна быть последней инструкцией в IT-блоке.

13.3.6.4 Флаги

Данная инструкция не влияет на состояние флагов.

13.3.6.5 Примеры

LDM R8,{R0,R2,R9} – LDMIA - синоним LDM
STMDB R1!,{R3-R6,R11,R12}

13.3.6.6 Примеры неправильного использования

STM R5!,{R5,R4,R9} – Сохраненное значение R5 является непредсказуемым;
LDM R2, {} – Список должен содержать хотя бы один регистр.

13.3.7 PUSH и POP

Загружает или считывает регистры в стек или из стека, растущего вниз, с указателем на последний загруженный элемент (full-descending stack).

13.3.7.1 Синтаксис

PUSH{cond} reglist
POP{cond} reglist

где cond – необязательный код условия, см. пункт 13.2.7 «Условное исполнение».
reglist – заключенный в фигурные скобки список из одного или нескольких регистров, которые должны быть записаны или считаны. В списке можно указывать диапазон номеров регистров. Начальный и конечный регистр диапазона разделены знаком "-". Элементы списка (отдельные регистры или диапазоны) разделяются запятыми.

Команды PUSH и POP являются синонимами команд STMDB и LDM (LDMIA) в которых базовый адрес памяти содержится в регистре указателя стека SP, а режим записи обратной записи значения базового регистра включен.

Мнемокоды PUSH и POP являются предпочтительными вариантами записи.

13.3.7.2 Описание

PUSH – сохраняет регистры в стеке в порядке уменьшения номеров регистров, при этом регистр с большим номером сохраняется в память с большим значением адреса.

POP – восстанавливает значения регистров из стека в порядке увеличения номеров регистров, при этом регистр с меньшим номером считывается из памяти с меньшим значением адресом.

Подробнее см. пункт 13.3.6 «LDM и STM».

13.3.7.3 Ограничения

В данных инструкциях:

- список регистров reglist не должен содержать указатель стека SP;
- в инструкции PUSH список регистров не должен содержать счетчик команд PC;
- в инструкции POP список регистров не должен одновременно содержать регистры PC и LR.

В случае если инструкция POP содержит в списке reglist счетчик команд PC:

- бит [0] загружаемого значения должен быть равен 1, передача управления при этом осуществляется по выровненному по границе полуслова адресу;
- если инструкция является условной, то она должна быть последней инструкцией в IT-блоке.

13.3.7.4 Флаги

Данная инструкция не влияет на состояние флагов.

13.3.7.5 Примеры

PUSH {R0,R4-R7}

PUSH {R2,LR}

POP {R0,R10,PC}

13.3.8 LDREX и STREX

Эксклюзивное чтение и запись регистров.

13.3.8.1 Синтаксис

LDREX{cond} Rt, [Rn {, #offset}]

STREX{cond} Rd, Rt, [Rn {, #offset}]

LDREXB{cond} Rt, [Rn]

STREXB{cond} Rd, Rt, [Rn]

LDREXH{cond} Rt, [Rn]

STREXH{cond} Rd, Rt, [Rn]

где cond – необязательный код условия, см. пункт 13.2.7 «Условное исполнение».

Rd – регистр-приемник, содержащий признак успешного выполнения операции.

Rt – записываемый (считываемый) регистр.

Rn – регистр, содержащий базовый адрес памяти.

offset – необязательный параметр – смещение данных относительно базового адреса. В случае если параметр опущен, он предполагается равным 0.

13.3.8.2 Описание

Команды LDREX, LDREXB и LDREXH позволяют загрузить соответственно слово, байт или полуслово данных из области памяти с заданным адресом.

Команды STREX, STREXB и STREXH осуществляют попытку записи соответственно слова, байта или полуслова данных в область памяти с заданным адресом.

Адрес, используемый в инструкции эксклюзивной записи должен совпадать с адресом последней выполненной команды эксклюзивного чтения. Кроме того, значение, сохраняемое с помощью инструкции эксклюзивной записи, должно иметь тот же размер данных, что и значение, считанное предшествующей инструкцией эксклюзивного чтения.

Таким образом, программное обеспечение должно всегда использовать инструкции эксклюзивного чтения и записи совместно, в интересах решения задач синхронизации – см. подраздел 9.13 «Примитивы синхронизации».

В случае если инструкция эксклюзивной записи успешно выполнила операцию, она записывает в регистр-получатель значение 0. В противном случае она возвращает 1. Запись в регистр-получатель значения 0 гарантирует, что никакие другие процессы не смогут получить доступ к данной ячейке памяти в промежутке времени между выполнением команд эксклюзивного чтения и эксклюзивной записи.

В интересах обеспечения высокой производительности необходимо свести количество операций между командами эксклюзивного чтения и эксклюзивной записи к минимуму.

Результат выполнения операции эксклюзивной записи по адресу, отличному от использованного ранее в инструкции эксклюзивного чтения, непредсказуем.

13.3.8.3 Ограничения

В рассматриваемых командах:

- нельзя использовать счетчик команд PC;
- нельзя использовать указатель стека SP в качестве регистров Rd и Rt;
- в инструкции STREX регистр Rd должен не совпадать с регистрами Rt и Rn;
- значение смещения offset должно быть кратно 4 и лежать в диапазоне от 0

до 1020.

13.3.8.4 Флаги

Данная инструкция не влияет на состояние флагов.

13.3.8.5 Примеры

MOV R1, #0x1 – Записать в регистр R1 значение, соответствующее блокировке ресурса try;

LDREX R0, [LockAddr] – Считать значение признака блокировки

CMP R0, #0 – ресурс свободен?

Мнемокод	Краткое описание
MOVW	Загрузка 16-битной константы
MVN	Загрузка инверсного значения
ORN	Логическое ИЛИ-НЕ
ORR	Логическое ИЛИ
RBIT	Обратить порядок бит
REV	Изменить на обратный порядок байтов в слове
REV16	Изменить на обратный порядок байтов в полусловах
REVSH	Изменить на обратный порядок байт в младшем полуслове, произвести распространение знакового бита в старшее полуслово
ROR	Циклический сдвиг вправо
RRX	Циклический сдвиг вправо на один бит с учетом переноса
RSB	Вычитание с противоположным порядком аргументов
SBC	Вычитание с учетом переноса
SUB	Вычитание
SUBW	Вычитание
TEQ	Проверка равенства
TST	Проверка значения бит по маске

13.4.1 ADD, ADC, SUB, SBC и RSB

Сложение, сложение с переносом, вычитание, вычитание с переносом, вычитание с противоположным порядком аргументов.

13.4.1.1 Синтаксис

ор{S}{cond} {Rd,} Rn, Operand2

ор{cond} {Rd,} Rn, #imm12 ; только для команд ADD и SUB.

где ор – один из кодов операции:

- ADD – сложение;
- ADC – сложение с учетом переноса;
- SUB – вычитание;
- SBC – вычитание с учетом переноса;
- RSB – вычитание с противоположным порядком аргументов;

S – необязательный суффикс. Если он указан, результат выполнения операции приводит к установке соответствующих флагов, см. пункт 13.2.7 «Условное исполнение»;

cond – необязательный суффикс условного исполнения, см. пункт 13.2.7 «Условное исполнение»;

Rd – регистр-получатель результата. В случае если регистр Rd не указан, результат записывается в Rn.

Rn – регистр, содержащий значение первого операнда;

Operand2 – второй операнд, см. пункт 13.2.3 «Формат второго операнда»;

imm12 – любое число в диапазоне от 0 до 4095.

13.4.1.2 Описание

Команда ADD складывает значение Operand2 или imm12 со значением регистра Rn.

Команда ADC складывает вместе значения Rn и Operand2, а также флага переноса.

Команда SUB вычитает значение Operand2 или imm12 из значения регистра Rn.

Команда SBC вычитает значение Operand2 из значения регистра Rn. Если флаг переноса не установлен, результат дополнительно уменьшается на единицу.

Команда RSB вычитает значение регистра Rn из значения Operand2. Этот вариант команды полезен, так как существует широкий выбор вариантов построения Operand2.

Инструкции ADC и SBC полезны при реализации вычислений с повышенной разрядностью, см. подпункт 13.4.1.6 «Арифметика с повышенной разрядностью».

См. также описание команды ADR.

Команда ADDW эквивалентна команде ADD, однако использует 12-разрядный непосредственный операнд imm12. Команда SUBW эквивалентна команде SUB, однако использует 12-разрядный непосредственный операнд imm12.

13.4.1.3 Ограничения

Для рассматриваемых инструкций:

- в качестве Operand2 нельзя использовать SP или PC;
- использовать SP в качестве регистра Rd допустимо только в командах ADD и SUB, со следующими дополнительными ограничениями:
 - в качестве Rn также должен использоваться SP;
 - сдвиг в Operand2 должен быть не более 3 бит в режиме LSL;
 - указатель стека SP может использоваться в качестве Rn только в командах ADD и SUB;
- счетчик команд PC может использоваться в качестве Rd только в команде: ADD{cond} PC, PC, Rm причем:
 - не допускается использование суффикса S;
 - в качестве Rm не допускается использовать PC и SP;
 - если инструкция условная, то она должна быть последней в IT-блоке.
- в качестве регистра Rn можно использовать счетчик команд PC только в инструкциях ADD и SUB (за исключением команды ADD{cond} PC, PC, Rm) с дополнительными ограничениями:
 - не допускается использование суффикса S;
 - второй операнд должен находиться в интервале от 0 до 4095.
 - при использовании PC в операциях сложения или вычитания биты [1:0] счетчика команд округляются до 0b00 перед выполнением операции, обеспечивая выравнивание адреса по границе слова;
 - при необходимости сформировать адрес инструкции, необходимо скорректировать значение смещения относительно PC. ARM рекомендует использовать вместо этого инструкцию ADR, так как в этом случае ассемблер автоматически сгенерирует правильное смещение;
 - в случае если PC используется в качестве Rd в команде ADD{cond} PC, PC, Rmбит[0] значения, записываемого в PC, будет проигнорирован, передача управления будет осуществляться по адресу, соответствующему нулевому значению этого бита.

13.4.1.4 Флаги

В случае, если в команде указан суффикс S, процессор устанавливает флаги N, Z, C и V в соответствии с результатом выполнения операции.

13.4.1.5 Примеры

ADD R2, R1, R3

SUBS R8, R6, #240 – установить флаги по результату операции вычитания

RSB R4, R4, #1280 – вычесть содержимое регистра R4 из 1280

ADCHI R11, R0, R3 – операция выполняется только в случае, если флаг C установлен, а флаг Z сброшен

13.4.1.6 Арифметика с повышенной разрядностью

64-разрядное сложение

Следующий пример показывает, как осуществить сложение 64-разрядного целого числа, записанного в паре регистров R2 и R3, с другим 64-разрядным числом, записанным в паре регистров R0 и R1, после чего записывает результат в пару регистров R4 и R5.

ADDS R4, R0, R2 – сложить младшие значащие слова

ADC R5, R1, R3 – сложить старшие значащие слова с учетом переноса

96-разрядное вычитание

Данные с повышенной разрядностью не обязательно содержать в смежных регистрах. В примере, приведенном ниже, показан фрагмент кода, осуществляющий вычитание 96-разрядного целого числа, записанного в регистрах R9, R1 и R11, из другого числа, содержащегося в R6, R2 и R8. Результат записывается в регистрах R6, R9 и R2.

SUBS R6, R6, R9 – вычитание младших значащих слов;

SBCS R9, R2, R1 – вычитание средних значащих слов с переносом;

SBC R2, R8, R11 – вычитание старших значащих слов с переносом.

13.4.2 AND, ORR, EOR, BIC и ORN

Логические операции: И, ИЛИ, Исключающее ИЛИ, сброс бит по маске, ИЛИ-НЕ.

13.4.2.1 Синтаксис

op{S}{cond}{Rd,} Rn, Operand2

где op – один из кодов операции:

- AND – логическое И;
- ORR – логическое ИЛИ;
- EOR – логическое Исключающее ИЛИ;
- BIC – сброс бит по маске;
- ORN – логическое ИЛИ-НЕ;

S – необязательный суффикс. Если он указан, результат выполнения операции приводит к установке соответствующих флагов, см. пункт 13.2.7 «Условное исполнение»;

cond – необязательный суффикс условного исполнения, см. пункт 13.2.7 «Условное исполнение»;

Rd – регистр-получатель результата;

Rn – регистр, содержащий значение первого операнда;

Operand2 – второй операнд, см. пункт 13.2.3 «Формат второго операнда».

13.4.2.2 Описание

Инструкции AND, ORR и EOR осуществляют, соответственно, логические операции И, ИЛИ и исключающего ИЛИ между первым операндом, содержащимся в регистре Rn, и вторым операндом Operand2.

Инструкция BIC выполняет операцию логического И между первым операндом, содержащимся в регистре Rn, и инверсным значением второго операнда Operand2.

Инструкция ORN выполняет операцию логического ИЛИ между первым операндом Rn и инверсным значением второго операнда Operand2.

13.4.2.3 Ограничения

Не допускается использованием указателя стека SP и счетчика команд PC.

13.4.2.4 Флаги

В случае если в команде указан суффикс S, процессор:

- устанавливает флаги N и Z в соответствии с результатом выполнения операции;
- может изменить флаг C в ходе вычисления значения второго операнда, см. пункт 13.2.3 «Формат второго операнда»;
- не влияет на значение флага V.

13.4.2.5 Примеры

AND R9, R2, #0xFF00

ORREQ R2, R0, R5

ANDS R9, R8, #0x19

EORS R7, R11, #0x18181818

BIC R0, R1, #0xab

ORN R7, R11, R14, ROR #4

ORNS R7, R11, R14, ASR #32

13.4.3 ASR, LSL, LSR, ROR и RRX

Арифметический сдвиг вправо, логический сдвиг влево, логический сдвиг вправо, циклический сдвиг вправо и циклический сдвиг вправо с переносом.

13.4.3.1 Синтаксис

op{S}{cond} Rd, Rm, Rs

op{S}{cond} Rd, Rm, #n

RRX{S}{cond} Rd, Rm

где op – один из кодов операции:

- ASR – арифметический сдвиг вправо;
- LSL – логический сдвиг влево;
- LSR – логический сдвиг вправо;

– ROR – циклический сдвиг вправо;

S – необязательный суффикс. Если он указан, результат выполнения операции приводит к установке соответствующих флагов, см. пункт 13.2.7 «Условное исполнение»;

cond – необязательный суффикс условного исполнения, см. пункт 13.2.7 «Условное исполнение»;

Rd – регистр-получатель результата;

Rm – регистр, значение которого должно быть подвергнуто сдвигу;

Rs – регистр, содержащий параметр сдвига. Процессор анализирует только младший значащий байт регистра, таким образом, параметр сдвига может принимать значения от 0 до 255;

n – параметр сдвига. Диапазон допустимых значений параметра зависит от инструкции:

– ASR – от 1 до 32;

– LSL – от 0 до 31;

– LSR – от 1 до 32;

– ROR – от 1 до 31.

Команду LSL {S} {cond} Rd, Rm, #0

рекомендуется записывать в формате MOV {S} {cond} Rd, Rm.

13.4.3.2 Описание

Команда ASR, LSL, LSR и ROR сдвигает биты регистра Rm влево или вправо на заданное количество позиций, определяемое константой n или содержимым регистра Rs.

Команда RRX осуществляет сдвиг Rm вправо на одну позицию с учетом переноса.

Во всех указанных инструкциях результат записывается в регистр Rd, при этом содержание регистра Rm остается неизменным. Детальное описание операций сдвига приведено в пункте 13.2.4 «Операции сдвига».

13.4.3.3 Ограничения

Не допускается использованием указателя стека SP и счетчика команд PC.

13.4.3.4 Флаги

В случае если в команде указан суффикс S, процессор:

– устанавливает флаги N и Z в соответствии с результатом выполнения операции;

– флаг C устанавливается в значение последнего сдвинутого бита, за исключением случая параметра сдвига, равного нулю (см. пункт 13.2.4 «Операции сдвига»).

13.4.3.5 Примеры

ASR R7, R8, #9 – Арифметический сдвиг вправо на 9 бит

LSLS R1, R2, #3 – Логический сдвиг влево на 3 бита с установкой флагов

LSR R4, R5, #6 – Логический сдвиг вправо на 6 бит

ROR R4, R5, R6 – Циклический сдвиг вправо на количество бит, указанное в младшем байте регистра R6

RRX R4, R5 – Циклический сдвиг вправо через бит переноса.

13.4.4 CLZ

Определить количество ведущих нулей.

13.4.4.1 Синтаксис

CLZ{cond} Rd, Rm

где cond – необязательный суффикс условного исполнения, см. пункт 13.2.7 «Условное исполнение»;

Rd – регистр-получатель результата;

Rm – регистр операнда.

13.4.4.2 Описание

Инструкция CLZ выполняет подсчет количества ведущих нулей в двоичной записи значения, записанного в регистре Rm, и возвращает результат в регистр Rd. Результат, равный 32, возвращается в случае, если в регистре Rm нет установленных бит, а результат, равный 0 – в случае, если установлен бит [31].

13.4.4.3 Ограничения

Не допускается использование указателя стека SP и счетчика команд PC.

13.4.4.4 Флаги

Данная инструкция не влияет на состояние флагов.

13.4.4.5 Примеры

CLZ R4,R9

CLZNE R2,R3.

13.4.5 CMP и CMN

Сравнение и сравнение с противоположным знаком.

13.4.5.1 Синтаксис

CMP{cond} Rn, Operand2

CMN{cond} Rn, Operand2

где cond – необязательный суффикс условного исполнения, см. пункт 13.2.7 «Условное исполнение»;

Rm – регистр, содержащий первый операнд;

Operand2 – второй операнд, см. пункт 13.2.3 «Формат второго операнда».

13.4.5.2 Описание

Данные инструкции осуществляют сравнение значений регистра и второго операнда. По результатам сравнения устанавливаются соответствующие флаги, однако сам результат в регистр не записывается.

Команда CMP вычитает из регистра Rn значение второго операнда Operand2. Она аналогична инструкции SUBS, за исключением того, что не сохраняет результат вычитания.

Команда CMN складывает значения регистра Rn и второго операнда Operand2. Она аналогична инструкции ADDS, за исключением того, что не сохраняет результат вычитания.

13.4.5.3 Ограничения

В данных инструкциях:

- не допускается использованием PC;
- в качестве второго операнда Operand2 нельзя использовать SP.

13.4.5.4 Флаги

Процессор устанавливает флаги N, Z, C и V в соответствии с результатом сравнения.

13.4.5.5 Примеры

CMP R2, R9

CMN R0, #6400

CMPGT SP, R7, LSL #2

13.4.6 MOV и MVN

Загрузка в регистр прямого или инверсного значения второго операнда.

13.4.6.1 Синтаксис

MOV{S}{cond} Rd, Operand2

MOV{cond} Rd, #imm16

MVN{S}{cond} Rd, Operand2

где S – необязательный суффикс. Если он указан, результат выполнения операции приводит к установке соответствующих флагов, см. пункт 13.2.7 «Условное исполнение»;

cond – необязательный суффикс условного исполнения, см. пункт 13.2.7 «Условное исполнение».

Rd – регистр-получатель результата;

Operand2 – второй операнд. См. пункт 13.2.3 «Формат второго операнда»;

imm16 – любое значение в диапазоне от 0 до 65535.

13.4.6.2 Описание

Инструкция MOV копирует значение второго операнда Operand2 в регистр Rd. В случае если Operand2 является регистром с параметром сдвига, отличным от LSL #0, рекомендуется использовать вместо команды MOV соответствующую команду сдвига:

- ASR{S}{cond} Rd, Rm, #n вместо MOV{S}{cond} Rd, Rm, ASR #n;
- LSL{S}{cond} Rd, Rm, #n вместо MOV{S}{cond} Rd, Rm, LSL #n при n != 0;
- LSR{S}{cond} Rd, Rm, #n вместо MOV{S}{cond} Rd, Rm, LSR #n;
- ROR{S}{cond} Rd, Rm, #n вместо MOV{S}{cond} Rd, Rm, ROR #n;
- RRX{S}{cond} Rd, Rm вместо MOV{S}{cond} Rd, Rm, RRX.

Кроме того, допускается использовать дополнительные формы построения второго операнда Operand2 в инструкции MOV как синонимы соответствующих операций сдвига:

- MOV{S}{cond} Rd, Rm, ASR Rs является синонимом ASR{S}{cond} Rd, Rm, Rs;
- MOV{S}{cond} Rd, Rm, LSL Rs является синонимом LSL{S}{cond} Rd, Rm, Rs
- MOV{S}{cond} Rd, Rm, LSR Rs является синонимом LSR{S}{cond} Rd, Rm, Rs
- MOV{S}{cond} Rd, Rm, ROR Rs является синонимом ROR{S}{cond} Rd, Rm, Rs.

См. также описание инструкций ASR, LSL, LSR, ROR и RRX.

Инструкция MVN считывает значение второго операнда Operand2, производит его побитную инверсию, после чего помещает результат в регистр Rd.

Инструкция MOVW функционирует также, как и инструкция MOV, однако в качестве второго операнда в ней можно использовать только непосредственно задаваемое значение imm16.

13.4.6.3 Ограничения

Регистры SP и PC допускается использовать только с инструкцией MOV, при следующих ограничениях:

- второй операнд должен быть регистром без указания параметра сдвига;
- суффикс S не должен быть указан.

В случае если в качестве Rd используется счетчик команд PC:

- бит [0] значения, загружаемого в PC, игнорируется;
- передача управления осуществляется по адресу, соответствующему загруженному значению с битом [0], принудительно установленным в 0.

Несмотря на то, что существует возможность использовать инструкцию MOV в качестве команды ветвления, ARM настоятельно рекомендует использовать для этих целей исключительно инструкции BX и BLX, в интересах обеспечения переносимости программного обеспечения.

13.4.6.4 Флаги

В случае если в команде указан суффикс S, процессор:

- устанавливает флаги N и Z в соответствии с результатом выполнения операции;
- может изменить флаг C в ходе вычисления значения второго операнда, см. пункт 13.2.3 «Формат второго операнда»;
- не влияет на значение флага V.

13.4.6.5 Примеры

MOVS R11, #0x000B – Записать значение 0x000B в R11, флаги устанавливаются

MOV R1, #0xFA05 – Записать значение 0xFA05 в R1, флаги не устанавливаются

MOVS R10, R12 – Записать регистр R12 в R10, флаги устанавливаются

MOV R3, #23 – Записать значение 23 в R3

MOV R8, SP – Записать значение указателя стека в регистр R8

MVNS R2, #0xF – Записать значение 0xFFFFFFFF0 (инверсия значения 0x0F) в регистр R2, установить флаги.

13.4.7 MOVТ

Записать в старшее полуслово регистра.

13.4.7.1 Синтаксис

MOVТ{cond} Rd, #imm16

где cond – необязательный суффикс условного исполнения, см. пункт 13.2.7 «Условное исполнение»;

Rd – регистр-получатель результата;

imm16 – любое значение в диапазоне от 0 до 65535.

13.4.7.2 Описание

Инструкция MOVТ записывает 16-разрядное непосредственное значение imm16 в старшее полуслово регистра-приемника Rd[31:16]. Младшее полуслово Rd[15:0] остается неизменным.

Комбинация команд MOV и MOVТ позволяет загрузить в регистр любую 32-битную константу.

13.4.7.3 Ограничения

В качестве Rd нельзя использовать указатель стека SP и счетчик команд PC.

13.4.7.4 Флаги

Данная инструкция не влияет на состояние флагов.

13.4.7.5 Примеры

MOVТ R3, #0xF123 – Загрузить 0xF123 в старшее полуслово R3, младшее полуслово и регистр состояния APSR остаются неизменными

13.4.8 REV, REV16, REVSH и RBIT

Изменение порядка бит или байтов в слове.

13.4.8.1 Синтаксис

op{cond} Rd, Rn

где op – один из кодов операции:

REV – изменить на обратный порядок байтов в слове;

REV16 – изменить на обратный порядок байтов в полусловах;

REVSH – изменить на обратный порядок байт в младшем полуслове, произвести распространение знакового бита в старшее полуслово;

RBIT – изменить порядок бит в 32-разрядном слове;

cond – необязательный суффикс условного исполнения, см. пункт 13.2.7 «Условное исполнение»;

Rd – регистр-получатель результата;

Rn – регистр, содержащий операнд.

13.4.8.2 Описание

Инструкции предназначены для изменения формата представления (endianness) данных:

- REV – преобразует 32-разрядное число в формате big-endian в число в формате little-endian и наоборот.

- REV16 – преобразует 32-разрядное число в формате big-endian в число в формате little-endian и наоборот.

- REVSH – выполняет одно из следующих преобразований:

- 16-разрядное число со знаком в формате big-endian в 32-разрядное число со знаком в формате little-endian;

- 16-разрядное число со знаком в формате little-endian в 32-bit 32-разрядное число со знаком в формате big-endian.

- RBIT – изменяет на обратный порядок бит в 32-разрядном слове.

13.4.8.3 Ограничения

Нельзя использовать указатель стека SP и счетчик команд PC.

13.4.8.4 Флаги

Данная инструкция не влияет на состояние флагов.

13.4.8.5 Примеры

REV R3, R7 – Обратить порядок следования байтов в R7, записать в R3

REV16 R0, R0 – Обратить порядок байтов в каждом 16-битном полуслове R0

REVSH R0, R5 – Обратить полуслово со знаком

REVHS R3, R7 – Обратить порядок при условии "больше или равно" (HS)

RBIT R7, R8 – Обратить порядок бит в R8, записать результат в R7

13.4.9 TST и TEQ

Проверить значение бит по маске, проверить равенство.

13.4.9.1 Синтаксис

TST{cond} Rn, Operand2

TEQ{cond} Rn, Operand2

где cond – необязательный суффикс условного исполнения, см. пункт 13.2.7 «Условное исполнение».

Rn – регистр, содержащий первый операнд.

Operand2 – второй операнд. См. пункт 13.2.3 «Формат второго операнда».

13.4.9.2 Описание

Данные инструкции позволяют проверить значение регистра с учетом значения второго операнда Operand2. По результату устанавливаются флаги, сам результат не сохраняется.

Команда TST выполняет побитную операцию логического И между значениями Rn и Operand2. Она совпадает с инструкцией ANDS, за исключением того, что не сохраняет результат. Для того чтобы проверить, что заданный бит регистра Rn равен 0 или 1, рекомендуется использовать команду TST со вторым операндом Operand2 в виде константы, в которой соответствующий бит равен 1, а все остальные – равны 0.

Команда TEQ выполняет побитную операцию Иключающее ИЛИ между значениями Rn и Operand2. Она совпадает с инструкцией EORS, за исключением того, что не сохраняет результат. Команда TEQ позволяет проверить равенство двух величин, не меняя значения флагов V и C. Кроме того, эта инструкция полезна для проверки знака числа. После сравнения флаг N является результатом операции Иключающее ИЛИ знаковых разрядов двух операндов.

13.4.9.3 Ограничения

Нельзя использовать указатель стека SP и счетчик команд PC.

13.4.9.4 Флаги

В случае если в команде указан суффикс S, процессор:

- устанавливает флаги N и Z в соответствии с результатом выполнения операции;
- может изменить флаг C в ходе вычисления значения второго операнда, см. пункт 13.2.3 «Формат второго операнда»;
- не влияет на значение флага V.

13.4.9.5 Примеры

TST R0, #0x3F8 – Побитное И между R0 и числом 0x3F8, устанавливаются флаги, результат не сохраняется

TEQEQ R10, R9 – Условное исполнение проверки равенства регистров R10 и R9, устанавливаются флаги, результат не сохраняется.

13.5 Инструкции умножения и деления

В таблице 61 представлена информация о командах умножения и деления:

Таблица 61 – Инструкции умножения и деления

Мнемокод	Краткое описание
MLA	Умножение и сложение, 32-битный результат
MLS	Умножение и вычитание, 32-битный результат
MUL	Умножение, 32-разрядный результат
SDIV	Деление чисел со знаком
SMLAL	Умножение чисел со знаком с накоплением (32 x 32 + 64), 64-битный результат
SMULL	Умножение чисел со знаком, 64-битный результат
UDIV	Деление чисел без знака
UMLAL	Умножение чисел без знака с накоплением (32 x 32 + 64), 64-битный результат
UMULL	Умножение чисел без знака, 64-битный результат

13.5.1 MUL, MLA и MLS

Умножение или умножение с накоплением (сложением, вычитанием) с использованием 32-разрядных операндов и выдающее 32-разрядный результат.

13.5.1.1 Синтаксис

MUL{S}{cond} {Rd}, Rn, Rm – Умножение

MLA{cond} Rd, Rn, Rm, Ra – Умножение и сложение

MLS{cond} Rd, Rn, Rm, Ra – Умножение и вычитание

где S – необязательный суффикс. Если он указан, результат выполнения операции приводит к установке соответствующих флагов, см. пункт 13.2.7 «Условное исполнение».

cond – необязательный суффикс условного исполнения, см. пункт 13.2.7 «Условное исполнение».

Rd – регистр-получатель результата. Если регистр Rd не указан, то в качестве получателя используется регистр Rn.

Rn, Rm – регистры, содержащий значения первого и второго сомножителей.

Ra – регистр, содержащий значение, к которому должно быть прибавлено или вычтено произведение.

13.5.1.2 Описание

Команда MUL выполняет перемножение значений, содержащихся в регистрах Rn и Rm, после чего сохраняет 32 младших значащих бита произведения в Rd.

Команда MLA перемножает содержимое регистров Rn и Rm, прибавляет к произведению значение Ra, после чего сохраняет 32 младших значащих бита результата в Rd.

Команда MLS перемножает содержимое регистров Rn и Rm, вычитает произведение из регистра Ra, после чего сохраняет 32 младших значащих бита результата в Rd.

Результат выполнения операций не зависит от того, используются ли в качестве операндов числа со знаком или без знака.

13.5.1.3 Ограничения

Нельзя использовать указатель стека SP и счетчик команд PC.

В случае если инструкция MUL используется с суффиксом установки флагов S:

- регистры Rd, Rn и Rm должны находиться в диапазоне от R0 до R7;
- регистр Rd должен совпадать с Rm;
- не допускается использование суффикса условного исполнения cond.

13.5.1.4 Флаги

В случае если в команде указан суффикс S, процессор:

- устанавливает флаги N и Z в соответствии с результатом выполнения операции;
- не влияет на значение флагов C и V.

13.5.1.5 Примеры

MUL R10, R2, R5 – R10 = R2 x R5

MLA R10, R2, R1, R5 – $R10 = (R2 \times R1) + R5$
 MULS R0, R2, R2 – $R0 = R2 \times R2$, установить флаги
 MULLT R2, R3, R2 – условное исполнение $R2 = R3 \times R2$
 MLS R4, R5, R6, R7 – $R4 = R7 - (R5 \times R6)$

13.5.2 UMULL, UMLAL, SMULL и SMLAL

Умножение чисел со знаком или без знака, с возможностью накопления, 32-битные операнды, 64-битный результат.

13.5.2.1 Синтаксис

op{cond} RdLo, RdHi, Rn, Rm

где op – один из кодов операции:

UMULL – умножение чисел без знака;

UMLAL – умножение чисел без знака с накоплением;

SMULL – умножение чисел со знаком;

SMLAL – умножение чисел со знаком с накоплением;

cond – необязательный суффикс условного исполнения, см. пункт 13.2.7 «Условное исполнение»;

RdLo, RdHi – регистры-получатели младшей и старшей частей результата, соответственно. Для инструкций UMLAL и SMLAL они также содержат накапливаемое значение.

Rn, Rm – регистры, содержащие значения первого и второго сомножителей.

13.5.2.2 Описание

Инструкция UMULL перемножает значения регистров Rn и Rm, интерпретируя их как целые числа без знака. Результат умножения размещается в паре регистров RdHi (старшие 32 бита) и RdLo (младшие 32 бита).

Инструкция UMLAL перемножает значения регистров Rn и Rm, интерпретируя их как целые числа без знака. Результат прибавляется к 64-разрядному целому числу без знака, записанному в паре регистров RdHi и RdLo, после чего сохраняется обратно в паре регистров RdHi и RdLo.

Инструкция SMULL перемножает значения регистров Rn и Rm, интерпретируя их как целые числа со знаком, записанные в дополнительном коде. Результат умножения размещается в паре регистров RdHi (старшие 32 бита) и RdLo (младшие 32 бита).

Инструкция SMLAL перемножает значения регистров Rn и Rm, интерпретируя их как целые числа со знаком, записанные в дополнительном коде. Результат прибавляется к 64-разрядному целому числу со знаком, записанному в паре регистров RdHi и RdLo, после чего сохраняется обратно в паре регистров RdHi и RdLo.

13.5.2.3 Ограничения

Нельзя использовать указатель стека SP и счетчик команд PC.

Пара регистров RdHi и RdLo должна состоять из разных регистров.

13.5.2.4 Флаги

Данная инструкция не влияет на состояние флагов.

13.5.2.5 Примеры

UMULL R0, R4, R5, R6 – Беззнаковое умножение $(R4, R0) = R5 \times R6$

SMLAL R4, R5, R3, R8 – Операция со знаком $(R5, R4) = (R5, R4) + R3 \times R8$

13.5.3 SDIV и UDIV

Деление чисел со знаком или без знака.

13.5.3.1 Синтаксис

SDIV{cond} {Rd,} Rn, Rm

UDIV{cond} {Rd,} Rn, Rm

где cond – необязательный суффикс условного исполнения, см. пункт 13.2.7 «Условное исполнение»;

Rd – регистр-получатель результата. Если Rd не указан, результат сохраняется в Rn;

Rn – регистр, содержащий значение делимого;

Rm – регистр, содержащий значение делителя.

13.5.3.2 Описание

Команда SDIV осуществляет деление целого числа со знаком, содержащегося в регистре Rn, на целое число со знаком, содержащееся в регистре Rm.

Команда UDIV осуществляет деление целого числа без знака, содержащегося в регистре Rn, на целое число без знака, содержащееся в регистре Rm.

В случае если число в Rn не делится нацело на число в Rm, результат округляется в сторону 0.

13.5.3.3 Ограничения

Нельзя использовать указатель стека SP и счетчик команд PC.

13.5.3.4 Флаги

Данная инструкция не влияет на состояние флагов.

13.5.3.5 Примеры

SDIV R0, R2, R4 – Деление чисел со знаком, $R0 = R2/R4$

UDIV R8, R8, R1 – Деление чисел без знака, $R8 = R8/R1$.

13.6 Инструкции преобразования данных с насыщением

В разделе рассмотрены инструкции преобразования данных с насыщением SSAT и USAT.

13.6.1 SSAT и USAT

Преобразование 32-разрядного числа в n-разрядное со знаком или без знака с насыщением.

13.6.1.1 Синтаксис

op{cond} Rd, #n, Rm {, shift #s}

где op – один из кодов операции:

– SSAT – преобразует число со знаком в число со знаком, лежащее в заданном диапазоне значений, с насыщением;

– USAT – преобразует число со знаком в число без знака, лежащее в заданном диапазоне значений, с насыщением;

cond – необязательный суффикс условного исполнения, см. пункт 13.2.7 «Условное исполнение»;

Rd – регистр-получатель результата;

Rm – регистр, содержащий преобразуемое значение;

n – определяет разрядность данных после преобразования с насыщением:

– n находится в диапазоне от 1 до 32 для инструкции SSAT;

– n находится в диапазоне от 0 до 31 для инструкции USAT.

shift #s – необязательный параметр сдвига, применяемый к регистру Rm перед преобразованием с насыщением. Он может принимать следующие значения:

– ASR #s, где s находится в диапазоне от 1 до 31;

– LSL #s, где s находится в диапазоне от 0 до 31.

13.6.1.2 Описание

Инструкции преобразуют 32-разрядное число в n-разрядное число со знаком или без знака. Преобразование осуществляется с насыщением.

Команда SSAT подвергает операнд заданной операции сдвига, после чего приводит его к диапазону значений $-2^{(n-1)} \leq x \leq 2^{(n-1)}-1$ в соответствии со следующими правилами:

- если значение после сдвига меньше $-2^{(n-1)}$, сохраняется результат $-2^{(n-1)}$;
- если значение после сдвига больше $2^{(n-1)}-1$, сохраняется результат $2^{(n-1)}-1$;
- в противном случае, результат сохраняется без изменений.

Команда USAT подвергает операнд заданной операции сдвига, после чего приводит его к диапазону значений $0 \leq x \leq 2^n-1$ в соответствии со следующими правилами:

- если значение после сдвига меньше 0, сохраняется результат 0;
- если значение после сдвига больше 2^n-1 , сохраняется результат 2^n-1 ;
- в противном случае, результат сохраняется без изменений.

В случае если значение операнда после сдвига отличается от сохраненного результата, возникает ситуация, называемая насыщением, при этом процессор устанавливает в слове состояния приложения APSR флаг Q в 1. В случае если в ходе преобразования данных насыщения не возникло, флаг Q сохраняет свое прежнее значение.

Для того чтобы сбросить признак насыщения Q в 0, необходимо выполнить команду MSR, см. описание этой команды.

Проверить состояние флага Q можно с помощью команды MRS.

13.6.1.3 Ограничения

Нельзя использовать указатель стека SP и счетчик команд PC.

13.6.1.4 Флаги

Данная инструкция не влияет на состояние флагов, за исключением флага Q. Флаг Q устанавливается в 1 в случае, если при преобразовании данных произошло насыщение.

13.6.1.5 Примеры

SSAT R7, #16, R7, LSL #4 – Логический сдвиг R7 влево на 4 бита, далее приведение его к 16-разрядному числу со знаком с насыщением, сохранить результат в R7

USATNE R0, #7, R5 – Условная операция: преобразовать с насыщением значение R5 к семиразрядному числу без знака, сохранить результат в R0/

13.7 Команды работы с битовыми полями

В таблице 62 показаны инструкции, позволяющие манипулировать последовательностями смежных бит данных в регистрах или битовых полях.

Таблица 62 – Инструкции упаковки и распаковки данных

Мнемокод команд	Краткое описание
BFC	Запись нуля в битовое поле
BFI	Запись заданного значения битового поля
SBFX	Чтение значения битового поля, интерпретируемого как число со знаком
SXTB	Преобразовать байт со знаком в слово
SXTH	Преобразовать полуслово со знаком в слово
UBFX	Чтение значения битового поля, интерпретируемого как число без знака
UXTB	Преобразовать байт без знака в слово
UXTH	Преобразовать полуслово без знака в слово

13.7.1 BFC и BFI

Сброс в ноль и запись заданного значения битового поля.

13.7.1.1 Синтаксис

BFC{cond} Rd, #lsb, #width

BFI{cond} Rd, Rn, #lsb, #width

где cond – необязательный суффикс условного исполнения, см. “Условное исполнение”;

Rd – регистр-получатель результата;

Rm – регистр-источник данных;

lsb – позиция младшего значащего разряда битового поля. Значение lsb должно находиться в интервале от 0 до 31;

width – ширина битового поля, значение которой должно находиться в интервале от 1 до 32-lsb.

13.7.1.2 Описание

Инструкция BFC очищает битовое поле, размещенное в регистре Rd, имеющее длину width бит и расположенное, начиная с бита с номером lsb. Остальные биты регистра Rd сохраняются без изменений.

Инструкция BFI копирует битовое поле шириной в width бит, расположенное в регистре Rn, начиная с позиции 0, в битовое поле шириной в width бит, расположенное в регистре Rd, начиная с позиции lsb. Остальные биты регистра Rd сохраняются без изменений.

13.7.1.3 Ограничения

Нельзя использовать указатель стека SP и счетчик команд PC.

13.7.1.4 Флаги

Данная инструкция не влияет на состояние флагов.

13.7.1.5 Примеры

BFC R4, #8, #12 – Очистить 12-битовое поле, расположенное с 8 по 19 бит R4.

BFI R9, R2, #8, #12 – Записать в 12-битовое поле, расположенное с 8 по 19 бит R9 значение из 12-битового поля, расположенного с 0 по 11 бит регистра R2.

13.7.2 SBFX и UBFX

Чтение значения битового поля, интерпретируемого как число со знаком или без знака.

13.7.2.1 Синтаксис

SBFX{cond} Rd, Rn, #lsb, #width

UBFX{cond} Rd, Rn, #lsb, #width

где cond – необязательный суффикс условного исполнения, см. пункт 13.2.7 «Условное исполнение»;

Rd – регистр-получатель результата;

Rm – регистр-источник данных;

lsb – позиция младшего значащего разряда битового поля. Значение lsb должно находиться в интервале от 0 до 31;

width – ширина битового поля, значение которой должно находиться в интервале от 1 до 32-lsb.

13.7.2.2 Описание

Инструкция SBFX считывает значение битового поля из регистра-источника, производит распространение знакового бита в старшие биты 32-разрядного слова, сохраняет результат в регистр-получатель.

Инструкция UBFX считывает значение битового поля из регистра-источника, заполняет нулями старшие биты 32-разрядного слова, сохраняет результат в регистр-получатель.

13.7.2.3 Ограничения

Нельзя использовать указатель стека SP и счетчик команд PC.

13.7.2.4 Флаги

Данная инструкция не влияет на состояние флагов.

13.7.2.5 Примеры

SBFX R0, R1, #20, #4 – Извлечь 4 бита (с 20 по 23) из R1, интерпретируя их как число со знаком, записать в R0

UBFX R8, R11, #9, #10 – Извлечь 10 бит (с 9 по 18) из R11, интерпретируя их как число без знака, записать в R8.

13.7.3 SXT и UXT

Преобразование байта или полуслова в слово с распространением знакового бита или нулей в старшие значащие разряды.

13.7.3.1 Синтаксис

SXTextend{cond} {Rd}, Rm {, ROR #n}

UXTextend{cond} {Rd}, Rm {, ROR #n}

где суффикс *extend* может принимать одно из следующих значений:

- B – преобразование 8-битного числа в 32-битное;
- H – преобразование 16-битного числа в 32-битное;

cond – необязательный суффикс условного исполнения, см. пункт 13.2.7 «Условное исполнение»;

Rd – регистр-получатель результата;

Rm – регистр-источник данных;

ROR #n – параметр сдвига, который может принимать одно из значений:

- ROR #8 – значение в Rm циклически сдвигается вправо на 8 бит;
- ROR #16 – значение в Rm циклически сдвигается вправо на 16 бит;
- ROR #24 – значение в Rm циклически сдвигается вправо на 24 бит;

если параметр не указан, сдвиг не производится.

13.7.3.2 Описание

Команда SXTB осуществляет циклический сдвиг содержимого регистра Rm вправо на заданное число бит, извлекает из результата младшие восемь бит [7:0], преобразует их в 32-разрядное число со знаком путем копирования знакового разряда [7] в биты [31:8], сохраняет результат в регистре Rd.

Команда UXTB осуществляет циклический сдвиг содержимого регистра Rm вправо на заданное число бит, извлекает из результата младшие восемь бит [7:0], преобразует их в 32-разрядное число без знака путем копирования нуля в биты [31:8], сохраняет результат в регистре Rd.

Команда SXTH осуществляет циклический сдвиг содержимого регистра Rm вправо на заданное число бит, извлекает из результата младшие восемь бит [15:0],

преобразует их в 32-разрядное число со знаком путем копирования знакового разряда [15] в биты [31:16], сохраняет результат в регистре Rd.

Команда UXTH осуществляет циклический сдвиг содержимого регистра Rm вправо на заданное число бит, извлекает из результата младшие восемь бит [15:0], преобразует их в 32-разрядное число без знака путем копирования нуля в биты [31:16], сохраняет результат в регистре Rd.

13.7.3.3 Ограничения

Нельзя использовать указатель стека SP и счетчик команд PC.

13.7.3.4 Флаги

Данная инструкция не влияет на состояние флагов.

13.7.3.5 Примеры

SXTH R4, R6, ROR #16 – сдвинуть R6 вправо на 16 бит, извлечь из результата младшее полуслово, преобразовать в 32-разрядное число с распространением знака, записать в R4.

UXTB R3, R10 – извлечь младший байт из R10, преобразовать в 32-разрядное число, старшие байты заполнить нулями, записать результат в R3.

13.8 Инструкции передачи управления

В таблице 63 приведен список инструкций передачи управления.

Таблица 63 – Инструкции передачи управления

Мнемокод команды	Краткое описание
B	Переход
BL	Переход со связью
BLX	Косвенный переход со связью
BX	Косвенный переход
CBNZ	Сравнение с нулем и переход по неравенству
CBZ	Сравнение с нулем и переход по равенству
IT	Начало блока условно исполняемых инструкций
TBB	Табличный переход по индексу, смещения - байты
TBN	Табличный переход по индексу, смещения - полусллова

13.8.1 B, BL, BX и BLX

Команды ветвления.

13.8.1.1 Синтаксис

B{cond} label

BL{cond} label

BX{cond} Rm

BLX{cond} Rm

где B – переход по непосредственно заданному адресу;
 BL – переход со связью по непосредственно заданному адресу;
 BX – косвенный переход по адресу, заданному значением регистра;

BLX – косвенный переход со связью;

cond – необязательный код условия, см. пункт 13.2.7 «Условное исполнение»;

label – относительный адрес, см. пункт 13.3.5 «LDR, адресация относительно счетчика команд PC»;

Rm – регистр, содержащий адрес, на который необходимо передать управления. Бит [0] этого регистра должен быть установлен в 1, однако передача управления будет выполнена по адресу, соответствующему нулевому значению бита [0].

13.8.1.2 Описание

Все рассматриваемые в данном разделе инструкции осуществляют передачу управления на адрес, заданный меткой, либо содержащийся в регистре Rm. Кроме того:

- команды BL и BLX записывают адрес следующей инструкции в регистр связи LR (R14);

- команды BX и BLX формируют отказ (usage fault) в случае, если bit[0] регистра Rm равен 0.

Инструкция вида B cond label – это единственный тип команды, который может находиться за пределами ИТ-блока. Все остальные условно исполняемые инструкции передачи управления должны располагаться внутри ИТ-блока, а за пределами этого блока должны использоваться только в безусловной форме. Подробнее см. в пункте 13.8.3 «ИТ».

В таблице 64 представлен диапазон адресуемых переходов для различных команд ветвления. Для достижения максимального диапазона может потребоваться указать суффикс .W размера инструкции. Подробности см. в пункте 13.2.8 «Выбор размера кода инструкции».

Таблица 64 – Диапазон адресуемых переходов для команд ветвления

Инструкция	Диапазон адресации
B label	от -16 Мбайт до +16 Мбайт относительно текущей позиции
B cond label (вне ИТ-блока)	от -1 Мбайт до +1 Мбайт относительно текущей позиции
B cond label (внутри ИТ-блока)	от -16 Мбайт до +16 Мбайт относительно текущей позиции
BL{cond} label	от -16 Мбайт до +16 Мбайт относительно текущей позиции
BX{cond} Rm	любое значение, записанное в регистре
BLX{cond} Rm	любое значение, записанное в регистре

13.8.1.3 Ограничения

в команде BLX не допускается использование регистра PC;

в командах BX и BLX, бит [0] регистра Rm должен быть установлен в 1, при этом передача управления будет, тем не менее, осуществлена по адресу, соответствующему нулевому значению бита [0];

внутри ИТ-блока любая из инструкций ветвления должна располагаться последней.

B cond – единственная условно исполняемая команда, которую допустимо использовать за пределами ИТ-блока. Тем не менее, внутри ИТ-блока она обеспечивает более широкий диапазон адресуемых переходов.

13.8.1.4 Флаги

Данная инструкция не влияет на состояние флагов.

13.8.1.5 Примеры

B loopA – передача управления на метку loopA

BLE ng – условная передача управления на метку ng

B.W target – переход на метку target, расположенную в пределах ± 16 Мбайт

BEQ target – условный переход на метку target

BEQ.W target – условный переход на метку target в пределах ± 1 Мбайт

BL funC – переход со связью (вызов функции) funC, адрес возврата будет записан в регистре LR

BX LR – возврат из функции

BXNE R0 – условный переход по адресу, записанному в R0

BLX R0 – переход со связью (вызов функции) по адресу, записанному в R0.

13.8.2 CBZ и CBNZ

Сравнение и условная передача управления, по равенству или неравенству 0.

13.8.2.1 Синтаксис

CBZ Rn, label

CBNZ Rn, label

где Rn – регистр, содержащий операнд.

label – метка, на которую должен быть осуществлен переход.

13.8.2.2 Описание

Инструкции CBZ и CBNZ позволяют осуществить проверку на равенство нулю с условным переходом, при этом, не влияя на значения флагов и снижая общее количество инструкций.

Команда CBZ Rn, label не влияет на флаги, а в остальном эквивалентна следующей последовательности инструкций:

- CMP Rn, #0;
- BEQ label.

Команда CBNZ Rn, label не влияет на флаги, а в остальном эквивалентна следующей последовательности инструкций:

- CMP Rn, #0;
- BNE label.

13.8.2.3 Ограничения

- в качестве Rn допустимо использовать регистры с R0 по R7;
- адрес перехода должен быть расположен после инструкции на расстоянии от 4 до 130 байт;
- данные команды нельзя использовать внутри IT-блока.

13.8.2.4 Флаги

Данная инструкция не влияет на состояние флагов.

13.8.2.5 Примеры

CBZ R5, target – Условный переход вперед при R5 = 0
 CBNZ R0, target – Условный переход вперед при R0 != 0.

13.8.3 IT

Начало блока условно исполняемых инструкций.

13.8.3.1 Синтаксис

IT{x{y{z}}}} cond

где x – определяет выбор условия для второй инструкции в IT-блоке;
 y – определяет выбор условия для третьей инструкции в IT-блоке;
 z – определяет выбор условия для четвертой инструкции в IT-блоке;
 cond – определяет условие для первой инструкции в IT-блоке.

Суффиксы выбора условия для второй, третьей и четвертой инструкций IT-блока могут принимать одно из следующих значений:

T – Then. Инструкция выполняется, если условие cond истинно;

E – Else. Инструкция выполняется, если условие cond ложно.

Существует возможность использовать в IT-блоке на месте cond условие AL (всегда истинное). В этом случае все инструкции в IT-блоке должны быть безусловными, а суффиксы выбора условия x, y и z должны быть равны T, либо опущены.

13.8.3.2 Описание

Команда IT делает условными до четырех следующих за ней инструкций. Условия могут либо совпадать, либо быть логически противоположными. Условные инструкции, следующие за командой IT, формируют IT-блок.

Мнемокоды команд внутри IT-блока, в том числе и команд ветвления, должны включать в себя суффикс условного исполнения {cond}.

Ассемблеры некоторых производителей способны автоматически генерировать необходимые инструкции IT, предшествующие условно исполняемым командам, избавляя разработчика от необходимости делать эту работу вручную. Подробности следует уточнить в документации на Ваш ассемблер.

Команда ВКРТ внутри IT-блока всегда выполняется, вне зависимости от истинности или ложности условия.

Обработка исключений внутри IT-блока, а также непосредственно после инструкции IT допускается. При этом осуществляется переход на соответствующий обработчик с предварительным сохранением регистра PSR в стеке и необходимой для корректного возврата информации в регистре LR. Возврат из обработчика осуществляется стандартным образом, при этом корректное выполнение IT-блока продолжается с прерванной позиции. Это единственный допустимый способ передачи управления внутрь IT-блока с помощью команд, модифицирующих счетчик команд PC.

13.8.3.3 Ограничения

Следующие инструкции нельзя использовать внутри IT-блока:

– IT, CBZ и CBNZ, CPSI D и CPSI E.

Кроме того, существуют следующие ограничения при использовании IT-блоков:

- ветвление, а также любая другая команда, модифицирующая счетчик команд PC, должны передавать управление либо за пределы IT-блока, либо на последнюю инструкцию IT-блока

Инструкции, модифицирующие счетчик команд:

- ADD PC, PC, Rm;
- MOV PC, Rm;
- B, BL, BX, BLX;
- любая инструкция LDM, LDR или POP, приводящая к записи значения в PC;
- TBB and TBH.
- не допускается передача управления на инструкцию внутри IT-блока, за исключением случая возврата из обработчика исключения;
- все условные инструкции, за исключением B cond, должны находиться внутри IT-блока. Команда B cond может быть расположена как внутри, так и вне IT-блока, однако внутри IT-блока она обеспечивает более широкий диапазон адресуемых переходов;
- каждая инструкция внутри IT-блока должна быть снабжена суффиксом условного исполнения с кодом, либо совпадающим, либо противоположным коду условия IT-блока.

Ассемблер конкретного производителя может накладывать дополнительные ограничения, например, возможен запрет на использование директив внутри IT-блока.

13.8.3.4 Флаги

Данная инструкция не влияет на состояние флагов.

13.8.3.5 Примеры

ITTE NE – Следующие три инструкции - условные

ANDNE R0, R0, R1 – ANDNE не изменяет состояние флагов

ADDSNE R2, R2, #1 – ADDSNE изменяет состояние флагов

MOVEQ R2, R3 – условное копирование

CMP R0, #9 – преобразование R0 (от 0 до 15) в код ASCII шестнадцатеричного числа ('0'-'9', 'A'-'F')

ITE GT – следующие две инструкции - условные

ADDGT R1, R0, #55 – [R0 > 9] преобразуем число 0xA -> в код 'A'

ADDLE R1, R0, #48 – [R0 <= 9] преобразуем число 0x0 -> в код '0'

IT GT – IT-блок с одной единственной условной инструкцией

ADDGT R1, R1, #1 – условное увеличение R1 на единицу

ITTEE EQ – следующие четыре инструкции - условные

MOVEQ R0, R1 – условное копирование

ADDEQ R2, R2, #10 – условное сложение

ANDNE R3, R3, #1 – условное логическое И

- | | |
|---------------|--|
| BNE.W dloop | – условный переход. должен быть последним в блоке |
| IT NE | – следующая инструкция - условная |
| ADD R0, R0,R1 | – синтаксическая ошибка: не указан код условия в IT-блоке. |

13.8.4 TBB и TBH

Табличный переход по индексу.

13.8.4.1 Синтаксис

TBB [Rn, Rm]

TBH [Rn, Rm, LSL #1]

где Rn – регистр, содержащий адрес таблицы длин переходов. Если в качестве регистра Rn используется PC, то первый байт таблицы переходов следует непосредственно после инструкции TBB или TBH;

Rm – регистр, содержащий индекс в таблице переходов. Для таблиц, содержащих полуслова, добавляется операция сдвига LSL #1, что обеспечивает корректную адресацию по смещению в таблице.

13.8.4.2 Описание

Данные инструкции позволяют выполнить переход вперед относительно текущего значения счетчика команд PC на заданное смещение, выбранное из таблицы смещений, имеющих размер байта (для команды TBB) или полуслова (для команды TBH).

Регистр Rn содержит указатель на начало таблицы, а регистр Rm - индекс требуемого элемента.

Для команды TBB смещение вычисляется путем умножения на два значения байта из заданной ячейки таблицы, интерпретируемого как целое число без знака.

Для команды TBH смещение вычисляется путем умножения на два значения полуслова из заданной ячейки таблицы, интерпретируемого как целое число без знака.

Передача управления по соответствующему смещению осуществляется немедленно после выполнения инструкции TBB или TBH.

13.8.4.3 Ограничения

- в качестве регистра Rn нельзя использовать SP;
- в качестве регистра Rm нельзя использовать SP и PC;
- при использовании инструкций TBB или TBH внутри IT-блока они должны быть последней командой блока.

13.8.4.4 Флаги

Данная инструкция не влияет на состояние флагов.

13.8.4.5 Примеры

ADR.W R0, BranchTable_Byte

TBB [R0, R1] – R1 – индекс, R0 - базовый адрес таблицы переходов

Case1 – код для варианта R1 = 0

Case2 – код для варианта R1 = 1

Case3 – код для варианта R1 = 2

BranchTable_Byte

DCB 0 – смещение для Case1

DCB ((Case2-Case1)/2) – смещение для Case2

DCB ((Case3-Case1)/2) – смещение для Case3

TBH [PC, R1, LSL #1] – R1 – индекс, таблица переходов расположена непосредственно после команды TBH

BranchTable_H

DCI ((CaseA - BranchTable_H)/2) – смещение для CaseA

DCI ((CaseB - BranchTable_H)/2) – смещение для CaseB

DCI ((CaseC - BranchTable_H)/2) – смещение для CaseC

CaseA – код для CaseA

CaseB – код для CaseB

CaseC – код для CaseC

13.9 Прочие инструкции

В таблице 65 представлен список инструкций процессора Cortex-M3, не рассмотренных в предыдущих разделах.

Таблица 65 – Прочие инструкции

Мнемокод	Краткое описание
BKPT	Точка останова
CPSID	Изменить состояние процессора, запретить прерывания
CPSIE	Изменить состояние процессора, разрешить прерывания
DMB	Барьер синхронизации доступа к памяти данных
DSB	Барьер синхронизации доступа к памяти данных
ISB	Барьер синхронизации доступа к инструкциям
MRS	Загрузка из специального регистра в регистр общего назначения
MSR	Записать регистр общего назначения в специальный регистр
NOP	Нет операции
SEV	Установить признак события
SVC	Вызов супервизора
WFE	Ожидать событие
WFI	Ожидать прерывание

13.9.1 CPS

Изменить состояние процессора.

13.9.1.1 Синтаксис

CPSeffect iflags

где effect – один из возможных суффиксов:

- IE – сбрасывает специальный регистр в 0;
- ID – устанавливает специальный регистр в 1.

Iflags – последовательность флагов:

- I – сбрасывает или устанавливает регистр PRIMASK;
- F – сбрасывает или устанавливает регистр FAULTMASK.

13.9.1.2 Описание

Команда CPS позволяет изменить значение специальных регистров PRIMASK и FAULTMASK. Подробнее см. пункт 12.3.9 «Регистр маски исключений Exception mask».

13.9.1.3 Ограничения

- команда CPS доступна только из привилегированного приложения, при вызове из непривилегированного приложения она игнорируется;
- команда CPS не допускает условного исполнения и, таким образом, не должна использоваться внутри IT-блока.

13.9.1.4 Флаги

Данная инструкция не влияет на состояние флагов.

13.9.1.5 Примеры

- CPSID i – Запретить прерывания и конфигурируемые обработчики отказов
- CPSID f – Запретить прерывания и все обработчики отказов
- CPSIE i – Разрешить прерывания и конфигурируемые обработчики отказов
- CPSIE f – Разрешить прерывания и все обработчики отказов.

13.9.2 DMB

Барьер синхронизации доступа к памяти данных.

13.9.2.1 Синтаксис

DMB{cond}

где cond – необязательный код условия, см. пункт 13.2.7 «Условное исполнение».

13.9.2.2 Описание

Команда DMB выполняет функцию барьерной синхронизации доступа к памяти данных. Она гарантирует, что все явные операции доступа к памяти, которые были инициированы перед выполнением инструкции DMB, будут завершены до того, как начнется выполнение любой операции доступа к памяти после этой инструкции.

Команда DMB не влияет на очередность и порядок выполнения инструкций, не выполняющих доступа к памяти.

13.9.2.3 Флаги

Данная инструкция не влияет на состояние флагов.

13.9.2.4 Примеры

DMB – Барьер синхронизации доступа к памяти данных.

13.9.3 DSB

Барьер синхронизации доступа к памяти данных.

13.9.3.1 Синтаксис

DSB{cond}

Где cond – необязательный код условия, см. пункт 13.2.7 «Условное исполнение».

13.9.3.2 Описание

Инструкция DSB выполняет функцию барьерной синхронизации доступа к памяти данных. Команды, которые будут следовать в порядке выполнения после DSB, не начнут исполняться до ее завершения. Инструкция DSB завершает свою работу после того, как будут выполнены все инициированные перед ней явные операции доступа к памяти.

13.9.3.3 Флаги

Данная инструкция не влияет на состояние флагов.

13.9.3.4 Примеры

DSB – Data Synchronization Barrier.

13.9.4 ISB

Барьер синхронизации доступа к инструкциям.

13.9.4.1 Синтаксис

ISB{cond}

где cond – необязательный код условия, см. пункт 13.2.7 «Условное исполнение».

13.9.4.2 Описание

Команда ISB выполняет функцию барьерной синхронизации выполнения команд. Она осуществляет сброс конвейера инструкций процессора, гарантируя таким образом, что все команды, расположенные после инструкции ISB, по окончании ее исполнения будут загружены в конвейер повторно.

13.9.4.3 Флаги

Данная инструкция не влияет на состояние флагов.

13.9.4.4 Примеры

ISB – Барьер синхронизации доступа к инструкциям.

13.9.5 MRS

Считать содержимое специального регистра в регистр общего назначения.

13.9.5.1 Синтаксис

MRS{cond} Rd, spec_reg

где cond – необязательный код условия, см. пункт 13.2.7 «Условное исполнение»;
Rd – регистр-получатель результата;

спес_reg – один из специальных регистров: APSR, IPSR, EPSR, IEPSR, IAPSR, EAPSR, PSR, MSP, PSP, PRIMASK, BASEPRI, BASEPRI_MAX, FAULTMASK или CONTROL.

13.9.5.2 Описание

Инструкции MRS совместно с MSR используются для чтения-модификации-записи элементов PSR, например, для сброса флага Q.

В коде, отвечающем за переключение процессов, необходимо обеспечить сохранение состояния приостановленного процесса, и восстановление состояния активизированного процесса. Необходимой составной частью сохраняемой (восстанавливаемой) информации является значение регистра PSR. При этом на этапе сохранения состояния используется команда MRS, а на этапе восстановления – команда MSR.

При использовании команды MRS регистр BASEPRI_MAX является синонимом регистра BASEPRI.

См. также описание инструкции MSR.

13.9.5.3 Ограничения

В качестве регистра-получателя Rd нельзя использовать SP или PC.

13.9.5.4 Флаги

Данная инструкция не влияет на состояние флагов.

13.9.5.5 Примеры

MRS R0, PRIMASK – Считать значение PRIMASK и записать значение в R0.

13.9.6 MSR

Записать регистр общего назначения в специальный регистр.

13.9.6.1 Синтаксис

MSR{cond} espec_reg, Rn

Где cond – необязательный код условия, см. пункт 13.2.7 «Условное исполнение»;

Rn – регистр-источник данных;

спес_reg – один из специальных регистров: APSR, IPSR, EPSR, IEPSR, IAPSR, EAPSR, PSR, MSP, PSP, PRIMASK, BASEPRI, BASEPRI_MAX, FAULTMASK или CONTROL.

13.9.6.2 Описание

Доступ к специальным регистрам в команде MSR различен для привилегированных и непривилегированных приложений. Непривилегированному приложению доступен только регистр APSR (см. пункт 12.3.6 «Программный регистр состояния приложения APSR»). При этом попытки записи в нераспределенные биты, а также в EPSR игнорируются.

Привилегированное приложение имеет доступ ко всем специальным регистрам.

При записи данных в регистр BASEPRI_MAX инструкция записывает данные в регистр BASEPRI только при выполнении одного из условий:

- Rn не равен нулю и текущее значение BASEPRI равно 0;
- Rn не равен нулю и меньше текущего значения BASEPRI.

См. также описание инструкции MRS.

13.9.6.3 Ограничения

В качестве регистра-источника данных Rn нельзя использовать SP или PC.

13.9.6.4 Флаги

Данная инструкция не влияет на состояние флагов.

13.9.6.5 Примеры

MSR CONTROL, R1 – Записать значение регистра R1 в регистр CONTROL

13.9.7 NOP

Нет операции.

13.9.7.1 Синтаксис

NOP{cond}

где cond – необязательный код условия, см. пункт 13.2.7 «Условное исполнение».

13.9.7.2 Описание

Инструкция NOP ничего не делает. В частности, эта инструкция в некоторых случаях может быть автоматически исключена из конвейера команд, и таким образом, выполнена за ноль тактов. Команду NOP рекомендуется использовать для заполнения, например, с целью разместить очередную инструкцию по адресу, выровненному по 64-битной границе.

13.9.7.3 Флаги

Данная инструкция не влияет на состояние флагов.

13.9.7.4 Примеры

NOP – нет операции.

13.9.8 SEV

Установить признак события.

13.9.8.1 Синтаксис

SEV{cond}

где cond – необязательный код условия, см. пункт 13.2.7 «Условное исполнение».

13.9.8.2 Описание

Инструкция SEV используется для передачи информации о событии всем процессорам в составе многопроцессорной системы. Кроме того, он устанавливает собственный регистр события в 1.

См. также подраздел 31.8 «Управление электропитанием».

13.9.8.3 Флаги

Данная инструкция не влияет на состояние флагов.

13.9.8.4 Примеры

SEV – Послать признак события.

13.9.9 SVC

Вызов супервизора.

13.9.9.1 Синтаксис

SVC{cond} #imm

где cond – необязательный код условия, см. пункт 13.2.7 «Условное исполнение»;
imm – константное выражение, целое число в диапазоне от 0 до 255 (8-битное число).

13.9.9.2 Описание

Инструкция SVC вызывает формирование исключения SVC. Параметр imm игнорируется процессором. При необходимости он может быть получен обработчиком исключения для определения запрошенного приложением варианта обслуживания.

13.9.9.3 Флаги

Данная инструкция не влияет на состояние флагов.

13.9.9.4 Примеры

SVC 0x32 – Вызов супервизора (обработчик SVC может извлечь параметр по сохраненному в стеке, адресу PC приложения).

13.9.10 WFE

Ожидать событие.

13.9.10.1 Синтаксис

WFE{cond}

где cond – необязательный код условия, см. пункт 13.2.7 «Условное исполнение».

13.9.10.2 Описание

В случае если регистр события равен 0, выполнение команды WFE приводит к приостановке исполнения команд до тех пор, пока не произойдет одно из следующих событий:

- исключение, не запрещенное путем установки маски или текущим уровнем приоритета;
- перевод исключения в состояние ожидания обслуживания при установленном в 1 бите SEVONPEND регистра управления системой SCR;
- получение запроса на переход в режим отладки, в случае, если отладка разрешена;

– получение сигнала о событии от периферийного устройства или от другого процессора (по команде SEV) в многопроцессорной системе.

В случае если регистр события равен 1, команда WFE сбрасывает его в 0, после чего завершает свое функционирование без приостановки процессора.

Подробнее см. подраздел 31.8 «Управление электропитанием».

13.9.10.3 Флаги

Данная инструкция не влияет на состояние флагов.

13.9.10.4 Примеры

WFE – Ожидание события.

13.9.11 WFI

Ожидание прерывание.

13.9.11.1 Синтаксис

WFI{cond}

где cond – необязательный код условия, см. пункт 13.2.7 «Условное исполнение».

13.9.11.2 Описание

Команда WFI приостанавливает процессор до тех пор, пока не произойдет одно из следующих событий:

- исключение;
- запрос на перевод в режим отладки, вне зависимости от того, разрешен или запрещен этот режим.

13.9.11.3 Флаги

Данная инструкция не влияет на состояние флагов.

13.9.11.4 Примеры

WFI – Ожидание прерывания.

14 Системный таймер SysTick

Процессор имеет 24-разрядный системный таймер, SysTick, который считает вниз от загруженного в него значения до нуля; перезагрузка (возврат в начало) значения в регистр LOAD происходит по следующему фронту синхросигнала, затем счет продолжается по последующему фронту.

Когда процессор остановлен для отладки, таймер не декрементируется.

14.1 Описание регистров системного таймера SysTick

Таблица 66 – Описание регистров системного таймера SysTick

Адрес	Название	Тип	Доступ	Значение после сброса	Описание
0xE000E010	SysTick				Системный таймер SYSTICK
0xE000E010	CTRL	RW	привилегированный	0x00000004	SysTick->CTRL
0xE000E014	LOAD	RW	привилегированный	0x00000000	SysTick->LOAD
0xE000E018	VAL	RW	привилегированный	0x00000000	SysTick->VAL
0xE000E01C	CALIB	RO	привилегированный	0x00000000	SysTick->CAL

14.1.1 SysTick->CTRL

Регистр CTRL разрешает основные функции системного таймера.

Таблица 67 – Регистр контроля и статуса CTRL

Номер	31...17	16	15...3	2	1	0
Доступ						
Сброс						
	-	COUNTFLAG	-	CLKSOURCE	TICKINT	ENABLE

14.1.1.1 COUNTFLAG

Возвращает 1, если таймер досчитал до 0 с последнего момента чтения.

14.1.1.2 CLKSOURCE

Указывает источник синхросигнала:

0 – LSI;

1 – HCLK.

14.1.1.3 TCKINT

Разрешает запрос на прерывание от системного таймера:

0 – таймер досчитает до нуля и прерывание не возникнет;

1 – таймер досчитывает до нуля и возникает запрос на прерывание.

Программное обеспечение может использовать бит COUNTFLAG, чтобы определить, досчитал таймер до 0 или нет.

14.1.1.4 ENABLE

Разрешает работу таймера:

0 – работа таймера запрещена;

1 – работа таймера разрешена.

Когда ENABLE установлен в 1, таймер загружает значение RELOAD из регистра LOAD и затем начинает декрементироваться. По достижению значения 0 таймер устанавливает бит COUNTFLAG и в зависимости от TCKINT генерирует запрос на прерывание. Затем загружается значение RELOAD и продолжается счет.

14.1.2 SysTick->LOAD

Регистр LOAD устанавливает стартовое значение, загружаемое в регистр VAL.

Таблица 68 – Регистр перегружаемого значения LOAD

Номер	31...24	23...0
Доступ		
Сброс		
	-	RELOAD

14.1.2.1 RELOAD

Значение, загружаемое в регистр VAL, когда таймер разрешен и когда достигается значение нуля.

Расчет значения RELOAD

Значение RELOAD может быть любым в диапазоне 0x00000001-0x00FFFFFF. Значение 0 допустимо, но не оказывает эффекта, потому что запрос на прерывание и активизация бита COUNTFLAG происходит только при переходе таймера из состояния 1 в 0.

Расчет значения RELOAD происходит в соответствии с использованием таймера:

- для формирования мультикороткого таймера с периодом N процессорных циклов применяется значение RELOAD, равное N-1. Например, если требуется прерывание каждые 100 циклов, то устанавливается значение RELOAD, равное 99;
- для формирования одиночного прерывания после задержки в N тактов процессора используется значение N. Например, если требуется прерывание после 400 тактов процессора, то устанавливается RELOAD, равное 400.

14.1.3 SysTick->VAL

Регистр VAL содержит текущее значение системного таймера.

Таблица 69 – Регистр текущего значения таймера VAL

Номер	31...24	23...0
Доступ		
Сброс		
	-	CURRENT

14.1.3.1 CURRENT

Чтение возвращает текущее значение системного таймера.

Запись любого значения очищает регистр в 0, и также очищает бит COUNTFLAG регистра CTRL.

14.1.4 SysTick->CAL

Регистр CALIB показывает калибровочное значение системного таймера. Калибровка системного таймера не реализована.

Таблица 70 – Регистр калибровочного значения таймера CAL

Номер	31	30	29...24	23...0
Доступ				
Сброс				
	NOREF	SKEW	-	TENMS

14.1.4.1 NOREF

Читается как 0.

14.1.4.2 SKEW

Читается как 0.

14.1.4.3 TENMS

Читается как 0.

14.2 Советы и особенности при применении системного таймера

Системный таймер работает от процессорного синхросигнала. Если синхросигнал останавливается в режиме пониженного энергопотребления, то системный таймер останавливается.

Гарантируйте, чтобы программа использовала доступ к регистрам системного таймера, выровненный по словам.

15 Модуль защиты памяти MPU

Модуль защиты памяти MPU делит карту памяти на регионы, и определяет положение, размер, разрешение на доступ и атрибуты памяти для каждого из них. Поддерживается:

- независимая установка атрибута для каждого региона;
- наложение (перекрытие) регионов;
- экспортирование атрибутов памяти в систему.

Атрибуты памяти влияют на доступ к памяти в регионе. Cortex-M3 MPU определяет:

- восемь независимых регионов, 0-7;
- фоновый регион.

Если регионы памяти перекрываются, на доступ к памяти влияют атрибуты региона с большим номером. Например, атрибуты региона 7 получают первенство над атрибутами любых других регионов, перекрывающихся с 7.

Фоновый регион имеет такие же атрибуты доступа к памяти, как и default карта памяти, но доступен только через привилегированные инструкции программы.

Карта памяти Cortex-M3 унифицированная. Это означает, что атрибуты доступа к инструкциям и данным одинаковые.

Если происходит программный запрос в запрещенную область памяти MPU, процессор генерирует ошибку управления памятью. Это вызывает прерывание по ошибке и может вызвать прерывание процессов в переменном окружении OS.

В переменном окружении OS ядро может обновлять настройки региона MPU динамически, основываясь на выполняемых процессах. Обычно встроенные OS используют MPU для защиты памяти.

Конфигурация регионов MPU основывается на типе памяти, см. подраздел 9.10 «Регионы памяти, типы и атрибуты».

В таблице 71 приведены возможные атрибуты регионов MPU. Здесь включены такие атрибуты памяти, как *shareable* и кэшируемость, которые не существенны во многих реализациях микросхем.

Таблица 71 – Обзор атрибутов памяти

Тип памяти	Атрибут <i>shareable</i>	Другие атрибуты	Описание
Строго упорядоченная	-	-	Весь доступ к строго упорядоченной памяти осуществляется под программным управлением. Все строго упорядоченные регионы могут быть общими
Устройство	Общая	-	Общая периферийная память для нескольких процессоров
	Не общая	-	Периферийная память только для одного процессора
Обычная	Общая		Обычная общая память для нескольких процессоров
	Не общая		Обычная память только для одного процессора

15.1 Описание регистров MPU

Применяются следующие регистры MPU для определения регионов и их атрибутов.

Таблица 72 – Обзор регистров MPU

Адрес	Обозначение	Тип	Доступ	Значение после сброса	Описание
0xE000ED90	MPU				Модуль защиты памяти MPU
0x000	TYPE	RO	привилегированный	0x00000800	MPU->TYPE
0x004	CTRL	RW	привилегированный	0x00000000	MPU->CTRL
0x008	RNR	RW	привилегированный	0x00000000	MPU->RNR
0x00C	RBAR	RW	привилегированный	0x00000000	MPU->RBAR
0x010	RASR	RW	привилегированный	0x00000000	MPU->RASR
0x014	RBAR_A1	RW	привилегированный	0x00000000	Обозначение RBAR
0x018	RASR_A1	RW	привилегированный	0x00000000	Обозначение RASR
0x01C	RBAR_A2	RW	привилегированный	0x00000000	Обозначение RBAR
0x020	RASR_A2	RW	привилегированный	0x00000000	Обозначение RASR
0x024	RBAR_A3	RW	привилегированный	0x00000000	Обозначение RBAR
0x028	RASR_A3	RW	привилегированный	0x00000000	Обозначение RASR

15.1.1 MPU->TYPE

Регистр TYPE показывает, присутствует или нет MPU, и как много регионов поддерживается.

Таблица 73 – Регистр TYPE

Номер	31...24	23...16	15...8	7...1	0
Доступ					
Сброс					
	-	IREGION	DREGION	-	SEPARATE

15.1.1.1 IREGION

Указывает количество поддерживаемых MPU регионов инструкций.

Всегда содержит 0x00. Карта памяти MPU унифицированная и описывается полем DREGION.

15.1.1.2 DREGION

Указывает количество поддерживаемых MPU регионов данных.

0x08 – Восемь MPU регионов.

15.1.1.3 SEPARATE

Указывает, поддерживается унифицированная или отдельная карта памяти для инструкций и данных:

0 – унифицированная.

15.1.2 MPU->CTRL

Регистр CTRL:

- разрешает MPU;
- разрешает default карту памяти как фоновый регион;
- разрешает применение MPU, при возникновении аппаратной ошибки, немаскируемое прерывание (NMI), FAULTMASK вызываемый обработчик.

Таблица 74 – Регистр CTRL

Номер	31...4	3	2	1	0
Доступ					
Сброс					
	-	PRIVDEFENA		HFNMIENA	ENABLE

15.1.2.1 PRIVDEFENA

Разрешает привилегированный программный доступ к default карте памяти:

0 – если MPU разрешен, запрещение применяется к default карте памяти. Любой доступ к памяти, не покрываемой разрешенным регионом, вызывает ошибку;

1 – если MPU разрешен, разрешает применение default карты памяти как фонового региона для привилегированного программного доступа.

Когда разрешено, фоновый регион считается как номер региона -1. Любой регион, который определен и разрешен, имеет приоритет выше этой default памяти.

Если MPU запрещен, то процессор игнорирует этот бит.

15.1.2.2 HFNMIENA

Разрешает операции MPU во время возникновения аппаратной ошибки, NMI, и FAULTMASK обработчик.

Если MPU разрешен:

0 – MPU запрещен во время возникновения аппаратной ошибки, NMI, FAULTMASK обработчик, несмотря на значения бита ENABLE;

1 – MPU разрешен во время возникновения аппаратной ошибки, NMI, FAULTMASK обработчик.

Если MPU запрещен и этот бит устанавливается в 1, то поведение непредсказуемо.

15.1.2.3 ENABLE

Разрешает MPU:

0 – MPU запрещен;

1 – MPU разрешен.

Если ENABLE и PRIVDEFENA одновременно установлены в 1:

Для привилегированного доступа карта памяти default описана в разделе 9 «Организация памяти». Любой неадресованный доступ привилегированным программным обеспечением к разрешенному региону, ведет себя как определено default картой памяти. Любой неадресованный доступ непривилегированным программным обеспечением к разрешенному региону вызывает ошибку управления памятью.

XN и строго упорядоченные правила всегда применяются к управляющему системному пространству несмотря на значение бита ENABLE.

Когда ENABLE установлен в единицу, по крайней мере, один регион карты памяти должен быть разрешен для системных функций, за исключением PRIVDEFENA установлен в единицу. Если PRIVDEFENA установлен в единицу и нет разрешенных регионов, тогда только привилегированное программное обеспечение может исполняться.

Когда ENABLE установлен в ноль, система использует default карту памяти. Это аналогично памяти с атрибутами, как если бы MPU не применялся. К default карте памяти доступ осуществляется как с помощью привилегированного, так и непривилегированного программного обеспечения.

Когда MPU разрешен, доступ к системному пространству управления и таблице векторов всегда разрешен. К другим областям доступ базируется на регионах и состоянии бита PRIVDEFENA.

За исключением случая HFNMIENA установленного в 1, MPU не разрешает процессору выполнять обработчики прерываний с приоритетом -1 или -2. Эти приоритеты допустимы, только когда обрабатывается прерывание аппаратной ошибки или NMI, или, когда FAULTMASK разрешен. Установка бита HFNMIENA в единицу разрешает действовать этим двум приоритетам.

15.1.3 MPU->RNR

Регистр RNR выбирает, на какой регион памяти ссылаются регистры RBAR и RASR.

Таблица 75 – Регистр номера региона RNR

Номер	31...8	7...0
Доступ		
Сброс		
	-	REGION

15.1.3.1 REGION

Указывает MPU регион, на который ссылаются регистры RBAR и RASR.

MPU поддерживает 8 регионов памяти, поэтому разрешенное значение для этого поля от 0 до 7.

Обычно вы записываете требуемое значение номера региона в этот регистр перед обращением в RBAR и RASR. Однако вы можете изменить номер региона записью в RBAR с установленным в единицу битом VALID. Эта запись обновляет значение поля REGION.

15.1.4 MPU->RBAR

Регистр RBAR определяет базовый адрес региона MPU, выбранного RNR, вы можете изменить значение RNR. Запись RBAR с битом VALID установленным в единицу изменяет текущий номер региона и обновляет RNR.

Таблица 76 – Регистр базового адреса региона RBAR

Номер	31...N	N-1...6	5	4	3...0
Доступ					
Сброс					
	ADDR	-	VALID		REGION

15.1.4.1 ADDR

Поле базового адреса региона. Значение N зависит от размера региона. Для более подробной информации смотрите раздел “Поле ADDR”.

15.1.4.2 VALID

Бит верности номера региона MPU:

Запись:

0 – RNR не изменяется, и процессор:

- обновляет базовый адрес для региона, определенного в RNR;
- игнорирует значение поля REGION.

1 – процессор:

- обновляет значение RNR на значение из поля REGION;
- обновляет базовый адрес региона, определенного в поле REGION.

Всегда читается как ноль.

15.1.4.3 REGION

Поле MPU региона:

- поведение при записи описано выше (см. описание бита VALID);
- при чтении возвращает текущий номер региона, который определен в регистре RNR.

15.1.4.4 Поле ADDR

Поле ADDR это [31:N] бит регистра RBAR. Размер региона определяется полем SIZE в регистре RASR, как:

$$N = \log_2(\text{Размер региона в байтах}) . \quad (3)$$

Если размер региона сконфигурирован равным 4 ГБ, в RASR, то значение поля ADDR неверно. В этом случае, регион занимает всю карту памяти, и базовый адрес его равен 0x00000000.

Базовый адрес выравнивается под размер региона. Например, 64КБ регион должен быть кратно 64КБ, например, 0x00010000 или 0x00020000.

15.1.5 MPU->RASR

Регистр RASR определяет размер и атрибуты памяти MPU региона, выбранного RNR, а также разрешает регион и любые подрегионы.

RASR доступен в режиме слова или полуслова:

- старшее значащее полуслово содержит атрибуты региона;
- младшее значащее полуслово содержит размер региона и биты разрешения региона и подрегионов.

Таблица 77 – Назначение бит регистра RASR.

Номер	31...29	28	27	26...24	23, 22	21...19	18	17	16	15...8	7, 6	5...1	0
Доступ													
Сброс													
	-	XN	-	AP	-	TEX	S	C	B	SRD	-	SIZE	ENABLE

15.1.5.1 XN

Бит запрещения доступа инструкций:

- 0 – выборка инструкций разрешена;
- 1 – выборка инструкций запрещена.

15.1.5.2 AP

Поле разрешения доступа, см. таблицу 81.

15.1.5.3 TEX, C, B

Атрибуты доступа к памяти, см. таблицу 79.

15.1.5.4 S

Бит общего доступа, см. таблицу 78.

15.1.5.5 SRD

Бит запрещения подрегиона. Для каждого бита в этом поле:

- 0 – соответствующий подрегион разрешен;
- 1 – соответствующий подрегион запрещен.

Для более подробной информации см. подпункт 15.1.8.3 «Подрегионы».

Регион размером 128 байт и менее не поддерживает подрегионы. Когда записываются атрибуты для такого региона, записывайте поле SRD равным 0x00.

15.1.5.6 SIZE

Определяет размер MPU региона. Минимальное разрешенное значение 3(b00010), для более подробной информации см. подпункт 15.1.5.8 «Значения поля SIZE»

15.1.5.7 ENABLE

Бит разрешения региона.

Для более подробной информации о разрешении доступа, см. пункт 15.1.6 «Атрибуты разрешения доступа MPU».

15.1.5.8 Значения поля SIZE

Поле SIZE определяет размер памяти MPU региона, выбранного регистром RNR следующим образом:

$$(\text{Размер региона в байтах}) = 2^{(SIZE+1)} . \quad (4)$$

Наименьший разрешенный размер региона 32 байт, соответствует значению SIZE, равному 4. В таблице 78 представлены примеры значений SIZE, соответствующие размеру региона и значению N регистра RBAR.

Таблица 78 – Пример значений поля SIZE

Значение SIZE	Размер региона	Значение N ⁽¹⁾	Комментарий
b00100 (4)	32 байт	5	Минимальный разрешенный размер
b01001 (9)	1 кбайт	10	-
b10011 (19)	1 Мбайт	20	-
b11101 (29)	1 Гбайт	30	-
b11111 (31)	4 Гбайт	b01100	Максимальный разрешенный размер
(1) – Содержится в RBAR, см. пункт 15.1.4 «MPU->RBAR»			

15.1.6 Атрибуты разрешения доступа MPU

Раздел описывает атрибуты разрешения доступа. Биты разрешения доступа TEX, C, B, S, AP и XN регистра RASR контролируют доступ к соответствующему региону памяти. Если происходит доступ к области памяти без разрешения доступа, то MPU генерирует ошибку доступа.

Таблица 79 – Кодирование бит разрешения доступа TEX, C, B, S

TEX	C	B	S	Тип памяти	Возможность общего доступа	Другие атрибуты
b000	0	0	x ⁽¹⁾	Строго упорядоченная	Общий доступ	
		1	x ⁽¹⁾	Устройство	Общий доступ	
	1	0	0	Обычная	Не общий доступ	Внешний и внутренний кэш, синхронное обновление памяти. Запись без кэширования
			1		Общий доступ	
		1	0	Обычная	Не общий доступ	Внешний и внутренний кэш, отложенное обновление памяти. Запись без кэширования
			1		Общий доступ	
b001	0	0	0	Обычная	Не общий доступ	
			1		Общий доступ	
		1	x ⁽¹⁾	Зарезервировано		-
	1	0	x ⁽¹⁾	Реализация определяется атрибутами		-
			0	Обычная	Не общий доступ	Внешний и внутренний кэш, отложенное обновление памяти. Запись и чтение пакетные
		1	1		Общий доступ	

TEX	C	B	S	Тип памяти	Возможность общего доступа	Другие атрибуты
b010	0	0	x ⁽¹⁾	Устройство	Не общий доступ	Индивидуальное устройство
		1	x ⁽¹⁾	Зарезервировано		-
	1	x ⁽¹⁾	x ⁽¹⁾	Зарезервировано		-
b1BV	A	A	0	Обычное	Не общий доступ	
			1		общий доступ	
(1) – MPU игнорирует значение этих бит						

Таблица 80 поясняет кодирование режима кэша атрибутом TEX в диапазоне значений атрибута от 4 до 7.

Таблица 80 – Кодирование режима кэша атрибутом TEX

Значение AA или BB при TEX=1xx	Соответствующий режим кэша
00	Не кэшируемая
01	Отложенное обновление, запись и чтение пакетные
10	Синхронное обновление, запись без кэширования
11	Отложенное обновление, запись без кэширования

Таблица 81 поясняет кодирование бит AP, определяющих разрешение на доступ для привилегированного и непривилегированного программного обеспечения (ПО).

Таблица 81 – Кодирование привилегий доступа в поле AP

AP[2:0]	Привилегированный доступ	Непривилегированный доступ	Описание
000	нет доступа	нет доступа	Любой доступ приводит к ошибке доступа
001	RW	нет доступа	Доступ только для привилегированного ПО
010	RW	RO	Запись непривилегированным ПО приводит к ошибке доступа
011	RW	RW	Полный доступ
100	непредсказуемо	непредсказуемо	Зарезервировано
101	RO	нет доступа	Чтение только привилегированным ПО
110	RO	RO	Только чтение и привилегированным, и непривилегированным ПО
111	RO	RO	Только чтение и привилегированным, и непривилегированным ПО

15.1.7 Несоответствие MP

Когда происходит нарушение разрешения доступа MPU, процессор генерирует ошибку управления памятью, см. раздел 31 «Прерывания и исключения». Регистр MMFSR указывает причину ошибки. Для более подробной информации см. подпункт 33.1.10.1 «Поле MMFSR».

15.1.8 Обновление MPU региона

Атрибуты для MPU региона обновляют через регистры RNR, RBAR и RASR. Вы можете программировать каждый регистр независимо или использовать для программирования возможность множественной записи всех этих регистров. Вы можете использовать обозначения RBAR и RASR, чтобы запрограммировать до 4 регионов одновременно, используя инструкцию STM.

15.1.8.1 Обновление MPU региона через отдельные регистры

Простой код для одного региона:

R1 = номер региона

R2 = размер/разрешение

R3 = атрибуты

R4 = адрес

LDR R0,=MPU_RNR – 0xE000ED98, регистр номера региона MPU

STR R1, [R0, #0x0] – номер региона

STR R4, [R0, #0x4] – базовый адрес региона

STRH R2, [R0, #0x8] – размер региона и разрешение

STRH R3, [R0, #0xA] – атрибуты региона

Запрещение региона перед записью новых настроек в MPU, если этот регион перед этим был разрешен. Например:

R1 = номер региона

R2 = размер/разрешение

R3 = атрибуты

R4 = адрес

LDR R0,=MPU_RNR – 0xE000ED98, регистр номера региона MPU

STR R1, [R0, #0x0] – номер региона

BIC R2, R2, #1 – запрещение

STRH R2, [R0, #0x8] – размер региона и разрешение

STR R4, [R0, #0x4] – базовый адрес региона

STRH R3, [R0, #0xA] – атрибуты региона

ORR R2, #1 – разрешение

STRH R2, [R0, #0x8] – размер региона и разрешение.

Программное обеспечение должно применить barrier инструкции:

- если перед установкой MPU будет невыполненная пересылка в память, такая как буферная запись, то это может повлиять на изменение настроек MPU;

- после установки MPU, если это включает пересылку в память, должны использоваться новые настройки MPU.

Однако не требуются barrier инструкции памяти, если процесс установки MPU начинается с помощью входа в обработчик прерывания, или сопровождается возвращением из прерывания, потому что вход и выход из прерывания сопровождается механизмом barrier для памяти.

Программному обеспечению не требуются barrier инструкции памяти во время установки MPU, потому что этот доступ осуществляется через PPB, который строго упорядоченный регион памяти.

Например, если вы хотите, чтобы все изменения доступа к памяти имели место непосредственно после программной последовательности, используйте инструкции DSB и ISB. Инструкции DSB требуются после изменения настроек MPU, в конце переключения контекста. Инструкции ISB требуются, если код, который программирует MPU регион или регионы вызывается с использованием инструкций перехода (branch) или вызова подпрограммы (call). Если программная последовательность вызывается инструкцией выхода из прерывания (return), или прерыванием, то ISB не требуется.

15.1.8.2 Обновление MPU региона через множественную запись регистров

Вы можете программировать напрямую, используя запись множества регистров, в зависимости от того, как распределена информация.

R1 = номер региона

R2 = адрес

R3 = размер, атрибуты

LDR R0, =MPU_RNR – 0xE000ED98, регистр номера региона MPU

STR R1, [R0, #0x0] – номер региона

STR R2, [R0, #0x4] – базовый адрес региона

STR R3, [R0, #0x8] – атрибут региона, размер и разрешение.

Оптимизация при использовании STM инструкции:

R1 = номер региона

R2 = адрес

R3 = размер, атрибуты

LDR R0, =MPU_RNR – 0xE000ED98, регистр номера региона MPU

STM R0, {R1-R3} – номер региона, адрес, атрибут, размер и разрешение.

Вы можете использовать два слова для предварительной упаковки информации. Это значит, что RBAR содержит требуемый номер региона и имеет бит VALID, установленный в единицу, см. пункт 15.1.4 «MPU->RBAR». Это применимо, если данные упакованы статически, например, в начальном загрузчике:

R1 = адрес и номер региона;

R2 = размер и атрибуты;

LDR R0, =MPU_RBAR – 0xE000ED9C, регистр базового адреса MPU

STR R1, [R0, #0x0] – базовый адрес и номер региона, совмещенные с битом VALID, установленным в 1

STR R2, [R0, #0x4] – атрибут региона, размер и разрешение.

Оптимизация при использовании STM инструкции:

R1 = адрес и номер региона

R2 = размер и атрибуты

LDR R0, =MPU_RBAR – 0xE000ED9C, регистр базового адреса MPU

STM R0, {R1-R2} – базовый адрес региона, номер региона и бит VALID, и атрибут региона, размер и разрешение.

15.1.8.3 Подрегионы

Регионы величиной в 256 байт или более делятся на восемь равных подрегионов. Установите соответствующий бит в поле SRD регистра RASR для запрещения подрегиона, см. пункт 15.1.5 «MPU->RASR». Младший значащий бит SRD контролирует первый подрегион, и старший значащий бит контролирует последний подрегион. Запрещение подрегиона означает, что другой регион перекрывает запрещенную область. Если другой разрешенный регион не перекрывает запрещенный регион, то MPU выдает ошибку.

Регионы размером 32, 64 и 128 не поддерживают подрегионы, с этими регионами вы должны установить поле SRD равным 0x00, иначе поведение MPU непредсказуемо.

15.1.8.4 Пример применения SRD

Два региона с одинаковым базовым адресом перекрываются. Регион размером 128 кбайт и регион размером 512 кбайт. Убедитесь, что атрибуты для региона один установлены для первых 128 кбайт, установите SRD поле для региона два в значение b00000011 для запрещения первых двух подрегионов, как показано на рисунке 27.

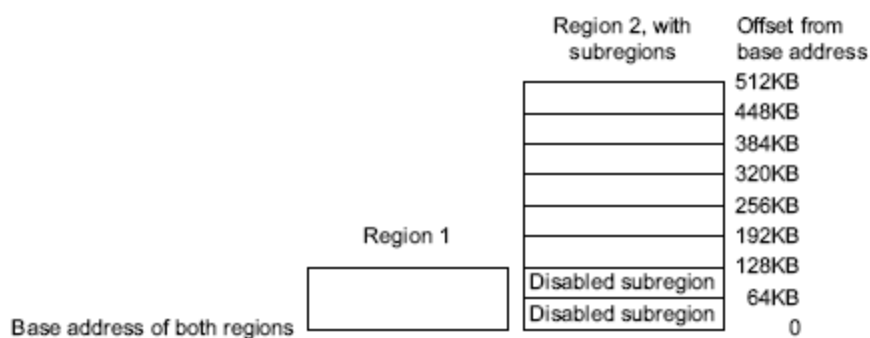


Рисунок 27 – Применение SRD

15.2 Советы и особенности применения MPU

Во избежание непредвиденных ситуаций, запретите прерывания перед обновлением атрибутов региона, к которому может осуществляться доступ в обработчике прерываний.

Убедитесь, что программное обеспечение использует корректный доступ, соответствующий размеру регистров MPU:

- за исключением RASR, необходимо использовать доступ по словам;
- для RASR может использоваться доступ по байтам, полусловам или словам.

Процессор не поддерживает невыровненный доступ к регистрам MPU.

Если MPU перенастраивается, то запретите неиспользуемые регионы для предотвращения любых предыдущих настроек регионов от их влияния на новые настройки.

15.2.1 Конфигурация MPU для микросхемы

Обычно, микросхемные системы имеют только один процессор и не имеют кэша. В таких системах MPU программируется следующим образом:

Таблица 82 – Атрибуты регионов памяти для микросхемы

Регион памяти	TEX	C	B	S	Типа памяти и атрибут
Флеш-память	b000	1	0	0	Обычная память, не общий доступ, сквозная запись
Внутренняя SRAM	b000	1	0	1	Обычная память, общий доступ, сквозная запись
Внешняя SRAM	b000	1	1	1	Обычная память, общий доступ, обратная запись, выделенная запись
Периферия	b000	0	1	1	Память устройства, общий доступ

В большинстве микросхемных приложениях, установка атрибутов общего доступа и кэширования не влияет на поведение системы. Однако применение этих настроек для MPU регионов может сделать код приложений более переносимым. Это имеет большую важность в обычных ситуациях. В специальных системах, таких как многопроцессорные или с отдельным DMA устройством, атрибуты общего доступа очень важны. В этих случаях обращайтесь к рекомендациям производителей устройств памяти.

16 Сигналы тактовой частоты MDR_RST_CLK

Микросхема имеет два встроенных и два внешних генератора, а также специализированный блок формирования тактовой синхронизации микросхемы.

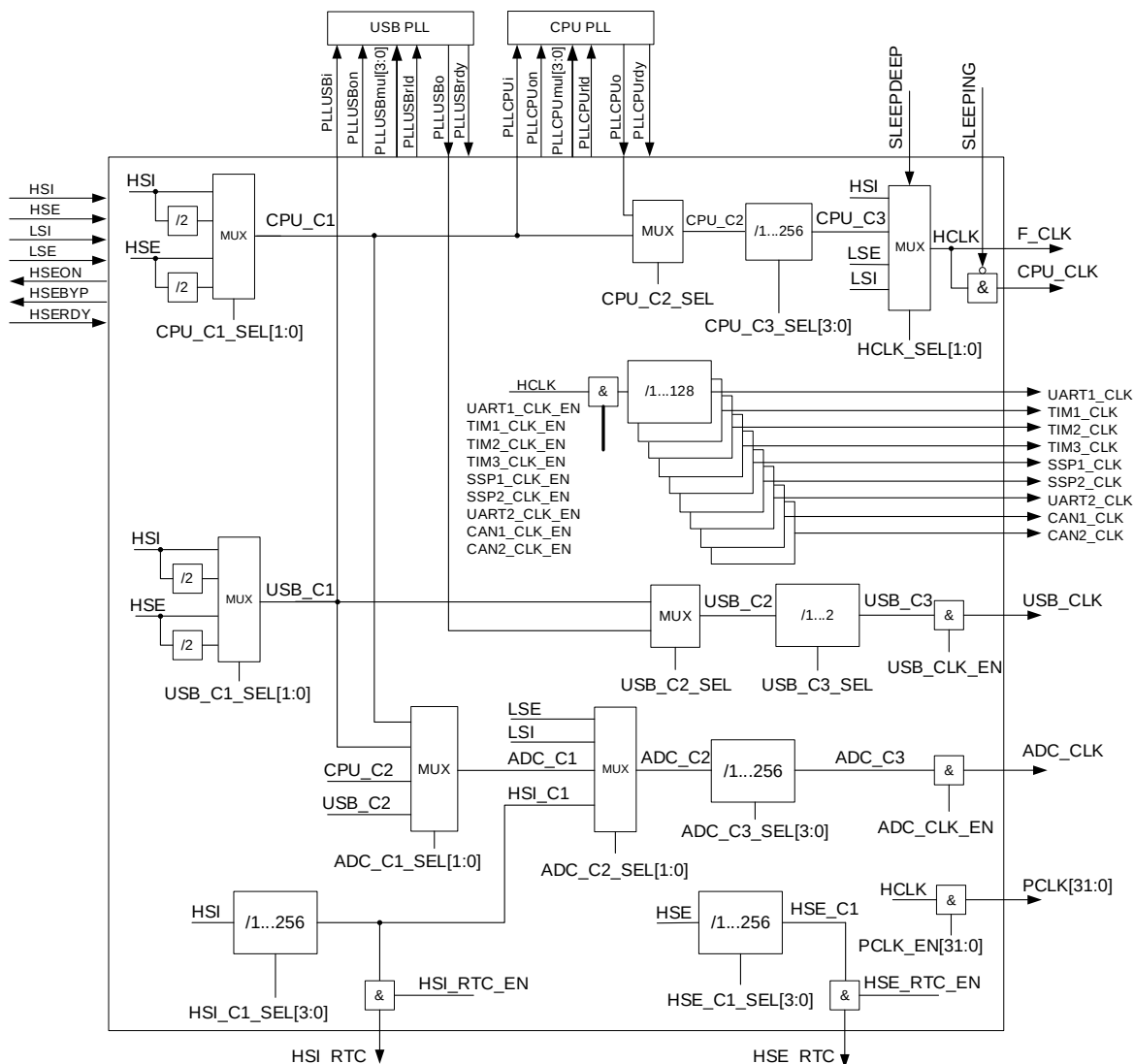


Рисунок 28 – Структурная блок-схема формирования тактовой частоты

Управление тактовыми частотами ведется через периферийный блок RST_CLK. При включении питания микросхема запускается на частоте HSI генератора. Выдача тактовых сигналов синхронизации для всех периферийных блоков, кроме RST_CLK, отключена. Для начала работы с нужным периферийным блоком необходимо включить его тактовую частоту в регистре PER_CLOCK. Некоторые контроллеры интерфейсов (UART, CAN, USB, Таймеры) могут работать на частотах, отличных от частоты процессорного ядра, поэтому в соответствующих регистрах (UART_CLOCK, CAN_CLOCK, USB_CLOCK, TIM_CLOCK) могут быть заданы их скорости работы. Для изменения тактовой частоты ядра можно перейти на другой генератор и/или воспользоваться блоком умножения тактовой частоты. Для корректной смены тактовой частоты сначала должны быть сформированы необходимые тактовые частоты и затем осуществлено переключение на них на соответствующих мультиплексорах, управляемых регистрами CPU_CLOCK и USB_CLOCK.

Для переключения с одного источника частоты на другой, нужно, чтобы оба источника оставались включенными.

При смене тактовой частоты процессорного ядра следует:

1. Настроить число тактов паузы для доступа к Flash-памяти, задаваемое полем Delay в регистре MDR_EEPROM->EEPROM_CMD.

2. Настроить режим работы встроенного регулятора напряжения DU_{CC}, задаваемый полем LOW регистра MDR_BKP->REG_0E совместно с дополнительной стабилизирующей нагрузкой для DU_{CC}, задаваемой полем SelectRI регистра MDR_BKP->REG_0E.

Если настраиваемая частота процессорного ядра выше текущей, то данные настройки нужно проводить до смены частоты, в противном случае – после её смены.

16.1 Встроенный RC-генератор HSI

Генератор HSI вырабатывает тактовую частоту f_{O_HSI} с типовым значением 8 МГц. Генератор автоматически запускается при появлении питания U_{CC} и при выходе в нормальный режим работы вырабатывает сигнал HSIRDY в регистре батарейного домена BKP_REG_0F. Первоначально процессорное ядро запускается на тактовой частоте HSI. При дальнейшей работе генератор HSI может быть отключен при помощи сигнала HSION в регистре BKP_REG_0F. Также генератор может быть подстроен при помощи сигнала HSITRIM в регистре BKP_REG_0F.

16.2 Встроенный RC-генератор LSI

Генератор LSI вырабатывает тактовую частоту f_{O_LSI} с типовым значением 40 кГц. Генератор автоматически запускается при появлении питания U_{CC} и при выходе в нормальный режим работы вырабатывает сигнал LSIRDY в регистре BKP_REG_0F. Первоначально тактовая частота генератор LSI используется для формирования дополнительной задержки трог. При дальнейшей работе генератор LSI может быть отключен при помощи сигнала LSION в регистре BKP_REG_0F.

16.3 Внешний генератор HSE

Генератор HSE предназначен для выработки тактовой частоты 2...16 МГц с помощью внешнего резонатора. Генератор запускается при появлении питания U_{CC} и сигнала разрешения HSEON в регистре HS_CONTROL. При выходе в нормальный режим работы вырабатывает сигнал HSERDY в регистре CLOCK_STATUS. Также этот генератор может работать в режиме HSEBYP, когда входная тактовая частота с входа OSC_IN проходит напрямую на выход HSE. Выход OSC_OUT находится в этом режиме в третьем состоянии.

16.4 Внешний генератор LSE

Генератор LSE предназначен для выработки тактовой частоты 32 кГц с помощью внешнего резонатора. Генератор запускается при появлении питания BDU_{CC} и сигнала разрешения LSEON в регистре BKP_REG_0F. При выходе в нормальный режим работы вырабатывает сигнал LSERDY в регистре BKP_REG_0F. Также осциллятор может

работать в режиме LSEBYP, когда входная тактовая частота с входа OSC_IN32 проходит напрямую на выход LSE. Выход OSC_OUT32 находится в этом режиме третьем состоянии. Так как генератор LSE питается от напряжения питания BDU_{CC} и его регистр управления BKP_REG_0F расположен в батарейном домене, то генератор может продолжать работать при пропадании основного питания U_{CC}. Генератор LSE используется для работы часов реального времени.

16.5 Встроенный блок умножения системной тактовой частоты

Блок умножения позволяет провести умножение входной тактовой частоты на коэффициент от 4 до 16, задаваемый на входе PLLCPUMUL[3:0] в регистре PLL_CONTROL. Входная частота блока умножителя должна быть в диапазоне от 6 до 16 МГц, выходная – до 100 МГц. При выходе блока умножителя тактовой частоты в расчетный режим вырабатывается сигнал PLLCPURDY в регистре CLOCK_STATUS. Блок включается с помощью сигнала PLLCPUON в регистре PLL_CONTROL. Выходная частота используется как основная частота процессора и периферии.

16.6 Встроенный блок умножения USB тактовой частоты

Блок умножения позволяет провести умножение входной тактовой частоты на коэффициент от 4 до 16, задаваемый на входе PLLUSBMUL[3:0] в регистре PLL_CONTROL. Входная частота блока умножителя должна быть в диапазоне от 6 до 16 МГц, выходная должна составлять 48 МГц. При выходе блока умножителя тактовой частоты в расчетный режим вырабатывается сигнал PLLUSBRDY в регистре CLOCK_STATUS. Блок включается с помощью сигнала PLLUSBON в регистре PLL_CONTROL. Выходная частота используется как основная частота протокольной части USB интерфейса.

16.7 Описание регистров блока контроллера тактовой частоты

Таблица 83 – Описание регистров блока контроллера тактовой частоты

Базовый Адрес	Название	Описание
0x4002_0000	MDR_RST_CLK	Контроллер тактовой частоты
Смещение		
0x00	CLOCK_STATUS	MDR_RST_CLK->CLOCK_STATUS Регистр состояния блока управления тактовой частотой
0x04	PLL_CONTROL	MDR_RST_CLK->PLL_CONTROL Регистр управления блоками умножения частоты
0x08	HS_CONTROL	MDR_RST_CLK->HS_CONTROL Регистр управления высокочастотным генератором и осциллятором
0x0C	CPU_CLOCK	MDR_RST_CLK->CPU_CLOCK Регистр управления тактовой частотой процессорного ядра
0x10	USB_CLOCK	MDR_RST_CLK->USB_CLOCK Регистр управления тактовой частотой контроллера USB

Базовый Адрес	Название	Описание
0x14	ADC_MCO_CLOCK	MDR_RST_CLK->ADC_MCO_CLOCK Регистр управления тактовой частотой АЦП
0x18	RTC_CLOCK	MDR_RST_CLK->RTC_CLOCK Регистр управления формированием высокочастотных тактовых сигналов блока RTC
0x1C	PER_CLOCK	MDR_RST_CLK->PER_CLOCK Регистр управления тактовой частотой периферийных блоков
0x20	CAN_CLOCK	MDR_RST_CLK->CAN_CLOCK Регистр управления тактовой частотой CAN
0x24	TIM_CLOCK	MDR_RST_CLK->TIM_CLOCK Регистр управления тактовой частотой TIMER
0x28	UART_CLOCK	MDR_RST_CLK->UART_CLOCK Регистр управления тактовой частотой UART
0x2C	SSP_CLOCK	MDR_RST_CLK->SSP_CLOCK Регистр управления тактовой частотой SSP

16.7.1 MDR_RST_CLK->CLOCK_STATUS

Таблица 84 – Регистр CLOCK_STATUS

Номер	31...3	2	1	0
Доступ	U	RO	RO	RO
Сброс	0	0	0	0
	-	HSERDY	PLLCPURDY	PLLUSBRDY

Таблица 85 – Описание бит регистра CLOCK_STATUS

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...3	-	Зарезервировано
2	HSE RDY	Флаг выхода в рабочий режим осциллятора HSE: 0 – осциллятор не запущен или не стабилен; 1 – осциллятор запущен и стабилен
1	PLL CPU RDY	Флаг выхода в рабочий режим CPU PLL: 0 – PLL не запущена или не стабильна; 1 – PLL запущена и стабильна. Для обновления состояния PLL_CPU_RDY при PLL_CPU_RDY = 1 перед изменением PLL_CPU_ON в 1 (вкл.) или в 0 (выкл.) установить PLL_CPU_RLD = 1. И затем при изменений PLL_CPU_ON в 1 или в 0 одновременно записать PLL_CPU_RLD = 0
0	PLL USB RDY	Флаг выхода в рабочий режим USB PLL: 0 – PLL не запущена или не стабильна; 1 – PLL запущена и стабильна. Для обновления состояния PLL_USB_RDY при PLL_USB_RDY = 1 перед изменением PLL_USB_ON в 1 (вкл.) или в 0 (выкл.) установить PLL_USB_RLD = 1. И затем при

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
		изменений PLL_USB_ON в 1 или в 0 одновременно записать PLL_USB_RLD = 0

16.7.2 MDR_RST_CLK->PLL_CONTROL

Таблица 86 – Регистр PLL_CONTROL

Номер	31...12	11...8	7...4	3	2	1	0
Доступ	U	R/W	R/W	R/W	R/W	R/W	R/W
Сброс	0	0	0	0	0	0	0
	-	PLL CPU MUL[3:0]	PLL USB MUL[3:0]	PLL CPU RLD	PLL CPU ON	PLL USB RLD	PLL USB ON

Таблица 87 – Описание бит регистра PLL_CONTROL

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...12	-	Зарезервировано
11...8	PLL CPU MUL[3:0]	Коэффициент умножения для CPU PLL: $PLL_{CPU0} = PLL_{CPUi} \times (PLL_{CPUMUL} + 1)$. Примечание – Значения 1 и 2 для данного поля не допустимы
7...4	PLL USB MUL[3:0]	Коэффициент умножения для USB PLL: $PLL_{USB0} = PLL_{USBi} \times (PLL_{USBMUL} + 1)$. Примечание – Значения 1 и 2 для данного поля не допустимы
3	PLL CPU RLD	Бит перезапуска PLL. При смене коэффициента умножения в рабочем режиме необходимо задать равным 1, а после этого сбросить в ноль
2	PLL CPU ON	Бит включения PLL: 0 – PLL выключена; 1 – PLL включена
1	PLL USB RLD	Бит перезапуска PLL. При смене коэффициента умножения в рабочем режиме необходимо задать равным 1, а после этого сбросить в ноль
0	PLL USB ON	Бит включения PLL: 0 – PLL выключена; 1 – PLL включена

16.7.3 MDR_RST_CLK->HS_CONTROL

Таблица 88 – Регистра HS_CONTROL

Номер	31...2	1	0
Доступ	U	R/W	R/W
Сброс	0	0	0
	-	HSE BYP	HSE ON

Таблица 89 – Описание бит регистра HS_CONTROL

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...2	-	Зарезервировано
1	HSE BYP	Бит управления HSE осциллятором: 0 – режим осциллятора; 1 – режим внешнего генератора. Устанавливать совместно с HSE_ON = 1
0	HSE ON	Бит управления HSE осциллятором: 0 – выключен; 1 – включен

16.7.4 MDR_RST_CLK->CPU_CLOCK

Таблица 90 – Регистр CPU_CLOCK

Номер	31...10	9...8	7...4	3	2	1...0
Доступ	U	R/W	R/W	U	R/W	R/W
Сброс	0	0	0	0	0	0
	-	HCLK SEL[1:0]	CPU C3 SEL[3:0]	-	CPU C2 SEL	CPU C1 SEL[1:0]

Таблица 91 – Описание бит регистра CPU_CLOCK

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...10	-	Зарезервировано
9...8	HCLK SEL[1:0]	Биты выбора источника для HCLK: 00 – HSI 01 – CPU_C3 10 – LSE 11 – LSI
7...4	CPU C3 SEL[3:0]	Биты выбора делителя для CPU_C3: 0xxx – CPU_C3 = CPU_C2 1000 – CPU_C3 = CPU_C2 / 2 1001 – CPU_C3 = CPU_C2 / 4 1010 – CPU_C3 = CPU_C2 / 8 ... 1111 – CPU_C3 = CPU_C2 / 256
3	-	Зарезервировано
2	CPU C2 SEL	Биты выбора источника для CPU_C2: 0 – CPU_C1 1 – PLLCPUo
1...0	CPU C1 SEL[1:0]	Биты выбора источника для CPU_C1: 00 – HSI 01 – HSI/2 10 – HSE 11 – HSE/2

16.7.5 MDR_RST_CLK->USB_CLOCK

Таблица 92 – Регистр USB_CLOCK

Номер	31...9	8	7...5	4	3	2	1, 0
Доступ	U	R/W	U	R/W	U	R/W	R/W
Сброс	0	0	0	0	0	0	0
	-	USB CLK EN	-	USB C3 SEL	-	USB C2 SEL	USB C1 SEL[1:0]

Таблица 93 – Описание бит регистра USB_CLOCK

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...9	-	Зарезервировано
8	USB CLK EN	Бит разрешения тактирования USB: 0 – нет тактовой частоты; 1 – есть тактовая частота
7...5	-	Зарезервировано
4	USB C3 SEL	Биты выбора делителя для USB_C3: 0 – USB_C3 = USB_C2 1 – USB_C3 = USB_C2 / 2
3	-	Зарезервировано
2	USB C2 SEL	Биты выбора источника для USB_C2: 0 – USB_C1 1 – PLLUSB0
1, 0	USB C1 SEL[1:0]	Биты выбора источника для USB_C1: 00 – HSI 01 – HSI/2 10 – HSE 11 – HSE/2

16.7.6 MDR_RST_CLK->ADC_MCO_CLOCK

Таблица 94 – Регистр ADC_MCO_CLOCK

Номер	31...14	13	12	11...8	7...6	5, 4	3, 2	1, 0
Доступ	U	R/W	U	R/W	U	R/W	U	R/W
Сброс	0	0	0	0	0	0	0	0
	-	ADC CLK EN	-	ADC C3 SEL[3:0]	-	ADC C2 SEL[1:0]	-	ADC C1 SEL[1:0]

Таблица 95 – Описание бит регистра ADC_MCO_CLOCK

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...14	-	Зарезервировано
13	ADC CLK EN	Бит разрешения выдачи тактовой частоты ADC CLK: 0 – запрещен; 1 – разрешен
12	-	Зарезервировано

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
11...8	ADC C3 SEL[3:0]	Биты выбора делителя для ADC_C3: 0xxx – ADC_C3 = ADC_C2 1000 – ADC_C3 = ADC_C2 / 2 1001 – ADC_C3 = ADC_C2 / 4 1010 – ADC_C3 = ADC_C2 / 8 ... 1111 – ADC_C3 = ADC_C2 / 256
7, 6	-	Зарезервировано
5, 4	ADC C2 SEL[1:0]	Биты выбора источника для ADC_C2: 00 – LSE 01 – LSI 10 – ADC_C1 11 – HSI_C1
3, 2	-	Зарезервировано
1, 0	ADC C1 SEL[1:0]	Биты выбора источника для ADC_C1: 00 – CPU_C1 01 – USB_C1 10 – CPU_C2 11 – USB_C2

16.7.7 MDR_RST_CLK->RTC_CLOCK

Таблица 96 – Регистр RTC_CLOCK

Номер	31...10	9	8	7...4	3...0
Доступ	U	R/W	R/W	R/W	R/W
Сброс	0	0	0	0	0
	-	HSI RTC EN	HSE RTC EN	HSI_C1 SEL[1:0]	HSE_C1 SEL[1:0]

Таблица 97 – Описание бит регистра RTC_CLOCK

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...10	-	Зарезервировано
9	HSI RTC EN	Бит разрешения выдачи тактовой частоты HSI_RTC: 0 – запрещен; 1 – разрешен
8	HSE RTC EN	Бит разрешения выдачи тактовой частоты HSE_RTC: 0 – запрещен; 1 – разрешен
7...4	HSI_C1 SEL[3:0]	Биты выбора делителя для HSI_C1: 0xxx – HSI_C1 = HSI 1000 - HSI_C1 = HSI / 2 1001 - HSI_C1 = HSI / 4 1010 - HSI_C1 = HSI / 8 ... 1111 - HSI_C1 = HSI / 256

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
3...0	HSE_C1 SEL[3:0]	Биты выбора делителя для HSE_C1: 0xxx – HSE_C1= HSE 1000 - HSE_C1= HSE / 2 1001 - HSE_C1= HSE / 4 1010 - HSE_C1= HSE / 8 ... 1111 - HSE_C1= HSE / 256

16.7.8 MDR_RST_CLK->PER_CLOCK

Таблица 98 – Регистр PER_CLOCK

Номер	31...5	4	3...0
Доступ	R/W	R/W	R/W
Сброс	0	1	0
	PCLK_EN[31:5]	PCLK_EN[4]	PCLK_EN[3:0]

Таблица 99 – Описание бит регистра PER_CLOCK

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...5	PCLK EN[31:5]	Биты разрешения тактирования периферийных блоков: 0 – запрещено; 1 – разрешено. PCLK[5] – DMA PCLK[6] – UART1 PCLK[7] – UART2 PCLK[8] – SSP1 PCLK[9] – зарезервировано PCLK[10] – I2C1 PCLK[11] – POWER PCLK[12] – WWDG PCLK[13] – IWDG PCLK[14] – TIMER1 PCLK[15] – TIMER2 PCLK[16] – TIMER3 PCLK[17] – ADC PCLK[18] – DAC PCLK[19] – COMP PCLK[20] – SSP2 PCLK[21] – PORTA PCLK[22] – PORTB PCLK[23] – PORTC PCLK[24] – PORTD PCLK[25] – PORTE PCLK[26] – зарезервировано PCLK[27] – BKP PCLK[28] – зарезервировано PCLK[29] – PORTF PCLK[30] – EXT_BUS_CNTRL PCLK[31] – зарезервировано
4	PCLK	Биты разрешения тактирования периферийных блоков:

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
	EN[4]	0 – запрещено; 1 – разрешено. PCLK[4] – RST_CLK. После сброса в состоянии 1
3...0	PCLK EN[3:0]	Биты разрешения тактирования периферийных блоков: 0 – запрещено; 1 – разрешено. PCLK[0] – CAN1 PCLK[1] – CAN2 PCLK[2] – USB PCLK[3] – EEPROM_CNTRL

16.7.9 MDR_RST_CLK->CAN_CLOCK

Таблица 100 – Регистр CAN_CLOCK

Номер	31...26	25	24	23...16	15...8	7...0
Доступ	U	R/W	R/W	U	R/W	R/W
Сброс	0	0	0	0	0	0
	-	CAN2 CLK EN	CAN1 CLK EN	-	CAN2 BRG [7:0]	CAN1 BRG [7:0]

Таблица 101 – Описание бит регистра CAN_CLOCK

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...26	-	Зарезервировано
25	CAN2 CLK EN	Разрешение тактовой частоты на CAN2: 0 – нет частоты; 1 – есть частота
24	CAN1 CLK EN	Разрешение тактовой частоты на CAN1: 0 – нет частоты; 1 – есть частота
23..16	-	Зарезервировано
15...8	CAN2 BRG [7:0]	Делитель тактовой частоты CAN2 xxxxxx000 – CAN2_CLK == HCLK xxxxxx001 – CAN2_CLK == HCLK/2 xxxxxx010 – CAN2_CLK == HCLK/4 ... xxxxxx111 – CAN2_CLK == HCLK/128
7...0	CAN1 BRG [7:0]	Делитель тактовой частоты CAN1 xxxxxx000 – CAN1_CLK == HCLK xxxxxx001 – CAN1_CLK == HCLK/2 xxxxxx010 – CAN1_CLK == HCLK/4 ... xxxxxx111 – CAN1_CLK == HCLK/128

16.7.10 MDR_RST_CLK->TIM_CLOCK

Таблица 102 – Регистр TIM_CLOCK

Номер	31...27	26	25	24	23...16	15...8	7...0
Доступ	U	R/W	R/W	R/W	R/W	R/W	R/W
Сброс	0	0	0	0	0	0	0
	-	TIM3 CLK EN	TIM2 CLK EN	TIM1 CLK EN	TIM3 BRG [7:0]	TIM2 BRG [7:0]	TIM1 BRG [7:0]

Таблица 103 – Описание бит регистра TIM_CLOCK

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...27	-	Зарезервировано
26	TIM3 CLK EN	Разрешение тактовой частоты на TIM3: 0 – нет частоты; 1 – есть частота
25	TIM2 CLK EN	Разрешение тактовой частоты на TIM2: 0 – нет частоты; 1 – есть частота
24	TIM1 CLK EN	Разрешение тактовой частоты на TIM1: 0 – нет частоты; 1 – есть частота
23..16	TIM3 BRG [7:0]	Делитель тактовой частоты TIM3: xxxxx000 – TIM3_CLK == HCLK xxxxx001 – TIM3_CLK == HCLK/2 xxxxx010 – TIM3_CLK == HCLK/4 ... xxxxx111 – TIM3_CLK == HCLK/128
15...8	TIM2 BRG [7:0]	Делитель тактовой частоты TIM2: xxxxx000 – TIM2_CLK == HCLK xxxxx001 – TIM2_CLK == HCLK/2 xxxxx010 – TIM2_CLK == HCLK/4 ... xxxxx111 – TIM2_CLK == HCLK/128
7...0	TIM1 BRG [7:0]	Делитель тактовой частоты TIM1: xxxxx000 – TIM1_CLK == HCLK xxxxx001 – TIM1_CLK == HCLK/2 xxxxx010 – TIM1_CLK == HCLK/4 ... xxxxx111 – TIM1_CLK == HCLK/128

16.7.11 MDR_RST_CLK->UART_CLOCK

Таблица 104 – Регистр UART_CLOCK

Номер	31...26	25	24	23...16	15...8	7...0
Доступ	U	R/W	R/W	U	R/W	R/W
Сброс	0	0	0	0	0	0
	-	UART2 CLK EN	UART 1 CLK EN	-	UART 2 BRG [7:0]	UART 1 BRG [7:0]

Таблица 105 – Описание бит регистра UART_CLOCK

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...27	-	Зарезервировано
26	-	Зарезервировано
25	UART2 CLK EN	Разрешение тактовой частоты на UART2: 0 – нет частоты; 1 – есть частота
24	UART1 CLK EN	Разрешение тактовой частоты на UART 1: 0 – нет частоты; 1 – есть частота
23..16	-	Зарезервировано
15...8	UART2 BRG [7:0]	Делитель тактовой частоты UART 2: xxxxx000 – UART 2_CLK == HCLK xxxxx001 – UART 2_CLK == HCLK/2 xxxxx010 – UART 2_CLK == HCLK/4 ... xxxxx111 – UART 2_CLK == HCLK/128
7...0	UART1 BRG [7:0]	Делитель тактовой частоты UART1: xxxxx000 – UART 1_CLK == HCLK xxxxx001 – UART 1_CLK == HCLK/2 xxxxx010 – UART 1_CLK == HCLK/4 ... xxxxx111 – UART 1_CLK == HCLK/128

16.7.12 MDR_RST_CLK->SSP_CLOCK

Таблица 106 – Регистр SSP_CLOCK

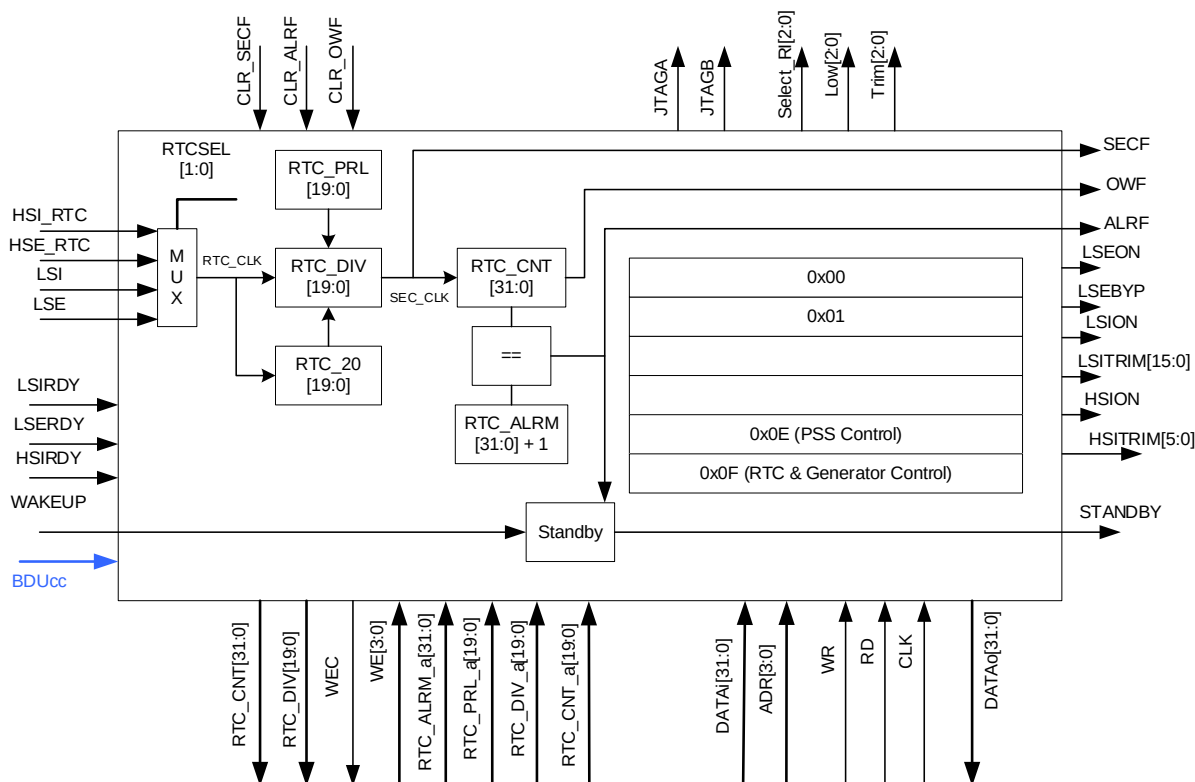
Номер	31...26	25	24	23...16	15...8	7...0
Доступ	U	R/W	R/W	U	R/W	R/W
Сброс	0	0	0	0	0	0
	-	SSP2 CLK EN	SSP 1 CLK EN	-	SSP 2 BRG [7:0]	SSP 1 BRG [7:0]

Таблица 107 – Описание бит регистра SSP_CLOCK

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...27	-	Зарезервировано
26	-	Зарезервировано
25	SSP2 CLK EN	Разрешение тактовой частоты на SSP 2: 0 – нет частоты; 1 – есть частота
24	SSP1 CLK EN	Разрешение тактовой частоты на SSP 1: 0 – нет частоты; 1 – есть частота
23...16	-	Зарезервировано

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
15...8	SSP2 BRG [7:0]	Делитель тактовой частоты SSP 2: xxxxxx000 – SSP 2_CLK == HCLK xxxxxx001 – SSP 2_CLK == HCLK/2 xxxxxx010 – SSP 2_CLK == HCLK/4 ... xxxxxx111 – SSP 2_CLK == HCLK/128
7...0	SSP1 BRG [7:0]	Делитель тактовой частоты SSP1: xxxxxx000 – SSP 1_CLK == HCLK xxxxxx001 – SSP 1_CLK == HCLK/2 xxxxxx010 – SSP 1_CLK == HCLK/4 ... xxxxxx111 – SSP 1_CLK == HCLK/128

Блок батарейного домена предназначен для обеспечения функций часов реального времени и сохранения некоторого набора пользовательских данных при отключении основного источника питания. При снижении питания U_{CC} в блоке SW происходит автоматическое переключение питания BDU_{CC} с U_{CC} на BU_{CC} . Если на BU_{CC} имеется отдельный источник питания (батарейка), то батарейный домен остается включенным и может выполнять свои функции.



Для отсчёта секунд в часах реального времени применяется 20-битный предварительный делитель входной тактовой частоты RTC CLK, на выходе которого

формируется тактовый сигнал SEC_CLK. Регистр RTC_DIV выступает в качестве счётчика предварительного делителя, который тактируется на частоте RTC_CLK. Регистр RTC_PRL задаёт коэффициент деления предварительного делителя, при этом счётчик RTC_DIV инкрементируется в интервале от 0 до RTC_PRL. Коэффициент деления в регистре RTC_PRL должен быть задан таким образом, чтобы частота SEC_CLK составляла 1 Гц.

Для калибровки тактовой частоты RTC_CLK используются биты RTC_CAL[7:0] (см. описание ошибки 0038 в документе K1986BE92FI, K1986BE92FII, K1986BE94GI Errata Notice). Значение RTC_CAL[7:0] определяет, какое число тактов RTC_CLK из 2^{20} будет замаскировано с помощью дополнительного счётчика RTC_20. Данный способ калибровки заключается в том, что один период RTC_DIV, который приходится на начало интервала 2^{20} , удлиняется на RTC_CAL[7:0] тактов частоты RTC_CLK, или на RTC_CAL[7:0]/RTC_CLK секунд. Таким образом, с помощью битов RTC_CAL[7:0] производится замедление хода часов, при этом единица замедления равна $1/2^{20}$, или $\sim 0,95$ ppm. Чтобы ускорить ход часов необходимо задать значение регистра RTC_PRL меньшее, чем требуется, а затем произвести замедление с помощью битов RTC_CAL[7:0]. Изменение значений битов RTC_CAL[7:0] может быть осуществлено в ходе работы часов реального времени.

Регистр RTC_CNT предназначен для отсчета времени в секундах и работает на выходной частоте предварительного делителя SEC_CLK. Регистр RTC_ALRM предназначен для задания времени, при превышении которого вырабатывается флаг ALRF для прерывания и пробуждения процессора. Таким образом, бит STANDBY, отключающий внутренний регулятор напряжения, автоматически сбрасывается при превышении RTC_CNT значения RTC_ALRM. Бит STANDBY также может быть сброшен с помощью вывода WAKEUP.

Процесс выполнения записи регистров RTC_CNT, RTC_DIV, RTC_PRL и RTC_ALRM отображается с помощью флага WEC в регистре RTC_CS. На время выполнения записи флаг WEC устанавливается в «1», по окончании записи флаг WEC сбрасывается в «0». Пока флаг WEC установлен в «1», новая запись в регистры RTC, а также регистры ВКР, запрещена.

17.2 Регистры аварийного сохранения

Батарейный домен имеет 16 встроенных 32-разрядных регистров аварийного сохранения. 16-й и 15-й регистры служат для хранения бит управления батарейным доменом, оставшиеся 14 регистров могут быть использованы разработчиком программы.

17.3 Описание регистров блока батарейного домена

Таблица 108 – Описание регистров блока батарейного домена

Базовый Адрес	Название	Описание
0x400D_8000	MDR_ВКР	Контроллер батарейного домена и часов реального времени
Смещение		

Базовый Адрес	Название	Описание
0x00	REG_00	Регистр MDR_BKP->REG_0E аварийного сохранения 0
...		
0x38	REG_0E	Регистр MDR_BKP->REG_0E аварийного сохранения 14
0x3C	REG_0F	Регистр MDR_BKP->REG_0F аварийного сохранения 15 и управления блоками RTC, LSE, LSI и HSI
0x40	RTC_CNT	Регистр MDR_BKP->RTC_CNT основного счетчика часов реального времени
0x44	RTC_DIV	Регистр MDR_BKP->RTC_DIV счётчика предварительного делителя
0x48	RTC_PRL	Регистр MDR_BKP->RTC_PRL коэффициента деления предварительного делителя
0x4C	RTC_ALRM	Регистр MDR_BKP->RTC_ALRM значения для сравнения с основным счетчиком RTC_CNT и выработки сигнала ALRF
0x50	RTC_CS	Регистр MDR_BKP->RTC_CS управления и состояния флагов часов реального времени

17.3.1 MDR_BKP->REG_[0D...00]

MDR_BKP->REG_00
MDR_BKP->REG_01
MDR_BKP->REG_02
MDR_BKP->REG_03
MDR_BKP->REG_04
MDR_BKP->REG_05
MDR_BKP->REG_06
MDR_BKP->REG_07
MDR_BKP->REG_08
MDR_BKP->REG_09
MDR_BKP->REG_0A
MDR_BKP->REG_0B
MDR_BKP->REG_0C
MDR_BKP->REG_0D

Таблица 109 – Регистры REG_[0D...00]

Номер	31...0
Доступ	R/W
Сброс	U
	BKP REG[31:0]

Таблица 110 – Описание бит регистров REG_[0D...00]

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...0	BKP REG[31:0]	Регистр аварийного сохранения

17.3.2 MDR_BKP->REG_0E

Таблица 111 – Регистр REG_0E

Номер	31...15	14...12	11	10...8	7	6	5...3	2...0
Доступ	U	R/W	R/S1	R/W	R/W	R/W	R/W	R/W
Сброс	0	U	1	0	1	0	0	0
	-	MODE [2:0]	FPOR	Trim [2:0]	JTAG_B	JTAG_A	SelectRI [2:0]	LOW [2:0]

Таблица 112 – Описание бит регистра REG_0E

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...15	-	Зарезервировано
14...12	MODE[2..0]	Режим работы микросхемы, определенный при включении питания по выводам MODE[2:0] (PF[6:4]): 000 – микроконтроллер в режиме отладки через JTAG_B; 001 – микроконтроллер в режиме отладки через JTAG_A; 010 – микропроцессор в режиме отладки через JTAG_B; 011 – микропроцессор в режиме отладки через JTAG_B; 100 – зарезервировано; 101 – UART-загрузчик без отладки; 110 – UART-загрузчик без отладки; 111 – тестовый режим работы. Подробнее см. в разделе 10 «Загрузочное ПЗУ и режимы работы микросхемы». При смене режима работы в данном регистре, изменение вступит в силу после сброса процессора через RESET, при сбросе по просадке питания режим будет определяться выводами MODE[2:0]
11	FPOR	Флаг срабатывания POR. Устанавливается в 1 загрузочным ПЗУ при первоначальном включении по появлению питания U _{CC} , при сбросе по питанию устанавливается в 0. Служит для анализа загрузочным ПЗУ, что сейчас идет выполнение программы после системного или программного сброса, либо после сброса по питанию
10...8	Trim[2:0]	Коэффициент настройки опорного напряжения встроенного регулятора напряжения DU _{CC} . Устанавливать только значение LDO_TRIM[2:0], считанное из информационной памяти (см. таблицу 24)
7	JTAG B	Разрешение работы порта JTAG B: 0 – запрещен; 1 – разрешен. При установке в единицу обоих бит JTAG B и JTAG A выбирается тестовый режим работы, при этом микросхема не работает в функциональном режиме (см. подраздел 10.4 «Тестовый режим работы»)
6	JTAG A	Разрешение работы порта JTAG A: 0 – запрещен; 1 – разрешен

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
5...3	SelectRI[2:0]	Выбор дополнительной стабилизирующей нагрузки для встроенного регулятора напряжения DU _{CC} : 000 – ~ 6 кОм (дополнительный ток потребления 300 мкА); 001 – ~ 270 кОм (дополнительный ток потребления 6,6 мкА); 010 – ~ 90 кОм (дополнительный ток потребления 20 мкА); 011 – ~ 24 кОм (дополнительный ток потребления 80 мкА); 100 – ~ 900 кОм (собственное потребление 2 мкА); 101 – ~ 2 кОм (дополнительный ток потребления 900 мкА); 110 – ~ 400 Ом (дополнительный ток потребления 4,4 мА); 111 – ~ 100 Ом (дополнительный ток потребления 19 мА)
2...0	LOW[2:0]	Выбор режима работы встроенного регулятора напряжения DU _{CC} . Значение LOW должно совпадать со значением SelectRI и выставляться в зависимости от тактовой частоты микросхемы: 000 – частота до 10 МГц; 001 – частота до 200 кГц; 010 – частота до 500 кГц; 011 – частота до 1 МГц; 100 – при выключении всех генераторов; 101 – частота до 40 МГц; 110 – частота до 80 МГц; 111 – частота более 80 МГц

17.3.3 MDR_BKP->REG_0F

Таблица 113 – Регистр REG_0F

Номер	31	30	29...24	23	22	21	20...16
Доступ	R/W	R/W	R/W	RO	R/W	RO	R/W
Сброс	0	0	100000	1	1	1	10000
	RTC RESET	STANDBY	HSI TRIM [5:0]	HSI RDY	HSI ON	LSI RDY	LSI TRIM [4:0]

Номер	15	14	13	12...5	4	3, 2	1	0
Доступ	R/W	U	RO	R/W	R/W	R/W	R/W	R/W
Сброс	1	0	0	0	0	0	0	0
	LSI ON	-	LSE RDY	RTC CAL[7:0]	RTC EN	RTC SEL[1:0]	LSE BYP	LSE ON

Таблица 114 – Описание бит регистра REG_0F

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31	RTC RESET	Сброс часов реального времени: 0 – часы не сбрасываются; 1 – часы сбрасываются

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
30	STANDBY	Режим отключения регулятора DU _{СС} : 0 – регулятор включен и выдает напряжение; 1 – регулятор выключен. Триггер сбрасывается флагом ALRF или по низкому уровню на выводе WAKEUP
29...24	HSI TRIM[5:0]	Коэффициент подстройки частоты генератора HSI. Устанавливать только значение HSI_TRIM[5:0], считанное из информационной памяти (см. таблицу 24). Зависимость частоты HSI от значения HSITRIM см. на рисунке 139
23	HSI RDY	Флаг выхода генератора HSI в рабочий режим: 0 – генератор не запущен или не вышел в режим; 1 – генератор работает в рабочем режиме
22	HSI ON	Бит управления генератором HSI: 0 – генератор выключен; 1 – генератор включен
21	LSI RDY	Флаг выхода генератора LSI в рабочий режим: 0 – генератор не запущен или не вышел в режим; 1 – генератор работает в рабочем режиме
20...16	LSI TRIM[4:0]	Коэффициент подстройки частоты генератора LSI: Устанавливать только значение LSI_TRIM[4:0], считанное из информационной памяти (см. таблицу 24). Зависимость частоты LSI от значения LSITRIM см. на рисунке 140
15	LSI ON	Бит управления генератором LSI: 0 – генератор выключен; 1 – генератор включен
14	-	Зарезервировано
13	LSE RDY	Флаг выхода генератора LSE в рабочий режим: 0 – генератор не запущен или не вышел в режим; 1 – генератор работает в рабочем режиме
12...5	RTC CAL[7:0]	Коэффициент подстройки тактовой частоты часов реального времени RTC_CLK. Из каждых 2 ²⁰ тактов RTC_CLK будет замаскировано RTC_CAL тактов: 00000000 – 0 тактов; 00000001 – 1 такт; 11111111 – 255 тактов. Таким образом, если исходная частота RTC_CLK равна 32768,00000 Гц, то после подстройки средняя частота тактирования составляет: RTC_CLK_C = 32768,00000 Гц при RTC_CAL = 0; RTC_CLK_C = 32767,96875 Гц при RTC_CAL = 1; ... RTC_CLK_C = 32760,03125 Гц при RTC_CAL = 255. В общем случае средняя тактовая частота RTC_CLK после подстройки составляет: $RTC_CLK_C = RTC_CLK \cdot (1 - RTC_CAL[7:0] / 2^{20})$ Единица подстройки (замедления) равна 1/2 ²⁰ , или ~0,95 ppm.

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
4	RTC EN	Бит разрешения работы часов реального времени: 0 – работа запрещена; 1 – работа разрешена. После записи задержка изменения бита составляет 1 такт частоты RTC_CLK
3, 2	RTC SEL[1:0]	Биты выбора источника тактовой синхронизации часов реального времени RTC_CLK: 00 – LSI; 01 – LSE; 10 – HSI_RTC (формируется в блоке MDR_RST_CLK); 11 – HSE_RTC (формируется в блоке MDR_RST_CLK)
1	LSE BYP	Бит управления генератором LSE: 0 – режим осциллятора; 1 – режим работы на проход (внешний генератор). Устанавливать совместно с LSE_ON = 1
0	LSE ON	Бит управления генератором LSE: 0 – генератор выключен; 1 – генератор включен

17.3.4 MDR_BKP->RTC_CNT

Таблица 115 – Регистр RTC_CNT

Номер	31...0
Доступ	R/W
Сброс	0
	RTC CNT[31:0]

Таблица 116 – Описание бит регистра RTC_CNT

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...0	RTC CNT[31:0]	Значение основного счетчика часов реального времени

17.3.5 MDR_BKP->RTC_DIV

Таблица 117 – Регистр RTC_DIV

Номер	31...20	19...0
Доступ	U	R/W
Сброс	0	0
	-	RTC DIV[19:0]

Таблица 118 – Описание бит регистра RTC_DIV

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...20	-	Зарезервировано
19...0	RTC DIV[19:0]	Значение счетчика предварительного делителя часов реального времени

17.3.6 MDR_BKP->RTC_PRL

Таблица 119 – Регистр RTC_PRL

Номер	31...20	19...0
Доступ	U	R/W
Сброс	0	0
	-	RTC PRL[19:0]

Таблица 120 – Описание бит регистра RTC_PRL

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...20	-	Зарезервировано
19...0	RTC PRL[19:0]	Коэффициент деления тактовой частоты RTC_CLK: $SEC_CLK = RTC_CLK / (RTC_PRL + 1)$, где RTC_CLK – частота тактирования с учетом подстройки RTC_CAL[7:0]

17.3.7 MDR_BKP->RTC_ALRM

Таблица 121 – Регистр RTC_ALRM

Номер	31...0
Доступ	R/W
Сброс	0
	RTC ALRM[31:0]

Таблица 122 – Описание бит регистра RTC_ALRM

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...0	RTC ALRM[31:0]	Значение для сравнения с основным счетчиком RTC_CNT и выработки сигнала ALRF. Сигнал ALRF вырабатывается в момент превышения основным счётчиком RTC_CNT значения в регистре RTC_ALRM: $RTC_CNT[31:0] == RTC_ALRM[31:0] + 1$

17.3.8 MDR_BKP->RTC_CS

Таблица 123 – Регистр RTC_CS

Номер	31...7	6	5	4	3	2	1	0
Доступ	U	RO	R/W	R/W	R/W	R/C1	R/C1	R/C1
Сброс	0	0	0	0	0	0	0	0
	-	WEC	ALRF_IE	SECF_IE	OWF_IE	ALRF	SECF	OWF

Таблица 124 – Описание бит регистра RTC_CS

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
30...7	-	Зарезервировано
6	WEC	Флаг выполнения записи в регистры RTC (RTC_CNT, RTC_DIV, RTC_PRL, RTC_ALRM): 0 – нет записи в регистр RTC; 1 – идет запись в регистр RTC, новая запись в регистры RTC и ВКР запрещена
5	ALRF_IE	Флаг разрешения прерывания по событию ALRF: 0 – прерывание запрещено; 1 – прерывание разрешено
4	SECF_IE	Флаг разрешения прерывания по событию SECF: 0 – прерывание запрещено; 1 – прерывание разрешено
3	OWF_IE	Флаг разрешения прерывания по событию OWF: 0 – прерывание запрещено; 1 – прерывание разрешено
2	ALRF	Флаг превышения основным счетчиком RTC_CNT значения RTC_ALRM: 0 – нет события превышения; 1 – было событие превышения. Флаг устанавливается при: RTC_CNT[31:0] == RTC_ALRM[31:0] + 1. Сброс флага осуществляется записью 1
1	SECF	Флаг совпадения значений регистров RTC_DIV и RTC_PRL: 0 – не было совпадения; 1 – было совпадение. По событиям совпадения RTC_DIV и RTC_PRL формируется частота SEC_CLK. Таким образом флаг SECF устанавливается на каждом такте частоты SEC_CLK. Сброс флага осуществляется записью 1
0	OWF	Флаг переполнения основного счетчика RTC_CNT: 0 – нет переполнения; 1 – было переполнение. Сброс флага осуществляется записью 1

18 Порты ввода-вывода MDR_PORTx

Микросхема имеет шесть портов ввода/вывода. Порты 16-разрядные и их выходы мультиплексируются между различными функциональными блоками, управление для каждого вывода отдельное. Для того, чтобы выходы порта перешли под управление того или иного периферийного блока, необходимо задать для нужных выводов выполняемую функцию и настройки.

При работе в режиме отладки не допускается изменение функций записью «1» в регистры RXTX и OE выводов, совмещенных с выводами JTAG. Это может привести к блокировке интерфейса отладки.

Таблица 125 – Порты ввода-вывода

Вывод	Аналоговая функция ANALOG_EN=0	Цифровая функция					
		Порт IO MODE[1:0]=00 ANALOG_EN=1	Основная MODE[1:0]=01 ANALOG_EN=1	Альтернативная MODE[1:0]=10 ANALOG_EN=1	Переопределенная MODE[1:0]=11 ANALOG_EN=1		
Порт A							
PA0	-	PA0	DATA0	1)	EXT_INT1	9)	-
PA1	-	PA1	DATA1		TMR1_CH1	3)	TMR2_CH1 15)
PA2	-	PA2	DATA2		TMR1_CH1N		TMR2_CH1N
PA3	-	PA3	DATA3		TMR1_CH2		TMR2_CH2
PA4	-	PA4	DATA4		TMR1_CH2N		TMR2_CH2N
PA5	-	PA5	DATA5		TMR1_CH3		TMR2_CH3
PA6	-	PA6	DATA6		CAN1_TX	2)	UART1_RXD 10)
PA7	-	PA7	DATA7		CAN1_RX		UART1_TXD
PA8	-	PA8	DATA8		TMR1_CH3N	3)	TMR2_CH3N 15)
PA9	-	PA9	DATA9		TMR1_CH4		TMR2_CH4
PA10	-	PA10	DATA10		nUART1DTR	10)	TMR2_CH4N
PA11	-	PA11	DATA11		nUART1RTS		TMR2_BLK
PA12	-	PA12	DATA12		nUART1RI		TMR2_ETR
PA13	-	PA13	DATA13		nUART1DCD		TMR1_CH4N 3)
PA14	-	PA14	DATA14		nUART1DSR		TMR1_BLK
PA15	-	PA15	DATA15		nUART1CTS		TMR1_ETR
Порт B							
PB0	-	PB0 JA_TDO	DATA16	1)	TMR3_CH1	12)	UART1_TXD 10)
PB1	-	PB1 JA_TMS	DATA17		TMR3_CH1N		UART2_RXD 14)
PB2	-	PB2 JA_TCK	DATA18		TMR3_CH2		CAN1_TX 2)
PB3	-	PB3 JA_TDI	DATA19		TMR3_CH2N		CAN1_RX
PB4	-	PB4 JA_TRST	DATA20		TMR3_BLK		TMR3_ETR 12)
PB5	-	PB5	DATA21		UART1_TXD	10)	TMR3_CH3
PB6	-	PB6	DATA22		UART1_RXD		TMR3_CH3N
PB7	-	PB7	DATA23		nSIROUT1		TMR3_CH4
PB8	-	PB8	DATA24		COMP_OUT	7)	TMR3_CH4N
PB9	-	PB9	DATA25		nSIRIN1	10)	EXT_INT4 9)
PB10	-	PB10	DATA26		EXT_INT2	9)	nSIROUT1 10)
PB11	-	PB11	DATA27		EXT_INT1		COMP_OUT 7)
PB12	-	PB12	DATA28		SSP1_FSS	16)	SSP2_FSS 13)
PB13	-	PB13	DATA29		SSP1_CLK		SSP2_CLK
PB14	-	PB14	DATA30		SSP1_RXD		SSP2_RXD
PB15	-	PB15	DATA31		SSP1_TXD		SSP2_TXD

Вывод	Аналоговая функция ANALOG_EN=0	Цифровая функция				
		Порт IO MODE[1:0]=00 ANALOG_EN=1	Основная MODE[1:0]=01 ANALOG_EN=1	Альтернативная MODE[1:0]=10 ANALOG_EN=1	Переопределенная MODE[1:0]=11 ANALOG_EN=1	
Порт C						
PC0	-	PC0	READY ¹⁷⁾	1) SCL1	11) SSP2_FSS	13)
PC1	-	PC1	OE	SDA1	SSP2_CLK	
PC2	-	PC2	WE	TMR3_CH1	12) SSP2_RXD	
PC3	-	PC3	BE0	TMR3_CH1N	SSP2_TXD	
PC4	-	PC4	BE1	TMR3_CH2	TMR1_CH1	3)
PC5	-	PC5	BE2	TMR3_CH2N	TMR1_CH1N	
PC6	-	PC6	BE3	TMR3_CH3	TMR1_CH2	
PC7	-	PC7	CLOCK	TMR3_CH3N	TMR1_CH2N	
PC8	-	PC8	CAN1_TX	2) TMR3_CH4	TMR1_CH3	
PC9	-	PC9	CAN1_RX	TMR3_CH4N	TMR1_CH3N	
PC10	-	PC10	-	TMR3_ETR	TMR1_CH4	
PC11	-	PC11	-	TMR3_BLK	TMR1_CH4N	
PC12	-	PC12	-	EXT_INT2	9) TMR1_ETR	
PC13	-	PC13	-	EXT_INT4	TMR1_BLK	
PC14	-	PC14	-	SSP2_FSS	13) CAN2_RX	4)
PC15	-	PC15	-	SSP2_RXD	CAN2_TX	
Порт D						
PD0	ADC0_REF+	5) PD0 JB_TMS	TMR1_CH1N	3) UART2_RXD	14) TMR3_CH1	12)
PD1	ADC1_REF-	PD1 JB_TCK	TMR1_CH1	UART2_TXD	TMR3_CH1N	
PD2	ADC2	PD2 JB_TRST	BUSY1	1) SSP2_RXD	13) TMR3_CH2	
PD3	ADC3	PD3 JB_TDI	-	SSP2_FSS	TMR3_CH2N	
PD4	ADC4	PD4 JB_TDO	TMR1_ETR	3) nSIROUT2	14) TMR3_BLK	
PD5	ADC5	PD5	CLE	1) SSP2_CLK	13) TMR2_ETR	15)
PD6	ADC6	PD6	ALE	SSP2_TXD	13) TMR2_BLK	
PD7	ADC7	PD7	TMR1_BLK	3) nSIRIN2	14) UART1_RXD	10)
PD8	ADC8	PD8	TMR1_CH4N	TMR2_CH1	15) UART1_TXD	
PD9	ADC9	PD9	CAN2_TX	4) TMR2_CH1N	SSP1_FSS	16)
PD10	ADC10	PD10	TMR1_CH2	3) TMR2_CH2	SSP1_CLK	
PD11	ADC11	PD11	TMR1_CH2N	TMR2_CH2N	SSP1_RXD	
PD12	ADC12	PD12	TMR1_CH3	TMR2_CH3	SSP1_TXD	
PD13	ADC13	PD13	TMR1_CH3N	TMR2_CH3N	CAN1_TX	2)
PD14	ADC14	PD14	TMR1_CH4	TMR2_CH4	CAN1_RX	
PD15	ADC15	PD15	CAN2_RX	4) BUSY2	1) EXT_INT3	9)
Порт E						
PE0	DAC2_OUT	6) PE0	ADDR16	1) TMR2_CH1	15) CAN1_RX	2)
PE1	DAC2_REF	PE1	ADDR17	TMR2_CH1N	CAN1_TX	
PE2	COMP_IN1	7) PE2	ADDR18	TMR2_CH3	TMR3_CH1	12)
PE3	COMP_IN2	PE3	ADDR19	TMR2_CH3N	TMR3_CH1N	
PE4	COMP_REF+	PE4	ADDR20	TMR2_CH4N	TMR3_CH2	
PE5	COMP_REF-	PE5	ADDR21	TMR2_BLK	TMR3_CH2N	
PE6	OSC_IN32	8) PE6	ADDR22	CAN2_RX	4) TMR3_CH3	
PE7	OSC_OUT32	PE7	ADDR23	CAN2_TX	TMR3_CH3N	
PE8	COMP_IN3	7) PE8	ADDR24	TMR2_CH4	15) TMR3_CH4	
PE9	DAC1_OUT	6) PE9	ADDR25	TMR2_CH2	TMR3_CH4N	
PE10	DAC1_REF	PE10	ADDR26	TMR2_CH2N	TMR3_ETR	

Выход	Аналоговая функция ANALOG_EN=0	Цифровая функция					
		Порт IO MODE[1:0]=00 ANALOG_EN=1	Основная MODE[1:0]=01 ANALOG_EN=1	Альтернативная MODE[1:0]=10 ANALOG_EN=1	Переопределенная MODE[1:0]=11 ANALOG_EN=1		
PE11	-	PE11	ADDR27	¹⁾ nSIRIN1	¹⁰⁾ TMR3_BLK	¹²⁾	
PE12	-	PE12	ADDR28	SSP1_RXD	¹⁶⁾ UART1_RXD	¹⁰⁾	
PE13	-	PE13	ADDR29	SSP1_FSS	UART1_TXD		
PE14	-	PE14	ADDR30	TMR2_ETR	¹⁵⁾ SCL1	¹¹⁾	
PE15	-	PE15	ADDR31	EXT_INT3	⁹⁾ SDA1		
Порт F							
PF0	-	PF0	ADDR0	¹⁾ SSP1TXD	¹⁶⁾ UART2_RXD	¹⁴⁾	
PF1	-	PF1	ADDR1	SSP1CLK	UART2_TXD		
PF2	-	PF2	ADDR2	SSP1FSS	CAN2_RX	⁴⁾	
PF3	-	PF3	ADDR3	SSP1RXD	CAN2_TX		
PF4	-	PF4 MODE[0]	ADDR4	-	-		
PF5	-	PF5 MODE[1]	ADDR5	-	-		
PF6	-	PF6 MODE[2]	ADDR6	TMR1_CH1	³⁾ -		
PF7	-	PF7	ADDR7	TMR1_CH1N	TMR3_CH1	¹²⁾	
PF8	-	PF8	ADDR8	TMR1_CH2	TMR3_CH1N		
PF9	-	PF9	ADDR9	TMR1_CH2N	TMR3_CH2		
PF10	-	PF10	ADDR10	TMR1_CH3	TMR3_CH2N		
PF11	-	PF11	ADDR11	TMR1_CH3N	TMR3_ETR		
PF12	-	PF12	ADDR12	TMR1_CH4	SSP2_FSS		
PF13	-	PF13	ADDR13	TMR1_CH4N	SSP2_CLK		
PF14	-	PF14	ADDR14	TMR1_ETR	SSP2_RXD		
PF15	-	PF15	ADDR15	TMR1_BLK	SSP2_TXD		
Примечания 1) Выводы управляются системной шиной EXT_BUS; 2) Выводы управляются контроллером интерфейса CAN1; 3) Выводы управляются Таймером 1; 4) Выводы управляются контроллером интерфейса CAN2; 5) Выводы используются АЦП; 6) Выводы используются ЦАП; 7) Выводы используются Компаратором; 8) Выводы используются генератором LSE; 9) Выводы используются контроллером прерываний; 10) Выводы управляются контроллером интерфейса UART1; 11) Выводы управляются контроллером интерфейса I2C; 12) Выводы управляются Таймером 3; 13) Выводы управляются контроллером интерфейса SSP2; 14) Выводы управляются контроллером интерфейса UART2; 15) Выводы управляются Таймером 2; 16) Выводы управляются контроллером интерфейса SSP1; 17) Реализовано только в K1986BE94GI							

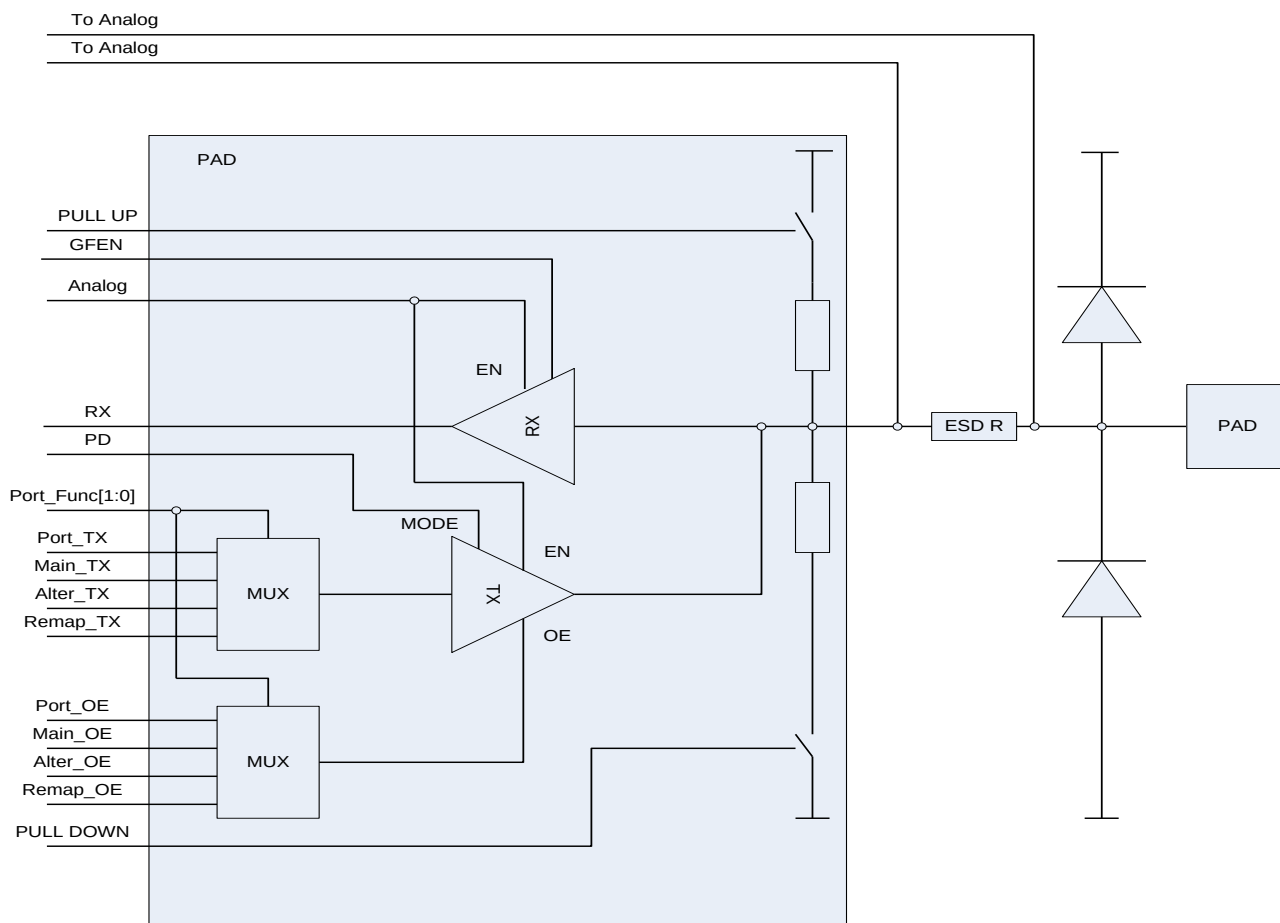


Рисунок 30 – Порты ввода-вывода

18.1 Описание регистров портов ввода-вывода

Таблица 126 – Описание регистров портов ввода-вывода

Базовый адрес	Название	Описание
0x400A_8000	MDR_PORTA	Порт А
0x400B_0000	MDR_PORTB	Порт В
0x400B_8000	MDR_PORTC	Порт С
0x400C_0000	MDR_PORTD	Порт D
0x400C_8000	MDR_PORTE	Порт E
0x400E_8000	MDR_PORTF	Порт F
Смещение		
0x00	RXTX[15:0]	MDR_PORTx->RXTX Данные порта
0x04	OE[15:0]	MDR_PORTx->OE Направление порта
0x08	FUNC[31:0]	MDR_PORTx->FUNC Режим работы порта
0x0C	ANALOG[15:0]	MDR_PORTx->ANALOG Аналоговый режим работы порта
0x10	PULL[31:0]	MDR_PORTx->PULL Подтяжка порта
0x14	PD[31:0]	MDR_PORTx->PD Режим работы выходного драйвера
0x18	PWR[31:0]	MDR_PORTx->PWR Режим мощности передатчика
0x1C	GFEN[15:0]	MDR_PORTx->GFEN Режим работы входного фильтра

18.1.1 MDR_PORTx->RXTX

Таблица 127 – Регистр RXTX

Номер	31...16	15...0
Доступ	U	R/W
Сброс	0	0
	-	PORT RXTX[15:0]

Таблица 128 – Описание бит регистра RXTX

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...16	-	Зарезервировано
15...0	PORT RXTX[15:0]	Режим работы контроллера. Данные для выдачи на выходы порта и для чтения. Биты читаются как 1, если соответствующие им выходы настроены в аналоговый режим работы

18.1.2 MDR_PORTx->OE

Таблица 129 – Регистр OE

Номер	31...16	15...0
Доступ	U	R/W
Сброс	0	0
	-	PORT OE[15:0]

Таблица 130 – Описание бит регистра OE

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...16	-	Зарезервировано
15...0	PORT OE[15:0]	Режим работы контроллера. Направление передачи данных на выводах порта: 1 – выход; 0 – вход

18.1.3 MDR_PORTx->FUNC

Таблица 131 – Регистр FUNC

Номер	31	30	...	3	2	1	0
Доступ	R/W	R/W	...	R/W	R/W	R/W	R/W
Сброс	0	0	...	0	0	0	0
	MODE15[1:0]		...	MODE1[1:0]		MODE0[1:0]	

Таблица 132 – Описание бит регистра FUNC

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...2	MODEx	Аналогично MODE0 для остальных бит порта
1...0	MODE0[1:0]	Режим работы вывода порта: 00 – порт; 01 – основная функция; 10 – альтернативная функция 11 – переопределенная функция

18.1.4 MDR_PORTx->ANALOG

Таблица 133 – Регистр ANALOG

Номер	31...16	15...0
Доступ	U	R/W
Сброс	0	0
	-	ANALOG EN[15:0]

Таблица 134 – Описание бит регистра ANALOG

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...16	-	Зарезервировано
15...0	ANALOG EN[15:0]	Режим работы контроллера: 0 – аналоговый; 1 – цифровой

18.1.5 MDR_PORTx->PULL

Таблица 135 – Регистр PULL

Номер	31...16	15...0
Доступ	R/W	R/W
Сброс	0	0
	PULL UP[15:0]	PULL DOWN[15:0]

Таблица 136 – Описание бит регистра PULL

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...16	PULL UP[15:0]	Режим работы контроллера. Разрешение подтяжки вверх: 0 – подтяжка в питание выключена; 1 – подтяжка в питание включена (есть подтяжка ~50 кОм)
15...0	PULL DOWN[15:0]	Режим работы контроллера. Разрешение подтяжки вниз: 0 – подтяжка в ноль выключена; 1 – подтяжка в ноль включена (есть подтяжка ~ 50 кОм)

18.1.6 MDR_PORTx->PD

Таблица 137 – Регистр PD

Номер	31...16	15...0
Доступ	R/W	R/W
Сброс	0	0
	PORT SHM[15:0]	PORT PD[15:0]

Таблица 138 – Описание бит регистра PD

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...16	PORT SHM[15:0]	Режим работы триггера Шмитта на входе: 0 – гистерезис 200 мВ; 1 – гистерезис 400 мВ
15...0	PORT PD[15:0]	Режим работы контроллера. Режим работы выхода: 0 – управляемый драйвер; 1 – открытый сток

18.1.7 MDR_PORTx->PWR

Таблица 139 – Регистр PWR

Номер	31	30	...	3	2	1	0
Доступ	R/W	R/W	...	R/W	R/W	R/W	R/W
Сброс	0	0	...	0	0	0	0
	PWR15[1:0]		...	PWR1[1:0]		PWR0[1:0]	

Таблица 140 – Описание бит регистра PORTx_PWR

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...2	PWRx	Аналогично PWR0 для остальных бит порта
1...0	PWR0[1:0]	Режим работы вывода порта: 00 – зарезервировано (передатчик отключен) 01 – медленный фронт (порядка 100 нс) 10 – быстрый фронт (порядка 20 нс) 11 – максимально быстрый фронт (порядка 10 нс)

18.1.8 MDR_PORTx->GFEN

Таблица 141 – Регистр GFEN

Номер	31...16	15...0
Доступ	R/W	R/W
Сброс	0	0
	-	GFEN[15:0]

Таблица 142 – Описание бит регистра GFEN

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...16	-	Зарезервировано
15...0	GFEN[15:0]	Режим работы входного фильтра: 0 – фильтр выключен; 1 – фильтр включен (фильтрация импульсов до 10 нс)

19 Детектор напряжения питания MDR_POWER

Блок детектора напряжения питания PVD предназначен для контроля питания U_{CC} и BU_{CC} при работе микросхемы. Блок PVD позволяет сравнивать внешние уровни напряжения с внутренними опорными уровнями и в случае превышения или снижения ниже опорного уровня выработать сигнал или прерывание для программной обработки.

Уровень опорного напряжения для сравнения с U_{CC} задается битами PLS[2:0] в регистре PVDCS, для сравнения с BU_{CC} задается битами PLBS[1:0] в регистре PVDCS. В соответствии с уровнями напряжения формируются флаги PVD и PVBD. Данные флаги выставляются при возникновении события и сбрасываются программно.

Таблица 143 – Типовые уровни напряжений детектора питания

Параметр	Не менее	Типовое	Не более
Входное напряжение, U_{CC} , В	2,0	-	3,6
Входное напряжение, BU_{CC} , В	1,8	-	3,6
Уровень срабатывания PVD от U_{CC} , при PLS = «000», В		2,0	
Уровень срабатывания PVD от U_{CC} , при PLS = «001», В		2,2	
Уровень срабатывания PVD от U_{CC} , при PLS = «010», В		2,4	
Уровень срабатывания PVD от U_{CC} , при PLS = «011», В		2,6	
Уровень срабатывания PVD от U_{CC} , при PLS = «100», В		2,8	
Уровень срабатывания PVD от U_{CC} , при PLS = «101», В		3,0	
Уровень срабатывания PVD от U_{CC} , при PLS = «110», В		3,2	
Уровень срабатывания PVD от U_{CC} , при PLS = «111», В		3,4	
Уровень срабатывания PVBD от BU_{CC} , при PBLBS = «00», В		1,8	
Уровень срабатывания PVBD от BU_{CC} , при PBLBS = «01», В		2,2	
Уровень срабатывания PVBD от BU_{CC} , при PBLBS = «10», В		2,6	
Уровень срабатывания PVBD от BU_{CC} , при PBLBS = «11», В		3,0	

Таблица 144 – Описание регистров блока PVD

Базовый Адрес	Название	Описание
0x4005_8000	MDR_POWER	Датчик подсистемы питания
Смещение		
0x00	PVDCS [12:0]	Регистр MDR_POWER->PVDCS управления и состояния датчика питания

19.1.1 MDR_POWER->PVDCS

Таблица 145 – Регистр PVDCS

Номер	31...12	11	10
Доступ	U	R/W	R/W
Сброс	0	0	0
	-	INV	INVB

Номер	9	8	7	6	5...3	2, 1	0
Доступ	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Сброс	0	0	0	0	000	00	0
	IEPVD	IEPVBD	PVD	PVBD	PLS [2:0]	PBLBS [1:0]	PVD EN

Таблица 146 – Описание бит регистра PVDCS

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...12	-	Зарезервировано
11	INV	Флаг инверсии выхода от датчика PVD: 0 – нет инверсии; 1 – инверсия. Если флаг не инвертируется, то флаг выставляется при превышении заданного уровня, если инвертируется, то при снижении ниже заданного уровня
10	INVB	Флаг инверсии выхода от датчика PVBD: 0 – нет инверсии; 1 – инверсия. Если флаг не инвертируется, то флаг выставляется при превышении заданного уровня, если инвертируется, то при снижении ниже заданного уровня
9	IEPVD	Флаг разрешения прерывания от датчика PVD: 0 – прерывание запрещено; 1 – прерывание разрешено. Очищается записью 0, если при очистке, датчик продолжает выдавать сигнал, то флаг не будет очищен
8	IEPVBD	Флаг разрешения прерывания от датчика PVBD: 0 – прерывание запрещено; 1 – прерывание разрешено. Очищается записью 0, если при очистке, датчик продолжает выдавать сигнал, то флаг не будет очищен
7	PVD	Результат сравнения напряжения основного питания Устанавливается событием, очищается записью 0. Если событие продолжается запись игнорируется: 0 – напряжение питания меньше чем уровень, задаваемый PLS; 1 – напряжение питания больше чем уровень, задаваемый PLS Примечание – сброс результата сравнения необходимо проводить с подтверждением - сбрасывать дважды.
6	PVBD	Результат сравнения напряжения батарейного питания Устанавливается событием, очищается записью 0. Если событие продолжается запись игнорируется: 0 – напряжение питания меньше чем уровень, задаваемый PLS; 1 – напряжение питания больше чем уровень, задаваемый PLS Примечание – сброс результата сравнения необходимо проводить с подтверждением - сбрасывать дважды.
5...3	PLS[2:0]	Уровень напряжения для сравнения с напряжением основного питания: 000 – 2,0 В 001 – 2,2 В 010 – 2,4 В 011 – 2,6 В 100 – 2,8 В 101 – 3,0 В 110 – 3,2 В 111 – 3,4 В

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
2, 1	PBLS[1:0]	Уровень напряжения для сравнения с напряжением батарейного питания: 00 – 1,8 В 01 – 2,2 В 10 – 2,6 В 11 – 3,0 В
0	PVDEN	Бит разрешения работы блока датчика напряжения питания: 0 – датчик отключен; 1 – датчик включен

20 Внешняя системная шина MDR_EBC

Внешняя системная шина позволяет работать с внешними микросхемами памяти и периферийными устройствами. Области адресного пространства микросхемы отведены для работы с внешней системной шиной.

Таблица 147 – Адресные диапазоны внешней системной шины

Адресный диапазон	Размер	Описание
0x1000_0000 – 0x1FFF_FFFF	256 Мбайт	Область памяти секции CODE отображаемая на внешнюю системную шину с доступом через I Code и D code шины. В режиме микропроцессора из этой области начинает выполняться программа
0x3000_0000 – 0x3FFF_FFFF	256 Мбайт	Область памяти секции DATA отображаемая на внешнюю системную шину с доступом через S Bus. К этой области имеет доступ DMA-контроллер
0x5000_0000 – 0x5FFF_FFFF	256 Мбайт	Область памяти секции PERIPHERAL и EXTERNAL BUS отображаемая на внешнюю системную шину с доступом через S Bus. К этой области имеет доступ DMA-контроллер
0x6000_0000 – 0xDFFF_FFFF	2 Гбайт	Область памяти секции PERIPHERAL и EXTERNAL BUS отображаемая на внешнюю системную шину с доступом через S Bus. К этой области имеет доступ DMA-контроллер

Контроллер внешней системной шины во всех режимах не формирует сигналов выборки чипа CE. При работе с внешними статическими ОЗУ, ПЗУ и периферийными устройствами в качестве сигнала выборки чипа можно использовать старшие линии шины адреса, не используемые для непосредственной адресации, либо использовать программно управляемые выходы портов для формирования сигналов CE. Для каждого из 4 регионов можно задавать собственные настройки длительности транзакции на шине.

20.1 Работа с внешними статическими ОЗУ, ПЗУ и периферийными устройствами

Для работы контроллера внешней системной шины с внешними микросхемами статического ОЗУ, ПЗУ или внешними периферийными устройствами необходимо задать режим работы через регистр EXT_BUS_CONROL. Бит RAM разрешает работу с внешними ОЗУ, бит ROM разрешает только чтение внешних ОЗУ или ПЗУ. В зависимости от скорости работы ядра микросхемы и внешних устройств необходимо задать времена транзакции на внешней системной шине через биты WAIT_STATE[3:0] при EXT_BUS_CTRL_MODE=0 или ENABLE=0 для соответствующего региона, или через биты WS_ACTIVE[6:0], WS_SETUP[2:0] и WS_HOLD[2:0] при EXT_BUS_CTRL_MODE=1 или ENABLE=1 для соответствующего региона. После этого все обращения в область памяти, отображаемой на внешнюю системную шину, будут транслироваться на выходы внешней системной шины ADDR, DATA и сигналы управления OE, WE, BE[3:0] и сигнал синхронизации CLOCK. На вывод CLOCK могут выдаваться одиночные импульсы внутри транзакции при EXT_BUS_CTRL_MODE=0

либо непрерывно тактовая частота ядра при EXT_BUS_CTRL_MODE=1 и CLOCKOUT_EN=1. Полярность сигнала на выходе CLOCK задается битом CPOL.

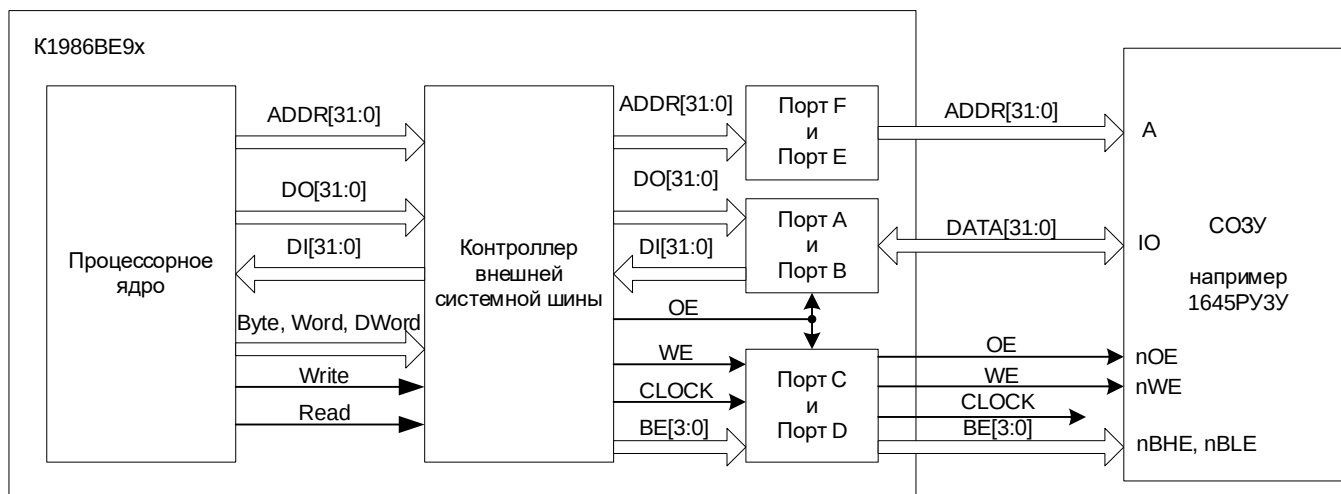


Рисунок 31 – Обмен по внешней системной шине при задании длительности через биты WAIT_STATE

Время цикла записи t_{cycle} задается битами WAIT_STATE[3:0]. Активный уровень сигналов WE, OE, BE[3:0] низкий. Если сигнал CLOCK не требуется, он может не использоваться.

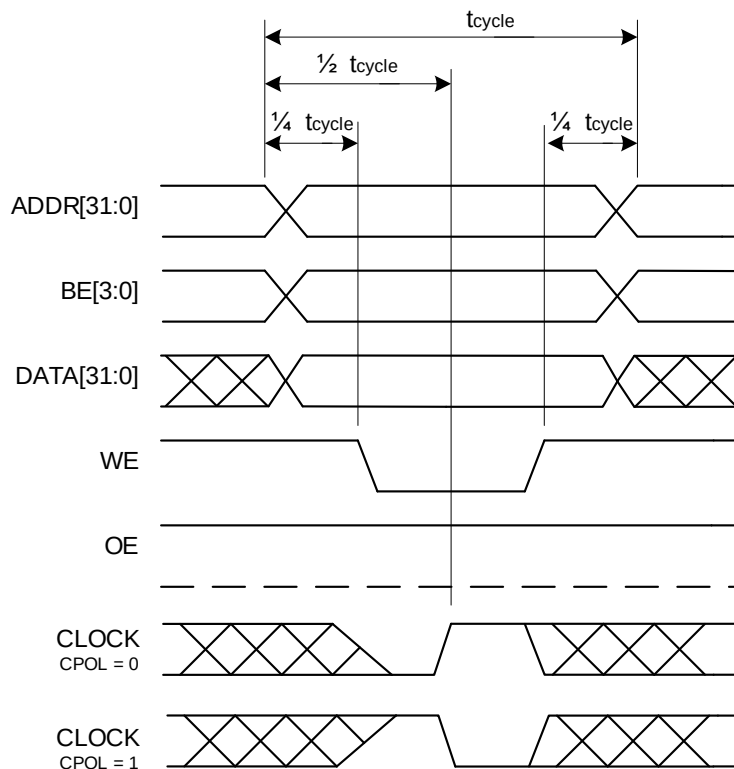


Рисунок 32 – Диаграмма записи

При чтении по внешней системной шине необходимо выбрать такую длительность времени t_{cycle} , чтобы выполнялось время скорости доступа к памяти. Время t_{dh} для микросхемы равно нулю.

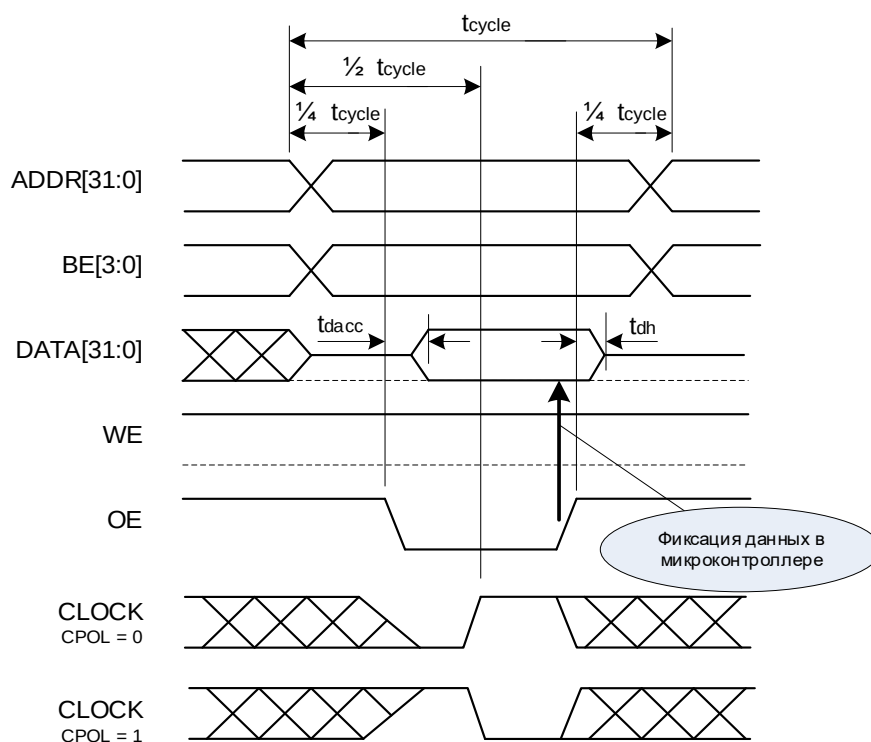


Рисунок 33 – Диаграмма чтения

Для уменьшения задержек в работе контроллера внешней системной шины направление шины данных вне цикла чтения/записи (неактивное состояние) зависит от последнего обращения:

- если выполнялась запись, то после цикла записи шина данных продолжает работать на выход, значение DATA[31:0] неопределено;
- если выполнялось чтение, то после цикла чтения шина данных продолжает работать на вход.

Шина адреса вне цикла чтения/записи работает на выход, значение ADDR[31:0] = 0xFFFF_FFFF.

Таблица 148 – Длительность фаз обращения в тактах процессора при EXT_BUS_CTRL_MODE=0 или ENABLE=0 для соответствующего региона

WAIT_STATE	Предустановка адреса и данных перед сигналом WE или OE	Длительность WE или OE	Удержание адреса и данных после сигнала WE или OE (не менее)
0	1	1	0
1	1	1	1
2	1	1	1
3	1	2	1
4	2	2	1
5	2	3	1
6	2	3	2
7	2	4	2
8	3	4	2
9	3	5	2
10	3	5	3
11	3	6	3
12	4	6	3

WAIT_STATE	Предустановка адреса и данных перед сигналом WE или OE	Длительность WE или OE	Удержание адреса и данных после сигнала WE или OE (не менее)
13	4	7	3
14	4	7	4
15	4	8	4

20.1.1 Обмен по внешней системной шине при задании длительности через WS_ACTIVE, WS_SETUP, WS_HOLD

Каждый адресный диапазон имеет собственный регистр конфигурации контроллера внешней шины REGx_CNTRL, содержащий следующие основные поля:

- WS_SETUP [2:0] – задает время предустановки сигналов WE/OE в цикле записи/чтения, выраженное в количестве тактов системной частоты (WS_SETUP + 1) в диапазоне от 1 до 8;
- WS_ACTIVE [6:0] – задает длительность низкого уровня сигналов WE/OE в цикле записи/чтения, выраженное в количестве тактов системной частоты (WS_ACTIVE + 1) в диапазоне от 1 до 128;
- WS_HOLD [2:0] – задает время удержания сигналов WE/OE в цикле записи/чтения, выраженное в количестве тактов системной частоты (WS_HOLD + 1) в диапазоне от 1 до 8;
- ENABLE – бит разрешения использования собственных настроек длительности:
 - 0 – для определения длительности используется биты WAIT_STATE;
 - 1 – при EXT_BUS_CNTRL_MODE = 1 для определения длительности используется биты WS_ACTIVE, WS_SETUP и WS_HOLD.

20.2 Работа с внешней NAND Flash-памятью

Для работы контроллера внешней системной шины с внешними NAND Flash микросхемами памяти необходимо задать режим работы через регистр EXT_BUS_CNTRL. Бит NAND разрешает работу с внешними NAND Flash микросхемами. В зависимости от скорости работы ядра микросхемы и внешних устройств необходимо задать времена выполнения различных этапов работы NAND Flash-памяти через регистр NAND_CYCLES. После этого обращения в область памяти, отображаемой на внешнюю системную шину, будут перекодироваться в командные, адресные и обмена данными циклы обращения с NAND Flash через выходы внешней системной шины DATA[7:0], ALE, CLE, BUSY1 и BUSY2.

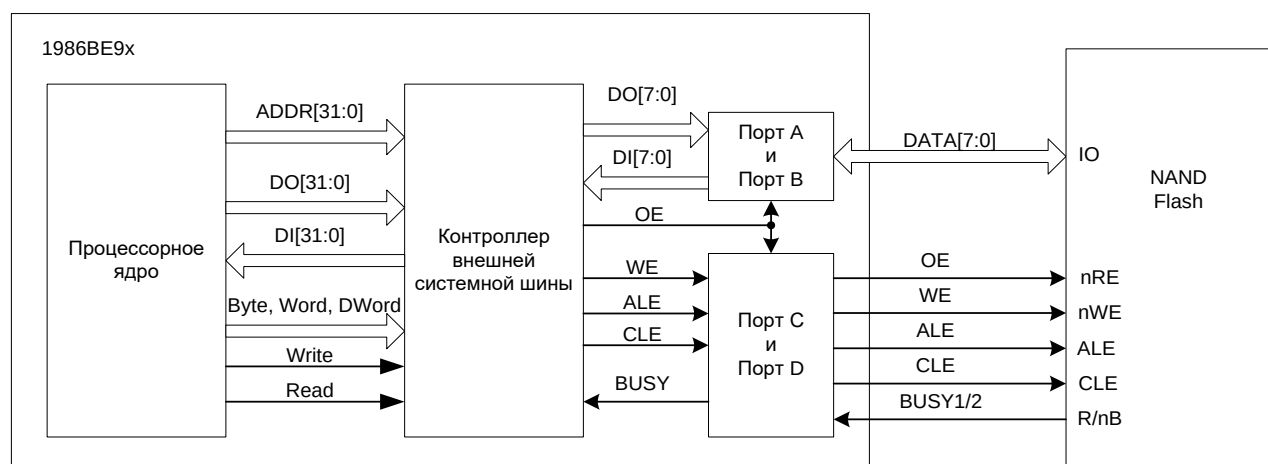


Рисунок 34 – Подключение внешней NAND Flash

Контроллер имеет два сигнала BUSY1 и BUSY2 для возможности подключения двух независимых микросхем NAND Flash. Оба сигнала объединяются по логическому И внутри контроллера и формируют общий сигнал BUSY. Если использование второго сигнала BUSY не требуется, то достаточно не задавать соответствующую функцию вывода порта D (BUSY1 – PD2 (основная функция) и BUSY2 – PD15 (альтернативная функция)).

При работе с NAND Flash-памятью тип выполняемой операции кодируется адресом обращения, а данные и адрес передаются данными при записи и чтении памяти. Формат кодирования адреса обращения представлен в таблице 149.

Таблица 149 – Формат кодирования адреса обращения

Адрес обращения	Фаза команды	Фаза данных
ADDR[31:24]	Не имеет значения, но должно попадать в адресные диапазоны внешней системной шины: 0x10...0x1F 0x30...0x3F 0x50...0xCF	
ADDR[23:21]	ADR_CYCLES[2:0] 000 – 0 циклов 001 – 1 цикл ... 111 – 7 циклов	Не имеет значения
ADDR[20]	Выполнение завершающей команды: 0 – не выполнять; 1 – выполнять	
ADDR[19]	Всегда 0	Всегда 1
ADDR[18:11]	Код завершающей команды ECMD[7:0] 0x10/0x11 – Page Program 0xD0 – Block Erase	
ADDR[10:3]	Код начальной команды SCMD[7:0] 0x00/0x01 – Read1 0x50 – Read2 0x90 – Read ID 0xFF – Reset 0x80 – Page Program 0x60 – Block Erase	Не имеет значения

	0x70 – Read Status	
ADDR[2:0]	Не имеет значения	

Более подробная информация о командах NAND Flash-памяти представлена в документации на этот тип микросхем.

Пример работы с NAND Flash-памятью:

```
// =====
// Инициализация контроллера внешней системной шины для работы с NAND Flash
// =====

NAND_CYCLES = 0x02A63466;
// время t_gr = 2 цикла HCLK или 20 нс при частоте HCLK 100 МГц
// время t_alea = 10 циклов
// время t_whr = 6 циклов
// время t_wp = 3 цикла
// время t_rea = 4 цикла
// время t_ws = 6 циклов
// время t_rc = 6 циклов

EXT_BUS_CONTROL = 0x00000004;
// NAND = 1;

// =====
// Чтение ID микросхемы
// =====

unsigned char IDH;
unsigned char IDL;

// Фаза команды
*((volatile unsigned char *) (0x77200480)) = 0x00;
// ADR_CYCLE = 1
// SCMD = 0x90 (READ)
// Address 1 cycle = 0x00

// Фаза данных
IDL = *((volatile unsigned char *) (0x77080000));
IDH = *((volatile unsigned char *) (0x77080000));

// =====
// Стирание блока памяти
// =====

// Фаза команды
*((volatile unsigned char *) (0x70768300)) = 0x11;
*((volatile unsigned char *) (0x70768301)) = 0x22;
*((volatile unsigned char *) (0x70768302)) = 0x33;
// ADR_CYCLE = 3
// выполнять завершающую команду
// ECMD = 0xD0
// SCMD = 0x60
```

```

// Address 1 cycle = 0x11
// Address 2 cycle = 0x22
// Address 1 cycle = 0x33
while (EXT_BUS_CONTROL!=0x080 ) {};
// Ждем R/nB

// Фаза команды
*((volatile unsigned char *)(0x70000380+addon))=0x00;
// ADR_CYCLE = 0
// SCMD = 0x70
// Фаза данных
IDL = *((volatile unsigned char *)(0x77080000));
If (IDL & 0x01==0x01) Error ();
// Если бит IO0==1, то стирание не выполнено

// =====
// Запись страницы
// =====

// Фаза команды
*((volatile unsigned char *)(0x70800400))=0x11;
*((volatile unsigned char *)(0x70800400))=0x22;
*((volatile unsigned char *)(0x70800400))=0x33;
*((volatile unsigned char *)(0x70800400))=0x44;
// ADR_CYCLE = 4
// SCMD = 0x80

// Фаза данных
*((volatile unsigned char *)(0x70088000+addon))=0xBB;
*((volatile unsigned char *)(0x70088000+addon))=0xCC;
*((volatile unsigned char *)(0x70088000+addon))=0xDD;
// не выполнять завершающую команду
// ECMD= 0x10
...
*((volatile unsigned char *)(0x70188000+addon))=0xEE;
// не выполнять завершающую команду
// ECMD= 0x10
// Данные 0 – 0xBB, 1 – 0xCC,... N – 0xEE
// N от 1 до 528
while (EXT_BUS_CONTROL!=0x080 ) {};
// Ждем R/nB

// Фаза команды
*((volatile unsigned char *)(0x70000380+addon))=0x00;
// ADR_CYCLE = 0
// SCMD = 0x70
// Фаза данных
IDL = *((volatile unsigned char *)(0x77080000));
If (IDL & 0x01==0x01) Error ();
// Если бит IO0==1, то запись не выполнена

// =====

```

```
// Чтение страницы
// =====

// Фаза команды
*((volatile unsigned char *) (0x70800000))=0x11;
*((volatile unsigned char *) (0x70800000))=0x22;
*((volatile unsigned char *) (0x70800000))=0x33;
*((volatile unsigned char *) (0x70800000))=0x44;
// ADR_CYCLE = 4
// SCMD = 0x00
while (EXT_BUS_CONTROL!=0x080 ) {};
// Ждем R/nB

// Фаза данных
IDL=*((volatile unsigned char *) (0x70080000));
IDH=*((volatile unsigned char *) (0x70080000));
If (IDL != 0xBB || IDH != 0xCC) Error ();
// Если считали не то, что записали, то ошибка
```

20.3 Описание регистров блока контроллера внешней системной шины

Таблица 150 – Описание регистров блока контроллера внешней системной шины

Базовый Адрес	Название	Описание
0x400F_0000	MDR_EBC	Контроллер внешней системной шины
Смещение		
0x50	NAND_CYCLES	Регистр MDR_EBC→NAND_CYCLES управления работой с NAND_Flash
0x54	CONTROL	Регистр MDR_EBC→CONTROL управления внешней системной шиной
0x58	REG0_CNTRL	Регистр MDR_EBC→REG0_CNTRL управления регионом 0 внешней системной шиной для адресного пространства 0x10000000-0x1FFFFFFF
0x5C	REG1_CNTRL	Регистр MDR_EBC→REG1_CNTRL управления регионом 1 внешней системной шиной для адресного пространства 0x30000000-0x3FFFFFFF
0x60	REG2_CNTRL	Регистр MDR_EBC→REG2_CNTRL управления регионом 2 внешней системной шиной для адресного пространства 0x50000000-0x5FFFFFFF
0x64	REG3_CNTRL	Регистр MDR_EBC→REG3_CNTRL управления регионом 3 внешней системной шиной для адресного пространства 0x60000000-0xDFFFFFFF

20.3.1 MDR_EBC->CONTROL

Таблица 151 – Регистр CONTROL

Номер	31...18	17	16	15...12	11	10	9	8
Доступ	U	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Сброс	0	0	0	1	0	0	0	0
	-	FF_ RDY	FS_ RDY	WAIT_ STATE [3:0]	EN_FF_ RDY	EN_FS_ RDY	EXT_BUS_ CNTRL_MODE	CLOCKOUT_ EN

Номер	7	6	5	4	3	2	1	0
Доступ	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Сброс	1	0	0	0	0	0	0	1
	BUSY	LOW16	LOW8	ENDIAN	CPOL	NAND	RAM	ROM

Таблица 152 – Описание бит регистра CONTROL

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...16	-	Зарезервировано
17	FF_RDY	Флаг возникновения ошибки на шине при отсутствии сигнала READY за отведенное время ожидания (очищается записью 1) 0 – нет ошибки 1 – есть ошибка
16	FS_RDY	Флаг возникновения ошибки на шине при отсутствии сигнала READY перед началом транзакции (очищается записью 1) 0 – нет ошибки 1 – есть ошибка
15...12	WAIT STATE[3:0]	Количество тактов шины АHB, необходимых для стандартного цикла записи/чтения. Сигналы OE/WE устанавливаются в момент времени $\frac{1}{4}$ WAIT_STATE, снимаются в момент времени $\frac{3}{4}$ WAIT_STATE: 0000 – 3 такта HCLK 0001 – 4 такта HCLK ... 1111 – 17 тактов HCLK
11	EN_FF_RDY	Бит разрешения формирования ошибки на шине при отсутствии сигнала READY за отведенное время ожидания (имеет смысл при работе с диапазоном, у которого выставлен бит USE_READY=1) 0 – не формировать ошибку BUS FAULT при отсутствии сигнала READY после превышения максимального времени ожидания 1 – формировать ошибку BUS FAULT при отсутствии сигнала READY после превышения максимального времени ожидания
10	EN_FS_RDY	Бит разрешения формирования ошибки на шине при отсутствии сигнала READY перед началом транзакции (имеет смысл при работе с диапазоном, у которого выставлен бит USE_READY=1) 0 – не формировать ошибку BUS FAULT при отсутствии сигнала READY перед началом транзакции 1 – формировать ошибку BUS FAULT при отсутствии сигнала READY перед началом транзакции
9	EXT_BUS_MODE	Общий бит разрешения работы с собственными настройками временных интервалов 0 – настройки задаются битами WAIT_STATE 1 – если для региона установлен ENABLE=1, то временные настройки для этого региона определяются битами WS_ACTIVE, WS_SETUP, WS_HOLD
8	CLOCKOUT_EN	Бит разрешения выдачи системной частоты микросхемы на вывод CLOCK 0 – выдается единичный сигнал тактирования внутри фазы WE или OE. 1 – при EXT_BUS_MODE=1 выдается системная частота
7	BUSY	Сигнал занятости NAND Flash-памяти: 1 – операция завершена; 0 – операция не завершена

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
6	LOW16	Выравнивание данных по 16 младшим разрядам внешней системной шины PA[15:0]: 0 – обычный режим работы шины; 1 – данные записываются и читаются всегда с 16 младших разрядов данных PA[15:0]. Чтение/запись 8-ми бит не выполняется. При чтении/записи 16-ти бит выполняется одно обращение. При чтении/записи 32-х бит выполняется два обращения
5	LOW8	Выравнивание данных по 8 младшим разрядам внешней системной шины PA[7:0]: 0 – обычный режим работы шины; 1 – данные записываются и читаются всегда с 8 младших разрядов данных PA[7:0]. Чтение/запись 16-ти бит не выполняется. При чтении/записи 8-ми бит выполняется одно обращение. При чтении/записи 32-х бит выполняется четыре обращения
4	ENDIAN	Всегда записывать ноль
3	CPOL	Бит задания полярности сигнала CLOCK: 0 – положительная полярность; 1 – отрицательная полярность
2	NAND	Бит глобального разрешения памяти NAND: 1 – выбрана NAND; 0 – NAND не выбрана. Одновременная установка нескольких бит 3..0 недопустима, в этом случае запрещается работа со всей памятью
1	RAM	Бит глобального разрешения памяти RAM: 1 – выбрана RAM; 0 – RAM не выбрана
0	ROM	Бит глобального разрешения памяти ROM: 1 – выбрана ROM; 0 – ROM не выбрана

20.3.2 MDR_EBC->NAND_CYCLES

Таблица 153 – Регистр NAND_CYCLES

Номер	31...28	27...24	23...20	19...16	15...12	11...8	7...4	3...0
Доступ	U	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Сброс		0	0	0	0	0	0	0
	-	t_rr	t_alea	t_whr	t_wp	t_rea	t_wc	t_rc

Таблица 154 – Описание бит регистра NAND_CYCLES

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...28		Зарезервировано
27...24	t_rr[3:0]	Время от снятия busy до операции чтения: 0000 – 0 HCLK циклов 0001 – 1 HCLK цикл 1111 – 15 HCLK циклов Типовое значение для памяти NAND Flash составляет 20 нс
23...20	t_alea[3:0]	Время доступа к регистрам ID. Аналогично t_rr. Типовое значение для памяти NAND Flash составляет 100 нс
19...16	t_whr[3:0]	Время доступа к регистру статуса. Аналогично t_rr. Типовое значение для памяти NAND Flash составляет 60 нс
15...12	t_wp[3:0]	Время доступа по записи. Аналогично t_rr. Типовое значение для памяти NAND Flash составляет 25 нс
11...8	t_rea[3:0]	Время доступа по чтению. Аналогично t_rr. Типовое значение для памяти NAND Flash составляет 35 нс
7...4	t_wc[3:0]	Время цикла записи. Аналогично t_rr. Типовое значение для памяти NAND Flash составляет 60 нс
3...0	t_rc[3:0]	Время цикла чтения. Аналогично t_rr. Типовое значение для памяти NAND Flash составляет 60 нс

20.3.3 MDR_EBC->REGx_CNTRL

Таблица 155 – Регистр REGx_CNTRL

Номер	31...15	14	13...11	10...8	7...1	0
Доступ	U	R/W	R/W	R/W	R/W	R/W
Сброс		0	010	010	100000	0
	-	USE_READY	WS_HOLD	WS_SETUP	WS_ACTIVE	ENABLE

Таблица 156 – Описание бит регистра REGx_CNTRL

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31..15		Зарезервировано
14	USE_READY	Разрешение опроса внешнего сигнала READY на выводе PC[0], динамически определяющего аппаратные состояния ожидания в цикле обмена по внешней системной шине: 1 – опрашивается; 0 – не опрашивается. Опрос сигнала READY производится на последнем такте фазы ACTIVE, если READY находится в активном состоянии – осуществляется переход к фазе HOLD и завершение цикла обмена, в противном случае производится повторение опроса на каждом последующем такте, пока количество тактов ожидания не превысит 256. После этого обмен завершается в любом случае
13..11	WS_HOLD[2:0]	Время удержания сигналов nWE/nOE в цикле записи/чтения, выраженное в количестве тактов системной частоты (WS_HOLD + 1) в диапазоне от 1 до 8
10..8	WS_SETUP[2:0]	Время предустановки сигналов nWE/nOE в цикле записи/чтения, выраженное в количестве тактов системной частоты (WS_SETUP + 1) в диапазоне от 1 до 8
7..1	WS_ACTIVE[6:0]	Длительность низкого уровня сигналов nWE/nOE в цикле записи/чтения, выраженная в количестве тактов системной частоты (WS_ACTIVE + 1) в диапазоне от 1 до 128
0	ENABLE	Разрешение настройки параметров обмена соответствующего диапазона адресов, имеет значение только при EXT_BUS_CNTRL_MODE = 1 1 – разрешена; 0 – запрещена

21 Контроллер интерфейса MDR_USB

Контролер USB реализует функции контроллера функционального устройства (Device) и управляющего устройства (Host) в соответствии со спецификацией USB 1.0.

Контроллер USB поддерживает следующие возможности: режимы работы Full Speed (12 Мбит/с) и Low Speed (1,5 Мбит/с), контроль ошибок с помощью циклического избыточного кода (CRC), NRZI код приема/передачи, управляющая (Control), сплошная (Bulk), изохронная (Isochronous) передачи и передача по прерываниям (Interrupt), конфигурирование USB Device от одной до четырех конечных точек; каждая конечная точка USB Device имеет собственную память FIFO размером 64 байта. USB Host поддерживает до 16 конечных точек. Возможности USB Host: FIFO размером 64 байта; автоматическая отправка SOF пакетов; вычисление оставшегося во фрейме времени.

21.1 Инициализация контроллера при включении

При включении питания в первую очередь должны быть заданы параметры тактового сигнала блока USB. Параметры задаются в блоке «Сигналы тактовой частоты». Источником тактового сигнала для блока USB может быть внешний генератор HSE. Блок USB функционирует на частоте 48 МГц. Требуемая частота может быть получена умножением частоты генератора до требуемого значения. Умножение выполняется встроенным блоком PLL_USB.

Блок умножения позволяет провести умножение входной тактовой частоты на коэффициент от 4 до 16, задаваемый в поле PLLUSBMUL регистра PLL_CONTROL. Входная частота блока умножителя должна быть в диапазоне от 6 до 16 МГц, выходная должна составлять 48 МГц. При выходе блока умножителя тактовой частоты в расчетный режим вырабатывается сигнал PLLRDY. Блок включается с помощью сигнала PLLUSBON. Выходная частота используется как основная частота протокольной части интерфейса USB.

Для задания тактовой частоты блока необходимо соблюдать следующий порядок работы. Установить бит разрешения тактирования блока (бит 3 регистра PER_CLOCK). В регистре USB_CLOCK установить бит USBCLKEN, задать источник тактового сигнала в полях USBC1SEL и USBC2SEL. Установить бит PLLUSBON и задать коэффициент умножения в поле PLLUSBMUL регистра PLL_CONTROL, если используется USBPLL.

После подачи тактового сигнала на блок USB необходимо выполнить сброс контроллера. Сброс выполняется установкой бита RESET_CORE в регистре USB_HSCR. Сигнал сброса необходимо удерживать как минимум 10 циклов тактовой частоты. После этого могут быть заданы параметры шины USB (скорость, полярность, наличие подтяжек).

21.2 Задание параметров шины USB и события подключения/отключения

Контроллер USB может быть сконфигурирован как USB Host или как USB Device. Конфигурация задается битом HOST_MODE в регистре HSCR (0 – режим Device, 1 – режим Host). Прием/передача через физический интерфейс USB разрешаются установкой

бит EN_RX и EN_TX в этом же регистре. В режиме приема имеется возможность отключить передатчик в целях экономии потребления (EN_TX=0). Отключение всего блока в целом осуществляется при EN_RX=0.

В режиме Device параметры шины задаются в регистре SC. Скорость задается битом SCFSR (0 – 1,5 Мбит/с, 1 – 12 Мбит/с), полярность битом SCFSP (0 – Low speed, 1 – Full speed) этого регистра.

В режиме Host параметры шины задаются в регистре HTXLC. Скорость задается битом FSLR (0 – 1,5 Мбит/с, 1 – 12 Мбит/с), полярность битом FSPL (0 – Low speed, 1 – Full speed) этого регистра.

В режиме Host контроллер автоматически определяет подключение или отключение устройства к шине. Бит CONEV регистра USB_HSI устанавливается в 1 при возникновении одного из событий.

В режиме HOST необходимо прямое подключение (без USB HUB) LOW SPEED устройства.

21.3 Задание адреса и инициализация оконечных точек

Функциональный адрес устройства USB задается в регистре SA.

Для инициализации конечной точки в первую очередь необходимо установить бит глобального разрешения всех оконечных точек (SCGEN = 1 в регистре SC). Биты EPEN в регистрах SEP[x].CTRL должны быть установлены, чтобы разрешить соответствующую оконечную точку. Если предполагается использовать изохронный тип передачи оконечной точки, то необходимо установить бит EPISEN в соответствующем регистре SEP[x].CTRL.

21.4 Транзакция IN (USB Device)

Если оконечная точка не готова (бит EPRDY = 0 в регистре SEP[x].CTRL), то контроллер отправляет NAK пакет (рисунок 35 а). Бит NAKSENT регистра SEP[x].STS устанавливается в 1.

Если оконечная точка готова и установлен бит EPSSTALL в регистре SEP[x].CTRL, то контроллер отправляет STALL пакет (рисунок 35 б). Бит SCSTALLSENT регистра SEP[x].STS устанавливается в 1.

Если оконечная точка готова (рисунок 35 в), биты SCTTYPE[1:0] в регистре SEP[x].TS устанавливаются в значение «1» для конечной точки с номером, содержащимся в поле пакета. Контроллер может передавать пакет данных. Пакет данных формируется записью в регистр EP[x].TXFD побайтно в FIFO оконечной точки. Запись «1» в EP[x].TXFC сбрасывает указатель FIFO передачи в 0. Максимальный размер передаваемого пакета составляет 64 байта. Попытка записи более 64 байтов подряд приведет к переполнению FIFO. Перед началом формирования очередного пакета необходимо выполнять сброс указателя FIFO.

Если в ответ на переданные данные хост отправляет ACK пакет, то бит SCACKRXED в регистре SEP[x].STS устанавливается в «1». Для отправки следующего пакета необходимо инвертировать бит EPDATASEQ в

регистре SEP[x].CTRL, чтобы соблюдалась очередность отправки пакетов DATA0, DATA1.

USB_SEPx.CTRL	
EPISOEN	0
EPSTALL	0
EPDATASEQ	X
EPRDY	0
EPEN	1

Host Token

SYNC	IN	ADDR	ENDP x	CRC5	EOP
------	----	------	-----------	------	-----

Device Handshake

SYNC	NAK	EOP
------	-----	-----

а)

NTTYPE[1:0]=1
(USB_SEPx].NTS)

SCNAKSENT=1
(USB_SEPx].STS)

USB_SEPx.CTRL	
EPISOEN	0
EPSTALL	1
EPDATASEQ	X
EPRDY	1
EPEN	1

При необходимости ответа на следующую транзакцию ACK необходимо программно сбросить бит EPSTALL в регистре USB_SEPx].CTRL

Host Token

SYNC	IN	ADDR	ENDP x	CRC5	EOP
------	----	------	-----------	------	-----

Device Handshake

SYNC	STALL	EOP
------	-------	-----

б)

SCTDONE=1
(USB_SIS)

SCTYPE[1:0]=1
(USB_SEPx].TS)

SCSTALLSENT=1
(USB_SEPx].STS)

USB_SEPx.CTRL	
EPISOEN	0
EPSTALL	0
EPDATASEQ	X
EPRDY	1
EPEN	1

1. Запись данных Data в FIFO оконечной точки (USB_EP[x].TXFD)
2. Задание типа пакета (EPDATASEQ)
3. Установка EPRDY=1 (USB_SEPx].CTRL)

Host Token

SYNC	IN	ADDR	ENDP x	CRC5	EOP
------	----	------	-----------	------	-----

Device DATA 0/1

SYNC	DATA 0/1	Data	CRC16	EOP
------	-------------	------	-------	-----

Host Handshake

SYNC	ACK	EOP
------	-----	-----

SCTYPE[1:0]=1
(USB_SEPx].TS)

SCTDONE=1 и SCACKRXED=1
(USB_SIS и USB_SEPx].STS)

в)

1. Проверить принятие подтверждения или наличия ошибки USB_SEPx].STS (Обработать ошибку)
2. Если принят ACK, сбросить указатель FIFO оконечной точки (USB_EP[x].TXFC=1)
3. Сбросить флаг SCTDONE

Рисунок 35 – Транзакция IN (USB Device)

а – оконечная точка не готова;

б – установлен бит EPSTALL;

в – оконечная точка готова

21.5 Транзакция SETUP/OUT (USB Device)

Если оконечная точка не готова (бит EPRDY = 0 в регистре SEP[x].CTRL), то контроллер отправляет NAK пакет (рисунок 36 а). Бит NAKSENT регистра SEP[x].ST устанавливается в «1».

Если оконечная точка готова и установлен бит EPSSTALL в регистре SEP[x].CTRL, то контроллер отправляет STALL пакет (рисунок 36 б). Бит SCSTALLSENT регистра SEP[x].STS устанавливается в «1».

Если оконечная точка готова (рисунок 36 в) и на шине был пакет SETUP, то биты SCTTYPE[1:0] в регистре SEP[x].TS устанавливаются в значение 00 для конечной точки с номером, содержащимся в поле пакета. Если пакет OUT, то значение SCTTYPE[1:0] = 2.

Когда на шине появляется DATA0/DATA1 пакет, данные начинают записываться побайтно в FIFO приема соответствующей оконечной точки. После записи каждого байта увеличивается на единицу счетчик принятых байтов. Принятые байты считываются через регистр EP[x].RXFD. Количество принятых байтов содержится в регистре EP[x].RXFDC. После приема очередного пакета необходимо выполнять сброс указателя FIFO приема записью «1» в регистр EP[x].RXFC.

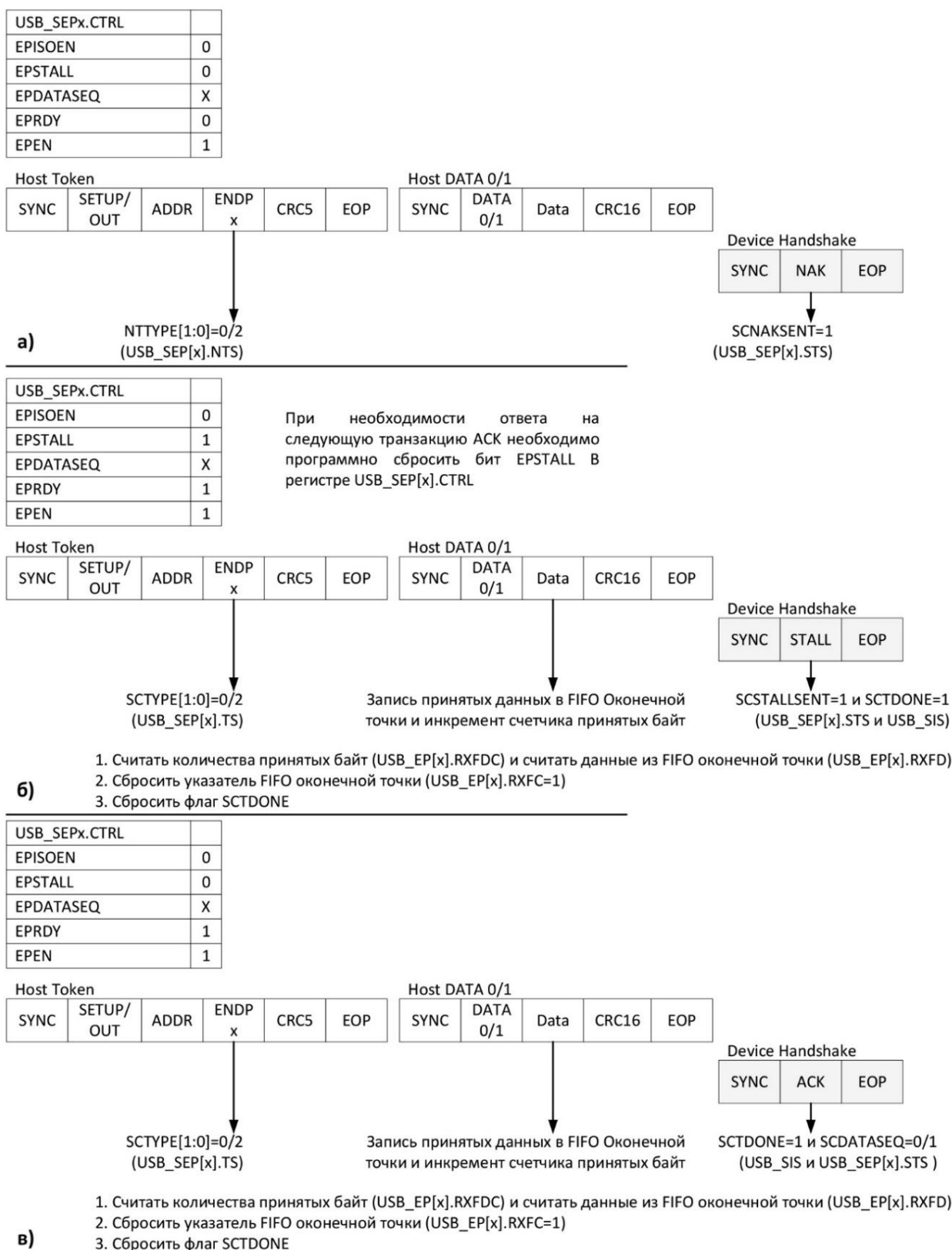


Рисунок 36 – Транзакция SETUP/OUT (USB Device)

а – оконечная точка не готова;

б – установлен бит EPSTALL;

в – оконечная точка готова

21.6 Транзакция SETUP/OUT (USB Host)

Для начала транзакции должны быть заданы адрес устройства (регистр HTXA), оконечная точка (регистр HTXE) и тип token пакета (регистр HTXT). Данные записываются побайтно в регистр HTXFD. Максимальный размер передаваемого пакета составляет 64 байта. Попытка записи более 64 байтов подряд приведет к переполнению FIFO. Запись «1» в HTXFC сбрасывает указатель FIFO передачи в «0». Перед началом формирования очередного пакета необходимо выполнять сброс указателя FIFO. Транзакция запускается при установке бита TREQ регистра HTXC. Host отправляет пакет Setup/Out и пакет данных.

После окончания транзакции бит TDONE = 1 (регистр HSI). Этот бит перед началом каждой транзакции должен быть очищен записью «1». PID принятого пакета записывается в регистре HRXP.

Если в ответ получен пакет NAK (рисунок 37 а), то бит NAKRXED = 1 (регистр HRXS).

Если в ответ получен пакет STALL (рисунок 37 б), то бит STALLRXED = 1 (регистр HRXS).

Если в ответ получен пакет ACK (рисунок 37 в), то бит ACKRXED = 1 (регистр HRXS).

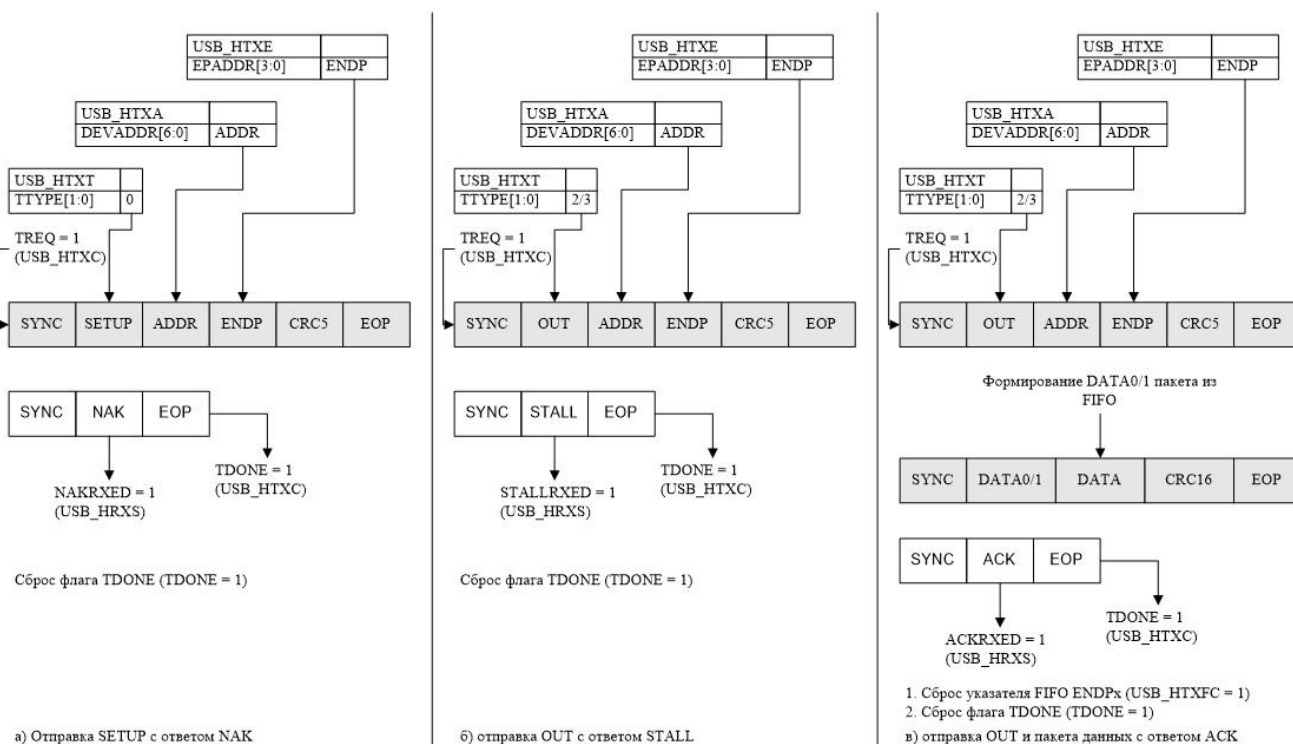


Рисунок 37 – Транзакция SETUP/OUT (USB Host)

21.7 Транзакция IN (USB Host)

Для начала транзакции должны быть заданы адрес устройства (регистр HTXA), оконечная точка (регистр HTXE) и тип token пакета (регистр HTXT). Транзакция запускается при установке бита TREQ регистра HTXC. Host отправляет IN пакет.

После окончания транзакции бит TDONE = 1 (регистр HSI). Этот бит перед началом каждой транзакцией должен быть очищен записью 1. PID принятого пакета записывается в регистре HRXP.

Если в ответ получен пакет NAK (рисунок 38 а), то бит NAKRXED = 1 (регистр HRXS).

Если в ответ получен пакет STALL (рисунок 38 б), то бит STALLRXED = 1 (регистр HRXS).

Если приходит DATA0/DATA1 пакет (рисунок 38 в), то данные начинают записываться побайтно в FIFO приема. После записи каждого байта увеличивается на единицу счетчик принятых байтов. Принятые байты считываются через регистр HRXFD. Количество принятых байтов содержится в регистре HRXFDC. После приема очередного пакета необходимо выполнять сброс указателя FIFO приема записью «1» в регистр HRXFC. Бит DATASEQ регистра HRXS отображает тип принятого пакета данных (0 – DATA0, 1 – DATA1).

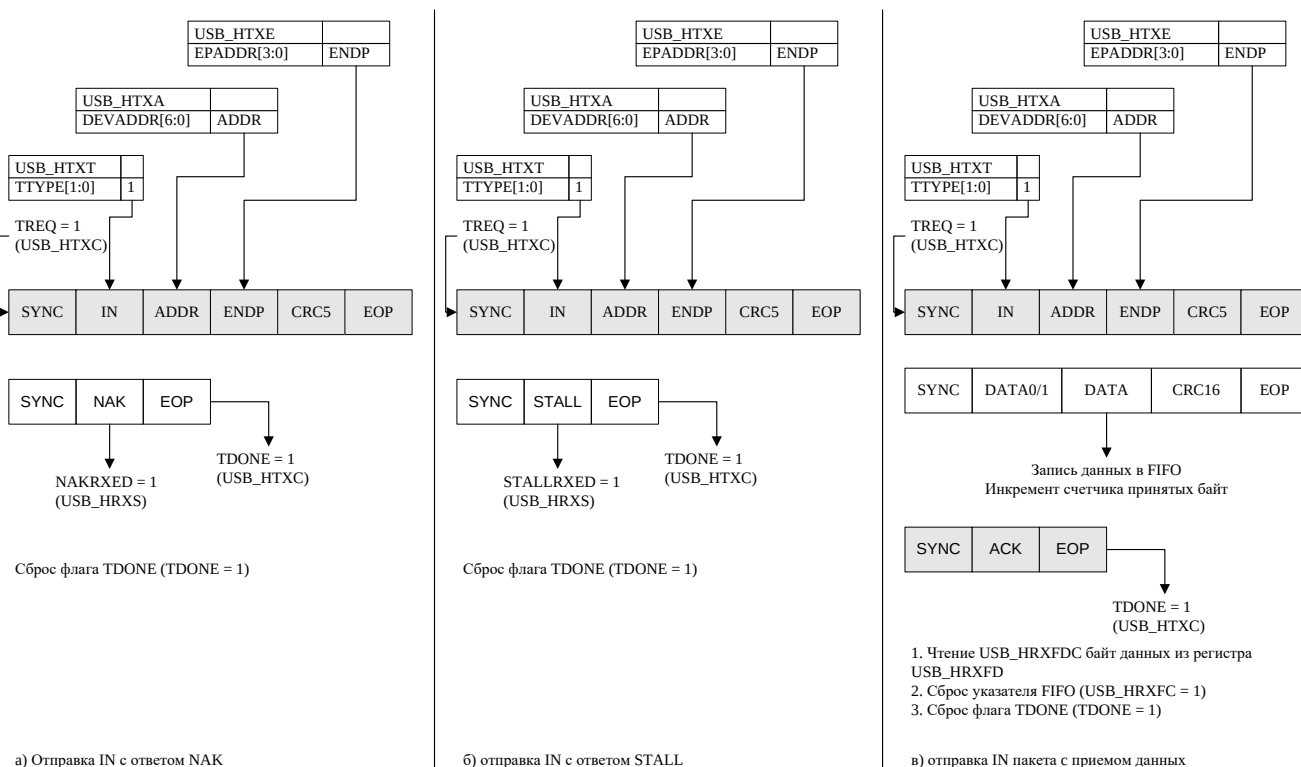


Рисунок 38 – Транзакция IN (USB Host)

21.8 Отправка SOF пакетов и отсчет времени (USB Host)

Для того чтобы контроллер автоматически отправлял SOF пакеты на Full speed, необходимо установить SOFEN в регистре HTXSE. Если FSPL = 1 (регистр TXLC), то SOF будет автоматически отсылаться каждые 1 мс. Если FSPL = 0, то автоматически будет отправляться EOP каждые 1 мс.

После отправки SOF пакета бит SOFS = 1 (регистр HSI). Этот бит должен быть очищен записью 1.

Контроллер ведет счет времени во фрейме таймером. Таймер увеличивается на частоте 48 МГц и имеет 48000 тактов в 1 мс фрейме. Старший байт таймера содержится

в регистре HSTM. Этот регистр может быть использован для вычисления, оставшегося во фрейме времени.

21.9 Описание регистров управления контроллером USB интерфейса

Таблица 157 – Описание регистров управления контроллером USB интерфейса

Базовый Адрес	Название	Описание
0x4001_0000	MDR_USB	Контроллер USB интерфейса
Смещение		
0x380	MDR_USB->HSCR	Общее управление для контроллера USB интерфейса
0x384	MDR_USB->HSVR	Версия аппаратного контроллера USB интерфейса
	Контроллер HOST	
0x00	MDR_USB->HTXC	Регистр управления передачей пакетов со стороны хоста
0x04	MDR_USB->HTXT	Регистр задания типа передаваемых пакетов со стороны хоста
0x08	MDR_USB->HTXLC	Регистр управления линиями шины USB
0x0C	MDR_USB->HTXSE	Регистр управление автоматической отправки SOF
0x10	MDR_USB->HTXA	Регистр задания адреса устройства для отправки пакета
0x14	MDR_USB->HTXE	Регистр задания номера конечной точки для отправки пакета
0x18 0x1C	MDR_USB->HFN_L MDR_USB->HFN_H	Регистр задания номера фрейма для отправки SOF
0x20	MDR_USB->HSI	Регистр флагов событий контроллера хост.
0x24	MDR_USB->HIM	Регистра флагов разрешения прерываний по событиям контроллера хоста
0x28	MDR_USB->HRXS	Регистр состояния очереди приема данных хоста
0x2C	MDR_USB->HRXP	Регистр отображения PID принятого пакета
0x30	MDR_USB->HRXA	Регистр отображения адреса устройства, от которого принят пакет
0x34	MDR_USB->HRXE	Регистр отображения номер конечной точки, от которой принят пакет
0x38	MDR_USB->HRXCS	Регистр отображения состояния подсоединения устройства
0x3C	MDR_USB->HSTM	Регистр расчета времени фрейма
0x80	MDR_USB->HRXFD	Данные очереди приема
0x8C	MDR_USB->HRXFDC	Число принятых данных в очереди
0x90	MDR_USB->HRXFC	Управление очередью приема

Базовый Адрес	Название	Описание
0xC0	MDR_USB->HTXFD	Данные для передачи
0xD0	MDR_USB->HTXFC	Управление очередью передачи
	Контроллер SLAVE	
0x100 0x110 0x120 0x130	MDR_USB->SEP[x].CTRL	Управление очередью нулевой оконечной точки
0x104 0x114 0x124 0x134	MDR_USB->SEP[x].STS	Состояние оконечной точки
0x108 0x118 0x128 0x138	MDR_USB->SEP[x].TS	Состояние типа передачи оконечной точки
0x10C 0x11C 0x12C 0x13C	MDR_USB->SEP[x].NTS	Состояние передачи NAK оконечной точки
0x140	MDR_USB->SC	Управление контроллеров SLAVE
0x144	MDR_USB->SLS	Отображение состояния линий USB шины
0x148	MDR_USB->SIS	Флаги событий контроллера SLAVE
0x14C	MDR_USB->SIM	Флаги разрешения прерываний от контроллера SLAVE
0x150	MDR_USB->SA	Функциональный адрес контроллера
0x154 0x158	MDR_USB->SFN_L MDR_USB->SFN_H	Номер фрейма
0x180 0x200 0x280 0x300	MDR_USB->SEP[x].RXFD	Принятые данные оконечной точки
0x18C 0x20C 0x28C 0x30C	MDR_USB->SEP[x].RXFDC	Число данных в оконечной точке
0x190 0x210 0x290 0x310	MDR_USB->SEP[x].RXFC	Управление очередью приема оконечной точки
0x1C0 0x240 0x2C0 0x340	MDR_USB->SEP[x].TXFD	Данные для передачи через оконечную точку
0x1D0 0x250 0x2D0 0x350	MDR_USB->SEP[x].TXFDC	Управление очередью передачи оконечной точки

21.9.1 MDR_USB->HSCR

Таблица 158 – Регистр HSCR

Номер	31...8	7	6	5	4	3	2	1	0
Доступ	U	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Сброс	0	0	0	0	0	0	0	0	0
	-	D- PULL DOWN	D- PULL UP	D+ PULL DOWN	D+ PULL UP	EN RX	EN TX	RESET CORE	HOST MODE

Таблица 159 – Описание бит регистра HSCR

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...8	-	Зарезервировано
7	D- PULLDOWN	Управление встроенной подтяжкой линии D-: 0 – нет подтяжки вниз; 1 – есть подтяжка вниз
6	D- PULLUP	Управление встроенной подтяжкой линии D-: 0 – нет подтяжки вверх; 1 – есть подтяжка вверх
5	D+ PULLDOWN	Управление встроенной подтяжкой линии D+: 0 – нет подтяжки вниз; 1 – есть подтяжка вниз
4	D+ PULLUP	Управление встроенной подтяжкой линии D+: 0 – нет подтяжки вверх; 1 – есть подтяжка вверх
3	EN_RX	Разрешение работы приемника USB: 0 – запрещен; 1 – разрешен. Может использоваться в энергосберегающих целях
2	EN_TX	Разрешение работы передатчика USB: 0 – запрещен; 1 – разрешен. Может использоваться в энергосберегающих целях
1	RESET_CORE	Программный сброс контроллера: 1 – сброс контроллера (удерживать минимум 10 циклов USBCLK); 0 – рабочий режим
0	HOST_MODE	Режим работы контроллера: 1 – режим HOST; 0 – режим Device

21.9.1.1 MDR_USB->HSVR

Таблица 160 – Регистр HSVR

Номер	31...8	7...4	3...0
Доступ	U	RO	RO
Сброс	0	0	0
	-	REVISION	VERSION

Таблица 161 – Описание бит регистра HSVR

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...8	-	Зарезервировано
7...4	REVISION	Номер Ревизии
3...0	VERSION	Номер Версии

21.9.2 Регистры HOST режима

21.9.2.1 MDR_USB->HTXC

Таблица 162 – Регистр HTXC

Номер	31...4	3	2	1	0
Доступ	U	R/W	R/W	R/W	R/W
Сброс	0	0	0	0	0
	-	ISOEN	PREEN	SOFS	TREQ

Таблица 163 – Описание бит регистра HTXC

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений.
31...4	-	Зарезервировано
3	ISOEN	Флаг разрешения изохронного режима: 1 – разрешение изохронного режима, ACK не посылается и не принимается. Необходимо, что бы TRANS_TYPE_REG был установлен в IN_TRANS или OUTDATA0_TRANS. Изохронный режим не применим с другими типами передачи; 0 – запрещение изохронного режима
2	PREEN	Рекомендуется оставлять 0
1	SOFS	Флаг задания синхронизации передачи с SOF: 1 – синхронизировать передачу с окончанием SOF. Передача будет запущена сразу за передачей SOF; 0 – передача не синхронизирована
0	TREQ	Флаг запроса передачи данных: 1 – запрос разрешения передачи данных, автоматически сбрасывается после передачи; 0 – запрещена передача

21.9.2.2 MDR_USB->HTXT

Таблица 164 – Регистр HTXT

Номер	31...2	1	0
Доступ	U	R/W	R/W
Сброс	0	0	0
	-	TTYTYPE	

Таблица 165 – Описание бит регистра HTXT

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...2	-	Зарезервировано
1...0	TTYTYPE	Тип передачи: 00 – setup_trans 01 – in_trans 10 – outdata0_trans 01 – outdata1_trans

21.9.2.3 MDR_USB->HTXLC

Таблица 166 – Регистр HTXLC

Номер	31...5	4	3	2	1	0
Доступ	U	R/W	R/W	R/W	R/W	R/W
Сброс	0	0	0	0	0	0
	-	FSLR	FSLP	DC	TXLS[1:0]	

Таблица 167 – Описание бит регистра HTXLC

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...5	-	Зарезервировано
4	FSLR	1 – 12 Мбит/с 0 – 1,5 Мбит/с
3	FSPL	1 – FULL SPEED полярность шины USB. 0 – LOW SPEED полярность шины USB. Если host работает с full speed устройством, full speed полярность должна быть установлена. Если работа ведется с low speed устройством на прямую, то должна быть установлена low speed полярность. Работа с low speed через hub не поддерживается
2	DC	Режим управления линиями шины USB: 1 – разрешение прямого управления состоянием линий USB шины; 0 – нормальный режим работы
1...0	TXLC[1:0]	Если установлен бит DIRECT_CONTROL_BIT, то отображается состояние шины USB: TXLC[0] = D- TXLC[1] = D+

21.9.2.4 MDR_USB->HTXSE

Таблица 168 – Регистр HTXSE

Номер	31...1	0
Доступ	U	R/W
Сброс	0	0
	-	SOFEN

Таблица 169 – Описание бит регистра HTXSE

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...1	-	Зарезервировано
0	SOFEN	1 – Если FSPL установлен, то SOF будет автоматически отсылаться каждые 1 мс. SOF отправляется на full speed независимо от состояния FSPL. Если FSPL не установлен, то автоматически будет отправляться EOP каждые 1 мс. Это необходимо при работе с low speed устройством напрямую (не через хаб). 0 – запрет автоматической отправки SOF/EOP и позволяет подсоединенным устройствам перейти в suspend режим

21.9.2.5 MDR_USB->HTXA

Таблица 170 – Регистр HTXA

Номер	31...7	6...0
Доступ	U	R/W
Сброс	0	0
	-	DEVADDR[6:0]

Таблица 171 – Описание бит регистра HTXA

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...7	-	Зарезервировано
6...0	DEVADDR[6:0]	USB Device address. Адрес устройства для обращения

21.9.2.6 MDR_USB->HTXE

Таблица 172 – Регистр HTXE

Номер	31...4	3...0
Доступ	U	R/W
Сброс	0	0
	-	EPADDR[3:0]

Таблица 173 – Описание бит регистра HTXE

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...4	-	Зарезервировано
3...0	EPADDR[3:0]	Endpoint address. Номер конечной точки устройства для обращения

21.9.2.7 MDR_USB->HFN

Таблица 174 – Регистр HFN

Номер	31...11	10...0
Доступ	U	R/W
Сброс	0	0
	-	FNUM[10:0]

Таблица 175 – Описание бит регистра HFN

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...11	-	Зарезервировано
10...0	FNUM[10:0]	Номер фрейма

21.9.2.8 MDR_USB->HSI

Таблица 176 – Регистр HSI

Номер	31...4	3	2	1	0
Доступ	U	R/W	R/W	R/W	R/W
Сброс	0	0	0	0	0
	-	SOFS	CONEV	RESUME	TDONE

Таблица 177 – Описание бит регистра HSI

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений.
31...4	-	Зарезервировано
3	SOFS	1 – автоматически устанавливается, когда SOF был отправлен. Должен быть очищен записью 1. 0 – не было SOF
2	CONEV	1 – автоматически устанавливается, когда подключение или отсоединение происходит. Должно быть очищено записью 1. 0 – события не было
1	RESUME	1 – автоматически устанавливается, когда возникнет состояние возобновления передачи. Должен быть очищен записью 1. 0 – не было повтора.
0	TDONE	1 – автоматически устанавливается, когда передача закончена. Должен быть очищен записью 1. 0 – передача не закончена или ее нет

21.9.2.9 MDR_USB->HIM

Таблица 178 – Регистр HIM

Номер	31...4	3	2	1	0
Доступ	U	R/W	R/W	R/W	R/W
Сброс	0	0	0	0	0
	-	SOFSIE	CONEVIE	RESUMEIE	TDONEIE

Таблица 179 – Описание бит регистра HIM

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...4	-	Зарезервировано
3	SOFIE	1 – разрешение выработки прерывания при передаче SOF. 0 – запрещение выработки прерывания
2	CONEVIE	1 – разрешение выработки прерывания при подсоединении или отсоединении. 0 – запрещение выработки прерывания
1	RESUMEIE	1 – разрешение выработки прерывания при возобновлении передачи. 0 – запрещение выработки прерывания
0	TDONEIE	1 – разрешение выработки прерывания при окончании передачи. 0 – запрещение выработки прерывания

21.9.2.10 MDR_USB->HRXS

Таблица 180 – Регистр HRXS

Номер	31...8	7	6	5	4	3	2	1	0
Доступ	U	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Сброс	0	0	0	0	0	0	0	0	0
	-	DATASEQ	ACK RXED	STALL RXED	NAK RXED	RX TO	RXOF	BSERR	CRCER

Таблица 181 – Описание бит регистра HRXS

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...8	-	Зарезервировано
7	DATASEQ	Если последняя транзакция была типа IN_TRANS, этот бит указывает номер последнего принятого пакета. DATA0 = 0, DATA1 = 1
6	ACK RXED	1 – получен ACK. 0 – не получен ACK
5	STALL RXED	1 – получен STALL. 0 – не получен STALL
4	NAK RXED	1 – получен NAK от устройства. 0 – не получен NAK
3	RXTO	1 – переполнения времени ожидания ответа от устройства. 0 – нет переполнения времени
2	RXOF	1 – обнаружена ошибка переполнения FIFO при приеме пакета. 0 – не было переполнения
1	BSERR	1 – обнаружена ошибка stuff при последней передаче. 0 – ошибки stuff не было
0	CRCERR	1 – обнаружена ошибка CRC при последней передаче. 0 – ошибки CRC не было

21.9.2.11 MDR_USB->HRXP

Таблица 182 – Регистр HRXP

Номер	31...4	3...0
Доступ	U	R/W
Сброс	0	0
	-	RPID[3:0]

Таблица 183 – Описание бит регистра HRXP

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...4	-	Зарезервировано
3...0	RPID[3:0]	Packet identifier от последнего принятого пакета

21.9.2.12 MDR_USB->HRXA

Таблица 184 – Регистр HRXA

Номер	31...7	6...0
Доступ	U	R/W
Сброс	0	0
	-	RADDR[6:0]

Таблица 185 – Описание бит регистра HRXA

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...7	-	Зарезервировано
6...0	RADDR[6:0]	Адрес последнего принятого пакета, который был послан

21.9.2.13 MDR_USB->HRXE

Таблица 186 – Регистр HRXE

Номер	31...4	3...0
Доступ	U	R/W
Сброс	0	0
	-	RXENDP[3:0]

Таблица 187 – Описание бит регистра HRXE

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений.
31...4	-	Зарезервировано
3...0	RXENDP[3:0]	Номер окончной точки в последнем принятом пакете, который был послан

21.9.2.14 MDR_USB->HRXCS

Таблица 188 – Регистр HRXCS

Номер	31...2	1...0
Доступ	U	R/W
Сброс	0	0
	-	RXLS[1:0]

Таблица 189 – Описание бит регистра HRXCS

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...2	-	Зарезервировано
1...0	RXLS[1:0]	Состояние линий шины USB: DISCONNECT = 0 LOW_SPEED_CONNECT = 1 FULL_SPEED_CONNECT = 2

21.9.2.15 MDR_USB->HSTM

Таблица 190 – Регистр HSTM

Номер	31...8	7...0
Доступ	U	R/W
Сброс	0	0
	-	HSTM[7:0]

Таблица 191 – Описание бит регистра HSTM

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...8	-	Зарезервировано
7...0	HSTM[7:0]	Старший байт SOF таймера, используемого для передачи SOF. Таймер увеличивается на частоте 48 МГц, и имеет 48000 тактов в 1 мс фрейме. Этот регистр может быть использован для вычисления оставшегося во фрейме времени

21.9.2.16 MDR_USB->HRXFD

Таблица 192 – Регистр HRXFD

Номер	31...8	7...0
Доступ	U	R/W
Сброс	0	0
	-	RX FIFO DATA[7:0]

Таблица 193 – Описание бит регистра HRXFD

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...8	-	Зарезервировано
7...0	RX FIFO DATA[7:0]	Если последняя транзакция была IN_TRANS, то порт содержит принятые данные, и они могут быть считаны

21.9.2.17 MDR_USB->HRXFDC

Таблица 194 – Регистр HRXFDC

Номер	31...16	15...0
Доступ	U	R/W
Сброс	0	0
	-	FIFO DATA COUNT[15:0]

Таблица 195 – Описание бит регистра HRXFDC

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...16	-	Зарезервировано
15...0	FIFO DATA COUNT[15:0]	Счетчик принятых байт в очереди

21.9.2.18 MDR_USB->HRXFC

Таблица 196 – Регистр HRXFC

Номер	31...1	0
Доступ	U	R/W
Сброс	0	0
	-	FIFO FORCE EMPTY

Таблица 197 – Описание бит регистра HRXFC

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...1	-	Зарезервировано
0	FIFO FORCE EMPTY	Запись 1 принудительно сбрасывает очередь

21.9.2.19 MDR_USB->HTXFD

Таблица 198 – Регистр HTXFD

Номер	31...8	7...0
Доступ	U	R/W
Сброс	0	0
	-	TX FIFO DATA[7:0]

Таблица 199 – Описание бит регистра HTXFD

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...8	-	Зарезервировано
7...0	TX FIFO DATA[7:0]	При запросах передачи OUTDATA0_TRANS или OUTDATA1_TRANS, через данный порт должны быть загружены данные для отправки

21.9.2.20 MDR_USB->HTXFC

Таблица 200 – Регистр HTXFC

Номер	31...1	0
Доступ	U	R/W
Сброс	0	0
	-	FIFO FORCE EMPTY

Таблица 201 – Описание бит регистра HTXFC

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...1	-	Зарезервировано
0	FIFO FORCE EMPTY	Запись 1 принудительно сбрасывает очередь

21.9.3 USB Slave (Device)

21.9.3.1 MDR_USB->SEP[x].CTRL

Таблица 202 – Регистр SEP[x].CTRL

Номер	31...5	4	3	2	1	0
Доступ	U	R/W	R/W	R/W	R/W	R/W
Сброс	0	0	0	0	0	0
	-	EPISOEN	EPSSTALL	EPDATASEQ	EPRDY	EPEN

Таблица 203 – Описание бит регистра USB_SEPx.CTRL

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...5	-	Зарезервировано
4	EPISOEN	0 – не изохронный режим передачи; 1 – разрешить изохронные передачи. В изохронном режиме не отсылаются какие-либо подтверждения передачи
3	EPSSTALL	0 – не отвечать STALL на запрос; 1 – если точка разрешена, готова, и не в изохронном режиме, то на запрос хоста будет отвечать STALL
2	EPDATASEQ	0 – отвечать на IN запрос от хоста с DATA0; 1 – отвечать на IN запрос от хоста с DATA1.
1	EPRDY	0 – оконечная точка не готова или закончила транзакцию; 1 – оконечная точка готова. Если точка разрешена и готова, то она может ответить на инициализированную хостом передачу. Автоматически сбрасывается в 0 после успешного завершения транзакции
0	EPEN	0 – оконечная точка запрещена; 1 – оконечная точка разрешена. Если точка запрещена она не отвечает на транзакции, если точка разрешена, но не готова и не находится в изохронном режиме, то отвечает NAK

21.9.3.2 MDR_USB->SEP[x].STS

Таблица 204 – Регистр SEP[x].STS

Номер	31...8	7	6	5	4	3	2	1	0
Доступ	U	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Сброс	0	0	0	0	0	0	0	0	0
	-	SC DATA SEQ	SC ACK RXED	SC STALL SENT	NAK SENT	SC RXTO	SC RXOF	SC BS ERR	SC CRC ERR

Таблица 205 – Описание бит регистра USB_SEPx.STS

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...8	-	Зарезервировано
7	SC DATA SEQ	Если предыдущий тип передачи был OUT_TRANS, то этот бит определяет тип принятого пакета DATA0 = 0, DATA1 = 1
6	SC ACK RXED	0 – нет подтверждения; 1 – получено подтверждение ACK от хоста на переданные данные.
5	SC STALL SENT	0 – не было STALL; 1 – признак отправки STALL
4	NAK SENT	1 – признак отправки NAK ответа. 0 – не было NAK
3	SC RXTO	1 – признак возникновения ошибки времени ожидания ответа от хоста. 0 – нет ошибки
2	SC RXOF	0 – нет переполнения; 1 – признак возникновения переполнения очереди при последней передаче
1	SC BS ERR	0 – нет ошибки; 1 – признак возникновения STUFF ошибки в последней передаче
0	SC CRC ERR	0 – нет ошибки; 1 – признак возникновения CRC ошибки в последней передаче

21.9.3.3 MDR_USB->SEP[x].TS

Таблица 206 – Регистр SEP[x].TS

Номер	31...2	1	0
Доступ	U	R/W	R/W
Сброс	0	0	0
	-	SCTTYPE[1:0]	

Таблица 207 – Описание бит регистра SEP[x].TS

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...2	-	Зарезервировано
1...0	SCTTYPE[1:0]	Отображает тип последней передачи перед тем, как ENDPOINT_READY_BIT был изменен с 1 на 0. SC_SETUP_TRANS = 0 SC_IN_TRANS = 1 SC_OUTDATA_TRANS = 2

21.9.3.4 MDR_USB->SEP[x].NTS

Таблица 208 – Регистр SEP[x].NTS

Номер	31...2	1	0
Доступ	U	R/W	R/W
Сброс	0	0	0
	-	NTTYPE[1:0]	

Таблица 209 – Описание бит регистра USB_SEPx.NTS

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...2	-	Зарезервировано
1...0	NTTYPE[1:0]	Тип последней передачи, в результате которой на хост был послан NAK. SC_SETUP_TRANS = 0 SC_IN_TRANS = 1 SC_OUTDATA_TRANS = 2

21.9.3.5 MDR_USB->SC

Таблица 210 – Регистр SC

Номер	31...6	5	4	3	2	1	0
Доступ	U	R/W	R/W	R/W	R/W	R/W	R/W
Сброс	0	0	0	0	0	0	0
	-	SCFSR	SCFSP	SCDC	SCTXLS[1:0]	SCGEN	

Таблица 211 – Описание бит регистра USB_SC

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...6	-	Зарезервировано
5	SCFSR	Флаг управления скоростью работы: 1 – 12 Мбит/с; 0 – 1,5 Мбит/с
4	SCFSP	Флаг выбора полярности линий USB шины: 1 – FULL SPEED; 0 – LOW SPEED
3	SCDC	Флаг прямого управления линиями USB шины: 1 – разрешено прямое управление 0 – запрещено прямое управление
2...1	SCTXL[1:0]	Если установлен бит SC_DIRECT_CONTROL_BIT, то через SC_TX_LINE_STATE осуществляется прямое управление состоянием линий USB шины: SC_TX_LINE_STATE [2] = D+ SC_TX_LINE_STATE [1] = D-
0	SCGEN	1 – разрешение для работы с разрешенных конечных точек 0 – все конечные точки запрещены

21.9.3.6 MDR_USB->SLS

Таблица 212 – Регистр SLS

Номер	31...2	1	0
Доступ	U	R/W	R/W
Сброс	0	0	0
	-	SCRXLS[1:0]	

Таблица 213 – Описание бит регистра SLS

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...2	-	Зарезервировано
1...0	SCRXLS[1:0]	Отображает состояние подключения на шине USB: RESET = 0 LOW_SPEED_CONNECT = 1 FULL_SPEED_CONNECT = 2

21.9.3.7 MDR_USB->SIS

Таблица 214 – Регистр SIS

Номер	31...6	5	4	3	2	1	0
Доступ	U	U	R/W	R/W	R/W	R/W	R/W
Сброс	0	1	0	0	0	0	0
	-	-	SC NAK SENT	SC SOF REC	SC RESET EV	SC RESUME	SC TDONE

Таблица 215 – Описание бит регистра USB_SIS

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...6	-	Зарезервировано
5	-	Зарезервировано
4	SC NAK SENT	При ответе NAK на запрос от хоста автоматически устанавливается в 1. Очищается записью 1
3	SC SOF REC	При принятии пакета SOF от хоста автоматически устанавливается в 1. Очищается записью 1
2	SC RESET EV	Автоматически устанавливается в 1 при наличии состояния сброса на шине USB. Очищается записью 1
1	SC RESUME	Автоматически устанавливается в 1 при обнаружении состояния возобновления передачи. Очищается записью 1
0	SC TDONE	Автоматически устанавливается в 1 после успешного выполнения передачи. Очищается записью 1
Примечание – Для организации надежного сброса статусных флагов прерываний необходимо учитывать требования, изложенные в подразделе 31.3		

21.9.3.8 MDR_USB->SIM

Таблица 216 – Регистр SIM

Номер	31...5	4	3	2	1	0
Доступ	U	R/W	R/W	R/W	R/W	R/W
Сброс	0	0	0	0	0	0
	-	SC NAK SENT IE	SC SOF RECIE	SC RESET EVIE	SC RESUME IE	SC TDONE IE

Таблица 217 – Описание бит регистра B_SIM

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...5	-	Зарезервировано

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
4	SC NAK SENT IE	Флаг управления разрешением прерывания при отправке NAK: 1 – разрешено прерывание; 0 – запрещено прерывание
3	SC SOF RECIE	Флаг управления разрешением прерывания при приеме SOF: 1 – разрешено прерывание; 0 – запрещено прерывание
2	SC RESET EVIE	Флаг управления разрешением прерывания при состоянии сброса на шине: 1 – разрешено прерывание; 0 – запрещено прерывание
1	SC RESUME IE	Флаг управления разрешением прерывания при состоянии возобновления передачи: 1 – разрешено прерывание; 0 – запрещено прерывание
0	SC TDONE IE	Флаг управления разрешением прерывания при окончании передачи: 1 – разрешено прерывание; 0 – запрещено прерывание

21.9.3.9 MDR_USB->SA

Таблица 218 – Регистр SA

Номер	31...7	6...0
Доступ	U	R/W
Сброс	0	0
	-	SDEVADDR[6:0]

Таблица 219 – Описание бит регистра SA

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...7	-	Зарезервировано
6...0	SDEVADDR[6:0]	Функциональный адрес устройства USB

21.9.3.10 MDR_USB->SFN

Таблица 220 – Регистр SFN

Номер	31...11	10...0
Доступ	U	R/W
Сброс	0	0
	-	FRAME NUM [10:0]

Таблица 221 – Описание бит регистра SFN

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...11	-	Зарезервировано
10...0	FRAME NUM [10:0]	Номер фрейма, принятый в последнем SOF

21.9.3.11 MDR_USB->SEP[x].RXFD

Таблица 222 – Регистр SEP[x].RXFD

Номер	31...8	7...0
Доступ	U	R/W
Сброс	0	0
	-	RX FIFO DATA[7:0]

Таблица 223 – Описание бит регистра SEP[x].RXFD

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...8	-	Зарезервировано
7...0	RX FIFO DATA[7:0]	После приема OUTDATA_TRANS или SETUP_TRANS пакета, принятые данные читаются из регистра RX_FIFO_DATA

21.9.3.12 MDR_USB->SEP[x].RXFDC

Таблица 224 – Регистр SEP[x].RXFDC

Номер	31...16	15...0
Доступ	U	R/W
Сброс	0	0
	-	FIFO DATA COUNT [15:0]

Таблица 225 – Описание бит регистра SEP[x].RXFDC

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...16	-	Зарезервировано
15...0	FIFO DATA COUNT [15:0]	Отображает число принятых байт в очереди

21.9.3.13 MDR_USB->SEP[x].RXFC

Таблица 226 – Регистр SEP[x].RXFC

Номер	31...1	0
Доступ	U	R/W
Сброс	0	0
	-	FIFO FORCE EMPTY

Таблица 227 – Описание бит регистра SEP[x].RXFC

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...1	-	Зарезервировано
0	FIFO FORCE EMPTY	Запись 1 очищает всю очередь

21.9.3.14 MDR_USB->SEP[x].TXFD

Таблица 228 – Регистр SEP[x].TXFD

Номер	31...8	7...0
Доступ	U	R/W
Сброс	0	0
	-	TX FIFO DATA[7:0]

Таблица 229 – Описание бит регистра SEP[x].TXFD

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...8	-	Зарезервировано
7...0	TX FIFO DATA [7:0]	Перед приемом IN_TRANS в очередь записываются данные для отправки

21.9.3.15 MDR_USB->SEP[x].TXFDC

Таблица 230 – Регистр SEP[x].TXFDC

Номер	31...1	0
Доступ	U	R/W
Сброс	0	0
	-	FIFO FORCE EMPTY

Таблица 231 – Описание бит регистра SEP[x].TXFDC

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...1	-	Зарезервировано
0	FIFO FORCE EMPTY	Запись 1 очищает всю очередь

22 Контроллер интерфейса MDR_CAN

В микросхеме реализовано два независимых контроллера интерфейса CAN. Они являются полнофункциональными CAN-узлами, отвечающими требованиям к активным и пассивным устройствам CAN 2.0A и 2.0B и поддерживающими передачу данных на скорости не более 1 Мбит/сек.

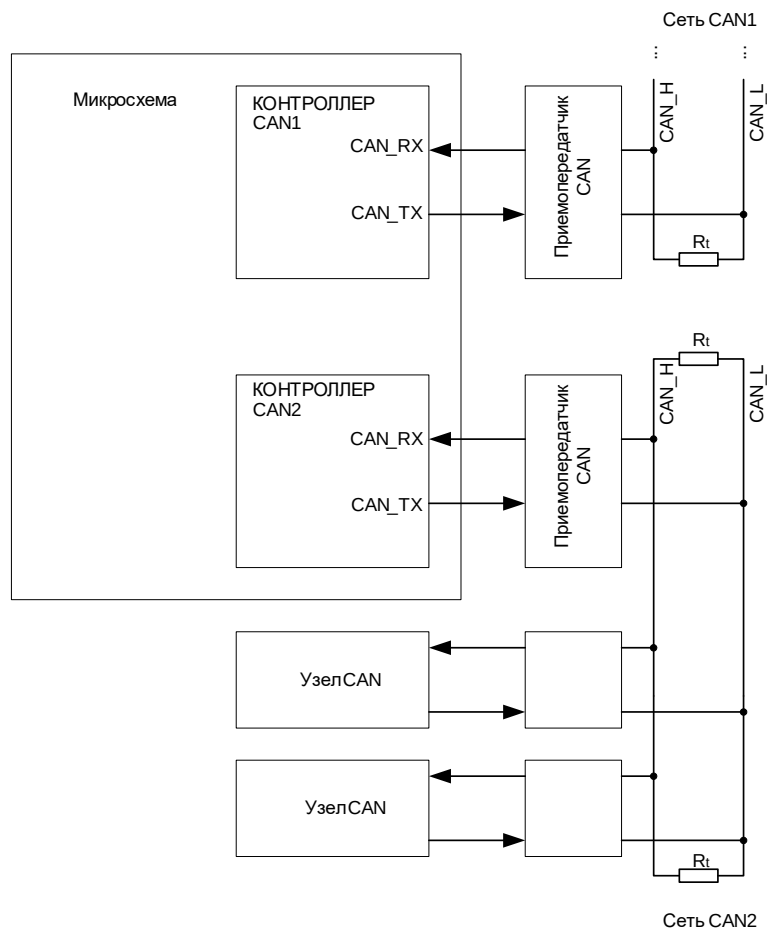


Рисунок 39 – Структурная блок – схема организации сети CAN

Интерфейс CAN позволяет обмениваться сообщениями в сети равноправных устройств. При передаче сообщения в сети CAN все узлы сети получают это сообщение. В сообщении передается уникальный идентификатор и данные. Все сообщения в протоколе CAN могут содержать не более восьми байтов данных. При возникновении коллизий (одновременная передача сообщений различными узлами) при передаче идентификатора происходит арбитраж, и узел, передающий сообщение с большим номером идентификатора, уступает сеть узлу, передающему сообщение с меньшим номером идентификатора.

Особенности:

- поддержка CAN протокола версии CAN 2.0 A и B;
- скорость передачи до 1 Мбит/с;
- 32 буфера приема/передачи;
- поддержка приоритетов сообщений;
- 32 фильтра приема;
- маскирование прерываний.

22.1 Режимы работы

CAN-контроллер поддерживает несколько режимов работы: нормальный режим для приема и передачи пакетов сообщений, режим работы только на прием, режим самотестирования и режим инициализации для задания параметров связи.

- Режим нормальной передачи (регистр CAN_STATUS : ROM = 0, STM = 0)

Выводы CAN_TX и CAN_RX подключены к шине.

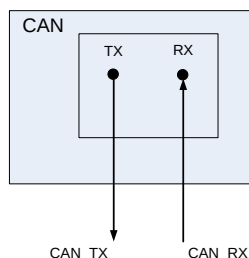


Рисунок 40 – Режим нормальной передачи

В этом режиме можно установить флаги разрешения приема своих пакетов и разрешения подтверждения своих пакетов посылкой ACK (регистр CAN_CONTROL поля SAP и ROP).

- Режим работы только на прием – Receive Only Mode (регистр CAN_STATUS: ROM = 1, STM = 0)

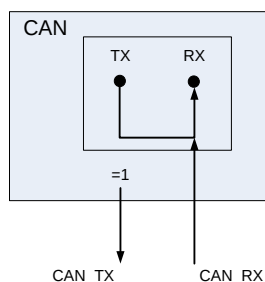


Рисунок 41 – Режим работы только на прием - Receive Only Mode

Контроллер CAN интерфейса принимает, но не посылает никакой информации, т.е. линия TX всегда в «1», но внутри контроллера все управляющие сигналы проходят.

- Режим самотестирования – Self Test Mode (регистр CAN_STATUS : STM = 1, ROM = 0)

Выводы CAN_TX и CAN_RX отключены, вся передаваемая информация видна только внутри контроллера.

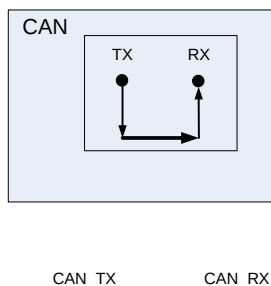


Рисунок 42 – Режим самотестирования - Self Test Mode

Для успешного приема своих сообщений необходимо установить флаги разрешения приема своих пакетов и разрешения подтверждения своих пакетов посылкой ACK (регистр CAN_CONTROL поля SAP и ROP). В этом режиме передаваемые сообщения сразу же принимаются в приемный буфер. Режим самотестирования полезен в период отладки кода программы.

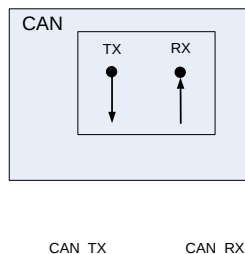


Рисунок 43 – Режим инициализации для задания параметров связи

Еще одна важная функция CAN-контроллера – фильтрация получаемых сообщений. Поскольку CAN является широковещательной шиной, каждое переданное сообщение принимается всеми узлами шины. В CAN-шине любой разумной степени сложности передается достаточно большое число сообщений. Задачей каждого подключенного к CAN-узлу ЦПУ является реагирование на CAN-сообщения. Таким образом, чтобы избавить CAN-контроллер от проблемы приема в буфер нежелательных сообщений, необходима их фильтрация. У CAN-контроллера имеется 32 регистра фильтров и 32 регистра масок, которые можно использовать для блокировки всех CAN-сообщений, кроме избранных сообщений или групп сообщений.

22.2 Типы пакетов сообщений

Информация на шине представлена в виде фиксированных сообщений различной, но ограниченной длины. Когда шина свободна, любой подключенный узел может начать передавать новое сообщение. При передаче информации с помощью протокола CAN используется четыре типа пакетов:

- **пакет удаленного запроса данных** передается узлом, чтобы запросить передачу пакета данных с тем же самым идентификатором;
- **пакет ошибки** передается любым узлом при обнаружении ошибочного состояния на шине. Пакет ошибки передается сразу же после обнаружения ошибки и накладывается на передаваемый пакет так, чтобы испортить его окончательно. Таким образом, если один из узлов обнаружил ошибку, он усиливает ошибку для того, чтобы ее обнаружили и другие узлы;
- **пакет перегрузки** используется для обеспечения дополнительной задержки между предшествующим и последующим кадрами данных или кадрами удаленного запроса данных. Он передается в редких случаях, подробнее можно прочесть в стандарте ISO 11898-1. Контроллер CAN интерфейса отправляет пакет перегрузки в соответствии со стандартом;
- основными пакетами на шине CAN являются **пакеты данных**. В пакете данных узел передает данные от передатчика приемнику. Пакеты могут быть стандартными и расширенными. Отличие пакетов заключается в размере полей

идентификатора. Пакеты с 11-разрядным идентификатором – называются стандартными пакетами, пакеты, содержащие 29 разрядные идентификаторы, называются расширенными пакетами. При передаче идентификационной информации происходит автоматический арбитраж на шине CAN таким образом, чтобы пакет с меньшим значением поля ID остался на шине. На шине не допускается одновременная передача двух или более сообщений с одним и тем же идентификатором. Размер передаваемых данных кодируется в поле DLC и может составлять от 0 до 8 байт. После передачи поля данных контроллер автоматически передает рассчитанное значение CRC. Если хотя бы один из узлов принял пакет, то он выставляет ACK подтверждение на шине, если хотя бы один из узлов обнаружит ошибку, то на шину будет выставлен пакет ошибки. Таким образом, гарантируются доставки сообщений.

Пакеты данных и пакеты удаленного запроса данных отделяются от предшествующих пакетов межкадровым пространством.

22.3 Структура пакета данных (Data Frame)

Пакет данных состоит из семи различных полей:

- «начало пакета» (SOF-start of frame);
- «поле арбитража» (arbitration field);
- «поле контроля» (control field);
- «поле данных» (data field);
- «поле CRC» (CRC field);
- «поле подтверждения» (ACK field);
- «конец пакета» (end of frame).

Поле данных может иметь нулевую длину.

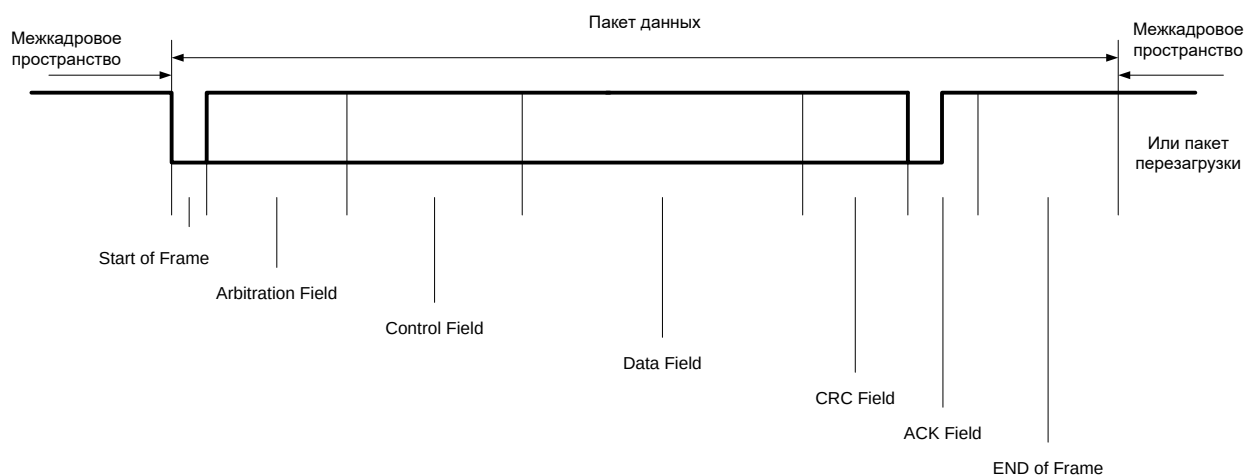


Рисунок 44 – Пакет сообщения CAN

В терминах протокола CAN логическая единица называется рецессивным битом, а логический ноль называется доминантным битом. Во всех случаях доминантный бит будет подавлять рецессивный. То есть, если несколько узлов выставят на шину рецессивный бит, а один – доминантный, то обратно всеми узлами будет считан доминантный бит.

22.3.1 Начало пакета (Start of frame)

Начало пакета отмечает начало пакета данных или пакета удаленного запроса данных. Это поле состоит из одиночного доминантного бита. Узлу разрешено начать передачу, когда шина свободна. Все узлы должны синхронизироваться по фронту, вызванному передачей поля «начало пакета» узла, начавшего передачу первым.

22.3.2 Поле арбитража (Arbitration field)

Формат поля арбитража отличается для стандартного и расширенного форматов:

– в стандартном формате поле арбитража состоит из 11-разрядного идентификатора и бита RTR;

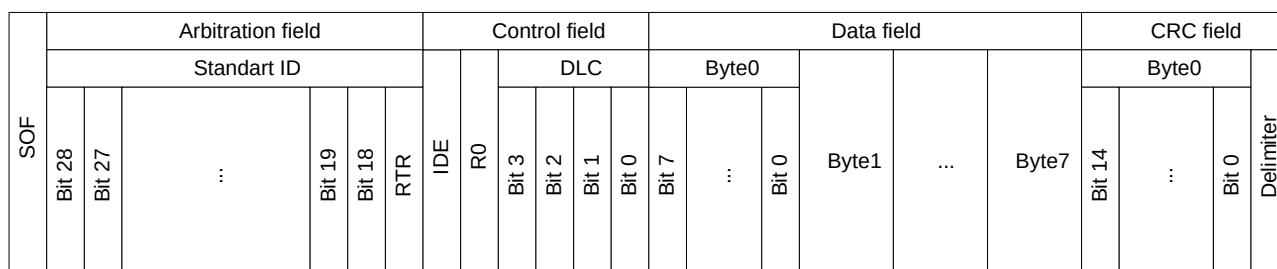


Рисунок 45 – Структура стандартного пакета данных

– в расширенном формате поле арбитража состоит из 29-разрядного идентификатора, бита SRR, бита IDE и бита RTR.

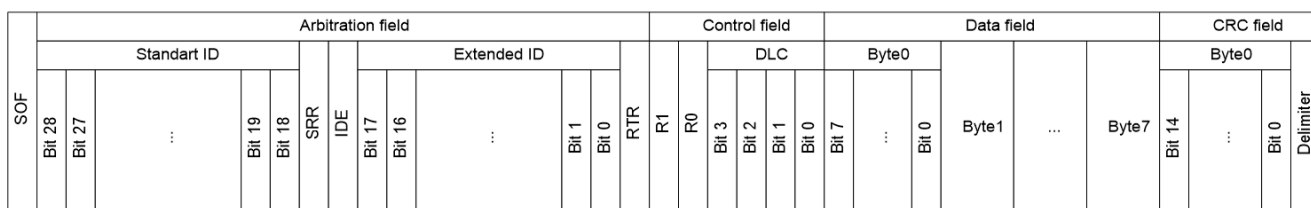


Рисунок 46 – Структура расширенного пакета данных

22.3.2.1 Идентификатор

Идентификатор – стандартный формат. Длина идентификатора – 11 бит и соответствует Standart ID в расширенном формате. Эти биты передаются в порядке Bit28 ... Bit18. Самый младший бит - Bit18. 7 старших бит (Bit28 - Bit 22) не должны быть все единичными битами.

Идентификатор – расширенный формат. В отличие от стандартного идентификатора, расширенный идентификатор состоит из 29 бит. Его формат содержит две секции:

- Standart ID – 11 бит;
- Extended ID – 18 бит.

Standart ID состоит из 11 бит. Эта секция передается в порядке от Bit28 ... Bit18. Это эквивалентно формату стандартного идентификатора. Standart ID определяет базовый приоритет расширенного пакета.

Extended ID состоит из 18 бит. Эта секция передается в порядке от Bit17 до Bit0. В стандартном пакете идентификатор сопровождается битом RTR.

22.3.2.2 Бит RTR

Бит запроса удаленной передачи. В пакетах данных бит RTR должен быть передан нулевым уровнем. Внутри пакета удаленного запроса данных бит RTR должен быть единичным. В расширенном пакете сначала передается Standart ID, с последующими битами IDE и SRR. Extended ID передается после бита SRR.

22.3.2.3 Бит SRR (расширенный формат)

Заменитель бита удаленного запроса. Бит SRR – единичный бит. Он передается в расширенных пакетах в позиции бита RTR. Таким образом, он заменяет бит RTR – бит стандартного пакета.

Следовательно, при одновременной передаче стандартного пакета и расширенного пакета, Standart ID которого совпадает с идентификатором стандартного пакета, стандартный пакет преобладает над расширенным пакетом.

22.3.2.4 Бит IDE (расширенный формат)

Бит расширения идентификатора

Бит IDE принадлежит:

- полю арбитража для расширенного формата;
- полю управления для стандартного формата.

Бит IDE в стандартном формате передается нулевым уровнем, в расширенном формате бит IDE – единичный уровень.

22.3.3 Поле управления (Control field)

Поле управления состоит из шести бит. Формат поля управления отличается для стандартного и расширенного формата.

Пакеты в стандартном формате включают: код длины данных (DLC), бит IDE, который передается нулевым уровнем (см. выше), и зарезервированный бит r0.

Пакеты в расширенном формате включают код длины данных и два зарезервированных бита r1 и r0. Зарезервированные биты должны быть посланы нулевым уровнем, но приемники принимают единичные и нулевые уровни биты во всех комбинациях.

22.3.3.1 Код длины данных (Data length code)

Число байт в поле данных обозначается кодом длины данных. Этот код длины данных, размером четыре бита, передается внутри поля управления. Допустимое число байт данных: {0,1, ..., 7,8}. Другие величины использоваться не могут.

22.3.4 Поле данных (Data field)

Поле данных состоит из данных, которые будут переданы внутри пакета данных. Оно может содержать от 0 до 8 байт, каждый содержит 8 бит, которые передаются, начиная со старшего значащего бита.

22.3.5 Поле CRC (CRC field)

Содержит последовательность CRC и CRC – разделитель. При вычислении 15-битного кода CRC используется последовательность бит, состоящая из полей: «начало пакета», «поле арбитража», «управляющее поле», «поле данных» (если есть). Последовательность CRC сопровождается разделителем CRC, который состоит из одного единичного бита.

22.3.6 Поле подтверждения (ACK field)

Поле подтверждения имеет длину два бита и содержит: «область подтверждения» и разделитель подтверждения. В поле подтверждения передающий узел посылает два бита с единичным уровнем. Приемник, который получил сообщение правильно (CRC соответствует), сообщает об этом передатчику, посылая бит с нулевым уровнем в течение приема поля "область подтверждения".

22.3.7 Конец пакета (End of frame)

Каждый пакет данных и пакет удаленного запроса данных ограничен последовательностью флагов, состоящей из семи единичных бит.

22.3.8 Структура пакета удаленного запроса данных (Remote frame)

Узел, действующий как приемник некоторых данных, может инициировать передачу соответственных данных исходными узлами, посылая пакет удаленного запроса данных. Пакет удаленного запроса данных существует и в стандартном формате, и в расширенном формате. В обоих случаях он состоит из шести битовых полей:

- «начало пакета» (Start of frame);
- «поле арбитража» (Arbitration field);
- «управляющее поле» (Control field);
- «поле CRC» (CRC - field);
- «поле подтверждения» (ACK field);
- «конец пакета» (End of frame).

В отличие от обычного пакета данных, бит RTR пакета удаленного запроса данных – единичный. В этом пакете отсутствует поле данных. При этом значение кода длины данных может принимать любое значение в пределах допустимого диапазона [0,8]. Значение кода длины данных соответствует коду длины данных кадра данных. Бит RTR указывает, является ли переданный кадр кадром данных.

22.3.9 Арбитраж на шине

Арбитраж сообщений гарантирует, что наиболее важное сообщение захватит шину и будет передано без задержки. Затем будут переданы приостановленные сообщения согласно их приоритетам (сообщение с наименьшим идентификатором передается первым).

Если планируется передача сообщения, и шина свободна, то сообщение будет передано и сможет быть принято любым заинтересованным в нем узлом. Если передача сообщения запланирована, а шина активна, то прежде чем приступить к передаче

сообщения, необходимо дождаться освобождения шины. Если запланирована передача нескольких сообщений, то при освобождении шины они начнут передаваться одновременно, синхронизируясь по признаку начала пакета. В этом случае на шине начнется процесс арбитража, задача которого – определить, какое именно из сообщений захватит шину и будет передано.

Арбитраж сообщений на шине CAN осуществляется методом, который называется «неразрушающий побитовый арбитраж».

На рисунке 47 изображены три сообщения, ожидающие передачи. После освобождения шины и синхронизации пакетов сообщений по старт-биту на шину начинают выдаваться все три идентификатора. При передаче первых двух бит все три узла выставляют на шину одинаковые логические уровни и соответственно считывают те же значения, поэтому они все продолжают передачу. Однако при передаче третьего бита узлы А и С выставляют на шину доминантный бит, а узел В выставляет рецессивный бит, но при этом считывает с шины доминантный. В результате узел В освобождает шину и начинает следить за ее состоянием. Узлы А и С продолжают передачу, пока ситуация не повторится; теперь узел С выдает рецессивный бит, а узел А – доминантный. При этом узел С прекращает передачу и начинает следить за состоянием шины. С этого момента шина захватывается узлом А. После передачи сообщения узлом А узлы В и С начинают передачу, причем узел С захватит шину и передает свое сообщение. Если бы узлу А снова надо было передавать сообщение, он снова захватил бы шину. Таким образом, первым на шине CAN передается сообщение с наименьшим идентификатором.

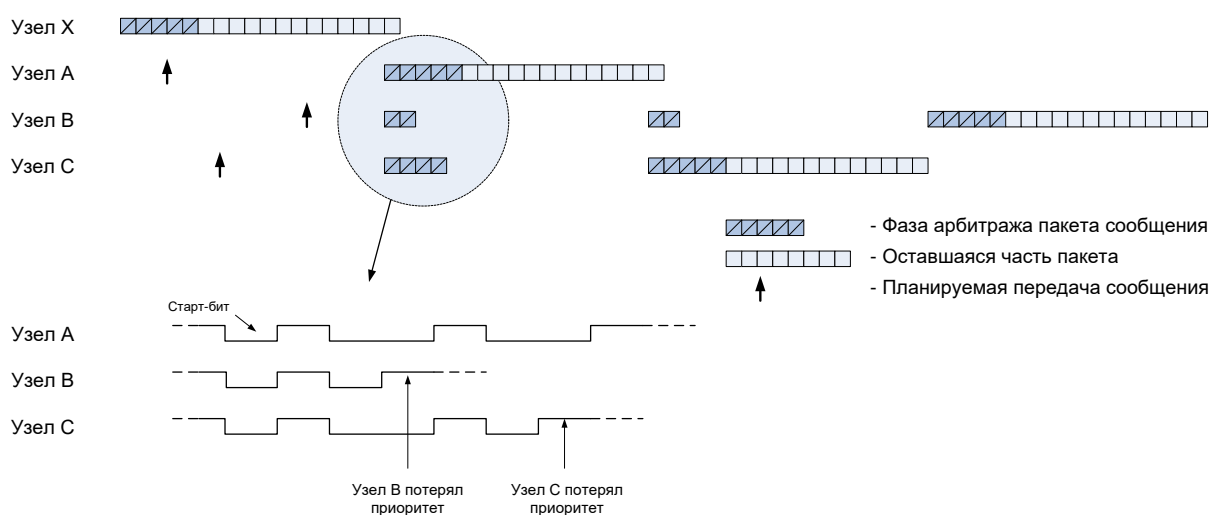


Рисунок 47 – Арбитраж на шине CAN

В случае «проигрыша» арбитража в регистре статуса контроллера CAN будет установлен флаг ID_LOWER.

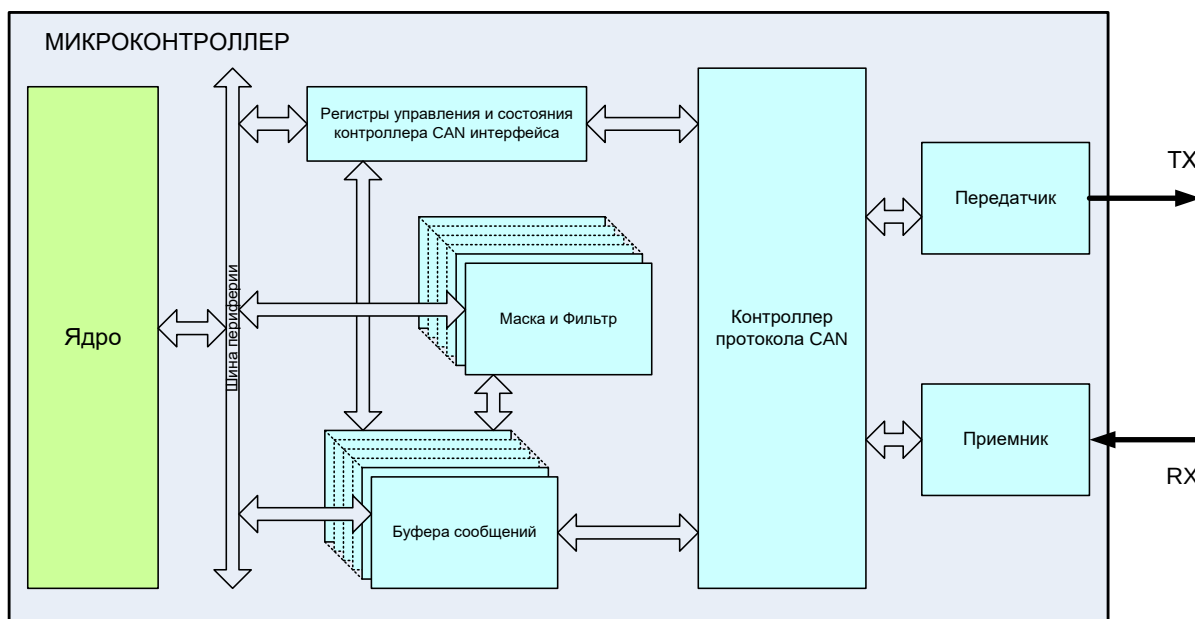


Рисунок 48 – Структурная блок-схема контроллера CAN

22.4 Инициализация

Перед началом работы с контроллерами CAN в первую очередь должны быть заданы параметры их тактового сигнала. Параметры задаются в блоке «Сигналы тактовой частоты».

Для задания тактовой частоты блока необходимо установить бит разрешения тактирования блока (бит 0 для CAN1, бит 1 для CAN2 регистра PER_CLOCK). В регистре CAN_CLOCK установить бит CANyCLKEN, чтобы разрешить тактовую частоту для определенного контроллера CAN, задать коэффициент деления тактовой частоты HCLK для каждого CAN контроллера.

После подачи тактового сигнала на контроллер CAN можно приступить к работе с ним.

Для работы контроллера шины CAN он должен быть настроен на соответствующую скорость шины CAN. Для этого должны быть заданы соответствующим образом поля SB, SJW, SEG2, SEG1, PSEG и BRP в регистре CAN_BITTMNG. После этого должны быть заданы работающие буфера сообщений путем задания бит EN (разрешение работы) RXTXn (1 – прием, 0 – передача) в регистре BUF_xx_CON. После этого должен быть выдан общий сигнал разрешения работы контроллера через задание бита CANEN в регистре CONTROL. После этого контроллер CAN начинает работу.

22.5 Передача сообщений

Для передачи сообщения необходимо в разрешенный для работы и конфигурируемый на передачу буфер записать сообщение для передачи (задать значения регистрам CAN_BUF[x].ID, CAN_BUF[x].DLC, CAN_BUF[x].DATA1 и CAN_BUF[x].DATAH), после чего установить бит TX_REQ. После установки этого бита сообщение будет поставлено в очередь на отправку. После отправки сообщения бит TX_REQ будет автоматически сброшен. Если в нескольких буферах есть сообщения

на отправку, то порядок отправки определяется по полю PRIOR_0. Если у сообщения бит PRIOR_0 выставлен в ноль, то оно отправляется в первую очередь. Если есть несколько сообщений с одинаковым приоритетом, то порядок отправки определяется порядковым номером буфера, буфер с меньшим порядковым номером имеет больший приоритет. Значение полей ID для выбора порядка отправки в рамках контроллера CAN (одного узла) значения не имеет. По ID выбирается приоритет между различными узлами.

22.6 Передача сообщений по Remote Transmit Request (RTR)

Для автоматической отправки сообщения по запросу Remote Transmit Request (далее – RTR) необходимо задать режим маскирования для данного буфера таким образом, чтобы он принимал только сообщения от устройства, которое может выслать запрос RTR. В регистре INT_TX при необходимости настроить генерацию прерывания передачи для соответствующего буфера. В регистре управления этим буфером (BUFF_CON[x]) проверить, что флаг TX_REQ = 0, задать приоритет отправляемого сообщения PRIOR_0, установить разрешение ответа при приеме RTR в буфер (RTR_EN=1), задать RX_TX = 0 для разрешения отправки сообщения и задать EN = 1 для разрешения работы буфера. В регистре идентификации задать необходимые SID и EID, в регистре BUF_xx_DLC указать формат пакета (расширенный или стандартный) и указать длину передаваемых данных в поле DLC. В регистрах данных CAN_BUF[x].DATA1 и CAN_BUF[x].DATA2 задать необходимые для отправки данные. Далее можно переходить к выполнению остальной части программы с отправкой сообщений CAN. Отправка сообщения буфером будет произведена по запросу RTR, удовлетворяющему механизму фильтрации для принимаемых сообщений, который выбран для данного буфера.

Если запрос RTR удовлетворяет механизму фильтрации нескольких буферов, то ответы будут отправлены из каждого из этих буферов.

22.7 Прием сообщений

Для приема сообщений необходимо иметь свободные и разрешенные для работы буфера, сконфигурированные на прием сообщений. При этом если по шине CAN будут передаваться сообщения от других узлов, они будут сохраняться в этих буферах.

22.8 Автоматическая фильтрация принимаемых сообщений

Для уменьшения затрат процессорного ядра на обработку принимаемых сообщений, контроллер CAN интерфейса может автоматически фильтровать принимаемые сообщения. Для каждого буфера могут быть заданы маска (CAN_BUF_FILTER[x].MASK) и фильтр (CAN_BUF_FILTER[x].FILTER) таким образом, что в этот буфер будут приниматься только те сообщения, для которых выполняется условие

$$ID \& CAN_BUF_FILTER[x].MASK == CAN_BUF_FILTER[x].FILTER.$$

Если принимаемое сообщение не может быть помещено ни в один из буферов, оно будет проигнорировано. Если сообщение может быть принято более чем одним

буфером, оно будет помещено в буфер с меньшим порядковым номером. При инициализации после включения питания или сброса CAN_BUF_FILTER[x].MASK и CAN_BUF_FILTER[x].FILTER для всех буферов имеют произвольное значение, таким образом, необходимо перед началом работы их проинициализировать. Для приема всех сообщений без фильтрации необходимо задать им нулевое значение. Специального бита для включения или выключения фильтрации нет.

22.9 Перезапись принятых сообщений

В буфере может быть включено разрешение перезаписи принятого сообщения. Если принимаемое сообщение не может быть сохранено в свободный буфер, то оно может быть сохранено в буфер с ранее полученным сообщением, если для него выставлен бит OVER_EN. При этом выставляется флаг OVER_WR. Таким образом, если у буфера разрешена перезапись принятых сообщений, после прочтения сообщения необходимо проверить флаг OVER_WR. Если он выставлен в 1, то необходимо сбросить OVER_WR (не сбрасывая флаг RX_FULL), затем еще раз прочесть сообщение, после чего снова проверить флаг OVER_WR и, если он не выставлен повторно, то сбросить флаг RX_FULL. И считанное значение считать корректным.

Прибегать к помощи механизма перезаписи принятых сообщений можно только в случае, когда допустима потеря сообщений, работа с перезаписью сообщений не гарантирует прием всех сообщений, а только позволяет принять сообщение корректно, так как момент чтения сообщения может совпасть с моментом сохранения нового сообщения. При этом первая часть считанного процессорным ядром сообщения будет от первого сообщения, вторая от второго. Если же между сбросом флага OVER_WR, чтением сообщения и при следующей проверке OVER_WR он оказался не выставлен, это означает, что в момент чтения сообщения из буфера в него не сохранялось новое сообщение.

22.10 Задание скорости передачи и момента семплирования

Все узлы шины CAN должны работать на одной скорости. Протокол CAN использует кодирование без возврата в ноль (NRZ). Также при передаче не передаются тактовые сигналы. Таким образом, приемники должны синхронизоваться с тактовым сигналом передатчика. Поскольку все узлы имеют свои индивидуальные тактовые генераторы, все приемники имеют специальный блок синхронизации DPLL.

Максимальная скорость передачи CAN 1 Мбит/сек. Время битового интервала Nominal Bit Time определяется как

$$TBIT = 1/\text{Скорость передачи}$$

Блок DPLL разбивает битовый интервал на интервалы Time Quanta (TQ). Битовый интервал состоит из 4 частей:

- Synchronization Segment (Sync_Seg);
- Propagation Time Segment (PSEG);
- Phase Buffer Segment 1 (SEG1);
- Phase Buffer Segment 2 (SEG2).

По определению Nominal Bit Time программируется длительностью от 8 до 25 TQ. В этом случае

$$Normal\ Bit\ Time = TQ \cdot (Sync_Seg + PSEG + SEG1 + SEG2) . \quad (5)$$

Время TQ фиксировано и определяется периодом генератора и программируемым делителем BRP со значением от 1 до 65536:

$$TQ(\mu s) = \frac{BRP + 1}{CANx_CLK(MHz)} \quad (6)$$

или

$$TQ(\mu s) = (BRP + 1) \cdot Tclk(\mu s) . \quad (7)$$

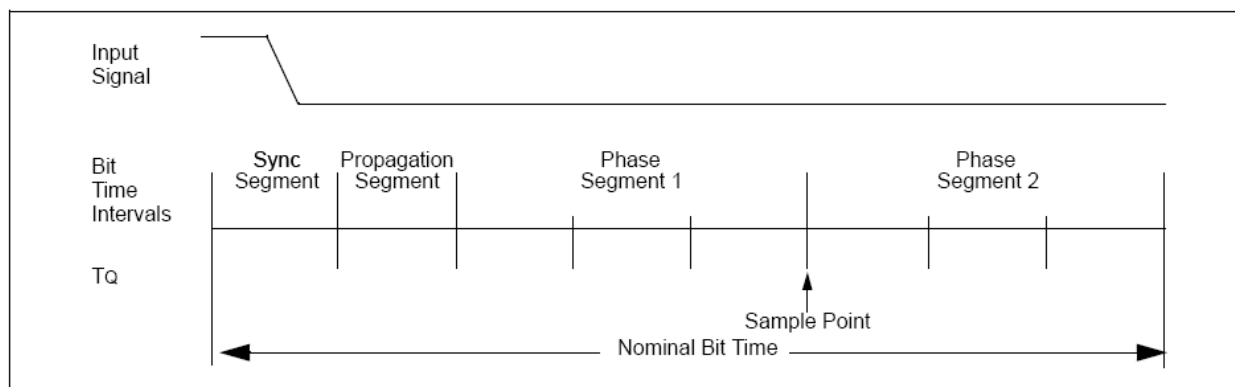


Рисунок 49 – Структура битового интервала

Synchronization Segment

Эта часть битового интервала, в которой должно происходить переключение сигнала. Длительность этого интервала 1 TQ. Если переключение происходит в этой области, то приемник синхронизирован с передатчиком.

Propagation Time Segment

Эта часть предназначена, чтобы компенсировать физические задержки времени распространения сигнала в шине и внутренние задержки в узлах. Длительность этого интервала может быть запрограммирована от 1 до 8 TQ

Phase Buffer Segments

Эти интервалы предназначены для более точной установки точки семплирования, которая располагается между ними. Длительности этих интервалов могут быть запрограммированы между 1 и 8 TQ.

22.11 Синхронизация

При обнаружении фронта принимаемого сигнала этот момент принимается как граница между битовыми интервалами; в зависимости от того, на какой интервал

приходится фронт, DPLL выполняет различного рода действия по подсинхронизации данных.

Hard Synchronization

Жесткая синхронизация выполняется однократно во время начала приема сообщения. Независимо от того, в каком состоянии находился DPLL при возникновении фронта, он переводится в Sync_Seg.

Resynchronization

Если фронт принимаемого сигнала отклоняется от Sync_Seg, длительность Phase Segment 1 может быть увеличена, а Phase Segment 2 уменьшена, чтобы в следующий раз фронт прошел в нужном месте. Величина изменения Phase Segment 1 и Phase Segment 2 варьируется в зависимости от значения отклонения фронта, но не превышает значения Synchronization Jump Width (SJW).

22.12 Обработка ошибок

В спецификации протокола CAN определено пять методов ограничения распространения ошибок, реализованных на аппаратном уровне. При обнаружении любой ошибки передающее устройство повторяет посылку пакета, поэтому ядру не нужно вмешиваться до тех пор, пока не возникнет грубая ошибка. Предусмотрено три метода обнаружения ошибок на уровне пакетов (контроль формата, CRC и подтверждение) и два метода на уровне бит (контроль бит и битстаффинг). Для реализации этих методов используется несколько полей, добавляемых к основному сообщению. При приеме осуществляется проверка, все ли поля присутствуют в сообщении. Если нет, то сообщение игнорируется, генерируется кадр ошибки и в регистре статуса контроллера STATUS устанавливается флаг ошибки формата пакета FRAME_ERR.

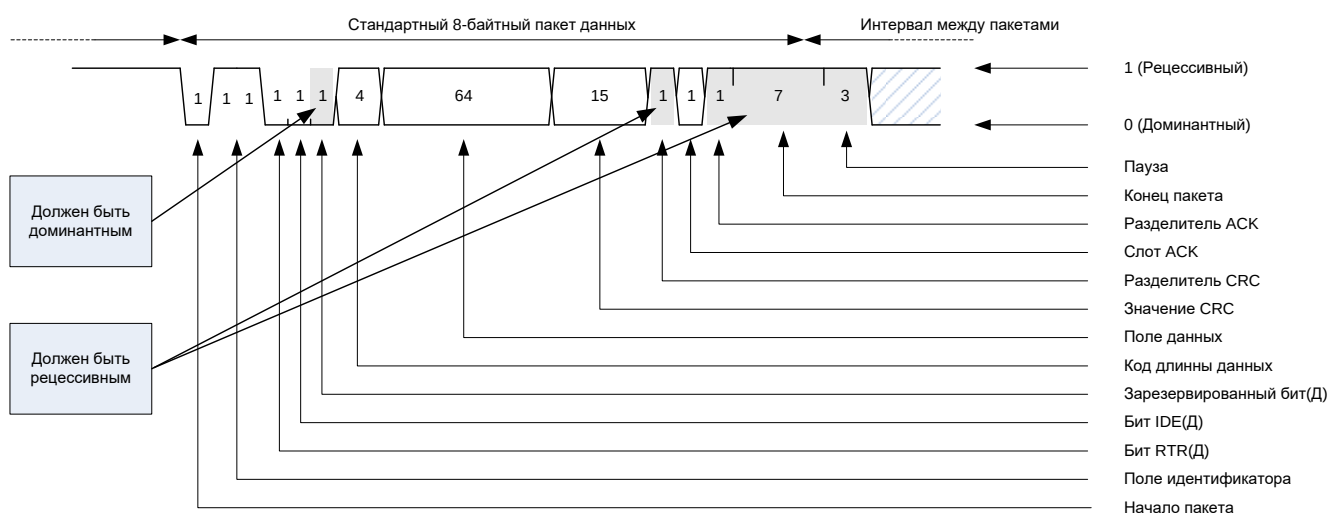


Рисунок 50 – Контроль формата пакета

Каждое сообщение должно подтверждаться вставкой доминантного бита в поле подтверждения. Если подтверждения нет, передающий узел будет передавать сообщение до тех пор, пока не получит подтверждение, при этом в регистре статуса контроллера STATUS будет установлен флаг ошибки подтверждения ACK_ERR.

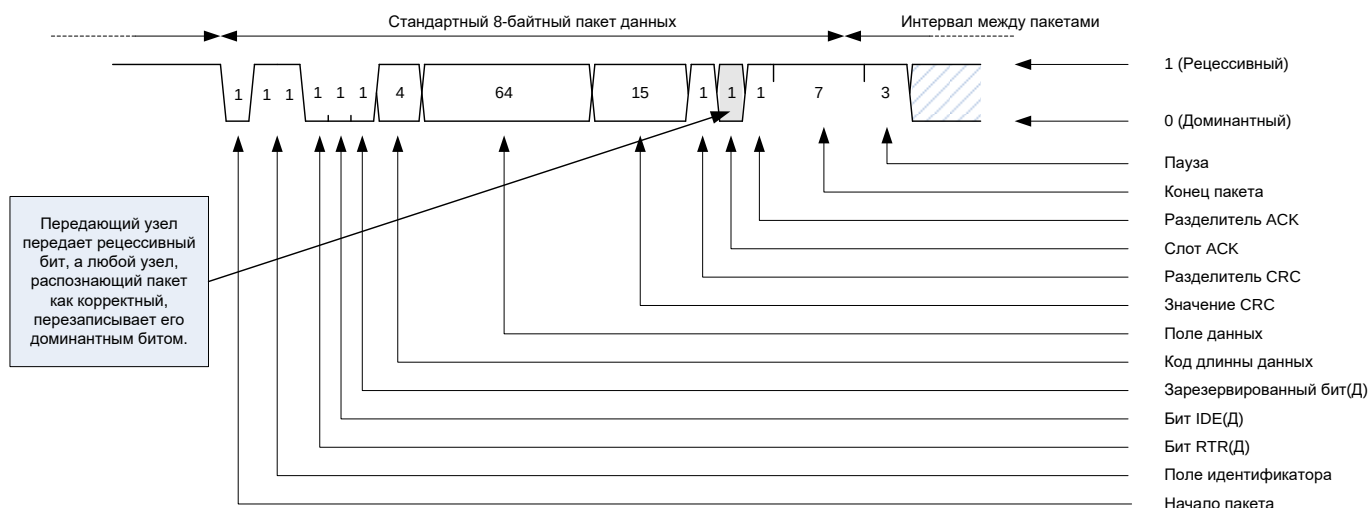


Рисунок 51 – Контроль подтверждения

Пакет сообщения CAN содержит 15-битное значение CRC, которое автоматически генерируется передатчиком и проверяется приемником. С помощью этого кода можно обнаружить и исправить ошибку в четырех битах сообщения от начала кадра до начала поля CRC. Если CRC неверен и сообщение игнорируется, то передается кадр ошибки и в регистре статуса контроллера STATUS будет установлен флаг ошибки контрольной суммы пакета CRC_ERR.

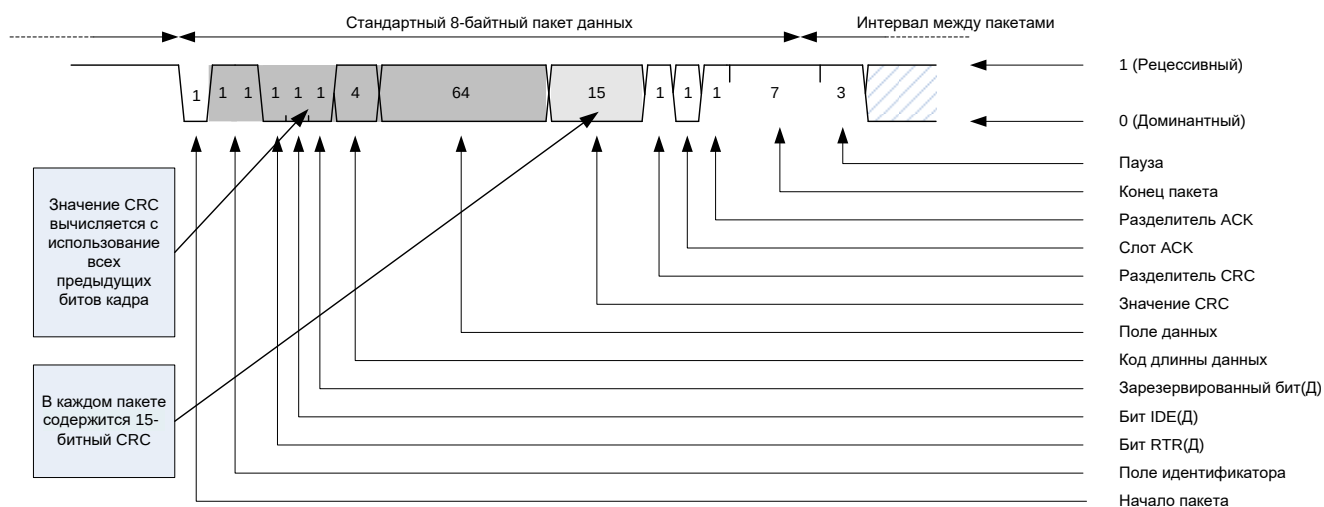


Рисунок 52 – Контроль CRC

После того, как узел выиграет арбитраж, он начинает передачу своего сообщения по шине. Как и во время арбитража, CAN-контроллер считывает обратно каждый бит, выдаваемый им на шину. Поскольку узел уже выиграл арбитраж, больше никто не должен передавать данные на шину, поэтому значение каждого выданного на шину бита должно соответствовать значению, считанному обратно с шины. Если считано неверное значение, передатчик генерирует кадр ошибки, в регистре статуса контроллера STATUS устанавливает флаг ошибки передаваемых бит пакета BIT_ERR и сообщение снова ставит в очередь. Это сообщение будет послано в следующем слоте сообщений, однако при этом оно должно пройти через процесс арбитража с другими запланированными сообщениями.

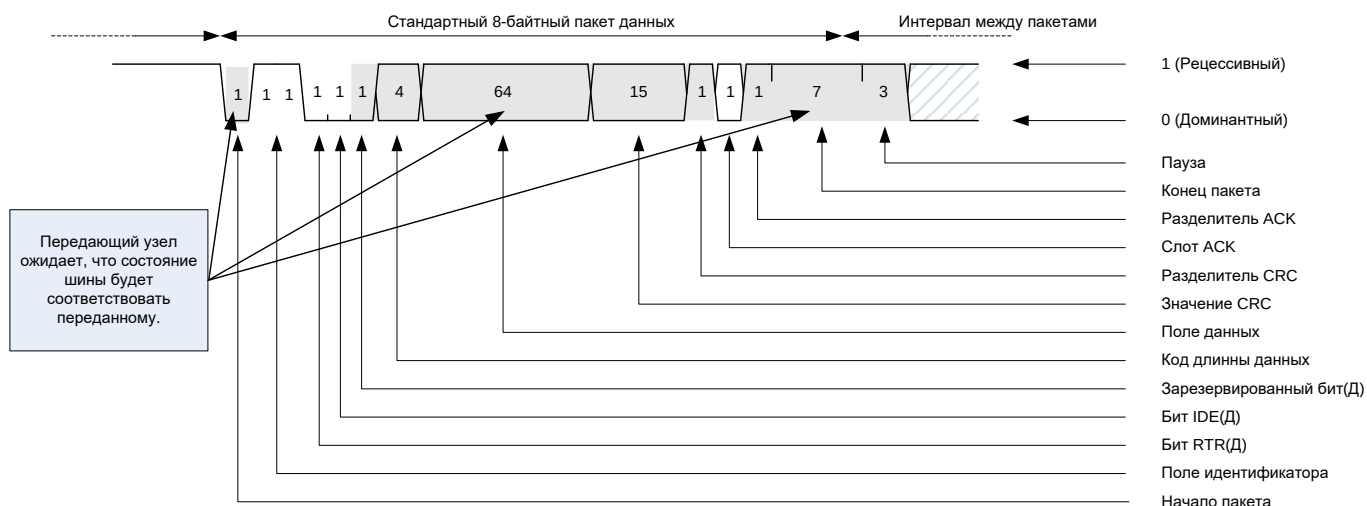


Рисунок 53 – Контроль передаваемых бит

На уровне бит в протоколе CAN реализован также метод вставки бита (битстаффинг). После каждой последовательности из пяти доминантных бит вставляется рецессивный бит; если рецессивный бит не обнаружен, в регистре статуса устанавливается флаг ошибки вставленных бит пакета BIT_STUF_ERR. Этот метод позволяет предотвратить появление на шине постоянных уровней и обеспечивает наличие в потоке бит достаточного количества переходов, используемых для повторной синхронизации. Кадр ошибки в протоколе CAN представляет собой простую последовательность из шести доминантных бит. Это позволяет любому контроллеру CAN формировать на шине сообщение об ошибке сразу после ее обнаружения, не дожидаясь конца сообщения.

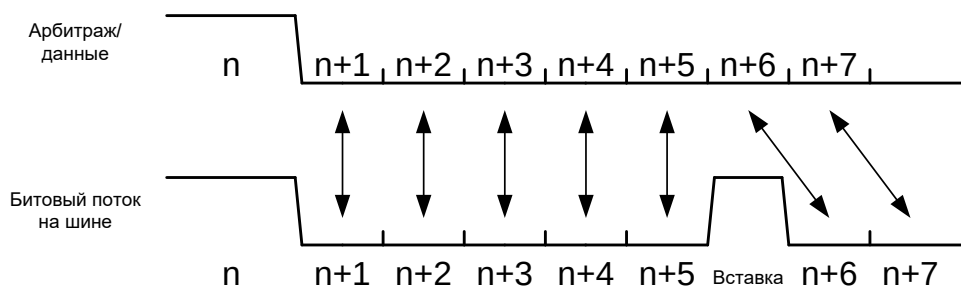


Рисунок 54 – Битстаффинг

В каждом CAN контроллере имеется два счетчика. Этими счетчиками являются счетчик ошибок приема (регистр STATUS, поле RX_ERR_CNT) и счетчик ошибок передачи (регистр STATUS, поле TX_ERR_CNT). Изменение состояния этих счетчиков происходит при приеме или передаче кадра ошибки. Когда любой счетчик достигает значения 128, контроллер CAN переходит в режим «error passive». В этом режиме он продолжает отзываться на кадры ошибки, однако при генерации кадра ошибки он вместо доминантных бит выставляет на шину рецессивные. Если счетчик ошибок передачи достигает значения 255, то контроллер CAN переходит в режим «bus-off» и больше не принимает участия в обмене по шине. Для возобновления обмена (переход CAN-контроллера в состояние Error Active) необходимо либо вмешательство ядра микросхемы,

которое повторно инициализирует CAN-контроллер, либо на шине CAN не должно быть активности (шина в рецессивном состоянии) в течение времени, равного времени передачи $128 \times 11 = 1408$ бит. Текущий статус состояния контроллера можно посмотреть в регистре статуса контроллера STATUS.

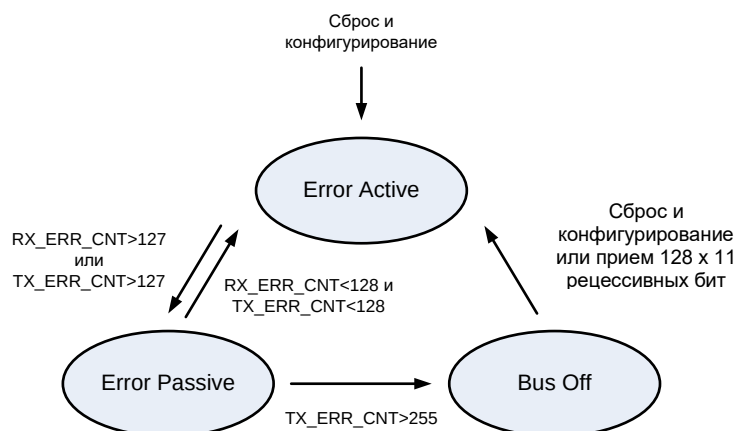


Рисунок 55 – Счетчики ошибок

Контроллер CAN имеет несколько механизмов обнаружения ошибок. Во-первых, из регистра состояния контроллера CAN_STATUS можно считать текущее состояние счетчиков ошибок приема и передачи. Также в этом регистре содержится флаг превышения счетчиками ошибок порогового значения ERROR_OVER. Это значение произвольно и записывается в регистр CAN_OVER. Как и регистры синхронизации, регистр CAN_OVER можно изменять только при нахождении контроллера в состоянии сброса.

22.13 Прерывания

В контроллере CAN в качестве источников прерывания выступают буфера сообщений. Генерируемые прерывания делятся на три группы:

- прерывания передачи (по одному для каждого буфера);
- прерывания приема (по одному для каждого буфера);
- прерывания ошибки.

При возникновении какого-либо прерывания и наличии сигналов разрешения этих прерываний, буфер вырабатывает прерывание. Контроллер CAN объединяет прерывания приема, передачи и ошибки в каждом буфере и вырабатывает прерывание, отображаемое в регистре прерываний периферии. Если прерывание разрешено в регистре, процессор выполняет переход на обработчик прерываний. Обработчик прерываний должен выполнить действия по обработке прерывания и снять его выставление. Прерывание передачи/приема для каждого буфера может быть замаскировано путем установления соответствующего бита в регистрах CAN_INT_TX/CAN_INT_RX. Также есть возможность группового маскирования прерываний по приему, по передаче и по ошибке (см. регистр CAN_INT_EN).

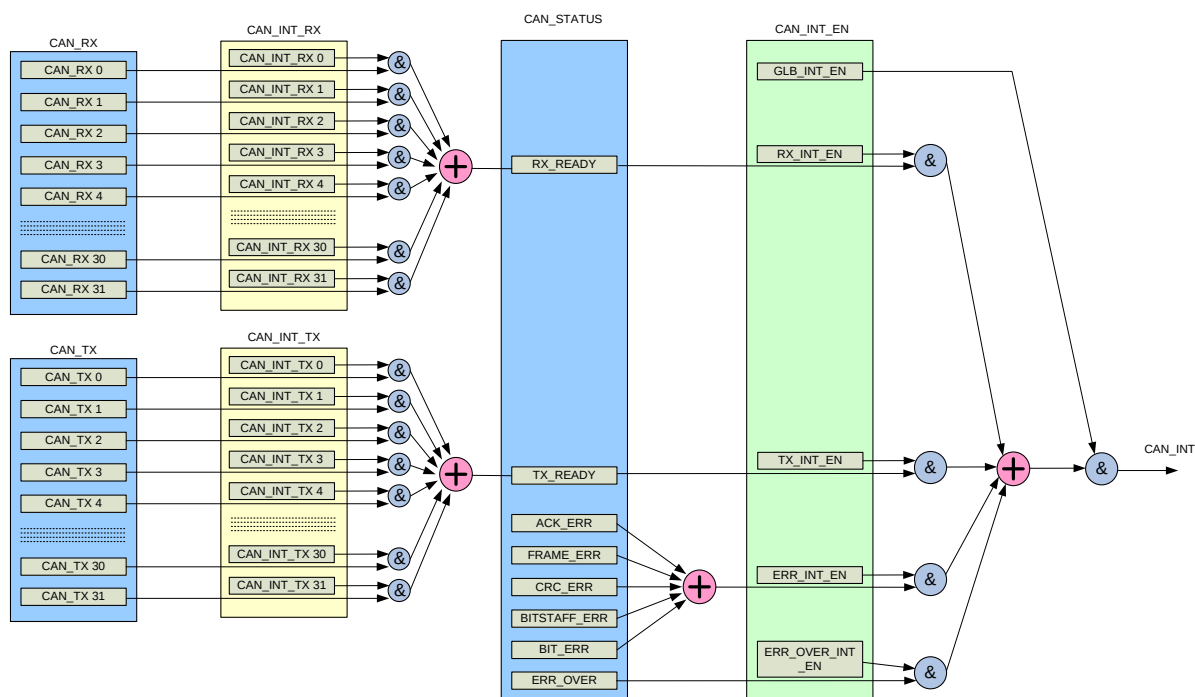


Рисунок 56 – Схема формирования прерывания блока CAN

22.14 Описание регистров контроллера CAN

Таблица 232 – Описание регистров контроллера CAN

Базовый адрес	Название	Описание
0x4000_0000	MDR_CAN1	Контроллер интерфейса CAN1
0x4000_8000	MDR_CAN2	Контроллер интерфейса CAN2
Смещение		
0x00	CONTROL	MDR_CANx->CONTROL Регистр управление контроллером CAN
0x04	STATUS	MDR_CANx->STATUS Регистр состояния контроллера CAN
0x08	BITTMNG	MDR_CANx->BITTMNG Регистр задания скорости работы
0x10	INT_EN	MDR_CANx->INT_EN Регистр разрешения прерываний контроллера
0x1C	OVER	MDR_CANx->OVER Регистр границы счетчика ошибок
0x20	RXID	MDR_CANx->RXID Регистр принятого ID сообщения
0x24	RXDLC	MDR_CANx->RXDLC Регистр принятого DLC сообщения
0x28	RXDATAL	MDR_CANx->RXDATAL Регистр принятых данных
0x2C	RXDATAN	MDR_CANx->RXDATAN Регистр принятых данных

Базовый адрес	Название	Описание
0x30	TXID	MDR_CANx->TXID Регистр передаваемого ID сообщения
0x34	TXDLC	MDR_CANx->TXDLC Регистр передаваемого DLC сообщения
0x38	DATAL	MDR_CANx->TXDATAL Регистр передаваемых данных
0x3C	DATAH	MDR_CANx->TXDATAH Регистр передаваемых данных
0x40	BUF_CON[0]	MDR_CANx->BUF_CON[x] Регистр управления буфером 01
	...	
0xBC	BUF_CON[31]	MDR_CANx->BUF_CON[x] Регистр управления буфером 32
0xC0	INT_RX	Флаги разрешения прерываний от приемных буферов MDR_CANx->INT_RX
0xC4	RX	Флаги RX_FULL от приемных буферов MDR_CANx->RX
0xC8	INT_TX	Флаги разрешения прерываний от передающих буферов MDR_CANx->INT_TX
0xCC	TX	Флаги ~TX_REQ от передающих буферов MDR_CANx->TX
0x200	CAN_BUF[0].ID	MDR_CANx->CAN_BUF[x].ID ID сообщения буфера 01
0x204	CAN_BUF[0].DLC	MDR_CANx->CAN_BUF[x].DLC DLC сообщения буфера 01
0x208	CAN_BUF[0].DATAL	MDR_CANx->CAN_BUF[x].DATAL Данные сообщения буфера 01
0x20C	CAN_BUF[0].DATAH	MDR_CANx->CAN_BUF[x].DATAH Данные сообщения буфера 01
0x210	CAN_BUF[1].ID	MDR_CANx->CAN_BUF[x].ID ID сообщения буфера 02
	...	
0x3FC	CAN_BUF[31].DATAH	MDR_CANx->CAN_BUF[x].DATAH Данные сообщения буфера 32
0x500	CAN_BUF_FILTER[0].MASK	MDR_CANx->CAN_BUF_FILTER[x].MASK Маска для приема сообщения в буфер 01
0x504	CAN_BUF_FILTER[0].FILTER	MDR_CANx->CAN_BUF_FILTER[x].FILTER Фильтр для приема сообщения в буфер 01
0x508	CAN_BUF_FILTER[1].MASK	MDR_CANx->CAN_BUF_FILTER[x].MASK Маска для приема сообщения в буфер 02
	...	
0x5FC	CAN_BUF_FILTER[31].FILTER	MDR_CANx->CAN_BUF_FILTER[x].FILTER Фильтр для приема сообщения в буфер 32
Примечание – Регистры RXID, RXDLC, RXDATAL/H (0x20-0x2C) и TXID, TXDLC, DATAL/H (0x30 – 0x3C) предназначены для временного хранения данных принятого или передаваемого пакета соответственно. Использовать только для отладки		

22.14.1 MDR_CANx->CONTROL

Таблица 233 – Регистр управления контроллером CONTROL

Номер	31...5	4	3	2	1	0
Доступ	U	R/W	R/W	R/W	R/W	R/W
Сброс	0	0	0	0	0	0
	-	ROP	SAP	STM	ROM	CAN EN

Таблица 234 – Описание бит регистра CONTROL

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...5	-	Зарезервировано
4	ROP	Прием собственных пакетов (Receive own packets): 1 – контроллер принимает собственные пакеты; 0 – контроллер принимает только чужие пакеты
3	SAP	Подтверждение собственных пакетов (Send ACK on own packets): 1 – контроллер подтверждает прием собственных пакетов; 0 – контроллер подтверждает прием только чужих пакетов
2	STM	Режим самотестирования (Self Test Mode): 1 – контроллер работает в режиме самотестирования; 0 – контроллер работает в нормальном режиме
1	ROM	Режим «Только прием» (Read Only Mode): 1 – контроллер работает только на прием; 0 – контроллер работает в нормальном режиме
0	CAN_EN	Режим работы контроллера CAN: 1 – разрешение работы; 0 – работа запрещена

22.14.2 MDR_CANx->STATUS

Таблица 235 – Регистр состояния контроллера STATUS

Номер	31...24	23...16	15...13	12	11	10...9	8
Доступ	RO	RO	U	RO	RO	RO	R/W
Сброс	0	0	0	0	0	0	0
	TX ERR CNT [7:0]	RX ERR CNT [7:0]	-	TX ERR CNT8	RX ERR CNT8	ERR STATUS[1:0]	ID LOWER

Номер	7	6	5	4	3	2	1	0
Доступ	R/W	R/W	R/W	R/W	R/W	R/W	RO	RO
Сброс	0	0	0	0	0	0	0	0
	ACK ERR	FRAME ERR	CRC ERR	BIT STUFF ERR	BIT ERR	ERROR OVER	TX READY	RX READY

Таблица 236 – Описание бит регистра STATUS

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...24	TX ERR CNT [7:0]	Счетчик ошибок передатчика TEC, биты [7:0]: TEC > 127, ERROR PASSIVE
23...16	RX ERR CNT [7:0]	Счетчик ошибок приемника REC, биты [7:0]: REC > 127, ERROR PASSIVE
15...13	-	Зарезервировано
12	TX ERR CNT8	Счетчик ошибок передатчика TEC, бит 8: 0 – TEC менее 255; 1 – TEC более 255
11	RX ERR CNT8	Счетчик ошибок приемника REC, бит 8: 0 – REC менее 255; 1 – REC более 255
10...9	ERR STATUS[1:0]	Статус состояния контроллера CAN: 00 – ERROR ACTIVE, при возникновении ошибки отсылается флаг активной ошибки; 01 – ERROR PASSIVE, при возникновении ошибки отсылается флаг пассивной ошибки; 1x – BUS OFF, ожидается восстановление шины
8	ID LOWER	Флаг «проигрыша» арбитража: 0 – при передаче не было проигрыша арбитража; 1 – при передаче был проигран арбитраж
7	ACK ERR	Флаг ошибки подтверждения приема: 0 – нет ошибки; 1 – есть ошибка. Сбрасывается записью «0»
6	FRAME ERR	Флаг ошибки формата пакета: 0 – нет ошибки; 1 – есть ошибка. Сбрасывается записью «0»
5	CRC ERR	Флаг ошибки контрольной суммы пакета: 0 – нет ошибки; 1 – есть ошибка. Сбрасывается записью «0»
4	BIT STUFF ERR	Флаг ошибки вставленных бит пакета: 0 – нет ошибки; 1 – есть ошибка. Сбрасывается записью «0»
3	BIT ERR	Флаг ошибки передаваемых бит пакета: 0 – нет ошибки; 1 – есть ошибка. Сбрасывается записью «0»
2	ERROR OVER	Флаг превышения TEC и REC уровня, заданного ERROR_MAX: 0 – ERROR_MAX ≤ TEC и REC; 1 – ERROR_MAX > TEC или REC. Сбрасывается записью «0»

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
1	TX READY	Флаг наличия данных в буфере для отправки: 0 – буферы пусты, нет сообщений готовых к передаче; 1 – есть не пустой буфер, в котором сообщение готово к передаче. Примечание – Биты TX READY регистра STATUS выставляются при условии, когда установлен бит TX_INT_EN = 1 (регистр CAN_INT_EN) и выставлен флаг разрешения прерывания от выполняющего отправки буфера в регистрах CAN_INT_TX
0	RX READY	Флаг наличия принятых сообщений: 0 – буферы пусты, нет принятых сообщений; 1 – есть буфер, содержащий принятые сообщения. Примечание – Биты RX READY регистра STATUS выставляются при условии, когда установлен бит RX_INT_EN = 1 (регистр CAN_INT_EN) и выставлен флаг разрешения прерывания от выполняющего прием буфера в регистрах CAN_INT_RX
Примечание – Для организации надежного сброса статусных флагов прерываний необходимо учитывать требования, изложенные в подразделе 31.3		

22.14.3 MDR_CANx->BITTMNG

Таблица 237 – Регистр задания скорости работы BITTMNG

Номер	31...28	27	26, 25	24...22	21...19	18...16	15...0
Доступ	U	R/W	R/W	R/W	R/W	R/W	R/W
Сброс	0	0	0	0	0	0	0
	-	SB	SJW [1:0]	SEG2 [2:0]	SEG1 [2:0]	PSEG [2:0]	BRP [15:0]

Таблица 238 – Описание бит регистра BITTMNG

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...28	-	Зарезервировано
27	SB	Семплирование: 0 – однократное; 1 – трехкратное с мажоритарным контролем
26, 25	SJW [1:0]	Значение размера фазы SJW: 11 = Synchronization jump width time = 4 x TQ 10 = Synchronization jump width time = 3 x TQ 01 = Synchronization jump width time = 2 x TQ 00 = Synchronization jump width time = 1 x TQ SJW – это максимальное значение, на которое происходит подстройка приема и передачи при работе на шине CAN. Приемник подстраивается на значение ошибки, но не более чем SJW

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
24...22	SEG2 [2:0]	Значение размера фазы SEG2: 111 = Phase Segment 2 time = 8 x TQ 110 = Phase Segment 2 time = 7 x TQ 101 = Phase Segment 2 time = 6 x TQ 100 = Phase Segment 2 time = 5 x TQ 011 = Phase Segment 2 time = 4 x TQ 010 = Phase Segment 2 time = 3 x TQ 001 = Phase Segment 2 time = 2 x TQ 000 = Phase Segment 2 time = 1 x TQ SEG2 – это время, используемое для сокращения битового интервала при подстройке
21...19	SEG1 [2:0]	Значение размера фазы SEG1: 111 = Phase Segment 1 time = 8 x TQ 110 = Phase Segment 1 time = 7 x TQ 101 = Phase Segment 1 time = 6 x TQ 100 = Phase Segment 1 time = 5 x TQ 011 = Phase Segment 1 time = 4 x TQ 010 = Phase Segment 1 time = 3 x TQ 001 = Phase Segment 1 time = 2 x TQ 000 = Phase Segment 1 time = 1 x TQ SEG1 – это время, используемое для увеличения битового интервала при подстройке
18...16	PSEG[2:0]	Значение размера фазы PSEG 111 = Propagation time = 8 x TQ 110 = Propagation time = 7 x TQ 101 = Propagation time = 6 x TQ 100 = Propagation time = 5 x TQ 011 = Propagation time = 4 x TQ 010 = Propagation time = 3 x TQ 001 = Propagation time = 2 x TQ 000 = Propagation time = 1 x TQ PSEG - это время компенсирующее задержку распространения сигналов в шине CAN
15...0	BRP [15:0]	Предделитель системной частоты: $CLK = CANx_CLK / (BRP + 1)$ $TQ(us) = 1 / CLK(MHz) = (BRP + 1) / CANx_CLK(MHz)$

22.14.4 MDR_CANx->INT_EN

Таблица 239 – Регистр разрешения прерываний INT_EN

Номер	31...5	4	3	2	1	0
Доступ	U	U	R/W	R/W	R/W	R/W
Сброс	0	0	0	0	0	0
	-	ERR OVER INT EN	ERR INT EN	TX INT EN	RX INT EN	GLB INT EN

Таблица 240 – Описание бит регистра INT_EN

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...5	-	Зарезервировано
4	ERR OVER INT EN	Флаг разрешения прерывания по превышению TEC или REC допустимого значения в ERROR_MAX: 0 – запрещено прерывание; 1 – разрешено прерывание
3	ERR INT EN	Флаг разрешения прерывания по возникновению ошибки: 0 – запрещено прерывание; 1 – разрешено прерывание
2	TX INT EN	Флаг разрешения прерывания по возможности передачи: 0 – запрещено прерывание; 1 – разрешено прерывание
1	RX INT EN	Флаг разрешения прерывания по приему сообщений: 0 – запрещено прерывание; 1 – разрешено прерывание
0	GLB INT EN	Общий флаг разрешения прерывания блока CAN: 0 – запрещено прерывание; 1 – разрешено прерывание

22.14.5 MDR_CANx->OVER

Таблица 241 – Регистр границы счета ошибок OVER

Номер	31...8	7...0
Доступ	U	R/W
Сброс	0	0
	-	ERROR_MAX[7:0]

Таблица 242 – Описание бит регистра OVER

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...8	-	Зарезервировано
7...0	ERROR MAX [7:0]	Регистр границы счетчика ошибок Допустимое значение счетчиков ошибок TEC и REC, при превышении которого вырабатывается флаг ERROR_OVER

22.14.6 MDR_CANx->BUF_CON[x]

Таблица 243 – Регистр управления буфером BUF_CON[x]

Номер	31...8	7	6	5	4	3	2	1	0
Доступ	U	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Сброс	0	0	0	0	0	0	0	0	0
	-	OVER WR	RX FULL	TX REQ	PRIOR_0	RTR EN	OVER EN	RX TXn	EN

Таблица 244 – Описание бит регистра BUF_CON[x]

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...8	-	Зарезервировано
7	OVER_WR	Флаг перезаписи принятого сообщения: 0 – не было перезаписи; 1 – была перезапись принятого сообщения
6	RX_FULL	Флаг готовности приема: 0 – нет принятого сообщения; 1 – принятое сообщение в буфере
5	TX_REQ	Запрос на отправку сообщения: 0 – нет запроса или отправлено; 1 – запрос на отправку
4	PRIOR_0	Приоритет при отправке: 0 – приоритет; 1 – нет приоритета
3	RTR_EN	Режим ответа на RTR: 0 – не отвечать при приеме RTR; 1 – ответить при приеме RTR в буфер
2	OVER_EN	Разрешение перезаписи принятого сообщения: 0 – не разрешена перезапись; 1 – разрешена перезапись сообщения
1	RX_TXn	Режим работы буфера: 0 – на передачу; 1 – на прием
0	EN	Разрешение работы буфера: 0 – отключен; 1 – работает

22.14.7 MDR_CANx->INT_RX

Таблица 245 – Регистр разрешения прерываний от приемных буферов INT_RX

Номер	31...0
Доступ	R/W
Сброс	0
	CAN_INT_RX[31:0]

Таблица 246 – Описание бит регистра INT_RX

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...0	CAN INT RX[31:0]	Флаги разрешения прерываний от буферов по приему сообщений: CAN_INT_RX[0] – для первого буфера; CAN_INT_RX[1] – для второго буфера и так далее

22.14.8 MDR_CANx->RX

Таблица 247 – Регистр RX флагов RX_FULL от приемных буферов

Номер	31...0
Доступ	RO
Сброс	0
	CAN_RX[31:0]

Таблица 248 – Описание бит регистра RX

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...0	CAN RX[31:0]	Флаги RX_FULL разрешенных на прием буферов: CAN_RX[0] – флаг RX_FULL от первого буфера CAN_RX[1] – флаг RX_FULL от второго буфера и так далее, доступны только на чтение

22.14.9 MDR_CANx->INT_TX

Таблица 249 – Регистр разрешения прерываний от передающих буферов INT_TX

Номер	31...0
Доступ	R/W
Сброс	0
	CAN_INT_TX[31:0]

Таблица 250 – Описание бит регистра INT_TX

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...0	CAN INT TX[31:0]	Флаги разрешения прерываний от буферов по передаче сообщений: CAN_INT_TX[0] – для первого буфера CAN_INT_TX[1] – для второго буфера и так далее

22.14.10 MDR_CANx->TX

Таблица 251 – Регистр TX флагов ~TX_REQ от передающих буферов

Номер	31...0
Доступ	RO
Сброс	0
	CAN_TX[31:0]

Таблица 252 – Описание бит TX

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...0	CAN TX[31:0]	Флаги ~TX_REQ разрешенных на передачу буферов: CAN_TX[0] – флаг ~TX_REQ от первого буфера CAN_TX[1] – флаг ~TX_REQ от второго буфера и так далее, доступны только на чтение

22.14.11 MDR_CANx->RXID
MDR_CANx->TXID
MDR_CANx->CAN_BUF[x].ID
MDR_CANx->CAN_BUF_FILTER[x].MASK
MDR_CANx->CAN_BUF_FILTER[x].FILTER

Таблица 253 – Регистры RXID, TXID и CAN_BUF[x].ID идентификаторов

Номер	31...29	28...18	17...0
Доступ	U	R/W	R/W
Сброс	0	0	0
	-	SID [10:0]	EID [17:0]

Таблица 254 – Описание бит регистров RXID, TXID и CAN_BUF[x].ID

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...29	-	Зарезервировано
28...18	SID [10:0]	Поле SID. Для стандартного и расширенного пакетов CAN. Чем меньше значение поля, тем больший приоритет имеет пакет при арбитраже
17...0	EID [17:0]	Поле EID. Для расширенных пакетов CAN. Чем меньше значение поля, тем больший приоритет имеет пакет при арбитраже

22.14.12 MDR_CANx->RXDLC
MDR_CANx->TXDLC
MDR_CANx->CAN_BUF[x].DLC

Таблица 255 – Регистры RXDLC, TXDLC и CAN_BUF[x].DLC сообщения

Номер	31...13	12	11	10	9	8	7...4	3...0
Доступ	U	R/W	R/W	R/W	R/W	R/W	U	R/W
Сброс	0	0	0	0	0	0	0	0
	-	IDE	SRR	R0	R1	RTR	-	DLC [3:0]

Таблица 256 – Описание бит регистров RXDLC, TXDLC и CAN_BUF[x].DLC

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...13	-	Зарезервировано
12	IDE	Поле IDE. Поле, обозначающее формат пакета: 0 – стандартный пакет; 1 – расширенный пакет
11	SRR	Поле SRR, расширенного формата. Всегда должно быть равно “1”
10	R0	Поле R0. Всегда должно быть равно “0”

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
9	R1	Поле R1, расширенного формата. Всегда должно быть равно «0»
8	RTR	Поле RTR, запроса обратного ответа: 0 – нет запроса; 1 – есть запрос. Если узел получил пакет с запросом обратного ответа, то он должен ответить
7...4	-	Зарезервировано
3...0	DLC[3:0]	Поле DLC, длина передаваемых данных в пакете: 0000 – нет данных 0001 – 1 байт 0010 – 2 байт 0011 – 3 байт 0100 – 4 байт 0101 – 5 байт 0110 – 6 байт 0111 – 7 байт 1000 – 8 байт 1xxx – 8 байт и недопустимо

**22.14.13 MDR_CANx->RXDATAL
MDR_CANx->TXDATAL
MDR_CANx->CAN_BUF[x].DATAL**

Таблица 257 – Регистры RXDATAL, TXDATAL и CAN_BUF[x].DATAL данных сообщения

Номер	31...24	23...16	15...8	7...0
Доступ	R/W	R/W	R/W	R/W
Сброс	0	0	0	0
	DB3[7:0]	DB2[7:0]	DB1[7:0]	DB0[7:0]

Таблица 258 – Описание бит регистров RXDATAL, TXDATAL и CAN_BUF[x].DATAL

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...24	DB3[7:0]	Поле DB3. Четвертый байт, передаваемый в пакете
23...16	DB2[7:0]	Поле DB2. Третий байт, передаваемый в пакете
15...8	DB1[7:0]	Поле DB1. Второй байт, передаваемый в пакете
7...0	DB0[7:0]	Поле DB0. Первый байт, передаваемый в пакете

**22.14.14 MDR_CANx->RXDATAH
MDR_CANx->TXDATAH
MDR_CANx->CAN_BUF[x].DATAH**

Таблица 259 – Регистры RXDATAH, TXDATAH и CAN_BUF[x].DATAH данных сообщения

Номер	31...24	23...16	15...8	7...0
Доступ	R/W	R/W	R/W	R/W
Сброс	0	0	0	0
	DB7[7:0]	DB6[7:0]	DB5[7:0]	DB4[7:0]

Таблица 260 – Описание бит регистров RXDATAH, TXDATAH и CAN_BUF[x].DATAH

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...24	DB7[7:0]	Поле DB7. Восьмой байт, передаваемый в пакете
23...16	DB6[7:0]	Поле DB6. Седьмой байт, передаваемый в пакете
15...8	DB5[7:0]	Поле DB5. Шестой байт, передаваемый в пакете
7...0	DB4[7:0]	Поле DB4. Пятый байт, передаваемый в пакете

23 Таймеры общего назначения MDR_TIMERx

В микросхеме реализовано три блока таймеров общего назначения, каждый из которых может быть использован для широкого спектра применений, включая:

- подсчет циклов частоты TIM_CLK или каких-либо внешних событий;
- формирование прерываний и запросов DMA по заданным событиям;
- захват входных сигналов, в том числе измерение длительности импульсов

входных сигналов;

- генерацию различных форм выходных сигналов.

Основу таймеров составляет 16-битный перезагружаемый счетчик. Счет может быть прямой, обратный или двунаправленный. В качестве источника синхросигнала может выступать внутренняя тактовая частота TIM_CLK, внешние сигналы или другие таймеры.

В каждый блок таймера входит до четырех каналов, которые имеют в своем составе схему захвата и блок ШИМ с функциями формирования «мертвой зоны» и аппаратной блокировки.

Каждый из таймеров позволяет генерировать прерывания и запросы DMA.

23.1 Основные характеристики

Основные характеристики блока таймера:

- 16-битный счетчик: счет прямой, обратный или двунаправленный;
- 16-разрядный предварительный делитель частоты TIM_CLK;
- схема выбора источника тактирования основного счетчика от внешних сигналов или от других таймеров;
- до четырех независимых каналов, каждый канал может работать в одном из режимов:
 - режим захвата: позволяет захватить (сохранить) текущее значение счетчика при изменении некоторого входного сигнала;
 - режим ШИМ: позволяет осуществлять непрерывное сравнение заданных значений со значением счетчика для формирования выходных сигналов;
 - формирование выходных сигналов в режиме ШИМ:
 - сброс в НИЗКИЙ уровень при совпадении;
 - установка в ВЫСОКИЙ уровень при совпадении;
 - переключение (инвертирование) при совпадении;
 - переключение при некотором условии;
 - формирование прерываний и запросов DMA по событиям:
 - обновление счетчика;
 - захват;
 - сравнение;
 - внешние события по входам ETR и BRK.

23.1.1 Структурная схема

Структурная схема блока «Таймер» представлена на рисунке 57.

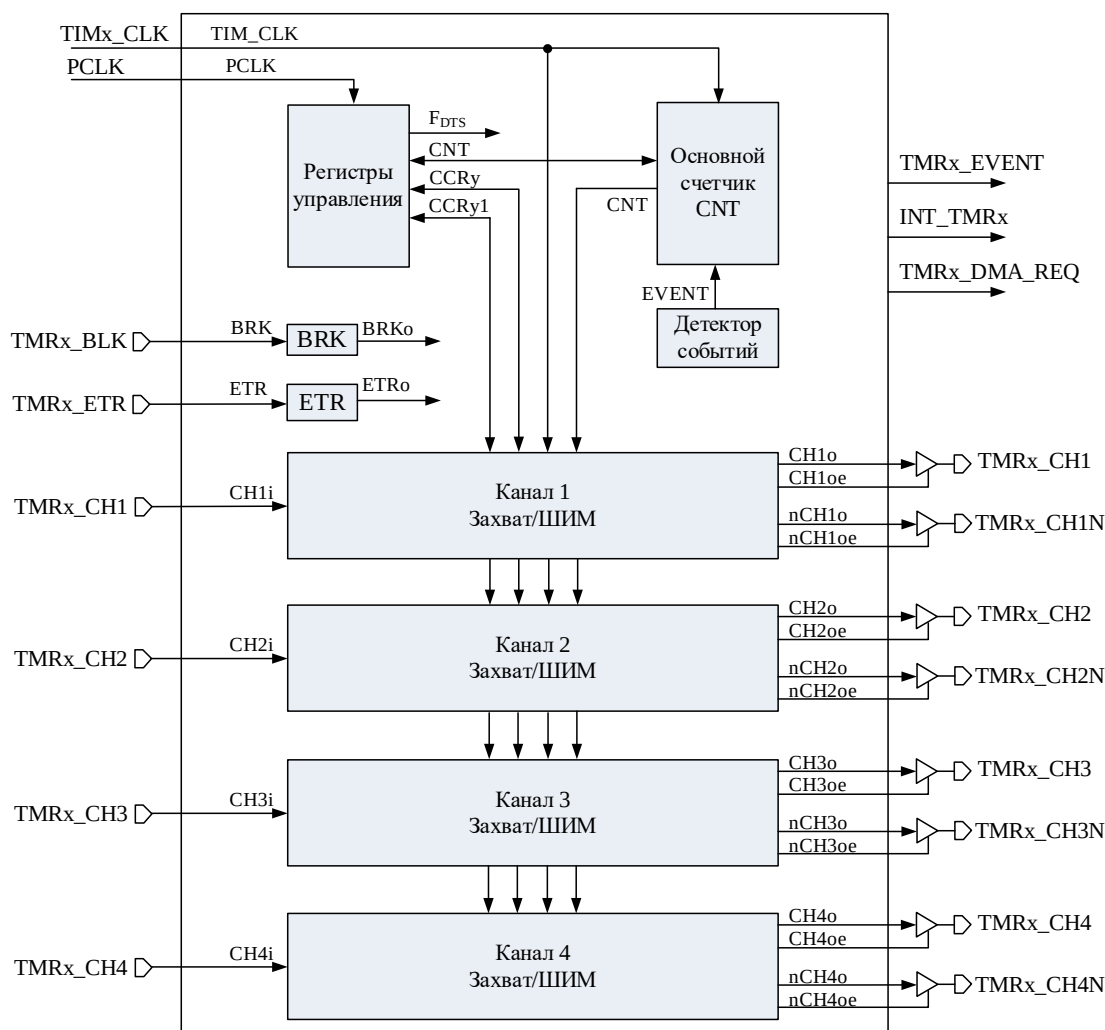


Рисунок 57 – Структурная схема блока «Таймер»

Таймер содержит основной 16-битный счетчик CNT, блок регистров управления и четыре канала схем захвата/ШИМ.

Таймер позволяет работать в режимах:

- таймер;
- расширенный таймер, с объединением нескольких таймеров;
- схема захвата;
- схема ШИМ.

23.2 Базовый блок таймера

Таймер построен на базе 16-битного счетчика. Базовый блок таймера включает в себя:

- основной счетчик таймера (CNT);
- основание счета (максимальное значение) основного счетчика (ARR);
- делитель частоты TIM_CLK (PSG), используемый для тактирования основного счетчика;
- регистр управления основным счетчиком (CNTRL).

Сигналом для изменения основного счетчика CNT может служить как внутренняя частота TIM_CLK, так и события в других счетчиках, либо внешние входные сигналы (см. подраздел 23.3 «Источники событий для счета»).

23.2.1 Инициализация тактирования таймера

Перед началом работы с таймером в первую очередь должны быть включены тактовые сигналы. Параметры задаются в блоке «Сигналы тактовой частоты MDR_RST_CLK».

Таймер общего назначения тактируется частотами PCLK и TIM_CLK. Частота PCLK используется для записи/чтения регистров блока по шине APB. Работа блоков таймера осуществляется на частоте TIM_CLK.

Для разрешения тактовой частоты PCLK необходимо установить бит тактирования блока в регистре PER_CLOCK: бит 14 для таймера 1, бит 15 для таймера 2, бит 16 для таймера 3. Задание тактовой частоты TIM_CLK осуществляется в регистре TIM_CLOCK: в поле TIMxBRG[7:0] устанавливается коэффициент деления тактовой частоты HCLK для формирования частоты TIM_CLK, разрешение подачи частоты TIM_CLK на блок таймера управляется битом TIMxCLKEN.

После подачи тактовых сигналов можно приступить к работе с таймером.

23.2.2 Инициализация основного счетчика таймера

Чтобы запустить работу основного счетчика необходимо задать:

- начальное значение основного счетчика таймера в регистре CNT;
- значение основания счета для основного счетчика в регистре ARR;
- режим работы счетчика в регистре CNTRL;
- выбрать источник события переключения счетчика EVENT_SEL[3:0];
- режим счета основного счетчика CNT_MODE[1:0]:
 - значения 00 и 01 при тактировании внутренней частотой;
 - значения 10 при тактировании внешними сигналами;
 - направление счета основного счетчика DIR;

– при тактировании внутренней частотой установить значение предварительного делителя в регистре PSG, основной счетчик при этом будет считать на частоте

$$TIM_CLKd = TIM_CLK / (PSG + 1);$$

- разрешить работу счетчика CNT_EN.

Значения регистров CNT, PSG и ARR можно изменять даже во время работы счетчика. Значения регистров CNT и PSG вступят в силу мгновенно после их записи. Значение регистра основания счета (ARR) может вступить в силу сразу после записи, если в регистре CNTRL бит ARRB_EN = 0.

При установленном бите ARRB_EN = 1 записанное значение ARR применяется при CNT == ARR. Необходимо учитывать, что если установлен прямой счет таймера, то новое значение ARR будет использоваться в следующем периоде счета. Если установлен

обратный счет таймера, то новое значение ARR будет использовано через один период счета.

Поле CNT_MODE[1:0] в регистре CNTRL определяет режим работы основного счетчика:

- CNT_MODE[1:0] = 00 или 10 – направление счета определяется битом DIR:
- DIR = 0 – счет прямой;
- DIR = 1 – счет обратный;
- CNT_MODE[1:0] = 01 – счет двунаправленный с автоматическим изменением

DIR.

23.2.3 Режимы счета

23.2.3.1 Счет прямой: CNT_MODE[1:0] = 00, DIR = 0

MDR_TIMERx->CNTRL = 0x00000000; //Режим инициализации таймера

//Настраиваем работу основного счетчика

MDR_TIMERx->CNT = 0x00000004; //Начальное значение счетчика

MDR_TIMERx->PSG = 0x00000000; //Предделитель частоты TIM_CLK

MDR_TIMERx->ARR = 0x00000013; //Основание счета

//Разрешение работы таймера

MDR_TIMERx->CNTRL = 0x00000001; //Счет прямой по TIM_CLK

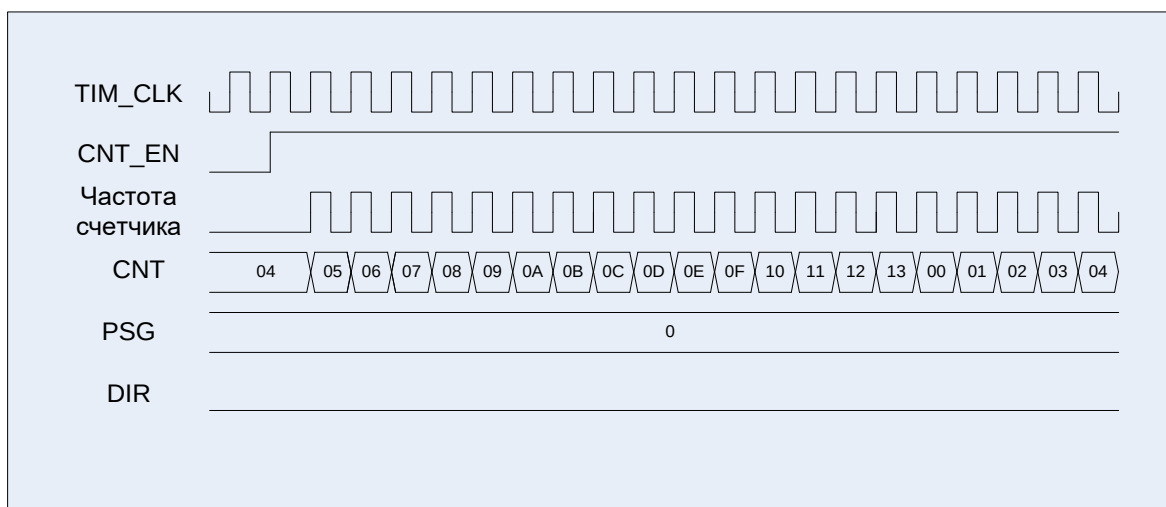


Рисунок 58 – Диаграммы работы таймера, счет прямой от 0 до 0x13, стартовое значение 0x04

23.2.3.2 Счет обратный: $CNT_MODE[1:0] = 00$, $DIR = 1$

$MDR_TIMERx \rightarrow CNTRL = 0x00000000$; //Режим инициализации таймера

//Настраиваем работу основного счетчика

$MDR_TIMERx \rightarrow CNT = 0x00000004$; //Начальное значение счетчика

$MDR_TIMERx \rightarrow PSG = 0x00000000$; //Предделитель частоты TIM_CLK

$MDR_TIMERx \rightarrow ARR = 0x00000013$; //Основание счета

//Разрешение работы таймера.

$MDR_TIMERx \rightarrow CNTRL = 0x00000009$; //Счет обратный по TIM_CLK

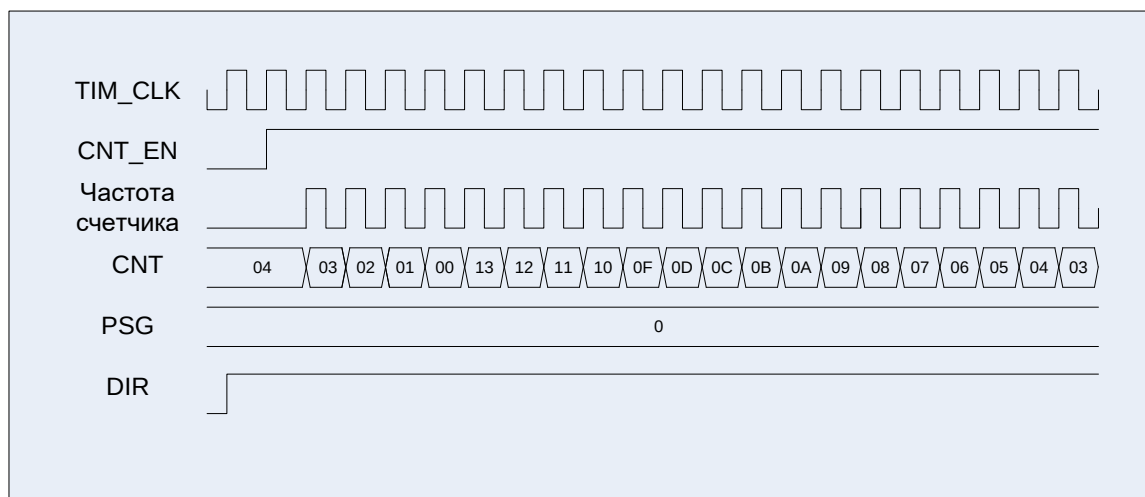


Рисунок 59 – Диаграммы работы таймера, счет обратный от 0x13 до 0, стартовое значение 0x04

23.2.3.3 Счет двунаправленный: $CNT_MODE = 01$, $DIR = 0$

$MDR_TIMERx \rightarrow CNTRL = 0x00000000$; //Режим инициализации таймера

//Настраиваем работу основного счетчика

$MDR_TIMERx \rightarrow CNT = 0x00000004$; //Начальное значение счетчика

$MDR_TIMERx \rightarrow PSG = 0x00000000$; //Предделитель частоты TIM_CLK

$MDR_TIMERx \rightarrow ARR = 0x00000013$; //Основание счета

//Разрешение работы таймера.

$MDR_TIMERx \rightarrow CNTRL = 0x00000041$; //Счет двунаправленный по TIM_CLK

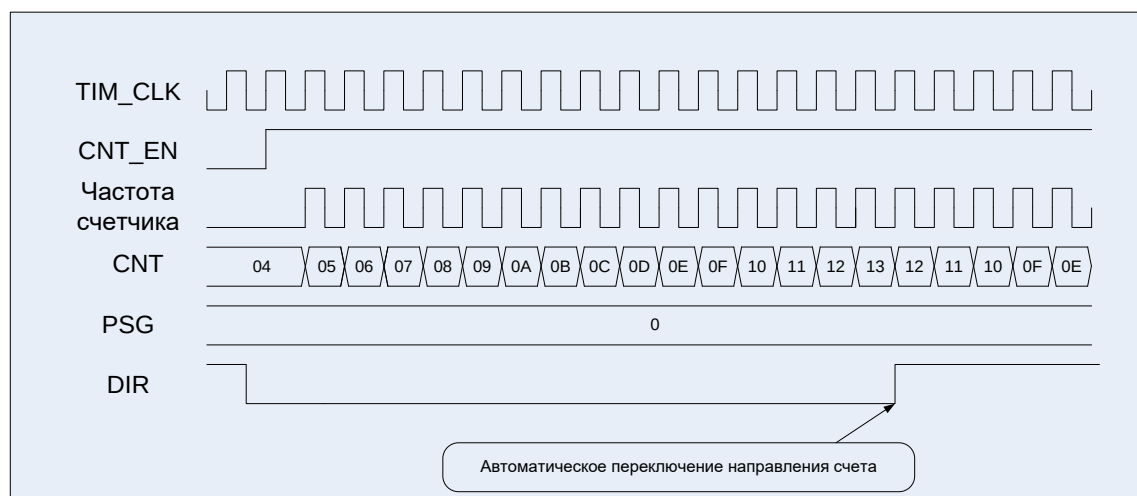


Рисунок 60 – Диаграммы работы таймера, счет двунаправленный, сначала прямой

23.2.3.4 Счет двунаправленный: CNT_MODE = 01, DIR = 1

MDR_TIMERx->CNTRL = 0x00000000; //Режим инициализации таймера

//Настраиваем работу основного счетчика

MDR_TIMERx->CNT = 0x00000004; //Начальное значение счетчика

MDR_TIMERx->PSG = 0x00000000; //Предделитель частоты TIM_CLK

MDR_TIMERx->ARR = 0x00000013; //Основание счета

//Разрешение работы таймера.

MDR_TIMERx->CNTRL = 0x00000049; //Счет двунаправленный по TIM_CLK

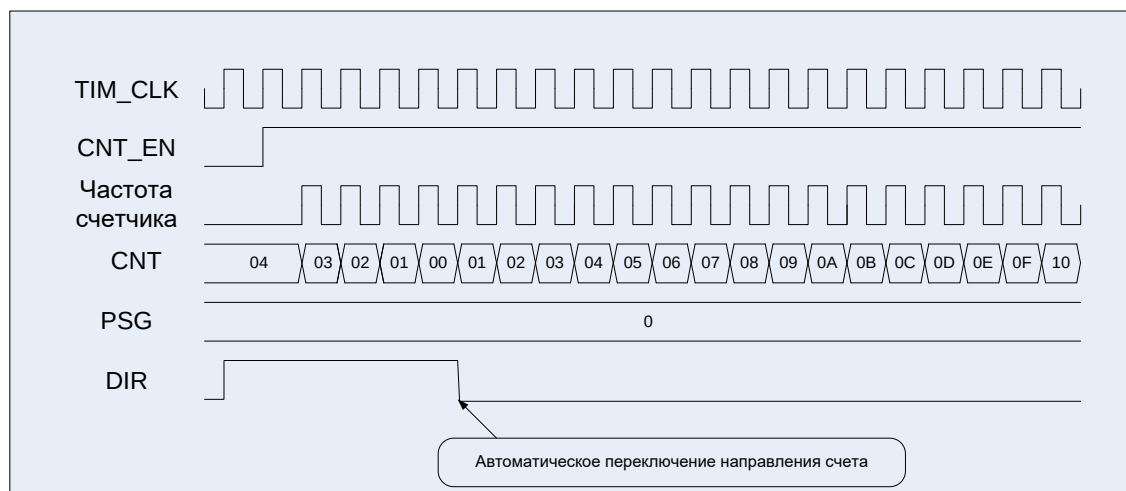


Рисунок 61 – Диаграммы работы таймера, счет двунаправленный, сначала обратный

23.2.4 Тактовая частота F_{DTS}

В блоке таймера предусмотрено формирование дополнительной тактовой частоты F_{DTS}, которая может использоваться для работы генератора «мертвой зоны» и цифровых фильтров на входах ETR и CНуi.

Тактовая частота F_{DTS} формируется из частоты TIM_CLK путём прореживания на заданный коэффициент (1, 2, 3 или 4). Настройка частоты F_{DTS} осуществляется в регистре CNTRL, поле FDTs[1:0].

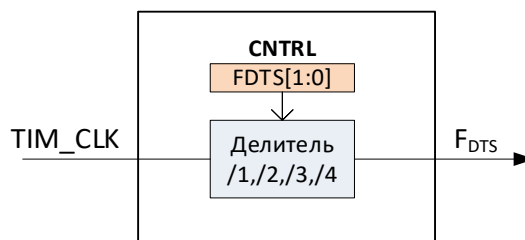


Рисунок 62 – Схема формирования тактовой частоты F_{DTS}

Диаграмма возможных частот F_{DTS} приведена на рисунке 63.

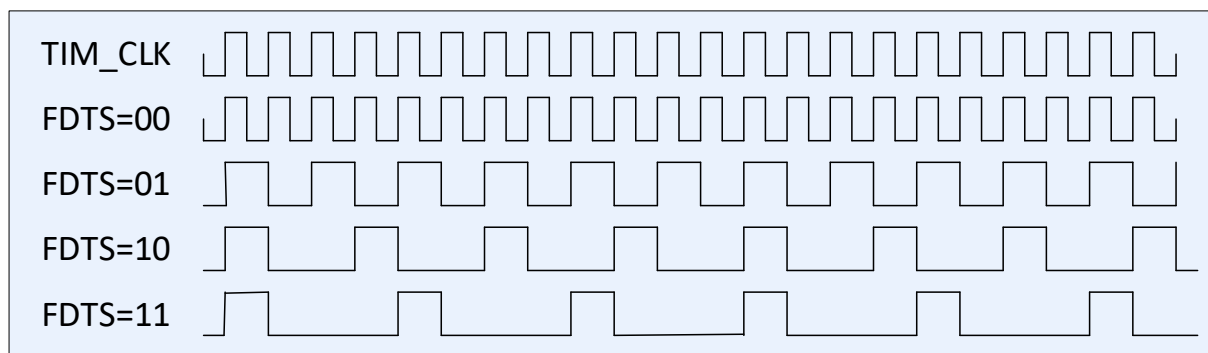


Рисунок 63 – Диаграмма тактовой частоты F_{DTS} в зависимости от значения $FDTs[1:0]$ в регистре CNTRL

23.3 Источники событий для счета

Тактирование основного счетчика таймера может осуществляться от следующих источников:

- внутренний тактовый сигнал (TIM_CLKd);
- событие в другом таймере (CNT==ARR);
- внешний тактовый сигнал, «Режим 1»: событие переднего фронта на входе канала CHyі;
- внешний тактовый сигнал, «Режим 2»: событие переднего или заднего фронта на входе ETR.

Выбор источника тактирования основного счетчика осуществляется в регистре CNTRL, поле EVENT_SEL[3:0]. При выборе любого источника, кроме внутреннего тактового сигнала (EVENT_SEL[3:0] = 0000), необходимо также установить CNT_MODE[1:0] = 10 в регистре CNTRL.

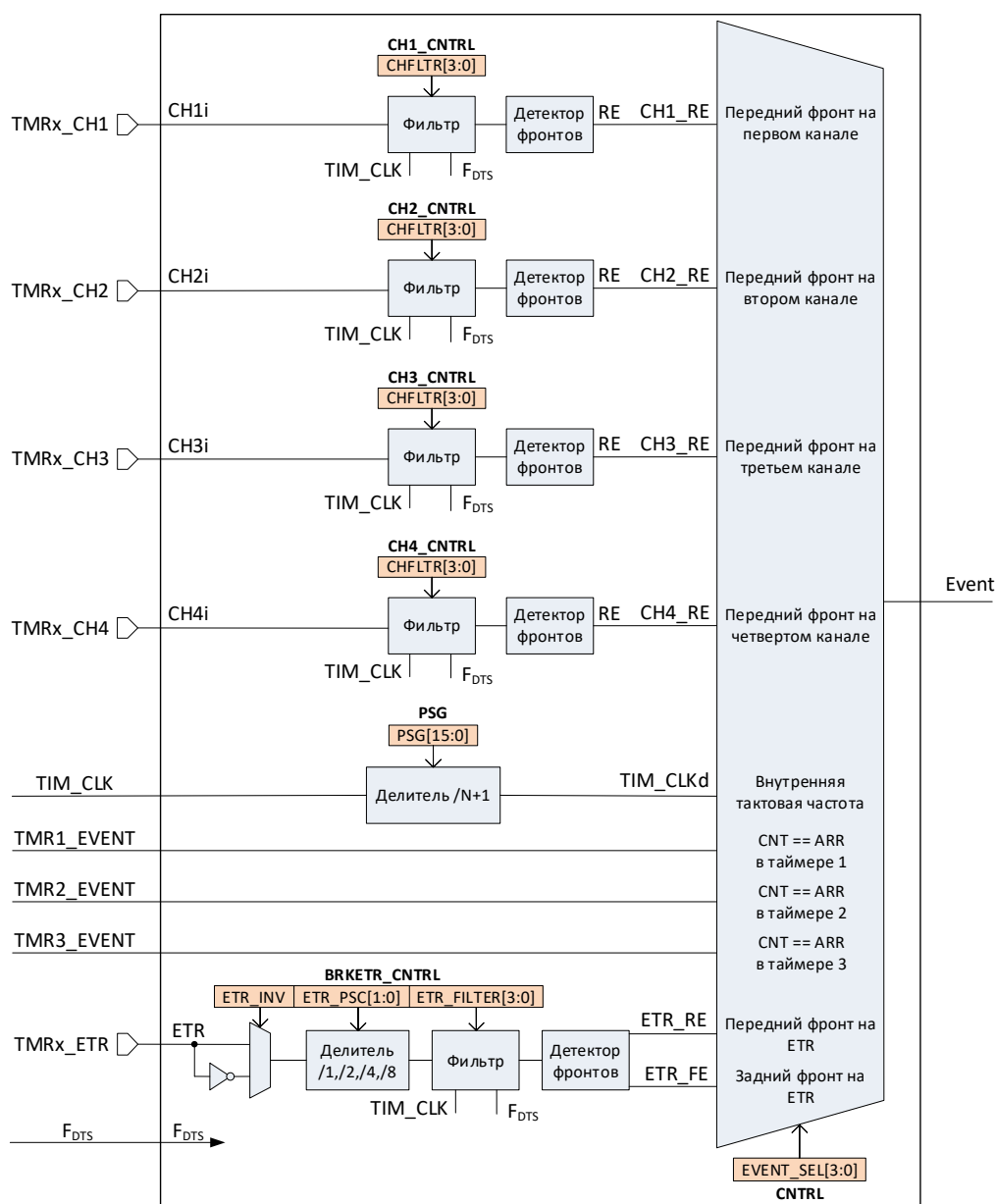


Рисунок 64 – Структурная схема формирования события для счета

23.3.1 Внутренний тактовый сигнал (TIM_CLKd)

Режим выбирается, когда $EVENT_SEL[3:0] = 0000$ и $CNT_MODE[1:0] = 0x$ в регистре CNTRL. Основной счетчик таймера тактируется от внутренней частоты TIM_CLKd, которая формируется путем деления частоты TIM_CLK в соответствии с коэффициентом деления, записанным в регистре PSG.

Если значение предварительного делителя основного счетчика (PSG) не равно нулю, то счетный регистр делителя будет инкрементироваться по каждому импульсу сигнала TIM_CLK до тех пор, пока не достигнет значения, находящегося в регистре делителя. Далее счетный регистр делителя сбрасывается в ноль, содержимое основного счетчика таймера изменяется на 1 и счет начинается заново. Таким образом выходная частота предварительного делителя составляет

$$TIM_CLKd = \frac{TIM_CLK}{PSG+1}. \quad (8)$$

Значение регистра PSG можно изменять даже во время работы счетчика, новое значение предделителя вступит в силу сразу после записи. На рисунках 65 и 66 приведены диаграммы работы счетчика при обновлении значения PSG.

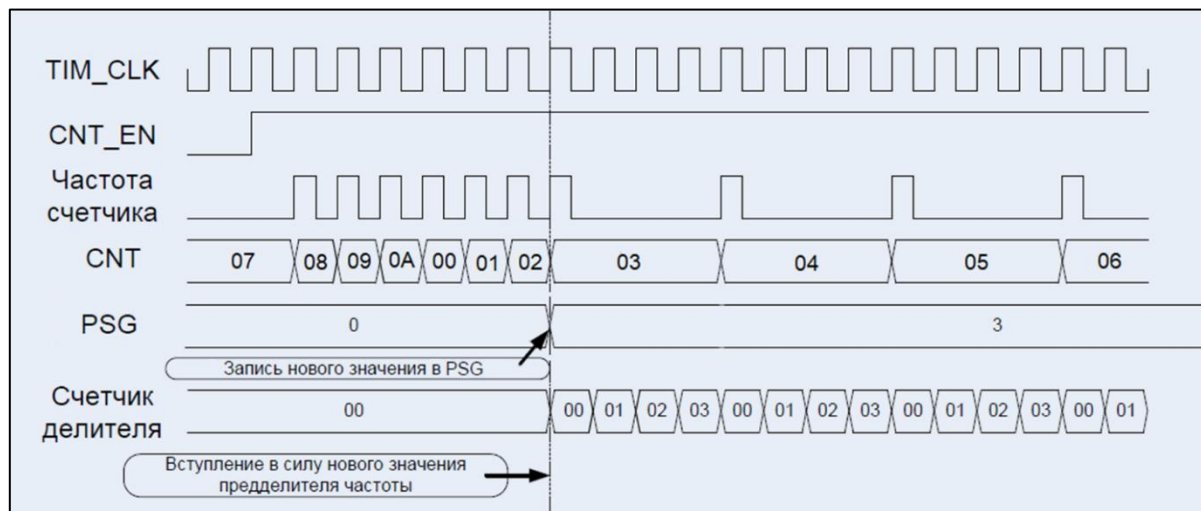


Рисунок 65 – Диаграмма работы счетчика: счет прямой (CNT_MODE[1:0] = 00, EVENT_SEL[3:0] = 0000, DIR = 0)

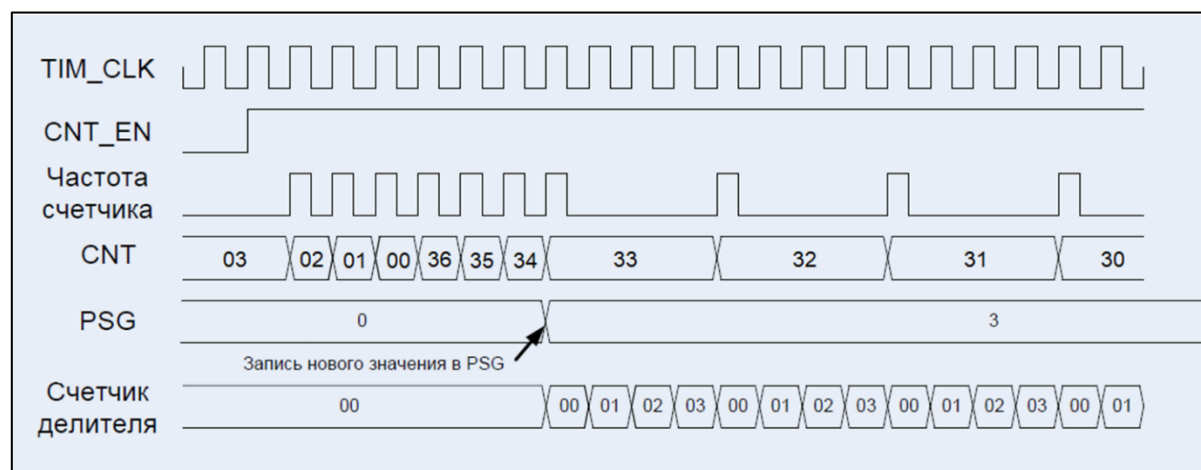


Рисунок 66 – Диаграмма работы счетчика: счет обратный (CNT_MODE[1:0] = 00, EVENT_SEL[3:0] = 0000, DIR = 1)

23.3.2 Событие в другом таймере (CNT==ARR)

Все таймеры полностью независимы друг от друга, но при этом у них предусмотрена возможность синхронизированной работы. Это позволяет создавать более сложные массивы таймеров, которые работают полностью автономно и не требуют написания какого-либо кода программы для выполнения сложных временных функций.

У каждого таймера имеется выход запуска TMRx_EVENT, который соединен с входами других таймеров. Тактирование от другого таймера выбирается в случае, когда EVENT_SEL[3:0] = 0001 – 0011 и CNT_MODE[1:0] = 10 в регистре CNTRL. Основной счетчик таймера тактируется от другого таймера по сигналу TMRx_EVENT, который устанавливается при CNT == ARR. Пересинхронизация сигнала TMRx_EVENT (CNT == ARR) с одного таймера на другой происходит с задержкой один такт частоты TIM_CLK.

Синхронизация таймеров возможна в различных режимах. На рисунке 67 показан пример каскадного соединения таймеров, диаграммы работы данных таймеров приведены на рисунке 68.

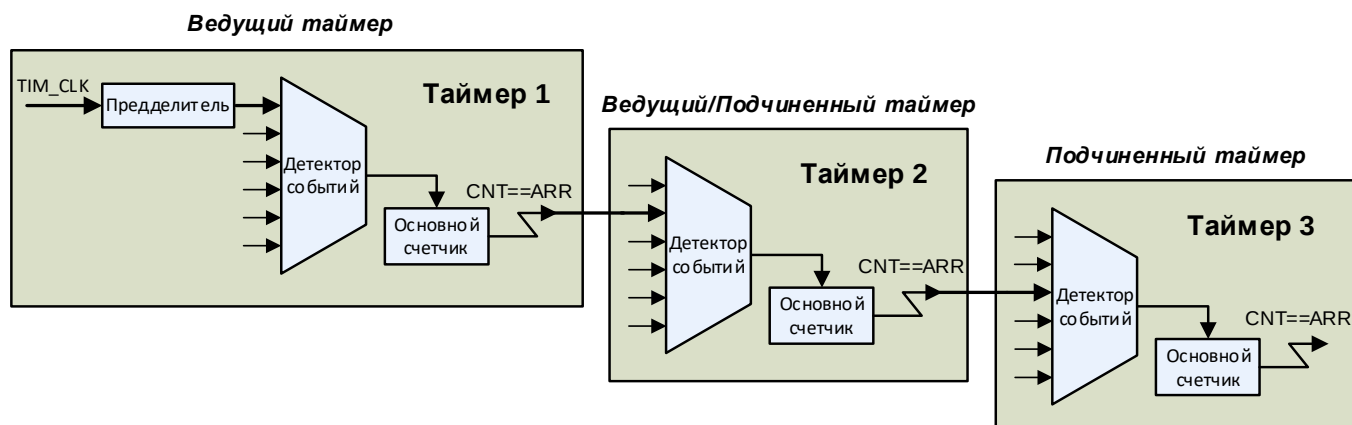


Рисунок 67 – Пример каскадного соединения таймеров

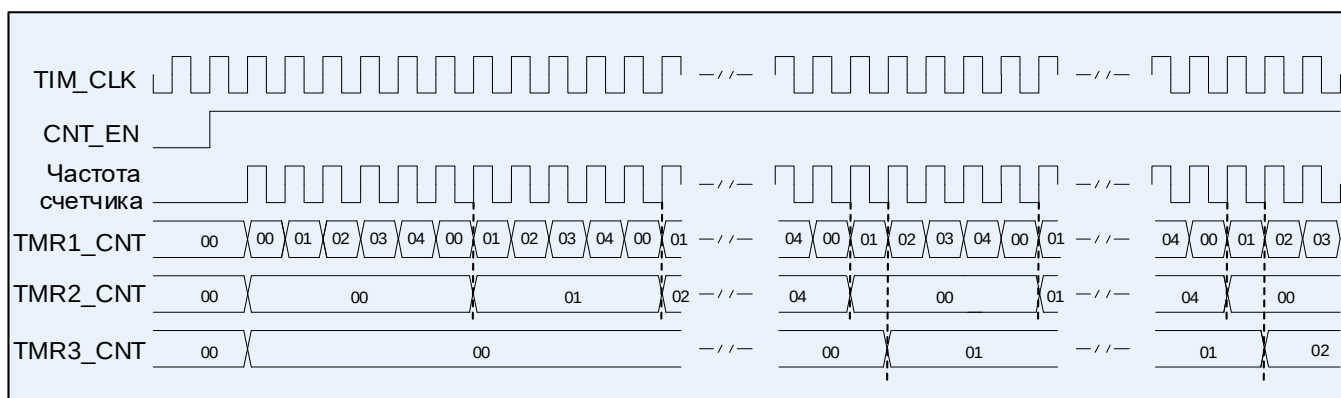


Рисунок 68 – Диаграммы работы трех таймеров в каскаде

MDR_TIMER1: DIR = 0, EVENT_SEL[3:0] = 0000, CNT_MODE[1:0] = 00;
MDR_TIMER2: DIR = 0, EVENT_SEL[3:0] = 0001, CNT_MODE[1:0] = 10;
MDR_TIMER3: DIR = 0, EVENT_SEL[3:0] = 0010, CNT_MODE[1:0] = 10.

23.3.3 Внешний тактовый сигнал, «Режим 1»: событие переднего фронта на входе канала CHy1

Режим выбирается, когда EVENT_SEL[3:0] = 01xx и CNT_MODE[1:0] = 10. Основной счетчик таймера считает по переднему фронту внешнего сигнала, поступающего на вход канала CHy1. Биты CHSEL[1:0] регистра CHy_CNTRL не оказывают влияния, так как они применяются для работы канала таймера только в режиме захвата.

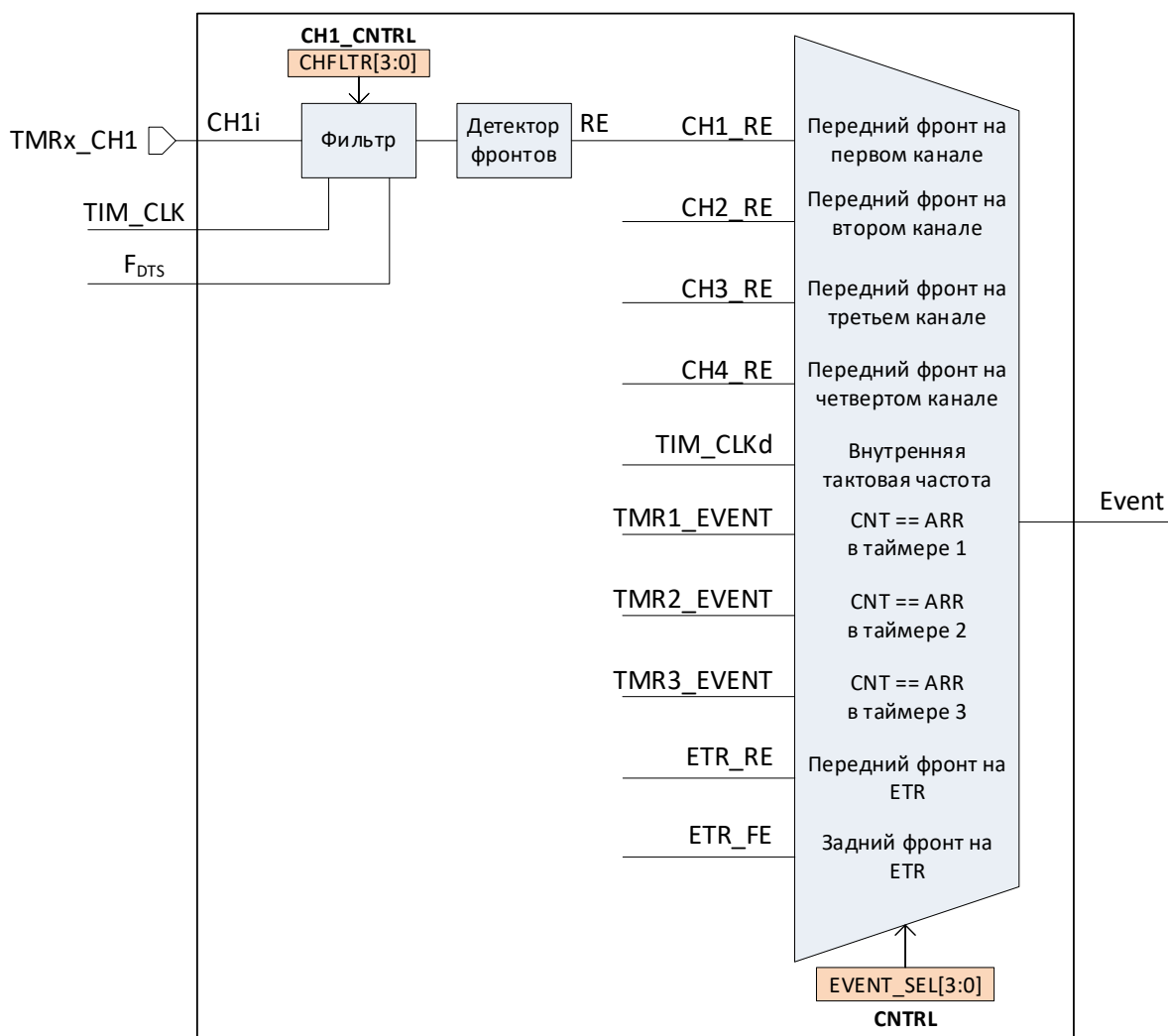


Рисунок 69 – Схема тактирования сигналом со входа первого канала

Со входа CH1i внешний тактовый сигнал поступает в блок цифрового фильтра. Данный блок позволяет отфильтровать входной сигнал с целью устранения импульсов, длительность которых меньше заданного порога (см. «Блок цифрового фильтра»). Настройки фильтра задаются в поле CHFLTR[3:0] регистра CH1_CNTRL.

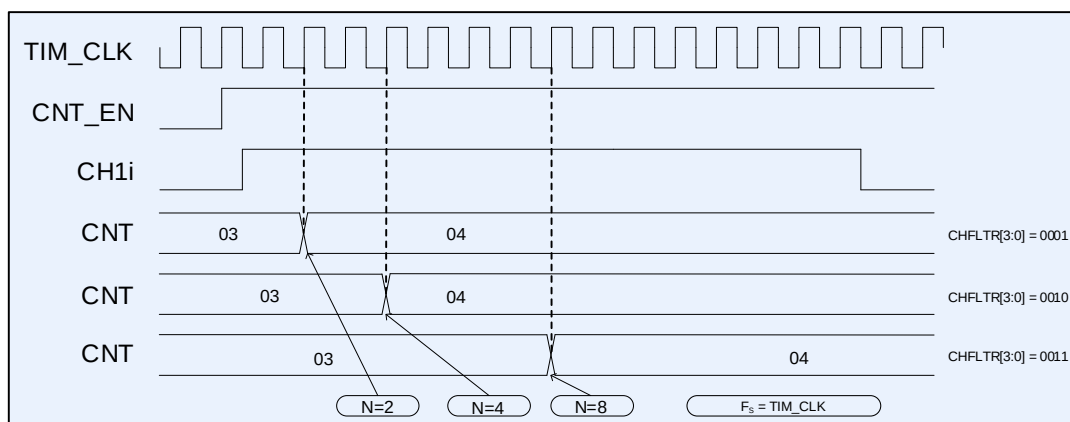


Рисунок 70 – Диаграмма внешнего тактирования с разными вариантами фильтра

23.3.4 Внешний тактовый сигнал, «Режим 2»: событие переднего или заднего фронта на входе ETR

Данный режим выбирается, когда $EVENT_SEL[3:0] = 100x$ и $CNT_MODE[1:0] = 10$ в регистре CNTRL. Основной счетчик таймера может тактироваться по переднему или по заднему фронту внешнего сигнала, поступающего на вход ETR, в зависимости от значения в поле $EVENT_SEL[3:0]$.

Конфигурация тактового сигнала со входа ETR задается в регистре BRKETR_CNTRL. Бит ETR_INV позволяет установить инверсию входного сигнала. Поле ETR_PSC[1:0] задает коэффициент деления асинхронного предделителя внешней частоты (1, 2, 4 или 8). После предделителя тактовый сигнал поступает в блок цифрового фильтра, где он может быть дополнительно отфильтрован с целью устранения импульсов, длительность которых меньше заданного порога (см. подраздел 23.6 «Блок цифрового фильтра»). Настройки фильтра задаются в поле ETR_FILTER[3:0].

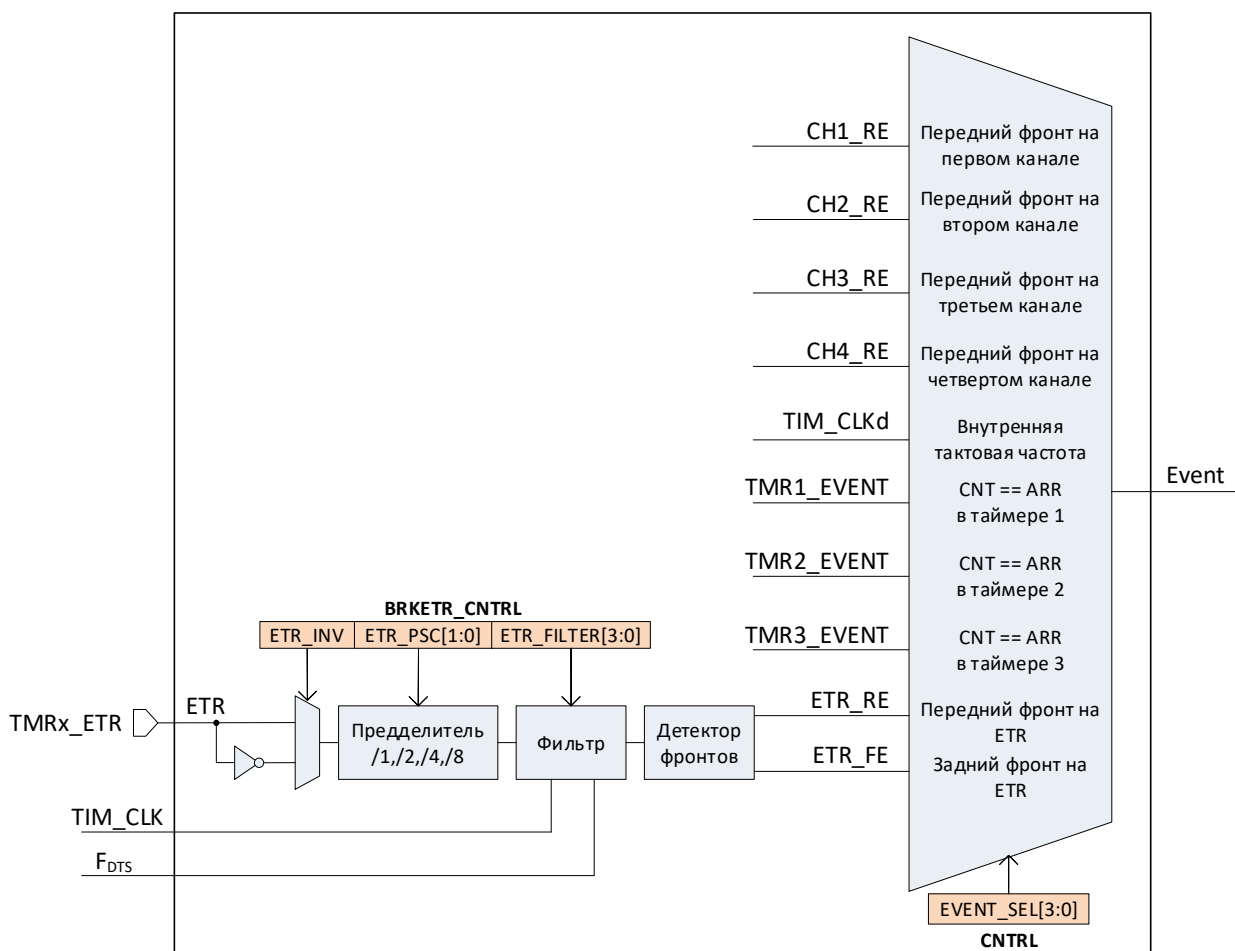


Рисунок 71 – Схема тактирования сигналом со входа ETR

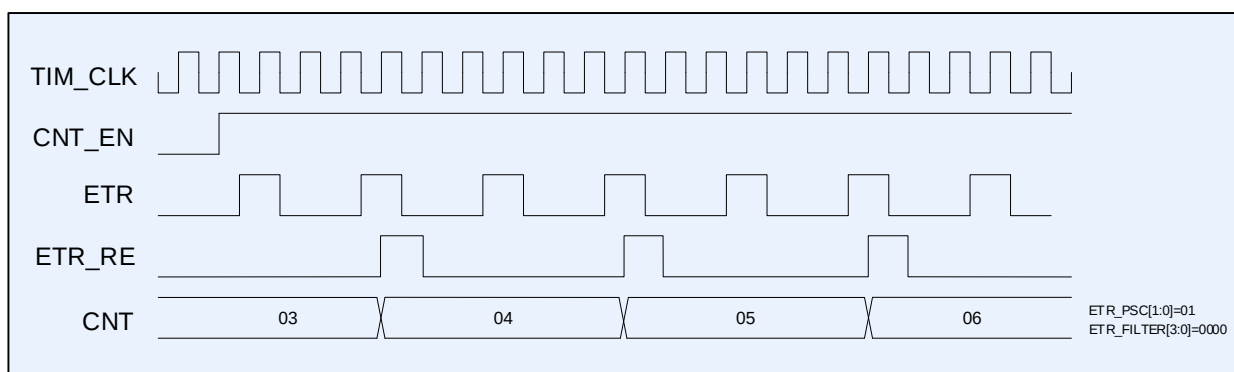


Рисунок 72 – Диаграмма тактирования со входа ETR, EVENT_SEL[3:0] = 1000

23.4 Режим захвата

Каждый канал таймера может быть независимо переведен в режим захвата. В режиме захвата по событию от внешнего входного сигнала происходит фиксация значения основного счетчика CNT в регистры CCRy (CCR) и CCRy1 (CCR1). Регистрация событий осуществляется только по входам CHy_i (выводы, настроенные в функцию TMRx_CHy).

Структурная схема блока захвата представлена на рисунке 73.

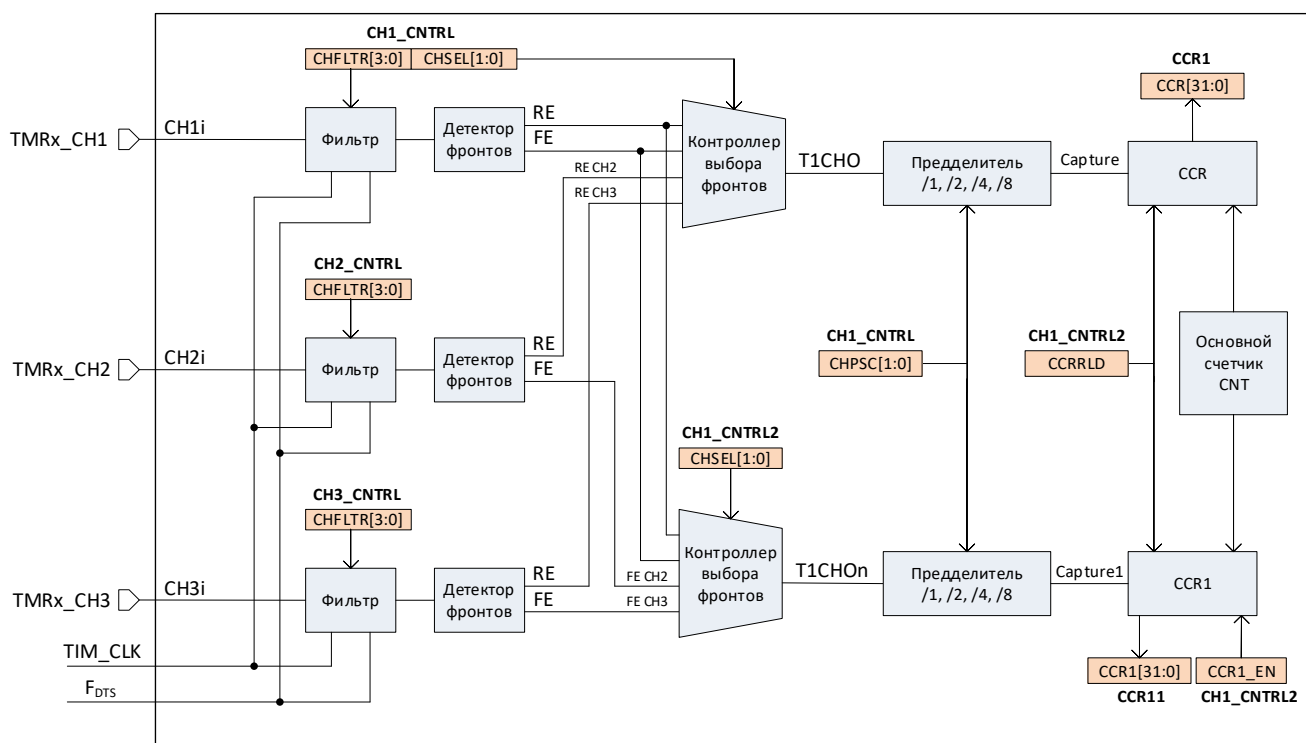


Рисунок 73 – Структурная схема блока захвата на примере канала 1

Для включения режима захвата для определенного канала необходимо записать «1» в бит CAP_NPWM регистра управления каналом CHy_CNTRL. Для использования регистра CCRy1(CCR1) необходимо записать «1» в бит CCR1_EN регистра CHy_CNTRL2.

Внешний сигнал со входа CHy_i сначала поступает в блок фильтра. Данный блок позволяет отфильтровать входной сигнал с целью устранения импульсов, длительность

которых меньше заданного порога (см. подраздел 23.6 «Блок цифрового фильтра»). Настройки фильтра задаются в поле CHFLTR[3:0] регистра CHy_CNTRL.

Сигнал с блока фильтра поступает в блок «Детектор фронтов». При обнаружении положительного фронта входного сигнала данный блок вырабатывает сигнал RE, а при обнаружении отрицательного фронта входного сигнала – сигнал FE.

В блоке «Контроллер выбора фронтов» производится выбор используемого для захвата сигнала между положительным фронтом канала, отрицательным фронтом канала и положительными и отрицательными фронтами сигналов от других каналов. Настройка блока «Контроллер выбора фронтов» для регистра CCRy осуществляется в поле CHSEL[1:0] регистра CHy_CNTRL, а для регистра CCRy1 – в поле CHSEL[1:0] регистра CHy_CNTRL2. Выбранный для захвата сигнал поступает в предварительный делитель, который в зависимости от значения в поле CHPSC[1:0] регистра CHy_CNTRL позволяет фиксировать все события, либо каждое второе, каждое четвертое или каждое восьмое событие.

Предварительный делитель для регистра CCRy формирует сигнал Capture, а предварительный делитель для регистра CCRy1 формирует сигнал Capture1. По сигналам Capture и Capture1 выполняется запись текущего значения основного счетчика CNT в регистры CCRy и CCRy1.

На рисунке 74 показан пример захвата значения основного счетчика CNT в регистр CCRy по положительному фронту на входе канала, а в регистр CCRy1 – по отрицательному фронту на входе канала.

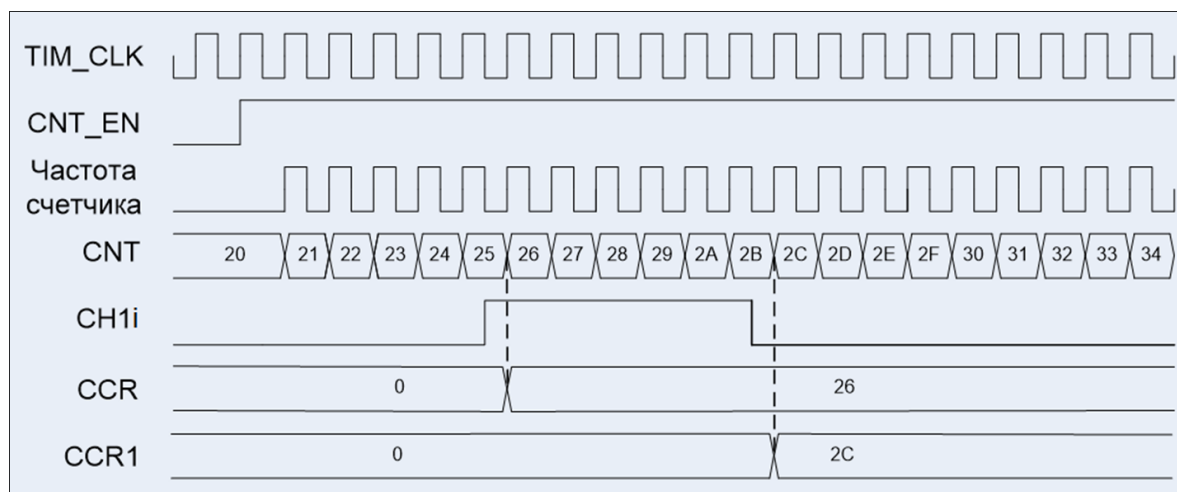


Рисунок 74 – Диаграмма захвата события со входа канала 1

По событию захвата на определенном канале в регистре IE можно разрешить выработку прерываний, а в регистре DMA_RE можно разрешить формирование запросов DMA.

Между формированием события захвата и записью текущего значения основного счетчика CNT в регистры CCRy и CCRy1 может быть установлена задержка с помощью бита EV_DELAY в регистре CHy_CNTRL2. Если бит EV_DELAY равен «0», то сначала формируется событие захвата, а затем через один такт сигнала синхронизации TIM_CLK выполняется запись CNT в регистры CCRy и CCRy1. Если бит EV_DELAY равен «1», то обновление информации в регистрах CCRy и CCRy1 происходит синхронно с событием

захвата. При реализации чтения регистров CCRy и CCRy1 по событию захвата рекомендуется устанавливать бит EV_DELAY в «1».

23.5 Режим ШИМ

Каждый канал таймера может быть независимо переведен в режим ШИМ для формирования выходных сигналов с возможностью задания «мертвой зоны». Структурная схема блока формирования ШИМ представлена на рисунке 75.

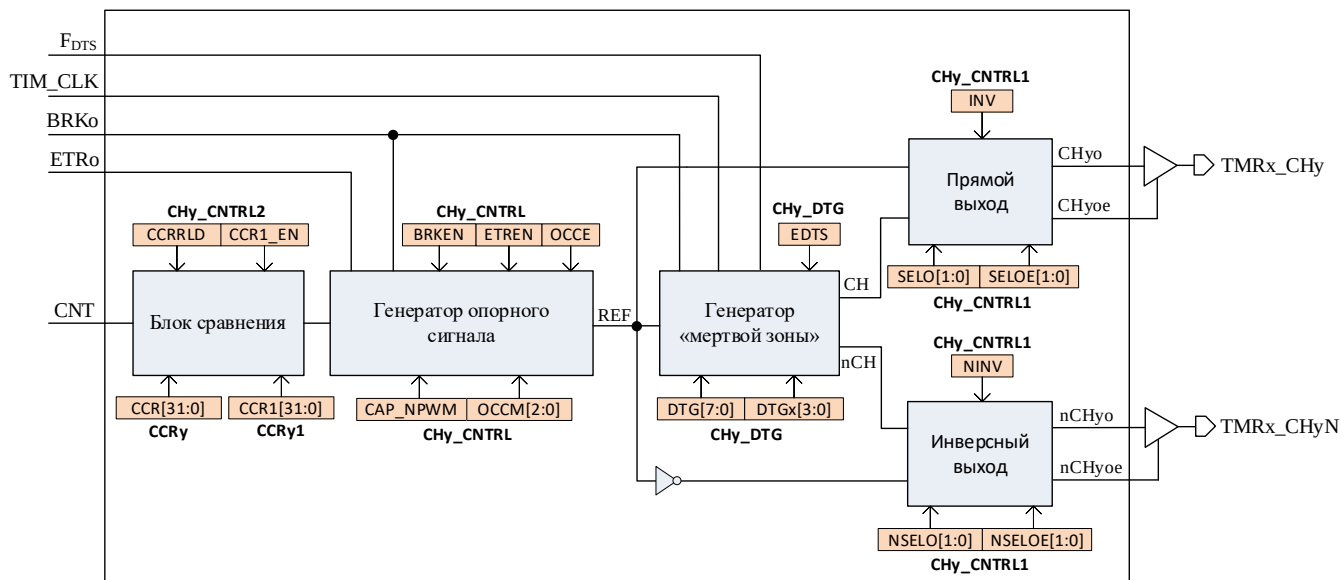


Рисунок 75 – Структурная схема блока формирования ШИМ

Для включения режима ШИМ для определенного канала необходимо в регистре управления каналом CHy_CNTRL записать «0» в бит CAP_NPWM.

23.5.1 Генератор опорного сигнала REF

При работе в режиме ШИМ блок генератора опорного сигнала формирует сигнал REF. Данный сигнал формируется на основании сравнения значения в регистрах CCRy (CCR), CCRy1 (CCR1) и основного счетчика CNT. Формат выработки сигнала REF устанавливается в регистре управления каналом таймера CHy_CNTRL, поле OCCM[2:0].

Если в регистре CHy_CNTRL2 бит CCR1_EN = 0, то для формирования сигнала REF используется только результат сравнения значения в регистре CCRy и основного счетчика CNT.

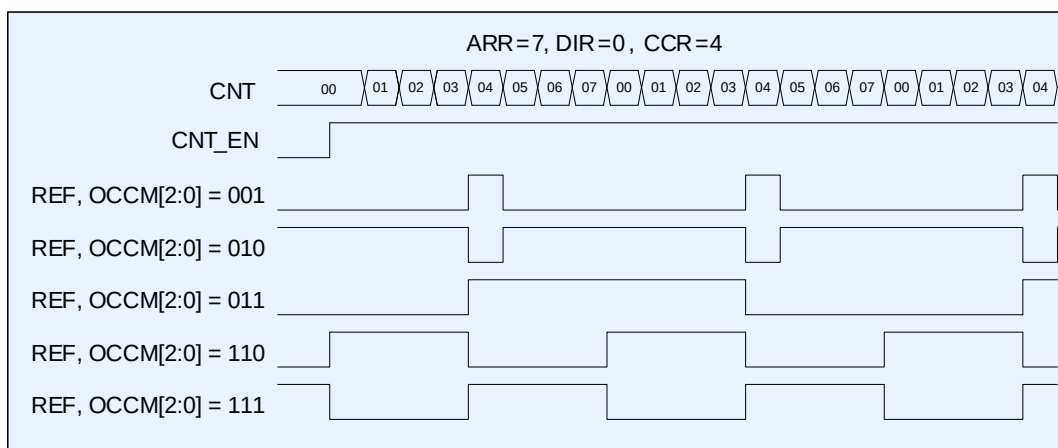


Рисунок 76 – Диаграмма работы в режиме ШИМ, CCR1_EN=0

Если в регистре CHy_CNTRL2 бит CCR1_EN = 1, то для формирования сигнала REF задействуются оба результата сравнения значения в регистрах CCRy, CCRy1 и основного счетчика CNT.

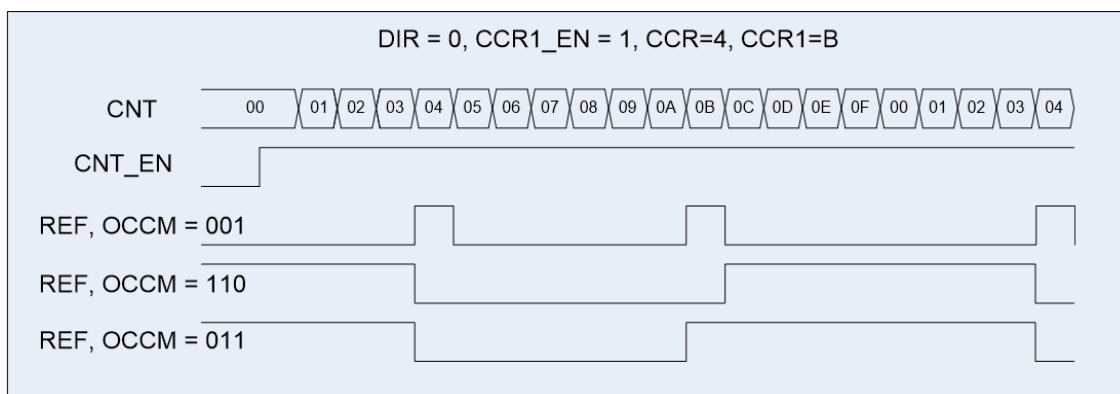


Рисунок 77 – Диаграмма работы в режиме ШИМ, CCR1_EN = 1

Запись новых значений в регистры CCRy и CCRy1 осуществляется немедленно, если в регистре CHy_CNTRL2 бит CCRRLD установлен в «0», иначе регистры CCRy и CCRy1 получают новые значения только при CNT == 0. Процесс обновления значений в регистрах CCRy и CCRy1 обозначается в регистре CHy_CNTRL с помощью флагов WR_CMPL и WR_CMPL1, соответственно. На время выполнения записи флаг WR_CMPL/WR_CMPL1 устанавливается в «1», по окончании записи флаг WR_CMPL/WR_CMPL1 сбрасывается в «0».

Сигнал REF может быть принудительно установлен в «0» с использованием внешнего сигнала сброса, поступающего со входа ETR (высокий активный уровень) или со входа BRK (низкий активный уровень). Активный уровень на входах ETR и BRK может быть изменён с помощью инверсии входного сигнала, регистр BRKETR_CNTRL, биты ETR_INV и BRK_INV, соответственно.

Для разрешения сброса сигнала REF по входу ETR необходимо установить бит ETREN и OCCE в регистре CHy_CNTRL. Активный уровень на входе ETR сбрасывает сигнал REF в «0». После снятия активного уровня на входе ETR сигнал REF остается в «0» до следующего события установки REF в «1» (см. рисунок 78).

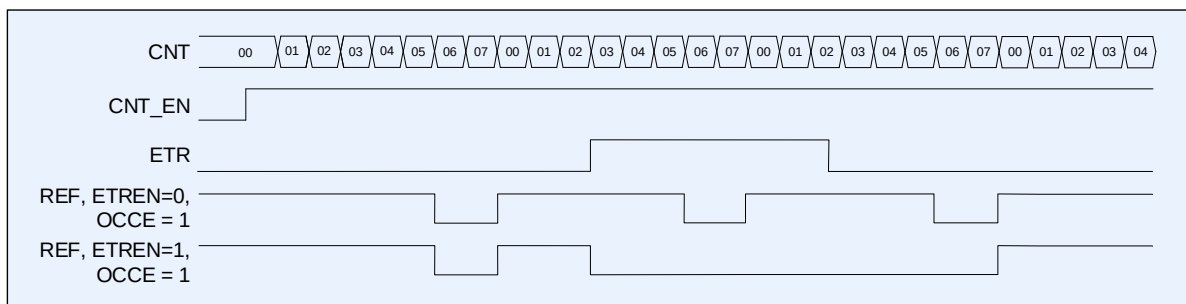


Рисунок 78 – Диаграмма сброса сигнала REF по выводу ETR

Для разрешения сброса сигнала REF по входу BRK необходимо установить бит BRKEN в регистре CHy_CNTRL. Активный уровень на входе BRK сбрасывает сигнал REF в «0» путем маскирования. После снятия активного уровня на входе BRK генерация сигнала REF сразу же восстанавливается (см. рисунок 79).

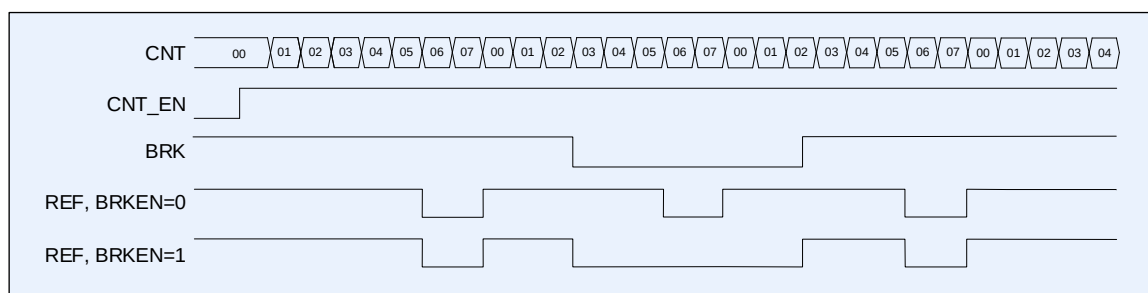


Рисунок 79 – Диаграмма сброса сигнала REF по выводу BRK

23.5.2 Генератор «мертвой зоны»

Блок генератора «мертвой зоны» (dead-time generator, DTG) позволяет на основе сигнала REF формировать комплементарную пару сигналов с «мертвой зоной». Выходные сигналы с блока DTG передаются на выходные блоки следующим образом:

- сигнал на прямом выходе (CHy_o, CHy_{oe}) представляет собой инвертированный сигнал REF, в котором передний фронт задержан на величину DTGdel относительно заднего фронта опорного сигнала REF;
- сигнал на инверсном выходе (nCHy_o, nCHy_{oe}) представляет собой сигнал REF, в котором передний фронт задержан на величину DTGdel относительно переднего фронта опорного сигнала REF.

Значение «мертвой зоны» между сигналами на прямом и инверсном выходах рассчитывается в тактах частоты TIM_CLK или F_{DTS} по формуле

$$DTGdel = DTG \cdot (DTGx + 1), \quad (9)$$

где DTGx – предварительный делитель частоты;
DTG – основной делитель частоты.

Управление блоком DTG осуществляется через регистр CHy_DTG. Выбор источника тактирования для задания «мертвой зоны» задается битом EDTS. Значения делителей DTGx и DTG задаются в полях DTGx[3:0] и DTG[7:0], соответственно. Если задержка DTGdel больше ширины импульса высокого уровня, то соответствующий импульс не генерируется.

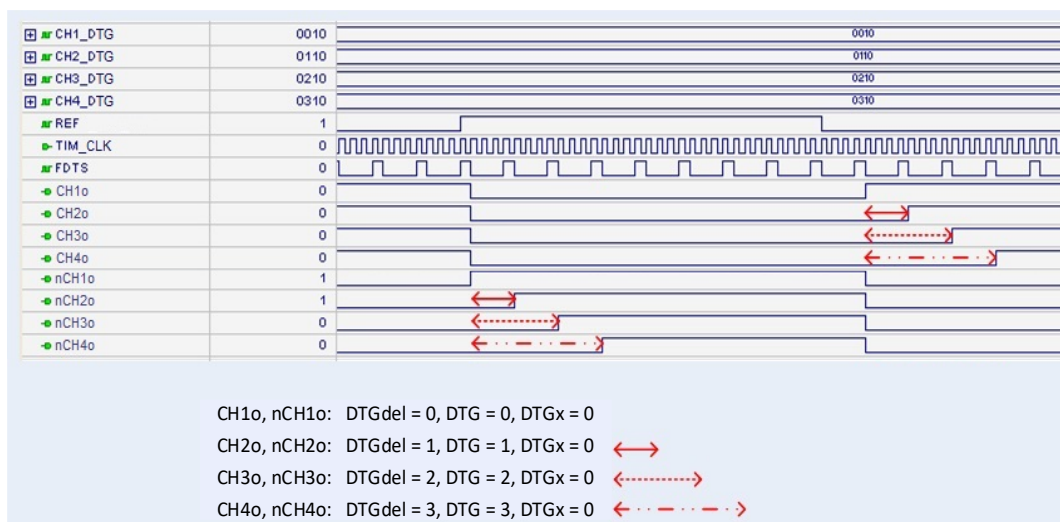


Рисунок 80 – Диаграмма работы блока DTG

Выходные сигналы блока DTG могут быть принудительно установлены в «0» с использованием внешнего сигнала сброса, поступающего со входа BRK (низкий активный уровень). Активный уровень на входе BRK может быть изменён с помощью инверсии входного сигнала, бит BRK_INV в регистре BRKETR_CNTRL. Для разрешения сброса выходных сигналов блока DTG по входу BRK необходимо установить бит BRKEN в регистре CHy_CNTRL. Активный уровень на входе BRK сбрасывает выходные сигналы блока DTG в «0» путем их маскирования.

23.5.3 Выходные блоки

Каждый канал таймера имеет два выходных блока – прямой и инверсный. Каждый выходной блок формирует как сигнал выдачи (CHyo, nCHyo), так и сигнал разрешения выдачи (CHyoe, nCHyoe). В качестве сигналов для прямого (CHyo, CHyoe) и инверсного (nCHyo, nCHyoe) выходов в регистре CHy_CNTRL1 могут быть заданы либо постоянные уровни (0 или 1), либо сигналы, формируемые на основе сигнала REF. К таким сигналам относится сам сигнал REF, а также сигналы, формируемые блоком DTG.

Выбор источника сигнала выдачи для прямого (CHyo) и инверсного (nCHyo) выходов задается в полях SELO[1:0] и NSELO[1:0] регистра CHy_CNTRL1. Дополнительно каждый сигнал выдачи для прямого (CHyo) и инверсного (nCHyo) выходов может быть инвертирован путем установки битов INV и NINV в регистре CHy_CNTRL1. Выбор источника сигнала разрешения выдачи для прямого (CHyoe) и инверсного (nCHyoe) выходов задается в полях SELOE[1:0] и NSELOE[1:0] регистра CHy_CNTRL1. При этом, если сигнал разрешения выдачи равен «0», то соответствующий вывод работает в режиме входа, если сигнал разрешения выдачи равен «1» – то в режиме выхода.

23.6 Блок цифрового фильтра

В тракте входа ETR и входов каналов таймера CHyi предусмотрен блок цифрового фильтра, который позволяет исключить из входного сигнала импульсы высокого и низкого уровня, длительность которых меньше заданного порога.

Конфигурация фильтра для входа ETR выполняется в поле ETR_FILTER[3:0] регистра BRKETR_CNTRL, для входов каналов CHy_i – в поле CHFLTR[3:0] регистра CHy_CNTRL. Значение в данных полях позволяет настроить два параметра фильтра:

- частота выборки F_S , на которой входной сигнал захватывается в сдвиговый регистр для накопления. В качестве частоты F_S может использоваться частота TIM_CLK или F_{DTS} ;
- количество выборок (длина фильтра) N , на протяжении которых входной сигнал должен оставаться стабильным, чтобы не подвергнуться фильтрации.

Если в течение заданного количества выборок N на частоте F_S входной сигнал не изменяется, то значение входного сигнала передаётся на выход фильтра. Иначе внутренний счетчик накопления сбрасывается и захват сигнала начинается заново.

Таким образом, настраивая частоту F_S и количество выборок N , задается минимальная длительность импульсов входного сигнала, которые не будут отфильтрованы. Диаграмма работы фильтра при использовании частоты TIM_CLK приведена на рисунке 81, частота $F_S = TIM_CLK$, количество выборок $N = 4$.

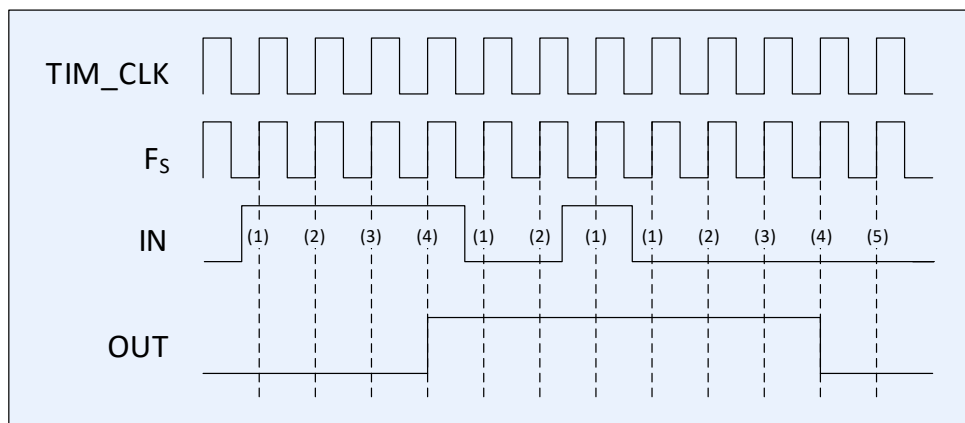


Рисунок 81 – Диаграмма работы фильтра, $F_S = TIM_CLK$, $N = 4$

Для задания длительных интервалов накопления входного сигнала имеется возможность использовать частоту F_{DTS} , которая формируется из частоты TIM_CLK путём прореживания на заданный коэффициент (см. п. 23.2.4 «Тактовая частота F_{DTS} »).

Диаграмма работы фильтра при использовании частоты F_{DTS} приведена на рисунке 82, частота выборки $F_S = F_{DTS}/2$, количество выборок $N = 6$.

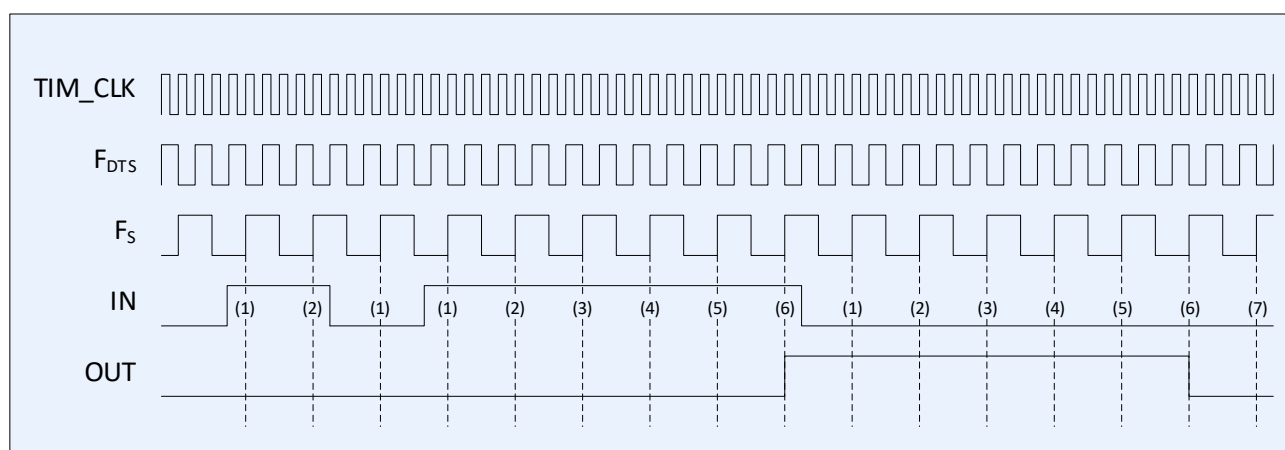


Рисунок 82 – Диаграмма работы фильтра, $F_S = F_{DTS}/2$, $N = 6$

Возможные варианты настройки блока фильтра приведены в таблице 261.

Таблица 261 – Возможные конфигурации фильтра

CHFLTR[3:0], ETR_FILTER[3:0]	Частота выборки F_S	Количество выборок N	Минимальная длительность импульсов, которые не будут отфильтрованы
0000	F_{DTS}	1	—
0001	TIM_CLK	2	$2 \times T_{TIM_CLK}$
0010	TIM_CLK	4	$4 \times T_{TIM_CLK}$
0011	TIM_CLK	8	$8 \times T_{TIM_CLK}$
0100	$F_{DTS}/2$	6	$12 \times T_{F_{DTS}}$
0101	$F_{DTS}/2$	8	$16 \times T_{F_{DTS}}$
0110	$F_{DTS}/4$	6	$24 \times T_{F_{DTS}}$
0111	$F_{DTS}/4$	8	$32 \times T_{F_{DTS}}$
1000	$F_{DTS}/8$	6	$48 \times T_{F_{DTS}}$
1001	$F_{DTS}/8$	8	$64 \times T_{F_{DTS}}$
1010	$F_{DTS}/16$	5	$80 \times T_{F_{DTS}}$
1011	$F_{DTS}/16$	6	$96 \times T_{F_{DTS}}$
1100	$F_{DTS}/16$	8	$128 \times T_{F_{DTS}}$
1101	$F_{DTS}/32$	5	$160 \times T_{F_{DTS}}$
1110	$F_{DTS}/32$	6	$192 \times T_{F_{DTS}}$
1111	$F_{DTS}/32$	8	$256 \times T_{F_{DTS}}$

23.7 Флаги состояний, прерывания и запросы DMA

В процессе работы блок таймера отслеживает состояние внутренних блоков и формирует 17 событий:

- CNT ZERO EVENT – совпадение значения счетчика CNT с нулем;
- CNT ARR EVENT – совпадение значения счетчика CNT со значением в регистре ARR;
- ETR RE EVENT – фиксация переднего фронта на входе ETR;
- ETR FE EVENT – фиксация заднего фронта на входе ETR;
- BRK EVENT – фиксация высокого уровня на входе BRK;

- CCR CAP EVENT[3:0] – запись значения счетчика CNT в регистр CCRy по захвату настроенного фронта на входе канала CHy, события формируются индивидуально для каждого канала;
- CCR REF EVENT[3:0] – фиксация переднего фронта на выходе генератора опорного сигнала REF, события формируются индивидуально для каждого канала;
- CCR CAP1 EVENT[3:0] – запись значения счетчика CNT в регистр CCRy1 по захвату настроенного фронта на входе канала CHy, события формируются индивидуально для каждого канала.

23.7.1 Флаги состояний

При возникновении события устанавливается соответствующий флаг в регистре STATUS. Сброс флагов в регистре STATUS осуществляется записью «0», запись «1» не оказывает влияния. Если запись «0» выполняется одновременно с новым событием, то приоритет у нового события.

Флаги состояний обновляются при включенном блоке таймера.

23.7.2 Прерывания

Блок таймера на основе флагов в регистре STATUS формирует один общий сигнал запроса прерывания INT_TMRx. Выбор флагов, формирующих запрос прерывания, осуществляется через регистр разрешения прерываний IE. При формировании запроса прерывания маскированные состояния флагов из регистра STATUS объединяются по схеме ИЛИ.

23.7.3 Запросы DMA

На основе отслеживаемых событий блок таймера формирует сигнал запроса DMA TMRx_DMA_REQ. Выбор событий, формирующих запрос DMA, осуществляется через регистр DMA_RE.

Запросы к блоку DMA после конфигурации регистра DMA_RE формируются в том числе, когда блок выключен (обратить внимание на событие CNT_ZERO_EVNT: по умолчанию CNT = 0).

23.8 Примеры

В данном разделе приведены примеры инициализации таймера 1 в различных режимах работы. Для других таймеров инициализация выполняется аналогично.

23.8.1 Обычный счетчик

```
MDR_RST_CLK->PER_CLOCK |= 0x4000; //Разрешение частоты PCLK для таймера 1
MDR_RST_CLK->TIM_CLOCK = 0x01000000; //Настройка и подача частоты TIM_CLK
                                //для таймера 1 (TIM_CLK = HCLK)
MDR_TIMER1->CNTRL = 0x00000000; //Режим инициализации таймера
//Настраиваем работу основного счетчика
MDR_TIMER1->CNT = 0x00000000; //Начальное значение счетчика
MDR_TIMER1->PSG = 0x00000000; //Предделитель частоты TIM_CLK
MDR_TIMER1->ARR = 0x0000000F; //Основание счета
```

MDR_TIMER1->IE = 0x00000002; //Разрешение генерировать прерывание при CNT = ARR
 //Разрешение работы таймера
 MDR_TIMER1->CNTRL = 0x00000001; //Счет прямой по TIM_CLK

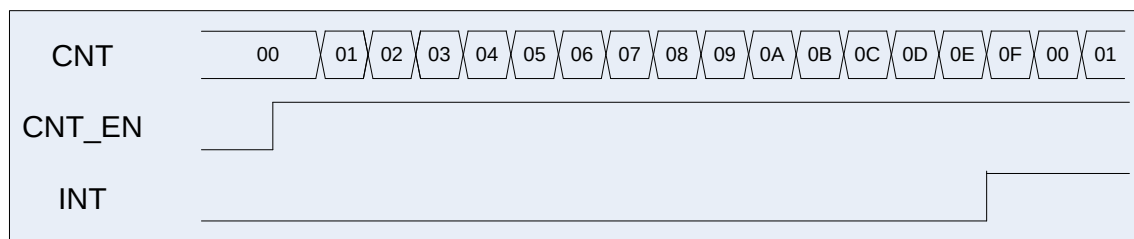


Рисунок 83 – Режим обычного счетчика

23.8.2 Режим захвата

MDR_RST_CLK->PER_CLOCK |= 0x4000; //Разрешение частоты PCLK для таймера 1
 MDR_RST_CLK->TIM_CLOCK = 0x01000000; //Настройка и подача частоты TIM_CLK
 //для таймера 1 (TIM_CLK = HCLK)
 MDR_TIMER1->CNTRL = 0x00000000; //Режим инициализации таймера
 //Настраиваем работу основного счетчика
 MDR_TIMER1->CNT = 0x00000000; //Начальное значение счетчика
 MDR_TIMER1->PSG = 0x00000000; //Предделитель частоты TIM_CLK
 MDR_TIMER1->ARR = 0x000000FF; //Основание счета
 MDR_TIMER1->IE = 0x000001E0; //Разрешение генерировать прерывание по событию
 //настроенного фронта на входах CH1i-CH4i
 MDR_TIMER1->CH1_CNTRL = 0x00008000; //Захват по положительному фронту сигнала
 //на входе CH1i, фильтрация отключена
 MDR_TIMER1->CH2_CNTRL = 0x00008010; //Захват по отрицательному фронту сигнала
 //на входе CH2i, фильтрация отключена
 MDR_TIMER1->CH3_CNTRL = 0x00008001; //Захват по положительному фронту сигнала
 //на входе CH3i, фильтрация выполняется
 //по 2 выборкам на частоте TIM_CLK
 MDR_TIMER1->CH4_CNTRL = 0x00008011; //Захват по отрицательному фронту сигнала
 //на входе CH4i, фильтрация выполняется
 //по 2 выборкам на частоте TIM_CLK
 //Режим работы выхода канала – канал работает на вход
 MDR_TIMER1->CH1_CNTRL1 = 0x00000000;
 MDR_TIMER1->CH2_CNTRL1 = 0x00000000;
 MDR_TIMER1->CH3_CNTRL1 = 0x00000000;
 MDR_TIMER1->CH4_CNTRL1 = 0x00000000;
 //Разрешение работы таймера
 MDR_TIMER1->CNTRL = 0x00000001; //Счет прямой по TIM_CLK

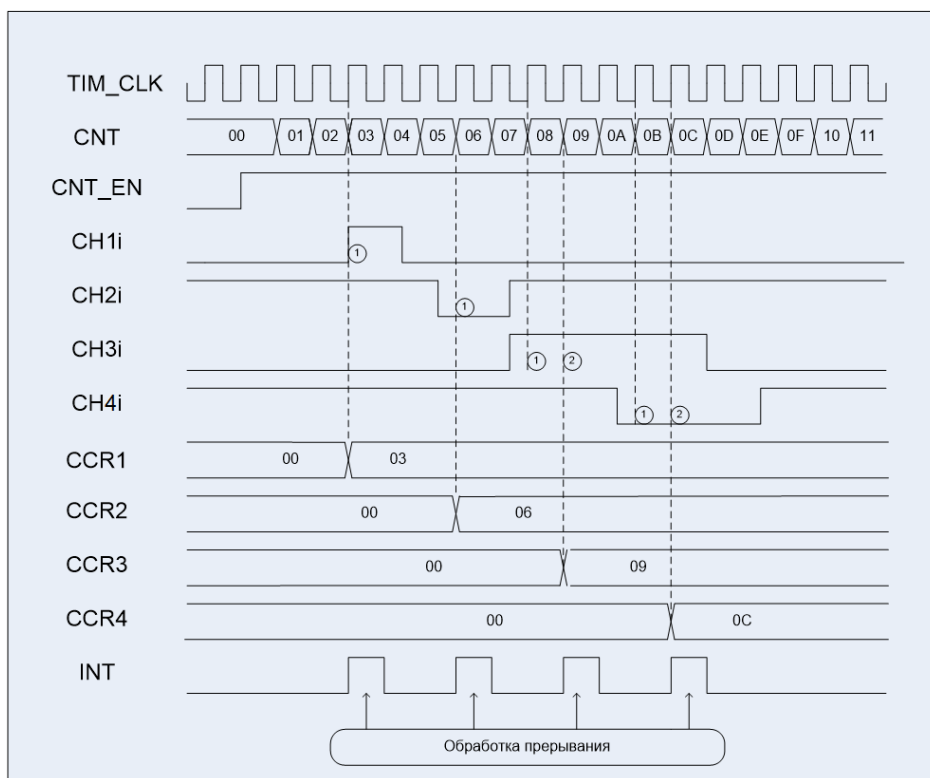


Рисунок 84 – Диаграммы примера работы в режиме захвата

23.8.3 Режим ШИМ

```
MDR_RST_CLK->PER_CLOCK |= 0x4000;    //Разрешение частоты PCLK для таймера 1
MDR_RST_CLK->TIM_CLOCK = 0x01000000; //Настройка и подача частоты TIM_CLK
                                     //для таймера 1 (TIM_CLK = HCLK)

MDR_TIMER1->CNTRL = 0x00000000; //Режим инициализации таймера
//Настраиваем работу основного счетчика
MDR_TIMER1->CNT = 0x00000000; //Начальное значение счетчика
MDR_TIMER1->PSG = 0x00000000; //Предделитель частоты TIM_CLK
MDR_TIMER1->ARR = 0x00000010; //Основание счета
MDR_TIMER1->IE = 0x00001E00;    //Разрешение генерировать прерывание по событию
                                //переднего фронта на выходе REF для всех каналов

//Режим работы каналов – ШИМ
MDR_TIMER1->CH1_CNTRL = 0x00000200; //REF = 1, если CNT == CCR
MDR_TIMER1->CH2_CNTRL = 0x00000200; //REF = 1, если CNT == CCR
MDR_TIMER1->CH3_CNTRL = 0x00000400; //REF = 0, если CNT == CCR
MDR_TIMER1->CH4_CNTRL = 0x00000600; //Переключение REF, если CNT == CCR
//Режим работы выхода канала – канал работает на выход,
//на выходы канала выдается сигнал REF
MDR_TIMER1->CH1_CNTRL1 = 0x00000909;
MDR_TIMER1->CH2_CNTRL1 = 0x00000909;
MDR_TIMER1->CH3_CNTRL1 = 0x00000909;
MDR_TIMER1->CH4_CNTRL1 = 0x00000909;
//Установка значений CCR, с которыми сравнивается CNT при работе в режиме ШИМ
MDR_TIMER1->CCR1 = 0x00000003;
MDR_TIMER1->CCR2 = 0x00000006;
MDR_TIMER1->CCR3 = 0x00000009;
```

MDR_TIMER1->CCR4 = 0x0000000F;

//Разрешение работы таймера

MDR_TIMER1->CNTRL = 0x00000001; //Счет прямой по TIM_CLK

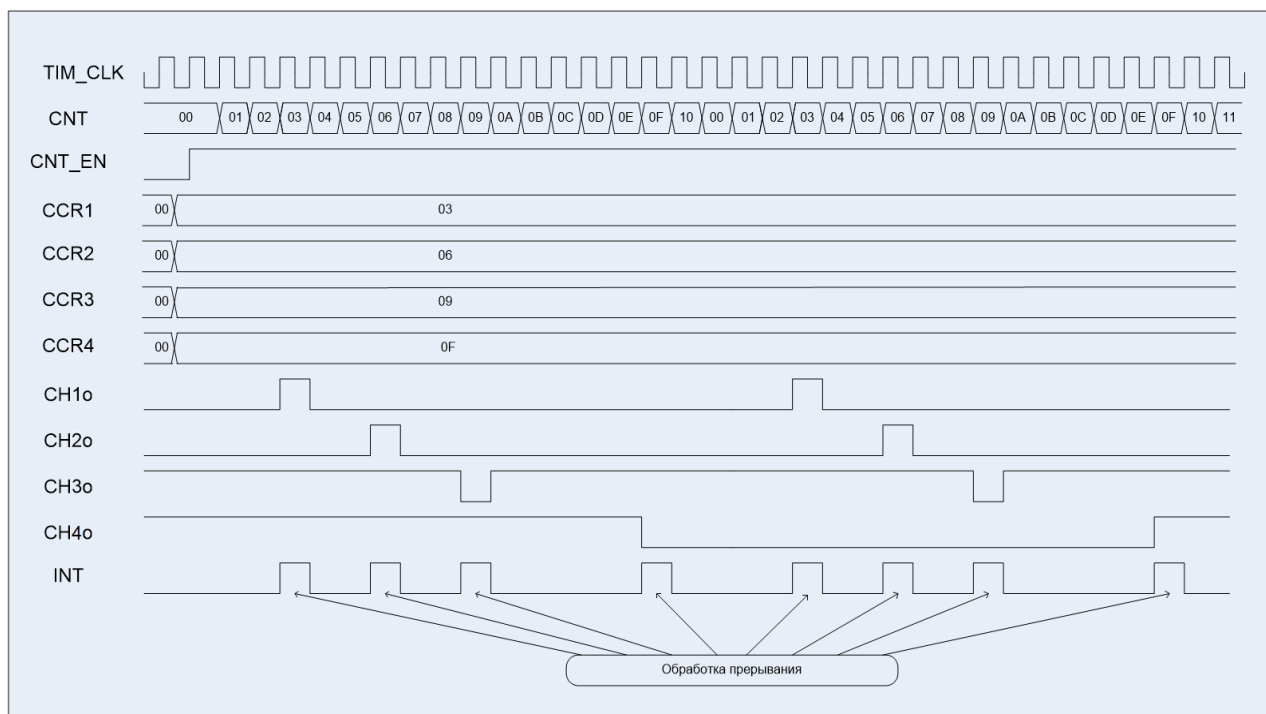


Рисунок 85 – Диаграммы примера работы в режиме ШИМ

23.9 Описание регистров блока таймера

Таблица 262 – Базовые адреса и смещения регистров управления таймера

Адрес	Название	Описание
0x4007_0000	MDR_TIMER1	Контроллер TIMER1
0x4007_8000	MDR_TIMER2	Контроллер TIMER2
0x4008_0000	MDR_TIMER3	Контроллер TIMER3
Смещение		
0x00	CNT[15:0]	Основной счетчик таймера MDR_TIMERx->CNT
0x04	PSG[15:0]	Делитель частоты TIM_CLK для тактирования основного счетчика MDR_TIMERx->PSG
0x08	ARR[15:0]	Основание счета основного счетчика MDR_TIMERx->ARR
0x0C	CNTRL[31:0]	Регистр управления основным счетчиком MDR_TIMERx->CNTRL
0x50	BRKETR_CNTRL[31:0]	Регистр управления входом BRK и ETR MDR_TIMERx->BRKETR_CNTRL
0x54	STATUS[31:0]	Регистр статуса таймера MDR_TIMERx->STATUS
0x58	IE[31:0]	Регистр разрешения прерываний таймера MDR_TIMERx->IE

Адрес	Название	Описание
0x5C	DMA_RE[31:0]	Регистр разрешения формирования запроса DMA MDR_TIMERx->DMA_RE
Канал 1		
0x10	CCR1[15:0]	Регистр сравнения/захвата для 1 канала таймера MDR_TIMERx->CCRx
0x20	CH1_CNTRL[31:0]	Регистр управления для 1 канала таймера MDR_TIMERx->CHx_CNTRL
0x30	CH1_CNTRL1[31:0]	Регистр управления 1 для 1 канала таймера MDR_TIMERx->CHx_CNTRL1
0x40	CH1_DTG[31:0]	Регистр управления DTG для 1 канала таймера MDR_TIMERx->CHx_DTG
0x60	CH1_CNTRL2[31:0]	Регистр управления 2 для 1 канала таймера MDR_TIMERx->CHx_CNTRL2
0x70	CCR11[15:0]	Регистр сравнения/захвата 1 для 1 канала таймера MDR_TIMERx->CCRx1
Канал 2		
0x14	CCR2[15:0]	Регистр сравнения/захвата для 2 канала таймера MDR_TIMERx->CCRx
0x24	CH2_CNTRL[31:0]	Регистр управления для 2 канала таймера MDR_TIMERx->CHx_CNTRL
0x34	CH2_CNTRL1[31:0]	Регистр управления 1 для 2 канала таймера MDR_TIMERx->CHx_CNTRL1
0x44	CH2_DTG[31:0]	Регистр управления DTG для 2 канала таймера MDR_TIMERx->CHx_DTG
0x64	CH2_CNTRL2[31:0]	Регистр управления 2 для 2 канала таймера MDR_TIMERx->CHx_CNTRL2
0x74	CCR21[15:0]	Регистр сравнения/захвата 1 для 2 канала таймера MDR_TIMERx->CCRx1
Канал 3		
0x18	CCR3[15:0]	Регистр сравнения/захвата для 3 канала таймера MDR_TIMERx->CCRx
0x28	CH3_CNTRL[31:0]	Регистр управления для 3 канала таймера MDR_TIMERx->CHx_CNTRL
0x38	CH3_CNTRL1[31:0]	Регистр управления 1 для 3 канала таймера MDR_TIMERx->CHx_CNTRL1
0x48	CH3_DTG[31:0]	Регистр управления DTG для 3 канала таймера MDR_TIMERx->CHx_DTG
0x68	CH3_CNTRL2[31:0]	Регистр управления 2 для 3 канала таймера MDR_TIMERx->CHx_CNTRL2
0x78	CCR31[15:0]	Регистр сравнения/захвата 1 для 3 канала таймера MDR_TIMERx->CCRx1

Адрес	Название	Описание
Канал 4		
0x1C	CCR4[15:0]	Регистр сравнения/захвата для 4 канала таймера MDR_TIMERx->CCRy
0x2C	CH4_CNTRL[31:0]	Регистр управления для 4 канала таймера MDR_TIMERx->CHy_CNTRL
0x3C	CH4_CNTRL1[31:0]	Регистр управления 1 для 4 канала таймера MDR_TIMERx->CHy_CNTRL1
0x4C	CH4_DTG[31:0]	Регистр управления DTG для 4 канала таймера MDR_TIMERx->CHy_DTG
0x6C	CH4_CNTRL2[31:0]	Регистр управления 2 для 4 канала таймера MDR_TIMERx->CHy_CNTRL2
0x7C	CCR41[15:0]	Регистр сравнения/захвата 1 для 4 канала таймера MDR_TIMERx->CCRy1

23.9.1 MDR_TIMERx->CNT

Таблица 263 – Основной счетчик таймера CNT

Номер	15...0
Доступ	R/W
Сброс	0
	CNT[15:0]

Таблица 264 – Описание бит регистра CNT

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
15...0	CNT[15:0]	Значение основного счетчика таймера

23.9.2 MDR_TIMERx->PSG

Таблица 265 – Делитель частоты TIM_CLK для счета основного счетчика PSG

Номер	15..0
Доступ	R/W
Сброс	0
	PSG[15:0]

Таблица 266 – Описание бит регистра PSG

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
15...0	PSG[15:0]	Значение предварительного делителя счетчика. Основной счетчик считает на частоте: $TIM_CLKd = TIM_CLK/(PSG+1)$

23.9.3 MDR_TIMERx->ARR

Таблица 267 – Основание счета основного счетчика ARR

Номер	15...0
Доступ	R/W
Сброс	0
	ARR[15:0]

Таблица 268 – Описание бит регистра ARR

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
15...0	ARR[15:0]	Основание счета для основного счетчика. CNT = [0...ARR]

23.9.4 MDR_TIMERx->CNTRL

Таблица 269 – Регистр управления основным счетчиком CNTRL

Номер	31...12	11...8
Доступ	U	R/W
Сброс	0	0
	-	EVENT_SEL[3:0]

Номер	7...6	5...9	3	2	1	0
Доступ	R/W	R/W	R/W	R/W	R/W	R/W
Сброс	00	00	0	0	0	0
	CNT_MODE[1:0]	FDTs[1:0]	DIR	WR_CMPL	ARRB_EN	CNT_EN

Таблица 270 – Описание бит регистра CNTRL

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...12	-	Зарезервировано
11...8	EVENT_SEL[3:0]	Биты выбора источника событий: 0000 – внутренняя тактовая частота TIM_CLKd (формируется путем деления частоты TIM_CLK); 0001 – CNT == ARR в таймере 1; 0010 – CNT == ARR в таймере 2; 0011 – CNT == ARR в таймере 3; 0100 – событие переднего фронта на канале 1, «Режим 1»; 0101 – событие переднего фронта на канале 2, «Режим 1»; 0110 – событие переднего фронта на канале 3, «Режим 1»; 0111 – событие переднего фронта на канале 4, «Режим 1»; 1000 – событие переднего фронта на ETR, «Режим 2»; 1001 – событие заднего фронта на ETR, «Режим 2»; 1010-1111 – зарезервировано
7..6	CNT_MODE[1:0]	Режим счета основного счетчика: 00 – счетчик прямой при DIR = 0; счетчик обратный при DIR = 1; 01 – счетчик двунаправленный с автоматическим изменением DIR при CNT == 0 или CNT == ARR; 10 – счетчик прямой при DIR = 0; счетчик обратный при DIR = 1; 11 – зарезервировано. Режим счета CNT_MODE[1:0] необходимо устанавливать в соответствии со значением в поле EVENT_SEL[3:0]: - EVENT_SEL[3:0] = 0000: CNT_MODE[1:0] = 00 или 01; - EVENT_SEL[3:0] != 0000: CNT_MODE[1:0] = 10
5...4	FDTs[1:0]	Делитель тактовой частоты FDTs: 00 – FDTs = TIM_CLK; 01 – FDTs = TIM_CLK/2; 10 – FDTs = TIM_CLK/3; 11 – FDTs = TIM_CLK/4
3	DIR	Направление счета основного счетчика: 0 – прямой, от 0 до ARR; 1 – обратный, от ARR до 0

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
2	WR_CMPL	Флаг выполнения записи нового значения в регистры CNT, PSG и ARR: 0 – новые данные можно записывать; 1 – данные не записаны и идет запись
1	ARRB_EN	Режим обновления регистра ARR: 0 – ARR будет перезаписан в момент записи в ARR; 1 – ARR будет перезаписан при CNT == ARR
0	CNT_EN	Разрешение работы таймера: 0 – таймер отключен; 1 – таймер включен

23.9.5 MDR_TIMERx->CCRy

Таблица 271 – Регистр сравнения/захвата для 'у' канала таймера CCRy

Номер	15...0
Доступ	R/W
Сброс	0
	CCR[15:0]

Таблица 272 – Описание бит регистра CCRy

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
15...0	CCR[15:0]	Режим захвата: значение CNT, при котором произошел факт захвата события. Режим ШИМ: значение CCR, с которым сравнивается CNT

23.9.6 MDR_TIMERx->CCRy1

Таблица 273 – Регистр сравнения/захвата для 'у' канала таймера CCRy1

Номер	15...0
Доступ	R/W
Сброс	0
	CCR1[15:0]

Таблица 274 – Описание бит регистра CCRy1

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
15...0	CCR1[15:0]	Режим захвата: значение CNT, при котором произошел факт захвата события. Режим ШИМ: значение CCR1, с которым сравнивается CNT

23.9.7 MDR_TIMERx->CHy_CNTRL

Таблица 275 – Регистр управления для ‘у’ канала таймера CHy_CNTRL

Номер	31...17	16	15	14	13
Доступ	U	RO	R/W	RO	R/W
Сброс	0	0	0	0	0
	-	WR_CMPL1	CAP_NPWM	WR_CMPL	ETREN

Номер	12	11...9	8	7...6	5...4	3...0
Доступ	R/W	R/W	R/W	R/W	R/W	R/W
Сброс	0	000	0	00	00	0000
	BRKEN	OCCM[2:0]	OCCE	CHPSC[1:0]	CHSEL[1:0]	CHFLTR[3:0]

Таблица 276 – Описание бит регистра CHy_CNTRL

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...17	-	Зарезервировано
16	WR_CMPL1	Флаг выполнения записи нового значения в регистр CCRy1: 0 – новые данные можно записывать; 1 – данные не записаны и идет запись
15	CAP_NPWM	Режим работы канала: 0 – канал работает в режиме ШИМ; 1 – канал работает в режиме захвата
14	WR_CMPL	Флаг выполнения записи нового значения в регистр CCRy: 0 – новые данные можно записывать; 1 – данные не записаны и идет запись
13	ETREN	Разрешение сброса сигнала REF в «0» при высоком уровне на входе ETR: 0 – запрещен; 1 – разрешен
12	BRKEN	Разрешение сброса сигналов REF и DTG в «0» при низком уровне на входе BRK: 0 – запрещен; 1 – разрешен

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
11...9	OCCM[2:0]	Формат выработки сигнала REF в режиме ШИМ: Если CCR1_EN=0: 000 – всегда 0; 001 – 1, если CNT==CCR; 010 – 0, если CNT==CCR; 011 – переключение REF, если CNT==CCR; 100 – всегда 0; 101 – всегда 1; 110 – 1, если DIR=0 (счет прямой), CNT<CCR, иначе 0; 0, если DIR=1 (счет обратный), CNT>CCR, иначе 1; 111 – 0, если DIR=0 (счет прямой), CNT<CCR, иначе 1; 1, если DIR=1 (счет обратный), CNT>CCR, иначе 0. Если CCR1_EN=1: 000 – всегда 0; 001 – 1, если CNT==CCR или CNT==CCR1; 010 – 0, если CNT==CCR или CNT==CCR1; 011 – переключение REF, если CNT==CCR или CNT==CCR1; 100 – всегда 0; 101 – всегда 1; 110 – 0, если DIR=0 (счет прямой), CCR≤CNT≤CCR1, иначе 1; 0, если DIR=1 (счет обратный), CCR<CNT<CCR1, иначе 1; 111 – 1, если DIR=0 (счет прямой), CCR≤CNT≤CCR1, иначе 0; 1, если DIR=1 (счет обратный), CCR<CNT<CCR1, иначе 0; Необходимо соблюдать условие CCR<CCR1
8	OCCE	Разрешение работы ETR: 0 – запрещен; 1 – разрешен
7...6	CHPSC[1:0]	Предварительный делитель входного канала: 00 – нет деления; 01 – /2; 10 – /4; 11 – /8
5...4	CHSEL[1:0]	Выбор события по входному каналу CHy_i для фиксации значения основного счетчика (регистр CNT) в регистр CCRy: 00 – положительный фронт на входном канале CHy_i; 01 – отрицательный фронт на входном канале CHy_i; 10 – положительный фронт от других каналов: для первого канала от второго канала; для второго канала от третьего канала; для третьего канала от четвертого канала; для четвертого канала от первого канала; 11 – положительный фронт от других каналов: для первого канала от третьего канала; для второго канала от четвертого канала; для третьего канала от первого канала; для четвертого канала от второго канала

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
3...0	CHFLTR[3:0]	Конфигурация фильтра на входе канала 'у'. Выбор частоты выборки F_s и количества выборок N : 0000 – нет фильтрации, $F_s = F_{DTS}$; 0001 – $F_s = TIM_CLK$, $N = 2$; 0010 – $F_s = TIM_CLK$, $N = 4$; 0011 – $F_s = TIM_CLK$, $N = 8$; 0100 – $F_s = F_{DTS}/2$, $N = 6$; 0101 – $F_s = F_{DTS}/2$, $N = 8$; 0110 – $F_s = F_{DTS}/4$, $N = 6$; 0111 – $F_s = F_{DTS}/4$, $N = 8$; 1000 – $F_s = F_{DTS}/8$, $N = 6$; 1001 – $F_s = F_{DTS}/8$, $N = 8$; 1010 – $F_s = F_{DTS}/16$, $N = 5$; 1011 – $F_s = F_{DTS}/16$, $N = 6$; 1100 – $F_s = F_{DTS}/16$, $N = 8$; 1101 – $F_s = F_{DTS}/32$, $N = 5$; 1110 – $F_s = F_{DTS}/32$, $N = 6$; 1111 – $F_s = F_{DTS}/32$, $N = 8$

23.9.8 MDR_TIMERx->CHy_CNTRL1

Таблица 277 – Регистр управления 1 для 'у' канала таймера CHy_CNTRL1

Номер	31...13	12	11...10	9...8	7...5	4	3...2	1...0
Доступ	U	R/W	R/W	R/W	U	R/W	R/W	R/W
Сброс	0	0	00	00	0	0	00	00
	-	NINV	NSELO [1:0]	NSELOE [1:0]	-	INV	SELO [1:0]	SELOE [1:0]

Таблица 278 – Описание бит регистра CHy_CNTRL1

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений.
31...13	-	Зарезервировано
12	NINV	Инверсия инверсного выхода nCHy: 0 – выход не инвертируется; 1 – выход инвертируется
11...10	NSELO[1:0]	Выбор источника сигнала для инверсного выхода nCHy: 00 – на nCHyо выдается 0; 01 – на nCHyо выдается 1; 10 – на nCHyо выдается сигнал nREF; 11 – на nCHyо выдается сигнал с DTG
9...8	NSELOE[1:0]	Режим работы инверсного выхода nCHy: 00 – на nCHyое выдается 0; 01 – на nCHyое выдается 1; 10 – на nCHyое выдается сигнал nREF; 11 – на nCHyое выдается сигнал с DTG. При nCHyое = 0 вывод канала в третьем состоянии, при nCHyое = 1 вывод канала работает в режиме выхода
7...5	-	Зарезервировано

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений.
4	INV	Инверсия прямого выхода СНу: 0 – выход не инвертируется; 1 – выход инвертируется
3...2	SELO[1:0]	Выбор источника сигнала для прямого выхода СНу: 00 – на СНуо выдается 0; 01 – на СНуо выдается 1; 10 – на СНуо выдается сигнал REF; 11 – на СНуо выдается сигнал с DTG
1...0	SELOE[1:0]	Режим работы прямого выхода СНу: 00 – на СНуое выдается 0; 01 – на СНуое выдается 1; 10 – на СНуое выдается сигнал REF; 11 – на СНуое выдается сигнал с DTG. При СНуое = 0 вывод канала работает в режиме входа, при СНуое = 1 вывод канала работает в режиме выхода

23.9.9 MDR_TIMERx->CHy_CNTRL2

Таблица 279 – Регистр управления 2 для ‘у’ канала таймера CHy_CNTRL2

Номер	31...5	4	3	2	1...0
Доступ	U	R/W	R/W	R/W	R/W
Сброс	0	0	0	0	00
	-	EV_DELAY	CCRRLD	CCR1_EN	CHSEL[1:0]

Таблица 280 – Описание бит регистра CHy_CNTRL2

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...5	-	Зарезервировано
4	EV_DELAY	Задержка события захвата до обновления регистров CCRy и CCRy1: 0 – сигнал события захвата устанавливается в момент обнаружения события, при этом обновление регистров CCRy и CCRy1 выполняется через один такт TIM_CLK; 1 – сигнал события захвата устанавливается синхронно с обновлением информации в регистрах CCRy и CCRy1
3	CCRRLD	Режим обновления регистров CCRy и CCRy1: 0 – обновление возможно в любой момент времени; 1 – обновление будет осуществлено только при CNT == 0
2	CCR1_EN	Разрешение работы регистра CCRy1: 0 – CCRy1 не используется; 1 – CCRy1 используется

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
1...0	CHSEL[1:0]	Выбор события по входному каналу CHy _i для фиксации значения основного счетчика (регистр CNT) в регистр CCRy1: 00 – положительный фронт на входном канале CHy _i ; 01 – отрицательный фронт на входном канале CHy _i ; 10 – отрицательный фронт от других каналов: для первого канала от второго канала; для второго канала от третьего канала; для третьего канала от четвертого канала; для четвертого канала от первого канала; 11 – отрицательный фронт от других каналов: для первого канала от третьего канала; для второго канала от четвертого канала; для третьего канала от первого канала; для четвертого канала от второго канала

23.9.10 MDR_TIMERx->CHy_DTG

Таблица 281 – Регистр управления генератором «мертвой зоны» CHy_DTG

Номер	31...16	15...8	7...5	4	3...0
Доступ	U	R/W	U	R/W	R/W
Сброс	0	00000000	000	0	0000
	-	DTG[7:0]	-	EDTS	DTGx[3:0]

Таблица 282 – Описание бит регистра CHy_DTG

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...16	-	Зарезервировано.
15...8	DTG[7:0]	Основной делитель частоты DTG. Задержка DTGdel = DTG • (DTGx + 1)
7...5	-	Зарезервировано.
4	EDTS	Частота работы DTG: 0 – TIM_CLK; 1 – F _{EDTS}
3...0	DTGx[3:0]	Предварительный делитель частоты DTG

23.9.11 MDR_TIMERx->BRKETR_CNTRL

Таблица 283 – Регистр BRKETR_CNTRL управления входом BRK и ETR

Номер	31...8	7...4	3...2	1	0
Доступ	U	R/W	R/W	R/W	R/W
Сброс	0	0000	00	0	0
	-	ETR_FILTER[3:0]	ETR_PSC[1:0]	ETR_INV	BRK_INV

Таблица 284 – Описание бит регистра BRKETR_CNTRL

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...8	-	Зарезервировано
7...4	ETR_FILTER[3:0]	Конфигурация фильтра на входе ETR. Выбор частоты выборки F_s и количества выборок N : 0000 – нет фильтрации, $F_s = F_{DTS}$; 0001 – $F_s = TIM_CLK$, $N = 2$; 0010 – $F_s = TIM_CLK$, $N = 4$; 0011 – $F_s = TIM_CLK$, $N = 8$; 0100 – $F_s = F_{DTS}/2$, $N = 6$; 0101 – $F_s = F_{DTS}/2$, $N = 8$; 0110 – $F_s = F_{DTS}/4$, $N = 6$; 0111 – $F_s = F_{DTS}/4$, $N = 8$; 1000 – $F_s = F_{DTS}/8$, $N = 6$; 1001 – $F_s = F_{DTS}/8$, $N = 8$; 1010 – $F_s = F_{DTS}/16$, $N = 5$; 1011 – $F_s = F_{DTS}/16$, $N = 6$; 1100 – $F_s = F_{DTS}/16$, $N = 8$; 1101 – $F_s = F_{DTS}/32$, $N = 5$; 1110 – $F_s = F_{DTS}/32$, $N = 6$; 1111 – $F_s = F_{DTS}/32$, $N = 8$
3...2	ETR_PSC[1:0]	Асинхронный предделитель частоты со входа ETR: 00 – без деления; 01 – /2; 10 – /4; 11 – /8
1	ETR_INV	Инверсия входа ETR: 0 – без инверсии; 1 – инверсия
0	BRK_INV	Инверсия входа BRK: 0 – без инверсии; 1 – инверсия

23.9.12 MDR_TIMERx->STATUS

Таблица 285 – Регистр статуса таймера STATUS

Номер	31...17	16...13	12...9	8...5
Доступ	U	R/W	R/W	R/W
Сброс	0	0	0	0
	-	CCR CAP1 EVENT[3:0]	CCR REF EVENT[3:0]	CCR CAP EVENT[3:0]

Номер	4	3	2	1	0
Доступ	R/W	R/W	R/W	R/W	R/W
Сброс	0	0	0	0	0
	BRK EVENT	ETR FE EVENT	ETR RE EVENT	CNT ARR EVENT	CNT ZERO EVENT

Таблица 286 – Описание бит регистра STATUS

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...17	-	Зарезервировано
16...13	CCR CAP1 EVENT[3:0]	Событие записи значения счетчика CNT в регистр CCRy1 по захвату настроенного фронта на входе канала CHy1: 0 – нет события; 1 – есть событие. Сбрасывается записью «0», если запись происходит одновременно с новым событием, приоритет у нового события. Бит 0 – первый канал; Бит 3 – четвертый канал
12...9	CCR REF EVENT[3:0]	Событие переднего фронта на выходе генератора опорного сигнала REF: 0 – нет события; 1 – есть событие. Сбрасывается записью «0», если запись происходит одновременно с новым событием, приоритет у нового события. Бит 0 – первый канал; Бит 3 – четвертый канал
8...5	CCR CAP EVENT[3:0]	Событие записи значения счетчика CNT в регистр CCRy по захвату настроенного фронта на входе канала CHy: 0 – нет события; 1 – есть событие. Сбрасывается записью «0», если запись происходит одновременно с новым событием, приоритет у нового события. Бит 0 – первый канал; Бит 3 – четвертый канал
4	BRK EVENT	Событие высокого уровня на входе BRK: 0 – нет события; 1 – есть событие. Сбрасывается записью «0», при условии наличия низкого уровня на входе BRK
3	ETR FE EVENT	Событие заднего фронта на входе ETR: 0 – нет события; 1 – есть событие. Сбрасывается записью «0», если запись происходит одновременно с новым событием, приоритет у нового события
2	ETR RE EVENT	Событие переднего фронта на входе ETR: 0 – нет события; 1 – есть событие. Сбрасывается записью «0», если запись происходит одновременно с новым событием, приоритет у нового события
1	CNT ARR EVENT	Событие совпадения CNT с ARR: 0 – нет события; 1 – есть событие. Сбрасывается записью «0», если запись происходит одновременно с новым событием совпадения, приоритет у нового события. Если с момента совпадения до момента программного сброса флага регистра CNT и ARR не изменили состояния, то флаг повторно не взводится

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
0	CNT ZERO EVENT	Событие совпадения CNT с нулем: 0 – нет события; 1 – есть событие. Сбрасывается записью «0», если запись происходит одновременно с новым событием совпадения, приоритет у нового события. Если с момента совпадения до момента программного сброса флага регистр CNT не изменил состояния, то флаг повторно не взводится
Примечание – Для организации надежного сброса статусных флагов прерываний необходимо учитывать требования, изложенные в подразделе 31.3		

23.9.13 MDR_TIMERx->IE

Таблица 287 – Регистр разрешения прерываний таймера IE

Номер	31...17	16...13	12...9	8...5
Доступ	U	R/W	R/W	R/W
Сброс	0	0	0	0
	-	CCR CAP1 EVENT IE[3:0]	CCR REF EVENT IE[3:0]	CCR CAP EVENT IE[3:0]

Номер	4	3	2	1	0
Доступ	R/W	R/W	R/W	R/W	R/W
Сброс	0	0	0	0	0
	BRK EVENT IE	ETR FE EVENT IE	ETR RE EVENT IE	CNT ARR EVENT IE	CNT ZERO EVENT IE

Таблица 288 – Описание бит регистра IE

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...17	-	Зарезервировано
16...13	CCR CAP1 EVENT IE[3:0]	Флаг разрешения прерывания по событию CCR CAP1 EVENT[3:0] в регистре STATUS: 0 – прерывание запрещено; 1 – прерывание разрешено. Бит 0 – первый канал; Бит 3 – четвертый канал
12...9	CCR REF EVENT IE[3:0]	Флаг разрешения прерывания по событию CCR REF EVENT[3:0] в регистре STATUS: 0 – прерывание запрещено; 1 – прерывание разрешено. Бит 0 – первый канал; Бит 3 – четвертый канал

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
8...5	CCR CAP EVENT IE[3:0]	Флаг разрешения прерывания по событию CCR CAP EVENT[3:0] в регистре STATUS: 0 – прерывание запрещено; 1 – прерывание разрешено. Бит 0 – первый канал; Бит 3 – четвертый канал
4	BRK EVENT IE	Флаг разрешения прерывания по событию BRK EVENT в регистре STATUS: 0 – прерывание запрещено; 1 – прерывание разрешено
3	ETR FE EVENT IE	Флаг разрешения прерывания по событию ETR FE EVENT в регистре STATUS: 0 – прерывание запрещено; 1 – прерывание разрешено
2	ETR RE EVENT IE	Флаг разрешения прерывания по событию ETR RE EVENT в регистре STATUS: 0 – прерывание запрещено; 1 – прерывание разрешено
1	CNT ARR EVENT IE	Флаг разрешения прерывания по событию CNT ARR EVENT в регистре STATUS: 0 – прерывание запрещено; 1 – прерывание разрешено
0	CNT ZERO EVENT IE	Флаг разрешения прерывания по событию CNT ZERO EVENT в регистре STATUS: 0 – прерывание запрещено; 1 – прерывание разрешено

23.9.14 MDR_TIMERx->DMA_RE

Таблица 289 – Регистр DMA_RE разрешения запросов DMA

Номер	31...17	16...13	12...9	8...5
Доступ	U	R/W	R/W	R/W
Сброс	0	0	0	0
	-	CCR CAP1 EVENT RE[3:0]	CCR REF EVENT RE[3:0]	CCR CAP EVENT RE[3:0]

Номер	4	3	2	1	0
Доступ	R/W	R/W	R/W	R/W	R/W
Сброс	0	0	0	0	0
	BRK EVENT RE	ETR FE EVENT RE	ETR RE EVENT RE	CNT ARR EVENT RE	CNT ZERO EVENT RE

Таблица 290 – Описание бит регистра DMA_RE

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...17	-	Зарезервировано

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
16...13	CCR CAP1 EVENT RE [3:0]	Флаг разрешения запроса DMA по событию CCR CAP1 EVENT[3:0]: 0 – запрос DMA запрещен; 1 – запрос DMA разрешен. Бит 0 – первый канал; Бит 3 – четвертый канал
12...9	CCR REF EVENT RE[3:0]	Флаг разрешения запроса DMA по событию CCR REF EVENT[3:0]: 0 – запрос DMA запрещен; 1 – запрос DMA разрешен. Бит 0 – первый канал; Бит 3 – четвертый канал
8...5	CCR CAP EVENT RE [3:0]	Флаг разрешения запроса DMA по событию CCR CAP EVENT[3:0]: 0 – запрос DMA запрещен; 1 – запрос DMA разрешен. Бит 0 – первый канал; Бит 3 – четвертый канал
4	BRK EVENT RE	Флаг разрешения запроса DMA по событию BRK EVENT: 0 – запрос DMA запрещен; 1 – запрос DMA разрешен
3	ETR FE EVENT RE	Флаг разрешения запроса DMA по событию ETR FE EVENT: 0 – запрос DMA запрещен; 1 – запрос DMA разрешен
2	ETR RE EVENT RE	Флаг разрешения запроса DMA по событию ETR RE EVENT: 0 – запрос DMA запрещен; 1 – запрос DMA разрешен
1	CNT ARR EVENT RE	Флаг разрешения запроса DMA по событию CNT ARR EVENT: 0 – запрос DMA запрещен; 1 – запрос DMA разрешен
0	CNT ZERO EVENT RE	Флаг разрешения запроса DMA по событию CNT ZERO EVENT: 0 – запрос DMA запрещен; 1 – запрос DMA разрешен

24 Контроллер MDR_ADC

В микросхеме реализовано два 12-разрядных АЦП (см. рисунок 86). С помощью АЦП можно оцифровать сигнал от 16 внешних аналоговых выводов порта D и от двух внутренних каналов, на которые выводятся датчик температуры и источник опорного напряжения. Скорость выборки составляет до 512 тысяч преобразований в секунду для каждого АЦП.

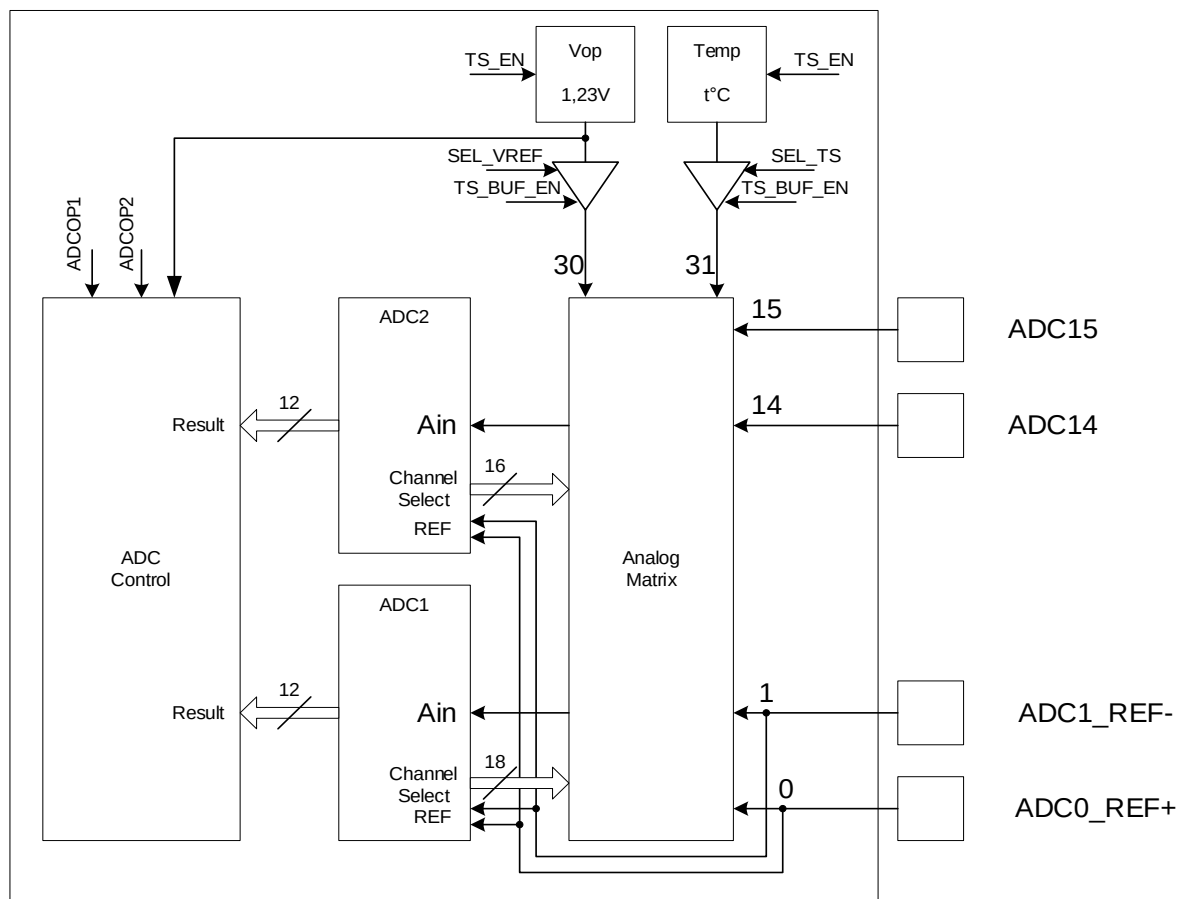


Рисунок 86 – Структурная схема контроллера АЦП

В качестве опорного напряжения преобразования могут выступать:

- питание АЦП с выводов AU_{CC} и AGND;
- внешние сигналы с выводов ADC0_REF+ и ADC1_REF-.

Контроллер АЦП позволяет:

- оцифровать один из 16 внешних каналов;
- оцифровать значение встроенного датчика температуры;
- оцифровать значение внутреннего источника опорного напряжения;
- осуществить автоматический опрос заданных каналов;
- выработать прерывание при выходе оцифрованного значения за заданные пределы;
- запускать два АЦП синхронно для увеличения скорости выборки.

Для осуществления преобразования требуется 28 тактов синхронизации C_ADC. В качестве синхросигнала может выступать частота процессора PCLKd или частота ADC_CLK, формируемая в блоке Сигналы тактовой частоты. Выбор частоты

осуществляется с помощью бита Cfg_REG_CLKS. Для получения частоты PCLKd в контроллере АЦП частота PCLK может быть поделена с помощью битов Cfg_REG_DIVCLK[3:0].

Для включения АЦП необходимо установить бит Cfg_REG_ADON. Для снижения тока потребления вместо собственного источника опорного напряжения в АЦП может использоваться источник опорного напряжения датчика температуры. Для этого необходимо включить блок датчика температуры и источник опорного напряжения, установив бит TS_EN в «1». После включения можно использовать источник опорного напряжения для первого и второго АЦП вместо их собственных. Для этого необходимо установить биты ADCx_OR в «1». Для преобразования необходимо, чтобы выводы, используемые АЦП у порта D, были сконфигурированы как аналоговые и были отключены какие-либо внутренние подтяжки.

24.1 Преобразование внешнего канала

В регистре ADCx_CFG в битах Cfg_REG_CHS[4:0] необходимо задать соответствующий выводу номер канала. Преобразование может осуществляться при внутренней опоре бит Cfg_M_REF = 0 и внешней Cfg_M_REF = 1, в этом случае опора берется с выводов ADC0_REF+ и ADC1_REF-. Биты Cfg_REG_CHCH, Cfg_REG_RNGC, Cfg_REG_SAMPLE, TS_BUF_EN, SEL_VREF, SEL_TS и Cfg_Sync_Conver должны быть сброшены.

Для начала преобразования необходимо записать «1» в бит Cfg_REG_GO.

После завершения преобразования будет взведен бит Flg_REG_EOCIF в регистре ADCx_STATUS, а в регистре ADCx_RESULT будет результат преобразования.

После считывания результата бит Flg_REG_EOCIF сбросится.

Если после первого преобразования результат не был считан, и было выполнено второе преобразование, то в регистре результата ADCx_RESULT будет значение от последнего преобразования, и помимо бита Flg_REG_EOCIF будет взведен бит Flg_REG_OVERWRITE. Флаг Flg_REG_OVERWRITE может быть сброшен только записью в регистр ADCx_STATUS.

24.2 Последовательное преобразование нескольких каналов

Для автоматического последовательного преобразования нескольких каналов или одного канала в регистре ADCx_CHSEL необходимо установить единицы в битах, соответствующих выбранным для преобразования каналам. Выставление данных бит необходимо обеспечить до установки конфигурации АЦП, то есть до записи в регистр ADCx_CFG. Преобразование может осуществляться при внутренней опоре бит Cfg_M_REF = 0 и внешней Cfg_M_REF = 1. В этом случае опора берется с выводов ADC0_REF+ и ADC1_REF-. Биты Cfg_REG_RNGC, TS_BUF_EN, SEL_VREF, SEL_TS и Cfg_Sync_Conver должны быть сброшены, а Cfg_REG_SAMPLE и Cfg_REG_CHCH должны быть установлены. С помощью бит Delay_GO можно задать паузу между преобразованиями при переборе каналов. Пауза определяется в тактах PCLKd, независимо от того на какой частоте ADC_CLK или PCLKd идет само преобразование. Для начала преобразования необходимо записать 1 в бит Cfg_REG_GO.

После завершения преобразования будет взведен бит Flg_REG_EOCIF в регистре ADCx_STATUS, а в регистре ADCx_RESULT будет результат преобразования.

После считывания результата бит Flg_REG_EOCIF сбросится.

Если после первого преобразования результат не был считан, и было выполнено второе преобразование, то в регистре результата ADCx_RESULT будет значение от последнего преобразования, и помимо бита Flg_REG_EOCIF будет взведен бит Flg_REG_OVERWRITE. Флаг Flg_REG_OVERWRITE может быть сброшен только записью в регистр ADCx_STATUS.

Для последовательного преобразования одного и того же канала можно в регистре ADCx_CHSEL выбрать только один канал и установить бит Cfg_REG_CHCH в 1, либо установить номер канала в битах Cfg_REG_CHS[4:0] и сбросить бит Cfg_REG_CHCH в 0. В этом случае процесс последовательного преобразования будет выполняться только для данного канала. Последовательное преобразование значения датчика температуры и источника опорного напряжения могут выполняться только в режиме последовательного преобразования одного канала.

24.3 Преобразование с контролем границ

При необходимости отслеживать нахождение оцифрованных значений в допустимых пределах можно задать нижнюю и верхнюю допустимые границы в регистрах ADCx_L_LEVEL и ADCx_H_LEVEL. При этом если установлен бит Cfg_REG_RNGC, то в случае, если результат преобразования выходит за границы, выставляется флаг Flg_REG_AWOIFEN, а в регистре результата будет полученное значение.

24.4 Внутренний источник опорного напряжения

С помощью первого АЦП можно осуществить преобразования внутреннего источника опорного напряжения (блок V_{OP} на рисунке 86). Для этого необходимо включить блок датчика температуры и источник опорного напряжения, установив бит TS_EN в «1». После включения можно использовать источник опорного напряжения для первого и второго АЦП вместо их собственных, что позволяет снизить ток потребления. Для этого необходимо установить биты ADCx_OP в «1». Для выбора источника опорного напряжения в качестве источника для преобразования необходимо в битах Cfg_REG_CHS установить значение 30 канала, установить биты TS_BUF_EN и SEL_VREF, после чего можно запустить процесс преобразования. Для запуска преобразования необходимо записать «1» в бит Cfg_REG_GO.

После завершения преобразования будет взведен бит Flg_REG_EOCIF в регистре ADC1_STATUS, а в регистре ADC1_RESULT будет результат преобразования.

После считывания результата бит Flg_REG_EOCIF сбросится.

Если после первого преобразования результат не был считан, и было выполнено второе преобразование, то в регистре результата ADC1_RESULT будет значение от последнего преобразования, а помимо бита Flg_REG_EOCIF будет взведен бит Flg_REG_OVERWRITE. Флаг Flg_REG_OVERWRITE может быть сброшен только записью в регистр ADC1_STATUS.

Для последовательного преобразования только источника опорного напряжения можно в регистре ADC1_CHSEL выбрать только 30 канал и установить бит Cfg_REG_CHCH в «1», либо установить номер 30 канала в битах Cfg_REG_CHS[4:0] и сбросить бит Cfg_REG_CHCH в 0. В этом случае процесс последовательного преобразования будет выполняться только для данного канала. При этом должны быть также установлены биты TS_BUF_EN и SEL_VREF.

Источник опорного напряжения может быть выбран для более точного результата преобразования АЦП и не может быть использован для задания опорного напряжения преобразования.

24.5 Датчик температуры

С помощью первого АЦП можно осуществить преобразования датчика опорного напряжения. Для этого необходимо включить блок датчика температуры и источник опорного напряжения, установив бит TS_EN в «1». После включения можно использовать источник опорного напряжения для первого и второго АЦП вместо их собственных, что позволяет снизить ток потребления. Для этого необходимо установить биты ADCx_OR в «1». Для выбора датчика температуры в качестве источника для преобразования необходимо в битах Cfg_REG_CHS установить значение 31 канала, установить биты TS_BUF_EN и SEL_TS, после чего можно запустить процесс преобразования. Для начала преобразования необходимо записать «1» в бит Cfg_REG_GO.

После завершения преобразования будет взведен бит Flg_REG_EOCIF в регистре ADC1_STATUS, а в регистре ADC1_RESULT будет результат преобразования.

После считывания результата бит Flg_REG_EOCIF сбросится.

Если после первого преобразования результат не был считан, и было выполнено второе преобразование, то в регистре результата ADC1_RESULT будет значение от последнего преобразования, и помимо бита Flg_REG_EOCIF будет взведен бит Flg_REG_OVERWRITE. Флаг Flg_REG_OVERWRITE может быть сброшен только записью в регистр ADC1_STATUS.

Для последовательного преобразования только датчика температуры можно в регистре ADC1_CHSEL выбрать только 31 канал и установить бит Cfg_REG_CHCH в 1, либо установить номер 31 канала в битах Cfg_REG_CHS[4:0] и сбросить бит Cfg_REG_CHCH в «0». В этом случае процесс последовательного преобразования будет выполняться только для данного канала. При этом должны быть также установлены биты TS_BUF_EN и SEL_TS.

Параметры температурного датчика не регламентируются. В зависимости от необходимой точности может быть достаточно провести градуировку в двух – трех точках. При необходимости более точных измерений необходимо построить градуировочную таблицу. Градуировка производится индивидуально для каждой микросхемы.

24.6 Синхронный запуск двух АЦП

Для ускорения оцифровки одного канала можно использовать оба АЦП, запускаемые с задержкой одного относительно другого по времени. Время задержки

запуска второго АЦП относительно первого задается битами Delay_ADC. При этом задержка Delay_ADC определяется в тактах PCLKd, независимо от того на какой частоте ADC_CLK или PCLKd идет само преобразование. Для одновременного запуска процесса преобразования необходимо установить бит Cfg_Sync_Conver и запустить процесс преобразования установкой бита Cfg_REG_GO. Синхронный запуск двух АЦП может работать также и в режиме последовательного преобразования нескольких каналов.

24.7 Время заряда внутренней емкости

Процесс преобразования состоит из двух этапов: сначала происходит заряд внутренней емкости до уровня внешнего сигнала, и затем происходит преобразование уровня заряда внутренней емкости в цифровой вид. Таким образом, для точного преобразования внешнего сигнала в цифровой вид, за время первого этапа внутренняя емкость должна зарядиться до уровня внешнего сигнала. Это время определяется соотношением номинальной внутренней емкости, входным сопротивлением тракта АЦП и выходным сопротивлением источника сигнала. Максимальное выходное сопротивление источника R_{AIN} для обеспечения качественного преобразования определяется по формулам (10), (11)

$$R_{AIN} < \frac{T_{track}}{C_{ADC} \cdot \ln(2^N)} - R_{ADC}, \quad (10)$$

$$T_{track} = 4 \cdot T_{C_ADC} + N_{PCLKd} \cdot T_{PCLKd} = \frac{4}{f_{C_ADC}} + \frac{(DelayGo + 1)}{f_{PCLKd}}, \quad (11)$$

где C_{ADC} – внутренняя емкость АЦП (~15 – 20 пФ);
 N – требуемая точность, в разрядах;
 R_{ADC} – входное сопротивление тракта АЦП (~500 Ом);
 T_{track} – время заряда внутренней емкости (определяется формулой (11)), с;
 f_{C_ADC} – рабочая частота АЦП (определяется Cfg REG CLKS в регистре ADC1_CFG), с⁻¹;
 f_{PCLKd} – определяется по формуле (12)

$$f_{PCLKd} = \frac{f_{PCLK}}{2^{Cfg_REG_DIVCLK}}. \quad (12)$$

Если необходимо обеспечить преобразование с точностью 12 разрядов $\pm 1/4$ LSB, то $N = 14$. Если необходимо обеспечить преобразование с точностью 10 разрядов ± 1 LSB, то $N = 10$. Время заряда T_{track} определяется битами DelayGo[2:0] и схемой самого АЦП и представлено в таблице 291.

Таблица 291 – Время заряда внутренней емкости АЦП и время преобразования

DelayGo[2:0]	Дополнительная задержка перед началом преобразования	Общее время T_{track} заряда емкости АЦП перед началом преобразования	Общее время преобразования АЦП
000	1xPCLKd	4xC_ADC+1xPCLKd	28xC_ADC+1xPCLKd
001	2xPCLKd	4xC_ADC+2xPCLKd	28xC_ADC+2xPCLKd
010	3xPCLKd	4xC_ADC+3xPCLKd	28xC_ADC+3xPCLKd
011	4xPCLKd	4xC_ADC+4xPCLKd	28xC_ADC+4xPCLKd
100	5xPCLKd	4xC_ADC+5xPCLKd	28xC_ADC+5xPCLKd
101	6xPCLKd	4xC_ADC+6xPCLKd	28xC_ADC+6xPCLKd
110	7xPCLKd	4xC_ADC+7xPCLKd	28xC_ADC+7xPCLKd
111	8xPCLKd	4xC_ADC+8xPCLKd	28xC_ADC+8xPCLKd

Помимо точности, определяемой временем зарядки внутренней емкости АЦП, точность преобразования имеет ошибки, связанные с технологическими разбросами схемы и шумами и определяемые параметрами E_{DLADC} , E_{ILADC} и E_{OFFADC} .

Задание режимов работы АЦП в регистре ADCx_CFG необходимо производить до задания бита Cfg_REG_GO, иначе новая конфигурация будет действовать со следующего преобразования.

24.8 Описание регистров блока контроллера АЦП

Таблица 292 – Описание регистров блока контроллера АЦП

Базовый адрес	Название	Описание
0x4008_8000	MDR_ADC	Контроллер ADC
Смещение		
0x00	MDR_ADC->ADC1_CFG	Регистр управления ADC1
0x04	MDR_ADC->ADC2_CFG	Регистр управления ADC2
0x08	ADC1_H_LEVEL	Регистр MDR_ADC->ADCx_H_LEVEL верхней границы ADC1
0x0C	ADC2_H_LEVEL	Регистр MDR_ADC->ADCx_H_LEVEL верхней границы ADC2
0x10	ADC1_L_LEVEL	Регистр MDR_ADC->ADCx_L_LEVEL нижней границы ADC1
0x14	ADC2_L_LEVEL	Регистр MDR_ADC->ADCx_L_LEVEL нижней границы ADC2
0x18	ADC1_RESULT	Регистр MDR_ADC->ADCx_RESULT результата ADC1
0x1C	ADC2_RESULT	Регистр MDR_ADC->ADCx_RESULT результата ADC2
0x20	ADC1_STATUS	Регистр MDR_ADC->ADCx_STATUS статуса ADC1
0x24	ADC2_STATUS	Регистр MDR_ADC->ADCx_STATUS статуса ADC2
0x28	ADC1_CHSEL	Регистр MDR_ADC->ADCx_CHSEL выбора каналов перебора ADC1
0x2C	ADC2_CHSEL	Регистр MDR_ADC->ADCx_CHSEL выбора каналов перебора ADC2

24.8.1 MDR_ADC->ADC1_CFG

Таблица 293 – Регистр ADC1_CFG

Номер	11	10	9	8...4	3	2	1	0
Доступ	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Сброс	0	0	0	0	0	0	0	0
	Cfg M_REF	Cfg REG RNGC	Cfg REG CHCH	Cfg REG CHS[4:0]	Cfg REG SAMPLE	Cfg REG CLKS	Cfg REG GO	Cfg REG ADON

Номер	31...28	27...25	24...21	20	19	18	17	16	15...12
Доступ	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Сброс	0	0	0	0	0	0	0	0	0
	Delay ADC [3:0]	Delay Go [2:0]	TR [3:0]	SEL VREF	SEL TS	TS_BUF EN	TS_EN	Cfg Sync Conver	Cfg REG DIVCLK [3:0]

Таблица 294 – Описание бит регистра ADC1_CFG

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...28	Delay ADC [3:0]	Задержка между началом преобразования ADC1 и ADC2 при последовательном переборе либо работе на один канал: 0000 – 1 такт PCLKd; 0001 – 2 такта PCLKd; ... 1111 – 16 тактов PCLKd
27...25	Delay Go [2:0]	Дополнительная задержка перед началом преобразования после выбора канала: 000 – 1 такт PCLKd; 001 – 2 такта PCLKd; ... 111 – 8 тактов PCLKd
24...21	TR[3:0]	Подстройка опорного напряжения. См. диаграмму на рисунке 87
20	SEL VREF	Выбор для оцифровки внутреннего источника опорного напряжения 1,23 В: 0 – не выбран; 1 – выбран. Должен использоваться совместно с выбором канала Cfg_REG_CHS = 30
19	SEL TS	Выбор для оцифровки датчика температуры: 0 – не выбран; 1 – выбран. Должен использоваться совместно с выбором канала Cfg_REG_CHS = 31
18	TS BUF EN	Включения выходного усилителя для датчика температуры и источника опорного напряжения: 0 – выключен; 1 – включен. Используется при TS_EN = 1

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
17	TS EN	Включения датчика температуры и источника опорного напряжения: 0 – выключен; 1 – включен. При включении датчика температуры и источника опорного напряжения выходной сигнал стабилизируется в течение времени 1 мс
16	Cfg Sync Conver	Запускает работу двух АЦП одновременно, при этом биты конфигурации второго АЦП, такие как Cfg_REG_DIVCLK, Cfg_REG_ADON, Cfg_M_REF и Cfg_REG_CHS берутся из регистра конфигурации первого: 0 – независимые АЦП; 1 – синхронные АЦП
15...12	Cfg REG DIVCLK [3:0]	Выбор коэффициента деления частоты процессора: 0000 – PCLKd = PCLK; 0001 – PCLKd = PCLK /2; 0010 – PCLKd = PCLK /4; 0011 – PCLKd = PCLK/8; ... 1011 – PCLKd = PCLK/2048; Остальные комбинации – PCLKd = PCLK
11	Cfg M_REF	Выбор источника опорных напряжений: 0 – внутреннее опорное напряжение (от AU _{CC} и AGND); 1 – внешнее опорное напряжение (от ADC0_REF+ и ADC1_REF-)
10	Cfg REG RNGC	Разрешение автоматического контроля уровней: 0 – не разрешена; 1 – разрешена выработка флага при выходе за диапазон в регистрах границы
9	Cfg REG CHCH	Выбор переключения каналов: 0 – используется только выбранный канал; 1 – переключение включено (перебираются каналы, выбранные в регистре выбора канала)
8...4	Cfg REG CHS [4:0]	Выбор аналогового канала, по которому поступает сигнал для преобразования: 00000 – 0 канал; 00001 – 1 канал; ... 11111 – 31 канал
3	Cfg REG SAMPLE	Выбор способа запуска АЦП: 0 – одиночное; 1 – последовательное. Автоматический запуск после завершения предыдущего преобразования
2	Cfg REG CLKS	Выбор источника синхросигнала C_ADC работы ADC: 0 – PCLKd (определяется по формуле (12)); 1 – ADC_CLK
1	Cfg REG GO	Начало преобразования. Запись «1» начинает процесс преобразования, сбрасывается автоматически
0	Cfg REG ADON	Включение АЦП: 0 – выключено; 1 – включено

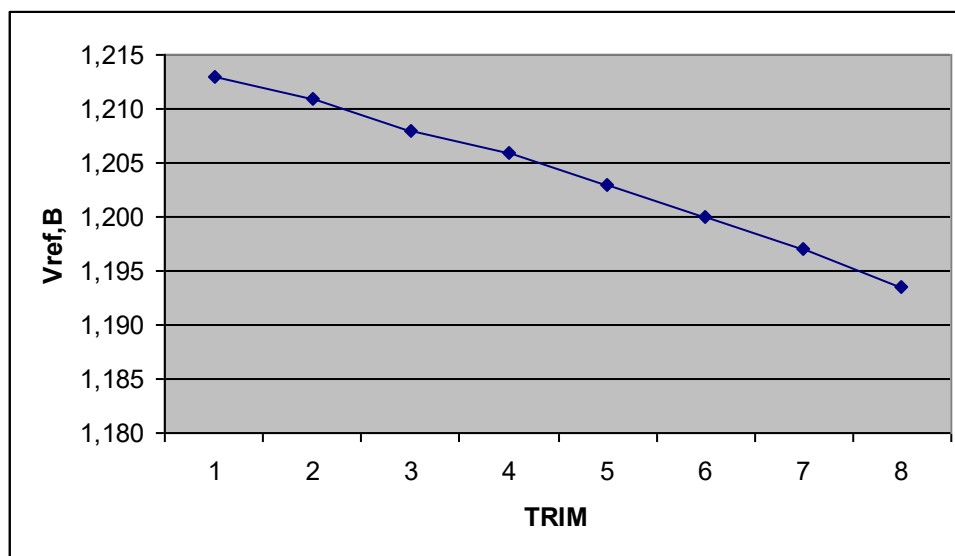


Рисунок 87 – Зависимость источника опорного напряжения от подстройки

24.8.2 MDR_ADC->ADC2_CFG

Таблица 295 – Регистр ADC2_CFG

Номер	31...28	27...25	24...19	18	17	16	15...12
Доступ	U	R/W	U	R/W	R/W	U	R/W
Сброс	0	0	0	0	0	0	0
	-	Delay Go [2:0]	-	ADC2 OP	ADC1 OP	-	Cfg REG DIVCLK [3:0]

Номер	11	10	9	8...4	3	2	1	0
Доступ	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Сброс	0	0	0	0	0	0	0	0
	Cfg M_REF	Cfg REG RNGC	Cfg REG CHCH	Cfg REG CHS[4:0]	Cfg REG SAMPLE	Cfg REG CLKS	Cfg REG GO	Cfg REG ADON

Таблица 296 – Описание бит регистра ADC2_CFG

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...28	-	Зарезервировано
27...25	Delay Go [2:0]	Задержка перед началом следующего преобразования после завершения предыдущего при последовательном переборе каналов: 000 – 1 такт PCLKd 001 – 2 такта PCLKd ... 111 – 8 тактов PCLKd
24...19	-	Зарезервировано

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
18	ADC2 OP	Выбор источника опорного напряжения: 0 – внутренний (неточный); 1 – от датчика температуры (точный). Необходимо, чтобы бит TS EN был равен 1
17	ADC1 OP	Выбор источника опорного напряжения: 0 – внутренний (неточный); 1 – от датчика температуры (точный). Необходимо, чтобы бит TS EN был равен 1
16	-	Зарезервировано
15...12	Cfg REG DIVCLK [3:0]	Выбор коэффициента деления частоты процессора: 0000 – PCLKd = PCLK; 0001 – PCLKd = PCLK /2; 0010 – PCLKd = PCLK /4; 0011 – PCLKd = PCLK/8; ... 1011 – PCLKd = PCLK/2048; Остальные комбинации – PCLKd = PCLK
11	Cfg M_REF	Выбор источника опорных напряжений: 0 – внутреннее опорное напряжение (от AU _{CC} и AGND); 1 – внешнее опорное напряжение (от ADC0_REF+ и ADC1_REF-)
10	Cfg REG RNGC	Разрешение автоматического контролирования уровней: 1 – разрешено, выработка прерывания при выходе за диапазон в регистрах границы обработки; 0 – не разрешено
9	Cfg REG CHCH	Выбор переключения каналов: 0 – используется только выбранный канал; 1 – переключение включено (перебираются каналы, выбранные в регистре выбора канала)
8...4	Cfg REG CHS [4:0]	Выбор аналогового канала, по которому поступает сигнал для преобразования: 00000 – 0 канал; 00001 – 1 канал; ... 11111 – 31 канал
3	Cfg REG SAMPLE	Выбор способа запуска АЦП: 0 – одиночное; 1 – последовательное. Автоматический запуск после завершения предыдущего преобразования.
2	Cfg REG CLKS	Выбор источника синхросигнала C_ADC работы ADC: 0 – PCLKd (определяется по формуле (12)); 1 – ADC_CLK
1	Cfg REG GO	Начало преобразования. Запись «1» начинает процесс преобразования. Сбрасывается автоматически
0	Cfg REG ADON	Включение АЦП: 0 – выключено; 1 – включено

24.8.3 MDR_ADC->ADCx_H_LEVEL

Таблица 297 – Регистр ADCx_H_LEVEL

Номер	31...12	11...0
Доступ	U	R/W
Сброс	0	0
	-	REG H LEVEL [11:0]

Таблица 298 – Описание бит регистра ADCx_H_LEVEL

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...12	-	Зарезервировано
11...0	REG H LEVEL [11:0]	Верхняя граница зоны допуска.

24.8.4 MDR_ADC->ADCx_L_LEVEL

Таблица 299 – Регистр ADCx_L_LEVEL

Номер	31...12	11...0
Доступ	U	R/W
Сброс	0	0
	-	REG L LEVEL [11:0]

Таблица 300 – Описание бит регистра ADCx_L_LEVEL

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...12	-	Зарезервировано
11...0	REG L LEVEL [11:0]	Нижняя граница зоны допуска

24.8.5 MDR_ADC->ADCx_RESULT

Таблица 301 – Регистр ADCx_RESULT

Номер	31...21	20...16	15...12	11...0
Доступ	U	RO	U	RO
Сброс	0	0	0	0
	-	CHANNEL [4:0]	-	RESULT[11:0]

Таблица 302 – Описание бит регистра ADCx_RESULT

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...21	-	Зарезервировано
20...16	CHANNEL [4:0]	Канал результата преобразования
15...12	-	Зарезервировано
11...0	RESULT [11:0]	Значение результата преобразования

24.8.6 MDR_ADC->ADCx_STATUS

Таблица 303 – Регистр ADCx_STATUS

Номер	31...5	4	3	2	1	0
Доступ	U	R/W	R/W	R/W	R/W	R/W
Сброс	0	0	0	0	0	0
	-	EOCIF IE	AWOIFIE	Flg REG EOCIF	Flg REG AWOIFEN	Flg REG OVERWRITE

Таблица 304 – Описание бит регистра ADCx_STATUS

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...5	-	Зарезервировано
4	EOCIF_IE	Флаг разрешения генерирования прерывания по событию Flg_REG_EOCIF: 0 – прерывания не генерируется; 1 – прерывание генерируется
3	AWOIF_IE	Флаг разрешения генерирования прерывания по событию Flg_REG_AWOIFEN: 0 – прерывания не генерируется; 1 – прерывание генерируется
2	Flg REG EOCIF	Флаг выставляется, когда закончено преобразования и данные еще не считаны. Очищается считыванием результата из регистра ADCx_RESULT: 1 – есть готовый результат преобразования; 0 – нет результата
1	Flg REG AWOIFEN	Флаг выставляется, когда результат преобразования выше верхней или ниже нижней границы автоматического контролирования уровней. Сбрасывается только при записи нуля в данный бит регистр флагов: 0 – результат в допустимой зоне; 1 – вне допустимой зоны
0	Flg REG OVERWRITE	Данные в регистре результата были перезаписаны, данный флаг сбрасывается только при записи нуля в данный бит регистр флагов: 0 – не было события перезаписи несчитанного результата; 1 – был результат преобразования, который не был считан
Примечание – Для организации надежного сброса статусных флагов прерываний необходимо учитывать требования, изложенные в подразделе 31.3		

24.8.7 MDR_ADC->ADCx_CHSEL

Таблица 305 – Регистр ADCx_CHSEL

Номер	31... 0
Доступ	R/W
Сброс	0
	Sl_Ch_Ch_REF[31:0]

Таблица 306 – Описание бит регистра ADCx_CHSEL

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...0	SI_Ch_Ch_REF[31:0]	Выбор каналов автоматического перебора: 0 – в соответствующем бите канал не участвует в переборе; 1 – канал участвует в переборе

25 Контроллер MDR_DAC

В микросхеме реализовано два ЦАП (см. рисунок 88). Для включения ЦАП необходимо установить бит Cfg_ON_DACx в 1, используемые выводы ЦАП порта E были сконфигурированы как аналоговые и были отключены какие-либо внутренние подтяжки. Оба ЦАП могут работать независимо или совместно. При независимой работе ЦАП (бит Cfg_SYNC_A = 0) после записи данных в регистр данных DACx_DATA на выходе DACx_OUT формируется уровень напряжения, соответствующий записанному значению. При синхронной работе (бит Cfg_SYNC_A = 1) данные обоих ЦАП могут быть обновлены одной записью в один из регистров DACx_DATA. ЦАП может работать от внутренней опоры Cfg_M_REFx = 0, тогда ЦАП формирует выходной сигнал в диапазоне от 0 до напряжения питания AU_{CC} . В режиме работы с внешней опорой Cfg_M_REFx = 1 ЦАП формирует выходное напряжение в диапазоне от 0 до значения DACx_REF.

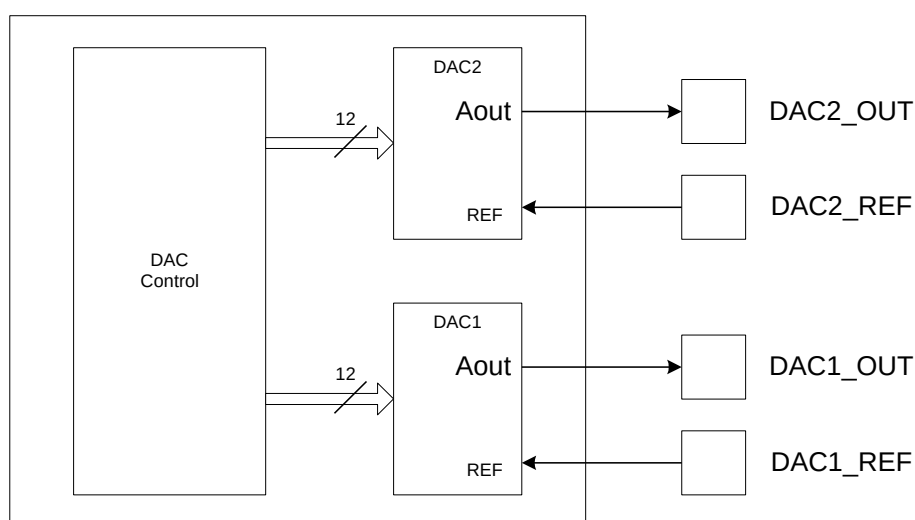


Рисунок 88 – Структурная схема контроллера ЦАП

25.1 Описание регистров блока контроллера ЦАП

Таблица 307 – Описание регистров блока контроллера ЦАП

Базовый Адрес	Название	Описание
0x4009_0000	MDR_DAC	Контроллер DAC
Смещение		
0x00	MDR_DAC->CFG	Регистр управления DAC
0x04	MDR_DAC->DAC1_DATA	Регистр данных DAC1
0x08	MDR_DAC->DAC2_DATA	Регистр данных DAC2

25.1.1 MDR_DAC->CFG

Таблица 308 – Регистр CFG

Номер	31...5	4	3	2	1	0
Доступ	U	R/W	R/W	R/W	R/W	R/W
Сброс	0	0	0	0	0	0
	-	Cfg SYNC_A	Cfg ON_DAC1	Cfg ON_DAC0	Cfg M_REF1	Cfg M_REF0

Таблица 309 – Описание бит регистра CFG

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...5	-	Зарезервировано
4	Cfg_SYNC_A	Синхронизация DAC1 и DAC2: 0 – асинхронные; 1 – синхронные
3	Cfg_ON_DAC1	Включение DAC2: 1 – включен; 0 – выключен
2	Cfg_ON_DAC0	Включение DAC1: 1 – включен; 0 – выключен
1	Cfg_M_REF1	Выбор источника опорного напряжения DAC2: 0 – в качестве опорного напряжения используется напряжение питания с вывода AU _{CC} ; 1 – в качестве опорного напряжения используется напряжение на входе опорного напряжения DAC2_REF
0	Cfg_M_REF0	Выбор источника опорного напряжения DAC1: 0 – в качестве опорного напряжения используется напряжение питания с вывода AU _{CC} ; 1 – в качестве опорного напряжения используется напряжение на входе опорного напряжения DAC1_REF

25.1.2 MDR_DAC->DAC1_DATA

Таблица 310 – Регистр DAC1_DATA

Номер	31...28	27...16	15...12	11...0
Доступ	U	R/W	U	R/W
Сброс	0	0	0	0
	-	DAC1_DATA[11:0]	-	DAC0_DATA[11:0]

Таблица 311 – Описание бит регистра DAC1_DATA

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...28	-	Зарезервировано
27...16	DAC1 DATA[11:0]	Данные DAC2 при Cfg_SYNC_A=1. При чтении всегда равны нулю. Читать из DAC2_DATA
15...12	-	Зарезервировано
11...0	DAC0 DATA[11:0]	Данные DAC1

25.1.3 MDR_DAC->DAC2_DATA

Таблица 312 – Регистр DAC2_DATA

Номер	31...28	27...16	15...12	11...0
Доступ	U	R/W	U	R/W
Сброс	0	0	0	0
	-	DAC0_DATA[11:0]	-	DAC1_DATA[11:0]

Таблица 313 – Описание бит регистра DAC2_DATA

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...28	-	Зарезервировано
27...16	DAC0 DATA[11:0]	Данные DAC1 при Cfg_SYNC_A=1. При чтении всегда равны нулю. Читать из DAC1_DATA
15...12	-	Зарезервировано
11...0	DAC1 DATA[11:0]	Данные DAC2

Примечание – Если бит конфигурации Cfg_SYNC_A установлен, то данные для DAC1 и DAC2 задаются записью в один из регистров DACx_DATA.

26 Контроллер схемы компаратора MDR_COMP

В микросхеме реализована схема компаратора, обеспечивающая следующие режимы работы:

- сравнение двух сигналов с трех различных выводов микросхемы;
- сравнение сигнала с трех различных выводов с внутренней шкалой напряжений;
- сравнение сигнала с вывода IN1 с внутренним источником опорного напряжения;
- формирование внутренней шкалы напряжений от питания микросхемы и от внешних выводов.

Для включения компаратора необходимо установить бит ON в 1, используемые выводы порта E должны быть сконфигурированы как аналоговые и должны быть отключены какие-либо внутренние подтяжки. После появления флага Ready компаратор готов к работе.

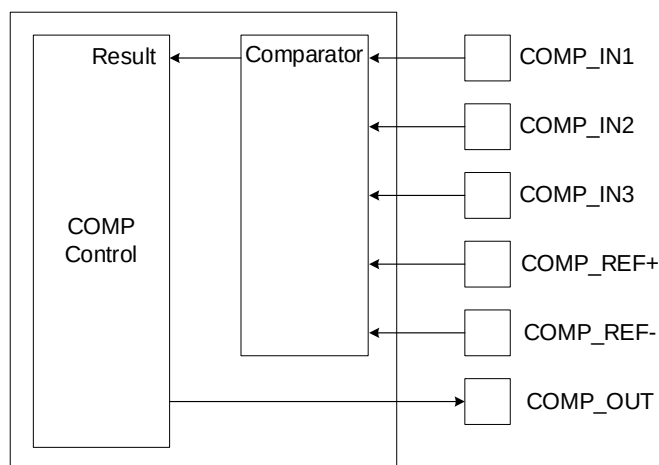
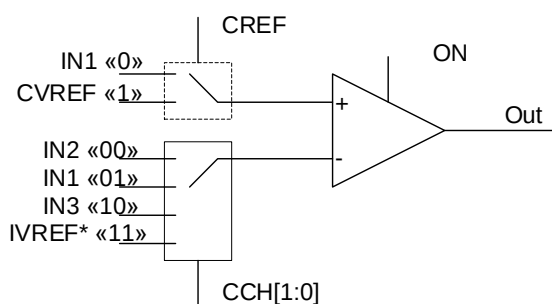


Рисунок 89 – Структура блока компаратора



*IVREF – выход внутреннего источника опорного напряжения 1,2 В

Рисунок 90 – Структура мультиплексирования входов компаратора

26.1 Сравнение внешних сигналов

Компаратор позволяет проводить сравнение двух сигналов, поступающих с трех выводов микросхемы. На вход «+» компаратора может быть подан сигнал IN1 (бит CREF=0), на вход «-» могут быть поданы сигналы IN1 (CCH=01), IN2 (CCH=00) и IN3 (CCH=10), при этом, если уровень на входе «+» будет больше уровня на входе «-», то выход Out установится в 1.

26.2 Сравнение сигнала с внутренним источником опорного напряжения

Компаратор позволяет проводить сравнение сигнала, поступающего с вывода IN1 микросхемы, с внутренним источником опорного напряжения IVREF. Для этого на вход «+» компаратора должен быть подан сигнал IN1 (бит CREF = 0), на вход «-» должен быть подан сигнал IVREF (CCH = 11), при этом, если уровень на входе «+» будет больше уровня на входе «-», то выход Out установится в 1.

26.3 Сравнение внешних сигналов с внутренней шкалой напряжений

Компаратор позволяет проводить сравнение внешних сигналов, поступающих с трех выводов микросхемы, со шкалой напряжений, формируемых внутри микросхемы. На вход «+» компаратора должен быть подан сигнал CVREF (бит CREF=1), на вход «-» могут быть поданы сигналы IN1 (CCH=01), IN2 (CCH=00) и IN3 (CCH=10), при этом, если уровень на входе «+» будет больше уровня на входе «-», то выход Out установится в 1.

26.4 Формирование внутренней шкалы напряжений

Внутренняя шкала напряжений формируется на резистивном делителе (см рисунок 91), который включается битом CVREN = 1.

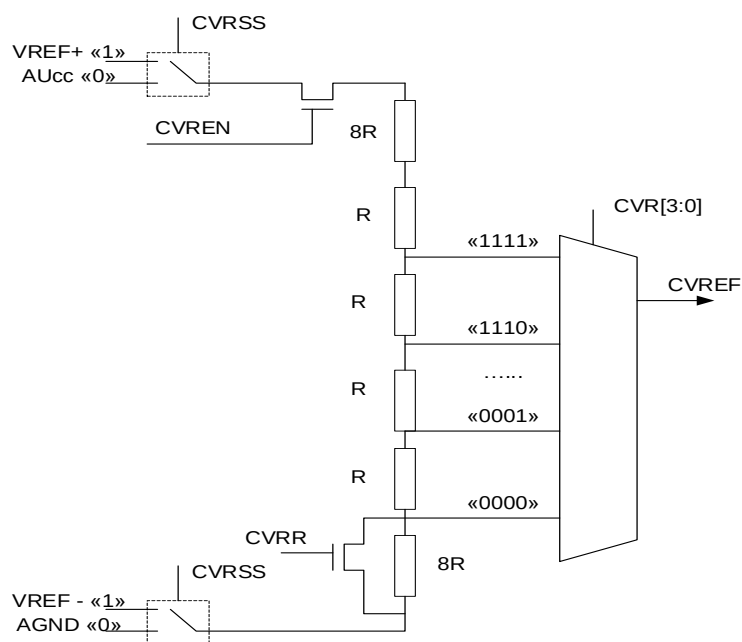


Рисунок 91 – Структура блока формирования CVREF

При этом в качестве опорного напряжения делителя может выступать питание микросхемы AU_{CC} ($CVRSS = 0$), либо напряжение на выводе $COMP_VREF+$ ($CVRSS = 1$). Нижнее опорное напряжение компаратора задается на выводе $COMP_VREF-$. Напряжение на выводе CVREF формируется на основании комбинации бит CVRR и CVR и приведены в таблице 314, как справочные данные. Реальные значения в конкретном кристалле могут отличаться за счет технологического разброса параметров.

Таблица 314 – Формирование внутренней шкалы напряжений CVREF

CVRR	CVR[3:0]	Отношение резисторов	Напряжение CVREF при $U_{CC}=3,3$ В, В	Входной импеданс VREF+, кОм	Примечание
0	0000	8/32	0,83	12	
	0001	9/32	0,93	13	
	0010	10/32	1,03	13,8	
	0011	11/32	1,13	14,4	
	0100	12/32	1,24	15	
	0101	13/32	1,34	15,4	
	0110	14/32	1,44	15,8	
	0111	15/32	1,55	15,9	
	1000	16/32	1,65	16	
	1001	17/32	1,75	15,9	
	1010	18/32	1,86	15,8	
	1011	19/32	1,96	15,4	
	1100	20/32	2,06	15	
	1101	21/32	2,17	14,4	
	1110	22/32	2,27	13,8	
	1111	23/32	2,37	12,9	
1	0000	0/24	0,00	0,5	
	0001	1/24	0,14	1,9	
	0010	2/24	0,28	3,7	
	0011	3/24	0,41	5,3	
	0100	4/24	0,55	6,7	
	0101	5/24	0,69	7,9	
	0110	6/24	0,83	9	
	0111	7/24	0,96	9,9	
	1000	8/24	1,10	10,7	
	1001	9/24	1,24	11,3	
	1010	10/24	1,38	11,7	
	1011	11/24	1,51	11,9	
	1100	12/24	1,65	12	
	1101	13/24	1,79	11,9	
	1110	14/24	1,93	11,7	
	1111	15/24	2,06	11,3	

Результат работы компаратора сигнал Out может быть проинвертирован с помощью бита INV и выдан на вывод микросхемы OUT_COMP. Также результат сравнения доступен внутри микросхемы. Комбинационный сигнал с компаратора отображается в бите Rslt_As (при чтении может быть считан как 1, но при этом не выработать прерывания). Зафиксированный в триггере по тактовой частоте HCLK сигнал сравнения отображается в бите Rslt_Sy. Флаг Rst_lch фиксирует событие появления положительного сигнала сравнения и устанавливается в 1 до тех пор, пока не будет считан регистр COMP_RESULT_LATCH.

26.5 Описание регистров блока контроллера компаратора

Таблица 315 – Описание регистров блока контроллера компаратора

Базовый Адрес	Название	Описание
0x4009_8000	MDR_COMP	Контроллер компаратора
Смещение		
0x00	MDR_COMP->CFG	Регистр управления компаратора
0x04	MDR_COMP->RESULT	Регистр результата компаратора
0x08	MDR_COMP->RESULT_LATCH	Регистр результата компаратора – защелка

26.5.1 MDR_COMP->CFG

Таблица 316 – Регистр CFG

Номер	31...14	13	12	11	10, 9	8	7...4	3	2	1	0
Доступ	U	R/W	RO	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Сброс	0	0	0	0	0	0	0	0	0	0	0
	-	CMP IE	Ready	INV	CCH [1:0]	CREF	CVR [3:0]	CVR EN	CVRSS	CVRR	ON

Таблица 317 – Описание бит регистра CFG

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...14	-	Зарезервировано
13	CMP IE	Флаг разрешения генерации прерывания по событию Rst_lch: 0 – запрещено прерывание; 1 – разрешено прерывание
12	Ready	Сигнал готовности аналогового компаратора при включении: 0 – компаратор не включен или не готов к работе; 1 – компаратор готов к работе
11	INV	Инверсия выхода компаратора: 0 – прямой; 1 – инверсный
10, 9	CCH [1:0]	Биты выбора сигнала управления мультиплексора канала: 00 – на «-» компаратора сигнал подается с IN2; 01 – на «-» компаратора сигнал подается с IN1; 10 – на «-» компаратора сигнал подается с IN3; 11 – на «-» компаратора сигнал подается с выхода внутреннего источника опорного напряжения 1.2 В (IVREF)
8	CREF	Бит выбора сигнала управления мультиплексора канала: 0 – на «+» компаратора сигнал подается с IN1; 1 – на «+» компаратора сигнал подается с CVREF
7...4	CVR [3:0]	Биты выбора сигнала управления мультиплексора выбора CVREF. См.таблицу 314
3	CVREN	Бит разрешения работы источника CVREF: 0 – не разрешен; 1 – разрешен

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
2	CVRSS	Бит выбора опоры CVREF: 0 – источник CVREF работает в границах AVdd AGND; 1 – источник CVREF работает в границах Vref+ Vref-
1	CVRR	Бит выбора диапазона CVREF: 0 – источник CVREF работает в верхнем диапазоне; 1 – источник CVREF работает в нижнем диапазоне
0	ON	Включение компаратора: 0 – выключен; 1 – включен

26.5.2 MDR_COMP->RESULT

Таблица 318 – Регистр RESULT

Номер	31...3	2	1	0
Доступ	U	R/W	R/W	R/W
Сброс	0	0	0	0
	-	Rst_lch	Rslt_As	Rslt_Sy

Таблица 319 – Описание бит регистра RESULT

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...3	-	Зарезервировано
2	Rst_lch	Значение компарирования хранится до момента считывания из регистра COMP_RESULT_LATCH, после чего сбрасывается. Вводится по переднему фронту сигнала с компаратора
1	Rslt_As	Значение компарирования непосредственно из компаратора
0	Rslt_Sy	Значение результата компарирования, синхронизированное с частотой HCLK

26.5.3 MDR_COMP->RESULT_LATCH

Таблица 320 – Регистр RESULT_LATCH

Номер	31...1	0
Доступ	U	R/W
Сброс	0	0
	-	Rst_lch

Таблица 321 – Описание бит регистра RESULT_LATCH

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...1	-	Зарезервировано
0	Rst_lch	Значение компарирования хранится до момента считывания из регистра COMP_RESULT_LATCH, после чего сбрасывается. Вводится по переднему фронту сигнала с компаратора

27 Контроллер интерфейса MDR_I2C

Интерфейс I2C является двухпроводным, двунаправленным последовательным каналом связи с простым и эффективным методом обмена данными между устройствами. Интерфейс применяется, когда надо организовать обмен на коротком расстоянии между несколькими устройствами.

Контроллер интерфейса I2C работает только в режиме ведущего устройства («мастера»). Реализована совместимость с многомастерными системами: предусмотрены механизмы детектирования коллизий и арбитража шины, предотвращающие потерю данных при обмене, когда два или более мастера пытаются осуществить передачу одновременно. Поддерживается работа с ведомыми устройствами, применяющими растяжение тактовой частоты SCL (clock stretching) для замедления скорости обмена данными.

Контроллер интерфейса I2C позволяет работать на типовых скоростях:

- нормальная: 100 кбит/с;
- быстрая: 400 кбит/с;
- очень быстрая: 1 Мбит/с.

27.1 Конфигурация контроллера I2C

Контроллер I2C формирует тактовый сигнал обмена данными SCL с использованием внутреннего предделителя частоты DIV, коэффициент деления которого задается в регистрах PRL (младшая часть) и PRH (старшая часть). Минимальное значение DIV равно 1. Если рассчитанное значение DIV не использует старшую часть, то регистр PRH должен быть установлен в 0. Настройка DIV должна выполняться только при выключенном контроллере I2C (EN_I2C = 0).

Расчетная скорость обмена данными по интерфейсу I2C определяется по формуле:

$$F_{SCL} = \frac{HCLK}{5 \cdot (DIV + 1)}. \quad (13)$$

Необходимо учитывать, что контроллер I2C отслеживает фактическое состояние линии SCL. Если после завершения формирования низкого уровня контроллер отпускает линию SCL, но она остаётся в состоянии логического нуля (например, из-за растяжения SCL ведомым устройством или медленного фронта нарастания сигнала SCL), то начало следующего тактового цикла откладывается до перехода SCL в высокий уровень. Таким образом фактическая скорость обмена может оказаться ниже расчётной, определяемой формулой (13).

Разрешение работы контроллера I2C осуществляется установкой бита EN_I2C в регистре CMD. Выключение контроллера I2C (сброс бита EN_I2C) допустимо только после завершения обмена на шине, то есть после окончания передачи сигнала STOP. Нарушение данного требования может привести к блокировке шины I2C.

Запись в регистр управления CMD возможна только при разрешенной работе контроллера I2C (EN_I2C = 1). Попытка записи в регистр CMD при неактивном состоянии контроллера I2C (EN_I2C = 0) игнорируется.

27.2 Конфигурация системы

I2C-системы используют последовательную линию данных SDA и линию тактового сигнала SCL. Все устройства, подсоединенные к этим двум линиям, должны работать в режиме открытого стока, обеспечивая тем самым создание на линии «проводного И» за счет внешних резисторов подтяжки обеих линий к питанию.

Передача данных между мастером и ведомым осуществляется по линии SDA и синхронизируется по линии SCL. После завершения передачи информации осуществляется передача в обратную сторону одного бита подтверждения. Каждый принимаемый бит фиксируется принимающей стороной при высоком уровне SCL и может изменяться передатчиком при низком уровне. Изменение линии SDA при высоком уровне SCL является командным состоянием (см. подразделы 27.4 «Сигнал START» и 27.7 «Сигнал STOP»).

27.3 Протокол I2C

Типовая передача по интерфейсу I2C содержит четыре фазы:

- сигнал START;
- передача адреса;
- передача данных;
- сигнал STOP.

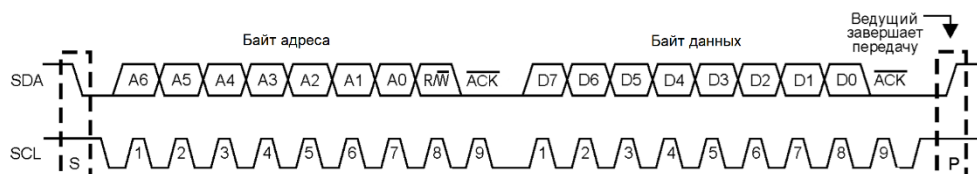


Рисунок 92 – Прием/передача по I2C в 7-битном режиме

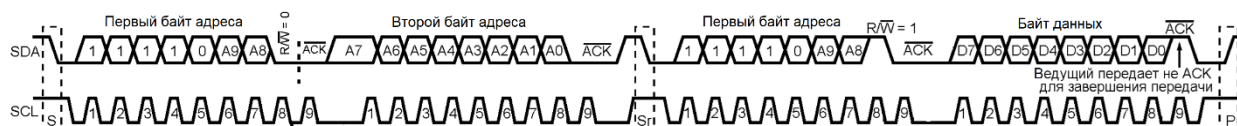


Рисунок 93 – Прием по I2C в 10-битном режиме

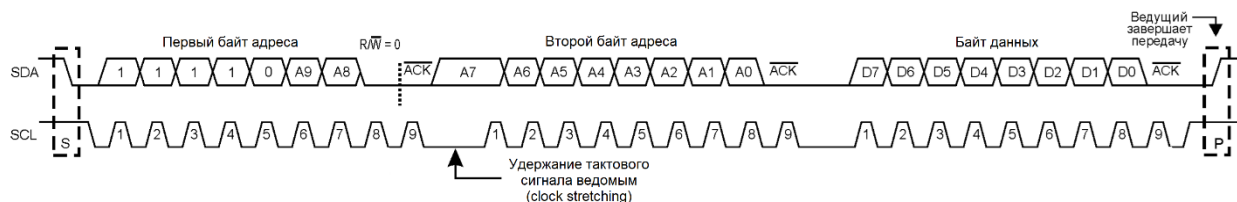


Рисунок 94 – Передача по I2C в 10-битном режиме

27.4 Сигнал START

Когда шина находится в свободном состоянии, то есть ни одно из устройств не осуществляет передачи (на линиях SCL и SDA высокий уровень), мастер может инициализировать процесс передачи через создание сигнала START на линии. Сигнал START или S-бит задается, когда уровень на линии SDA переходит из высокого в низкий при высоком уровне на линии SCL. Появление сигнала START не означает начала передачи данных.

Повторный сигнал START является обычным сигналом START, но без предварительно сгенерированного до этого сигнала STOP. Мастер может использовать метод для начала соединения с другим ведомым или с тем же ведомым, но с изменением режима работы (например, чтение после записи, или, наоборот) без перевода шины в свободное состояние.

Контроллер интерфейса генерирует сигнал START при записи единицы в бит START регистра CMD при установленных битах WR или RD. Для начала стандартного обмена необходимо установить биты START и WR в регистре CMD, что приведет к последовательной выдаче на линию сигнала START, а также адреса ведомого устройства и бита RW, которые записаны в регистр TXD. В зависимости от состояния линии SCL генерируется либо сигнал START, либо повторный сигнал START.

27.5 Передача адреса

Первым байтом данных, передаваемым мастером сразу после сигнала START, является адрес ведомого. Это 7-битный адрес и следующий за ним бит RW. Бит RW определяет дальнейшее направление передачи данных: 0 – запись данных в ведомое устройство, 1 – чтение данных из ведомого устройства. В системе не может быть несколько ведомых устройств с одним адресом. Ведомое устройство, у которого совпадает адрес с адресом в сообщении, подтверждает прием, выставляя ACK и опуская линию SDA в низкий уровень на 9-й тактовый импульс SCL.

Процесс выдачи 7-битного адреса выполняется совместно с выдачей сигнала START. Необходимо записать адрес ведомого и бит RW в регистр TXD, после чего установить биты START и WR в регистре CMD. Контроллер осуществит передачу сигнала START и адреса в линию.

Контроллер также поддерживает передачу 10-битного адреса ведомого путем генерации нескольких циклов передачи адреса (см. рисунки 93 и 94). Сначала необходимо выполнить передачу сигнала START и первого байта адреса ведомого с битом RW равным 0 (запись в ведомое устройство), как указано выше. После завершения передачи необходимо записать второй байт адреса ведомого в регистр TXD, после чего установить бит WR в регистре CMD. Если требуется смена направления передачи на RW = 1 (чтение из ведомого устройства), то после завершения передачи второго байта адреса необходимо выполнить повторную передачу сигнала START и первого байта адреса ведомого с битом RW равным 1.

27.6 Передача данных

После успешного подтверждения приема адреса одним ведомым устройством может быть начата передача данных в направлении, задаваемым битом RW в послылке мастера. Каждый передаваемый бит подтверждается ACK на 9-й тактовый импульс SCL. Если ведомое устройство выдало NACK (нет подтверждения), то мастер может сгенерировать либо сигнал STOP для прекращения передачи, либо повторный сигнал START для начала нового цикла передачи.

Если мастер является принимающим устройством и выдает NACK, то ведомое устройство отпускает линию SDA и мастер может сгенерировать сигнал STOP или повторный сигнал START.

Для записи данных в ведомое устройство нужно записать данные в регистр TXD и установить бит WR в регистре CMD. Для чтения данных из ведомого устройства нужно в регистре CMD установить бит RD, при этом одновременно записав нужное значение в бит ACK. На время выполнения передачи контроллер интерфейса выставляет флаг TR_PROG в регистре STA. Когда передача завершена, этот флаг снимается и устанавливается флаг INT. Если при этом установлен бит разрешения EN_INT в регистре CTR, то генерируется прерывание контроллеру прерываний. Регистр RXD содержит корректные принятые данные после сброса флага TR_PROG или установки флага INT в регистре STA. Новый цикл чтения или записи можно начать только тогда, когда флаг TR_PROG сброшен.

27.7 Сигнал STOP

Мастер может завершить соединение путем создания сигнала STOP. Сигнал STOP или P-бит определяется переходом линии SDA из низкого состояния в высокое, когда SCL находится в высоком состоянии.

Сигнал STOP генерируется при записи единицы в бит STOP регистра CMD.

27.8 Описание регистров контроллера I2C

Таблица 322 – Описание регистров контроллера I2C

Базовый Адрес	Название	Описание
0x4005_0000	MDR_I2C	Контроллер I2C
Смещение		
0x00	MDR_I2C->PRL	Младшая часть предделителя частоты
0x04	MDR_I2C->PRH	Старшая часть предделителя частоты
0x08	MDR_I2C->CTR	Управление контроллером I2C
0x0C	MDR_I2C->RXD	Принятые данные по I2C
0x10	MDR_I2C->STA	Статус I2C
0x14	MDR_I2C->TXD	Передаваемые данные по I2C
0x18	MDR_I2C->CMD	Управление I2C

27.8.1 MDR_I2C->PRL

Таблица 323 – Регистр PRL

Номер	31...8	7...0
Доступ	U	R/W
Сброс	0	0
	-	PR[7:0]

Таблица 324 – Описание бит регистра PRL

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...8	-	Зарезервировано
7...0	PR[7:0]	Младшая часть предделителя

27.8.2 MDR_I2C->PRH

Таблица 325 – Регистр PRH

Номер	31...8	7...0
Доступ	U	R/W
Сброс	0	0
	-	PR[15:8]

Таблица 326 – Описание бит регистра PRH

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...8	-	Зарезервировано
7...0	PR[15:8]	Старшая часть предделителя

27.8.3 MDR_I2C->CTR

Таблица 327 – Регистр CTR

Номер	31...8	7	6	5...0
Доступ	U	R/W	R/W	U
Сброс	0	0	0	0
	-	EN_I2C	EN_INT	-

Таблица 328 – Описание бит регистра CTR

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...8	-	Зарезервировано
7	EN_I2C	Разрешение работы контроллера I2C: 0 – выключен; 1 – включен
6	EN_INT	Разрешение прерывания от I2C: 0 – запрещено; 1 – разрешено
5...0	-	Зарезервировано

27.8.4 MDR_I2C->RXD

Таблица 329 – Регистр RXD

Номер	31...8	7...0
Доступ	U	RO
Сброс	0	0
	-	RXD[7:0]

Таблица 330 – Описание бит регистра RXD

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...8	-	Зарезервировано
7...0	RXD[7:0]	Последний полученный по I2C байт

27.8.5 MDR_I2C->STA

Таблица 331 – Регистр STA

Номер	31...8	7	6	5	4...2	1	0
Доступ	U	RO	RO	RO	U	RO	RO
Сброс	0	0	0	0	0	0	0
	-	RX_ACK	BUSY	LOST_ARB	-	TR_PROG	INT

Таблица 332 – Описание бит регистра STA

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...8	-	Зарезервировано
7	RX_ACK	Полученный от ведомого ACK/NACK: 0 – получен ACK; 1 – получен NACK. Бит устанавливается в 0 при потере арбитража
6	BUSY	Состояние шины I2C: 0 – после получения сигнала STOP; 1 – после получения сигнала START
5	LOST_ARB	Потеря арбитража: 0 – нет потери арбитража; 1 – потеря арбитража. Этот бит выставляется, если: – получен сигнал STOP, но он не был инициирован этим контроллером; – Если контроллер пытается выставить SDA в высокий уровень, но SDA остается в низком уровне. Бит устанавливается в 0 после получения сигнала START
4...2	-	Зарезервировано
1	TR_PROG	Процесс передачи: 0 – передача завершена; 1 – передаются данные. Бит является объединением битов RD и WR регистра CMD

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
0	INT	Флаг прерывания. 0 – нет прерывания; 1 – есть прерывание. Флаг выставляется, если: – завершена передача байта; – был потерян арбитраж. Прерывание для процессора выставляется, если установлен бит EN_INT

27.8.6 MDR_I2C->TXD

Таблица 333 – Регистр TXD

Номер	31...8	7...0
Доступ	U	R/W
Сброс	0	0
	-	TXD[7:0]

Таблица 334 – Описание бит регистра TXD

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...8	-	Зарезервировано
7...0	TXD[7:0]	Байт для отправки по I2C. При передаче первого байта адреса нулевой бит определяет режим передачи: 0 – запись в ведомое устройство; 1 – чтение из ведомого устройства

27.8.7 MDR_I2C->CMD

Таблица 335 – Регистр CMD

Номер	31...8	7	6	5	4	3	2, 1	0
Доступ	U	R/W	R/W	R/W	R/W	R/W	U	R/W
Сброс	0	0	0	0	0	0	0	0
	-	START	STOP	RD	WR	ACK	-	CLR_INT

Таблица 336 – Описание бит регистра CMD

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...8	-	Зарезервировано
7	START	Отправить сигнал START: 0 – нет действия; 1 – начать отправку сигнала START. Передача START выполняется только при установленных битах WR или RD. Сбрасывается автоматически при потере арбитража и после завершения передачи

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
6	STOP	Отправить сигнал STOP. 0 – нет действия; 1 – начать отправку сигнала STOP. Сбрасывается автоматически при потере арбитража и после завершения передачи
5	RD	Чтение из ведомого устройства: 0 – нет действия; 1 – начать чтение. Сбрасывается автоматически при потере арбитража и после завершения передачи
4	WR	Запись в ведомое устройство; 0 – нет действия; 1 – начать запись. Сбрасывается автоматически при потере арбитража и после завершения передачи
3	ACK	Отправить ACK при чтении: 0 – отправить ACK; 1 – отправить NACK. После завершения отправки NACK автоматически в 0 не сбрасывается, а очищается записью нуля
2, 1	-	Зарезервировано
0	CLR_INT	Очистить прерывание INT. Запись 1 очищает прерывание

28 Контроллер MDR_SSP

Модуль порта синхронной последовательной связи (SSP – Synchronous Serial Port) выполняет функции интерфейса последовательной синхронной связи в режиме ведущего и ведомого устройства и обеспечивает обмен данными с подключенным ведомым или ведущим периферийным устройством в соответствии с одним из протоколов:

- интерфейс SPI фирмы Motorola;
- интерфейс SSI фирмы Texas Instruments;
- интерфейс Microwire фирмы National Semiconductor.

Как в ведущем, так и в ведомом режиме работы модуль SSP обеспечивает:

- преобразование данных, размещенных во внутреннем буфере FIFO передатчика (восемь 16-разрядных ячеек данных) из параллельного в последовательный формат;
- преобразование данных из последовательного в параллельный формат и их запись в аналогичный буфер FIFO приемника (восемь 16-разрядных ячеек данных).

Модуль формирует сигналы прерываний по следующим событиям:

- необходимость обслуживания буферов FIFO приемника и передатчика;
- переполнение буфера FIFO приемника;
- наличие данных в буфере FIFO приемника по истечении времени таймаута.

Основные сведения о модуле представлены в следующих разделах:

- характеристики интерфейса SPI;
- характеристики интерфейса Microwire;
- характеристики интерфейса SSI.

28.1 Основные характеристики модуля SSP

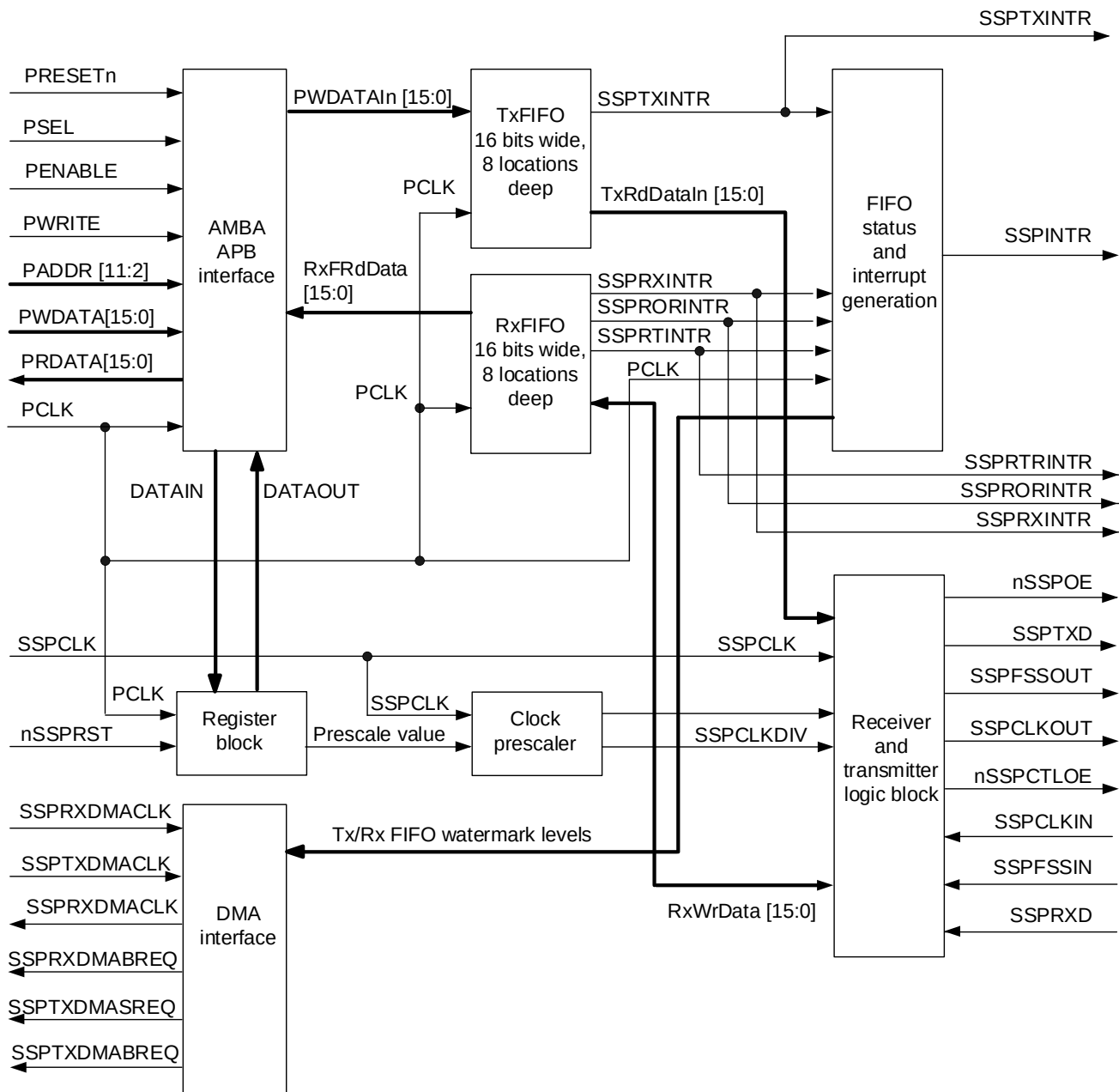
- может функционировать как в ведущем, так и в ведомом режиме;
- программное управление скоростью обмена;
- состоит из независимых буферов приема и передачи (8 ячеек по 16 бит) с организацией доступа типа FIFO (First In First Out – первый вошел, первый вышел);
- программный выбор одного из интерфейсов обмена: SPI, Microwire, SSI;
- программируемая длительность информационного кадра от 4 до 16 бит;
- независимое маскирование прерываний от буфера FIFO передатчика, буфера FIFO приемника, а также по переполнению буфера приемника;
- доступна возможность тестирования по шлейфу, соединяющему вход с выходом;
- поддержка прямого доступа к памяти (DMA).

Структурная схема модуля представлена далее на рисунке 95.

28.2 Программируемые параметры

Следующие ключевые параметры могут быть заданы программно:

- режим функционирования периферийного устройства – ведущее или ведомое;
- разрешение или запрещение функционирования;



28.3 Характеристики интерфейса SPI

Последовательный синхронный интерфейс SPI фирмы Motorola обеспечивает:

- полнодуплексный обмен данными по четырехпроводной линии;
- программное задание фазы и полярности тактового сигнала.

28.4 Характеристики интерфейса Microwire

Интерфейс Microwire фирмы National Semiconductor обеспечивает:

- полудуплексный обмен данными с использованием 8-битных управляющих последовательностей.

28.5 Характеристики интерфейса SSI

Интерфейс SSI фирмы Texas Instruments обеспечивает:

- полнодуплексный обмен данными по четырехпроводной линии;
- возможность перевода линии передачи данных в третье (высокоимпедансное) состояние.

28.6 Общий обзор модуля SSP

Модуль SSP представляет собой интерфейс синхронного последовательного обмена данными, способный функционировать в качестве ведущего или ведомого устройства и поддерживающий протоколы передачи данных SPI фирмы Motorola, Microwire фирмы National Semiconductor, а также SSI фирмы Texas Instruments.

Модуль выполняет следующие функции:

- преобразование данных, полученных от периферийного устройства, из последовательной в параллельную форму;
- преобразование данных, передаваемых на периферийное устройство, из параллельной в последовательную форму;
- центральный процессор читает и записывает данные, а также управляющую информацию и информацию о состоянии;
- прием и передача данных буферизуются с помощью буферов FIFO, обеспечивающих хранение до восьми слов данных шириной 16 бит независимо для режимов приема и передачи.

Последовательные данные передаются по линии SSP_TXD и принимаются с линии SSP_RXD.

Модуль SSP содержит программируемые делители частоты, формирующие тактовый сигнал обмена данными SSP_CLK из сигнала, поступающего на линию SSPCLK. Скорость передачи данных может достигать более 2 МГц, в зависимости от частоты SSPCLK и характеристик подключенного периферийного устройства.

Режим обмена данными, формат информационного кадра и количество бит данных задаются программно с помощью регистров управления CR0 и CR1.

Модуль формирует четыре независимо маскируемых прерывания:

SSPTXINTR – запрос на обслуживание буфера передатчика;

SSPRXINTR – запрос на обслуживание буфера приемника;

SSPRORINTR – переполнение приемного буфера FIFO;

SSPRTINTR – таймаут ожидания чтения данных из приемного FIFO.

Кроме того, формируется общий сигнал прерывания SSPINTR, возникающий в случае активности одного из вышеуказанных независимых немаскированных прерываний, который идет на контроллер NVIC.

Модуль также формирует сигналы запроса на прямой доступ к памяти (DMA) для совместной работы с контроллером DMA.

В зависимости от режима работы модуля сигнал SSPFSSOUT используется либо для кадровой синхронизации (интерфейс SSI, активное состояние – высокий уровень), либо для выбора ведомого режима (интерфейсы SPI и Microwire, активное состояние – низкий уровень).

28.6.1 Блок формирования тактового сигнала

В режиме ведущего устройства модуль формирует тактовый сигнал обмена данными SSP_CLK с помощью внутреннего делителя частоты, состоящего из двух последовательно соединенных счетчиков без цепи сброса.

Путем записи значения в регистр SSPCPSR можно задать коэффициент предварительного деления частоты в диапазоне от 2 до 254 с шагом 2. Так как младший значащий разряд коэффициента деления не используется, то исключается возможность деления частоты на нечетный коэффициент деления. Это, в свою очередь, гарантирует формирование тактового сигнала симметричной формы (с одинаковой длительностью полупериодов высокого и низкого уровней).

Сформированный описанным образом сигнал далее поступает на второй делитель частоты, с выхода которого и снимается тактовый сигнал обмена данными SSP_CLK.

Коэффициент деления второго делителя задается программно в диапазоне от 1 до 256, путем записи соответствующего значения в регистр управления SSPCR0.

28.6.2 Буфер FIFO передатчика

Буфер передатчика имеет ширину 16 бит, глубину 8 слов, схему организации доступа типа FIFO – «первый вошел, первый вышел». Данные от центрального процессора сохраняются в буфере до тех пор, пока не будут считаны блоком передачи данных.

28.6.3 Буфер FIFO приемника

Буфер приемника имеет ширину 16 бит, глубину 8 слов, схему организации доступа типа FIFO – «первый вошел, первый вышел». Принятые от периферийного устройства данные сохраняются в этом буфере блоком приема данных до тех пор, пока не будут считаны центральным процессором.

28.6.4 Блок приема и передачи данных

В режиме ведущего устройства модуль формирует тактовый сигнал обмена данными SSP_CLK для подключенных ведомых устройств. Как было описано ранее, данный сигнал формируется путем деления частоты сигнала SSPCLK.

Блок передатчика последовательно считывает данные из буфера FIFO передатчика и производит их преобразование из параллельной формы в последовательную. Далее поток последовательных данных и элементов кадровой синхронизации, тактированный сигналом SSP_CLK, передается по линии SSP_TXD к подключенным ведомым устройствам.

Блок приемника выполняет преобразование данных, поступающих синхронно с линии SSP_RXD, из последовательной в параллельную форму, после чего загружает их в буфер FIFO приемника, откуда они могут быть считаны процессором.

В режиме ведомого устройства тактовый сигнал обмена данными формируется одним из подключенных к модулю периферийных устройств и поступает по линии SSP_CLK.

При этом блок передатчика, тактируемый этим внешним сигналом, считывает данные из буфера FIFO, преобразует их из параллельной формы в последовательную, после чего выдает поток последовательных данных и элементов кадровой синхронизации в линию SSP_TXD.

Аналогично, блок приемника выполняет преобразование данных, поступающих с линии SSP_RXD синхронно с сигналом SSP_CLK, из последовательной в параллельную форму, после чего загружает их в буфер FIFO приемника, откуда они могут быть считаны процессором.

Примечание – В режиме работы ведомого устройства, запросы ведущим устройством на выдачу информации от ведомого, необходимо осуществлять при наличии данных в FIFO передатчика ведомого.

28.6.5 Блок формирования прерываний

Модуль SSP генерирует независимые маскируемые прерывания с активным высоким уровнем. Кроме того, формируется комбинированное прерывание путем объединения указанных независимых прерываний по схеме ИЛИ.

Комбинированный сигнал прерывания подается на контроллер прерываний NVIC, при этом появляется дополнительная возможность маскирования устройства в целом, что облегчает построение модульных драйверов устройств.

28.6.6 Интерфейс прямого доступа к памяти

Модуль обеспечивает интерфейс с контроллером DMA согласно схеме взаимодействия приемопередатчика и контроллера DMA.

28.6.7 Конфигурирование приемопередатчика

После сброса работа блоков приемопередатчика запрещается до выполнения процедуры задания конфигурации.

Для этого необходимо выбрать ведущий или ведомый режим работы устройства, а также используемый протокол передачи данных (SPI фирмы Motorola, SSI фирмы Texas Instruments, либо Microware фирмы National Semiconductor), после чего записать необходимую информацию в регистры управления CR0 и CR1.

Кроме того, для установки требуемой скорости передачи данных необходимо выбрать параметры блока формирования тактового сигнала с учетом значения частоты сигнала SSPCLK и записать соответствующую информацию в регистр PSR.

28.6.8 Разрешение работы приемопередатчика

Разрешение осуществляется путем установки бита SSE регистра управления CR1. Буфер FIFO передатчика может быть либо проинициализирован путем записи в него до восьми 16-разрядных слов заблаговременно перед установкой этого бита, либо может заполняться передаваемыми данными в процедуре обслуживания прерывания.

После разрешения работы модуля приемопередатчик начинает обмен данными по линиям SSP_TXD и SSP_RXD.

28.6.9 Соотношения между тактовыми сигналами

В модуле имеется ограничение на соотношение между частотами тактовых сигналов CPU_CLK и SSPCLK. Частота SSPCLK должна меньше или равна частоте CPU_CLK. Выполнение этого требования гарантирует синхронизацию сигналов управления, передаваемых из зоны действия тактового сигнала SSPCLK в зону действия сигнала CPU_CLK в течение времени, меньшего продолжительности передачи одного информационного кадра $FSSPCLK \leq FPCLK$.

В режиме ведомого устройства сигнал SSP_CLK от ведущего внешнего устройства поступает на схемы синхронизации, задержки и обнаружения фронта. Для того чтобы обнаружить фронт сигнала SSP_CLK, необходимо три такта сигнала SSPCLK. Сигнал SSP_TXD имеет меньшее время установки по отношению к заднему фронту SSP_CLK, по которому и происходит считывание данных из линии. Время установки и удержания сигнала SSP_RXD по отношению к сигналу SSP_CLK должно выбираться с запасом, гарантирующим правильное считывание данных. Для обеспечения корректной работы устройства необходимо, чтобы частота SSPCLK была как минимум в 12 раз больше, чем максимальная предполагаемая частота сигнала SSP_CLK.

Выбор частоты тактового сигнала SSPCLK должен обеспечивать поддержку требуемого диапазона скоростей обмена данными. Отношение минимальной частоты сигнала SSPCLK к максимальной частоте сигнала SSP_CLK в режиме ведомого устройства равно 12, в режиме ведущего – двум.

Так, в режиме ведущего устройства для обеспечения максимальной скорости обмена 1,8432 Мбит/с частота сигнала SSPCLK должна составлять не менее 3,6864 МГц. В этом случае в регистр CPSR должно быть записано значение 2, а поле SCR[7:0] регистра CR0 должно быть установлено в 0.

В режиме ведомого устройства для обеспечения той же информационной скорости необходимо использовать тактовый сигнал SSPCLK с частотой не менее 22,12 МГц. При этом в регистр CPSR должно быть записано значение 12, а поле SCR[7:0] регистра CR0 должно быть установлено в 0.

Соотношение между максимальной частотой сигнала SSPCLK и минимальной частотой SSPCLKOUT составляет 254×256 .

Минимальная допустимая частота сигнала SSPCLK определяется следующей системой соотношений, которые должны выполняться одновременно:

$$\text{FSSPCLK}(\min) \Rightarrow 2 \times \text{FSSPCLKOUT}(\max) \text{ [for master mode]}$$

$$\text{FSSPCLK}(\min) \Rightarrow 12 \times \text{FSSPCLKIN}(\max) \text{ [for slave mode]}.$$

Аналогично, максимальная допустимая частота сигнала SSPCLK определяется следующей системой соотношений, которые должны выполняться одновременно:

$$\text{FSSPCLK}(\max) \leq 254 \times 256 \times \text{FSSPCLKOUT}(\min) \text{ [for master mode]}$$

$$\text{FSSPCLK}(\max) \leq 254 \times 256 \times \text{FSSPCLKIN}(\min) \text{ [for slave mode]}.$$

28.6.10 Программирование регистра управления CR0

Регистр CR0 предназначен для:

- установки скорости информационного обмена;
- выбора одного из трех протоколов обмена данными;
- выбора размера слова данных.

Скорость информационного обмена зависит от частоты внешнего тактового сигнала SSPCLK и коэффициента деления блока формирования тактового сигнала. Последний задается совместно значением поля SCR (Serial Clock Rate – скорость информационного обмена) регистра SSPCR0 и значением поля CPSDVSR (clock prescale divisor value – коэффициент деления тактового сигнала) регистра SSPCPSR.

Формат информационного кадра задается путем установки значения поля FRF, а размер слова данных – путем установки значения поля DSS регистра SSPCR0.

Для протокола SPI фирмы Motorola также задаются полярность и фаза сигнала (биты SPH и SPO).

28.6.11 Программирование регистра управления CR1

Регистр SSPCR1 предназначен для:

- выбора ведущего или ведомого режима функционирования приемопередатчика;
- включения режима проверки канала по шлейфу;
- разрешения или запрещения работы модуля.

Выбор ведущего режима осуществляется путем записи 0 в поле MS регистра SSPCR1 (это значение устанавливается после сброса автоматически).

Запись 1 в поле MS переводит приемопередатчик в режим ведомого устройства. В этом режиме разрешение или запрещение формирования сигнала передатчика SSP_TXD осуществляется путем установки бита SOD (slave mode SSP_TXD output disable – запрет линии SSP_TXD для ведомого режима) регистра CR1. Указанная функция полезна при подключении к одной линии нескольких подчиненных устройств.

Для того чтобы разрешить функционирование приемопередатчика, необходимо установить в 1 бит SSE (Synchronous Serial Port Enable – разрешение последовательного синхронного порта).

28.6.12 Формирование тактового сигнала обмена данными

Тактовый сигнал обмена данными формируется путем деления частоты тактового сигнала SSPCLK. На первом этапе формирования частота этого сигнала делится на четный коэффициент CPSDVSR, лежащий в диапазоне от 2 до 254, доступный для программирования через регистр CPSR. Сформированный сигнал далее поступает на делитель частоты с коэффициентом $(1 + SCR)$ от 1 до 256, где значение SCR доступно для программирования через CR0.

Частота выходного тактового сигнала обмена данными SSP_CLK определяется следующим соотношением

$$F_{SSPCLKOUT} = \frac{F_{SSPCLK}}{CPSDVR \cdot (1 + SCR)} \cdot \quad (14)$$

Например, в случае, если частота сигнала SSPCLK составляет 3,6864 МГц, а значение CPSDVSR = 2, частота сигнала SSP_CLK лежит в интервале от 7,2 кГц до 1,8432 МГц.

28.6.13 Формат информационного кадра

Каждый информационный кадр содержит в зависимости от запрограммированного значения от 4 до 16 бит данных. Передача данных начинается со старшего значащего разряда. Можно выбрать три базовых структуры построения кадра:

- SSI фирмы Texas Instruments;
- SPI фирмы Motorola;
- Microwire фирмы National Semiconductor.

Во всех трех режимах построения кадра тактовый сигнал SSP_CLK формируется только тогда, когда приемопередатчик готов к обмену данными. Перевод сигнала SSP_CLK в неактивное состояние используется как признак таймаута приемника, то есть наличия в буфере приемника необработанных данных по истечении заданного интервала времени.

В режимах SPI и Microwire выходной сигнал кадровой синхронизации передатчика SSP_FSS имеет активный низкий уровень и поддерживается в низком уровне в течение всего периода передачи информационного кадра.

В режиме построения кадра SSI фирмы Texas Instruments перед началом каждого информационного кадра на выходе SSP_FSS формируется импульс с длительностью, равной одному тактовому интервалу обмена данными. В этом режиме приемопередатчик SSP, равно как и ведомые периферийные устройства, передает данные в линию по переднему фронту сигнала SSP_CLK, а считывает данные из линии по заднему фронту этого сигнала.

В отличие от полнодуплексных режимов передачи данных SSI и SPI, режим Microwire фирмы National Semiconductor использует специальный способ обмена данными между ведущим и ведомым устройством, функционирующий в режиме полудуплекса. В указанном режиме на внешнее ведомое устройство перед началом передачи информационного кадра посылается специальная 8-битная управляющая

последовательность. В течение всего времени передачи этой последовательности приемник не обрабатывает каких-либо входных данных. После того как сигнал передан и декодирован ведомым устройством, оно выдерживает паузу в один тактовый интервал после передачи последнего бита управляющей последовательности, после чего передает в адрес ведущего устройства запрошенные данные. Длительность блока данных от ведомого устройства может составлять от 4 до 16 бит, таким образом общая длительность информационного кадра составляет от 13 до 25 бит.

28.6.14 Формат синхронного обмена SSI фирмы Texas Instruments

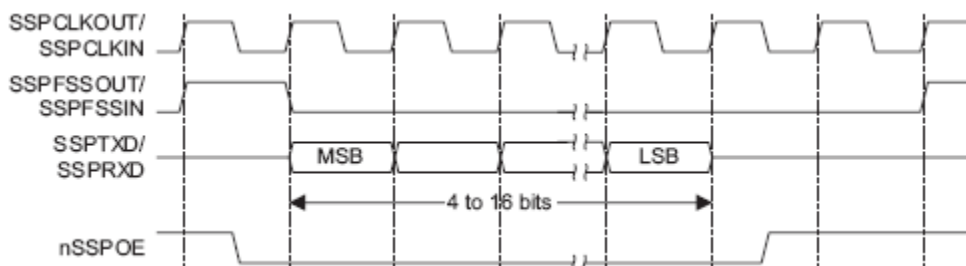


Рисунок 96 – Формат синхронного обмена протокола SSI (единичный обмен)

В данном режиме при неактивном приемопередатчике SSP сигналы SSP_CLK и SSP_FSS переводятся в низкий логический уровень, а линия передачи данных SSP_TXD поддерживается в третьем состоянии.

После появления хотя бы одного элемента в буфере FIFO передатчика сигнал SSP_FSS переводится в высокий логический уровень на время, соответствующее одному периоду сигнала SSP_CLK. При этом значение из буфера FIFO переносится в сдвиговый регистр блока передатчика. По следующему переднему фронту сигнала SSP_CLK старший значащий разряд информационного кадра (4 – 16 бит данных) выдается на выход линии SSP_TXD и т.д.

В режиме приема данных как модуль SSP, так и ведомое внешнее устройство последовательно загружают биты данных в сдвиговый регистр по заднему фронту сигнала SSP_CLK. Принятые данные переносятся из сдвигового регистра в буфер FIFO после загрузки в него младшего значащего бита данных по очередному переднему фронту сигнала SSP_CLK.

Временные диаграммы последовательного синхронного обмена по протоколу SSI фирмы Texas Instruments представлены на рисунках: рисунок 96 – передача единичного информационного кадра, рисунок 97 – передача последовательности кадров.

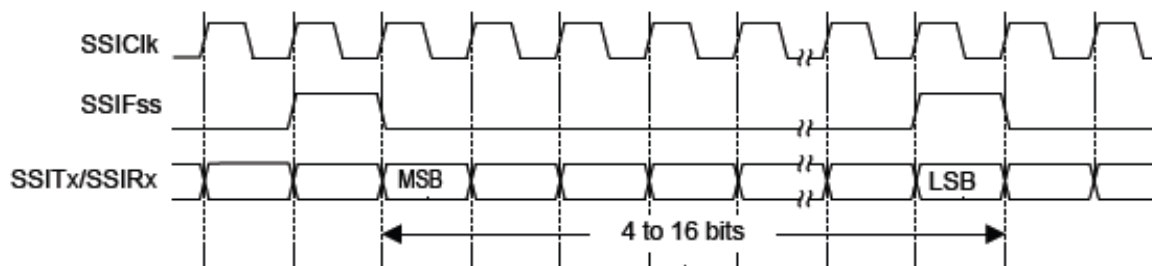


Рисунок 97 – Формат синхронного обмена протокола SSI (непрерывный обмен)

28.6.15 Формат синхронного обмена SPI фирмы Motorola

Интерфейс SPI фирмы Motorola осуществляется по четырем сигнальным линиям, при этом сигнал SSP_FSS выполняет функцию выбора ведомого устройства. Главной особенностью протокола SPI является возможность выбора состояния и фазы сигнала SSP_CLK в режиме ожидания (неактивном приемопередатчике) путем задания значений бит SPO и SPH регистра управления SSPSCR0.

Выбор полярности тактового сигнала – бит SPO

Если бит SPO равен 0, то в режиме ожидания линия SSP_CLK переводится в низкий логический уровень. В противном случае при отсутствии обмена данными линия SSP_CLK переводится в высокий логический уровень.

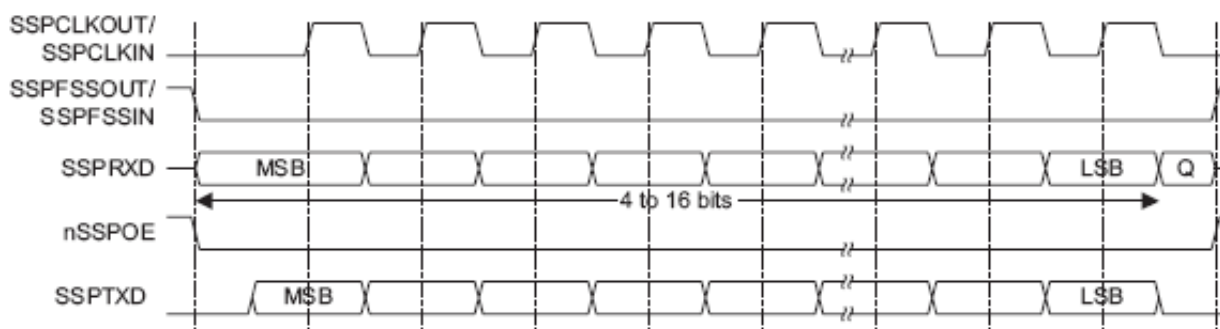
Выбор фазы тактового сигнала – бит SPH

Значение бита SPH определяет фронт тактового сигнала, по которому осуществляется выборка данных и изменение состояния на выходе линии.

В случае если бит SPH установлен в 0, регистрация данных приемником осуществляется после первого обнаружения фронта тактового сигнала, в противном случае – после второго.

28.6.15.1 Формат синхронного обмена SPI фирмы Motorola, SPO=0, SPH=0

Временные диаграммы последовательного синхронного обмена в режиме SPI с SPO=0, SPH=0 показаны на рисунках: рисунок 98 – одиночный обмен рисунок 99 – непрерывный обмен.



Q – сигнал с неопределенным уровнем

Рисунок 98 – Формат синхронного обмена протокола SPI, SPO=0, SPH=0
(одиночный обмен)

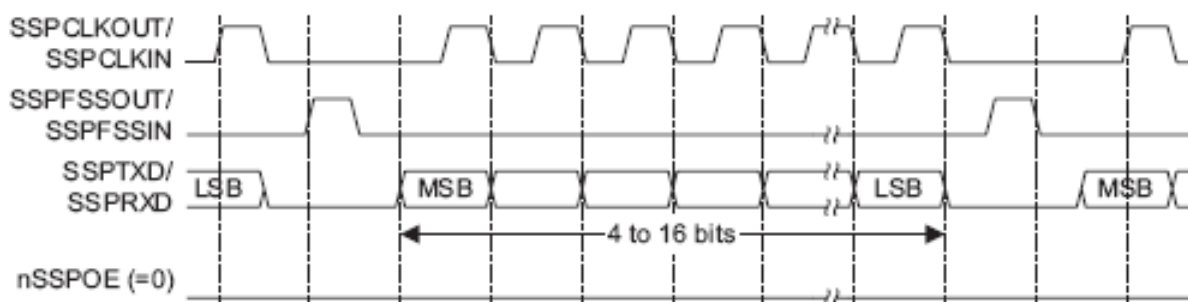


Рисунок 99 – Формат синхронного обмена протокола SPI, SPO=0, SPH=0
(непрерывный обмен)

В данном режиме во время ожидания приемопередатчика:

- сигнал SSP_CLK имеет низкий логический уровень;
- сигнал SSP_FSS имеет высокий логический уровень;
- сигнал SSP_TXD переводится в высокоимпедансное состояние.

Если работа модуля разрешена и в буфере FIFO передатчика содержатся корректные данные, сигнал SSP_FSS переводится в низкий логический уровень, что указывает на начало обмена данными и разрешает передачу данных от ведомого устройства на входную линию SSP_RXD ведущего. Контакт передатчика SSPTXD переходит из высокоимпедансного в активное состояние.

По истечении полутакта сигнала SSP_CLK на линии SSP_TXD формируется значение первого бита передаваемых данных. К этому моменту должны быть сформированы данные на линиях обмена, как ведущего, так и ведомого устройства. По истечении следующего полутакта сигнал SSP_CLK переводится в высокий логический уровень.

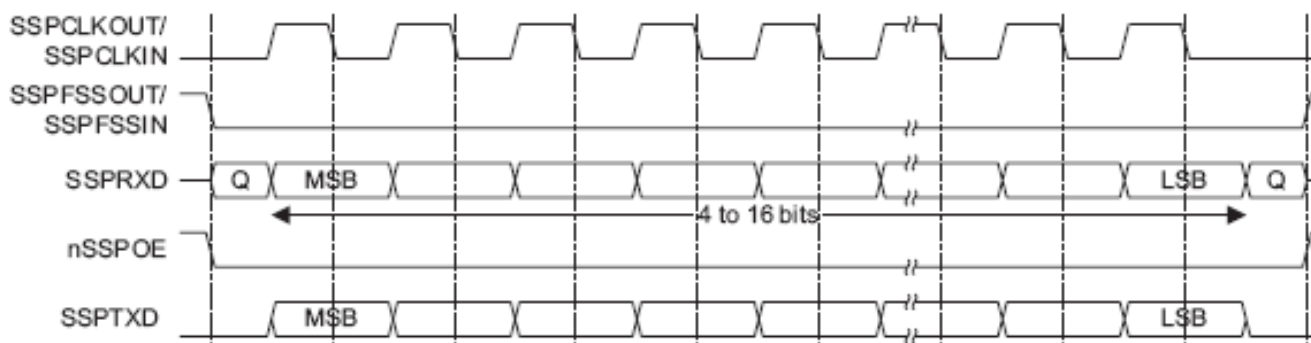
Далее данные регистрируются по переднему фронту и выдаются в линию по заднему фронту сигнала SSP_CLK.

В случае передачи одного слова данных после приема его последнего бита линия SSP_FSS переводится в высокий логический уровень по истечении одного периода тактового сигнала SSP_CLK.

В режиме непрерывной передачи данных на линии SSP_FSS должны формироваться импульсы высокого логического уровня между передачами каждого из слов данных. Это связано с тем, что в режиме SPH=0 линия выбора ведомого устройства в низком уровне блокирует запись в сдвиговый регистр. Поэтому ведущее устройство должно переводить линию SSP_FSS в высокий уровень по окончании передачи каждого кадра, разрешая, таким образом, запись новых данных. По окончании приема последнего бита блока данных линия SSP_FSS переводится в состояние, соответствующее режиму ожидания, по истечении одного такта сигнала SSP_CLK.

28.6.15.2 Формат синхронного обмена SPI фирмы Motorola, SPO=0, SPH=1

Временные диаграммы последовательного синхронного обмена в режиме SPI с SPO=0, SPH=1 показывает рисунок 100 - одиночный и непрерывный обмен.



Q – сигнал с неопределенным уровнем

Рисунок 100 – Формат синхронного обмена протокола SPI, SPO=0, SPH=1

В данном режиме во время ожидания приемопередатчика:

- сигнал SSP_CLK имеет низкий логический уровень;
- сигнал SSP_FSS имеет высокий логический уровень;
- сигнал SSP_TXD переводится в высокоимпедансное состояние.

Если работа модуля разрешена и в буфере FIFO передатчика содержатся корректные данные, сигнал SSP_FSS переводится в низкий логический уровень, что указывает на начало обмена данными и разрешает передачу данных от ведомого устройства на входную линию SSP_RXD ведущего. Выходной контакт передатчика SSPTXD переходит из высокоимпедансного в активное состояние.

По истечении полутакта сигнала SSP_CLK на линиях обмена, как ведущего, так и ведомого устройств будут сформированы значения первых бит передаваемых данных. В это же время включается линия SSP_CLK и на ней формируется передний фронт сигнала.

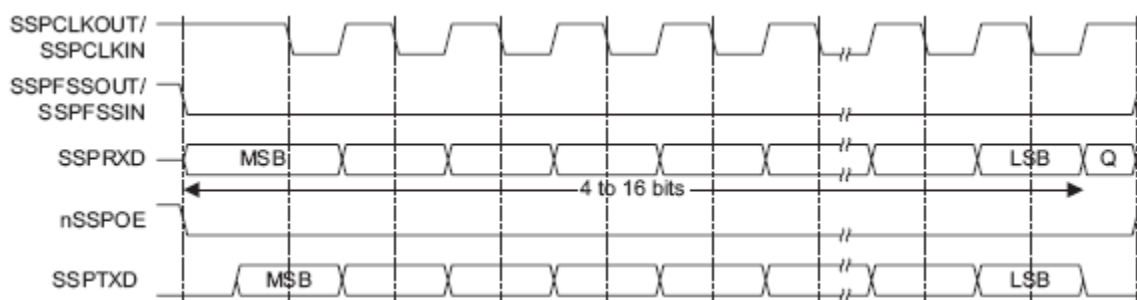
Далее данные регистрируются по заднему фронту и выдаются в линию по переднему фронту сигнала SSP_CLK.

В случае передачи одного слова данных после приема его последнего бита линия SSP_FSS переводится в высокий логический уровень по истечении одного периода тактового сигнала SSP_CLK.

В режиме непрерывной передачи данных линия SSP_FSS постоянно находится в низком логическом уровне, и переводится в высокий уровень по окончании приема последнего бита блока данных, как и в режиме передачи одного слова.

28.6.15.3 Формат синхронного обмена SPI фирмы Motorola, SPO=1, SPH=0

Временные диаграммы последовательного синхронного обмена в режиме SPI с SPO=1, SPH=0 показаны на рисунках: рисунок 101 – одиночный обмен и рисунок 102 – непрерывный обмен.



Q – сигнал с неопределенным уровнем

Рисунок 101 – Формат синхронного обмена протокола SPI, SPO=1, SPH=0 (одиночный обмен)

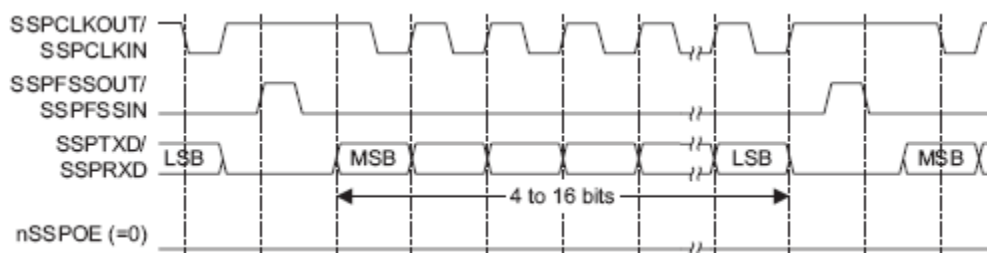


Рисунок 102 – Формат синхронного обмена протокола SPI, SPO=1, SPH=0 (непрерывный обмен)

В данном режиме во время ожидания приемопередатчика:

- сигнал SSP_CLK имеет высокий логический уровень;
- сигнал SSP_FSS имеет высокий логический уровень;
- сигнал SSP_TXD переводится в высокоимпедансное состояние.

Если работа модуля разрешена и в буфере FIFO передатчика содержатся корректные данные, сигнал SSP_FSS переводится в низкий логический уровень, что указывает на начало обмена данными и разрешает передачу данных от ведомого устройства на входную линию SSP_RXD ведущего. Выходной контакт передатчика SSPTXD переходит из высокоимпедансного в активное состояние.

По истечении полутакта сигнала SSP_CLK, на линии SSP_TXD формируется значение первого бита передаваемых данных. К этому моменту должны быть сформированы данные на линиях обмена, как ведущего, так и ведомого устройства. По истечении следующего полутакта сигнал SSP_CLK переводится в низкий логический уровень.

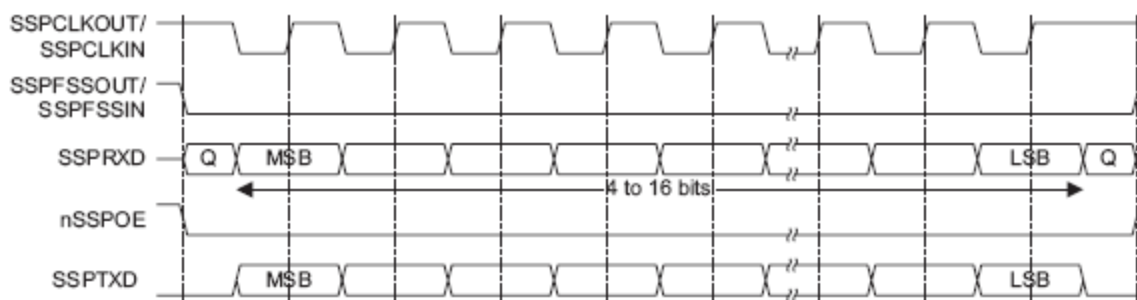
Далее данные регистрируются по заднему фронту и выдаются в линию по переднему фронту сигнала SSP_CLK.

В случае передачи одного слова данных после приема его последнего бита линия SSP_FSS переводится в высокий логический уровень по истечении одного периода тактового сигнала SSP_CLK.

В режиме непрерывной передачи данных на линии SSP_FSS должны формироваться импульсы высокого логического уровня между передачами каждого из слов данных. Это связано с тем, что в режиме SPH=0 линия выбора ведомого устройства в низком уровне блокирует запись в сдвиговый регистр. Поэтому ведущее устройство должно переводить линию SSP_FSS в высокий уровень по окончании передачи каждого кадра, разрешая, таким образом, запись новых данных. По окончании приема последнего бита блока данных линия SSP_FSS переводится в состояние, соответствующее режиму ожидания, по истечении одного такта сигнала SSP_CLK.

28.6.15.4 Формат синхронного обмена SPI фирмы Motorola, SPO=1, SPH=1

Временные диаграммы последовательного синхронного обмена в режиме SPI с SPO=1, SPH=1 показывает рисунок 103 – одиночный и непрерывный обмен.



Q – сигнал с неопределенным уровнем

Рисунок 103 – Формат синхронного обмена протокола SPI, SPO=1, SPH=1

В данном режиме во время ожидания приемопередатчика:

- сигнал SSP_CLK имеет высокий логический уровень;
- сигнал SSP_FSS имеет высокий логический уровень;
- сигнал SSP_TXD переводится в высокоимпедансное состояние.

Если работа модуля разрешена и в буфере FIFO передатчика содержатся корректные данные, сигнал SSP_FSS переводится в низкий логический уровень, что указывает на начало обмена данными и разрешает передачу данных от ведомого устройства на входную линию SSP_RXD ведущего. Выходной контакт передатчика SSP_TXD переходит из высокоимпедансного в активное состояние.

По истечении полутакта сигнала SSP_CLK на линиях обмена, как ведущего, так и ведомого устройств сформированы значения первых бит передаваемых данных. В это же время включается линия SSP_CLK и на ней формируется передний фронт сигнала.

Далее данные регистрируются по переднему фронту и выдаются в линию по заднему фронту сигнала SSP_CLK.

В случае передачи одного слова данных после приема его последнего бита линия SSP_FSS переводится в высокий логический уровень по истечении одного периода тактового сигнала SSP_CLK.

В режиме непрерывной передачи данных линия SSP_FSS постоянно находится в низком логическом уровне и переводится в высокий уровень по окончании приема последнего бита блока данных, как и в режиме передачи одного слова.

28.6.16 Формат синхронного обмена Microwire фирмы National Semiconductor

Временные диаграммы последовательного синхронного обмена в режиме Microwire показаны на рисунке 104 – одиночный обмен и на рисунке 105 – непрерывный обмен.

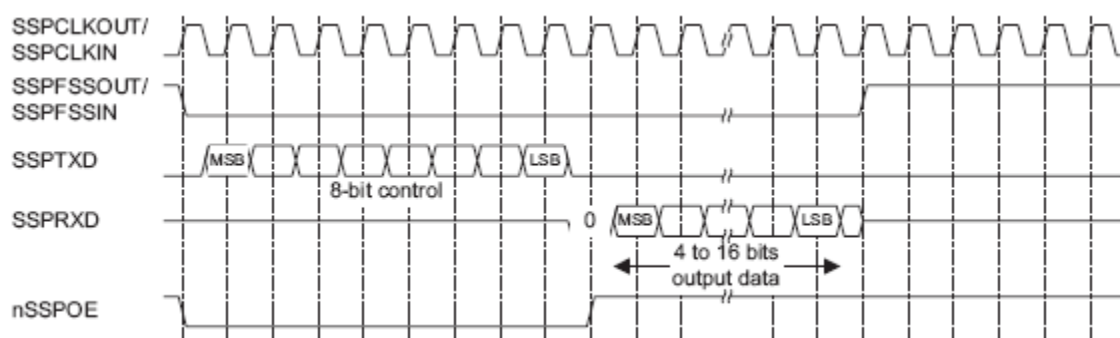


Рисунок 104 – Формат синхронного обмена протокола Microwire (одиночный обмен)

Протокол передачи данных Microwire во многом схож с протоколом SPI, за исключением того, что обмен в нем осуществляется в полудуплексном режиме, с использованием служебных последовательностей. Каждая информационный обмен начинается с передачи ведущим устройством специальной 8-битной управляющей последовательности. В течение всего времени ее передачи приемник не обрабатывает каких-либо входных данных. После того, как сигнал передан и декодирован ведомым устройством, оно выдерживает паузу в один тактовый интервал после передачи последнего бита управляющей последовательности, после чего передает в адрес ведущего устройства запрошенные данные. Длительность блока данных от ведомого устройства может составлять от 4 до 16 бит, таким образом, общая длительность информационного кадра составляет от 13 до 25 бит.

В данном режиме во время ожидания приемопередатчика:

- сигнал SSP_CLK имеет низкий логический уровень;
- сигнал SSP_FSS имеет высокий логический уровень;
- сигнал SSP_TXD переводится в высокоимпедансное состояние.

Переход в режим информационного обмена происходит после записи управляющего байта в буфер FIFO передатчика. По заднему фронту сигнала SSP_FSS данные из буфера переносятся в регистр сдвига блока передатчика, откуда, начиная со старшего значащего разряда, последовательно выдаются в линию SSP_TXD. Линия SSP_FSS остается в низком логическом уровне в течение всей передачи кадра. Линия SSP_RXD при этом находится в высокоимпедансном состоянии.

Внешнее ведомое устройство осуществляет прием бит данных по переднему фронту сигнала SSP_CLK. По окончании приема последнего бита управляющей последовательности она декодируется в течение одного тактового интервала, после чего ведомое устройство передает запрошенные данные в адрес модуля SSP. Биты данных выдаются в линию SSP_RXD по заднему фронту сигнала SSP_CLK. Ведущее устройство, в свою очередь, регистрирует их по переднему фронту этого тактового сигнала. В случае одиночного информационного обмена по окончании приема последнего бита слова данных сигнал SSP_FSS переводится в высокий уровень на время, соответствующее одному тактовому интервалу, что служит командой для переноса принятого слова данных из регистра сдвига в буфер FIFO приемника.

Примечание – Внешнее устройство может перевести линию приемника в третье состояние по заднему фронту сигнала SSP_CLK после приема последнего бита слова данных, либо после перевода линии SSP_FSS в высокий логический уровень.

Непрерывный обмен данными начинается и заканчивается также, как и одиночный обмен. Однако линия SSP_FSS удерживается в низком логическом уровне в течение всего сеанса передачи данных. Управляющий байт следующего информационного кадра передается сразу же после приема младшего значащего разряда текущего кадра. Данные из сдвигового регистра передаются в буфер приемника после регистрации младшего разряда очередного слова по заднему фронту сигнала SSP_CLK.

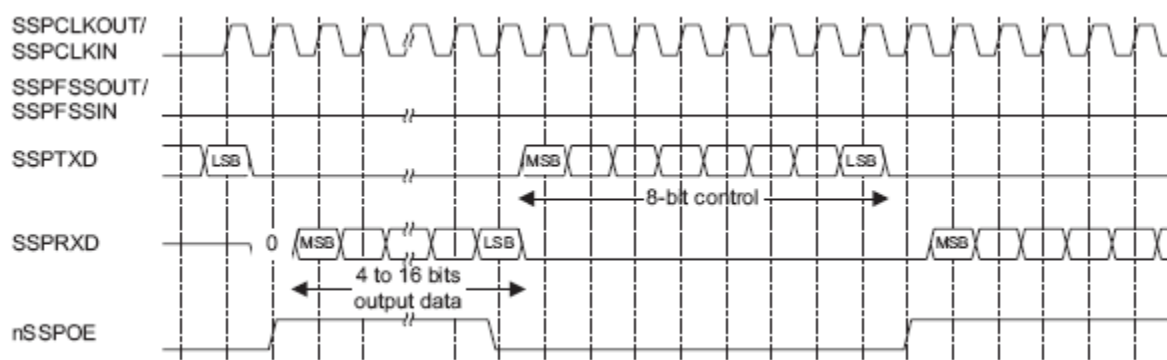


Рисунок 105 – Формат синхронного обмена протокола Microwire (непрерывный обмен)

28.6.16.1 Требования к временным параметрам сигнала SSP_FSS относительно тактового сигнала SSP_CLK в режиме Microwire

Модуль SSP, работающий в режиме Microwire как ведомое устройство, регистрирует данные по переднему фронту сигнала SSP_CLK после установки сигнала SSP_FSS в низкий логический уровень. Ведущие устройства, формирующие сигнал SSP_CLK, должны гарантировать достаточное время установки и удержания сигнала SSP_FSS по отношению к переднему фронту сигнала SSP_CLK.

Данные требования иллюстрирует рисунок 106. По отношению к переднему фронту сигнала SSP_CLK, по которому осуществляется регистрация данных в приемнике ведомого модуля SSP, время установки сигнала SSP_FSS должно быть как минимум в два раза больше периода SSP_CLK, на котором работает модуль. По отношению к предыдущему переднему фронту сигнала SSP_CLK должно обеспечиваться время удержания не менее одного периода этого тактового сигнала.

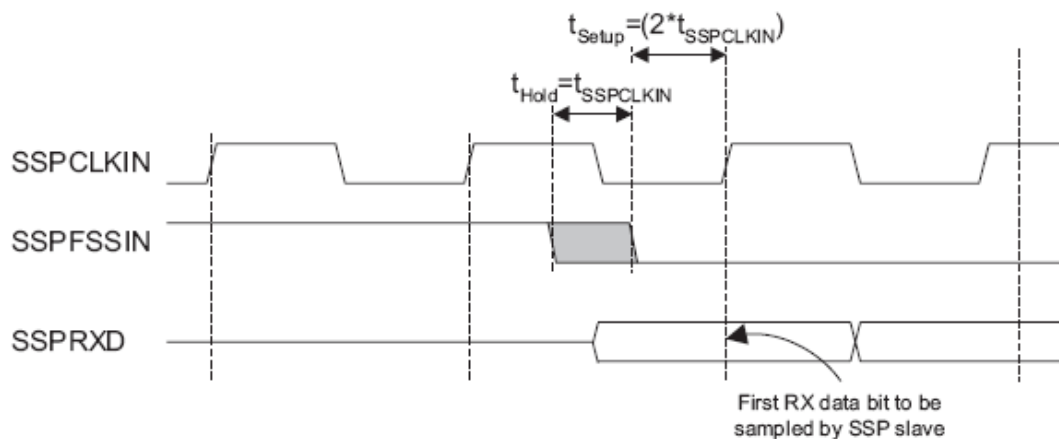


Рисунок 106 – Формат Microwire, требования к времени установки и удержания сигнала

28.6.17 Примеры конфигурации модуля в ведущем и ведомом режимах

На рисунках 107 – 109 приведены варианты подключения модуля SSP к периферийным устройствам, работающим в ведущем или ведомом режиме.

Примечание – Модуль SSP не поддерживает динамическое изменение режима «ведущий – ведомый». Каждый приемопередатчик должен быть изначально сконфигурирован в одном из этих режимов.

На рисунке 107 приведена совместная работа трех модулей SSP, один из которых сконфигурирован в качестве ведущего, а два – в качестве ведомых устройств. Ведущее устройство способно передавать данные циркулярно в адрес двух ведомых по линии SSP_TXD.

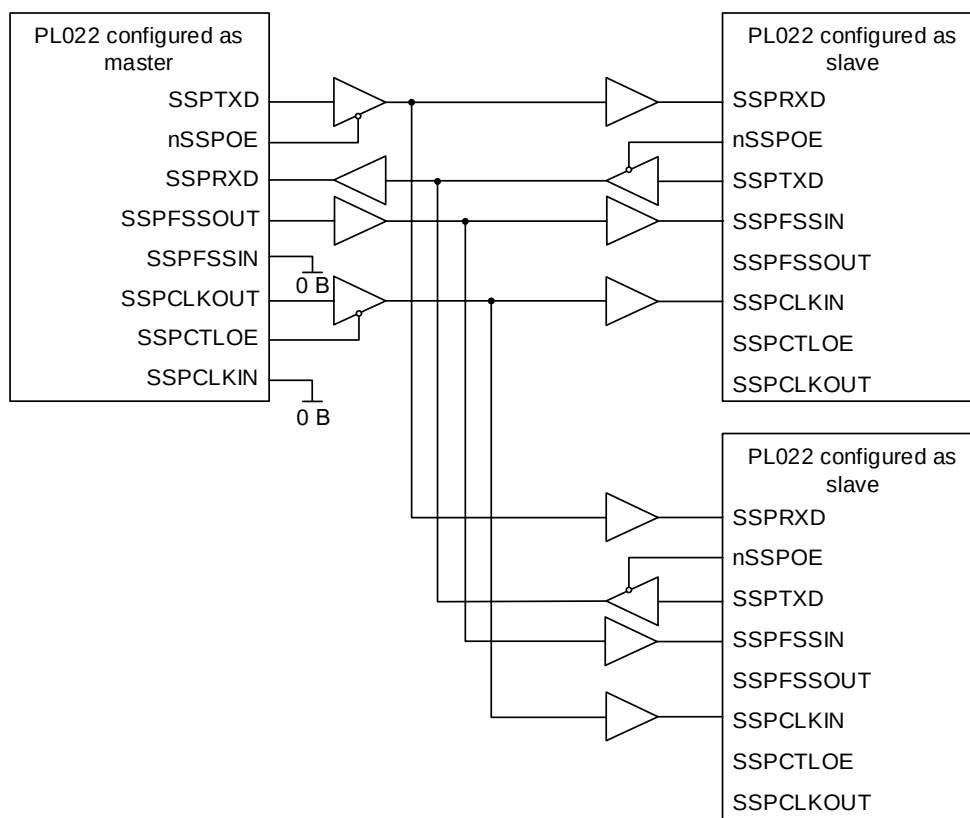


Рисунок 107 – Ведущее устройство SSP подключено к двум ведомым

Для ответной передачи данных один из ведомых модулей разрешает прохождение сигнала от своей линии SSP_TXD на вход SSP_RXD ведущего.

На рисунке 108 приведено подключение модуля SSP, сконфигурированного как ведущее устройство, к двум ведомым устройствам, поддерживающим протокол SPI фирмы Motorola. Внешние устройства сконфигурированы как ведомые путем установки в низкий логический уровень сигнала выбора ведомого устройства Slave Select (SS). Как и в предыдущем примере, ведущее устройство способно передавать данные в адрес ведомых циркулярно по линии SSP_TXD. Ответная передача данных на входную линию SSP_RXD ведущего устройства одновременно осуществляется только одним из ведомых по соответствующей линии MISO.

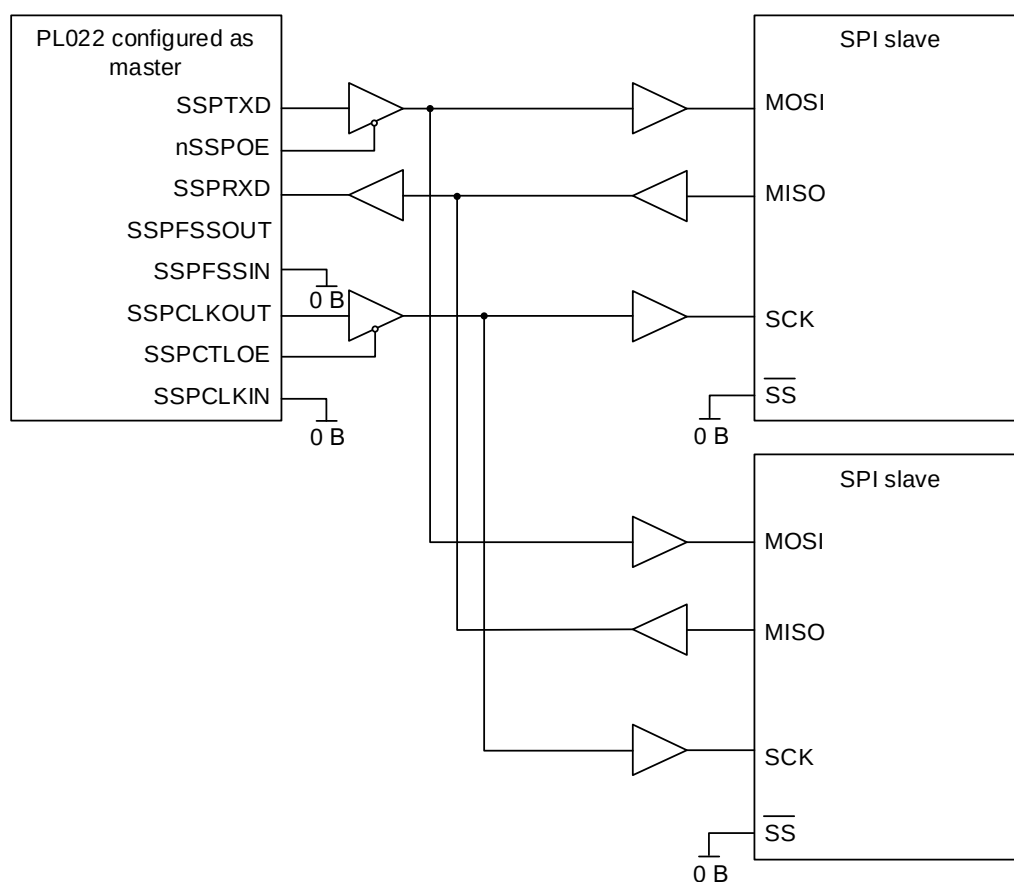
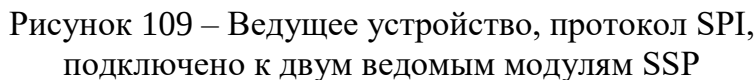


Рисунок 108 – Ведущее устройство SSP подключено к двум ведомым, поддерживающим SPI

Рисунок 109 показывает ведущее устройство, поддерживающее протокол SPI фирмы Motorola, соединенное с двумя модулями SSP, сконфигурированными для работы в ведомом режиме. Линия Slave Select (SS) ведущего устройства в этом случае установлена в высокий логический уровень. Ведущее устройство осуществляет передачу данных по линии MOSI циркулярно в адрес двух ведомых модулей.

Для ответной передачи данных один из ведомых модулей переводит линию SSP_TXD в активное состояние, разрешая, таким образом, прохождение сигнала от своей линии SSP_TXD на вход SSP_RXD ведущего.



Модуль SSP предоставляет интерфейс подключения к контроллеру прямого доступа к памяти. Работа в данном режиме контролируется регистром управления DMA IACR.

Для приема:

- Для передачи:

- 362

– SSPTXDMACLR – сброс запроса на DMA, инициируется контроллером DMA с целью сброса принятого запроса. В случае если был запрошен блочный обмен данными, сигнал сброса формируется в ходе передачи последнего символа данных в блоке.

Сигналы блочного и одноэлементного обмена данными не являются взаимоисключающими, они могут быть инициированы одновременно. Например, в случае, если заполнение данными буфера приемника превышает пороговое значение четыре, формируются как сигнал запроса одноэлементного обмена, так и сигнал запроса блочного обмена данными. В случае если количество данных в буфере приема меньше порогового значения, формируется только запрос одноэлементного обмена. Это бывает полезно в ситуациях, при которых объем данных меньше размера блока. Пусть, например, нужно принять 19 символов. Тогда контроллер DMA осуществит четыре передачи блоков по четыре символа, а оставшиеся три символа передаст в ходе трех одноэлементных обменов.

Примечание – Для оставшихся трех символов контроллер SSP не инициирует процедуру блочного обмена.

Каждый инициированный приемопередатчиком сигнал запроса DMA остается активным до момента его сброса соответствующим сигналом DMACLR.

После снятия сигнала сброса модуль приемопередатчика вновь получает возможность сформировать запрос на DMA в случае выполнения описанных выше условий. Все запросы DMA снимаются после запрета работы приемопередатчика, а также в случае снятия сигнала разрешения DMA.

В таблице 337 приведены значения порогов заполнения буферов приемника и передатчика, необходимых для срабатывания запросов блочного обмена DMABREQ.

Таблица 337 – Параметры срабатывания запросов блочного обмена данными в режиме DMA

Пороговый уровень	Длина блока обмена данными	
	Буфер передатчика (количество незаполненных ячеек)	Буфер приемника (количество заполненных ячеек)
1/2	4	4

Рисунок 110 показывает временные диаграммы одноэлементного и блочного запросов DMA, в том числе действие сигнала DMACLR. Все сигналы должны быть синхронизированы с PCLK.

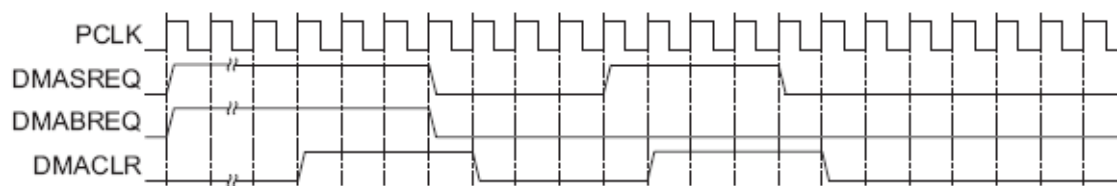


Рисунок 110 – Временные диаграммы обмена в режиме DMA

28.7 Программное управление модулем

28.7.1 Общая информация

В микросхеме реализовано два модуля SSP, базовые адреса каждого модуля указаны далее в таблице 338. Смещение каждого регистра относительно базового адреса постоянно. Следующие адреса являются резервными и не должны использоваться в нормальном режиме функционирования:

- адреса со смещениями в диапазоне +0x028 ... +0x07C и +0xFD0 ... +0xFDC зарезервированы для перспективных расширений возможностей модуля;
- адреса со смещениями в диапазоне +0x080 ... +0x088 зарезервированы для тестирования.

28.7.2 Описание регистров контроллера SSP

Данные о регистрах модуля SSP приведены в таблице 338.

Таблица 338 – Обобщенные данные о регистрах модуля SSP

Базовый адрес	Наименование	Тип	Значение после сброса	Размер, бит	Описание
0x4004_0000	MDR_SSP1				Контроллер SSP1
0x400A_0000	MDR_SSP2				Контроллер SSP2
Смещение					
0x000	CR0	RW	0x0000	16	Регистр MDR_SSPx->CR0MDR_SSPx->CR0 управления 0
0x004	CR1	RW	0x0	4	Регистр MDR_SSPx->CR1MDR_SSPx->CR1 управления 1
0x008	DR	RW	0x----	16	Буфера FIFO приемника (чтение) Буфер FIFO передатчика (запись) MDR_SSPx->DR
0x00C	SR	RO	0x03	3	Регистр MDR_SSPx->SR состояния
0x010	CPSR	RW	0x00	8	Регистр MDR_SSPx->CPSR делителя тактовой частоты
0x014	IMSC	RW	0x0	4	Регистр MDR_SSPx->IMSC маски прерывания
0x018	RIS	RO	0x8	4	Регистр MDR_SSPx->RIS состояния прерываний без учета маскирования
0x01C	MIS	RO	0x0	4	Регистр MDR_SSPx->MIS состояния прерываний с учетом маскирования
0x020	ICR	WO	0x0	4	Регистр MDR_SSPx->ICR сброса прерывания
0x024	DMACR	RW	0x0	2	Регистр MDR_SSPx->DMACR управления прямым доступом к памяти
Примечание – В поле «тип» указан вид доступа к регистру: RW – чтение и запись, RO – только чтение, WO – только запись					

28.7.2.1 MDR_SSPx->CR0

Регистр управления 0

Регистр CR0 содержит пять битовых полей, предназначенных для управления блоками модуля SSP. Назначение разрядов регистра представлено в таблице 339.

Таблица 339 – Формат регистра CR0

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...16	-	Зарезервировано
15...8	SCR	Скорость последовательного обмена. Значение поля SCR используется при формировании тактового сигнала обмена данными. Информационная скорость удовлетворяет соотношению: $F_SSPCLK / (CPSDVR \cdot (1 + SCR))$, где CPSDVR – четное число в диапазоне от 2 до 254 (см. регистр SSPCPSR), а SCR – число от 0 до 255
7	SPH	Фаза сигнала SSPCLKOUT (используется только в режиме обмена SPI фирмы Motorola). См. раздел «Формат SPI фирмы Motorola»
6	SPO	Полярность сигнала SSPCLKOUT (используется только в режиме обмена SPI фирмы Motorola). См. пункт 28.6.15 «Формат синхронного обмена SPI фирмы Motorola»
5, 4	FRF	Формат информационного кадра. 00 – протокол SPI фирмы Motorola; 01 – протокол SSI фирмы Texas Instruments; 10 – протокол Microwire фирмы National Semiconductor; 11 – резерв
3...0	DSS	Размер слова данных: 0000 – резерв 0001 – резерв 0010 – резерв 0011 – 4 бита 0100 – 5 бит 0101 – 6 бит 0110 – 7 бит 0111 – 8 бит 1000 – 9 бит 1001 – 10 бит 1010 – 11 бит 1011 – 12 бит 1100 – 13 бит 1101 – 14 бит 1110 – 15 бит 1111 – 16 бит

28.7.2.2 MDR_SSPx->CR1

Регистр управления 1

Регистр CR1 содержит четыре битовых поля, предназначенных для управления блоками модуля SSP. Назначение разрядов регистра представлено в таблице 340.

Таблица 340 – Регистр CR1

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
15...4		Резерв, при чтении результат не определен. При записи следует устанавливать в 0
3	SOD	Запрет выходных линий в режиме ведомого устройства. Бит используется только в режиме ведомого устройства (MS=1). Это позволяет организовать двусторонний обмен данными в системах, содержащих одно ведущее и несколько ведомых устройств. Бит SOD следует установить в случае, если данный ведомый модуль SSP не должен в настоящее время осуществлять передачу данных в линию SSP_TXD. При этом линии обмена данными ведомых устройств можно соединить параллельно. 0 – управление линией SSP_TXD в ведомом режиме разрешена. 1 – управление линией SSP_TXD в ведомом режиме запрещена
2	MS	Выбор ведущего или ведомого режима работы: 0 – ведущий модуль (устанавливается по умолчанию); 1 – ведомый модуль
1	SSE	Разрешение работы приемопередатчика: 0 – работа запрещена; 1 – работа разрешена
0	LBM	Тестирование по шлейфу: 0 – нормальный режим работы приемопередатчика; 1 – выход регистра сдвига передатчика соединен со входом регистра сдвига приемника

28.7.2.3 MDR_SSPx->DR

Регистр данных

Регистр SSPDR имеет разрядность 16 бит и предназначен для чтения принятых и записи передаваемых данных.

Операция чтения обеспечивает доступ к последней несчитанной ячейке буфера FIFO приемника. Запись данных в этот буфер FIFO осуществляет блок приемника.

Операция записи позволяет занести очередное слово в буфер FIFO передатчика. Извлечение данных из этого буфера осуществляет блок передатчика. При этом извлеченные данные помещаются в регистр сдвига передатчика, откуда последовательно выдаются на линию SSP_TXD с заданной скоростью информационного обмена.

В случае если выбран размер информационного слова менее 16 бит, перед записью в регистр SSPDR необходимо обеспечить выравнивание данных по правой границе. Блок передатчика игнорирует неиспользуемые биты. Принятые информационные слова автоматически выравниваются по правой границе в блоке приемника.

В режиме обмена данными Microwire фирмы National Semiconductor модуль SSP по умолчанию работает с восьмиразрядными информационными словами (старший значащий байт игнорируется). Размер принимаемых данных задается программно. Буфера FIFO приемника и передатчика автоматически не очищаются даже в случае, если бит SSE установлен в 0. Это позволяет заполнить буфер передатчика необходимой информацией заблаговременно, перед разрешением работы модуля.

Назначение разрядов регистра SSPDR описано в следующей таблице 341.

Таблица 341 – Формат регистра DR

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
15...0	DATA	Принимаемые данные (чтение). Передаваемые данные (запись). В случае если выбран размер информационного слова менее 16 бит, перед записью в регистр SSPDR необходимо обеспечить выравнивание данных по правой границе. Блок передатчика игнорирует неиспользуемые биты. Принятые информационные слова автоматически выравниваются по правой границе в блоке приемника

28.7.2.4 MDR_SSPx->SR

Регистр состояния

Регистр состояния доступен только для чтения и содержит информацию о состоянии буферов FIFO приемника и передатчика и занятости модуля SSP.

Назначение бит регистра SSPSR представлено в таблице 342.

Таблица 342 – Регистр SR

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
15...5		Резерв, при чтении результат не определен.
4	BSY	Флаг активности модуля: 0 – модуль SSP не активен; 1 – модуль SSP в настоящее время передает и/или принимает данные, либо буфер FIFO передатчика не пуст
3	RFF	Буфер FIFO приемника заполнен: 0 – не заполнен; 1 – заполнен
2	RNE	Буфер FIFO приемника не пуст: 0 – пуст; 1 – не пуст
1	TNF	Буфер FIFO передатчика не заполнен: 0 – заполнен; 1 – не заполнен
0	TFE	Буфер FIFO передатчика пуст: 0 – не пуст; 1 – пуст

28.7.2.5 MDR_SSPx->CPSR

Регистр делителя тактовой частоты

Регистр SSPCPSR используется для установки параметров делителя тактовой частоты. Записываемое значение должно быть целым числом в диапазоне от 2 до 254. Младший значащий разряд регистра принудительно устанавливается в ноль. Если записать в регистр SSPCPSR нечетное число, его последующее чтение даст результатом это число, но с установленным в ноль младшим битом.

Таблица 343 отображает назначение бит регистра SSPSR.

Таблица 343 – Регистр CPSR

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...8	-	Резерв. При чтении результат не определен. При записи следует заполнить нулями
7... 0	CPSDVSR	Коэффициент деления тактовой частоты. Записываемое значение должно быть целым числом в диапазоне от 2 до 254. Младший значащий разряд регистра принудительно устанавливается в ноль

28.7.2.6 MDR_SSPx->IMSC

Регистр установки и сброса маски прерывания

При чтении выдается текущее значение маски. При записи производится установка или сброс маски на соответствующее прерывание. При этом запись 1 в разряд разрешает соответствующее прерывание, запись 0 – запрещает.

После сброса все биты регистра маски устанавливаются в нулевое состояние.

Назначение бит регистра IMSC показано в таблице 344.

Таблица 344 – Регистр IMSC

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...4		Резерв. Не модифицируйте. При чтении выдаются нули
3	TXIM	Маска прерывания по заполнению на 50 % и менее буфера FIFO передатчика. 1 – не маскирована. 0 – маскирована
2	RXIM	Маска прерывания по заполнению на 50 % и более буфера FIFO приемника. 1 – не маскирована. 0 – маскирована
1	RTIM	Маска прерывания по таймауту приемника (буфер FIFO приемника не пуст и не было попыток его чтения в течение времени таймаута). 1 – не маскирована. 0 – маскирована
0	RORIM	Маска прерывания по переполнению буфера приемника. 1 – не маскирована. 0 – маскирована

28.7.2.7 MDR_SSPx->RIS

Регистр состояния прерываний

Этот регистр доступен только для чтения и содержит текущее состояние прерываний без учета маскирования. Данные, записываемые в регистр, игнорируются.

Таблица 345 отображает назначение бит в регистре RIS.

Таблица 345 – Регистр RIS

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31... 4		Резерв. Не модифицируйте. При чтении выдаются нули
3	TXRIS	Состояние до маскирования прерывания SSPTXINTR
2	RXRIS	Состояние до маскирования прерывания SSPRXINTR
1	RTRIS	Состояние до маскирования прерывания SSPRTINTR
0	RORRIS	Состояние до маскирования прерывания SSPRORINTR

28.7.2.8 MDR_SSPx->MIS

Регистр маскированного состояния прерываний

Этот регистр доступен только для чтения и содержит текущее состояние прерываний с учетом маскирования. Данные, записываемые в регистр, игнорируются.

Назначение бит в регистре SSPMIS представлено в таблице 346.

Таблица 346 – Регистр MIS

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...4		Резерв. Не модифицируйте. При чтении выдаются нули
3	TXMIS	Состояние маскированного прерывания SSPTXINTR
2	RXMIS	Состояние маскированного прерывания SSPRXINTR
1	RTMIS	Состояние маскированного прерывания SSPRTINTR
0	RORMIS	Состояние маскированного прерывания SSPRORINTR

28.7.2.9 MDR_SSPx->ICR

Регистр сброса прерываний

Этот регистр доступен только для записи и предназначен для сброса признака прерывания по заданному событию путем записи 1 в соответствующий бит. Запись в любой из разрядов регистра 0 игнорируется.

Назначение бит в регистре SSPICR представлено в таблице 347.

Таблица 347 – Регистр ICR

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31... 2		Резерв. Не модифицируйте. При чтении выдаются нули
1	RTIC	Сброс прерывания SSPRTINTR
0	RORIC	Сброс прерывания SSPRORINTR

28.7.2.10 MDR_SSPx->DMACR

Регистр управления прямым доступом к памяти

Регистр доступен по чтению и записи. После сброса все биты регистра обнуляются.

Назначение бит регистра UARTDMACR представлено в таблице 348.

Таблица 348 – Регистр DMACR

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...2		Резерв. Не модифицируйте. При чтении выдаются нули.
1	TXDMAE	Использование DMA при передаче. Если бит установлен в 1, разрешено формирование запросов DMA для обслуживания буфера FIFO передатчика
0	RXDMAE	Использование DMA при приеме. Если бит установлен в 1, разрешено формирование запросов DMA для обслуживания буфера FIFO приемника

28.8 Прерывания

В модуле предусмотрено пять маскируемых линий запроса на прерывание с выводом на один общий сигнал, представляющий собой комбинацию независимых по схеме ИЛИ.

Сигналы запроса на прерывание:

- SSPRXINTR – запрос на обслуживание буфера FIFO приемника;
- SSPTXINTR – запрос на обслуживание буфера FIFO передатчика;
- SSPRORINTR – переполнение буфера FIFO приемника;
- SSPRTINTR – таймаут приемника;
- SSPINTR – логическое ИЛИ сигналов SSPRXINTR, SSPTXINTR, SSPRTINTR и SSPRORINTR.

Каждый из независимых сигналов запроса на прерывание может быть маскирован путем установки соответствующего бита в регистре маски SSPIMSC. Установка бита в 1 разрешает соответствующее прерывание, а в 0 – запрещает.

Доступность индивидуальных линий и общей линии запроса позволяет организовать обслуживание прерываний в системе, как путем применения глобальной процедуры обработки, так и с помощью драйвера устройства, построенного по модульному принципу.

Прерывания от приемника и передатчика SSPRXINTR и SSPTXINTR выведены отдельно от прерываний по изменению состояния устройства. Это позволяет использовать данные сигналы запроса для обеспечения чтения и записи данных согласованно с достижением заданного порога заполнения буферов FIFO приемника и передатчика.

Признаки возникновения каждого из условий прерывания можно считать либо из регистра прерываний SSPRIS, либо из маскированного регистра прерываний SSPMIS.

28.8.1 SSPRXINTR

Прерывание по заполнению буфера FIFO приемника формируется в случае, если буфер приемника содержит четыре или более несчитанных слов данных.

28.8.2 SSPTXINTR

Прерывание по заполнению буфера FIFO передатчика формируется в случае, если буфер передатчика содержит четыре или менее корректных слов данных.

Состояние прерывания не зависит от значения сигнала разрешения работы модуля SSP. Это позволяет организовать взаимодействие программного обеспечения с передатчиком одним из двух способов. Во-первых, можно записать данные в буфер заблаговременно, перед активизацией передатчика и разрешения прерываний. Во-вторых, можно предварительно разрешить работу модуля и формирование прерываний и заполнять буфер передатчика в ходе работы процедуры обслуживания прерываний.

28.8.3 SSPRORINTR

Прерывание по переполнению буфера FIFO приемника формируется в случае, если буфер уже заполнен и блоком приемника осуществлена попытка записать в него еще одно слово. При этом принятое слово данных регистрируется в регистре сдвига приемника, но в буфер приемника не заносится.

28.8.4 SSPRTINTR

Прерывание по таймауту приемника возникает в случае, если буфер FIFO приемника не пуст, и на вход приемника не поступало новых данных в течение времени таймаута, равного 32 тактам частоты SSPCLKOUT (для ведущего и ведомого режимов работы). Данный механизм гарантирует, что пользователь будет знать о наличии в буфере приемника необработанных данных.

Прерывание по таймауту снимается либо после считывания данных из буфера приемника до его опустошения, либо после приема новых слов данных по входной линии SSP_RXD. Кроме того, оно может быть снято путем записи «1» в бит RTIC регистра сброса прерывания SSPTICR.

28.8.5 SSPINTR

Все описанные сигналы запроса на прерывание скомбинированы в общую линию путем объединения по схеме ИЛИ сигналов SSPRXINTR, SSPTXINTR, SSPRTINTR и SSPRORINTR с учетом маскирования. Общий выход может быть подключен к системному контроллеру прерывания, что позволит ввести дополнительное маскирование запросов на уровне периферийных устройств.

29 Контроллер MDR_UART

Модуль универсального асинхронного приемопередатчика (UART – Universal Asynchronous Receiver-Transmitter) представляет собой периферийное устройство микросхемы.

В состав контроллера включен кодек (ENDEC – ENcoder/DEcoder) последовательного интерфейса инфракрасной (ИК) передачи данных в соответствии с протоколом SIR (SIR – Serial Infra Red) ассоциации Infrared Data Association (IrDA).

29.1 Основные сведения

Основные сведения о модуле представлены в следующих разделах:

- основные характеристики;
- программируемые параметры;
- отличия от приемопередатчика 16C650.

29.1.1 Основные характеристики модуля UART

Может быть запрограммирован для использования, как в качестве универсального асинхронного приемопередатчика, так и для инфракрасного обмена данными (SIR).

Содержит независимые буферы приема (16x12) и передачи (16x8) типа FIFO (First In First Out – первый вошел, первый вышел), что позволяет снизить интенсивность прерываний центрального процессора.

Программное отключение FIFO позволяет ограничить размер буфера одним байтом.

Программное управление скоростью обмена. Обеспечивается возможность деления тактовой частоты опорного генератора в диапазоне (1x16 – 65535x16). Допускается использование нецелых коэффициентов деления частоты, что позволяет использовать любой опорный генератор с частотой более 3,6864 МГц.

Поддержка стандартных элементов асинхронного протокола связи – стартового и стопового бит, а также бита контроля четности, которые добавляются перед передачей и удаляются после приема.

Независимое маскирование прерываний от буфера FIFO передатчика, буфера FIFO приемника, по таймауту приемника, по изменению линий состояния модема, а также в случае обнаружения ошибки.

Поддержка прямого доступа к памяти.

Обнаружение ложных стартовых бит.

Формирование и обнаружения сигнала разрыва линии.

Поддержка функции управления модемом (линии CTS, DCD, DSR, RTS, DTR и RI).

Возможность организации аппаратного управления потоком данных.

Полностью программируемый асинхронный последовательный интерфейс с характеристиками:

- данные длиной 5, 6, 7 или 8 бит;
- формирование и контроль четности (проверочный бит выставляется по четности, нечетности, имеет фиксированное значение, либо не передается);

- формирование 1 или 2 стоповых бит;
- скорость передачи данных – от 0 до UARTCLK/16 Бод.

Кодек ИУ обмена данными IrDA SIR обеспечивает:

- программный выбор обмена данными по линиям асинхронного приемопередатчика либо кода ИК связи IrDA SIR;
- поддержку длительности бит для нормального режима (3/16) и для режима пониженного энергопотребления (1.41 – 2.23 мкс);
- программируемое деление опорной частоты UARTCLK для получения заданной длительности бит в режиме пониженного энергопотребления.

Наличие идентификационного регистра, однозначно идентифицирующего модуль, что позволяет операционной системе выполнять автоматическую конфигурацию.

29.1.2 Программируемые параметры

Следующие ключевые параметры могут быть заданы программно:

- скорость передачи данных – целая и дробная часть числа;
- количество бит данных;
- количество стоповых бит;
- режим контроля четности;
- разрешение или запрет использования буферов FIFO (глубина очереди данных – 16 элементов или один элемент, соответственно);
- порог срабатывания прерывания по заполнению буферов FIFO (1/8, 1/4, 1/2, 3/4 и 7/8);
- частота внутреннего тактового генератора (номинальное значение – 1,8432 МГц) может быть задана в диапазоне 1,42 – 2,12 МГц для обеспечения возможности формирования бит данных с укороченной длительностью в режиме пониженного энергопотребления;
- режим аппаратного управления потоком данных.

29.1.3 Отличия от контроллера UART 16C650

Контроллер отличается от промышленного стандарта асинхронного приемопередатчика 16C650 следующими характеристиками:

- пороги срабатывания прерывания по заполнению буфера FIFO приемника – 1/8, 1/4, 1/2, 3/4 и 7/8;
- пороги срабатывания прерывания по заполнению буфера FIFO передатчика – 1/8, 1/4, 1/2, 3/4 и 7/8;
- отличается распределение адресов внутренних регистров и назначение бит в регистрах;
- недоступны изменения сигналов состояния модема.

Следующие возможности контроллера 16C650 не поддерживаются:

- полуторная длительность стопового бита (поддерживается только 1 или 2 стоповых бита);
- независимое задание тактовой частоты приемника и передатчика.

29.2 Функциональные возможности

Устройство выполняет следующие функции:

- преобразование данных, полученных от периферийного устройства, из последовательной в параллельную форму;
- преобразование данных, передаваемых на периферийное устройство, из параллельной в последовательную форму.

Процессор читает и записывает данные, а также управляющую информацию и информацию о состоянии модуля. Прием и передача данных буферизуются с помощью внутренней памяти FIFO, позволяющей сохранить до 16 байтов независимо для режимов приема и передачи.

Модуль приемопередатчика:

- содержит программируемый генератор, формирующий тактовый сигнал одновременно для передачи и для приема данных на основе внутреннего тактового сигнала UARTCLK;
- обеспечивает возможности, сходные с возможностями промышленного стандарта - контроллера UART 16C650.

Режим работы приемопередатчика и скорость обмена данными контролируются регистром управления линией UARTLCR_N и регистрами делителя скорости передачи данных – целой части (UARTIBRD) и дробной части (UARTFBRD).

Устройство может формировать следующие сигналы:

- независимые маскируемые прерывания от приемника (в том числе по таймауту), передатчика, а также по изменению состояния модема и в случае обнаружения ошибки;
- общее прерывание, возникающее в случае, если возникло одно из независимых немаскированных прерываний;
- сигналы запроса на прямой доступ к памяти (DMA) для совместной работы с контроллером DMA.

В случае возникновения ошибки в структуре сигнала, четности данных, а также разрыва линии соответствующий бит ошибки устанавливается и сохраняется в буфере FIFO. В случае переполнения буфера немедленно устанавливается соответствующий бит в регистре переполнения, а доступ к записи в буфер FIFO блокируется.

Существует возможность программно ограничить размер буфера FIFO одним байтом, что позволяет реализовать общепринятый интерфейс асинхронной последовательной связи с двойной буферизацией.

Поддерживаются входные линии состояния модема: «готовность к приему» (Clear To Send, CTS), «обнаружен информационный сигнал» (Data Carrier Detected, DCD), «источник данных готов» (Data Set Ready, DSR) и «индикатор вызова» (Ring Indicator, RI), а также выходные линии: «запрос на передачу» (Request to Send, RTS) и «приемник данных готов» (Data Terminal Ready, DTR).

Доступна возможность аппаратного управления потоком данных по линиям nUARTCTS и nUARTRTS.

Блок последовательного интерфейса инфракрасной передачи данных в соответствии с протоколом IrDA SIR реализует протокол обмена данными ENDEC. В случае его активизации обмен информацией осуществляется не с помощью сигналов UARTTXD и UARTRXD, а посредством сигналов nSIROUT и SIRIN.

В этом случае устройство переводит линию UARTTXD в пассивное состояние (высокий уровень), и перестает реагировать на изменение состояния модема, а также сигнала на линии UARTRXD. Протокол SIR ENDEC обеспечивает возможность обмена данными исключительно в режиме полудуплекса, то есть он не может передавать во время приема данных и принимать во время передачи данных.

В соответствии со спецификацией физического уровня протокола IrDA SIR, задержка между передачей и приемом должна составлять не менее 10 мс.

29.3 Описание функционирования блока UART

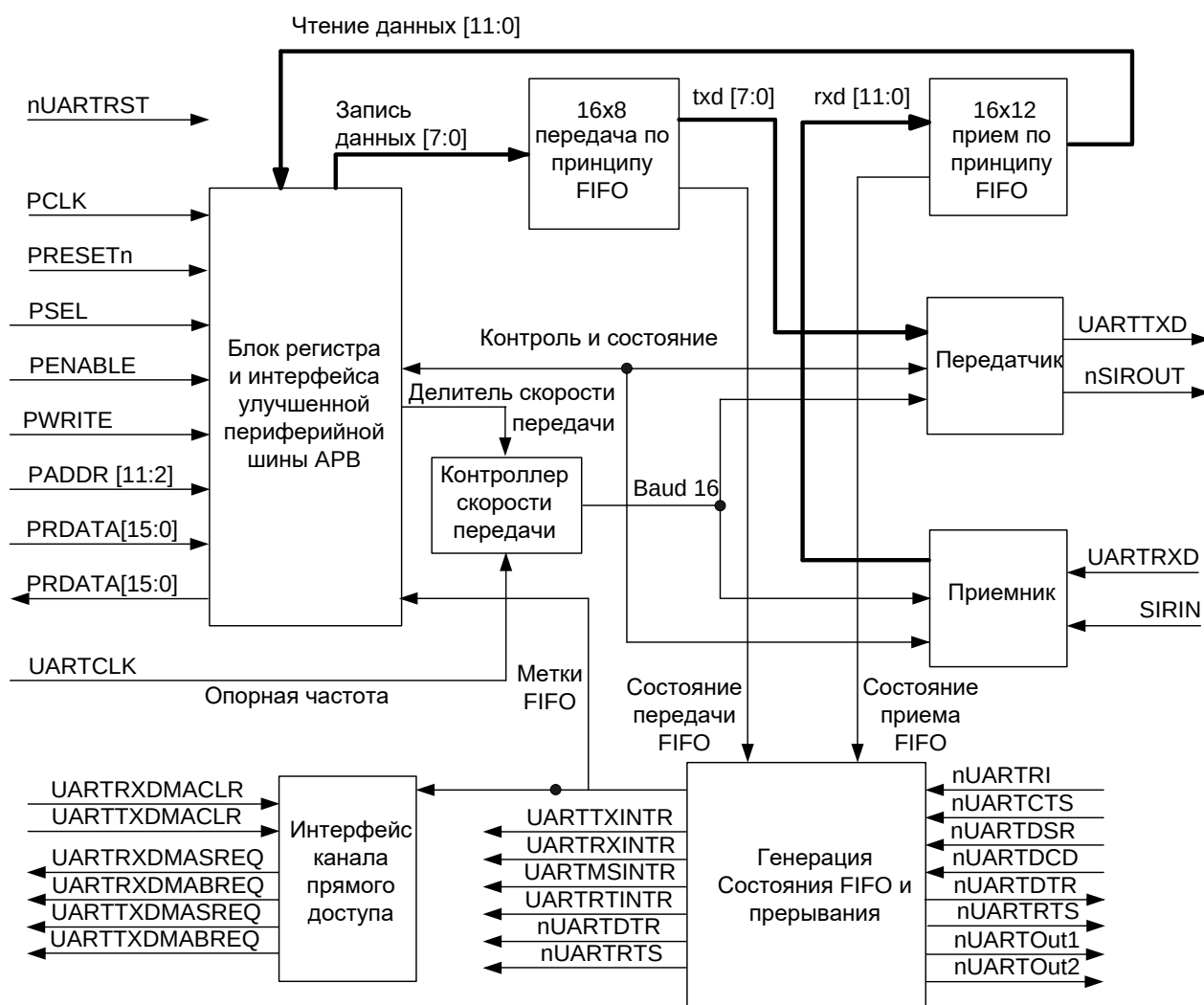


Рисунок 111 – Блок-схема универсального асинхронного приемопередатчика (UART)

29.3.1 Генератор тактового сигнала приемопередатчика

Генератор содержит счетчики без цепи сброса, формирующие внутренние тактовые сигналы Baud16 и IrLPBaud16.

Сигнал Baud16 используется для синхронизации схем управления приемником и передатчиком последовательного обмена данными. Он представляет собой

последовательность импульсов с шириной, равной одному периоду сигнала UARTCLK и частотой, в 16 раз выше скорости передачи данных.

Сигнал IrLPBaud16 предназначен для синхронизации схемы формирования импульсов с длительностью, требуемой для ИК-обмена данными в режиме с пониженным энергопотреблением.

29.3.2 Буфер FIFO передатчика

Буфер передатчика имеет ширину 8 бит, глубину 16 слов, схему организации доступа типа «первый вошел, первый вышел». Данные от центрального процессора, записанные через шину APB, сохраняются в буфере до тех пор, пока не будут считаны логической схемой передачи данных. Существует возможность запретить буфер FIFO передатчика, тогда он будет функционировать как однобайтовый буферный регистр.

29.3.3 Буфер FIFO приемника

Буфер приемника имеет ширину 12 бит, глубину 16 слов, схему организации доступа типа «первый вошел, первый вышел». Принятые от периферийного устройства данные и соответствующие коды ошибки сохраняются логикой приема данных в нем до тех пор, пока не будут считаны центральным процессором через шину APB. Буфер FIFO приемника может быть запрещен, в этом случае он будет действовать как однобайтовый буферный регистр.

29.3.4 Блок передатчика

Логические схемы передатчика осуществляют преобразование данных, считанных из буфера передатчика, из параллельной в последовательную форму. Управляющая логика выдает последовательный поток бит в порядке: стартовый бит, биты данных, начиная с младшего значащего разряда, бит проверки на четность, и, наконец, стоповые биты, в соответствии с конфигурацией, записанной в регистре управления.

29.3.5 Блок приемника

Логические схемы приемника преобразуют данные, полученные от периферийного устройства, из последовательной в параллельную форму после обнаружения корректного стартового импульса. Кроме того, производятся проверки переполнения буфера, проверки на ошибки контроля четности, на ошибки в структуре сигнала, а также на разрыв линии. Признаки обнаружения этих ошибок также сохраняются в выходном буфере.

29.3.6 Блок формирования прерываний

Контроллер генерирует независимые маскируемые прерывания с активным высоким уровнем. Кроме того, формируется комбинированное прерывание путем объединения указанных независимых прерываний по схеме ИЛИ.

Комбинированный сигнал прерывания может быть подан на внешний контроллер прерываний системы, при этом появится дополнительная возможность маскирования устройства в целом, что облегчает построение модульных драйверов устройств.

Другой подход состоит в подаче на системный контроллер прерываний независимых линий запроса на прерывание от приемопередатчика. В этом случае процедура обработки сможет одновременно считать информацию обо всех источниках прерывания. Данный подход привлекателен в случае, если скорость доступа к регистрам периферийных устройств значительно превышает тактовую частоту центрального процессора в системе реального времени.

Для более подробной информации см. подраздел 29.8 «Прерывания».

29.3.7 Интерфейс прямого доступа к памяти

Модуль обеспечивает интерфейс с контроллером DMA согласно схеме взаимодействия приемопередатчика и контроллера DMA.

29.3.8 Блок и регистры синхронизации

Контроллер поддерживает как асинхронный, так и синхронный режимы работы тактовых генераторов CPU_CLK и UARTCLK. Регистры синхронизации и логика квитирования постоянно находятся в активном состоянии. Это практически не отражается на характеристиках устройства и занимаемой площади. Синхронизация сигналов управления осуществляется в обоих направлениях потока данных, то есть как из области действия CPU_CLK в область действия UARTCLK, так и наоборот.

29.4 Описание функционирования ИК кодека IrDA SIR

Структурная схема кодека представлена на рисунке 112.

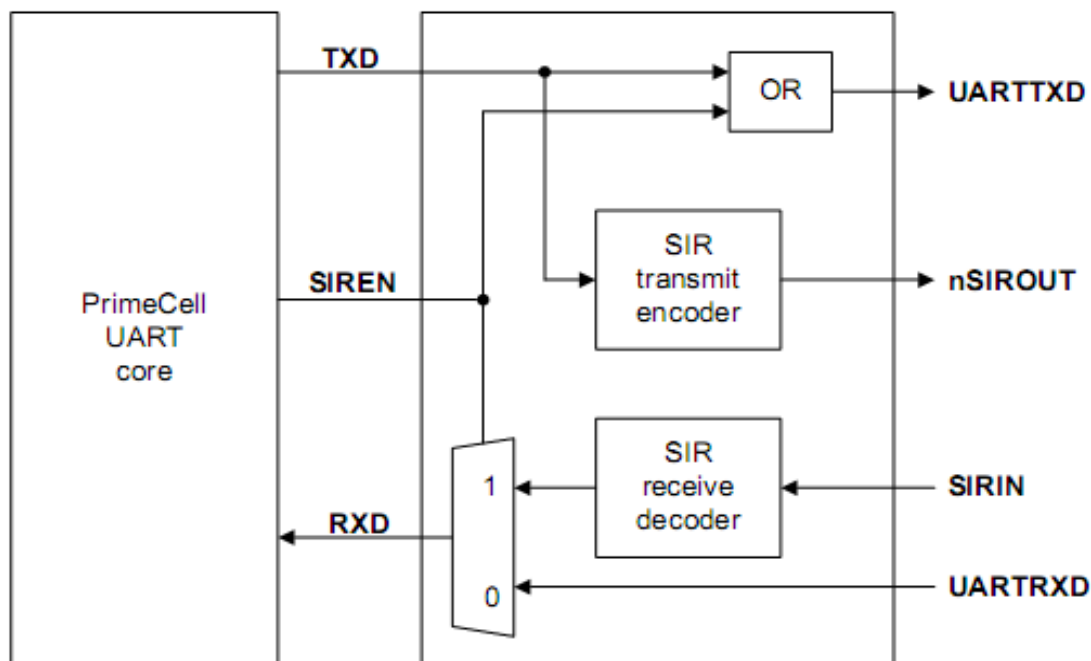


Рисунок 112 – Структурная схема кодека IrDA

29.4.1 Кодер ИК передатчика

Кодер преобразует поток данных с выхода асинхронного передатчика, сформированный по закону модуляции без возврата к нулю (NRZ). Спецификация физического уровня протокола IrDA SIR подразумевает использование модуляции с

возвратом к нулю и инверсией (RZI), в соответствии с которой передача логического нуля соответствует излучению одного светового ИК импульса. Сформированный выходной поток импульсов подается на усилитель и, далее, на ИК светодиод.

Длительность импульса в режиме IrDA составляет, согласно спецификации, три периода внутреннего тактового генератора с частотой Baud16, то есть 3/16 периода времени, выделенного на передачу одного бита.

В режиме IrDA с пониженным энергопотреблением ширина импульса задана как 3/16 периода, выделенного на передачу бита, при скорости передачи данных 115200 бит/с. Данное требование реализуется за счет формирования трех периодов тактового сигнала IrLPBaud16 с номинальной частотой 1,8432 МГц, в свою очередь, формируемого путем деления частоты UARTCLK. Значение частоты IrLPBaud16 задается путем записи соответствующего коэффициента деления частоты в регистр UARTILPR.

Выход кодера имеет активное низкое состояние. При передаче логической «1» выход кодера остается в низком состоянии, при передаче логического «0» – формируется импульс, при этом выход кратковременно переводится в высокое состояние.

Как в нормальном режиме, так и в режиме пониженного энергопотребления использование нецелых значений коэффициента деления скорости передачи данных увеличивает джиттер («дребезжание») фронтов импульсов данных. Наличие джиттера в случае использования дробных коэффициентов деления связано с тем, что интервалы между тактовыми импульсами Baud16 будут нерегулярными – период сигнала Baud16 в разное время будет содержать различное количество периодов сигнала UARTCLK. Можно показать, что в наихудшем случае величина джиттера в потоке ИК-импульсов может достигать трех периодов UARTCLK. В соответствии со спецификацией стандарта SIR, джиттер не должен превышать величины 13 %. В случае, если частота сигнала UARTCLK составляет более 3,6834 МГц, а скорость передачи данных меньше или равна 115200 бит/с, величина джиттера не превышает 9 %. Таким образом, требования стандарта выполняются.

29.4.2 Декодер ИК приемника

Декодер преобразует поток данных, сформированных по закону возврата к нулю, полученного от приемника ИК-сигнала, и выдает поток данных без возврата к нулю на вход приемника UART. В неактивном состоянии вход декодера находится нормально в высоком состоянии. Выходной сигнал кодера имеет полярность, противоположную полярности входа декодера.

Обнаружение стартового бита осуществляется при низком уровне сигнала на входе декодера.

Примечание – Для того, чтобы исключить ложные срабатывания UART от импульсных помех, на входе SIRIN игнорируются импульсы с длительностью менее, чем:

- 3/16 длительности Baud16 в режиме IrDA;
- 3/16 длительности IrLPBaud16 в режиме IrDA с пониженным энергопотреблением.

29.5 Описание работы UART

29.5.1 Сброс модуля

Приемопередатчик и кодек могут быть сброшены общим сигналом сброса процессора. Значения регистров после сброса описаны в подразделе 29.9 «Программное управление модулем».

29.5.2 Тактовые сигналы

Частота тактового сигнала UARTCLK должна обеспечивать поддержку требуемого диапазона скоростей передачи данных:

$$F_{\text{UARTCLK}}(\text{min}) \geq 16 \cdot \text{baud_rate_max};$$

$$F_{\text{UARTCLK}}(\text{max}) \leq 16 \cdot 65535 \cdot \text{baud_rate_min}.$$

Например, для поддержки скорости передачи данных в диапазоне от 110 до 460800 Бод частота UARTCLK должна находиться в интервале от 7,3728 МГц до 115,34 МГц.

Частота UARTCLK, кроме того, должна выбираться с учетом возможности установки скорости передачи данных в рамках заданных требований точности.

29.5.3 Работа универсального асинхронного приемопередатчика

Управляющая информация хранится в регистре управления линией UARTLCR. Этот регистр имеет внутреннюю ширину 30 бит, однако внешний доступ по шине APB к нему осуществляется через следующие регистры:

- UARTLCR_H – определяет:
 - параметры передачи данных;
 - длину слова;
 - режим буферизации;
 - количество передаваемых стоповых бит;
 - режим контроля четности;
 - формирование сигнала разрыва линии;
- UARTIBRD – определяет целую часть коэффициента деления для скорости передачи данных;
- UARTFBRD – определяет дробную часть коэффициента деления для скорости передачи данных.

29.5.4 Коэффициент деления частоты

Коэффициент деления для формирования скорости передачи данных состоит из 22 битов, при этом 16 бит выделено для представления его целой части, а 6 бит – дробной части. Возможность задания нецелых коэффициентов деления позволяет осуществлять обмен данными со стандартными информационными скоростями, при этом используя в качестве UARTCLK тактовый сигнал с произвольной частотой более 3,6864 МГц.

Целая часть коэффициента деления записывается в 16-битный регистр UARTIBRD. Шестиразрядная дробная часть записывается в регистр UARTFBRD.

Значение коэффициента деления связано с содержимым указанных регистров следующим образом

$$\text{Коэффициент деления} = \frac{UARTCLK}{16 \cdot \text{скорость передачи данных}} = BRD_I + BRD_F, \quad (15)$$

где BRD_I – целая часть коэффициента деления;

BRD_F – дробная часть коэффициента деления.

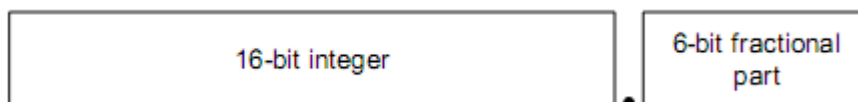


Рисунок 113 – Коэффициент деления

Шестибитное значение, записываемое в регистр UARTFBRD, вычисляется путем выделения дробной части требуемого коэффициента деления, умножения ее на 64 (то есть на 2^n , где n – ширина регистра UARTFBRD) и округления до ближайшего целого числа

$$M = \text{integer}(BRD_F \cdot 2^n + 0,5), \quad (16)$$

где integer – операция отсечения дробной части числа, $n = 6$.

В модуле формируется внутренний сигнал Baud16, представляющий собой последовательность импульсов с длительностью, равной периоду сигнала UARTCLK и средней частотой, в 16 раз большей требуемой скорости обмена данными.

29.5.5 Передача и прием данных

Принятые или передаваемые данные заносятся в 16-элементные буферы FIFO, при этом каждый элемент приемного буфера FIFO кроме байта данных хранит также четыре бита информации о состоянии модема.

Для передачи данные заносятся в буфер FIFO передатчика. Если работа приемопередатчика разрешена, начинается передача информационного кадра с параметрами, указанными в регистре управления линией UARTLCR_H. Передача данных продолжается до опустошения буфера FIFO передатчика. После записи элемента в буфер FIFO передатчика сигнал BUSY переходит в высокое состояние. Это состояние сохраняется в течение всего времени передачи данных. В низкое состояние сигнал BUSY переходит только после того, как буфер FIFO передатчика станет пуст, а последний бит данных (включая стоповые биты) будет передан. Сигнал BUSY может находиться в высоком состоянии даже в случае, если приемопередатчик будет переведен из разрешенного состояний в запрещенное.

Для каждого бита данных (в приемной линии) производится три измерения уровня, решение принимается по мажоритарному принципу.

В случае, если приемник находился в неактивном состоянии (на линии входного сигнала UART_RXD постоянно присутствовала единица) и произошел переход входного сигнала из высокого в низкий логический уровень (обнаружен стартовый бит), включается счетчик, тактируемый сигналом Baud16, после чего отсчеты сигнала на входе приемника регистрируются каждые восемь тактов (в режиме асинхронного

приемопередатчика) или каждые четыре такта (в режиме ИК обмена данными) сигнала Baud16. Более частая выборка данных в режиме ИК обмена связана с необходимостью корректной обработки импульсов данных согласно протоколу SIR IrDA.

Стартовый бит считается достоверным в случае, если сигнал на линии UART_RXD сохраняет низкий логический уровень в течение восьми отсчетов сигнала Baud16 с момента включения счетчика. В противном случае переход в ноль рассматривается как ложный старт и игнорируется.

В случае, если обнаружен достоверный стартовый бит, производится регистрация последовательности данных на входе приемника. Очередной бит данных фиксируются каждые 16 отсчетов тактового сигнала Baud16 (что соответствует длительности одного символа). Производится регистрация всех бит данных (согласно запрограммированным параметрам) и бита четности (если включен режим контроля четности).

Наконец, производится проверка присутствия корректного стопового бита (высокий логический уровень сигнала UART_RXD). В случае, если последнее условие не выполняется, устанавливается признак ошибки формирования кадра. После того, как слово данных принято полностью, оно заносится в буфер FIFO приемника, наряду с четырьмя битами признаков ошибки, связанных с принятым словом (см. таблицу 349).

29.5.6 Биты ошибки

Три бита признаков ошибки, ассоциированные с принятым символом данных, заносятся в разряды [10...8] слова данных в буфере FIFO приемника. Также предусмотрен признак ошибки переполнения буфера FIFO в разряде 11 слова данных. Таблица 349 описывает назначение всех бит слова данных в FIFO-буфере приемника.

Таблица 349 – Назначение бит слова данных в FIFO-буфере приемника

Бит буфера FIFO	Назначение
11	Признак переполнения буфера
10	Ошибка – «разрыв линии»
9	Ошибка проверки на четность
8	Ошибка формирования кадра
7...0	Принятые данные

29.5.7 Бит переполнения буфера

Бит переполнения непосредственно не связан с конкретным символом в буфере приемника. Признак переполнения фиксируется в случае, если буфер FIFO заполнен к моменту, когда очередной символ данных полностью принят (находится в регистре сдвига). При этом данные из регистра сдвига не попадают в буфер приемника и теряются с началом приема очередного символа. Как только в буфере приемника появляется свободное место, очередной принятый символ данных заносится в буфер FIFO вместе с текущим значением признака переполнения. После успешной записи данных в буфер признак переполнения сбрасывается.

29.5.8 Запрет буфера FIFO

Предусмотрена возможность отключения FIFO буферов приемника и передатчика. В этом случае приемная и передающая сторона контроллера UART располагают лишь однобайтными буферными регистрами. Бит переполнения буфера устанавливается при этом тогда, когда очередной символ данных уже принят, однако предыдущий еще не был считан.

В настоящей реализации модуля буферы FIFO физически не отключаются, необходимая функциональность достигается за счет логических манипуляций с флагами. При этом в случае, если буфер FIFO отключен, а сдвиговый регистр передатчика пуст (не используется), запись байта данных происходит непосредственно в регистр сдвига, минуя буферный регистр.

29.5.8.1 Проверка по шлейфу

Проверка по шлейфу (замыкание выхода передатчика на вход приемника) выполняется путем установки в 1 бита LBE в регистре управления контроллером UARTCR.

29.5.9 Работа кодека ИК-обмена данными IrDA SIR

Кодек обеспечивает сопряжение асинхронного потока данных, сформированного приемопередатчиком, с полудуплексным последовательным интерфейсом IrDA SIR. Какая-либо аналоговая обработка сигнала при этом не выполняется. Назначение кодека – сформировать цифровой поток данных на вход приемника асинхронного сигнала и обработать цифровой поток данных с выхода передатчика.

Предусмотрены два режима работы: IrDA и IrDA с пониженным энергопотреблением.

В режиме IrDA уровень логического нуля передается на линию nSIROUT в виде импульса с высоким логическим уровнем и длительностью 3/16 от выбранного периода следования бит данных. Логическая единица при этом передается в виде постоянного низкого уровня сигнала. Сформированный выходной сигнал далее подается на передатчик ИК-сигнала, обеспечивая излучение светового импульса всякий раз при передаче нулевого бита. На приемной стороне световые импульсы воздействуют на базу фототранзистора ИК-приемника, который в результате формирует низкий логический уровень. Это, в свою очередь, обуславливает низкий уровень на входе SIRIN.

В режиме IrDA с пониженным энергопотреблением длительность передаваемых импульсов ИК излучения устанавливается в три раза выше длительности импульсов внутреннего опорного сигнала IrLPBaud16 (равной 1,63 мкс при номинальной частоте 1,8432 МГц). Данный режим активизируется путем установки бита SIRLP в регистре управления UARTCR.

Как в нормальном режиме, так и в режиме пониженного энергопотребления:

- кодирование осуществляется на основе бит данных, сформированных асинхронным передатчиком модуля;
- в ходе приема данных декодированные биты далее обрабатываются блоком асинхронного приема.

В соответствии со спецификацией физического уровня протокола IrDA SIR, обмен данными должен осуществляться в режиме полудуплекса, при этом задержка между передачей и приемом данных должна составлять не менее 10 мс. Эта задержка должна формироваться программно. Необходимость ее введения обусловлена тем, что воздействие передающего ИК светодиода на находящийся рядом ИК приемник может привести к искажению принимаемого сигнала или даже ввести приемный тракт в состояние насыщения. Задержка между окончанием передачи и началом приема данных именуется латентность, или время установки (готовности) приемника.

Сигнал IrLPBaud16 формируется путем деления частоты сигнала UARTCLK в соответствии с коэффициентом деления, записанным в регистре UARTILPR.

Коэффициент деления вычисляется по формуле

$$\text{Коэффициент деления} = \frac{F_{\text{UARTCLK}}}{F_{\text{IrLPBaud16}}}, \quad (17)$$

где номинальное значение IrLPBaud16 составляет 1,8432 МГц.

Коэффициент деления должен быть выбран так, чтобы выполнялось соотношение:

$$1,42 \text{ МГц} < F_{\text{IrLPBaud16}} < 2,12 \text{ МГц}.$$

29.5.9.1 Проверка по шлейфу

Проверка по шлейфу выполняется после установки в «1» бита LBE регистра управления контроллером UARTCR с одновременной установкой в «1» бита SIRTEST регистра управления тестированием UARTTCCR.

В этом режиме данные, передаваемые на выход nSIROUT, должны подаваться на вход SIRIN.

Примечание – Это единственный случай использования тестового регистра в нормальном режиме функционирования модуля.

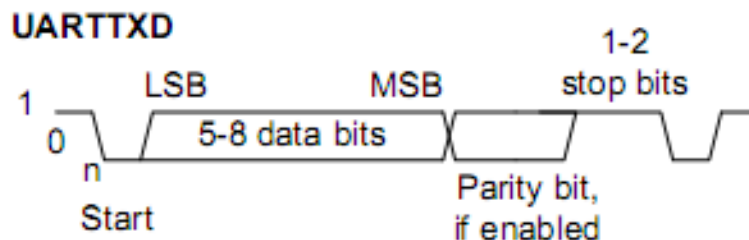


Рисунок 114 – Кадр передачи данных

29.5.10 Модуляция данных IrDA

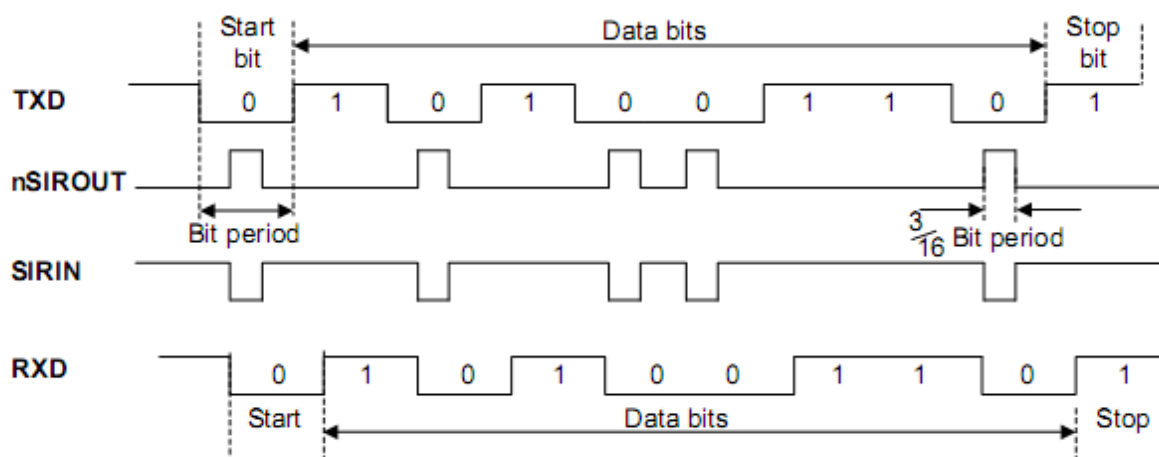


Рисунок 115 – Модуляция данных IrDA

29.6 Линии управления модемом

Модуль универсального асинхронного приемопередатчика может использоваться как в режиме оконечного оборудования (DTE), так и в режиме оборудования передачи данных (DCE). Сигналы модема в режиме DTE показаны ранее (см. рисунок 111).

Назначение сигналов в режимах DTE и DCE представлено в таблице 350.

Таблица 350 – Назначение управления модемом в режимах DTE и DCE

Сигнал	Назначение	
	Режим оконечного оборудования	Режим оборудования передачи данных
nUARTCTS	Готов к передаче данных	Запрос передачи данных
nUARTDSR	Источник данных готов	Приемник данных готов
nUARTDCD	Обнаружен информационный сигнал	-
nUARTRI	Индикатор вызова	-
nUARTRTS	Запрос передачи данных	Готов к передаче данных
nUARTDTR	Приемник данных готов	Источник данных готов
nUARTOUT1	-	Обнаружен информационный сигнал
nUARTOUT2	-	Индикатор вызова

29.6.1 Аппаратное управление потоком данных

Программно активизируемый режим аппаратного управления потоком данных позволяет контролировать (приостанавливать и возобновлять) информационный обмен с помощью сигналов nUARTRTS и nUARTCTS. Иллюстрация взаимодействия двух устройств последовательной связи с аппаратным управлением потоком данных представлена на рисунке 116.

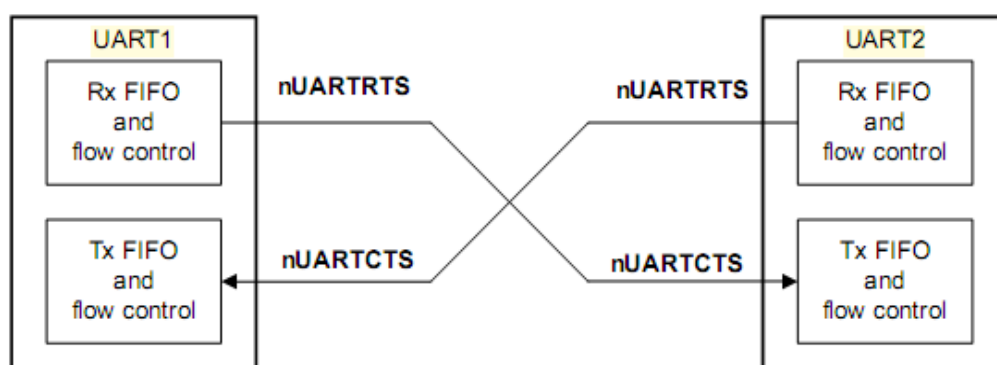


Рисунок 116 – Взаимодействие двух устройств последовательной связи с аппаратным управлением потоком данных

Если разрешено управление потоком данных по сигналу RTS, линия nUARTRTS переводится в активное состояние только после того, как в FIFO буфере приема появляется заданное количество свободных элементов.

Если разрешено управление потоком данных по сигналу CTS, передача данных осуществляется только после перевода линии nUARTCTS в активное состояние.

Режим аппаратного управления потоком данных задается путем установки значений бит RTSEn и CTSEn в регистре управления UARTCR. Таблица 351 показывает необходимые установки для различных режимов управления потоком данных.

Таблица 351 – Режимы управления потоком данных

CTSEn	RTSEn	Описание
1	1	Разрешено управление потоком данных по CTS и RTS
1	0	Управление потоком данных осуществляется по линии CTS
0	1	Управление потоком данных осуществляется по линии RTS
0	0	Управление потоком данных запрещено

Примечание – В случае если выбран режим управления потоком данных по RTS, программное обеспечение не может использовать бит RTSEn регистра UARTCR для проверки состояния линии RTS.

29.6.2 Управление потоком данных по линии RTS

Логика управления потоком данных по RTS использует данные о превышении пороговых уровней заполнения буфера FIFO приемника. В случае выбора режимов с управлением по RTS, сигнал на линии nUARTRTS переводится в активное состояние только после того, как в FIFO буфере приема появляется заданное количество свободных элементов. После достижения порогового уровня заполнения буфера приемника сигнал nUARTRTS снимается (переводится в пассивное состояние), указывая, таким образом, на отсутствие свободного места для сохранения принятых данных. При этом дальнейшая передача данных должна быть прекращена по завершении передачи текущего символа.

Обратно в активное состояние сигнал nUARTRTS переводится после считывания данных из приемного буфера FIFO в количестве, достаточном для того, чтобы заполнение буфера оказалось ниже порогового уровня.

В случае, если управление потоком данных по RTS запрещено, однако работа приемопередатчика UART разрешена, прием будет осуществляться до полного заполнения буфера FIFO, либо до завершения передачи данных.

29.6.3 Управление потоком данных по линии CTS

В случае выбора одного из режимов с управлением потоком данных по CTS передатчик осуществляет проверку состояния линии nUARTCTS перед началом передачи очередного байта данных. Передача осуществляется только в случае, если данная линия активна, и продолжается до тех пор, пока активное состояние линии сохраняется и буфер передатчика не пуст.

При переходе линии nUARTCTS в неактивное состояние модуль завершает выдачу текущего передаваемого символа, после чего передача данных прекращается.

Если управление потоком данных по CTS запрещено, и при этом работа приемопередатчика UART разрешена - данные будут выдаваться до опустошения буфера FIFO передатчика.

29.7 Интерфейс прямого доступа к памяти

Модуль универсального асинхронного приемопередатчика оснащен интерфейсом подключения к контроллеру прямого доступа к памяти. Работа в данном режиме контролируется регистром управления DMA UARTDMACR.

Интерфейс DMA включает в себя следующие сигналы:

Для приема:

UARTRXDMASREQ – запрос передачи отдельного символа, инициируется контроллером UART. Размер символа в режиме приема данных – до 12 бит. Сигнал переводится в активное состояние в случае, если буфер FIFO приемника содержит, по меньшей мере, один символ.

UARTRXDABREQ – запрос блочного обмена данными, инициируется модулем приемопередатчика. Сигнал переходит в активное состояние в случае, если заполнение буфера FIFO приемника превысило заданный порог. Порог программируется индивидуально для каждого буфера FIFO путем записи значения в регистр UARTIFLS.

UARTRXDACLRL – сброс запроса на DMA, инициируется модулем приемопередатчика с целью сброса принятого запроса. В случае, если был запрошен блочный обмен данными, сигнал сброса формируется в ходе передачи последнего символа данных в блоке.

Для передачи:

UARTTXDMASREQ – запрос передачи отдельного символа, инициируется модулем приемопередатчика. Размер символа в режиме передачи данных – до восьми бит. Сигнал переводится в активное состояние в случае, если буфер FIFO передатчика содержит, по меньшей мере, одну свободную ячейку.

UARTTXDABREQ – запрос блочного обмена данными, инициируется модулем приемопередатчика. Сигнал переводится в активное состояние в случае, если заполнение буфера FIFO передатчика ниже заданного порога. Порог программируется индивидуально для каждого буфера FIFO путем записи значения в регистр UARTIFLS.

UARTTXDACLRL – сброс запроса на DMA, инициируется контроллером DMA с целью сброса принятого запроса. В случае, если был запрошен блочный обмен данными, сигнал сброса формируется в ходе передачи последнего символа данных в блоке.

Сигналы блочного и одноэлементного обмена данными не являются взаимно исключаящими, они могут быть инициированы одновременно. Например, в случае, если заполнение данными буфера приемника превышает пороговое значение, формируется как сигнал запроса одноэлементного обмена, так и сигнал запроса блочного обмена данными. В случае, если количество данных в буфере приема меньше порогового значения формируется только запрос одноэлементного обмена. Это бывает полезно в ситуациях, при которых объем данных меньше размера блока. Пусть, например, нужно принять 19 символов, а порог заполнения буфера FIFO установлен равным четырем. Тогда контроллер DMA осуществит четыре передачи блоков по четыре символа, а оставшиеся три символа передаст в ходе трех одноэлементных обменов.

Примечание – Для оставшихся трех символов контроллер UART не может инициировать процедуру блочного обмена.

Каждый инициированный приемопередатчиком сигнал запроса DMA остается активным до момента его сброса соответствующим сигналом DMACLR.

После снятия сигнала сброса модуль приемопередатчика вновь получает возможность сформировать запрос на DMA в случае выполнения описанных выше условий. Все запросы DMA снимаются после запрета работы приемопередатчика, а также в случае установки в ноль бита управления DMA TXDMAE или RXDMAE в регистре управления DMA UARTDMACR.

В случае запрета буферов FIFO устройство способно передавать и принимать только одиночные символы; как следствие, контроллер может инициировать DMA только в одноэлементном режиме. При этом модуль в состоянии формировать только сигналы управления DMA UARTRXDMASREQ и UARTTXDMASREQ. Для информации о запрете буферов FIFO см. описание регистра управления линией UARTLCR_H.

Когда буферы FIFO включены, обмен данными может производиться в ходе как одноэлементных, так и блочных передач данных, в зависимости от установленной величины порога заполнения буферов и их фактического заполнения. Таблица 352 показывает значения параметров срабатывания запросов блочного обмена UARTRXDMABREQ и UARTTXDMABREQ в зависимости от порога заполнения буфера.

Таблица 352 – Параметры срабатывания запросов блочного обмена данными в режиме DMA

Пороговый уровень	Длина блока обмена данными	
	Буфер передатчика (количество незаполненных ячеек)	Буфер приемника (количество заполненных ячеек)
1/8	14	2
1/4	12	4
1/2	8	8
3/4	4	12
7/8	2	14

В регистре управления DMA UARTDMACR предусмотрен бит DMAONERR, который позволяет запретить DMA от приемника в случае активного состояния линии прерывания по обнаружению ошибки UARTEINTR. При этом соответствующие линии запроса DMA – UARTRXDMASREQ и UARTRXDMABREQ переводятся в неактивное

состояние (маскируются) до сброса UARTEINTR. На линии запроса DMA, обслуживающие передатчик, состояние UARTEINTR не влияет.

На рисунке 117 показаны временные диаграммы одноэлементного и блочного запросов DMA, в том числе действие сигнала DMACLR. Все сигналы должны быть синхронизированы с CPU_CLK. В интересах ясности изложения предполагается, что синхронизация сигналов запроса DMA в контроллере DMA не производится.

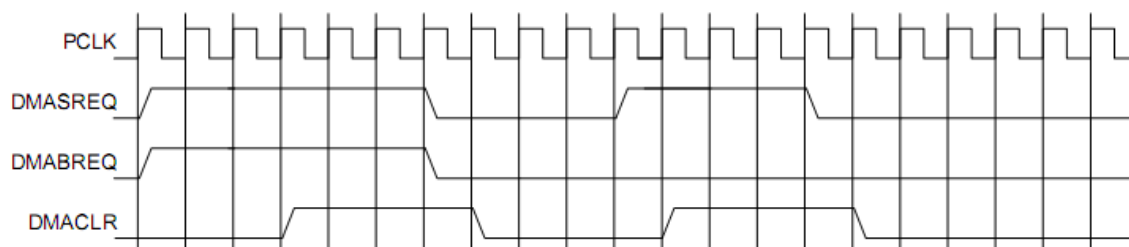


Рисунок 117 – Временные диаграммы одноэлементного и блочного запросов DMA

29.8 Прерывания

В модуле предусмотрено 11 маскируемых источников прерывания. В результате формируется один общий сигнал, представляющий собой комбинацию независимых сигналов, объединенных по схеме ИЛИ.

Сигналы запроса на прерывание:

- UARTRXINTR – прерывание от приемника.
- UARTTXINTR – прерывание от передатчика.
- UARTRTINTR – прерывание по таймауту приемника.
- UARTMSINTR – прерывание по состоянию модема:
- UARTRIINTR, изменение состояния линии nUARTRI;
- UARTCTSINTR, изменение состояния линии nUARTCTS;
- UARTDCDINTR, изменение состояния линии nUARTDCD;
- UARTDSRINTR, изменение состояния линии nUARTDSR.
- UARTEINTR – ошибка:
- UARTOEINTR, переполнение буфера;
- UARTBEINTR, прерывание приема – разрыв линии;
- UARTPEINTR, ошибка контроля четности;
- UARTFEINTR, ошибка в структуре кадра.
- UARTINTR – логическое ИЛИ сигналов UARTRXINTR, UARTTXINTR, UARTRTINTR, UARTMSINTR и UARTEINTR.

Каждый из независимых сигналов запроса на прерывание может быть маскирован путем установки соответствующего бита в регистре маски UARTIMSC. Установка бита в 1 разрешает соответствующее прерывание, в 0 – запрещает.

Доступность, как индивидуальных линий, так и общей линии запроса позволяет организовать обслуживание прерываний в системе, как путем применения глобальной процедуры обработки, так и с помощью драйвера устройства, построенного по модульному принципу.

Прерывания от приемника и передатчика UARTRXINTR и UARTTXINTR выведены отдельно от прерываний по изменению состояния устройства. Это позволяет использовать сигналы запроса UARTRXINTR и UARTTXINTR для обеспечения чтения и записи данных согласованно с достижением заданного порога заполнения буферов FIFO приемника и передатчика.

Прерывание по обнаружению ошибке UARTEINTR формируется в случае возникновения той или иной ошибки приема данных. Предусмотрен ряд условий формирования признака ошибки.

Прерывание по состоянию модема представляет собой комбинацию признаков изменения отдельных линий состояния модема.

Признаки возникновения каждого из условий прерывания можно считать либо из регистра прерываний UARTRIS, либо из маскированного регистра прерываний UARTMIS.

29.8.1 UARTMSINTR

Прерывание по состоянию модема возникает в случае изменения любой из линий состояний модема (nUARTCTS, nUARTDCD, nUARTDSR, nUARTRI). Сброс прерывания осуществляется путем записи 1 в соответствующий (в зависимости от линии состояния модема, вызвавшей прерывание) разряд регистра сброса прерывания UARTICR.

29.8.2 UARTRXINTR

Состояние прерывания от приемника может измениться в случае возникновения одного из следующих событий:

- буфер FIFO разрешен и его заполнение достигло заданного порогового значения. В этом случае линия прерывания переходит в высокое состояние. Сигнал прерывания переходит в низкое состояние после чтения данных из буфера приемника до тех пор, пока его заполнение не станет меньше порога, либо после сброса прерывания;
- буфер FIFO запрещен (имеет размер один символ), принят один символ данных. При этом линия прерывания переходит в высокое состояние. Сигнал прерывания переходит в низкое состояние после чтения одного байта данных, либо после сброса прерывания.

29.8.3 UARTTXINTR

Состояние прерывания от передатчика может измениться в случае возникновения одного из следующих событий:

- буфер FIFO разрешен и его заполнение меньше или равно заданному пороговому значению. В этом случае линия прерывания переходит в высокое состояние. Сигнал прерывания переходит в низкое состояние после записи данных в буфера передатчика до тех пор, пока его заполнение не станет больше порога, либо после сброса прерывания;
- буфер FIFO запрещен (имеет размер один символ), данные в буферном регистре передатчика отсутствуют. При этом линия прерывания переходит в высокое

состояние. Сигнал прерывания переходит в низкое состояние после записи одного байта данных, либо после сброса прерывания.

Для занесения данных в буфер FIFO передатчика необходимо записать данные в буфер либо перед разрешением работы приемопередатчика и прерываний, либо после разрешения работы приемопередатчика и прерываний.

Примечание – Прерывание передатчика основано на переходе через пороговое значение, а не на состоянии заполненности буфера FIFO передатчика относительно порогового значения. В случае, если модуль и прерывания от него разрешены до осуществления записи данных в буфер FIFO передатчика, прерывание не формируется. Прерывание возникает только при опустошении буфера FIFO.

29.8.4 UARTRTINTR

Прерывание по таймауту приемника возникает в случае, если буфер FIFO приемника не пуст, и на вход приемника не поступало новых данных в течение периода времени, необходимого для передачи 32 бит. Прерывание по таймауту снимается либо после считывания данных из буфера приемника до его опустошения (или считывания одного байта в случае, если буфер FIFO запрещен), либо путем записи 1 в соответствующий бит регистра сброса прерывания UARTICR.

29.8.5 UARTEINTR

Прерывание по обнаружению ошибки возникает в случае ошибки при приеме данных. Оно может быть вызвано рядом факторов:

- ошибка в структуре кадра;
- ошибка контроля четности;
- разрыв линии;
- переполнение буфера.

Причину возникновения прерывания можно определить, прочитав содержимое регистра прерываний UARTRIS, либо содержимое маскированного регистра прерываний UARTMIS.

Сброс прерывания осуществляется путем записи соответствующих бит в регистр сброса прерывания UARTICR. За прерываниями по обнаружению ошибки закреплены биты с 7 по 10.

29.8.6 UARTINTR

Все описанные сигналы запроса на прерывание скомбинированы в общую линию путем объединения по схеме ИЛИ сигналов UARTRXINTR, UARTRXINTR, UARTTXINTR, UARTRTINTR, UARTMSINTR и UARTEINTR с учетом маскирования. Общий выход может быть подключен к системному контроллеру прерывания, что позволит ввести дополнительное маскирование запросов на уровне периферийных устройств.

29.9 Программное управление модулем

29.9.1 Общая информация

Следующая информация применима ко всем регистрам контроллера:

- Базовый адрес контроллера не фиксирован и может быть различным в разных системах. Смещение каждого регистра относительно базового адреса постоянно.
- Не следует пытаться получить доступ к зарезервированным или неиспользуемым адресам. Это может привести к непредсказуемому поведению модуля.
- За исключением специально оговоренных в настоящем документе случаев:
- не следует изменять значения не определенных в документе разрядов регистров;
- не следует использовать значения не определенных в документе разрядов регистров;
- все биты регистров (за исключением некоторых случаев из таблицы 353 – Обобщенные данные о регистрах устройства) устанавливаются в значение 0 после сброса по включению питания или системного сброса.

Столбец «Тип» Таблица 353 определяет режим доступа к регистру в соответствии с обозначениями:

- RW – чтение и запись;
- RO – только чтение;
- WO – только запись.

Обобщенные данные о регистрах устройства

Данные о регистрах модуля универсального асинхронного приемопередатчика приведены в таблице 353.

Таблица 353 – Обобщенные данные о регистрах устройства

Смещение	Наименование	Тип	Значение после сброса	Размер, бит	Описание
0x40030000	MDR_UART1				Контроллер UART1
0x40038000	MDR_UART2				Контроллер UART2
0x000	DR	RW	0x---	12/8	MDR_UARTx->DR Регистр данных
0x004	RSR_ECR	RW	0x0	4/0	MDR_UARTx->RSR_ECR Регистр состояния приемника/ Сброс ошибки приемника
0x008-0x014					Зарезервировано
0x018	FR	RO	0b-10010---	9	MDR_UARTx->FR Регистр флагов
0x01C					Зарезервировано
0x020	ILPR	RW	0x00	8	MDR_UARTx->ILPR Регистр управления ИК-обменом в режиме пониженного энергопотребления

Смещение	Наименование	Тип	Значение после сброса	Размер, бит	Описание
0x024	IBRD	RW	0x0000	16	MDR_UARTx->IBRD Целая часть делителя скорости обмена данными
0x028	FBRD	RW	0x00	6	MDR_UARTx->FBRD Дробная часть делителя скорости обмена данными
0x02C	LCR_H	RW	0x00	8	MDR_UARTx->LCR_H Регистр управления линией
0x030	CR	RW	0x0300	16	MDR_UARTx->CR Регистр управления
0x034	IFLS	RW	0x12	6	MDR_UARTx->IFLS Регистр порога прерывания по заполнению буфера FIFO
0x038	IMSC	RW	0x000	11	MDR_UARTx->IMSC Регистр маски прерывания
0x03C	RIS	RO	0x00-	11	MDR_UARTx->RIS Регистр состояния прерываний
0x040	MIS	RO	0x00-	11	MDR_UARTx->MIS Регистр состояния прерываний с маскированием
0x044	ICR	WO	-	11	MDR_UARTx->ICR Регистр сброса прерывания
0x048	DMACR	RW	0x00	3	MDR_UARTx->DMACR Регистр управления DMA

29.9.2 MDR_UARTx->DR

Регистр данных

В ходе передаче данных:

Если буфер FIFO передатчика разрешен, то слово данных, записанное в рассматриваемый регистр, направляется в буфер FIFO передатчика.

В противном случае, записанное слово фиксируется в буферный регистр передатчика (последний элемент буфера FIFO).

Операция записи в регистр инициирует передачу данных. Слово данных предваряется стартовым битом, дополняется битом контроля четности (если режим контроля четности включен) и стоповым битом. Сформированное слово отправляется в линию передачи данных.

В ходе приема данных:

Если буфер FIFO приемника разрешен, байт данных и четыре бита состояния (разрыв, ошибка формирования кадра, четность, переполнение) сохраняются в 12-битном буфере.

В противном случае байт данных и биты состояния записываются в буферный регистр (последний элемент буфера FIFO).

Полученные из линии связи байты данных считываются путем чтения из регистра принятых данных совместно с соответствующими битами состояния. Информация о

состоянии также может быть получена путем чтения регистра UARTRSR/UARTECR. (См. таблицу 354).

Таблица 354 – Формат регистра UARTDR

Номер бита	Сигнал	Назначение
15...12		Резерв
11	OE	Переполнение буфера приемника. Бит устанавливается в 1 в случае, если на вход приемника поступают данные, в то время как буфер заполнен. Сбрасывается в 0 после того, как в буфере появится свободное место
10	BE	Разрыв линии. Устанавливается в 1 при обнаружении признака разрыва линии, то есть в случае наличия низкого логического уровня на входе приемника в течение времени, большего чем длительность передачи полного слова данных (включая стартовый, стоповый биты и бит проверки на четность). При включенном FIFO данная ошибка ассоциируется с последним символом, поступившим в буфер. В случае обнаружения разрыва линии в буфер загружается только один нулевой символ, прием данных возобновляется только после перехода линии в логическую «1» и последующего обнаружения корректного стартового бита
9	PE	Ошибка контроля четности. Устанавливается в 1 в случае, если четность принятого символа данных не соответствует установкам бит EPS и SPS в регистре управления линией UARTLCR_H. При включенном FIFO данная ошибка ассоциируется с последним символом, поступившим в буфер.
8	FE	Ошибка в структуре кадра. Устанавливается в 1 в случае, если в принятом символе не обнаружен корректный стоповый бит (корректный стоповый бит равен 1). При включенном FIFO данная ошибка ассоциируется с последним символом, поступившим в буфер
7...0	DATA	Принимаемые данные (чтение). Передаваемые данные (запись)

Примечание – Необходимо запрещать работу приемопередатчика перед любым перепрограммированием его регистров управления. Если приемопередатчик переводится в отключенное состояние во время передачи или приема символа, то перед остановкой он завершает выполняемую операцию.

29.9.3 MDR_UARTx->RSR_ECR

Регистр состояния приемника/сброса ошибки

Состояние приемника также может быть считано из регистра UARTRSR. В этом случае информация о состоянии признаков разрыва линии, ошибки контроля четности и ошибки в структуре кадра относится к последнему символу, считанному из регистра данных UARTDR. С другой стороны, признак переполнения буфера устанавливается немедленно после возникновения этого состояния (и не связан с последним считанным из регистра UARTDR байтом данных).

Запись в регистр UARTECR приводит к сбросу признаков ошибок переполнения, четности, структуры кадра, разрыва линии. Кроме того, все эти признаки устанавливаются в 0 после сброса устройства.

Таблица 355 показывает назначение бит регистра UARTRSR/UARTECR.

Таблица 355 – Регистр UARTRSR/UARTECR

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
7...4		Резерв, при чтении результат не определен
3	OE	Переполнение буфера приемника. Бит устанавливается в 1 в случае, если на вход приемника поступают данные, в то время как буфер заполнен. Сбрасывается в 0 после записи в регистр UARTECR. Содержимое буфера остается верным, так как перезаписан был только регистр сдвига. Центральный процессор должен считать данные для того, чтобы освободить буфер FIFO
2	BE	Разрыв линии. Устанавливается в 1 при обнаружении признака разрыва линии, то есть в случае наличия низкого логического уровня на входе приемника в течение времени, большего чем длительность передачи полного слова данных (включая стартовый, стоповый биты и бит проверки на четность). Бит сбрасывается в 0 после записи в регистр UARTECR. При включенном FIFO данная ошибка ассоциируется с символом, находящемся на вершине буфера. В случае обнаружения разрыва линии в буфер загружается только один нулевой символ, прием данных возобновляется только после перехода линии в логическую 1 и последующего обнаружения корректного стартового бита
1	PE	Ошибка контроля четности. Устанавливается в 1 в случае, если четность принятого символа данных не соответствует установкам бит EPS и SPS в регистре управления линией UARTECR_H. Бит сбрасывается в 0 после записи в регистр UARTECR. При включенном FIFO данная ошибка ассоциируется с символом, находящимся на вершине буфера
0	FE	Ошибка в структуре кадра. Устанавливается в 1 в случае, если в принятом символе не обнаружен корректный стоповый бит (корректный стоповый бит равен 1). Бит сбрасывается в 0 после записи в регистр UARTECR. При включенном FIFO данная ошибка ассоциируется с символом, находящимся на вершине буфера

Примечания:

1 Перед чтением регистра состояния UARTRSR необходимо считать данные, принятые из линии, путем обращения к регистру данных UARTDR. Противоположная последовательность действий не допускается, так как регистр UARTRSR обновляет свое состояние только после чтения регистра UARTDR. Вместе с тем, информация о состоянии приемника может быть получена непосредственно из регистра данных UARTDR.

2 Запись в регистр UARTRSR/UARTECR любого кода сбрасывает признаки ошибок формирования кадра, проверки на четность, разрыва линии и переполнения буфера.

29.9.4 MDR_UARTx->FR

Регистр флагов

После сброса биты регистра флагов TXFF, RXFF и BUSY устанавливаются в 0, а биты TXFE и RXFE – в 1. В таблице 356 представлена информация о назначении бит регистра UARTFR.

Таблица 356 – Регистр UARTFR

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
15...9		Резерв. Не модифицируйте. При чтении заполняются нулями
8	RI	Инверсия линии nUARTRI
7	TXFE	Буфер FIFO передатчика пуст. Значение бита зависит от состояния бита FEN регистра управления линией UARTLCR_H. Если буфер FIFO запрещен, бит устанавливается в 1, когда буферный регистр передатчика пуст. В противном случае он равен 1, если пуст буфер FIFO передатчика. Данный бит не дает никакой информации о наличии данных в регистре сдвига передатчика
6	RXFF	Буфер FIFO приемника заполнен. Значение бита зависит от состояния бита FEN регистра управления линией UARTLCR_H. Если буфер FIFO запрещен, бит устанавливается в 1, когда буферный регистр приемника занят. В противном случае он равен 1, если заполнен буфер FIFO приемника
5	TXFF	Буфер FIFO передатчика заполнен. Значение бита зависит от состояния бита FEN регистра управления линией UARTLCR_H. Если буфер FIFO запрещен, бит равен 1, когда буферный регистр передатчика занят. В противном случае он равен 1, если заполнен буфер FIFO передатчика
4	RXFE	Буфер FIFO приемника пуст. Значение бита зависит от состояния бита FEN регистра управления линией UARTLCR_H. Если буфер FIFO запрещен, бит устанавливается в 1, когда буферный регистр приемника пуст. В противном случае он равен 1, если пуст буфер FIFO приемника
3	BUSY	UART занят. Бит равен 1 в случае, если контроллер передает в линию данные. Бит остается установленным до тех пор, пока данные, включая стоповые биты, не будут полностью переданы. Кроме того, бит занятости устанавливается в 1 при наличии данных в буфере FIFO передатчика, вне зависимости от состояния приемопередатчика (даже, если он запрещен)
2	DCD	Инверсия линии nUARTDCD
1	DSR	Инверсия линии nUARTDSR
0	CTS	Инверсия линии nUARTCT

29.9.5 MDR_UARTx->ILPR

Регистр управления ИК-обменом в режиме пониженного энергопотребления

Этот восьмиразрядный регистр, доступный для чтения и записи, содержит значение коэффициента деления частоты UARTCLK, для формирования тактового сигнала IrLPBaud16. Назначение разрядов регистра показано в таблице 357.

Требуемое значение коэффициента деления для формирования сигнала IrLPBaud16 вычисляется по формуле

$$ILPDVSR = (F_UARTCLK)/(F_IrLPBaud16), \quad (18)$$

где номинальное значение частоты F_IrLPBaud16 составляет 1,8432 МГц.

Коэффициент деления должен быть установлен таким образом, чтобы выполнялось соотношение

$$1,42 \text{ МГц} < F_{IrLPBaud16} < 2,12 \text{ МГц},$$

что, в свою очередь, гарантирует формирование кодеком импульсов данных с длительностью 1,41 – 2,11 мкс (в три раза длиннее периода сигнала IrLPBaud16).

Таблица 357 – Регистр UARTILPR

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
7...0	ILPDVSR	Коэффициент деления частоты UARTCLK, для формирования тактового сигнала IrLPBaud16. После сброса устанавливается в 0. Примечание – Коэффициент 0 – запрещенное значение. В случае его установки импульсы IrLPBaud16 формироваться не будут

Примечание – В интересах подавления помех при работе в режиме IrDA с пониженным энергопотреблением кодек игнорирует поступающие на вход SIRIN импульсы с длительностью, меньшей трех периодов сигнала IrLPBaud16.

29.9.6 MDR_UARTx->IBRD

Регистр целой части делителя скорости передачи данных

Назначение бит регистра UARTBIRD показано в таблице 358.

Таблица 358 – Регистр UARTBIRD

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
15...0	BAUDDIV_INT	Целая часть коэффициента деления частоты для формирования тактового сигнала передачи данных. После сброса устанавливается в 0

29.9.7 MDR_UARTx->FBRD

Регистр дробной части делителя скорости передачи данных

Таблица 359 показывает назначение бит регистра UARTBFRD.

Таблица 359 – Регистр UARTBFRD

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
5...0	BAUDDIV_FRAC	Дробная часть коэффициента деления частоты для формирования тактового сигнала передачи данных. После сброса устанавливается в 0

Коэффициент деления вычисляется по формуле

$$BAUDDIV = \frac{FUARTCLK}{16 \cdot Baud_rate}, \quad (19)$$

где FUARTCLK – тактовая частота контроллера UART,
Baud_rate – требуемая скорость передачи данных.

Коэффициент BAUDDIV состоит из целой и дробной частей – BAUDDIV_INT и BAUDDIV_FRAC, соответственно.

Примечания

1 Изменения содержимого регистров UARTIBRD и UARTFBRD вступают в силу только после завершения передачи и приема текущего символа данных.

2 Минимальный допустимый коэффициент деления – 1, максимальный 65535 ($2^{16} - 1$). Таким образом, значение UARTIBRD, равное 0, является недопустимым, при этом значение регистра UARTFBRD игнорируется.

3 Аналогично, при UARTIBRD равном 65535 (0xFFFF), значение UARTFBRD не может быть больше нуля. Невыполнение этого условия приведет к прерыванию приема или передачи.

Далее приведен пример вычисления коэффициента деления.

Пример. Вычисление коэффициента деления

Пусть требуемая скорость передачи данных составляет 230400 бит/с, частота тактового сигнала UARTCLK равна 4 МГц. Тогда

$$\text{Коэффициент деления} = (4 \cdot 10^6) / (16 \cdot 230400) = 1,085.$$

Таким образом, BRDI = 1, BRDF = 0,085.

Следовательно, значение, записываемое в регистр UARTBFRD, равно

$$m = \text{integer} ((0,085 \cdot 64) + 0,5) = 5.$$

Реальное значение коэффициента деления = $1 + 5 / 64 = 1,078$.

Реальная скорость передачи данных = $(4 \cdot 10^6) / (16 \cdot 1,078) = 231911$ бит/с.

Ошибка установки скорости = $(231911 - 230400) / 230400 \cdot 100 \% = 0,656 \%$.

Максимальная ошибка установки скорости передачи данных с использованием шестизрядного регистра UARTBFRD = $1 / 64 \cdot 100 \% = 1,56 \%$. Такая ошибка возникает в случае $m = 1$, при этом разница накапливается в течение 64 тактовых интервалов.

В таблице 360 представлены значения коэффициента деления для типичных скоростей передачи данных при частоте UARTCLK = 7,3728 МГц. При таких параметрах дробная часть коэффициента деления не используется, следовательно, в регистр UARTFBRD должен быть записан ноль.

Таблица 360 – Коэффициенты деления при частоте UARTCLK = 7,3728 МГц

Коэффициент деления	Скорость передачи данных
0x0001	460800
0x0002	230400
0x0004	115200
0x0006	76800
0x0008	57600
0x000C	38400
0x0018	19200
0x0020	14400
0x0030	9600
0x00C0	2400
0x0180	1200
0x105D	110

В таблице 361 приведены значения коэффициента деления для типичных скоростей передачи данных при частоте UARTCLK = 4 МГц.

Таблица 361 – Коэффициенты деления при частоте UARTCLK = 4 МГц

Целая часть	Дробная часть	Требуемая скорость	Реальная скорость	Ошибка, %
0x001	0x05	230400	231911	0,656
0x002	0x0B	115200	115101	0,086
0x003	0x10	76800	76923	0,160
0x006	0x21	38400	38369	0,081
0x011	0x17	14400	14401	0,007
0x068	0x0B	2400	2400	~ 0
0x8E0	0x2F	110	110	~ 0

29.9.8 MDR_UARTx->LCR_H

Регистр управления линией

Данный регистр обеспечивает доступ к разрядам с 29 по 22 регистра UARTLCR. При сбросе все биты регистра UARTLCR_H обнуляются.

Таблица 362 показывает назначение разрядов регистра UARTLCR_H.

Таблица 362 – Регистр UARTLCR_H

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
15...8		Резерв. Не модифицируйте. При чтении выдаются нули.
7	SPS	Передача бита четности с фиксированным значением. 0 – запрещена; 1 – на месте бита четности передается инверсное значение бита EPS, оно же проверяется при приеме данных. (При EPS=0 на месте бита четности передается 1, при EPS=1 – передается 0). Значение бита SPS не играет роли в случае, если битом PEN формирование и проверка бита четности запрещен
6...5	WLEN	Длина слова – количество передаваемых или принимаемых информационных бит в кадре: 0b11 – 8 бит 0b10 – 7 бит 0b01 – 6 бит 0b00 – 5 бит
4	FEN	Разрешение работы буфера FIFO приемника и передатчика. 0 – запрещено; 1 – разрешено
3	STP2	Режим передачи двух стоповых бит. 0 – один стоповый бит; 1 – два стоповых бита. Приемник не проверяет наличие дополнительного стопового бита в кадре

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
2	EPS	Четность/нечетность. 0 – бит четности дополняет количество единиц в информационной части кадра до нечетного; 1 – до четного числа. Значение бита EPS не играет роли в случае, если битом PEN формирование и проверка бита четности запрещена
1	PEN	Разрешение проверки четности. 0 – кадр не содержит бита четности; 1 – бит четности передается в кадре и проверяется при приеме данных
0	BRK	Разрыв линии. Если этот бит установлен в 1, то по завершении передачи текущего символа на выходе UARTTXD устанавливается низкий уровень сигнала. Для правильного выполнения этой операции программное обеспечение должно обеспечить передачу сигнала разрыва в течение, как минимум, времени передачи двух информационных кадров. В нормальном режиме функционирования бит должен быть установлен в 0

Содержимое регистров UARTLCR_H, UARTIBRD и UARTFBRD совместно образует общий 30-разрядный регистр UARTLCR, который обновляется по стробу, формируемому при записи в UARTLCR_H. Таким образом, для того, чтобы изменение параметров коэффициента деления частоты обмена данными вступило в силу, после изменения значения регистров UARTIBRD и/или UARTFBRD необходимо осуществить запись данных в регистр UARTLCR_H.

Примечания

1 Изменение значений трех регистров можно осуществить корректно двумя способами:

- запись UARTIBRD, запись UARTFBRD, запись UARTLCR_H;
- запись UARTFBRD, запись UARTIBRD, запись UARTLCR_H.

2 Для того, чтобы изменить значение лишь одного из регистров (UARTIBRD или UARTFBRD) необходимо выполнить следующий шаг:

- запись UARTIBRD (или UARTFBRD), запись UARTLCR_H.

Таблица 363 показывает таблицу истинности для бит управления контролем четности PEN, EPS и SPS регистра управления линией UARTLCR_H.

Таблица 363 – Управление режимом контроля четности

PEN	EPS	SPS	Бит контроля четности
0	X	X	Не передается, не проверяется
1	1	0	Проверка четности слова данных
1	0	0	Проверка нечетности слова данных
1	0	1	Бит четности постоянно равен 1
1	1	1	Бит четности постоянно равен 0

Примечания

1 Регистры UARTLCR_H, UARTIBRD и UARTFBRD не должны изменяться:

- при разрешенной работе приемопередатчика;
- во время завершения приема или передачи данных в процессе остановки (перевода в запрещенное состояние) приемопередатчика.

2 Целостность данных в буферах FIFO не гарантируется в следующих случаях:

- после установки бита разрыва линии BRK;
- если программное обеспечение произвело остановку приемопередатчика при наличии данных в буферах FIFO после его повторного перевода в разрешенное состояние.

29.9.9 MDR_UARTx->CR

Регистр управления

После сброса все биты регистра управления, за исключением бит 9 и 8 устанавливаются в нулевое состояние. Биты 9 и 8 устанавливаются в единичное состояние.

Назначение разрядов регистра управления показано в таблице 364.

Таблица 364 – Регистр управления UARTCR

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
15	CTSEn	Разрешение управления потоком данных по CTS. 1 – разрешено, данные передаются в линию только при активном значении сигнала nUARTCTS.
14	RTSEn	Разрешение управления потоком данных по RTS. 1 – разрешено. Запрос данных от внешнего устройства осуществляется только при наличии свободного места в буфере FIFO приемника
13	Out2	Инверсия сигнала на линии состояния модема nUARTOut2. В режиме оконечного оборудования (DTE) эта линия может использоваться в качестве линии «сигнал вызова» (RI)
12	Out1	Инверсия сигнала на линии состояния модема nUARTOut1. В режиме оконечного оборудования (DTE) эта линия может использоваться в качестве линии «обнаружен информационный сигнал» (DCD)
11	RTS	Инверсия сигнала на линии состояния модема nUARTRTS
10	DTR	Инверсия сигнала на линии состояния модема nUARTDTR
9	RXE	Прием разрешен. Установка бита в 1 разрешает работу приемника. Прием данных осуществляется либо по интерфейсу асинхронного последовательного обмена, либо по интерфейсу ИК обмена SIR, в зависимости от значения бита SIREN. В случае перевода приемопередатчика в запрещенное состояние в ходе приема данных, он завершает прием текущего символа перед остановкой
8	TXE	Передача разрешена. Установка бита в 1 разрешает работу передатчика. Передача осуществляется либо по интерфейсу асинхронного последовательного обмена, либо по интерфейсу ИК обмена SIR, в зависимости от значения бита SIREN. В случае перевода приемопередатчик в запрещенное состояние в ходе передачи данных, он завершает передачу текущего символа: перед остановкой

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
7	LBE	0 – запрещено; 1 – шлейф разрешен. В режиме разрешенного шлейфа: Если установлены бит SIREN=1 и бит регистра управления тестированием UARTTCR SIRTEST=1, то сигнал с выхода кодека nSIROUT инвертируется и подается на вход кодека SIRIN. Бит SIRTEST устанавливается в 1 для того, чтобы вывести устройство из полудуплексного режима, характерного для интерфейса SIR. После окончания тестирования по шлейфу бит SIRTEST должен быть установлен в 0. Если бит SIRTEST=0, то выходная линия передатчика UARTTXD коммутируется на вход приемника UARTRXD. Как в режиме SIR, так и в режиме UART, выходные линии состояния модема коммутируются на соответствующие входные линии. После сброса бит устанавливается в 0
6...3		Резерв. Не модифицируйте. При чтении выдаются нули
2	SIRLP	Выбор режима ИК обмена с пониженным энергопотреблением: 0 – длительность импульсов данных равна 3/16 длительности передачи бита; 1 – длительность импульсов данных равна трем тактам сигнала IrLPBaud16 вне зависимости от выбранной скорости передачи данных. Выбор этого режима снижает энергопотребление, однако может привести к уменьшению дальности связи
1	SIREN	Разрешение работы кодека ИК передачи данных IrDA SIR: 0 – запрещено. Сигнал nSIROUT находится в низком состоянии, данные на входе SIRIN не обрабатываются. 1 – разрешено. Данные передаются на выход nSIROUT и принимаются с входа SIRIN. Линия UARTTXD находится в высоком состоянии. Данные на входе UARTRXD и линиях состояния модема не обрабатываются. В случае, если UARTEN=0 значение бита не играет роли
0	UARTEN	Разрешение работы приемопередатчика: 0 – работа запрещена. Перед остановкой завершается прием и/или передача обрабатываемого в текущий момент символа. 1 – работа разрешена. Производится обмен данными либо по линиям асинхронного обмена, либо по линиям ИК обмена SIR, в зависимости от состояния бита SIREN

Примечания

1 Для того, чтобы разрешить передачу данных, необходимо установить в логическую 1 биты TXE и UARTEN. Аналогично, для разрешения приема данных необходимо установить в 1 биты RXE и UARTEN.

2 Рекомендуются следующая последовательность действий для программирования регистров управления:

- остановите работу приемопередатчика;
- дождитесь окончания приема и/или передачи текущего символа данных;
- сбросьте буфер передатчика путем установки бита FEN регистра UARTLCR_H в 0;
- измените настройки регистра UARTCR;
- возобновите работу приемопередатчика.

29.9.10 MDR_UARTx->IFLS

Регистр порога прерывания по заполнению буфера FIFO

Данный регистр используется для установки порогового значения заполнения буферов передатчика и приемника, по достижению которых генерируется сигнал прерывания UARTTXINTR или UARTRXINTR, соответственно. Прерывание генерируется в момент перехода величины заполнения буфера через заданное значение.

После сброса в регистре устанавливается порог, соответствующий заполнению половины буфера. Формат регистра UARTIFLS и значения его бит представлены в таблице 365.

Таблица 365 – Регистр UARTIFLS

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...6		Резерв. Не модифицируйте. При чтении выдаются нули.
5...3	RXIFLSEL	Порог прерывания по заполнению буфера приемника: b000 = буфер заполнен на 1/8 b001 = буфер заполнен на 1/4 b010 = буфер заполнен на 1/2 b011 = буфер заполнен на 3/4 b100 = буфер заполнен на 7/8 b101-b111 = резерв
2...0	TXIFLSEL	Порог прерывания по заполнению буфера передатчика: b000 = буфер заполнен на 1/8 b001 = буфер заполнен на 1/4 b010 = буфер заполнен на 1/2 b011 = буфер заполнен на 3/4 b100 = буфер заполнен на 7/8 b101-b111 = резерв Также стоит помнить, что в случае, если сдвиговый регистр передатчика пуст, то слово, записанное в FIFO, будет сразу же переписано в сдвиговый регистр. Следовательно, для генерирования события прерывания от передатчика блока UART необходимо произвести запись в FIFO такого количества слов, которое превысит установленный порог хотя бы на одно слово с учетом описанного случая

29.9.11 MDR_UARTx->IMSC

Регистр установки сброса маски прерывания

При чтении выдается текущее значение маски. При записи производится установка или сброс маски на соответствующее прерывание.

После сброса все биты регистра маски устанавливаются в нулевое состояние.

Назначение бит регистра UARTIMSC показано в таблице 366.

Таблица 366 – Регистр UARTIMSC

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...11		Зарезервировано. Не модифицируйте. При чтении выдаются нули
10	OEIM	Маска прерывания по переполнению буфера UARTOEINTR: 1 – установлена; 0 – сброшена
9	BEIM	Маска прерывания по разрыву линии UARTBEINTR: 1 – установлена; 0 – сброшена
8	PEIM	Маска прерывания по ошибке контроля четности UARTPEINTR: 1 – установлена; 0 – сброшена
7	FEIM	Маска прерывания по ошибке в структуре кадра UARTFEINTR: 1 – установлена; 0 – сброшена
6	RTIM	Маска прерывания по таймауту приема данных UARTRTINTR: 1 – установлена; 0 – сброшена
5	TXIM	Маска прерывания от передатчика UARTTXINTR. 1 – установлена; 0 – сброшена
4	RXIM	Маска прерывания от приемника UARTRXINTR. 1 – установлена; 0 – сброшена
3	DSRMIM	Маска прерывания UARTDSRINTR по изменению состояния линии nUARTDSR: 1 – установлена; 0 – сброшена
2	DCDMIM	Маска прерывания UARTDCDINTR по изменению состояния линии nUARTDCD: 1 – установлена; 0 – сброшена
1	CTSMIM	Маска прерывания UARTCTSINTR по изменению состояния линии nUARTCTS: 1 – установлена; 0 – сброшена
0	RIMIM	Маска прерывания UARTRIINTR по изменению состояния линии nUARTRI: 1 – установлена; 0 – сброшена

29.9.12 MDR_UARTx->RIS

Регистр состояния прерываний

Этот регистр доступен только для чтения и содержит текущее состояние прерываний без учета маскирования. Данные, записываемые в регистр, игнорируются.

Предупреждение. После сброса все биты регистра, за исключением бит прерывания по состоянию модема (биты с 3 по 0), устанавливаются в 0. Значение бит прерывания по состоянию модема после сброса не определено.

Назначение бит регистра UARTRIS представлено в таблице 367.

Таблица 367 – Регистр UARTRIS

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...11		Зарезервировано. Не модифицируйте. При чтении выдаются нули
10	OERIS	Состояние прерывания по переполнению буфера UARTOEINTR
9	BERIS	Состояние прерывания по разрыву линии UARTBEINTR
8	PERIS	Состояние прерывания по ошибке контроля четности UARTPEINTR
7	FERIS	Состояние прерывания по ошибке в структуре кадра UARTFEINTR
6	RTRIS	Состояние прерывания по таймауту приема данных UARTRTINTR Бит RTRIS может быть установлен только при установленной маске прерывания по таймауту приема данных UARTRTINTR в регистре UARTIMSC. Это вызвано тем, что сигнал маски прерывания по таймауту используется в качестве разрешения перехода в режим пониженного энергопотребления. Чтение состояния прерывания по таймауту из регистров UARTRIS и UARTRIS приводит к одинаковым результатам.
5	TXRIS	Состояние прерывания от передатчика UARTTXINTR
4	RXRIS	Состояние прерывания от приемника UARTRXINTR
3	DSRRMIS	Состояние прерывания UARTDSRINTR по изменению линии nUARTDSR
2	DCDRMIS	Состояние прерывания UARTDCDINTR по изменению линии nUARTDCD
1	CTSRMIS	Состояние прерывания UARTCTSINTR по изменению линии nUARTCTS
0	RIRMIS	Состояние прерывания UARTRIINTR по изменению линии nUARTRI

29.9.13 MDR_UARTx->MIS

Регистр маскированного состояния прерываний

Этот регистр доступен только для чтения и содержит текущее состояние прерываний с учетом маскирования. Данные, записываемые в регистр, игнорируются.

После сброса все биты регистра, за исключением бит прерывания по состоянию модема (биты с 3 по 0), устанавливаются в 0. Значение бит прерывания по состоянию модема после сброса не определено.

Назначение бит регистра UARTMIS представлено в таблице 368.

Таблица 368 – Регистр UARTMIS

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...11		Зарезервировано. Не модифицируйте. При чтении выдаются нули
10	OEMIS	Маскированное состояние прерывания по переполнению буфера UARTOEINTR
9	BEMIS	Маскированное состояние прерывания по разрыву линии UARTBEINTR
8	PEMIS	Маскированное состояние прерывания по ошибке контроля четности UARTPEINTR
7	FEMIS	Маскированное состояние прерывания по ошибке в структуре кадра UARTFEINTR
6	RTMIS	Маскированное состояние прерывания по таймауту приема данных UARTRTINTR
5	TXMIS	Маскированное состояние прерывания от передатчика UARTTXINTR
4	RXMIS	Маскированное состояние прерывания от приемника UARTRXINTR
3	DSRMMIS	Маскированное состояние прерывания UARTDSRINTR по изменению линии nUARTDSR
2	DCDMMIS	Маскированное состояние прерывания UARTDCDINTR по изменению линии nUARTDCD
1	CTSMMIS	Маскированное состояние прерывания UARTCTSINTR по изменению линии nUARTCTS
0	RIMMIS	Маскированное состояние прерывания UARTRIINTR по изменению линии nUARTRI

29.9.14 MDR_UARTx->ICR

Регистр сброса прерываний

Этот регистр доступен только для записи и предназначен для сброса признака прерывания по заданному событию путем записи 1 в соответствующий бит. Запись нуля в любой из разрядов регистра игнорируется.

Назначение бит регистра UARTICR представлено в таблице 369.

Таблица 369 – Регистр UARTICR

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...11		Зарезервировано. Не модифицируйте. При чтении выдаются нули
10	OEIC	Сброс прерывания по переполнению буфера UARTOEINTR
9	BEIC	Сброс прерывания по разрыву линии UARTBEINTR
8	PEIC	Сброс прерывания по ошибке контроля четности UARTPEINTR
7	FEIC	Сброс прерывания по ошибке в структуре кадра UARTFEINTR
6	RTIC	Сброс прерывания по таймауту приема данных UARTRTINTR
5	TXIC	Сброс прерывания от передатчика UARTTXINTR
4	RXIC	Сброс прерывания от приемника UARTRXINTR
3	DSRMIC	Сброс прерывания UARTDSRINTR по изменению линии nUARTDSR
2	DCDMIC	Сброс прерывания UARTDCDINTR по изменению линии nUARTDCD
1	CTSMIC	Сброс прерывания UARTCTSINTR по изменению линии nUARTCTS
0	RIMIC	Сброс прерывания UARTRIINTR по изменению линии nUARTRI
Примечание – Для организации надежного сброса статусных флагов прерываний необходимо учитывать требования, изложенные в подразделе 31.3		

29.9.15 MDR_UARTx->DMACR

Регистр управления прямым доступом к памяти

Регистр доступен по чтению и записи. После сброса все биты регистра обнуляются.

Назначение бит регистра UARTDMACR представлено в таблице 370.

Таблица 370 – Регистр UARTDMACR

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...3		Зарезервировано. Не модифицируйте. При чтении выдаются нули
2	DMAONERR	Если бит установлен в 1, то в случае возникновения прерывания по обнаружению ошибки блокируются запросы DMA от приемника UARTRXDMASREQ и UARTRXDMAREQ
1	TXDMAE	Использование DMA при передаче. Если бит установлен в 1, то разрешено формирование запросов DMA для обслуживания буфера FIFO передатчика

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
0	RXDMAE	Использование DMA при приеме. Если бит установлен в 1, то разрешено формирование запросов DMA для обслуживания буфера FIFO приемника

30 Контроллер прямого доступа в память MDR_DMA

30.1 Основные свойства контроллера DMA

Основные свойства и отличительные особенности:

- 32 канала DMA;
- каждый канал DMA имеет свои сигналы управления передачей данных;
- каждый канал DMA имеет программируемый уровень приоритета;
- каждый уровень приоритета обрабатывается, исходя из уровня приоритета, определяемого номером канала DMA;
- поддержка различного типа передачи данных:
- память – память;
- память – периферия;
- периферия – память;
- поддержка различных типов DMA циклов;
- поддержка передачи данных различной разрядности;
- каждому каналу DMA доступна первичная и альтернативная структура управляющих данных канала;
- все управляющие данные канала хранятся в системной памяти;
- разрядность данных приемника равна разрядности данных передатчика;
- количество передач в одном цикле DMA может программироваться от 1 до 1024;
- инкремент адреса передачи может быть больше чем разрядность данных.

30.2 Термины и определения

При описании контроллера используются следующие термины:

Таблица 371 – Термины и определения

Альтернативная	Альтернативная структура управляющих данных канала. Вы можете установить соответствующий регистр для изменения типа структуры данных (см. подраздел 30.5 «Структура управляющих данных канала»)
C	Идентификатор номера канала прямого доступа. Например: C=1 - канал DMA 1 C=23 - канал DMA 23
Канал	Возможны конфигурации контроллера с числом каналов до 32. Каждый канал содержит независимые сигналы управления передачей данных, которые могут инициировать передачу данных по каналу DMA
Управляющие данные канала	Структура данных находится в системной памяти. Вы можете программировать эту структуру данных так, что контроллер может выполнять передачу данных по каналу DMA в желаемом режиме. Контроллер должен иметь доступ к области системной памяти, где находится эта информация. Примечание – Любое упоминание в документе структуры данных означает управляющие данные канала

Цикл DMA	Все передачи DMA, которые контроллер должен выполнить для передачи N пакетов данных
Передача DMA	Акция пересылки одного байта, полуслова или слова. Общее количество передач DMA, которые контроллер выполняет для канала
Пинг-понг	Режим работы для выбранного канала, при котором контроллер получает начальный запрос и затем выполняет цикл DMA, используя первичную или альтернативную структуру данных. После завершения этого цикла DMA-контроллер начинает выполнять новый цикл DMA, используя другую структуру данных. Контроллер сигнализирует об окончании каждого цикла DMA, позволяя главному процессору перенастраивать неактивную структуру данных. Контроллер продолжает переключаться от первичной к альтернативной структуре данных и обратно до тех пор, пока он не прочитает «неправильную» структуру данных или пока он не завершит цикл без переключения к другой структуре
Первичная	Первичная структура управляющих данных канала. Контроллер использует эту структуру данных, если соответствующий разряд в регистре <code>chnl_pri_alt_set</code> установлен в 0
R	Степень числа 2, устанавливающее число передач DMA, которые могут произойти перед сменой арбитража. Количество передач DMA программируется в диапазоне от 1 до 1024 двоичными шагами от 2^0 до 2^{10}
Исполнение с изменением конфигурации	Режим работы для выбранного канала, при котором контроллер получает запрос от периферии и выполняет 4 DMA передачи, используя первичную структуру управляющих данных, которые настраивают альтернативную структуру управляющих данных. После чего контроллер начинает цикл DMA, используя альтернативную структуру данных. После того, как цикл закончится и, если периферия устанавливает новый запрос на обслуживание, контроллер выполняет снова 4 DMA передачи, используя первичную структуру управляющих данных, которые опять перенастраивают альтернативную структуру управляющих данных. После чего контроллер начинает цикл DMA, используя альтернативную структуру данных. Контроллер будет продолжать работать вышеописанным способом до тех пор, пока не прочитает неправильную структуру данных или процессор не установит альтернативную структуру данных для обычного цикла. Контроллер устанавливает флаг <code>dma_done</code> , если окончание подобного режима работы происходит после выполнения обычного цикла

30.3 Функциональное описание

На рисунке 118 показана упрощенная структурная схема контроллера.

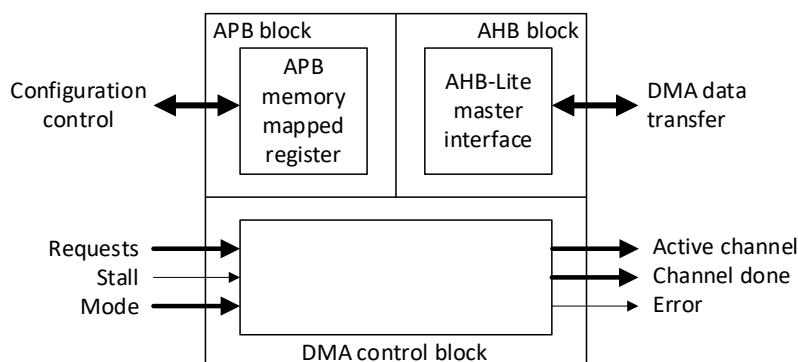


Рисунок 118 – Структурная схема контроллера

Контроллер состоит из следующих основных функциональных блоков:

- блок, подключенный к шине APB;
- блок, подключенный к шине AHB;
- управляющий блок DMA.

30.3.1 Распределение каналов DMA

Таблица 372 – Распределение каналов DMA

Номер канала	Источник sreq	Источник req	Тип	Описание
0	UART1 TX	UART1 TX		Запрос DMA от UART1 по передаче
1	UART1 RX	UART1 RX		Запрос DMA от UART1 по приему
2	UART2 TX	UART2 TX		Запрос DMA от UART2 по передаче
3	UART2 RX	UART2 RX		Запрос DMA от UART2 по приему
4	SSP1 TX	SSP1 TX		Запрос DMA от SSP1 по передаче
5	SSP1 RX	SSP1 RX		Запрос DMA от SSP1 по приему
6	SSP2 TX	SSP2 TX		Запрос DMA от SSP2 по передаче
7	SSP2 RX	SSP2 RX		Запрос DMA от SSP2 по приему
8	ADC1_EC	-		Запрос DMA от АЦП1
9	ADC2_EC	-		Запрос DMA от АЦП2
10	TIMER1	-		Запрос DMA от Timer1
11	TIMER2	-		Запрос DMA от Timer2
12	TIMER3	-		Запрос DMA от Timer3
13	-	-		Программный
14	-	-		Программный
15	-	-		Программный
16	-	-		Программный
17	-	-		Программный
18	-	-		Программный
19	-	-		Программный
20	-	-		Программный
21	-	-		Программный
22	-	-		Программный
23	-	-		Программный
24	-	-		Программный
25	-	-		Программный
26	-	-		Программный
27	-	-		Программный
28	-	-		Программный
29	-	-		Программный
30	-	-		Программный
31	-	-		Программный

30.3.2 Блок, подключенный к шине APB

Блок содержит набор регистров, позволяющих настраивать контроллер, используя ведомый APB интерфейс. Регистры занимают адресное пространство емкостью 4 Кбайт.

30.3.3 Блок, подключенный к шине АНВ

Контроллер содержит один блок типа «ведущий» шины DMA Bus, который позволяет, используя 32-разрядную шину, передавать данные от источника к приемнику. Источник и приемник являются ведомыми шины АНВ.

30.3.4 Управляющий блок DMA

Этот блок содержит схему управления, позволяющую реализовать следующие функции:

- осуществление арбитража поступающих запросов;
- индикацию активного канала;
- индикацию завершения обмена по каналу;
- индикацию состояния ошибки обмена по шине DMA Bus;
- разрешение медленным устройствам приостанавливать исполнение цикла DMA;
- ожидание запроса на очистку до завершения цикла DMA;
- осуществление одиночных или множественных передач DMA для каждого запроса;
- осуществление следующих типов DMA передач:
- память – память;
- память – периферия;
- периферия – память.

30.3.5 Типы передач

Контроллер интерфейса не поддерживает пакетные передачи. Контроллер выполняет одиночные передачи. Отсутствие возможности осуществлять пакетные передачи оказывает минимальное влияние на производительность системы, так как пакетные передачи более эффективны в одноуровневых системах с шиной АНВ, где блоки должны «захватывать» шину или обращаться к внешней памяти. В тоже время контроллер DMA предназначен для использования в многоуровневых системах с шиной АНВ, включающих встроенную память.

30.3.6 Разрядность передач данных

Контроллер интерфейса предоставляет возможность осуществлять передачу 8-, 16- и 32-разрядных данных. Таблица 373 перечисляет значения комбинаций шины HSIZE.

Таблица 373 – Комбинации шины HSIZE

HSIZE[2]*	HSIZE[1]	HSIZE[0]	Разрядность данных (бит)
0	0	0	8
0	0	1	16
	1	0	32
	1	1	**
* Сигнал постоянно удерживается в состоянии логический «0».			
** Запрещенная комбинация			

Контроллер всегда использует передачи 32-разрядными данными при обращении к управляющим данным канала. Необходимо устанавливать разрядность данных источника, соответствующую разрядности данных приемника.

30.3.7 Управление защитой данных

Контроллер позволяет устанавливать режимы защиты данных протокола АНВ-Lite, определяемые шиной HPROT[3:1]. Возможен выбор следующих режимов защиты:

- кэширование;
- буферизация;
- привилегированный.

Таблица 374 перечисляет значения комбинаций шины HPROT.

Таблица 374 – Режимы защиты данных

HPROT[3] Кэширование	HPROT[2] буферизация	HPROT[1] Привилегиро- ванный	HPROT[0] Данные/команда	Описание
-	-	-	1*	Доступ к данным
-	-	0	-	Пользовательский доступ
-	-	1	-	Привилегированный доступ
-	0	-	-	Без буферизации
-	1	-	-	Буферизированный
0	-	-	-	Без кэширования
1	-	-	-	Кэшированный
* Контроллер удерживает HPROT[0] в состоянии логической «1», чтобы обозначить доступ к данным				

Для каждого цикла DMA возможен выбор режимов защиты данных передач источника и приемника. Более подробно это описано в разделе «Настройка управляющих данных».

Для каждого канала DMA также возможен выбор режима защиты данных. Более подробно это описано в разделе Управление DMA.

30.3.8 Инкремент адреса

Контроллер позволяет управлять инкрементом адреса при чтении данных из источника и при записи данных в приемник. Инкремент адреса зависит от разрядности передаваемых данных. В таблице 375 перечислены возможные комбинации.

Таблица 375 – Инкремент адреса

Разрядность данных	Величина инкремента
8	Байт, полуслово, слово
16	Полуслово, слово
32	Слово

Минимальная величина инкремента адреса всегда соответствует разрядности передаваемых данных. Максимальная величина инкремента адреса, осуществляемая контроллером, одно слово. Более подробно о настройке инкремента адреса написано в разделе Настройка управляющих данных. Этот раздел описывает разряды управления величиной инкремента адреса в управляющих данных канала.

Примечание – Если необходимо оставлять адрес неизменным при чтении или записи данных, для примера, при работе с FIFO, можно соответствующим образом настроить контроллер на работу с фиксированным адресом (см. подраздел 30.5 «Структура управляющих данных канала»).

30.4 Управление DMA

30.4.1 Правила обмена данными

Контроллер использует правила обмена данными, перечисленные далее в таблице 376, при соблюдении следующих условий:

- канал DMA включен, что выполняется установкой в состояние логической единицы разрядов управления `chnl_enable_set[C]` и `master_enable`;
- флаги запроса `dma_req[C]` и `dma_sreq[C]` не замаскированы, что выполняется установкой в состояние логического нуля разряда управления `chnl_req_mask_set[C]`;

Таблица 376 – Правила, при которых передача данных по каналам разрешена, и запросы не маскируются

Правило	Описание
1	Если <code>dma_active[C]</code> установлен в 0, то установка в 1 <code>dma_req[C]</code> или <code>dma_sreq[C]</code> на один или более тактов сигнала <code>hclk</code> , следующих или не следующих друг за другом, инициирует передачу по каналу номер C
2	Контроллер осуществляет установку в 1 только одного разряда <code>dma_active[C]</code>
3	Контроллер устанавливает в 1 <code>dma_active[C]</code> в момент начала передачи по каналу C
4	Для типов циклов DMA, отличных от периферийного «Исполнение с изменением конфигурации», <code>dma_active[C]</code> остается в состоянии 1 до тех пор, пока контроллер не окончит передачи с номерами меньше, чем значение 2^R или чем число передач, указанное в регистре <code>n_minus_1</code> . В периферийном режиме «Исполнение с изменением конфигурации», <code>dma_active[C]</code> остается в состоянии 1 в течение каждой пары DMA передач, с использованием первичной и альтернативной структур управляющих данных. Таким образом, контроллер выполняет 2^R передач, используя первичную структуру управляющих данных, затем без осуществления арбитража выполняет передачи с номерами меньше, чем значение 2^R (или чем число передач, указанное в регистре <code>n_minus_1</code>), используя альтернативную структуру управляющих данных. По окончании последней передачи <code>dma_active[C]</code> сбрасывается в 0
5	Контроллер устанавливает <code>dma_active[C]</code> в 0 на как минимум один такт сигнала <code>hclk</code> перед тем, как снова установит <code>dma_active[C]</code> или <code>dma_active[]</code> в 1
6	Для каналов, по которым разрешена передача, контроллер осуществляет установку в 1 только одного <code>dma_done[]</code>
7	Если <code>dma_req[C]</code> устанавливается в состояние 1 в момент, когда <code>dma_active[C]</code> или <code>dma_stall</code> также в состоянии 1, то это означает, что контроллер обнаружил запрос
8	Если разряды <code>cycle_ctrl</code> для канала установлены в состояние 3'b100, 3'b101, 3'b110, 3'b111, то <code>dma_done[C]</code> никогда не будет установлен в 1

Правило	Описание
9	Если все передачи по каналу завершены, и разряды cycle_ctrl позволяют удержание dma_done[C], то по срезу сигнала dma_active[] произойдут события: - если dma_stall в состоянии 0, контроллер устанавливает dma_done[] в состояние 1 продолжительностью один такт hclk - если dma_stall в состоянии 1, работа контроллера приостановлена. После того, как dma_stall будет установлен в 0, контроллер устанавливает dma_done[] в состояние 1 продолжительностью один такт hclk
10	Состояние dma_waitonreq[C] можно изменять только при выключенном канале.
11	Если dma_waitonreq[C] в состоянии 1, то сигнал dma_active[C] не перейдет в состояние 0 до тех пор, пока: контроллер завершит 2 ^R передач (или число передач, указанное в регистре n_minus_1); dma_req[C] будет установлен в 0; dma_sreq[C] будет установлен в 0
12	Если за один такт сигнала hclk перед установкой dma_active[C] в 0 dma_stall устанавливается в 1, то контроллер установит dma_active[C] в 0 на следующем такте сигнала hclk; передача по каналу C не завершится, пока не будет сброшен в 0 dma_stall
13	Контроллер игнорирует dma_sreq[C], если dma_waitonreq[C] в состоянии 0
14	Контроллер игнорирует dma_sreq[C], если chnl_useburst_set[C] в состоянии 1*
15	Для циклов DMA, отличных по типу от периферийного режима «Исполнение с изменением конфигурации», по окончании 2 ^R передач контроллер устанавливает значение chnl_useburst_set[C] в состояние 0, если количество оставшихся передач меньше, чем 2 ^R . В периферийном режиме «Исполнение с изменением конфигурации» контроллер устанавливает значение chnl_useburst_set[C] в состояние 0 только, если количество оставшихся передач с использованием альтернативной структуры управляющих данных меньше, чем 2 ^R .
16	Для типов циклов DMA, отличных от периферийного режима «Исполнение с изменением конфигурации», если за один такт hclk до установки dma_active[C] в 1 dma_sreq[C] и dma_waitonreq[C] установлены в 1 и dma_req[C] установлен в 0, то контроллер выполняет одну DMA передачу. В периферийном режиме «Исполнение с изменением конфигурации», если за один такт hclk до установки dma_active[C] в 1 dma_sreq[C] и dma_waitonreq[C] установлены в 1 и dma_req[C] установлен в 0, контроллер выполняет 2 ^R передач с использованием первичной структуры управляющих данных. Затем без осуществления арбитража выполняет одну передачу, используя альтернативную структуру управляющих данных
17	Для типов циклов DMA, отличных от периферийного режима «Исполнение с изменением конфигурации», если за один такт hclk до установки dma_active[C] в 1, а dma_sreq[C] и dma_req[C] установлены в 1, то приоритет предоставляется dma_req[c], и контроллер выполняет 2 ^R (или число передач, указанное в регистре n_minus_1) DMA передач. В периферийном режиме «Исполнение с изменением конфигурации», если за один такт hclk до установки dma_active[C] в 1 dma_sreq[C] и dma_req[C] установлены в 1, то приоритет предоставляется dma_req[c], и контроллер выполняет 2 ^R передач с использованием первичной структуры управляющих данных, затем без осуществления арбитража выполняет передачи с номерами меньше, чем значение 2 ^R (или чем число передач, указанное в регистре n_minus_1), используя альтернативную структуру управляющих данных
18	Когда chnl_req_mask_set[C] установлен в 1, контроллер игнорирует запросы по dma_sreq[C] и dma_req[C]

Правило	Описание
	* Необходимо с осторожностью устанавливать эти разряды. Если значение, указанное в регистре <code>n_minus_1</code> меньше, чем значение 2^R , то контроллер не очистит разряды <code>chnl_useburst_set</code> и поэтому запросы по <code>dma_sreq[C]</code> будут маскированы. Если периферия не устанавливает <code>dma_req[C]</code> в состояние 1, то контроллер никогда не выполнит необходимых передач

При отключении канала контроллер осуществляет DMA передачи согласно правилам, представленным в таблице 377.

Таблица 377 – Правила осуществления DMA передач при «запрещенных» каналах

Правило	Описание
19	Если <code>dma_req[C]</code> установлен в 1, то контроллер устанавливает <code>dma_done[C]</code> в 1. Это позволяет контроллеру показать центральному процессору запрос готовности, даже если канал выключен (запрещен)
20	Если <code>dma_sreq[C]</code> установлен в 1, то контроллер устанавливает <code>dma_done[C]</code> в 1 при условии <code>dma_waitonreq[C]</code> в 1 и <code>chnl_useburst_set[C]</code> в состоянии 0. Это позволяет контроллеру показать центральному процессору запрос готовности, даже если канал выключен (запрещен)
21	<code>dma_active[C]</code> всегда удерживается в состоянии 0

30.4.2 Диаграммы работы контроллера DMA

Данный раздел описывает примеры функционирования контроллера с использованием правил обмена данными, представленных в таблице 376:

- импульсный запрос на обработку;
- запрос по уровню на обработку;
- флаги завершения;
- флаги ожидания запроса на обработку.

Примечание – Все диаграммы, показанные далее в этом подразделе на рисунках 119 – 123, подразумевают следующее:

- `hready` находится в состоянии 1;
- АНВ «ведомый» всегда дает ответ «ОКAY».

30.4.2.1 Импульсный запрос на обработку

Рисунок 119 показывает временную диаграмму работы контроллера DMA при получении импульсного запроса от периферии.

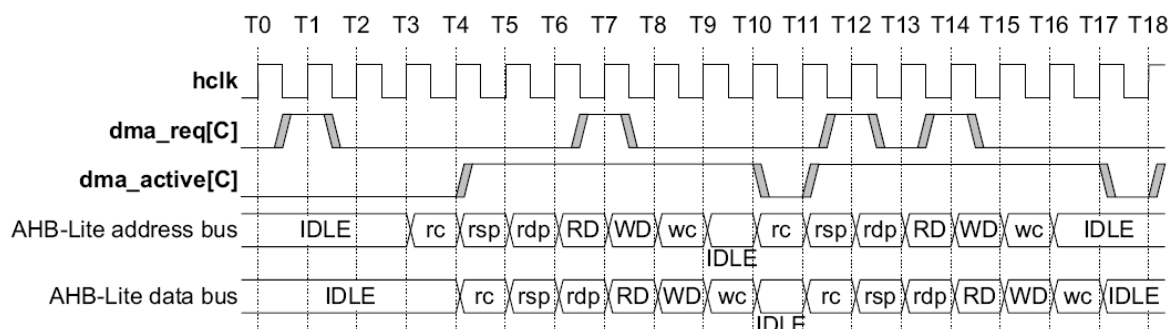


Рисунок 119 – Диаграмма работы при получении импульсного запроса

Пояснения к диаграмме на рисунке 119 приведены в таблице 378.

Таблица 378 – Пояснения к диаграмме работы при получении импульсного запроса

T1	Контроллер обнаружил запрос на обработку по каналу C (см. правило 1) при условии, что <code>chnl_req_mask_set[C]</code> находится в состоянии 0 (см. правило 18)
T4	Контроллер устанавливает <code>dma_active[C]</code> (см. правила 2 и 3) и начинает DMA передачи по каналу C
T4-T7	Контроллер считывает управляющую данные канала, где: <code>rc</code> – чтение настроек канала, <code>channel_cfg</code> ; <code>rsp</code> – чтение указателя адреса окончания данных источника, <code>src_data_end_ptr</code> ; <code>rdp</code> – чтение указателя адреса окончания данных приемника, <code>dst_data_end_ptr</code>
T7	При установленном <code>dma_active[C]</code> в 1 и при условии, что <code>chnl_req_mask_set[C]</code> находится в состоянии 0, контроллер обнаруживает импульс запроса на обработки по каналу C (см. правило 7). Контроллер обработает этот запрос в течение следующего арбитража
T7-T9	Контроллер выполняет передачу DMA по каналу C, где: <code>RD</code> – чтение данных; <code>WD</code> – запись данных
T9-T10	Контроллер осуществляет запись настроек канала, <code>channel_cfg</code> , где <code>wc</code> – запись настроек канала, <code>channel_cfg</code>
T10	Контроллер сбрасывает сигнал <code>dma_active[C]</code> , что указывает на окончание передачи DMA (см. правило 4)
T10-T11	Контроллер удерживает <code>dma_active[C]</code> как минимум на один такт <code>hclk</code> (см. правило 5)
T11	Если канал C имеет более высокий приоритет, то контроллер устанавливает <code>dma_active[C]</code> , так как ранее на такте T7 был получен запрос на обработку (см. правила 2 и 3)
T12	При установленном <code>dma_active[C]</code> в 1 и при условии, что <code>chnl_req_mask_set[C]</code> находится в состоянии 0, контроллер обнаруживает импульс запроса на обработки по каналу C (см. правило 7). Контроллер обработает этот запрос в течение следующего арбитража
T14	Контроллер игнорирует запрос по каналу C из-за отложенного запроса, полученного на такте T12
T17	Контроллер сбрасывает сигнал <code>dma_active[C]</code> , что указывает на окончание передачи DMA (см. правило 4)
T17-T18	Контроллер удерживает <code>dma_active[C]</code> , как минимум, на один такт <code>hclk</code> (см. правило 5)
T18	Если канал C имеет более высокий приоритет, то контроллер устанавливает <code>dma_active[C]</code> , так как ранее на такте T12 был получен запрос на обработку (см. правила 2 и 3)

30.4.2.2 Запрос на обработку по уровню

Рисунок 120 показывает временную диаграмму работы контроллера DMA при получении от периферии запроса на обработку по уровню.

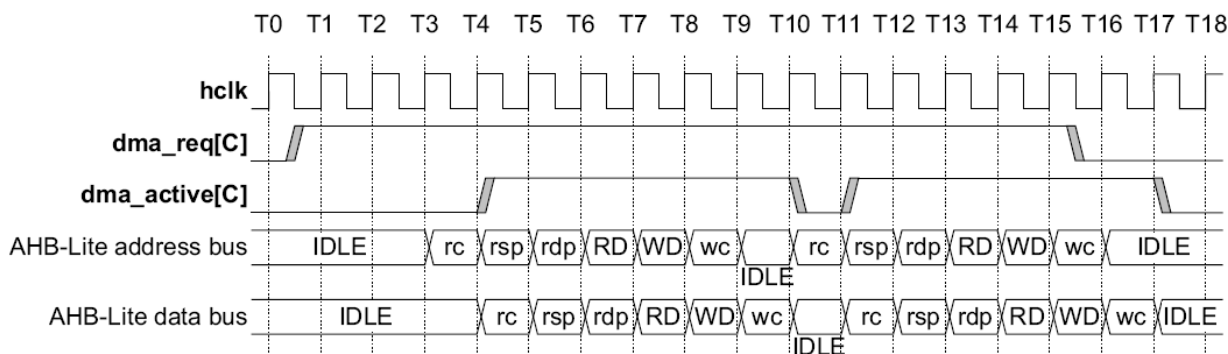


Рисунок 120 – Диаграмма работы при получении запроса на обработку по уровню.

Пояснения к диаграмме на рисунок 120 приведены в таблице 379.

Таблица 379 – Пояснения к диаграмме работы при получении запроса на обработку по уровню

T1	Контроллер обнаружил запрос на обработку по каналу C (таблица 376, правило 1) при условии, что <code>chnl_req_mask_set[C]</code> находится в состоянии 0 (см. правило 18)
T4	Контроллер устанавливает <code>dma_active[C]</code> (см. правила 2 и 3) и начинает DMA передачи по каналу C
T4-T7	Контроллер считывает управляющие данные канала, где: rc – чтение настроек канала, <code>channel_cfg</code> ; rsp – чтение указателя адреса окончания данных источника, <code>src_data_end_ptr</code> ; rdp – чтение указателя адреса окончания данных приемника, <code>dst_data_end_ptr</code>
T7-T9	Контроллер выполняет передачу DMA по каналу C, где: RD – чтение данных WD – запись данных
T9-T10	Контроллер осуществляет запись настроек канала, <code>channel_cfg</code> , где wc – запись настроек канала, <code>channel_cfg</code>
T10	Контроллер сбрасывает сигнал <code>dma_active[C]</code> , что указывает на окончание передачи DMA (см. правило 4). Контроллер обнаружил запрос на обработку по каналу C (см. правило 1) при условии, что <code>chnl_req_mask_set[C]</code> находится в состоянии 0 (см. правило 18).
T10-T11	Контроллер удерживает <code>dma_active[C]</code> на как минимум один такт <code>hclk</code> (см. правило 5)
T11	Если канал C имеет более высокий приоритет, то контроллер устанавливает <code>dma_active[C]</code> и начинает вторую DMA передачу по каналу C
T11-T14	Контроллер считывает управляющие данные канала
T14-T16	Контроллер выполняет передачу DMA по каналу C
T15-T16	Периферийный блок обнаруживает, что передача DMA началась и сбрасывает <code>dma_req[C]</code>
T16-T17	Контроллер осуществляет запись настроек канала <code>channel_cfg</code>
T17	Контроллер сбрасывает сигнал <code>dma_active[C]</code> , что указывает на окончание передачи DMA (см. правило 4)

При использовании запроса на обработку по уровню периферийный блок может не обладать достаточным быстродействием, чтобы вовремя снять сигнал запроса, в этом случае он должен установить сигнал `dma_stall`. Установка сигнала `dma_stall` предотвращает повторение выполненной передачи.

30.4.2.3 Флаги завершения

Рисунок 121 демонстрирует функционирование сигнала (флага) dma_done[] при следующих условиях:

- dma_stall и dma_waitonreq[] находятся в состоянии 0;
- dma_stall установлен в 1;
- dma_waitonreq[] установлен в 1.

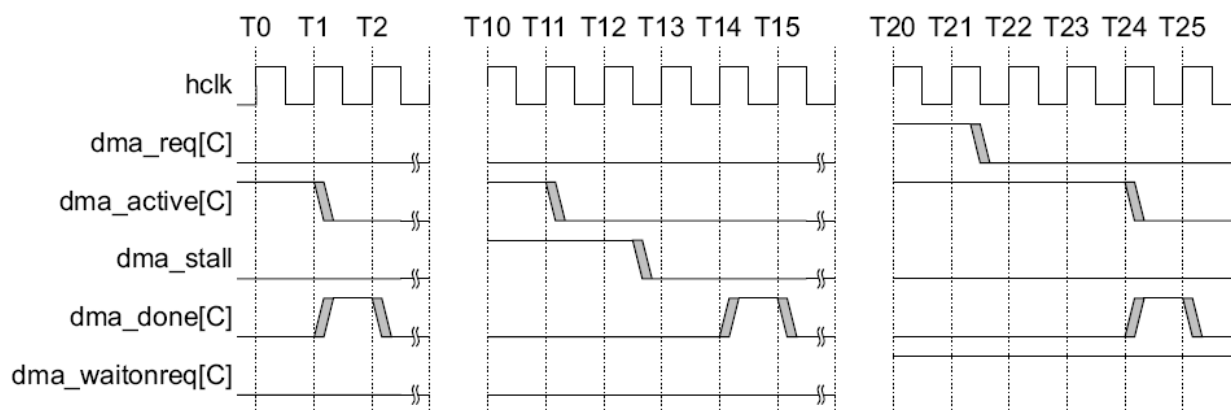


Рисунок 121 – Диаграммы функционирования dma_done

Пояснения к диаграмме на рисунке 121, такты от T0 до T2, приведены в таблице 380.

Таблица 380 – Пояснения функционирования dma_done, такты от T0 до T2

T1	Контроллер сбрасывает сигнал dma_active[C], что указывает на окончание передачи DMA (см. таблицу 376, правило 4)
T1-T2	Контроллер завершает цикл DMA и если cycle_ctrl[2] установлен в 0, то устанавливает в 1 dma_done[C] на один такт hclk (см. правила 8 и 9). Для других разрешенных каналов сигнал dma_done[C] останется в состоянии 0 (см. правило 6)

Пояснения к диаграмме на рисунке 121, такты от T10 до T15, приведены в таблице 381.

Таблица 381 – Пояснения функционирования dma_done, такты от T10 до T15

T11	Контроллер сбрасывает сигнал dma_active[C], что указывает на окончание передачи DMA (см. правило 4)
T12-T13	Периферийный блок сбрасывает сигнал dma_stall
T14-T15	Контроллер завершает цикл DMA и если cycle_ctrl[2] установлен в 0, то устанавливает в 1 dma_done[C] на один такт hclk (см. правила 8 и 9). Для других разрешенных каналов сигнал dma_done[C] останется в состоянии 0 (см. правило 6)

Примечание к T11 – Контроллер не устанавливает сигнал dma_done[C], так как сигнал dma_stall установлен в 1 в предшествующем такте hclk (см. правила 9 и 12).

Пояснения к диаграмме на рисунке 121, такты от T20 до T25, приведены в таблице 382.

Таблица 382 – Пояснения функционирования dma_done, такты от T20 до T25

T20	Контроллер выполнил передачу DMA, но из-за установленного в 1 dma_waitonreq[C] он должен ожидать сброса в 0 сигнала dma_req[C], перед тем как сбросить dma_active[C] (см. правило 11) и установить dma_done[C] (см. правило 9)
T21-T25	Периферийный блок сбрасывает dma_req[C]

T24	Контроллер сбрасывает сигнал dma_active[C], что указывает на окончание передачи DMA (см. правило 4)
T24-T25	Контроллер завершает цикл DMA и, если cycle_ctrl[2] установлен в 0, то устанавливает в 1 dma_done[C] на один такт hclk (см. правила 8 и 9). Для других разрешенных каналов сигнал dma_done[C] останется в состоянии 0 (см. правило 6)

30.4.2.4 Флаги ожидания запроса на обработку

На рисунках 122, 123 продемонстрированы примеры использования флагов ожидания запроса на обработку при выполнении 2^R передач и одиночных передач:

- диаграмма работы контроллера DMA при использовании периферией dma_waitonreq;
- диаграмма работы контроллера DMA при использовании периферией dma_waitonreq совместно с dma_sreq.

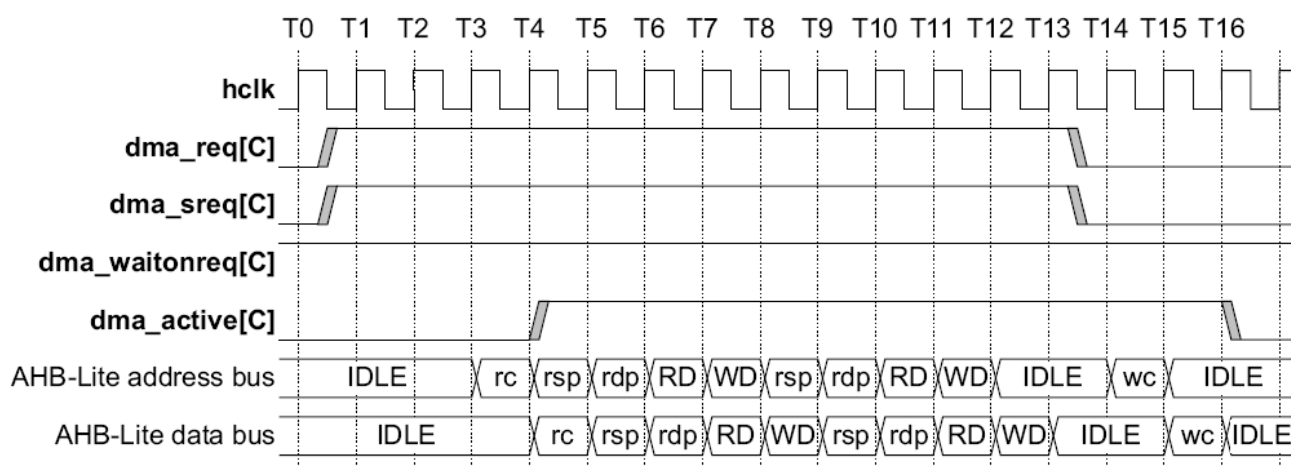


Рисунок 122 – Диаграмма работы контроллера DMA при использовании dma_waitonreq

Пояснения к диаграмме на рисунке 122 приведены в таблице 383.

Таблица 383 – Пояснения работы контроллера DMA при использовании dma_waitonreq

T0-T16	Периферийный блок должен оставлять состояние dma_waitonreq[C] постоянно (см. правило 10)
T0-T1	Контроллер обнаружил запрос на обработку по каналу C (см. правило 1) при условии, что chnl_req_mask_set[C] находится в состоянии 0 (см. правило 18)
T3-T4	Периферийный блок удерживает dma_req[C] и dma_sreq[C] в 1. Контроллер игнорирует dma_sreq[C] запрос и отвечает на dma_req[C] запрос (см. правила 16 и 17)
T4	Контроллер устанавливает dma_active[C] (см. правила 2 и 3) и начинает DMA передачи по каналу C
T4-T7	Контроллер считывает управляющие данные канала, где: rc – чтение настроек канала, channel_cfg; rsp – чтение указателя адреса окончания данных источника, src_data_end_ptr; rdp – чтение указателя адреса окончания данных приемника, dst_data_end_ptr
T7-T9	Контроллер выполняет передачу DMA по каналу C, где: RD – чтение данных; WD – запись данных
T9-T11	Контроллер считывает 2 указателя адреса окончания данных rsp и rdp
T11-T13	Периферийный блок сбрасывает сигналы dma_req[C] и dma_sreq[C]

T15-T16	Контроллер осуществляет запись настроек канала, channel_cfg, где wc – запись настроек канала, channel_cfg
T16	Контроллер сбрасывает сигнал dma_active[C], что указывает на окончание передачи DMA (см. правило 11). Контроллер устанавливает значение по чтению регистра chnl_useburst_set[C] в 0, если количество оставшихся передач менее 2^R (см. правило 15)

На рисунке 123 приведена работа контроллера DMA при установке dma_waitonreq в 1 и выполнении одиночной DMA передачи.

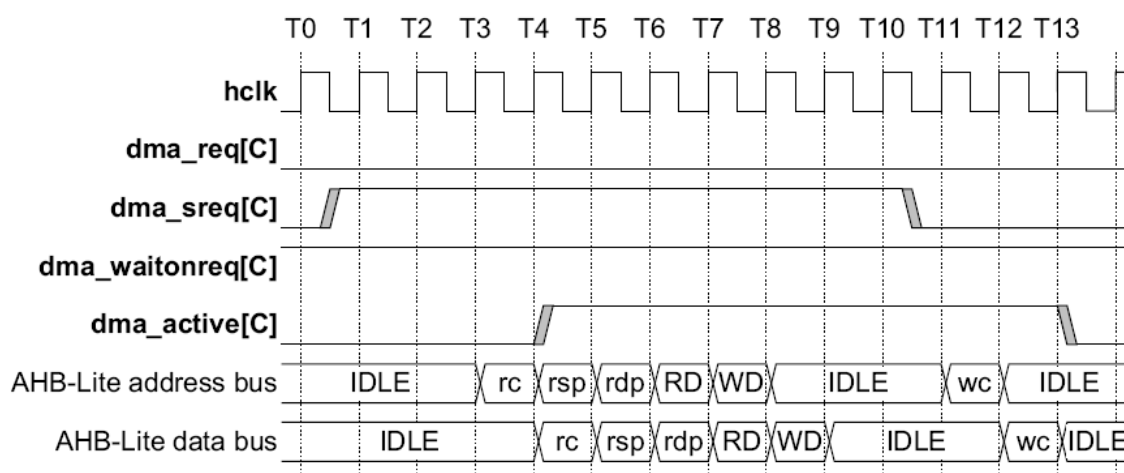


Рисунок 123 – Работа DMA при использовании dma_waitonreq совместно с dma_sreq

Пояснения к диаграмме на рисунке 123 приведены в таблице 384.

Таблица 384 – Пояснения работы DMA при использовании dma_waitonreq совместно с dma_sreq

T0-T13	Периферийный блок должен оставлять состояние dma_waitonreq[C] постоянно (см. правило 10)
T0-T1	Контроллер обнаружил запрос на обработку по каналу C (см. правило 1) при условии, что chnl_useburst_set[C] находится в состоянии 0 (см. правила 13 и 14)
T3-T4	Контроллер отвечает на dma_sreq[C] запрос (см. правила 16)
T4	Контроллер устанавливает dma_active[C] (см. правила 2 и 3) и начинает DMA передачи по каналу C
T4-T7	Контроллер считывает управляющие данные канала, где: rc – чтение настроек канала, channel_cfg; rsp – чтение указателя адреса окончания данных источника, src_data_end_ptr; rdp - чтение указателя адреса окончания данных приемника, dst_data_end_ptr
T7-T9	Контроллер выполняет передачу DMA по каналу C, где: RD – чтение данных; WD – запись данных. Это запрос в ответ на dma_sreq[], таким образом, R=0 и, следовательно, контроллер исполнит 1 DMA передачу
T10-T11	Периферийный блок сбрасывает сигнал dma_sreq[C]
T12-T13	Контроллер осуществляет запись настроек канала, channel_cfg, где wc – запись настроек канала, channel_cfg
T13	Контроллер сбрасывает сигнал dma_active[C], что указывает на окончание передачи DMA (см. правило 11)

30.4.3 Правила арбитража DMA

Контроллер имеет возможность настройки момента арбитража при передачах DMA. Эта возможность позволяет уменьшить время отклика при обслуживании каналов с высоким приоритетом.

Контроллер имеет 4 разряда, которые определяют количество транзакций по шине АНВ до повторения арбитража. Эти разряды задают степень R числа 2; изменение R напрямую устанавливает периодичность арбитража как 2 в степени R. Для примера, если R равно 4, то арбитраж будет проводиться через каждые 16 передач DMA.

Таблица 385 показывает возможную периодичность арбитража.

Таблица 385 – Периодичность арбитража в единицах передач по шине АНВ

Значение R	Периодичность арбитража каждые x передач DMA
b0000	1
b0001	2
b0010	4
b0011	8
b0100	16
b0101	32
b0110	64
b0111	128
b1000	256
b1001	512
b1010-b1111	1024

Примечание – Необходимо с осторожностью устанавливать большие значения R для низкоприоритетных каналов, так как это может привести к невозможности обслуживать запросы по высокоприоритетным каналам.

При $N > 2^R$ (N – номер передачи) и, если результат деления 2^R на N не целое число, контроллер всегда выполняет последовательность из 2^R передач до тех пор, пока не станет верным $N < 2^R$. Контроллер выполняет оставшиеся N передач в конце цикла DMA.

Разряды степени R числа 2 находятся в структуре управляющих данных канала. Местонахождение этих разрядов описано в разделе «Управляющие данные канала».

30.4.4 Приоритет

При проведении арбитража определяется канал для обслуживания в следующем цикле DMA. На выбор следующего канала влияют:

- номер канала
- уровень приоритета, присвоенного каналу.

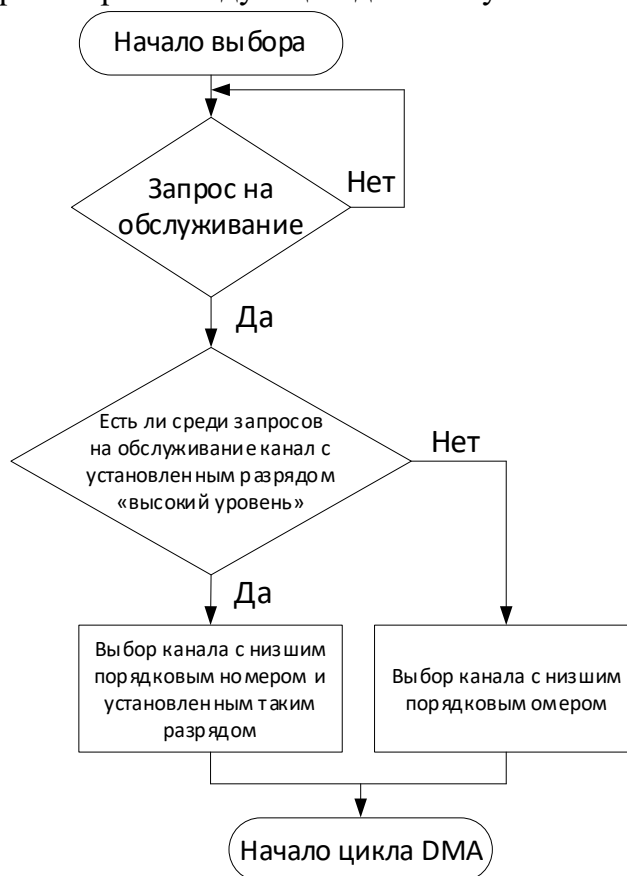
Каждому каналу может быть присвоен уровень приоритета по умолчанию (низкий) или высокий уровень приоритета. Присвоение уровня приоритета осуществляется установкой или сбросом разряда `chnl_priority_set`.

Канал номер 0 имеет высший уровень приоритета, и уровень приоритета снижается с увеличением номера канала. Таблица 386 показывает уровень приоритета каналов DMA в порядке его уменьшения.

Таблица 386 – Уровень приоритета каналов DMA

Уровень приоритета в порядке его уменьшения	Номер канала	Уровень приоритета, установленный битом <code>chnl_priority_set</code>
Наивысший уровень приоритета	0	Высокий
-	1	Высокий
-	2	Высокий
.....		
-	30	Высокий
-	31	Высокий
-	0	По умолчанию (низкий)
-	1	По умолчанию (низкий)
-	2	По умолчанию (низкий)
.....		
-	30	По умолчанию (низкий)
Низший уровень приоритета	31	По умолчанию (низкий)

После окончания цикла DMA-контроллер выбирает следующий для обслуживания



канал из всех включенных каналов DMA.

Рисунок 124 иллюстрирует процесс выбора следующего канала для обслуживания.

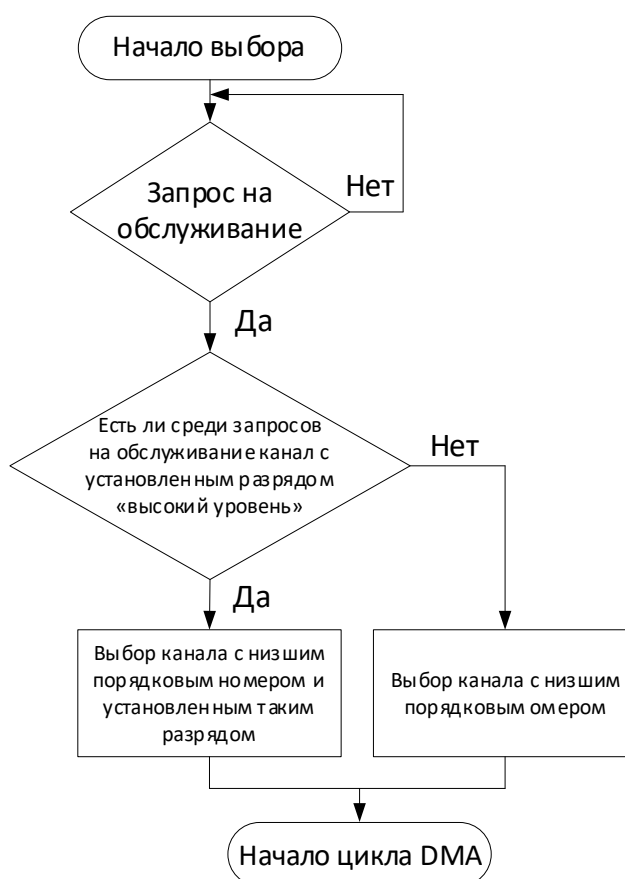


Рисунок 124 – Алгоритм выбора следующего канала для обслуживания

30.4.5 Типы циклов DMA

Разряды `cycle_ctrl` определяют, как контроллер будет выполнять циклы DMA. Описание значений этих разрядов приведено в таблице 387.

Таблица 387 – Типы циклов DMA

cycle_ctrl	Описание
b000	Структура управляющих данных канала в запрещенном состоянии
b001	Обычный цикл DMA
b010	Авто-запрос
b011	Режим «пинг-понг»
b100	Работа с памятью в режиме «Исполнение с изменением конфигурации» с использованием первичных управляющих данных канала
b101	Работа с памятью в режиме «Исполнение с изменением конфигурации» с использованием альтернативных управляющих данных канала
b110	Работа с периферией в режиме «Исполнение с изменением конфигурации» с использованием первичных управляющих данных канала
b111	Работа с периферией в режиме «Исполнение с изменением конфигурации» с использованием альтернативных управляющих данных канала

Примечание – Разряды `cycle_ctrl` находятся в области памяти, отведенной под `channel_cfg` – см. подраздел 30.5 «Структура управляющих данных канала».

Для всех типов циклов DMA повторный арбитраж происходит после 2^R передач DMA. Если установить длинный период арбитража на низкоприоритетном канале, то это заблокирует все запросы на обработку от других каналов до тех пор, пока не будут выполнены 2^R передач DMA по данному каналу. Поэтому, устанавливая значение R, необходимо учитывать, что это может привести к повышенному времени отклика на запрос на обработку от высокоприоритетных каналов.

Данный раздел описывает следующие **типы циклов DMA**:

- недействительный;
- основной;
- авто-запрос;
- «пинг-понг»;
- работа с памятью в режиме «исполнение с изменением конфигурации»;
- работа с периферией в режиме «исполнение с изменением конфигурации».

30.4.5.1 Недействительный

После окончания цикла DMA-контроллер устанавливает тип цикла в значение «недействительный» для предотвращения повтора выполненного цикла DMA.

30.4.5.2 Основной

В этом режиме контроллер работает только с основными или альтернативными управляющими данными канала. После того, как разрешена работа канала, и контроллер получил запрос на обработку, цикл DMA выглядит следующим образом:

1. Контроллер выполняет 2^R передач. Если число оставшихся передач 0, контроллер переходит к шагу 3.

2. Осуществление арбитража:

- если высокоприоритетный канал выдает запрос на обработку, то контроллер начинает обслуживание этого канала;
- если периферийный блок или программное обеспечение выдает запрос на обработку (повторный запрос на обработку по каналу), то контроллер переходит к шагу 1.

3. Контроллер устанавливает `dma_done[C]` в состояние 1 на один такт сигнала `hclk`. Это указывает центральному процессору на завершение цикла DMA.

30.4.5.3 Авто-запрос

Функционируя в данном режиме, контроллер ожидает получения одиночного запроса на обработку для разрешения работы и выполнения цикла DMA. Такая работа позволяет выполнять передачу больших пакетов данных без существенного увеличения времени отклика на обслуживание высокоприоритетных запросов и не требует множественных запросов на обработку от процессора или периферийных блоков.

Контроллер позволяет выбрать для использования первичную или альтернативную структуру управляющих данных канала. После того, как разрешена работа канала, и контроллер получил запрос на обработку, цикл DMA выглядит следующим образом:

1 Контроллер выполняет 2^R передач для канала C. Если число оставшихся передач 0, контроллер переходит к шагу 3.

2 Контроллер осуществляет арбитраж. Когда канал C становится каналом с самым высоким приоритетом, контроллер переходит к шагу 1.

3. Контроллер устанавливает `dma_done[C]` в состояние 1 на один такт сигнала `hclk`. Это указывает центральному процессору на завершение цикла DMA.

30.4.5.4 Пинг-понг

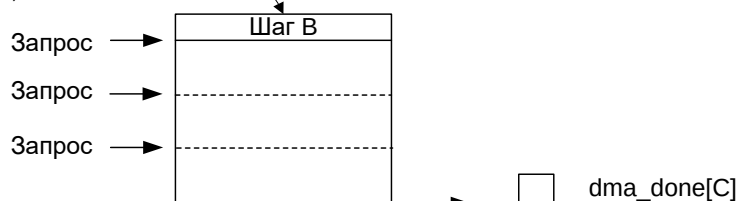
В данном режиме контроллер выполняет цикл DMA, используя одну из структур управляющих данных, а затем выполняет еще один цикл DMA, используя другую структуру управляющих данных. Контроллер выполняет циклы DMA с переключением структур до тех пор, пока не считает «неправильную» структуру данных или пока процессор не запретит работу канала.

Рисунок 125 демонстрирует пример функционирования контроллера в режиме «ПИНГ-ПОНГ».

Шаг А. Первичная структура, `cycle_ctrl = b011`,
 $2^R = 4$, $N = 6$



Шаг В. Альтернативная структура, `cycle_ctrl=b011`,
 $2^R = 4$, $N=12$



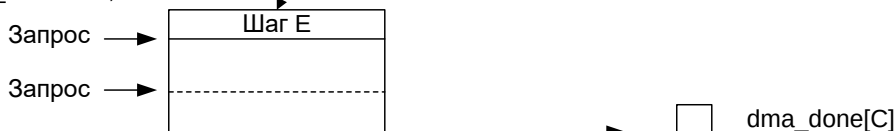
Шаг С. Первичная структура, `cycle_ctrl=b011`,
 $2^R = 2$, $N=2$



Шаг D. Альтернативная структура, `cycle_ctrl=b011`,
 $2^R = 4$, $N=5$



Шаг Е. Первичная структура, `cycle_ctrl=b011`,
 $2^R = 4$, $N=7$



Конец. Альтернативная структура, `cycle_ctrl=b000`

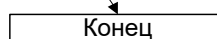


Рисунок 125 – Пример функционирования контроллера в режиме «пинг-понг»

Пояснения к схеме на рисунке 125:

Шаг А	Процессор устанавливает первичную структуру управляющих данных для шага А. Процессор устанавливает альтернативную структуру управляющих данных для шага В. Это позволит контроллеру переключиться к шагу В незамедлительно после выполнения шага А, при условии, что контроллер не получит запрос на обработку от высокоприоритетного канала. Контроллер получает запрос и выполняет четыре передачи DMA. Контроллер выполняет арбитраж. После получения запроса на обработку от этого же канала, контроллер продолжает цикл в ситуации отсутствия высокоприоритетных запросов. Контроллер выполняет оставшиеся две передачи DMA. Контроллер устанавливает dma_done[C] в состояние 1 на один такт сигнала синхронизации hclk и входит в процедуру арбитража
-------	--

После выполнения шага А процессор может установить первичные управляющие данные канала для шага С. Это позволит контроллеру переключиться к шагу С незамедлительно после выполнения шага В, при условии, что контроллер не получит запрос на обработку от высокоприоритетного канала.

После получения нового запроса на обработку от канала при условии его наивысшего приоритета исполняется шаг В:

Шаг В	Контроллер выполняет четыре передачи DMA. Контроллер выполняет арбитраж. После получения запроса на обработку от этого же канала контроллер продолжает цикл в ситуации отсутствия высокоприоритетных запросов. Контроллер выполняет четыре передачи DMA. Контроллер выполняет арбитраж. После получения запроса на обработку от этого же канала контроллер продолжает цикл в ситуации отсутствия высокоприоритетных запросов. Контроллер выполняет оставшиеся четыре передачи DMA. Контроллер устанавливает dma_done[C] в состояние 1 на один такт сигнала синхронизации hclk и входит в процедуру арбитража
-------	--

После выполнения шага В процессор может установить альтернативные управляющие данные канала для шага D.

После получения нового запроса на обработку от канала при условии его наивысшего приоритета исполняется шаг С:

Шаг С	Контроллер выполняет две передачи DMA. Контроллер устанавливает dma_done[C] в состояние 1 на один такт сигнала синхронизации hclk и входит в процедуру арбитража
-------	--

После выполнения шага С процессор может установить первичные управляющие данные канала для шага E.

После получения нового запроса на обработку от канала при условии его наивысшего приоритета исполняется шаг D:

Шаг D	Контроллер выполняет четыре передачи DMA.
-------	---

	Контроллер выполняет арбитраж. После получения запроса на обработку от этого же канала контроллер продолжает цикл в ситуации отсутствия высокоприоритетных запросов. Контроллер выполняет оставшуюся передачу DMA. Контроллер устанавливает dma_done[C] в состояние 1 на один такт сигнала синхронизации hclk и входит в процедуру арбитража
--	---

После получения нового запроса на обработку от канала при условии его наивысшего приоритета выполняется шаг E:

Шаг E	Контроллер выполняет четыре передачи DMA. Контроллер выполняет арбитраж. После получения запроса на обработку от этого же канала контроллер продолжает цикл в ситуации отсутствия высокоприоритетных запросов. Контроллер выполняет оставшиеся три передачи DMA. Контроллер устанавливает dma_done[C] в состояние 1 на один такт сигнала синхронизации hclk и входит в процедуру арбитража
-------	--

Если контроллер получит новый запрос на обработку от данного канала и этот запрос будет самым приоритетным, контроллер предпримет попытку выполнения следующего шага. Однако из-за того, что процессор не установил альтернативные управляющие данные, и по окончании шага D контроллер установил cycle_ctrl в состояние b000, передачи DMA прекращаются.

Примечание – Для прерывания цикла DMA, исполняемого в режиме «пинг-понг», также возможен перевод режима работы контроллера на шаге E в режим «Основной цикл DM» путем установки cycle_ctrl в 3'b001.

30.4.5.5 Режим работы с памятью «исполнение с изменением конфигурации»

В данном режиме контроллер, получая начальный запрос на обработку, выполняет четыре передачи DMA, используя первичные управляющие данные. По окончании этих передач контроллер начинает цикл DMA, используя альтернативные управляющие данные. Затем контроллер выполняет еще четыре передачи DMA, используя первичные управляющие данные. Контроллер продолжает выполнять циклы ПДА, меняя структуры управляющих данных, пока не произойдет одно из следующих условий:

- процессор переведет контроллер в режим «Основной» во время цикла с альтернативной структурой
- контроллер считает «неправильную» структуру управляющих данных.

Примечание – После исполнения контроллером N передач с использованием первичных управляющих данных он делает эти управляющие данные «неправильными» путем установки cycle_ctrl в 3'b000.

Контроллер устанавливает флаг dma_done[C] в этом режиме работы только тогда, когда передача DMA заканчивается с использованием основного цикла.

В данном режиме контроллер использует первичные управляющие данные для программирования альтернативных управляющих данных. Таблица 388 перечисляет

области памяти channel_cfg, как те, которые должны быть определены константами, так и те, значения которых определяются пользователем.

Таблица 388 – Channel_cfg для первичной структуры управляющих данных в режиме работы с памятью «исполнение с изменением конфигурации»

Номер бита	Обозначение	Значение	Описание
Области с константными значениями			
31...30	dst_inc	b'10	Контроллер производит инкремент адреса пословно
29...28	dst_size	b'10	Контроллер осуществляет передачу пословно
27...26	src_inc	b'10	Контроллер производит инкремент адреса пословно
25...24	src_size	b'10	Контроллер осуществляет передачу пословно
17...14	R_power	b'0010	Контроллер выполняет 4 передачи DMA
3	next_useburst	b'0	Для данного режима этот разряд должен быть равен 0
2...0	cycle_ctrl	b'100	Контроллер работает в режиме работы с периферией «исполнение с изменением конфигурации»
Области со значениями, определяемыми пользователем			
23...21	dst_prot_ctrl	-	Определяет состояние HPROT при записи данных в приемник
20...18	src_prot_ctrl	-	Определяет состояние HPROT при чтении данных из источника
13...4	n_minus_1	N*	Настраивает контроллер на выполнение N передач DMA, где N кратно 4

* Так как R_power задает значение 4, то необходимо задавать значение N, кратное 4. Число, равное N/4, это количество раз, которое нужно настраивать альтернативные управляющие данные.

Рисунок 126 демонстрирует пример функционирования в режиме работы с памятью «Исполнение с изменением конфигурации».

	src_data_end_ptr	dst_data_end_ptr	channel_cfg	Unused
Data for Task A	0x0A000000	0x0AE00000	cycle_ctrl = b101, $2^R = 4$, N = 3	0xFFFFFFFF
Data for Task B	0x0B000000	0x0BE00000	cycle_ctrl = b101, $2^R = 2$, N = 8	0xFFFFFFFF
Data for Task C	0x0C000000	0x0CE00000	cycle_ctrl = b101, $2^R = 8$, N = 5	0xFFFFFFFF
Data for Task D	0x0D000000	0x0DE00000	cycle_ctrl = b001, $2^R = 4$, N = 4	0xFFFFFFFF

Memory scatter-gather transaction:

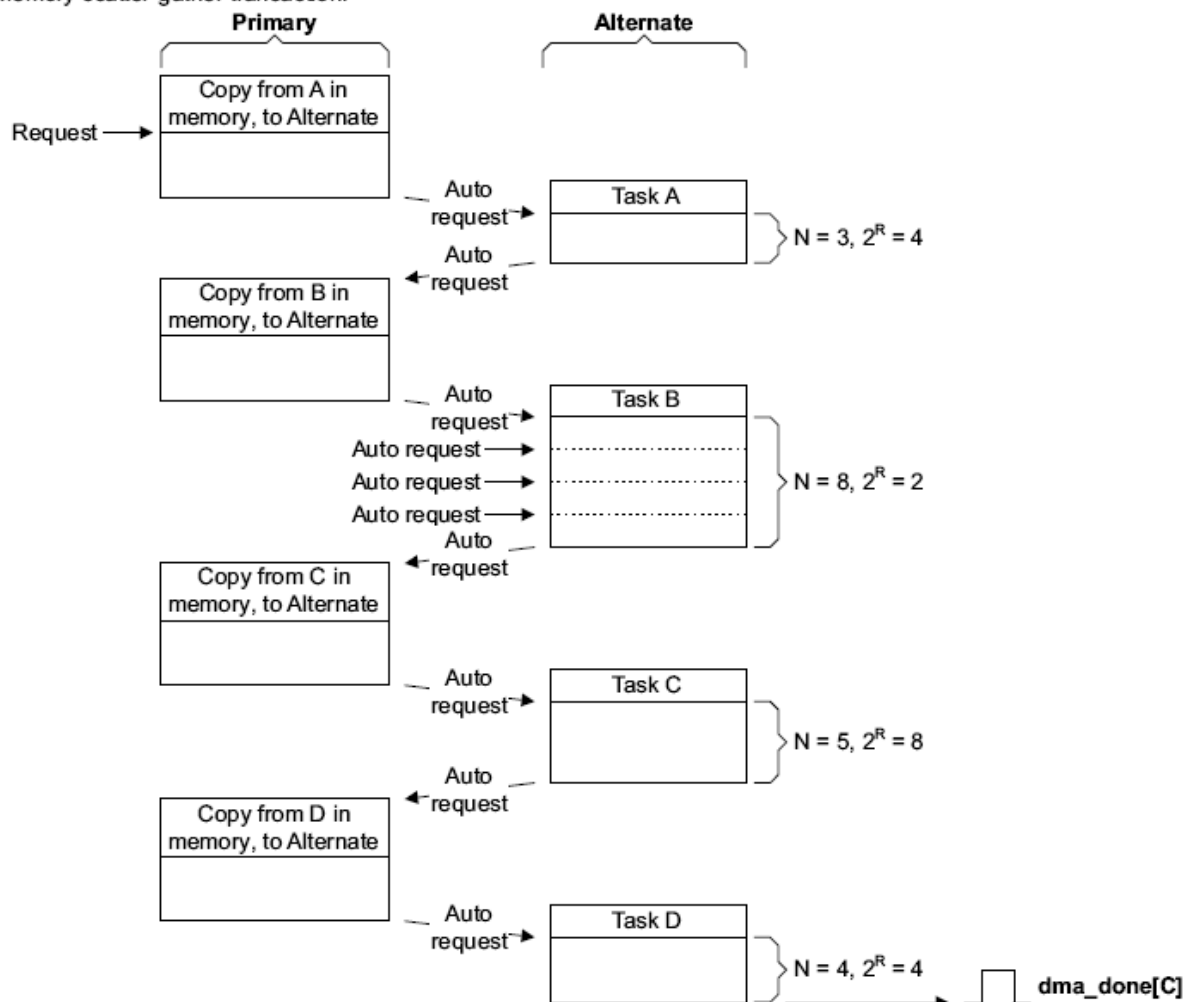


Рисунок 126 – Пример работы DMA в режиме с «Исполнением с изменением конфигурации»

Пояснения к схеме на рисунке:

Инициализация:

1 Процессор настраивает первичную структуру управляющих данных для работы в режиме работы с памятью «исполнение с изменением конфигурации» путем установки cycle_ctrl в b100. Так как управляющие данные канала состоят из четырех слов, необходимо установить 2^R в 4. В этом примере количество задач равно четырем и поэтому N установлен в 16.

2 Процессор записывает управляющие данные для шагов A, B, C, D в область памяти с адресом, указанным в src_data_end_ptr.

3 Процессор разрешает работу канала DMA.

Передачи в данном режиме начинают исполняться при получении контроллером запроса на обслуживание по `dma_req[]` или запроса от процессора. Порядок выполнения следующий:

Первичная, копирование А

По получению запроса на обслуживание контроллер выполняет четыре передачи DMA. Эти передачи записывают альтернативную структуру управляющих данных для шага А.

Контроллер генерирует автозапрос для канала, после чего проводит процедуру арбитража.

Шаг А

Контроллер выполняет шаг А. По окончании контроллер генерирует автозапрос для канала и проводит процедуру арбитража.

Первичная, копирование В

Контроллер выполняет четыре передачи DMA. Эти передачи записывают альтернативную структуру управляющих данных для шага В.

Контроллер генерирует автозапрос для канала, после чего проводит процедуру арбитража.

Шаг В

Контроллер выполняет шаг В. По окончании контроллер генерирует автозапрос для канала и проводит процедуру арбитража.

Первичная, копирование С

Контроллер выполняет четыре передачи DMA. Эти передачи записывают альтернативную структуру управляющих данных для шага С.

Контроллер генерирует автозапрос для канала, после чего проводит процедуру арбитража.

Шаг С

Контроллер выполняет шаг С. По окончании контроллер генерирует автозапрос для канала и проводит процедуру арбитража.

Первичная, копирование D

Контроллер выполняет четыре передачи DMA. Эти передачи записывают альтернативную структуру управляющих данных для шага D.

Контроллер устанавливает `cycle_ctrl` первичных управляющих данных в `b000` для индикации о том, что эта структура управляющих данных является «неправильной».

Контроллер генерирует автозапрос для канала, после чего проводит процедуру арбитража.

Шаг D

Контроллер выполняет шаг D, используя основной цикл DMA.

Контроллер устанавливает флаг `dma_done[C]` в состояние 1 на один такт сигнала `hclk` и входит в процедуру арбитража.

30.4.5.6 Режим работы с периферией «исполнение с изменением конфигурации»

В данном режиме контроллер, получая начальный запрос на обработку, выполняет 4 передачи DMA, используя первичные управляющие данные. По окончании этих передач контроллер начинает цикл DMA, используя альтернативные управляющие данные без осуществления арбитража и не устанавливая сигнал dma_active[C] в 0.

Примечание – Это единственный случай, при котором контроллер не осуществляет процедуру арбитража после выполнения передачи DMA, используя первичные управляющие данные.

После того, как этот цикл завершился, контроллер выполняет арбитраж и по получении запроса на обслуживание от периферии, имеющего наивысший приоритет, он выполняет еще 4 передачи DMA, используя первичные управляющие данные. По окончании этих передач контроллер начинает цикл DMA, используя альтернативные управляющие данные без осуществления арбитража и не устанавливая сигнал dma_active[C] в 0.

Контроллер продолжает выполнять циклы ПДА, меняя структуры управляющих данных, пока не произойдет одно из следующих условий:

- процессор переведет контроллер в режим «Основной» во время цикла с альтернативной структурой;
- контроллер считает «неправильную» структуру управляющих данных.

Примечание – После исполнения контроллером N передач с использованием первичных управляющих данных, он делает эти управляющие данные «неправильными» путем установки cycle_ctrl в 3'b000.

Контроллер устанавливает флаг dma_done[C] в этом режиме работы только тогда, когда передача DMA заканчивается с использованием основного цикла.

В данном режиме контроллер использует первичные управляющие данные для программирования альтернативных управляющих данных.

Таблица 389 перечисляет области памяти channel_cfg, как те, которые должны быть определены константами, так и те, значения которых определяются пользователем.

Таблица 389 – Channel_cfg для первичной структуры управляющих данных в режиме работы с периферией «Исполнение с изменением конфигурации»

Номер бита	Обозначение	Значение	Описание
Области с константными значениями			
31...30	dst_inc	b'10	Контроллер производит инкремент адреса пословно
29...28	dst_size	b'10	Контроллер осуществляет передачу пословно
27...26	src_inc	b'10	Контроллер производит инкремент адреса пословно
25...24	src_size	b'10	Контроллер осуществляет передачу пословно
17...14	R_power	b'0010	Контроллер выполняет четыре передачи DMA
2...0	cycle_ctrl	b'110	Контролер работает в режиме работы с периферией «исполнение с изменением конфигурации»
Области со значениями, определяемыми пользователем			
23...21	dst_prot_ctrl	-	Определяет состояние HPROT при записи данных в приемник

20...18	src_prot_ctrl	-	Определяет состояние HPROT при чтении данных из источника
13...4	n_minus_1	N*	Настраивает контроллер на выполнение N передач DMA, где N кратно 4
3	next_useburst	-	При установке в 1 контроллер установит chnl_useburst_set[C] в 1 после выполнения передачи с альтернативной структурой
* Так как R_power задает значение 4, то необходимо задавать значение N, кратное 4. Число, равное N/4, это количество раз, которое нужно настраивать альтернативные управляющие данные			

На рисунке 127 приведен пример функционирования в режиме работы с периферией «исполнение с изменением конфигурации».

Инициализация:

1. Настройка первичных управляющих данных для разрешения копирования A, B, C и D: cycle_ctrl=b110, $2^R=4$, N=16.
2. Запись первичных данных в память с использованием структуры, показанной на рисунке 127.

	src_data_end_ptr	dst_data_end_ptr	channel_cfg	Unused
Data for Task A	0x0A000000	0x0AE00000	cycle_ctrl = b111, $2^R = 4$, N = 3	0xFFFFFFFF
Data for Task B	0x0B000000	0x0BE00000	cycle_ctrl = b111, $2^R = 2$, N = 8	0xFFFFFFFF
Data for Task C	0x0C000000	0x0CE00000	cycle_ctrl = b111, $2^R = 8$, N = 5	0xFFFFFFFF
Data for Task D	0x0D000000	0x0DE00000	cycle_ctrl = b001, $2^R = 4$, N = 4	0xFFFFFFFF

Peripheral scatter-gather transaction:

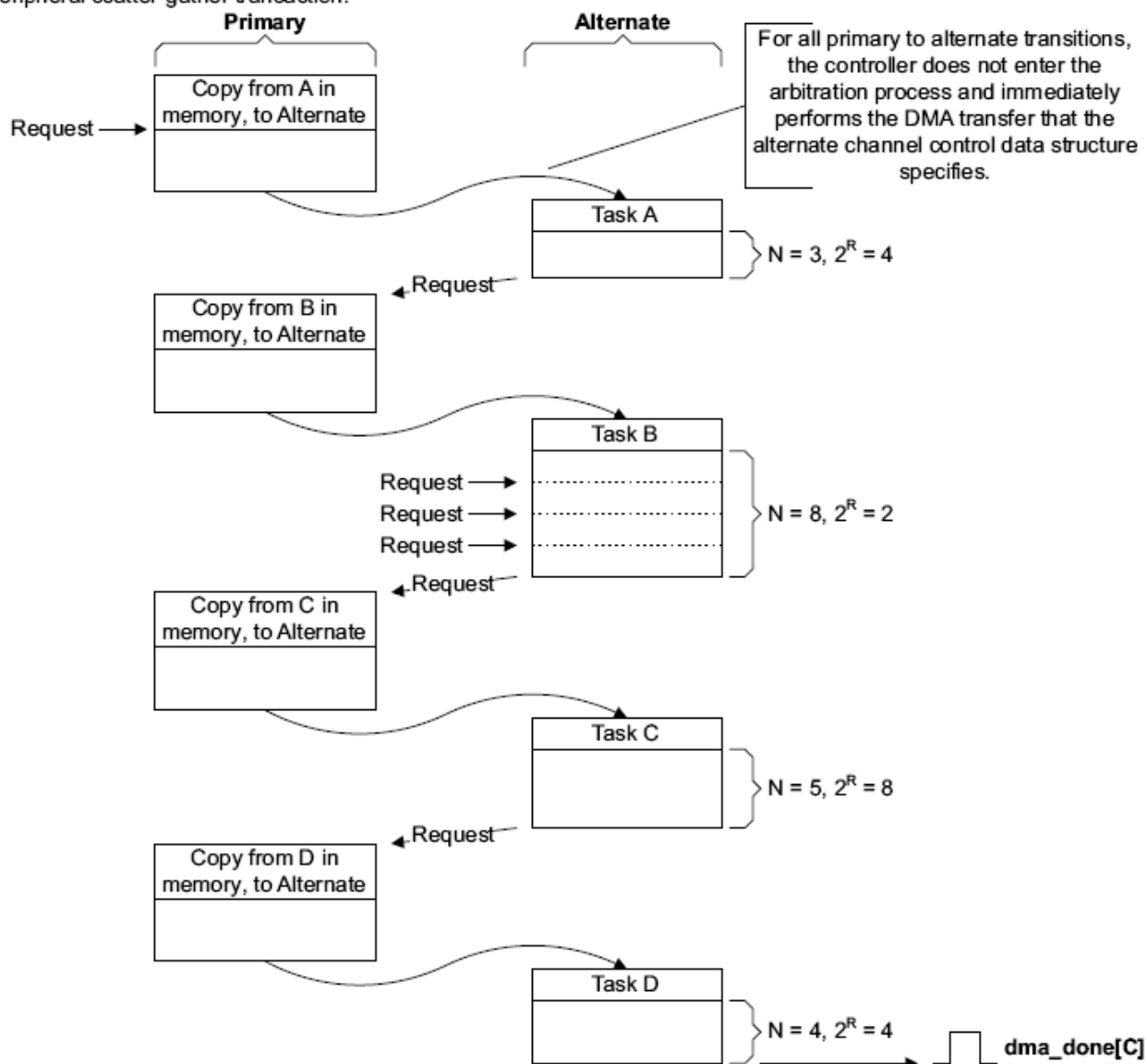


Рисунок 127 – Пример работы DMA в режиме с «Исполнением с изменением конфигурации»

Пояснения к схеме на рисунке:

Инициализация:

1 Процессор настраивает первичную структуру управляющих данных для работы в режиме работы с периферией «исполнение с изменением конфигурации» путем установки cycle_ctrl в b110. Так как управляющие данные канала состоят из четырех слов, необходимо установить 2^R в 4. В этом примере количество задач равно четырем и поэтому N установлен в 16.

2 Процессор записывает управляющие данные для шагов A, B, C, D в область памяти с адресом, указанным в src_data_end_ptr.

3 Процессор разрешает работу канала DMA.

Передачи в данном режиме начинают исполняться при получении контроллером запроса на обслуживание по dma_req[]. Передачи выполняются следующим образом:

Первичная, копирование из области А памяти

По получению запроса на обслуживание, контроллер выполняет четыре передачи DMA. Эти передачи записывают альтернативную структуру управляющих данных для шага А.

Шаг А

Контроллер выполняет шаг А.

По окончании контроллер проводит процедуру арбитража.

Первичная, копирование из области В памяти

Контроллер выполняет четыре передачи DMA. Эти передачи записывают альтернативную структуру управляющих данных для шага В.

Шаг В

Контроллер выполняет шаг В. Для завершения задачи периферия должна установить последовательно три запроса.

По окончании контроллер проводит процедуру арбитража.

Первичная, копирование из области С памяти

Контроллер выполняет четыре передачи DMA. Эти передачи записывают альтернативную структуру управляющих данных для шага С.

Шаг С

Контроллер выполняет шаг С.

По окончании контроллер проводит процедуру арбитража.

После выставления периферией нового запроса на обслуживание, при условии, что этот запрос является наиболее приоритетным, процесс продолжается следующим образом:

Первичная, копирование из области D памяти

Контроллер выполняет четыре передачи DMA. Эти передачи записывают альтернативную структуру управляющих данных для шага D.

Контроллер устанавливает cycle_ctrl первичных управляющих данных в b000 для индикации о том, что эта структура управляющих данных является «неправильной».

Шаг D

Контроллер выполняет шаг D, используя основной цикл DMA.

Контроллер устанавливает флаг dma_done[C] в состояние 1 на один такт сигнала hclk и входит в процедуру арбитража.

30.4.6 Индикация ошибок

При получении контроллером по шине АНВ ответа об ошибке, он выполняет следующие действия:

- отключает канал, связанный с ошибкой;
- устанавливает флаг dma_err в состояние 1.

После обнаружения процессором флага dma_err процессор определяет номер канала, который был активен в момент появления ошибки. Для этого он осуществляет следующее:

- чтение регистра chnl_enable_set с целью создания списка отключенных каналов;
 - если канал установил флаг dma_done[], то контроллер отключает канал.
- Программа, выполняемая процессором, должна всегда хранить данные о каналах, которые недавно установили флаги dma_done[];
- процессор должен сравнить список выключенных каналов, полученный в шаге 1, с данными о каналах, которые недавно устанавливали флаги dma_done[]. Канал, по которому отсутствуют данные об установке флага dma_done[], это и есть канал, с которым связана ошибка.

30.5 Структура управляющих данных канала

В системной памяти должна быть отведена область для хранения управляющих данных каналов. Системная память должна:

- предоставлять смежную область системной памяти, к которой контроллер и процессор имеют доступ;
- иметь базовый адрес, который целочисленно кратен общему размеру структуры управляющих данных канала.

Рисунок 128 показывает область памяти, необходимую контроллеру для структур управляющих данных канала, при использовании всех 32 каналов и опциональной альтернативной структуры управляющих данных.

Alternate data structure		Primary data structure	
Alternate_Ch_31	0x3F0	Primary_Ch_31	0x1F0
Alternate_Ch_30	0x3E0	Primary_Ch_30	0x1E0
Alternate_Ch_29	0x3D0	Primary_Ch_29	0x1D0
Alternate_Ch_28	0x3C0	Primary_Ch_28	0x1C0
Alternate_Ch_27	0x3B0	Primary_Ch_27	0x1B0
Alternate_Ch_26	0x3A0	Primary_Ch_26	0x1A0
Alternate_Ch_25	0x390	Primary_Ch_25	0x190
Alternate_Ch_24	0x380	Primary_Ch_24	0x180
Alternate_Ch_23	0x370	Primary_Ch_23	0x170
Alternate_Ch_22	0x360	Primary_Ch_22	0x160
Alternate_Ch_21	0x350	Primary_Ch_21	0x150
Alternate_Ch_20	0x340	Primary_Ch_20	0x140
Alternate_Ch_19	0x330	Primary_Ch_19	0x130
Alternate_Ch_18	0x320	Primary_Ch_18	0x120
Alternate_Ch_17	0x310	Primary_Ch_17	0x110
Alternate_Ch_16	0x300	Primary_Ch_16	0x100
Alternate_Ch_15	0x2F0	Primary_Ch_15	0x0F0
Alternate_Ch_14	0x2E0	Primary_Ch_14	0x0E0
Alternate_Ch_13	0x2D0	Primary_Ch_13	0x0D0
Alternate_Ch_12	0x2C0	Primary_Ch_12	0x0C0
Alternate_Ch_11	0x2B0	Primary_Ch_11	0x0B0
Alternate_Ch_10	0x2A0	Primary_Ch_10	0x0A0
Alternate_Ch_9	0x290	Primary_Ch_9	0x090
Alternate_Ch_8	0x280	Primary_Ch_8	0x080
Alternate_Ch_7	0x270	Primary_Ch_7	0x070
Alternate_Ch_6	0x260	Primary_Ch_6	0x060
Alternate_Ch_5	0x250	Primary_Ch_5	0x050
Alternate_Ch_4	0x240	Primary_Ch_4	0x040
Alternate_Ch_3	0x230	Primary_Ch_3	0x030
Alternate_Ch_2	0x220	Primary_Ch_2	0x020
Alternate_Ch_1	0x210	Primary_Ch_1	0x010
Alternate_Ch_0	0x200	Primary_Ch_0	0x000

Unused	0x00C
Control	0x008
Destination End Pointer	0x004
Source End Pointer	0x000

Рисунок 128 – Карта памяти для 32-х каналов, включая альтернативную структуру

Пример, показанный на рисунке 128, использует 1 Кбайт системной памяти. В этом примере контроллер использует младшие 0x10 разрядов адреса для доступа ко всем элементам структуры управляющих данных, и поэтому базовый адрес структуры должен быть 0xXXXXX000, далее 0xXXXXX400, далее 0xXXXXX800, далее 0xXXXXXC00.

Возможно, установить базовый адрес для первичной структуры управляющих данных путем записи соответствующего значения в регистр ctrl_base_ptr.

Необходимый размер области системной памяти зависит:

- от количества каналов, используемых в контроллере;
- от того, используется или нет альтернативная структура управляющих данных.

Таблица 390 перечисляет разряды адреса, обеспечивающие контроллеру доступ к различным элементам структуры управляющих данных, в зависимости от количества каналов, используемых в контроллере.

Таблица 390 – Разряды адреса, соответствующие элементам структуры управляющих данных

Количество каналов, используемых в контроллере	Разряды адреса						
	[9]	[8]	[7]	[6]	[5]	[4]	[3:0]
1						A	0x0 0x4 0x8
2					A	C[0]	
3-4				A	C[1]	C[0]	
5-8			A	C[2]	C[1]	C[0]	
9-16		A	C[3]	C[2]	C[1]	C[0]	
17-32	A	C[4]	C[3]	C[2]	C[1]	C[0]	

Где А выбирает одну из структур управляющих данных канала:

- А = 0 выбирает первичную структуру управляющих данных;
- А = 1 выбирает альтернативную структуру управляющих данных.

C[x:0] – Выбирает канал DMA.

Address[3:0] – Выбирает один из управляющих элементов:

0x0 – Выбирает указатель конца данных источника;

0x4 – Выбирает указатель конца данных приемника;

0x8 – Выбирает конфигурацию управляющих данных;

0xC – Контроллер не имеет доступа к этому адресу. Если это необходимо, то возможно разрешить процессору использовать эти адреса в качестве системной памяти.

Примечание – Совсем не обязательно вычислять базовый адрес альтернативной структуры управляющих данных, так как регистр alt_ctrl_base_ptr содержит эту информацию.

Рисунок 129 демонстрирует пример реализации контроллера с использованием 3 каналов DMA и с альтернативной структурой управляющих данных.

Перевод текста рисунка

Destination end pointer – указатель конца данных приемника;

Source end pointer – указатель конца данных источника;

Control – управление.

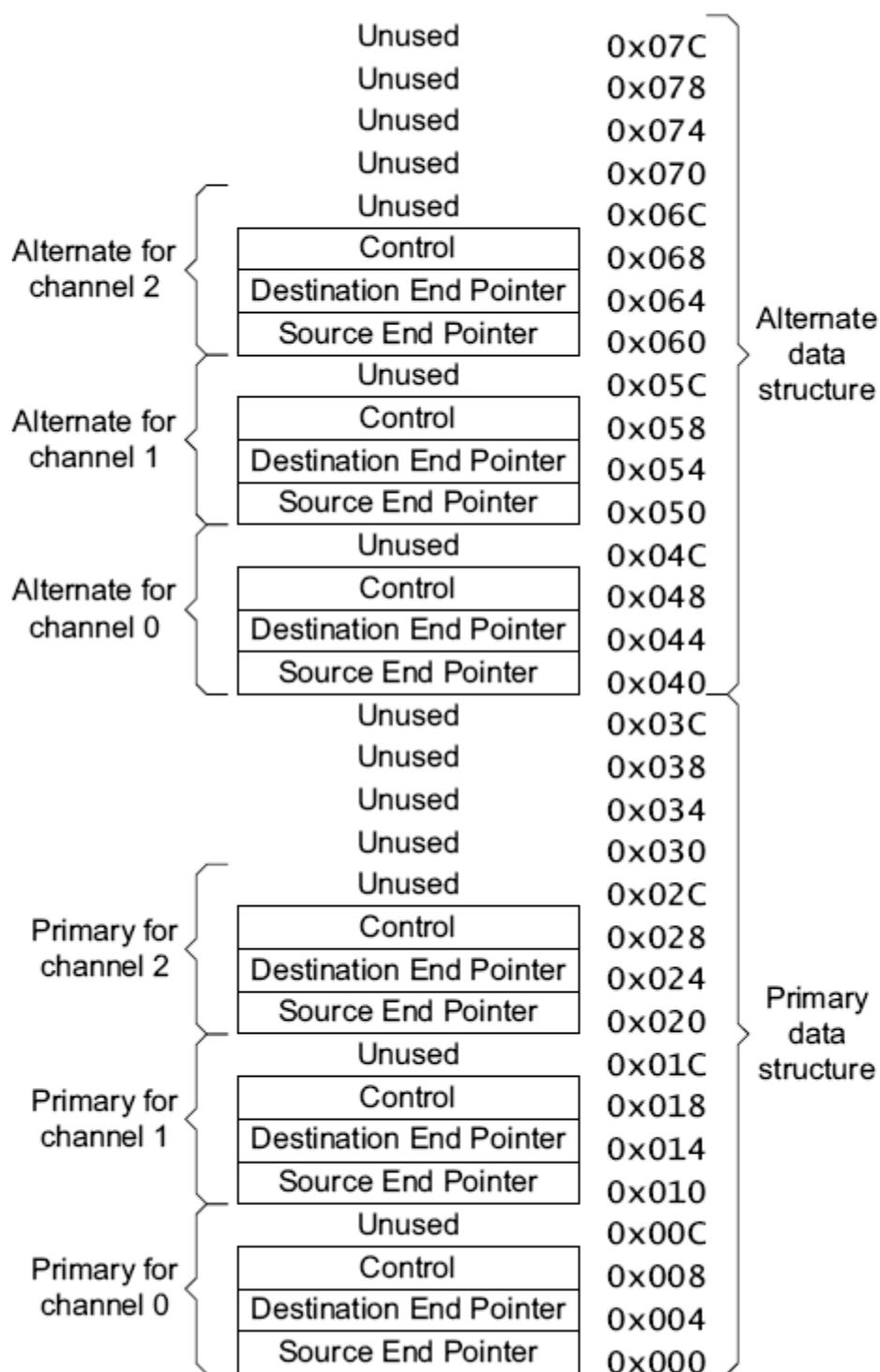


Рисунок 129 – Карта памяти для трех каналов DMA, включая альтернативную структуру

Этот пример структуры управляющих данных использует 128 байт системной памяти. В нем контроллер использует младшие 0x06 разрядов адреса для доступа ко всем элементам структуры управляющих данных. Поэтому базовый адрес структуры должен быть 0xXXXXXX00, далее 0xXXXXXX80.

Таблица 391 перечисляет все разрешенные значения базового адреса для первичной структуры управляющих данных в зависимости от количества каналов DMA, использованных в контроллере.

Таблица 391 – Разрешенные базовые адреса

Количество каналов DMA	Разрешенные значения базового адреса для первичной структуры управляющих данных
17-32	0хXXXXXX000, 0хXXXXXX400, 0хXXXXXX800, 0хXXXXXXC00

Контроллер использует системную память для доступа к двум указателям адреса конца данных и разрядам управления каждого канала. Эти 32-х разрядные области памяти и процедуру вычисления контроллером адреса передачи DMA описывают следующие подразделы:

- указатель конца данных источника;
- указатель конца данных приемника;
- разряды управления;
- вычисление адреса.

Указатель конца данных источника

Область памяти под названием src_data_end_ptr содержит указатель на последний адрес месторасположения данных источника.

Таблица 392 – Значения разрядов src_data_end_ptr

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...0	src_data_end_ptr	Указатель последнего адреса данных источника

Перед тем, как контроллер выполнит передачу DMA, необходимо определить эту область памяти. Контроллер считывает значение этой области перед началом 2^R передачи DMA.

Примечание – Контроллер не имеет доступа по записи в эту область памяти.

Указатель конца данных приемника

Область памяти под названием dst_data_end_ptr содержит указатель на последний адрес месторасположения данных приемника.

Таблица 393 – Значения разрядов dst_data_end_ptr

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...0	dst_data_end_ptr	Указатель на последний адрес данных приемника

Перед тем, как контроллер выполнит передачу DMA, необходимо определить эту область памяти. Контроллер считывает значение этой области перед началом 2^R передачи DMA.

Примечание – Контроллер не имеет доступа по записи в эту область памяти.

Разряды управления

Область памяти под названием channel_cfg обеспечивает управление каждой передачей DMA.

Таблица 394 – Название разрядов области памяти channel_cfg

Номер	31, 30	29, 28	27, 26	25, 24	23...21	20...18	17...14	13...4	3	2...0
Доступ										
Сброс										
	dst_inc	dst_size	src_inc	src_size	dst_prot_ctrl	Src_prot_ctrl	R_power	n_minus_1	next_use_burst	cycle_ctrl

Таблица 395 объясняет назначение разрядов этой области памяти.

Таблица 395 – Назначение разрядов channel_cfg

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31, 30	dst_inc	Шаг инкремента адреса приемника. Шаг инкремента адреса зависит от разрядности данных источника. Разрядность данных источника = байт: b00 = байт; b01 = полуслово (16 разрядов); b10 = слово (32 разряда); b11 = нет инкремента. Адрес остается равным значению области памяти dst_data_end_ptr. Разрядность данных источника = полуслово: b00 = зарезервировано; b01 = полуслово; b10 = слово; b11 = нет инкремента. Адрес остается равным значению области памяти dst_data_end_ptr. Разрядность данных источника = слово: b00 = зарезервировано; b01 = зарезервировано; b10 = слово (32 разряда); b11 = нет инкремента. Адрес остается равным значению области памяти dst_data_end_ptr
29, 28	dst_size	Размерность данных приемника Примечание – Значение этого поля должно быть равно значению поля src_size
27, 26	src_inc	Шаг инкремента адреса источника. Шаг инкремента адреса зависит от разрядности данных источника. Разрядность данных источника = байт: b00 = байт; b01 = полуслово; b10 = слово (32 разряда); b11 = нет инкремента. Адрес остается равным значению области памяти src_data_end_ptr.

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
		Разрядность данных источника = полуслово: b00 = зарезервировано b01 = полуслово b10 = слово b11 = нет инкремента. Адрес остается равным значению области памяти src_data_end_ptr. Разрядность данных источника = слово: b00 = зарезервировано; b01 = зарезервировано; b10 = слово; b11 = нет инкремента. Адрес остается равным значению области памяти src_data_end_ptr
25, 24	src_size	Задаёт размерность данных источника: b00 = байт; b01 = полуслово (в русском обычно слово); b10 = слово (в русском обычно двойное слово); b11 = зарезервировано
23...21	dst_prot_ctrl	Задаёт состояние HPROT[3:1], когда контроллер записывает данные в приемник. Разряд 23 управляет разрядом HPROT[3]: 0 = HPROT[3] в состоянии 0 и доступ не кэшируется; 1 = HPROT[3] в состоянии 1 и доступ кэшируется. Разряд 22 управляет разрядом HPROT[2]: 0 = HPROT[2] в состоянии 0 и доступ не буферизуется; 1 = HPROT[2] в состоянии 1 и доступ буферизуется. Разряд 21 управляет разрядом HPROT[1]: 0 = HPROT[1] в состоянии 0 и доступ непривилегированный; 1 = HPROT[1] в состоянии 1 и доступ привилегированный
20...18	src_prot_ctrl	Задаёт состояние HPROT[3:1], когда контроллер считывает данные из источника. Разряд 20 управляет разрядом HPROT[3]: 0 = HPROT[3] в состоянии 0 и доступ не кэшируется; 1 = HPROT[3] в состоянии 1 и доступ кэшируется. Разряд 19 управляет разрядом HPROT[2]: 0 = HPROT[2] в состоянии 0 и доступ не буферизуется; 1 = HPROT[2] в состоянии 1 и доступ буферизуется. Разряд 18 управляет разрядом HPROT[1]: 0 = HPROT[1] в состоянии 0 и доступ непривилегированный; 1 = HPROT[1] в состоянии 1 и доступ привилегированный
17...14	R_power	Задаёт количество передач DMA до выполнения контроллером процедуры арбитража. Возможные значения: b0000 – арбитраж производится после каждой передачи DMA; b0001 – арбитраж производится после 2 передач DMA; b0010 – арбитраж производится после 4 передач DMA; b0011 – арбитраж производится после 8 передач DMA; b0100 – арбитраж производится после 16 передач DMA; b0101 – арбитраж производится после 32 передач DMA; b0110 – арбитраж производится после 64 передач DMA; b0111 – арбитраж производится после 128 передач DMA; b1000 – арбитраж производится после 256 передач DMA;

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
		b1001 – арбитраж производится после 512 передач DMA; b1010-b1111 – арбитраж производится после 1024 передач DMA. Это означает, что арбитраж не производится, так как максимальное количество передач DMA равно 1024
13...4	n_minus_1	Перед выполнением цикла DMA эти разряды указывают общее количество передач DMA, из которых состоит цикл DMA. Необходимо установить эти разряды в значение, соответствующее размеру желаемого цикла DMA. 10-разрядное число плюс 1 задает количество передач DMA. Возможные значения: b0000000000 = 1 передача DMA; b0000000001 = 2 передачи DMA; b0000000010 = 3 передачи DMA; b0000000011 = 4 передачи DMA; b0000000100 = 5 передач DMA; b0000000101 = 6 передач DMA; b1111111111 = 1024 передачи DMA. Контроллер обновит это поле перед тем, как произвести процесс арбитража. Это позволяет контроллеру хранить количество оставшихся передач DMA до завершения цикла DMA
3	next_useburst	Контролирует, не установлен ли chnl_useburst_set[C] в состояние 1, если контроллер работает в режиме работы с периферией «Исполнение с изменением конфигурации» и, если контроллер завершает цикл DMA, используя альтернативные управляющие данные. Примечание – Перед завершением цикла DMA, использующего альтернативные управляющие данные, контроллер устанавливает chnl_useburst_set[C] в значение 0, если количество оставшихся передач DMA меньше, чем 2^R. Установка next_useburst разряда определяет, будет ли контроллер дополнительно переопределять разряд chnl_useburst_set[C]. Если контроллер выполняет цикл DMA в режиме работы с периферией «Исполнение с изменением конфигурации», то после окончания цикла, использующего альтернативные управляющие данные, происходит следующее в зависимости от состояния next_useburst: 0 – контроллер не изменяет значение chnl_useburst_set[C]. Если chnl_useburst_set[C] установлен в 0, то для всех оставшихся циклов DMA в режиме работы с периферией «Исполнение с изменением конфигурации», контроллер отвечает на запросы по dma_req[] и dma_sreq[], при выполнении циклов DMA он использует альтернативные управляющие данные. 1 – контроллер изменяет значение chnl_useburst_set[C] в состояние 1. Поэтому для оставшихся циклов DMA в режиме работы с периферией «Исполнение с изменением конфигурации», контроллер реагирует только на запросы по dma_req[], при выполнении циклов DMA он использует альтернативные управляющие данные.
2...0	cycle_ctrl	Режим работы при выполнении цикла DMA:

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
		b000 – Стоп. Означает, что структура управляющих данных является «неправильной»; b001 – Основной. Контроллер должен получить новый запрос для окончания цикла DMA, перед этим он должен выполнить процедуру арбитража; b010 – Авто-запрос. Контроллер автоматически осуществляет запрос на обработку по соответствующему каналу в течение процедуры арбитража. Это означает, что начального запроса на обработку достаточно для выполнения цикла DMA; b011 – Пинг-понг. Контроллер выполняет цикл DMA, используя одну из структур управляющих данных. По окончании выполнения цикла DMA, контроллер выполняет следующий цикл DMA, используя другую структуру. Контроллер сигнализирует об окончании каждого цикла DMA, позволяя процессору перенастраивать неактивную структуру данных. Контроллер продолжает выполнять циклы DMA, до тех пор, пока он не прочитает «неправильную» структуру данных или пока процессор не изменит cycle_ctrl поле в состояние b001 или b010; b100 – Режим работы с памятью «Исполнение с изменением конфигурации». Смотрите соответствующий раздел. При работе контроллера в данном режиме значение этого поля в первичной структуре управляющих данных должно быть b100; b101 – Режим работы с памятью «Исполнение с изменением конфигурации». Смотрите соответствующий раздел. При работе контроллера в данном режиме значение этого поля в альтернативной структуре управляющих данных должно быть b101; b110 – Режим работы с периферией «исполнение с изменением конфигурации». Смотрите соответствующий раздел. При работе контроллера в данном режиме значение этого поля в первичной структуре управляющих данных должно быть b110; b111 – Режим работы с периферией «исполнение с изменением конфигурации». Смотрите соответствующий раздел. При работе контроллера в данном режиме значение этого поля в альтернативной структуре управляющих данных должно быть b111

В начале цикла DMA или 2^R передачи DMA-контроллер считывает значение channel_cfg из системной памяти. После выполнения 2^R или N передач он сохраняет обновленное значение channel_cfg в системную память.

Контроллер не поддерживает значений dst_size, отличных от значений src_size. Если контроллер обнаруживает неравные значения этих полей, он использует значение src_size в качестве размера данных и приемника, и источника и при ближайшем обновлении поля n_minus_1, он также устанавливает значение поля dst_size, равное src_size.

После выполнения контроллером N передач, контроллер устанавливает значение поля `cycle_ctrl` в `b000`, делая тем самым `channel_cfg` данные «неправильными». Это позволяет избежать повторения выполненной передачи DMA.

Вычисление адреса

Для вычисления адреса источника передачи DMA, контроллер выполняет сдвиг влево значения `n_minus_1` на количество разрядов, соответствующее полю `src_inc`, и затем вычитает получившееся значение от значения указателя адреса конца данных источника. Подобным образом вычисляется адрес передатчика передачи DMA, контроллер выполняет сдвиг влево значения `n_minus_1` на количество разрядов, соответствующее полю `dst_inc`, и затем вычитает получившееся значение от значения указателя адреса конца данных приемника.

В зависимости от значения полей `src_inc` и `dst_inc` вычисления адресов приемника и источника выполняются по следующим уравнениям:

`src_inc=b00 and dst_inc=b00`

- адрес источника = `src_data_end_ptr - n_minus_1`
- адрес приемника = `dst_data_end_ptr - n_minus_1`.

`src_inc=b01 and dst_inc=b01`

- адрес источника = `src_data_end_ptr - (n_minus_1<<1)`
- адрес приемника = `dst_data_end_ptr - (n_minus_1<<1)`.

`src_inc=b01 and dst_inc=b10`

- адрес источника = `src_data_end_ptr - (n_minus_1<<2)`
- адрес приемника = `dst_data_end_ptr - (n_minus_1<<2)`.

`src_inc=b11 and dst_inc=b11`

- адрес источника = `src_data_end_ptr`
- адрес приемника = `dst_data_end_ptr`.

Таблица 396 перечисляет адреса приемника цикла DMA для 6 слов.

Таблица 396 – Цикла DMA для 6 слов с пословным инкрементом

Начальные значения <code>channel_cfg</code> перед циклом DMA				
<code>src_size=b10, dst_inc=b10, n_minus_1=b101, cycle_ctrl=1</code>				
DMA передачи	Указатель конца данных	Счетчик	Отличие*	Адрес
	0x2AC	5	0x14	0x298
	0x2AC	4	0x10	0x29C
	0x2AC	3	0xC	0x2A0
	0x2AC	2	0x8	0x2A4
	0x2AC	1	0x4	0x2A8
	0x2AC	0	0x0	0x2AC
Конечные значения <code>channel_cfg</code> после цикла DMA				
<code>src_size=b10, dst_inc=b10, n_minus_1=0, cycle_ctrl=0</code>				
* Это значение, полученное после сдвига влево значения счетчика на количество разрядов, соответствующее <code>dst_inc</code>				

Таблица 397 перечисляет адреса приемника для передач DMA 12 байт с использованием «полусловного» инкремента.

Таблица 397 – Цикла DMA для 12 байт с «полусловным» инкрементом

Начальные значения channel_cfg перед циклом DMA				
src_size=b00, dst_inc=b01, n_minus_1=b1011, cycle_ctrl=1, R_power=b11				
DMA передачи	Указатель конца данных	Счетчик	Отличие*	Адрес
	0x5E7	11	0x16	0x5D1
	0x5E7	10	0x14	0x5D3
	0x5E7	9	0x12	0x5D5
	0x5E7	8	0x10	0x5D7
	0x5E7	7	0xE	0x5D9
	0x5E7	6	0xC	0x5DB
	0x5E7	5	0xA	0x5DD
	0x5E7	4	0x8	0x5DF
Значения channel_cfg после 2 ^R передач DMA				
src_size=b00, dst_inc=b01, n_minus_1=b011, cycle_ctrl=1, R_power=b11				
DMA передачи	0x5E7	3	0x6	0x5E1
	0x5E7	2	0x4	0x5E3
	0x5E7	1	0x2	0x5E5
	0x5E7	0	0x0	0x5E7
Конечные значения channel_cfg после цикла DMA				
src_size=b00, dst_inc=b01, n_minus_1=0, cycle_ctrl=0**, R_power=b11				
* Это значение, полученное после сдвига влево значения счетчика на количество разрядов, соответствующее dst_inc. ** После окончания цикла DMA-контроллер делает channel_cfg «неправильным», сбрасывая в 0 поле cycle_ctrl				

30.6 Описание регистров контроллера DMA

Данный раздел описывает регистры контроллера и управление контроллером через них.

Раздел содержит следующие сведения:

- о регистровой модели контроллера;
- описание регистров.

Основные положения регистровой модели контроллера:

- нужно избегать адресации при доступе к зарезервированным или неиспользованным адресам, так как это может привести к непредсказуемым результатам;
- необходимо заполнять неиспользуемые или зарезервированные разряды регистров нулями при записи и игнорировать значения таких разрядов при считывании, кроме случаев, специально описанных в разделе;
- системный сброс или сброс по установке питания сбрасывает все регистры в состояние 0, кроме случаев, специально описанных в разделе;

– все регистры поддерживают доступ по чтению и записи, кроме случаев, специально описанных в разделе. Доступ по записи обновляет содержание регистра, а доступ по чтению возвращает содержимое регистра.

Таблица 398 – Перечень регистров контроллера

Смещение относительно базового адреса	Наименование	Тип	Значение по сбросу	Описание
0x40028000	MDR_DMA			Контроллер DMA
0x000	STATUS	RO	0x-0nn0000*	MDR_DMA->STATUS Статусный регистр DMA
0x004	CFG	WO	-	MDR_DMA->CFG Регистр конфигурации DMA
0x008	CTRL_BASE_PTR	R/W	0x00000000	MDR_DMA->CTRL_BASE_PTR Регистр базового адреса управляющих данных каналов
0x00C	ALT_CTRL_BASE_PTR	RO	0x000000nn**	MDR_DMA->ALT_CTRL_BASE_PTR Регистр базового адреса альтернативных управляющих данных каналов
0x010	WAITONREQ_STATUS	RO	0x00000000	MDR_DMA->WAITONREQ_STATUS Регистр статуса ожидания запроса на обработку каналов
0x014	CHNL_SW_REQUEST	WO	-	MDR_DMA->CHNL_SW_REQUEST Регистр программного запроса на обработку каналов
0x018	CHNL_USEBURST_SET	R/W	0x00000000	MDR_DMA->CHNL_USEBURST_SET Регистр установки пакетного обмена каналов
0x01C	CHNL_USEBURST_CLR	WO	-	MDR_DMA->CHNL_USEBURST_CLR Регистр сброса пакетного обмена каналов
0x020	CHNL_REQ_MASK_SET	R/W	0x00000000	MDR_DMA->CHNL_REQ_MASK_SET Регистр маскирования запросов на обслуживание каналов
0x024	CHNL_REQ_MASK_CLR	WO	-	MDR_DMA->CHNL_REQ_MASK_CLR Регистр очистки маскирования запросов на обслуживание каналов
0x028	CHNL_ENABLE_SET	R/W	0x00000000	MDR_DMA->CHNL_ENABLE_SET Регистр установки разрешения каналов
0x02C	CHNL_ENABLE_CLR	WO	-	MDR_DMA->CHNL_ENABLE_CLR Регистр сброса разрешения каналов
0x030	CHNL_PRI_ALT_SET	R/W	0x00000000	MDR_DMA->CHNL_PRI_ALT_SET Регистр установки первичной/альтернативной структуры управляющих данных каналов
0x034	CHNL_PRI_ALT_CLR	WO	-	MDR_DMA->CHNL_PRI_ALT_CLR Регистр сброса первичной/альтернативной структуры управляющих данных каналов

Смещение относительно базового адреса	Наименование	Тип	Значение по сбросу	Описание
0x038	CHNL_PRIORITY_SET	R/W	0x00000000	MDR_DMA->CHNL_PRIORITY_SET Регистр установки приоритета каналов
0x03C	CHNL_PRIORITY_CLR	WO	-	MDR_DMA->CHNL_PRIORITY_CLR Регистр сброса приоритета каналов
0x040-0x048	-		-	Зарезервировано
0x04C	ERR_CLR	R/W	0x00000000	MDR_DMA->ERR_CLR Регистр сброса флага ошибки
0x050-0xDFC	-	-		Зарезервировано
* Значение по сбросу зависит от количества каналов DMA, использованных в контроллере, а также от того, интегрирована ли схема тестирования. ** Значение по сбросу зависит от количества каналов DMA, использованных в контроллере				

30.6.1 MDR_DMA->STATUS

Статусный регистр DMA

Данный регистр имеет доступ только на чтение. При чтении регистр возвращает состояние контроллера. Если контроллер находится в состоянии сброса, то чтение регистра запрещено. Назначение разрядов регистра приведены в таблице 400.

Таблица 399 – Регистр status

Номер	31...28	27...21	20...16	15...8	7...4	3...1	0
Доступ	RO	U	RO	U	RO	U	RO
Сброс	0	0	0	0	0	0	0
	test_status	-	chnls_minus1	-	State	-	master_enable

Таблица 400 – Назначение разрядов регистра status

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...28	test_status	Значение при чтении: 0x0 = контроллер не имеет интегрированной схемы тестирования; 0x1 = контроллер имеет интегрированную схему тестирования; 0x2 – 0xF = не определено
27...21	-	Не определено
20...16	chnls_minus1	Количество доступных каналов DMA минус 1. Например: b00000 = контроллер имеет 1 канал DMA; b00001 = контроллер имеет 2 канала DMA; b00010 = контроллер имеет 3 канала DMA; ... b11111 = контроллер имеет 32 канала DMA
15...8	-	Не определено
7...4	state	Текущее состояние автомата управления контроллера. Состояние может быть одним из следующих: b0000 = в покое; b0001 = чтение управляющих данных канала;

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
		b0010 = чтение указателя конца данных источника; b0011 = чтение указателя конца данных приемника; b0100 = чтение данных источника; b0101 = запись данных в приемник; b0110 = ожидание запроса на выполнение DMA; b0111 = запись управляющих данных канала; b1000 = приостановлен; b1001 = выполнен; b1010 = режим работы с периферией «Исполнение с изменением конфигурации»; b1011-b1111 = не определено
3...1	-	Не определено
0	master_enable	Состояние контроллера: 0 = работа контроллера запрещена; 1 = работа контроллера разрешена

30.6.2 MDR_DMA->CFG

Регистр конфигурации DMA

Данный регистр имеет доступ только на запись. Регистр определяет состояние контроллера. Назначение разрядов регистра приведено в таблице 402.

Таблица 401 – Регистр cfg

Номер	31...8	7...5	4...1	0
Доступ	U	WO	U	WO
Сброс	0	0	0	0
	-	chnl_prot_ctrl	-	master_enable

Таблица 402 – Назначение разрядов регистра cfg

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...8	-	Не определено, следует записывать 0.
7...5	chnl_prot_ctrl	Определяет уровни индикации сигналов HPROT[3:1] защиты шины АНВ-Lite: Разряд 7 управляет сигналом HPROT[3], с целью индикации о появлении доступа с кэшированием; Разряд 6 управляет сигналом HPROT[2], с целью индикации о появлении доступа с буферизацией; Разряд 5 управляет сигналом HPROT[1], с целью индикации о появлении привилегированного доступа. Примечания 1 Если разряд[n] = 1, то соответствующий сигнал HPROT в состоянии 1; 2 Если разряд[n] = 0, то соответствующий сигнал HPROT в состоянии 0
4...1	-	Не определено. Следует записывать 0.
0	master_enable	Определяет состояние контроллера: 0 – запрещает работу контроллера;

		1 – разрешает работу контроллера
--	--	----------------------------------

30.6.3 MDR_DMA->CTRL_BASE_PTR

Регистр базового адреса управляющих данных каналов

Данный регистр имеет доступ на запись и чтение. Регистр определяет базовый адрес системной памяти размещения управляющих данных каналов.

Примечание – Контроллер не содержит внутреннюю память для хранения управляющих данных каналов.

Размер системной памяти, предназначенной контроллеру, зависит от количества каналов DMA, использующихся контроллером, а также от возможности использования альтернативных управляющих данных каналов. Поэтому количество разрядов регистра, необходимых для задания базового адреса, варьируется и зависит от варианта построения системы.

Если контроллер находится в состоянии сброса, то чтение регистра запрещено. Назначение разрядов регистра приведено в таблице 404.

Таблица 403 – Регистр ctrl_base_ptr

Номер	31...10	9...0
Доступ	R/W	U
Сброс	0	0
	ctrl_base_ptr	-

Таблица 404 – Назначение разрядов регистра ctrl_base_ptr

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...10	ctrl_base_ptr	Указатель на базовый адрес первичной структуры управляющих данных
9...0	-	Не определено. Следует записывать 0

30.6.4 MDR_DMA->ALT_CTRL_BASE_PTR

Регистр базового адреса альтернативных управляющих данных каналов

Данный регистр имеет доступ только на чтение. Регистр возвращает при чтении указатель базового адреса альтернативных управляющих данных каналов. Если контроллер находится в состоянии сброса, то чтение регистра запрещено. Этот регистр позволяет не производить вычисления базового адреса альтернативных управляющих данных каналов.

Назначение разрядов регистра приведено в таблице 406.

Таблица 405 – Регистр alt_ctrl_base_ptr

Номер	31... 0
Доступ	RO
Сброс	0
	alt_ctrl_base_ptr

Таблица 406 – Назначение разрядов регистра alt_ctrl_base_ptr

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...0	alt_ctrl_base_ptr	Указатель базового адреса альтернативной структуры управляющих данных

30.6.5 MDR_DMA->WAITONREQ_STATUS

Регистр статуса ожидания запроса на обработку каналов

Данный регистр имеет доступ только на чтение. Регистр возвращает при чтении состояние сигналов dma_waitonreq[]. Если контроллер находится в состоянии сброса, то чтение регистра запрещено. Назначения разрядов регистра приведены в таблице 408.

Таблица 407 – Регистр dma_waitonreq_status

Номер	31		2	1	0
Номер	RO		RO	RO	RO
Доступ	0		0	0	0
	dma_waitonreg_status for dma_waitnreg [31]		dma_waitonreg_status for dma_waitnreg [2]	dma_waitonreg_status for dma_waitnreg [1]	dma_waitonreg_status for dma_waitnreg [0]

Таблица 408 – Назначение разрядов регистра dma_waitonreq_status

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...0	dma_waitonreq_status	Состояние сигналов ожидания запроса на обработку каналов DMA. При чтении: Разряд [C] =0 означает, что dma_waitonreq[C] в состоянии 0 Разряд [C] =1 означает, что dma_waitonreq[C] в состоянии 1

30.6.6 MDR_DMA->CHNL_SW_REQUEST

Регистр программного запроса на обработку каналов

Данный регистр имеет доступ только на запись. Регистр позволяет устанавливать программно запрос на выполнение цикла DMA. Назначения разрядов регистра приведены в таблице 410.

Таблица 409 – Регистр chnl_sw_request

Номер	31		2	1	0
Доступ	WO		WO	WO	WO
Сброс	0		0	0	0
	chnl_sw_request for channel [31]		chnl_sw_request for channel [2]	chnl_sw_request for channel [1]	chnl_sw_request for channel [0]

Таблица 410 – Назначение разрядов регистра chnl_sw_request

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...0	chnl_sw_request	Устанавливает соответствующий разряд для генерации программного запроса на выполнение цикла DMA по соответствующему каналу DMA. При записи: Разряд [C] = 0 означает, что запрос на выполнение цикла DMA по каналу C не будет установлен; Разряд [C] = 1 означает, что запрос на выполнение цикла DMA по каналу C будет установлен. Запись разряда, соответствующего нереализованному каналу, означает, что запрос на выполнение цикла DMA не будет установлен

30.6.7 MDR_DMA->CHNL_USEBURST_SET

Регистр установки пакетного обмена каналов

Данный регистр имеет доступ на чтение и запись. Регистр отключает выполнение одиночных запросов по установке dma_sreq[] и поэтому будут обрабатываться и исполняться только запросы по dma_req[]. Регистр возвращает при чтении состояние установок пакетного обмена. Назначения разрядов регистра приведены в таблице 412.

Таблица 411 – Регистр chnl_useburst_set

Номер	31		2	1	0
Доступ	R/W		R/W	R/W	R/W
Сброс	0		0	0	0
	chnl_useburst_set for channel [31]		chnl_useburst_set for channel [2]	chnl_useburst_set for channel [1]	chnl_useburst_set for channel [0]

Таблица 412 – Назначение разрядов регистра chnl_useburst_set

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...0	chnl_useburst_set	Отключает обработку запросов на выполнение циклов DMA от dma_sreq[] и возвращает при чтении состоянии этих настроек. При чтении: Разряд [C] = 0 означает, что канал DMA C выполняет циклы DMA в ответ на запросы, полученные от dma_sreq[] и dma_req[]. Контроллер выполняет одиночные передачи или 2^R передач. Разряд [C] = 1 означает, что канал DMA C выполняет циклы DMA в ответ на запросы, полученные только от dma_req[]. Контроллер выполняет 2^R передач. При записи: Разряд [C] = 0 не дает эффекта. Необходимо использовать chnl_useburst_clr регистр и установить соответствующий разряд C в 0; Разряд [C] = 1 отключает возможность обрабатывать запросы на выполнение циклов DMA, полученные от dma_sreq[]. Контроллер выполняет 2^R передач. Запись разряда, соответствующего нереализованному каналу, не дает никакого эффекта

После выполнения предпоследней передачи из 2^R передач, в том случае, если число оставшихся передач (N) меньше чем 2^R , контроллер сбрасывает разряд `chnl_useburst_set` в 0. Это позволяет выполнять оставшиеся передачи, используя `dma_sreq[]` и `dma_req[]`.

Примечание – При программировании `channel_cfg` значением N меньшим, чем 2^R , запрещена установка соответствующего разряда `chnl_useburst_set` в случае, если периферийный блок не поддерживает сигнал `dma_req[]`.

В режиме работы с периферией «исполнение с изменением конфигурации», если разряд `next_useburst` установлен в `channel_cfg`, то контроллер устанавливает `chnl_useburst_set [C]` в 1 после окончания цикла DMA, использующего альтернативные управляющие данные.

30.6.8 MDR_DMA->CHNL_USEBURST_CLR

Регистр сброса пакетного обмена каналов

Данный регистр имеет доступ только на запись. Регистр разрешает выполнение одиночных запросов по установке `dma_sreq[]`. Назначения разрядов регистра приведены в таблице 414.

Таблица 413 – Регистр `chnl_useburst_clr`

Номер	31		2	1	0
Доступ	WO		WO	WO	WO
Сброс	0		0	0	0
	<code>chnl_useburst_clr</code> for channel [31]		<code>chnl_useburst_clr</code> for channel [2]	<code>chnl_useburst_clr</code> for channel [1]	<code>chnl_useburst_clr</code> for channel [0]

Таблица 414 – Назначение разрядов регистра `chnl_useburst_clr`

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...0	<code>chnl_useburst_clr</code>	Установка соответствующего разряда разрешает обработку запросов на выполнение циклов DMA от <code>dma_sreq[]</code>. При записи: Разряд [C] = 0 не дает эффекта. Необходимо использовать <code>chnl_useburst_set</code> регистр для отключения обработки запросов от <code>dma_sreq[]</code>; Разряд [C] = 1 разрешает обрабатывать запросы на выполнение циклов DMA, полученные от <code>dma_sreq[]</code>. Запись разряда, соответствующего нереализованному каналу, не дает никакого эффекта

30.6.9 MDR_DMA->CHNL_REQ_MASK_SET

Регистр маскирования запросов на обслуживание каналов

Данный регистр имеет доступ на чтение и запись. Регистр отключает установку запросов на выполнение циклов DMA на `dma_sreq[]` и `dma_req[]`. Регистр возвращает при

чтении состояние установок маскирования запросов от dma_sreq[] и dma_req[] на обслуживание каналов. Назначения разрядов регистра приведены в таблице 416.

Таблица 415 – Регистр chnl_req_mask_set

Номер	31		2	1	0
Доступ	R/W		R/W	R/W	R/W
Сброс	0		0	0	0
	chnl_reg_mask_set for dma_req [31] and dma_sreq [31]		chnl_reg_mask_set for dma_req[2] and dma_sreq[2]	chnl_reg_mask_set for dma_req[1] and dma_sreq[1]	chnl_reg_mask_set for dma_req[0] and dma_sreq[0]

Таблица 416 – Назначение разрядов регистра chnl_req_mask_set

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...0	chnl_req_mask_set	Отключает обработку запросов по dma_sreq[] и dma_req[] на выполнение циклов DMA от каналов и возвращает при чтении состоянии этих настроек. При чтении: Разряд [C] = 0 означает, что канал DMA C выполняет циклы DMA в ответ на поступающие запросы; Разряд [C] = 1 означает, что канал DMA C не выполняет циклы DMA в ответ на поступающие запросы. При записи: Разряд [C] = 0 не дает эффекта. Необходимо использовать chnl_req_mask_clr регистр для разрешения установки запросов; Разряд [C] = 1 отключает установку запросов на выполнение циклов DMA, по dma_sreq[] и dma_req[]. Запись разряда, соответствующего нереализованному каналу, не дает никакого эффекта

30.6.10 MDR_DMA->CHNL_REQ_MASK_CLR

Регистр очистки маскирования запросов на обслуживание каналов.

Данный регистр имеет доступ только на запись. Регистр разрешает установку запросов на выполнение циклов DMA на dma_sreq[] и dma_req[]. Назначения разрядов регистра приведены в таблице 418.

Таблица 417 – Регистр очистки маскирования запросов на обслуживание каналов

Номер	31		2	1	0
Доступ	WO		WO	WO	WO
Сброс	0		0	0	0
	chnl_reg_mask_clr for dma_req [31] and dma_sreq [31]		chnl_reg_mask_clr for dma_req [2] and dma_sreq [2]	chnl_reg_mask_clr for dma_req [1] and dma_sreq [1]	chnl_reg_mask_clr for dma_req [0] and dma_sreq [0]

Таблица 418 – Назначение разрядов регистра chnl_req_mask_clr

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...0	chnl_req_mask_clr	Установка соответствующего разряда разрешает установку запросов по dma_sreq[] и dma_req[] на выполнение циклов DMA от каналов. При записи: Разряд [C] = 0 не дает эффекта. Необходимо использовать chnl_req_mask_set регистр для отключения установки запросов; Разряд [C] = 1 разрешает установку запросов на выполнение циклов DMA, по dma_sreq[] и dma_req[]. Запись разряда, соответствующего нереализованному каналу, не дает никакого эффекта

30.6.11 MDR_DMA->CHNL_ENABLE_SET

Регистр установки разрешения каналов

Данный регистр имеет доступ на чтение и запись. Регистр разрешает работу каналов DMA, возвращает при чтении состояние разрешений работы каналов DMA. Назначения разрядов регистра приведены в таблице 420.

Таблица 419 – Регистр chnl_enable_set

Номер	31	...	2	1	0
Доступ	R/W	...	R/W	R/W	R/W
Сброс	0	...	0	0	0
	chnl_enable_set for channel [31]	...	chnl_enable_set for channel [2]	chnl_enable_set for channel [1]	chnl_enable_set for channel [0]

Таблица 420 – Назначение разрядов регистра chnl_enable_set

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...0	chnl_enable_set	Разрешает работу каналов DMA и возвращает при чтении состоянии этих настроек. При чтении: Разряд [C] = 0 означает, что канал DMA C отключен; Разряд [C] = 1 означает, что работа канала DMA C разрешена. При записи: Разряд [C] = 0 не дает эффекта. Необходимо использовать chnl_enable_clr регистр для отключения канала; Разряд [C] = 1 разрешает работу канала DMA C. Запись разряда, соответствующего нереализованному каналу, не дает никакого эффекта

30.6.12 MDR_DMA->CHNL_ENABLE_CLR

Регистр сброса разрешения каналов

Регистр запрещает работу каналов DMA, имеет доступ только на запись.

Назначения разрядов регистра приведены в таблице 422.

Таблица 421 – Регистр chnl_enable_clr

Номер	31		2	1	0
Доступ	WO		WO	WO	WO
Сброс	0		0	0	0
	chnl_enable_clr for channel 31		chnl_enable_clr for channel 2	chnl_enable_clr for channel 1	chnl_enable_clr for channel 0

Таблица 422 – Назначение разрядов регистра chnl_enable_clr

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...0	chnl_enable_clr	Установка соответствующего разряда запрещает работу соответствующего канала DMA. При записи: Разряд [C] = 0 не дает эффекта. Необходимо использовать chnl_enable_set регистр для разрешения работы канала; Разряд [C] = 1 запрещает работу канала DMA С. Запись разряда, соответствующего нереализованному каналу, не дает никакого эффекта. Примечание – Контроллер может отключить канал DMA, установив соответствующий разряд в следующих случаях: – при завершении цикла DMA; – при чтении из channel_cfg с полем cycle_ctrl установленным в b000; – при появлении ошибки на шине AHB-Lite

30.6.13 MDR_DMA->CHNL_PRI_ALT_SET

Регистр установки первичной/альтернативной структуры управляющих данных каналов

Данный регистр имеет доступ на запись и чтение. Регистр разрешает работу канала DMA с использованием альтернативной структуры управляющих данных. Чтение регистра возвращает состояние каналов DMA (какую структуру управляющих данных использует каждый канал DMA). Назначения разрядов регистра приведены в таблице 424.

Таблица 423 – Регистр chnl_pri_alt_set

Номер	31		2	1	0
Доступ	R/W		R/W	R/W	R/W
Сброс	0		0	0	0
	chnl_pri_alt_set for channel [31]		chnl_pri_alt_set for channel [2]	chnl_pri_alt_set for channel [1]	chnl_pri_alt_set for channel [0]

Таблица 424 – Назначение разрядов регистра chnl_pri_alt_set

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...0	chnl_pri_alt_set	Установка соответствующего разряда подключает использование альтернативных управляющих данных для соответствующего канала DMA, чтение возвращает состояние этих настроек. При чтении: Разряд [C] = 0 означает, что канал DMA C использует первичную структуру управляющих данных; Разряд [C] = 1 означает, что канал DMA C использует альтернативную структуру управляющих данных. При записи: Разряд [C] = 0 не дает эффекта. Необходимо использовать chnl_pri_alt_clr регистр для сброса разряда [C] в 0; Разряд [C] = 1 подключает использование альтернативной структуры управляющих данных каналом DMA C. Запись разряда, соответствующего нереализованному каналу, не дает никакого эффекта. Примечание – Контроллер может переключить значение разряда chnl_pri_alt_set[C] в следующих случаях: – при завершении четырех передач DMA указанных в первичной структуре управляющих данных при выполнении цикла DMA в режимах работы с памятью или периферией «исполнение с изменением конфигурации»; – при завершении всех передач DMA указанных в первичной структуре управляющих данных при выполнении цикла DMA в режиме «Пинг-понг»; – при завершении всех передач DMA указанных в альтернативной структуре управляющих данных при выполнении цикла DMA в режимах: – «пинг-понг»; – работа с памятью «Исполнение с изменением конфигурации»; – работа с периферией «Исполнение с изменением конфигурации»

30.6.14 MDR_DMA->CHNL_PRI_ALT_CLR

Регистр сброса первичной/альтернативной структуры управляющих данных каналов

Данный регистр имеет доступ только на запись. Регистр разрешает работу канала DMA с использованием первичной структуры управляющих данных. Назначение разрядов регистра приведены в таблице 426.

Таблица 425 – Регистр chnl_pri_alt_clr

Номер	31		2	1	0
Доступ	WO		WO	WO	WO
Сброс	0		0	0	0
	chnl_pri_alt_clr for channel [31]		chnl_pri_alt_clr for channel [2]	chnl_pri_alt_clr for channel [1]	chnl_pri_alt_clr for channel [0]

Таблица 426 – Назначение разрядов регистра chnl_pri_alt_clr

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...0	chnl_pri_alt_clr	Установка соответствующего разряда подключает использование первичных управляющих данных для соответствующего канала DMA. При записи: Разряд [C] = 0 не дает эффекта. Необходимо использовать chnl_pri_alt_set регистр для выбора альтернативных управляющих данных; Разряд [C] = 1 подключает использование первичной структуры управляющих данных каналом DMA C. Запись разряда, соответствующего нереализованному каналу, не дает никакого эффекта. Примечание – Контроллер может переключить значение разряда chnl_pri_alt_clr[C] в следующих случаях: – при завершении 4-х передач DMA указанных в первичной структуре управляющих данных при выполнении цикла DMA в режимах работы с памятью или периферией «исполнение с изменением конфигурации»; – при завершении всех передач DMA указанных в первичной структуре управляющих данных при выполнении цикла DMA в режиме «пинг-понг»; – при завершении всех передач DMA указанных в альтернативной структуре управляющих данных при выполнении цикла DMA в режимах: – «пинг-понг»; – работа с памятью «Исполнение с изменением конфигурации»; – работа с периферией «Исполнение с изменением конфигурации»

30.6.15 MDR_DMA->CHNL_PRIORITY_SET

Регистр установки приоритета каналов

Данный регистр имеет доступ на запись и чтение. Регистр позволяет присвоить высокий приоритет каналу DMA. Чтение регистра возвращает состояние приоритета каналов DMA. Назначение разрядов регистра приведены в таблице 428.

Таблица 427 – Регистр chnl_priority_set

Номер	31	...	2	1	0
Доступ	R/W	...	R/W	R/W	R/W
Сброс	0	...	0	0	0
	chnl_priorit_set for channel [31]	...	chnl_priority_set for channel [2]	chnl_priority_set for channel [1]	chnl_priority_set for channel [0]

Таблица 428 – Назначение разрядов регистра chnl_priority_set

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...0	chnl_priority_set	Установка высокого приоритета каналу DMA, чтение возвращает состояние приоритета каналов DMA. При чтении: Разряд [C] = 0 означает, что каналу DMA C присвоен уровень приоритета по умолчанию; Разряд [C] = 1 означает, что каналу DMA C присвоен высокий уровень приоритета. При записи: Разряд [C] = 0 не дает эффекта. Необходимо использовать chnl_priority_clr регистр для установки каналу C уровня приоритета по умолчанию; Разряд [C] = 1 устанавливает каналу DMA C высокий уровень приоритета. Запись разряда, соответствующего нереализованному каналу, не дает никакого эффекта

30.6.16 MDR_DMA->CHNL_PRIORITY_CLR

Регистр сброса приоритета каналов

Данный регистр имеет доступ только на запись. Регистр позволяет присвоить каналу DMA уровень приоритета по умолчанию. Назначение разрядов регистра приведены в таблице 430.

Таблица 429 – Регистр chnl_priority_clr

Номер	31	...	2	1	0
Доступ	WO	...	WO	WO	WO
Сброс	0	...	0	0	0
	chnl_priorit_clr for channel [31]	...	chnl_priority_clr for channel [2]	chnl_priority_clr for channel [1]	chnl_priority_clr for channel [0]

Таблица 430 – Назначение разрядов регистра chnl_priority_clr

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
[31:0]	chnl_priority_clr	Установка разряда присваивает соответствующему каналу DMA уровень приоритета по умолчанию. При записи: Разряд [C] = 0 не дает эффекта. Необходимо использовать chnl_priority_set регистр для установки каналу C высокого уровня приоритета. Разряд [C] = 1 устанавливает каналу DMA C уровень приоритета по умолчанию. Запись разряда, соответствующего нереализованному каналу, не дает никакого эффекта

30.6.17 MDR_DMA->ERR_CLR

Регистр сброса флага ошибки

Данный регистр имеет доступ на запись и чтение. Регистр позволяет сбрасывать сигнал dma_err в 0. Чтение регистра возвращает состояние сигнала dma_err.

Назначение разрядов регистра приведены в таблице 432.

Таблица 431 – Регистр сброса флага ошибки

Номер	31...1	0
Доступ	U	R/W
Сброс	0	0
	-	err_clr

Таблица 432 – Назначение разрядов регистра err_clr

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...1	-	Не определено. Следует записывать 0
0	err_clr	Установка сигнала в состояние 0, чтение возвращает состояние сигнала (флага) dma_err. При чтении: Разряд [C] = 0 означает, что dma_err находится в состоянии 0; Разряд [C] = 1 означает, что dma_err находится в состоянии 1. При записи: Разряд [C] = 0 не дает эффекта. Состояние dma_err останется неизменным; Разряд [C] = 1 сбрасывает сигнал (флаг) dma_err в состояние 0. Примечание – При сбросе сигнала dma_err одновременно с появлением ошибки на шине АНВ-Lite, приоритет отдается ошибке, и, следовательно, значение регистра (и dma_err) остается неизменным (несброшенным)

31 Прерывания и исключения

Состояние исключений:

Inactive – исключение не находится в стадии Active или Pending

Pending – исключение находится в состоянии ожидания обработки процессором. Запрос прерывания от периферийных блоков или программы может изменить состояние соответствующего прерывания на состояние Pending

Active – исключение начало обрабатываться процессором, но еще не закончено. Обработчик исключения может быть прерван другим обработчиком исключения. В этом случае оба исключения находятся в состоянии Active

Active и Pending – исключение начало обрабатываться процессором, но появилось новое исключение в состоянии pending от того же источника.

31.1 Типы исключений

31.1.1 RESET

RESET вызывается при включении питания и горячем сбросе. Модель исключений трактует RESET как специальную форму исключения. Когда выставляется RESET, работа процессора останавливается потенциально в любой точке инструкций. Когда RESET убирается, выполнение перезапускается с адреса, заданного в таблице векторов для сброса. Выполнение перезапускается с уровнем privileged в thread режиме.

31.1.2 NON MASKABLE INTERRUPT (NMI)

Немаскируемое прерывание (NMI) может быть вызвано только программным путем. Это самое высокоприоритетное исключение после сброса. Всегда разрешено и имеет фиксированный приоритет – 2.

NMI не может быть:

- замаскировано или предотвращено от активации из другого исключения;
- прерывает любые исключения, кроме RESET.

31.1.3 Hard Fault

Исключение Hard Fault возникает при ошибке при обработке исключений или потому, что исключение не может быть обработано каким-либо другим механизмом. Hard fault имеет фиксированный приоритет -1, означающий, что оно имеет больший приоритет, чем любое из исключений с конфигурируемым приоритетом.

31.1.4 Memory Management fault

Исключение Memory Management fault возникает при срабатывании по защите памяти. Блок MPU или фиксированные защитные настройки определяют это исключение, как для данных, так и для инструкций. Исключение используется для прерывания доступа за инструкцией в область EXECUTE NEVER (XN), если блок MPU не используется.

31.1.5 Bus Fault

Исключение возникает при ошибке памяти при выполнении выборки инструкций или обращения за данными. Это может быть при возникновении ошибки на шинах доступа к памяти, например, обращение в несуществующую память.

31.1.6 Usage Fault

Исключение USAGE FAULT возникает при сбоях при выполнении инструкции.

Например:

- выполнение неизвестной инструкции;
- обращение к некорректно выровненным данным;
- некорректное состояние при выполнении инструкции;
- ошибка при возвращении из обработчика.

Данное исключение может быть сконфигурировано и используется для обработки следующих ситуаций:

- невыровненный адрес при обращении за полусловами halfword и словами word;
- деление на ноль.

31.1.7 SVCall

Исключение Supervisor Call (SVCALL) возникает при выполнении инструкции SVC. В приложениях с использованием Операционных Сред инструкция SVC может использоваться для доступа к функциям ОС и драйверам устройств.

31.1.8 PendSV

Исключение PendSV является прерыванием запросом сервисов системного уровня. В приложениях с использованием ОС PendSV используется для переключения контекстов, когда нет других активных исключений.

31.1.9 SysTick

Исключение SysTick генерируется системным таймером, когда он обнуляется. Программное обеспечение также может генерировать исключение SysTick. В приложениях с использованием ОС процессор может использовать это исключение для подсчета системных циклов

31.2 Прерывания (IRQ)

Прерывания или IRQ – это исключения, вызываемые периферийными устройствами или программными запросами. Все прерывания асинхронны по отношению к выполняемым инструкциям. В системе прерывания используются для коммуникации периферии и процессора

Таблица 433 – Различные типы исключений

Номер исключения	Номер IRQ	Тип	Приоритет	Адрес вектора обработчика (смещение)	Активация
1	-	RESET	-3, наивысший	0x0000_0004	Асинхронный
2	-14	NMI	-2	0x0000_0008	Асинхронный
3	-13	Hard Fault	-1	0x0000_000C	-
4	-12	Memory Management Fault	Конфигурируемый	0x0000_0010	Синхронный
5	-11	Bus Fault	Конфигурируемый	0x0000_0014	Синхронный/ Асинхронный
6	-10	Usage Fault	Конфигурируемый	0x0000_0018	Синхронный
7-10	-	-	-	Зарезервировано	-
11	-5	SVCall	Конфигурируемый	0x0000_002C	Синхронный
12-13	-	-	-	Зарезервировано	-
14	-2	PendSV	Конфигурируемый	0x0000_0038	Асинхронный
15	-1	SysTick	Конфигурируемый	0x0000_003C	Асинхронный
16 и выше	0 и выше	IRQ	Конфигурируемый	0x0000_0040 и выше	Асинхронный

Для асинхронных исключений, кроме RESET, процессор может выполнить другие инструкции между возникновением сигнала исключения и входом в обработчик.

Программа в режиме Privileged может запретить прерывания, имеющие конфигурируемый приоритет.

31.3 Обработчики исключений

Для обработки исключений используются:

Процедуры обработки прерываний (Interrupt Service Routines – ISRs)

Прерывания с IRQ0 по IRQ31 обрабатываются процедурами ISR.

Если программный код, сбрасывающий флаг прерывания на уровне периферийного блока, находится в конце функции обработчика прерывания, ядро может не успеть выполнить очистку флага в момент выхода из обработчика, из-за чего прерывание будет вызвано повторно. Чтобы этого избежать, необходимо после сброса флага прерывания сделать фиктивное чтение, например, регистра статусных флагов периферийного блока в отдельную переменную.

Обработчики ошибок (Fault Handlers)

Обрабатывают исключения Hard fault, memory management fault, usage fault и bus fault.

Системные обработчики (System handlers)

Обрабатывают исключения NMI, PendSV, SVCall и SysTick.

31.4 Таблица векторов

Таблица векторов содержит указатель стека, вектор входа по RESET и стартовые адреса обработчиков, также называемых векторами. Рисунок 130 представляет последовательность векторов в таблице. Младший бит всех векторов должен быть равен 1, указывая на то, что обработчик выполняется в Thumb режиме.

Exception number	IRQ number	Offset	Vector
47	31	0x00BC	IRQ31
.	.	.	
.	.	.	
.	.	0x004C	
18	2	0x0048	IRQ2
17	1	0x0044	IRQ1
16	0	0x0040	IRQ0
15	-1	0x003C	Systick
14	-2	0x0038	PendSV
13			Reserved
12			Reserved for Debug
11	-5	0x002C	SVCall
10			
9			
8			
7			
6	-10	0x0018	Usage fault
5	-11	0x0014	Bus fault
4	-12	0x0010	Memory management fault
3	-13	0x000C	Hard fault
2	-14	0x0008	Reserved
1		0x0004	Reset
		0x0000	Initial SP value

Рисунок 130 – Таблица векторов исключений и прерываний

При системном сбросе таблица векторов располагается по фиксированному адресу 0x00000000. Программное обеспечение в privileged режиме может перенести таблицу в другое место памяти через регистр VTOR. Таблица может располагаться в адресах от 0x00000080 до 0x3ffff80. Подробнее в описании регистра VTOR.

31.5 Приоритеты исключений

Исключения имеют следующие приоритеты:

- меньшее значение приоритета означает больший приоритет;
- конфигурируемы все приоритеты, кроме RESET и Hard Fault;
- если программное обеспечение не задает приоритетов, то все они имеют приоритет 0;

- конфигурируемый приоритет может быть в диапазоне от 0 до 7. Это означает, что RESET, Hard Fault и NMI, имеющие отрицательное значение приоритета, всегда имеют больший приоритет;
- если имеется несколько исключений с одинаковым приоритетом, то больший приоритет имеет исключение с меньшим порядковым номером;
- если процессор выполняет обработчик исключения и происходит исключение с большим приоритетом, то происходит переход на обработчик исключения с большим приоритетом;
- если при выполнении обработчика произошло исключение с таким же приоритетом, то это исключение будет выполнено по завершении текущего обработчика, несмотря на порядковый номер исключения;

31.5.1 Группировка приоритетов прерываний

Для увеличения управляемости приоритетов в системах с прерываниями контроллер прерываний NVIC поддерживает группировку приоритетов. Это достигается за счет разбиения регистра приоритета прерывания на две части:

- верхняя часть определяет группу приоритетов;
- нижняя часть задает подприоритет в группе.

Только приоритет группы определяет последовательность обработки прерываний. Когда процессор выполняет обработку прерывания, другое прерывание с таким же приоритетом группы не прервет обработку первоначального обработчика. При возникновении нескольких прерываний, имеющих одинаковый приоритет группы, подприоритеты определяют последовательность их обработки. При возникновении нескольких прерываний с одинаковым приоритетом группы и подприоритетом первым обрабатывается прерывание с меньшим номером.

31.6 Вход в обработчик и выход из обработчика

31.6.1 Приоритетное прерывание

Выполнение процессором процедуры обработки исключительной ситуации (далее по тексту – исключения), может быть прервано в случае возникновения исключения с приоритетом выше, чем у обрабатываемого. Подробнее данный вопрос рассмотрен в пункте 31.5.1 «Группировка приоритетов прерываний». В случае если внутри обработчика исключения возникает прерывание более высокого приоритета, возникает ситуация, называемая вложенным исключением. Подробнее данный вопрос рассмотрен в пункте 31.6.5 «Вход в процедуру обработки исключения».

31.6.2 Возврат

Возврат из программы-обработчика осуществляется по завершении обработки исключительной ситуации, с одновременным выполнением следующих условий:

- в системе отсутствуют необработанные исключения с достаточным приоритетом;

– завершённый обработчик не обрабатывал запоздавшее исключение (late-arriving exception).

Процессор обращается к стеку и восстанавливает состояние, имевшее место до вызова обработчика. Более подробная информация дана в пункте 31.6.6 «Возврат из обработчика исключения»

31.6.3 Передача управления без восстановления контекста (tail-chaining)

Данный механизм ускоряет процесс обработки исключений. По завершении выполнения обработчика осуществляется проверка наличия необработанных исключений и в случае, если исключения, требующие вызова обработчика, присутствуют, восстановление состояния процессора из стека не производится, а управление передается непосредственно на новый обработчик.

31.6.4 Запоздавшее исключение (late-arriving exception)

В случае, если во время сохранения состояния при входе в обработчик возникла исключительная ситуация с более высоким приоритетом, процессор передает управление непосредственно высокоприоритетному обработчику.

Подобный способ обработки высокоприоритетного исключения возможен до момента начала выполнения первой инструкции процедуры обработки исключительной ситуации. После возврата из обработчика запоздавшего исключения осуществляется передача управления на прерванный низкоприоритетный обработчик без восстановления контекста.

31.6.5 Вход в процедуру обработки исключения

Вызов процедуры обработки исключения происходит в случае наличия необработанных исключительных ситуаций с достаточным приоритетом и при выполнении одного из следующих условий:

- процессор находится в режиме приложения (thread mode);
- новая исключительная ситуация имеет приоритет выше, чем обрабатываемая в текущий момент времени, что приводит к приоритетному прерыванию выполнения текущего обработчика. В этом случае возникает вложение одного исключения в другое.

При необходимости вызова обработчика, за исключением случаев обработки запоздавшего исключения и передачи управления на обработчик без восстановления контекста, процессор заносит в текущий стек восемь слов данных, называемые далее стековым фреймом (см. рисунок 131). Стековый фрейм включает в себя следующие значения:

- регистры R0-R3, R12;
- регистр LR;
- адрес возврата;
- регистр xPSR.

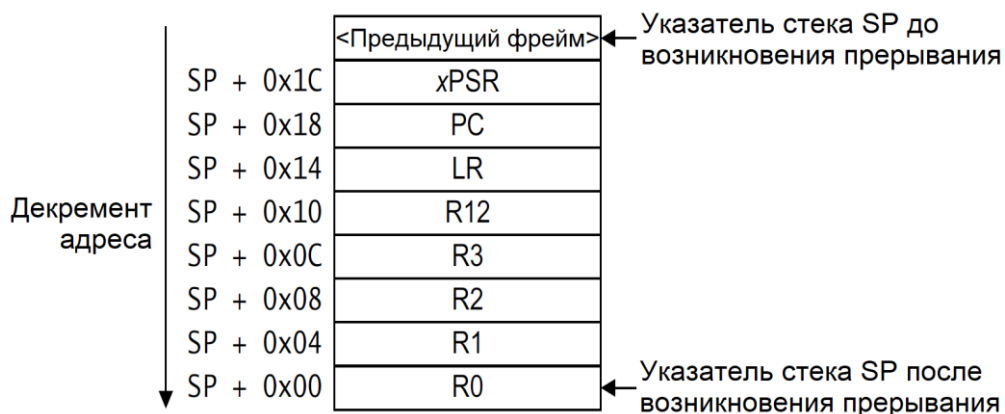


Рисунок 131 – Структура стекового фрейма

Указанная операция далее будет называться сохранением контекста. Непосредственно после ее выполнения указатель стека равен младшему адресу стекового фрейма.

В случае если бит STKALIGN в регистре управления конфигурацией (CCR) установлен в «1», во время сохранения контекста производится выравнивание адреса стека по границе двойного слова.

Стековый фрейм содержит адрес возврата, указывающий на ближайшую невыполненную инструкцию прерванной программы. По завершении процедуры обработки исключения значение адреса возврата заносится в счетчик команд, после чего выполнение программы возобновляется с прерванной точки.

Одновременно с сохранением контекста процессор осуществляет выборку адреса точки входа в процедуру обработки исключения из таблицы векторов исключений. По завершении операции сохранения контекста процессор передает управление на полученный из таблицы адрес.

Одновременно в регистр LR записывается специальное значение EXC_RETURN, позволяющее определить, какой из двух указателей стека соответствует данному стековому фрейму, и в каком режиме находился процессор перед входом в обработчик.

Если во время передачи управления не возникло исключения с более высоким приоритетом, процессор начинает выполнение вызванной процедуры обработки и автоматически изменяет состояние текущего прерывания с ожидающего обработки на активное.

В противном случае процессор передает управление обработчику высокоприоритетной исключительной ситуации без изменения состояния отложенного прерывания в соответствии с правилами, изложенными в пункте 31.6.4 «Запоздавшее исключение (late-arriving exception)».

31.6.6 Возврат из обработчика исключения

Возврат из обработчика исключения осуществляется в случае, если процессор находится в режиме обработчика (handler mode) и выполняет загрузку значения EXC_RETURN в регистр PC с помощью одной из следующих инструкций:

- инструкцию POP с аргументом PC;
- инструкцию BX с любым регистром;

- инструкции LDR или LDM с регистром PC в качестве приемника.

Значение EXC_RETURN загружается в регистр LR по входу в обработчик исключения. Механизм обработки исключений использует это значение для того, чтобы определить, завершил ли процессор выполнение процедуры обработки исключительной ситуации. Младшие четыре бита EXC_RETURN содержат информацию о состоянии стека и режиме работы процессора. Информация о назначении разрядов EXC_RETURN[3:0] и особенности процесса возврата из обработчика исключения представлены в таблице 434.

Процессор устанавливает биты EXC_RETURN [31:4] в 0xFFFFFFFF. Загрузка данного значения в PC указывает на завершение процедуры обработки исключения и заставляет процессор выполнить необходимые действия для возврата из обработчика. При выполнении возврата из обработчика исключения процессор производит восстановление контекста, который ранее был сохранён в стеке при входе в обработчик исключения.

Таблица 434 – Возврат из обработчика исключения

EXC_RETURN[3:0]	Описание
bXXX0	Зарезервирован
b0001	Возврат в режим обработчика. Восстановление контекста осуществляется из стека MSP. Дальнейшая работа осуществляется со стеком MSP
b0011	Зарезервирован
b01X1	Зарезервирован
b1001	Возврат в режим приложения. Восстановление контекста осуществляется из стека MSP. Дальнейшая работа осуществляется со стеком MSP
b1101	Возврат в режим приложения. Восстановление контекста осуществляется из стека PSP. Дальнейшая работа осуществляется со стеком PSP
b1X11	Зарезервирован

31.7 Обработка отказов

Отказы являются частным случаем исключений. Отказы могут возникать по следующим причинам:

- ошибка шины в ходе:
- чтения инструкции или вектора обработчика;
- доступа к данным;
- ошибка, обнаруженная процессором, например, неопределенная инструкция или попытка изменить состояние процессора с помощью команды BX;
- попытка выполнить инструкцию, расположенную в области памяти, помеченной как неисполняемая (Non-Executable – XN);
- отказ блока защиты памяти MPU вследствие нарушения прав доступа или вследствие попытки доступа к неподдерживаемой области адресного пространства.

31.7.1 Типы отказов

В таблице 435 представлены типы отказов, обработчики, вызываемые при их возникновении, соответствующие данному типу отказа регистры состояния, и биты регистра, указывающие на конкретный отказ.

Таблица 435 – Типы отказов

Тип отказа	Обработчик	Наименование бита регистра	Регистр отказа
Ошибка доступа к шине при чтении вектора	Тяжелый отказ	VECTTBL	«Регистр состояния тяжелых отказов»
Эскалация отказа		FORCED	

Тип отказа	Обработчик	Наименование бита регистра	Регистр отказа
Ошибка доступа к памяти:	Отказ доступа к памяти	-	«Регистр состояния отказов доступа к памяти», «Регистр адреса отказа доступа к памяти»
- при чтении команды		IACCVIOL	
- при доступе к данным		DACCVIOL	
- при сохранении контекста		MSTKERR	
- при восстановлении контекста		MUNSKERR	
Ошибка шины:	Отказ доступа к шине	-	«Регистр состояния отказа доступа к шине», «Регистр адреса отказа доступа к шине»
- при сохранении контекста		STKERR	
- при восстановлении контекста		UNSTKERR	
- при загрузке инструкции		IBUSERR	
локализованная ошибка шины данных		PRECISERR	
нелокализованная ошибка шины данных		IMPRECISERR	
Попытка доступа к сопроцессору	Отказ, вызванный ошибками программирования	NOCP	«Регистр состояния отказов, вызванных ошибками программирования»
Неизвестная инструкция		UNDEFINSTR	
Попытка выбора неверного набора инструкций*		INVSTATE	
Неверное значение EXC_RETURN		INVPC	
Запись или чтение по неверно выровненному адресу		UNALIGNED	
Деление на 0		DIVBYZERO	
* Попытка выбора набора инструкций, не поддерживаемого процессором			

31.7.2 Эскалация отказов и тяжелые отказы

Всем типам исключительных ситуаций по отказу, за исключением тяжелых отказов (hard fault) можно задать приоритет обработки, см. пункт 33.1.8 «SCB->SHP[x]». Выполнение данных обработчиков можно программно запретить, см. пункт 33.1.9 «SCB->SHCSR».

Как правило, приоритет обработки исключения, наряду со значениями регистров маскирования исключений, определяет, будет ли вызываться данный обработчик отказа, а также сможет ли он прервать выполнение другого обработчика.

В некоторых ситуациях отказ с конфигурируемым уровнем приоритета рассматривается системой как тяжелый. Такая ситуация именуется эскалацией отказа (escalation). Это возможно в следующих случаях:

- обработчик отказа во время своего выполнения вызвал отказ того же типа. Этот тип эскалации обусловлен тем фактом, что обработчик не может прервать собственное выполнение, так как его приоритет равен текущему;
- обработчик отказа вызвал отказ другого типа с приоритетом, меньшим или равным собственному. В этом случае новый обработчик также не может быть активизирован вследствие недостаточного уровня приоритета;
- обработчик исключительной ситуации вызвал отказ с приоритетом обработки, меньшим или равным текущему;
- возник отказ, обработчик которого не разрешен.

Если отказ обращения к шине возник во время загрузки данных в стек при передаче управления на обработчик отказа доступа к шине – эскалации не происходит. Таким образом, в случае, если отказ возник вследствие разрушения стека, передача управления на обработчик отказа выполняется, несмотря на то, что сохранение контекста не было осуществлено.

Обработка тяжелых отказов имеет фиксированный приоритет. Она может быть прервана только по сигналу сброса Reset или немаскируемого прерывания NMI. Сам обработчик способен прерывать обработку любых исключительных ситуаций, кроме ситуаций сброса Reset, NMI, а также другого тяжелого отказа.

31.7.3 Регистры состояния и адреса отказа

Регистры состояния отказа содержат информацию о причине отказа. Для обработки отказов шины и доступа к памяти предусмотрены регистры адреса отказа, содержащие адрес, по которому произошло обращение, вызвавшее отказ. Подробная информация приведена в таблице 436.

Таблица 436 – Регистры состояния и адреса отказа

Обработчик	Регистр состояния	Регистр адреса	Описание регистров
Тяжелый отказ	HFSR	-	Регистр состояния тяжелых отказов
Отказ доступа к памяти	MMFSR	MMFAR	Регистр состояния отказов доступа к памяти Регистр адреса отказа доступа к памяти
Отказ доступа к шине	BFSR	BFAR	Регистр состояния отказов доступа к шине Регистр адреса отказа доступа к шине
Отказ, вызванный ошибками программирования	UFSR	-	Регистр состояния отказов, вызванных ошибками программирования

31.7.4 Блокировка

Процессор переходит в состояние блокировки в случае, если тяжелый отказ возник во время выполнения программы-обработчика тяжелого отказа.

После перехода в состояние блокировки процессор перестает выполнять какие-либо команды. В этом состоянии он будет находиться до момента сброса.

31.8 Управление электропитанием

В процессоре Cortex-M3 предусмотрены следующие режимы ожидания (пониженного энергопотребления):

- Deep Sleep;
- Sleep;
- Standby.

Выбор процессором конкретного режима ожидания определяется значением бита SLEEPDEEP регистра SCR (см. пункт 33.1.6 «SCB->SCR»).

Далее в разделе описаны механизмы перехода в режим пониженного энергопотребления и условия выхода из этого режима.

31.8.1 Переход в режим пониженного энергопотребления

Система может формировать ложные сигналы событий, выводящие процессор из ожидания. Например, эти сигналы возникают при работе отладчика. Следовательно, программное обеспечение должно быть способным перевести процессор обратно в указанный режим ожидания. Для этого можно, например, организовать в программе пустой цикл.

31.8.2 Ожидание прерывания

Инструкция ожидания прерывания WFI (wait for interrupt) после своего выполнения немедленно переводит процессор в режим пониженного энергопотребления.

31.8.3 Ожидание события

Инструкция ожидания сигнала события WFE (wait for event) переводит или не переводит процессор в режим пониженного энергопотребления в зависимости от результата проверки одноразрядного регистра события. При этом процессор проверяет значение регистра события, и в случае, если он равен 0, приостанавливает дальнейшее выполнение команд и переходит в состояние ожидания. В случае если он равен 1, процессор записывает в регистр события 0 и продолжает нормальную работу без перехода в режим ожидания.

31.8.4 Переход в режим ожидания по выходу из обработчика исключения (режим Sleep)

В случае если бит SLEEPONEXIT регистра SCR установлен в 1, по завершении выполнения обработчика исключения процессор возвращается в режим приложения, после чего немедленно переходит в состояние пониженного энергопотребления.

Данный механизм рекомендуется использовать в задачах, в которых процессор используется только для обработки исключений.

31.8.5 Выход из состояния ожидания

Условия выхода процессора из режима ожидания зависят от причины, по которой он был переведен в этот режим.

31.8.5.1 Выход из ожидания по команде WFI и в режиме Sleep

Как правило, процессор выходит из режима ожидания только в случае возникновения исключительной ситуации с приоритетом, достаточным для активизации соответствующего обработчика.

В некоторых приложениях может возникнуть необходимость выполнения процедур восстановления системы после выхода процессора из режима пониженного энергопотребления, однако до того, как он начнет выполнять обслуживание прерываний. Для того чтобы добиться этого, достаточно установить бит PRIMASK в 1, а бит FAULTMASK – в 0. В случае возникновения в системе разрешенного прерывания с приоритетом выше текущего приоритета, процессор будет выведен из ожидания, однако не сможет передать управление обработчику прерывания до тех пор, пока бит PRIMASK не будет установлен в 0.

31.8.5.2 Выход из ожидания по команде WFE

Процессор выходит из режима ожидания в случае обнаружения исключительной ситуации с приоритетом, достаточным для активизации обработчика.

Кроме того, в случае установки бита SEVONPEND регистра SCR в 1, любое новое необслуженное прерывание формирует сигнал события и выводит процессор из ожидания, даже если это прерывание запрещено или имеет приоритет, недостаточно высокий для запуска обработчика.

Подробнее о регистре SCR см. пункт 33.1.6 «SCB->SCR».

31.8.6 Рекомендации по программированию режима энергопотребления

В стандарте ANSI языка C отсутствует возможность непосредственной генерации инструкций WFI и WFE. В CMSIS предусмотрены встроенные функции, предназначенные для включения в код этих инструкций:

```
void __WFE(void) // Wait for Event
void __WFI(void) // Wait for Interrupt
```

Периферийные блоки формируют прерывания с IRQ0 до IRQ31.

Таблица 437 – Формирование прерывания с IRQ0 до IRQ31

Прерывания	Блок	Принцип формирования
IRQ0	CAN1	Сигнал прерывания от блока CAN. Возникает при установленном бите GLB_INT_EN и при сигналах RX_INT_EN[31:0] и RX_INT[31:0] или EX_INT_EN[31:0] и EX_INT[31:0] или ERR_INT_EN и (ACK_ERR или FRAM_EERR или CRC_ERR или BS_ERR или BIT_ERR) или ERR_OVER_INT_EN и REC > CAN_ERR_MAX или TEC > CAN_ERR_MAX
IRQ1	CAN2	Аналогично IRQ0
IRQ2	USB	Прерывания от USB Host при наличии соответствующих флагов разрешения: HostSOFSent, или HostConnEvent, или HostResume, или HostTransDone. Прерывания от USB Slave при наличии соответствующих флагов разрешения SlaveNAKSent или SlaveSOFRXed или SlaveResetEvent или SlaveResume или SlaveTransDone
IRQ3...IRQ4	Зарезервировано	
IRQ5	DMA	Прерывания от DMA DMA_ERR или DMA_DONE. Обработка прерываний от DMA в соответствии с разделом Error signaling технического описания DMA
IRQ6	UART1	Сигнал UARTINTR
IRQ7	UART2	Сигнал UARTINTR
IRQ8	SSP1	Сигнал SSPINTR
IRQ9	Зарезервировано	
IRQ10	I2C	Сигнал INT при EN_INT
IRQ11	POWER	Сигнал прерывания от POWER Detecor

Прерывания	Блок	Принцип формирования
IRQ12	WWDG	Сигнал прерывания от WWDG
IRQ13	Зарезервировано	
IRQ14	Timer 1	Сигнал прерывания от Таймера. TIM_STATUS и TIM_IE
IRQ15	Timer 2	Аналогично IRQ14
IRQ16	Timer 3	Аналогично IRQ14
IRQ17	ADC	Сигналы прерываний от АЦП. EOCIF_1 или AWOIF_1 или EOCIF_2 или AWOIF_2
IRQ18	Зарезервировано	
IRQ19	COMP	Сигнал Rslt_Sy1
IRQ20	SSP2	Сигнал SSPINTR
IRQ21 ... IRQ26	Зарезервировано	
IRQ27	BACKUP	Прерывание от ВКР и часов реального времени
IRQ28	Внешнее прерывание 1	Сигнал EXT_INT1. Прерывание по уровню: 0 – нет прерывания; 1 – есть прерывание
IRQ29	Внешнее прерывание 2	Сигнал EXT_INT2. Прерывание по уровню: 0 – нет прерывания; 1 – есть прерывание
IRQ30	Внешнее прерывание 3	Сигнал EXT_INT3. Прерывание по уровню: 0 – нет прерывания; 1 – есть прерывание
IRQ31	Внешнее прерывание 4	Сигнал EXT_INT4. Прерывание по уровню: 0 – нет прерывания; 1 – есть прерывание
Примечание – Выводы в функции EXT_INT1-EXT_INT4 являются входами прерываний по уровню. Т.е. пока на вывод EXT_INTx подан сигнал высокого уровня, генерируется прерывание. Если на входе сигнал низкого уровня, то прерывание не генерируется.		

32 Контроллер прерываний NVIC

В разделе описан векторный контроллер прерываний с возможностью вложения (NVIC – Nested Vectored Interrupt Controller) и используемые им регистры.

Контроллер обеспечивает следующие возможности:

- программное задание уровня приоритета в диапазоне от 0 до 7 независимо каждому прерыванию. Более высокое значение уровня соответствует меньшему приоритету, таким образом, уровень 0 отвечает наивысшему приоритету прерывания;
- срабатывание сигнала прерывания по импульсу и по уровню;
- динамическое изменение приоритета прерываний;
- разделение исключений по группам с одинаковым приоритетом и по подгруппам внутри одной группы;
- передача управления из одного обработчика исключения в другой без восстановления контекста.

Процессор автоматически сохраняет в стеке свое состояние (контекст) по входу в обработчик прерывания и восстанавливает его по завершению обработчика, без необходимости непосредственного программирования этих операций. Это обеспечивает обработку исключительных ситуаций с малой задержкой.

Назначение регистров контроллера прерываний приведено в таблице 438.

Таблица 438 – Обобщенная информация о регистрах контроллера NVIC

Адрес	Название	Тип	Доступ	Значение после сброса	Описание
0xE000E100	NVIC				Контроллер прерываний NVIC
0x000	ISER[0]	RW	Привилегированный	0x00000000	Регистр разрешения прерываний ISER
...					
0x01C	ISER[7]				
...					
0x080	ICER[0]	RW	Привилегированный	0x00000000	Регистр запрета прерываний ICER
...					
0x09C	ICER[7]				
...					
0x100	ISPR[0]	RW	Привилегированный	0x00000000	Регистр установки состояния ожидания для прерывания ISPR
...					
0x11C	ISPR[7]				
...					
0x180	ICPR[0]	RW	Привилегированный	0x00000000	Регистр сброса состояния ожидания для прерывания ICPR
...					
0x19C	ICPR[7]				
...					
0x200	IABR[0]	RO	Привилегированный	0x00000000	Регистр активных прерываний IABR
...					
0x21C	IABR[7]				
...					

Адрес	Название	Тип	Доступ	Значение после сброса	Описание
0x300	IPR[0]	RW	Привилегированный	0x00000000	Регистр приоритета прерываний IP
0x304	IPR[1]				
0x308	IPR[2]				
0x30C	IPR[3]				
0x310	IPR[4]				
0x314	IPR[5]				
0x318	IPR[6]				
0x31C	IPR[7]				
...					
0xE00	STIR	WO	В зависимости от конфигурации*	0x00000000	Регистр программного формирования прерываний STIR
* Более подробную информацию см. в описании регистра					

32.1 Логика работы прерываний контроллера NVIC

В данном разделе описывается функционирование контроллера NVIC при поступлении на его вход запросов прерываний IRQ от различных модулей периферии микросхемы.

Первоначальным условием работы прерывания является его разрешение в модуле NVIC. За это отвечают регистры:

- ISER – за разрешение прерываний;
- ICER – за запрет прерываний.

В случае, когда соответствующий запрос разрешен (при данном условии рассмотрены все диаграммы в разделе), и приходит сигнал активации прерывания – запрос IRQ request, то возникает признак отложенного прерывания IRQ pending. Данный признак переводит прерывание в состояние ожидания его обработки ядром.

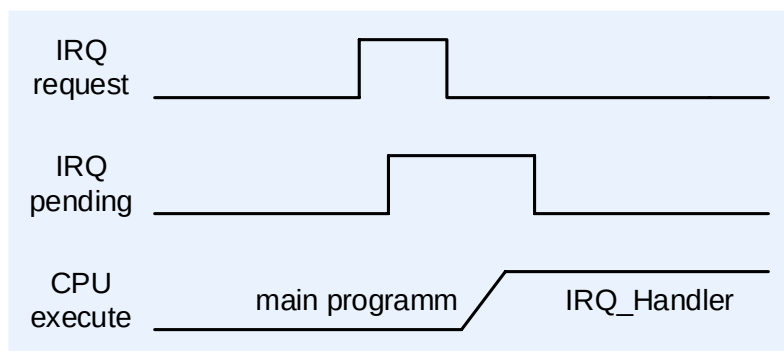


Рисунок 132 – Выставление отложенного запроса на прерывание и последующая его обработка

Pending биты выставляются в регистрах ISPR/ICPR, которые в свою очередь позволяют программно управлять признаком отложенного прерывания. ISPR – для установки pending бит, ICPR – для сброса соответственно. Если после прихода запроса на

прерывание IRQ request сбросить pending бит в регистре ICPR до того, как ядро приступит к его обработке, то прерывание будет проигнорировано – рисунок 133.

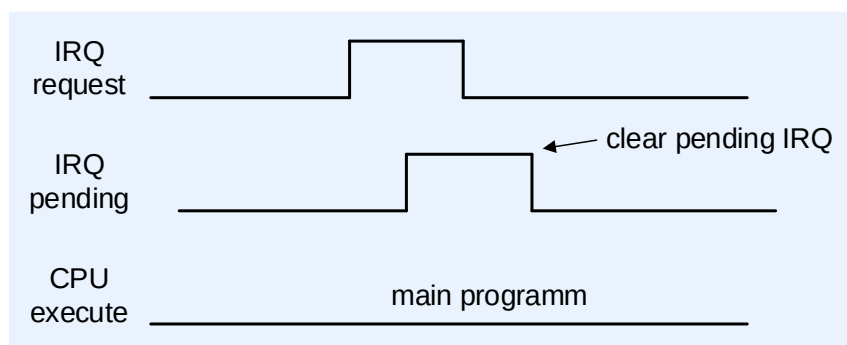


Рисунок 133 – Сброс признака отложенного прерывания, до обработки ядром

Если произойдет снятие запроса IRQ request от источника, «защелкивание» признака отложенного прерывания гарантирует отработку его ядром в соответствии с приоритетом – рисунок 134. Сам IRQ pending признак снимается автоматически, когда прерывание становится активным, о чем сигнализирует признак IRQ active. Информация об активности соответствующего прерывания содержится в регистре IABR[x].

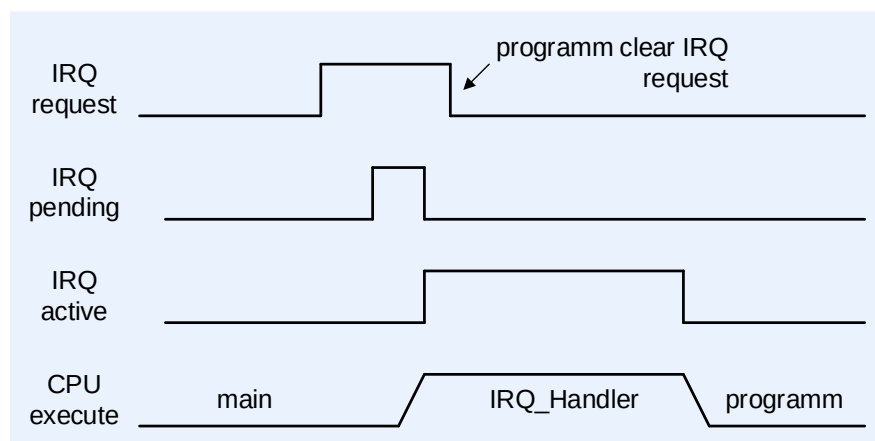


Рисунок 134 – Сброс признака отложенного прерывания, до обработки ядром

После того как прерывание стало активным, повторно запустить обработчик того же прерывания будет невозможно до тех пор, пока не будет завершена процедура обработки прерывания командой выхода из исключения. После выполнения команды выхода происходит сброс признака активности IRQ active.

При удержании источником на входе NVIC запроса на обработку IRQ request, по окончании обработки прерывания и снятия признака активного прерывания IRQ active, происходит повторное выставление признака отложенного прерывания IRQ pending – «защелкивание» pending бита, сброс которого в дальнейшем инициирует повторную активность и обработку того же исключения – рисунок 135.

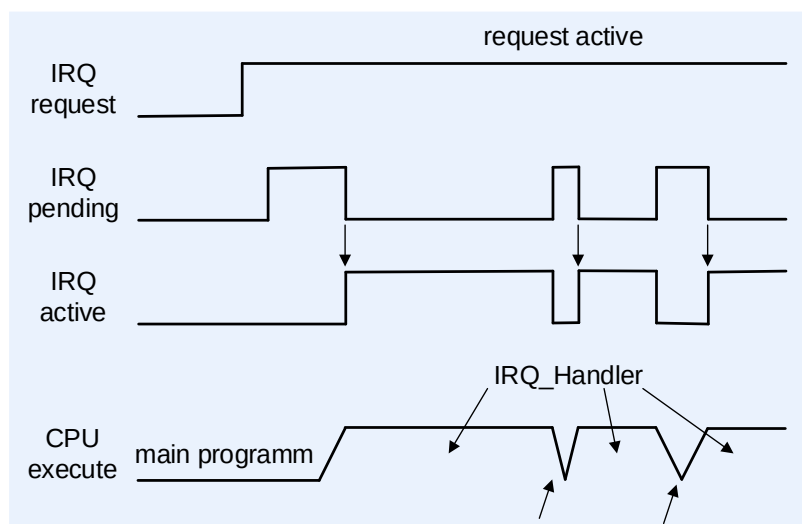


Рисунок 135 – Повторная обработка прерываний при удержании запроса от источника

Необходимо учитывать, что если источник прерываний выдает многократную установку и снятие запроса IRQ request на входе контроллера NVIC, то в таком случае только первый запрос выставляет признак отложенного прерывания IRQ pending, а остальные запросы до начала процедуры обработки прерывания (в момент активного признака отложенного прерывания) будут проигнорированы ядром – рисунок 136.

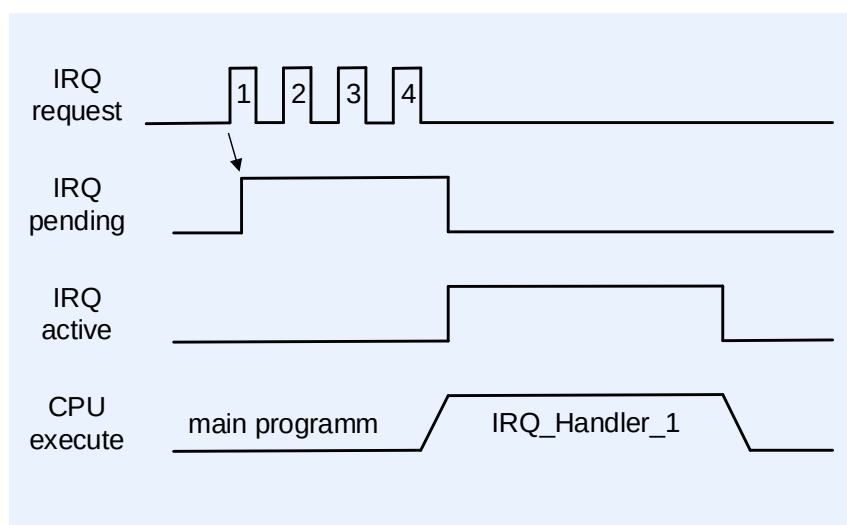


Рисунок 136 – Многократная установка снятие запроса IRQ request

Если запрос на прерывание пришел в момент активного прерывания, то в такой ситуации уже будут отработаны оба запроса на прерывание. В отличие от случая, изображенного на рисунке 136, запрос приходит тогда, когда признак отложенного прерывания IRQ pending уже сброшен, и новый запрос как раз его выставляет, что в дальнейшем позволяет провести повторную обработку прерывания – рисунок 137.

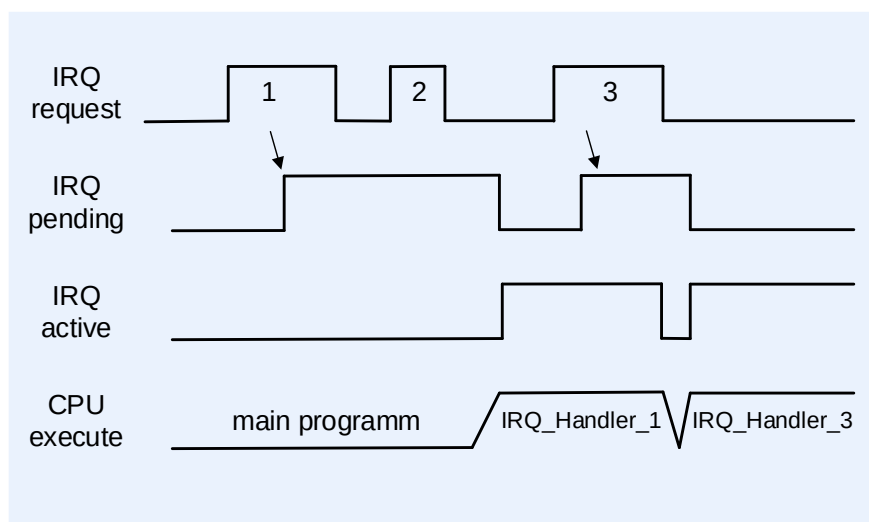


Рисунок 137 – Повторная установка запроса на прерывание в момент выполнения обработчика исключения

Выставление признака отложенного прерывания возможно даже в тех случаях, когда соответствующее прерывание запрещено. Все отложенные прерывания будут отражены в ISPR/ICPR, и в случае разрешения таких прерываний регистром ISPR, ядро тут же приступит к их обработке. Рекомендуется перед разрешением соответствующего прерывания убедиться в отсутствии признака отложенного запроса и при необходимости сбросить его.

32.2 Упрощенный доступ к регистрам контроллера прерываний

В целях повышения эффективности разработки программного обеспечения в CMSIS предусмотрен упрощенный доступ к регистрам контроллера прерываний NVIC из среды разработки программного обеспечения:

- регистры разрешения, запрета, установки и сброса состояния ожидания прерываний, а также регистр активных прерываний отображаются на массивы 32-разрядных целых чисел, а именно:

- массив ISER[0] соответствует регистру ISER0;
- массив ICER[0] соответствует регистру ICER0;
- массив ISPR[0] соответствует регистру ISPR0;
- массив ICPR[0] соответствует регистру ICPR0;
- массив IABR[0] соответствует регистру IABR0;
- трехбитные поля регистра приоритета прерываний отображаются на массив 3-разрядных целых чисел, а именно:

- массив IP[0]...IP[29] соответствует регистрам IPR0-IPR7, причем элемент массива IP[n] соответствует приоритету прерывания с номером n.

CMSIS генерирует код, гарантированно обеспечивающий в условиях многозадачности корректный непрерываемый (atomic) доступ к регистрам приоритета. Более подробная информация изложена в описании функции NVIC_SetPriority в подразделе 32.5 «Рекомендации по работе с контроллером прерываний».

В таблице 439 показано отображение прерываний (номеров запросов IRQ) в регистрах прерываний и соответствующие переменные CMSIS, для которых предусмотрено по одному биту на прерывание.

Таблица 439 – Распределение прерываний в переменных прерывания

Номер прерывания	Элементы массивов CMSIS [*]				
	Разрешение	Запрет	Установка режима ожидания	Сброс режима ожидания	Признак активности
0 – 31	ISER[0]	ICER[0]	ISPR[0]	ICPR[0]	IABR[0]
* Каждый элемент массива соответствует одному регистру контроллера прерываний NVIC, например, элемент ICER[1] соответствует регистру ICER1					

32.2.1 NVIC->ISER[x]

Регистр ISER0 предназначен для разрешения прерываний (запись) и определения, какие из прерываний разрешены (чтение).

Таблица 440 – Регистр разрешения прерываний

Номер	31...0
Доступ	R/W
Сброс	0
	SETENA bits

Назначение бит **SETENA**:

запись: 0 – не влияет,

1 – разрешение прерывания;

чтение: 0 – прерывание запрещено,

1 – прерывание разрешено.

При разрешении прерывания, находящегося в состоянии ожидания обработки, контроллер NVIC активизирует его в зависимости от приоритета. Запрос запрещенного прерывания переводит его в состояние ожидания обработки, однако контроллер NVIC не активизирует его вне зависимости от приоритета.

32.2.2 NVIC->ICER[x]

Регистр запрета прерываний

Регистр ICER0 предназначен для запрета прерываний (запись) и определения, какие из прерываний разрешены (чтение).

Таблица 441 – Регистр запрета прерываний

Номер	31... 0
Доступ	R/W
Сброс	0
	CLRENA

Назначение бит **CLRENA**:

запись: 0 – не влияет,

1 – запрет прерывания;

чтение: 0 – прерывание запрещено,

1 – прерывание разрешено.

32.2.3 NVIC->ISPR[x]

Регистр установки состояния ожидания для прерывания

Регистр ISPR0 предназначен для принудительного перевода прерываний в состояние ожидания обслуживания (запись) и определения, какие из прерываний находятся в этом состоянии (чтение).

Таблица 442 – Регистр установки состояния ожидания для прерывания

Номер	31...0
Доступ	R/W
Сброс	0
	SETPEND

Назначение бит **SETPEND**:

запись: 0 – не влияет,

1 – перевод прерывания в состояние ожидания;

чтение: 0 – прерывание не ожидает обслуживания,

1 – прерывание ожидает обслуживания.

Запись 1 в бит регистра ISPR, соответствующий:

- прерыванию, уже ожидающему обслуживания – не влияет на работу системы;
- запрещенному прерыванию – переводит его в состояние ожидания.

32.2.4 NVIC->ICPR[x]

Регистр сброса состояния ожидания для прерывания

Регистр ICPR0 предназначен для принудительного сброса состояния ожидания обслуживания прерывания (запись) и определения, какие из прерываний находятся в состоянии ожидания (чтение).

Таблица 443 – Регистр сброса состояния ожидания для прерывания

Номер	31...0
Доступ	R/W
Сброс	0
	CLRPEND

Назначение бит **CLRPEND**:

запись: 0 – не влияет,

1 – сброс состояния ожидания;

чтение: 0 – прерывание не ожидает обслуживания,

1 – прерывание ожидает обслуживания.

Запись 1 в разряд регистра ICPR, соответствующий прерыванию в активном состоянии, не влияет на работу системы.

32.2.5 NVIC->IABR[x]

Регистр активных прерываний

Регистр ICPR0 показывает, какие из прерываний находятся в активном состоянии. Этот регистр доступен только для чтения (Таблица 444).

Таблица 444 – Регистр активных прерываний

Номер	31...0
Доступ	RO
Сброс	0
	ACTIVE

Назначение бит **ACTIVE**:

чтение: 0 – прерывание не активно;

1 – прерывание активно и обслуживается, либо активно и ожидает обслуживания.

32.2.6 NVIC->IPR[x]

Регистры приоритета прерываний

Этот регистр используется для присвоения приоритета от нулевого до седьмого каждому из доступных прерываний. Ноль – наивысший приоритет, а семь – самый низший. Три бита приоритета хранятся в битах [7:5] каждого байта. Процессор обеспечивает доступ только к битам IP[x] [7:5] приоритета, биты [4:0] при чтении всегда равны нулю, а при записи игнорируются.

Адрес регистра: 0xE000E400 - 0xE000E41C

Доступ: Чтение/запись

Значение после сброса: 0x00000000

Таблица 445 – Назначения бит регистра

	31.. 29	28 24	23.. 21	20 16	15 13	12 8	7 .. 5	4 .. 0
E000E400	IP_3	Зарезерв.	IP_2	Зарезерв.	IP_1	Зарезерв.	IP_0	Зарезерв
E000E404	IP_7		IP_6		IP_5		IP_4	
E000E408	IP_11		IP_10		IP_9		IP_8	
E000E40C	IP_15		IP_14		IP_13		IP_12	
E000E410	IP_19		IP_18		IP_17		IP_16	
E000E414	IP_23		IP_22		IP_21		IP_20	
E000E418	IP_27		IP_26		IP_25		IP_24	
E000E41C	IP_31		IP_30		IP_29		IP_28	

Таблица 446 – Регистр приоритета прерываний IPR[x]

Биты	Поле	Функция
7...5	IP_n	Приоритет прерывания n

32.2.7 NVIC->STIR

Регистр программного формирования прерывания

Запись в регистр STIR приводит к формированию в системе программного прерывания (SGI – Software Generated Interrupt).

В случае если бит USERSETMPEND в регистре SCR установлен в «1», возможен доступ к регистру STIR из непривилегированных приложений (см. пункт 33.1.6 «SCB->SCR»). Установка этого бита возможна только из привилегированного режима работы процессора.

Таблица 447 – Регистр программного формирования прерывания

Номер	31...9	8...0
Доступ	U	R/W
Сброс	0	0
	-	INTID

INTID – идентификатор формируемого прерывания в диапазоне 0 – 239.

Например, значение b000000011 соответствует прерыванию IRQ3.

32.3 Прерывания, срабатывающие по уровню сигнала

Процессор способен обрабатывать прерывания, сформированные по уровню сигнала. Формирование запроса на прерывание по уровню происходит при условии удержания сигнала не менее двух тактов процессорного ядра.

Прерывание такого типа считается активным до тех пор, пока периферийное устройство не снимет активный уровень сигнала запроса. Как правило, это происходит после соответствующего обращения процедуры обработки прерывания к периферийному устройству.

После того, как процессор передал управление на обработчик, он автоматически снимает признак ожидания обслуживания прерывания (см. подраздел 32.4 «Аппаратное и программное управление прерываниями»). Если прерывание формируется по уровню сигнала, а сигнал запроса не снят до возврата из обработчика, процессор вновь переведет прерывание в состояние ожидания обслуживания, что, в свою очередь, приведет к повторному вызову его обработчика. Таким образом, периферийное устройство может поддерживать сигнал запроса прерывания в активном состоянии до тех пор, пока не перестанет нуждаться в обслуживании.

32.4 Аппаратное и программное управление прерываниями

Процессор Cortex-M3 регистрирует все поступающие прерывания. Перевод прерывания, сформированного периферийным устройством, в состояние ожидания обслуживания осуществляется в одном из следующих случаев:

- контроллер прерываний NVIC обнаруживает, что сигнал запроса имеет высокий логический уровень, а прерывание неактивно;
- контроллер прерываний NVIC обнаруживает передний фронт сигнала запроса прерывания;

программное обеспечение осуществляет запись в соответствующий разряд регистра ISPR0 (см. пункт 32.2.3 «NVIC->ISPR[x]») или соответствующего значения в регистр STIR (см. пункт 0 «Таблица 445 – Назначения бит регистра

	31.. 29	28 24	23.. 21	20 16	15 13	12 8	7 .. 5	4 .. 0
E000E400	IP_3	Зарезерв.	IP_2	Зарезерв.	IP_1	Зарезерв.	IP_0	Зарезерв.
E000E404	IP_7		IP_6		IP_5		IP_4	
E000E408	IP_11		IP_10		IP_9		IP_8	
E000E40C	IP_15		IP_14		IP_13		IP_12	
E000E410	IP_19		IP_18		IP_17		IP_16	
E000E414	IP_23		IP_22		IP_21		IP_20	
E000E418	IP_27		IP_26		IP_25		IP_24	
E000E41C	IP_31		IP_30		IP_29		IP_28	

Таблица 446 – Регистр приоритета прерываний IPR[x]

Биты	Поле	Функция
7...5	IP_n	Приоритет прерывания n

– NVIC->STIR»).

Прерывание находится в состоянии ожидания до тех пор, пока не произойдет одно из следующих событий:

– процессор передаст управление процедуре обработки прерывания. В этом случае прерывание переходит в активное состояние, после чего:

– по завершении обработки прерывания, срабатывающего по уровню, контроллер NVIC проверяет состояние сигнала запроса на прерывание. Если этот сигнал активен, прерывание вновь переводится в состояние ожидания обслуживания, что приводит к немедленной повторной передаче управления на обработчик. В противном случае прерывание переводится в неактивное состояние;

– если в период выполнения процедуры обработки прерывания, настроенного на срабатывание по фронту, не было зафиксировано импульсов на линии запроса, прерывание переводится в неактивное состояние.

– программное обеспечение осуществляет запись в соответствующий разряд регистра сброса состояния ожидания прерывания.

32.5 Рекомендации по работе с контроллером прерываний

Доступ к регистрам контроллера из программного обеспечения должен осуществляться по корректно выровненным адресам. Процессор не поддерживает возможность доступа к контроллеру по невыровненным адресам. Требования по выравниванию приведены в описании регистров.

Прерывание может быть переведено в состояние ожидания обслуживания даже в случае, если оно запрещено.

Перед установкой нового адреса таблицы векторов прерывания необходимо убедиться, что элементы новой таблицы корректно проинициализированы адресами обработчиков отказов и всех разрешенных исключений, в частности, прерываний. Более подробная информация представлена в пункте 33.1.4 «SCB->VTOR».

Программное разрешение или запрещение прерываний может осуществляться с помощью инструкций CPSIE I и CPSID I. В CMSIS предусмотрены следующие встроенные функции, генерирующие эти инструкции:

```
void __disable_irq(void) // Disable Interrupts
void __enable_irq(void) // Enable Interrupts
```

Кроме того, в CMSIS имеется ряд дополнительных функций, обеспечивающих управление контроллером прерываний NVIC (см. таблицу 448).

Таблица 448 – Функции CMSIS для управления контроллером прерываний

Функция	Описание
void NVIC_SetPriorityGrouping (uint32_t priority_grouping)	Установить группировку приоритетов
void NVIC_EnableIRQ (IRQn_t IRQn)	Разрешить IRQn
void NVIC_DisableIRQ (IRQn_t IRQn)	Запретить IRQn
uint32_t NVIC_GetPendingIRQ (IRQn_t IRQn)	Вернуть TRUE, если прерывание IRQn ожидает обслуживания, FALSE – в противном случае
void NVIC_SetPendingIRQ (IRQn_t IRQn)	Перевести IRQn в состояние ожидания обслуживания
void NVIC_ClearPendingIRQ (IRQn_t IRQn)	Сбросить состояние ожидания обслуживания для IRQn
uint32_t NVIC_GetActive (IRQn_t IRQn)	Вернуть номер IRQ текущего активного прерывания
void NVIC_SetPriority (IRQn_t IRQn, uint32_t priority)	Установить приоритет для IRQn
uint32_t NVIC_GetPriority (IRQn_t IRQn)	Считать приоритет IRQn
void NVIC_SystemReset (void)	Сбросить систему

Более подробную информацию см. в документации на CMSIS.

33 Блок управления системой

Блок управления системой SCB обеспечивает доступ к информации о конфигурации и управление работой системы. Регистры блока управления системой приведены в таблице 449.

Таблица 449 – Обобщенная информация о регистрах блока управления системой

Адрес	Имя	Тип	Доступ	Значение после сброса	Описание
0xE000E000	InterruptType				
0x008	ACTLR	RW	Привилегированный	0x00000000	Дополнительный регистр управления
0xE000ED00	SCB				Блок управления системой
0x000	CPUID	RO	Привилегированный	0x412FC230	Регистр идентификации процессора
0x004	ICSR	RW	Привилегированный	0x00000000	Регистр управления прерываниями
0x008	VTOR	RW	Привилегированный	0x00000000	Регистр смещения таблицы векторов прерываний
0x00C	AIRCR	RW	Привилегированный	0xFA050000	Регистр управления прерываниями и программного сброса
0x010	SCR	RW	Привилегированный	0x00000000	Регистр управления системой
0x014	CCR	RW	Привилегированный	0x00000200	Регистр конфигурации и управления
0x018	SHPR1	RW	Привилегированный	0x00000000	Регистр №1 приоритета системных обработчиков
0x01C	SHPR2	RW	Привилегированный	0x00000000	Регистр №2 приоритета системных обработчиков
0x020	SHPR3	RW	Привилегированный	0x00000000	Регистр №3 приоритета системных обработчиков
0x024	SHCRS	RW	Привилегированный	0x00000000	Регистр управления и состояния системных обработчиков
0x028	CFSR	RW	Привилегированный	0x00000000	Регистр состояния отказов с конфигурируемым приоритетом
0x028	MMSR	RW	Привилегированный	0x00	Регистр состояния отказов доступа к памяти
0x029	BFSR	RW	Привилегированный	0x00	Регистр состояния отказов доступа к шине
0x02A	UFSR	RW	Привилегированный	0x0000	Регистр состояния отказов, вызванных ошибками программирования
0x02C	HFSR	RW	Привилегированный	0x00000000	Регистр состояния тяжелого отказа
0x034	MMAR	RW	Привилегированный	Не определено	Регистр адреса отказа доступа к памяти
0x038	BFAR	RW	Привилегированный	Не определено	Регистр адреса отказа доступа к шине

33.1 Упрощенный доступ к регистрам блока управления системой

В целях повышения эффективности в CMSIS предусмотрен упрощенный доступ к регистрам SCB из среды разработки программного обеспечения, а именно, регистры SHPR1-SHPR3 в CMSIS отображаются на массив байтов SHP[0]...SHP[12].

33.1.1 InterruptType->ACTLR

Дополнительный регистр управления

Регистр ACTLR (таблица 450) позволяет разрешить или запретить следующие возможности процессора:

- вложение условных инструкций (IT folding);
- использование буферизации записи в режиме отображения памяти по умолчанию (default memory map);
- прерывание многоэлементных инструкций чтения и записи регистров.

Таблица 450 – Регистр ACTLR

Номер	31...3	2	1	0
Доступ	U	R/W	R/W	R/W
Сброс	0	0	0	0
	-	DISFOLD	DISDEFWBUFF	DISMCYCINT

DISFOLD – установка разряда в 1 запрещает вложение условных инструкций (IT folding) (см. ниже п. «О вложении условных инструкций»).

DISDEFWBUFF – установка в 1 запрещает использование буфера записи при работе в режиме отображения памяти по умолчанию (default memory map). Это обеспечивает возможность локализовать любые отказы шины, однако приводит к снижению производительности системы, так как все операции записи данных в память должны быть завершены до того, как процессор перейдет к выполнению следующей инструкции. Данный бит влияет исключительно на функционирование буферов записи, реализованных в процессоре Cortex-M3.

DISMCYCINT – установка бита в 1 запрещает прерывание многоэлементных инструкций чтения и записи регистров (LDM и STM). Это приводит к увеличению задержки обработки прерываний, вследствие необходимости завершения выполнения инструкций LDM или STM перед началом сохранения контекста и передачи управления обработчику прерывания.

О вложении условных инструкций

В некоторых случаях процессор может начать выполнение первой инструкции в IT-блоке, все еще выполняя инструкцию IT. Эта возможность, называемая далее вложением условных инструкций (IT folding), позволяет увеличить производительность системы, однако может привести к непостоянству времени выполнения тела цикла программы («джиттеру»). В случае если в разрабатываемом приложении это нежелательно, следует установить бит DISFOLD в 1.

33.1.2 SCB->CPUID

Регистр идентификации процессора

Регистр CPUID (таблица 451) содержит информацию о модели процессора, версии и варианте его реализации.

Таблица 451 – Регистр CPUID

Номер	31...24	23...20	19...16	15...4	3...0
Доступ	RO	RO	RO	RO	RO
Сброс	0x41	0x2	0xF	0xC23	0x0
	Implementer	Variant	Constant	PartNo	Revision

Implementer – код разработчика 0x41 = ARM.

Variant – значение r в номере версии rnpn изделия: 0x2 = r2p0;

Constant – постоянное значение 0xF;

PartNo – номер модели процессора: 0xC23 = Cortex-M3;

Revision – значение p в номере версии rnpn изделия: 0x0 = r2p0.

33.1.3 SCB->ICSR

Регистр управления прерываниями

Регистр ICSR обеспечивает возможность установки и сброса состояния ожидания обслуживания для исключений PendSV и SysTick, а также доступ к следующей информации:

- номер текущего обрабатываемого исключения;
- наличие активных исключений, обработка которых была прервана;
- номер исключения, ожидающего обслуживания, с наивысшим приоритетом;
- наличие прерываний, ожидающих обслуживания.

Таблица 452 – Регистр ICSR

Номер	31...29	28	27	26	25	24	23	22	21...12	11	10, 9	8...0
Доступ	U	R/W	R/W	R/W	R/W	U	R/W	R/W	R/W	R/W	U	R/W
Сброс	0	0	0	0	0	0	0	0	0	0	0	0
	-	PENDSVSET	PENDSVCLR	PENDSTSET	PENDSTCLR	-	Reserved for Debug	ISR_PENDING	VECT_PENDING	RETTOBASE	-	VECT_ACTIVE

PENDSVSET (RW) – бит установки состояния ожидания обслуживания для исключения PendSV.

Запись 0 – не влияет на работу системы,

1 – переводит исключение PendSV в состояние ожидания обслуживания.

Чтение 0 – исключение PendSV не ожидает обслуживания,

1 – ожидает.

Запись 1 – это единственно возможный способ перевода исключения PendSV в состояние ожидания обслуживания.

PENDSVCLR (WO) – бит сброса состояния ожидания обслуживания для исключения PendSV.

Запись 0 – не влияет на работу системы.

Запись 1 – сбрасывает состояние ожидания обслуживания для исключения PendSV.

PENDSTSET (RW) – бит установки состояния ожидания обслуживания для исключения SysTick.

Запись 0 – не влияет на работу системы.

Запись 1 – переводит исключение SysTick в состояние ожидания обслуживания.

Чтение 0 – исключение SysTick не ожидает обслуживания.

1 – ожидает.

PENDSTCLR (WO) – бит сброса состояния ожидания обслуживания для исключения SysTick.

Запись 0 – не влияет на работу системы.

Запись 1 – сбрасывает состояние ожидания обслуживания для исключения SysTick.

Данный бит доступен только для записи, при чтении результат не определен.

Reserved for Debug use (RO) – этот бит зарезервирован для целей отладки, при чтении вне режима отладки возвращает значение 0.

ISRPENDING (RO) – флаг наличия в системе прерываний (за исключением отказов), ожидающих обслуживания. 0 – ожидающие обслуживания прерывания отсутствуют, 1 – присутствуют.

VECTPENDING (RO) – содержит номер ожидающего обслуживания исключения с наивысшим приоритетом, обработка которого в системе разрешена. 0 – необслуженных исключений нет, другое число – номер ожидающего обслуживания исключения.

Значение данного поля формируется с учетом полей BASEPRI и FAULTMASK, однако не учитывает влияние поля PRIMASK.

RETTOBASE (RO) – показывает наличие в системе активных исключений, обслуживание которых было прервано. 0 – присутствуют, 1 – отсутствуют.

VECTACTIVE (RO) – содержит номер активного исключения. 0 – режим приложения, другое число – номер текущего обслуживаемого исключения. Для получения номера запроса прерывания (IRQ) из значения VECTACTIVE необходимо вычесть 16.

Запись в регистр ICSR может привести к непредсказуемым результатам в случае:

- одновременной установки в 1 бит PENDSVSET и PENDSVCLR;
- одновременной установки в 1 бит PENDSTSET и PENDSTCLR.

33.1.4 SCB->VTOR

Регистр смещения таблицы векторов прерываний

Регистр VTOR содержит смещение базового адреса таблицы векторов прерываний относительно адреса 0x00000000.

Таблица 453 – Регистр смещения таблицы векторов прерываний

Номер	31, 30	29...7	6...0
Доступ	U	R/W	R/W
Сброс	0	0	0
	-	TBLOFF	Reserved

TBLOFF – смещение базового адреса таблицы векторов относительно нижней границы карты распределения памяти. Собственно, смещение хранится в битах [28:7]. Бит [29] определяет, размещена ли таблица в области кода или в области памяти SRAM: 0 = область кода, 1 = SRAM. Бит [29] может также обозначаться как TBLBASE.

При установке значения TBLOFF требуется обеспечить выравнивание базового адреса таблицы векторов. Минимальный размер выравнивания – по границе блока из 32 слов, достаточен для хранения 16 векторов прерываний. Для поддержки большего количества прерываний необходимо увеличить размер выравнивания до ближайшей степени двойки, большей или равной размеру таблицы. Например, для хранения 21 вектора прерываний таблицу следует выровнять по границе блока из 64 слов, так как ее объем составляет 37 слов, а ближайшая степень двойки, большая или равная 37, равна 64.

Учитывая описанные выше требования по выравниванию, разряды [6...0] смещения всегда равны 0.

33.1.5 SCB->AIRCR

33.1.5.1 Регистр управления прерываниями и программного сброса

Регистр AIRCR позволяет группировать исключения по приоритетам, задавать порядок следования байтов в слове (endian) при доступе к данным, а также управлять процессом сброса системы.

Для записи данных в регистр необходимо установить его поле VECTKEY в значение 0x05FA, в противном случае попытка записи будет проигнорирована процессором.

Таблица 454 – Регистр AIRCR

Номер	31...16	15	14...11	10...8	7...3	2	1	0
Доступ	R/W	R/W	U	R/W	U	R/W	R/W	R/W
Сброс	0	0	0	0	0	0	0	0
	On Read: VECTKEYSTAT, On Write: VECTKEY	ENDIANESS	-	PRIGROUP	-	SYSRESETREQ	VECTCLRACTIVE	VECTRESET

VECTKEYSTAT – ключ доступа к регистру. При чтении возвращает 0xFA05.

VECTKEY – ключ доступа к регистру. При записи должен быть равен 0x05FA, в противном случае попытка записи в регистр будет проигнорирована процессором.

ENDIANESS (RO) – порядок следования значащих разрядов при доступе к данным. 0 – младший байт идет первым (little-endian), 1 – старший байт идет первым (big-endian). Значение поля устанавливается, исходя из уровня конфигурационного сигнала BIGEND в момент сброса системы.

PRIGROUP (RW) – группировка приоритетов исключений. Значение данного поля определяет положение двоичной точки, разделяющей поле приоритета на поля номера группы и подгруппы приоритетов.

PRIGROUP – определяет позицию двоичной точки, разделяющей поля PRI_n регистров приоритета прерываний на два подполя – номер группы и номер подгруппы. Зависимость этого разбиения от значения PRIGROUP приведена в таблице 455.

Таблица 455 – Группировка приоритетов прерываний

PRIGROUP	Значение приоритета в поле PRI_N[7:0]			Общее количество	
	Положение двоичной точки	Биты номера группы	Биты номера подгруппы	Групп	подгрупп
0b000, 0b001, 0b010, 0b011, 0b100	bxxx.00000	[7:5]	None	8	1
0b101	bxx.y00000	[7:6]	5	4	2
0b110	bx.yy00000	[7]	[6:5]	2	4
0b111	b.yyy00000	None	[7:5]	1	8

SYSRESETREQ (WO) – запрос сброса системы. 0 – не влияет на работу, 1 – инициирует сигнал сброса процессора. При чтении возвращает 0.

VECTCLRACTIVE (WO) – зарезервировано для целей отладки. При чтении возвращает 0. При записи данных в регистр значение поля должно быть равно 0, в противном случае результат непредсказуем.

VECTRESET (WO) – зарезервировано для целей отладки. При чтении возвращает 0. При записи данных в регистр значение поля должно быть равно 0, в противном случае результат непредсказуем.

33.1.6 SCB->SCR

Регистр управления системой

Регистр SCR позволяет определить требования к переходу в режим и выходу из режима пониженного энергопотребления.

Таблица 456 – Регистр SCR

Номер	31...5	4	3	2	1	0
Доступ	U	R/W	U	R/W	R/W	U
Сброс	0	0	0	0	0	0
	-	SEVONPEND	-	SLEEPDEEP	SLEEONEXIT	-

SEVONPEND – разрешает или запрещает формирование сигнала события при переводе исключения в состояние ожидания обработки. 0 – выход из режима пониженного энергопотребления по прерыванию могут инициировать только разрешенные прерывания или события; 1 – выход может инициироваться разрешенными событиями и любыми, в том числе запрещенными, прерываниями.

Перевод прерывания в состояние ожидания обслуживания формирует событие, что в свою очередь приводит к выходу процессора из режима пониженного потребления, инициированного инструкцией WFE, либо к регистрации факта события, если эта инструкция еще не выполнялась.

Кроме того, процессор может быть выведен из режима пониженного энергопотребления при поступлении внешнего события, а также после выполнения инструкции SEV.

SLEEPDEEP – определяет режим пониженного энергопотребления процессора:

0 – спящий режим (Sleep);

1 – режим глубокого сна (Deep Sleep).

SLEEONEXIT – разрешает или запрещает перевод процессора в режим пониженного энергопотребления при выходе из обработчика события в режим выполнения прикладной программы: 0 – не переводить, 1 – переводить.

33.1.7 SCB->CCR

Регистр конфигурации и управления

Регистр CCR управляет процессом перехода процессора в режим приложения, а также позволяет запретить или разрешить:

– игнорирование отказов доступа к шине в обработчиках тяжелых отказов и при эскалации отказа по FAULTMASK;

– генерацию исключений при делении на ноль и при доступе по невыровненному адресу;

доступ к регистру STIR из непривилегированного приложения (см. пункт 0 «Таблица 445 – Назначения бит регистра

	31.. 29	28 24	23.. 21	20 16	15 13	12 8	7 .. 5	4 .. 0
E000E400	IP_3	Зарезерв.	IP_2	Зарезерв.	IP_1	Зарезерв.	IP_0	Зарезерв
E000E404	IP_7		IP_6		IP_5		IP_4	
E000E408	IP_11		IP_10		IP_9		IP_8	
E000E40C	IP_15		IP_14		IP_13		IP_12	
E000E410	IP_19		IP_18		IP_17		IP_16	
E000E414	IP_23		IP_22		IP_21		IP_20	
E000E418	IP_27		IP_26		IP_25		IP_24	
E000E41C	IP_31		IP_30		IP_29		IP_28	

Таблица 446 – Регистр приоритета прерываний IPR[x]

Биты	Поле	Функция
7...5	IP_n	Приоритет прерывания n

– NVIC->STIR»).

Таблица 457 – Регистр CCR

Номер	31...10	9	8	7...5	4	3	2	1	0
Доступ	U	R/W	R/W	U	R/W	R/W	U	R/W	R/W
Сброс	0	0	0	0	0	0	0	0	0
	-	STKALIGN	BFHFNMIGN	-	DIV_O_TRP	UNALIGN_TRP	-	USERSETMPEND	NONBASETHRDENA

STKALIGN определяет режим выравнивания адреса стека при обработке исключений:

0 – выравнивание по границе 4 байт;

1 – выравнивание по границе 8 байт.

При передаче управления на обработчик исключения процессор анализирует бит [9] сохраненного в стеке слова состояния PSR и определяет по нему режим выравнивания стека. При возврате из обработчика процессор использует сохраненный в стеке бит этого слова для восстановления требуемого режима выравнивания.

BFHFNMIGN разрешает обработчикам с уровнем приоритета -1 и -2 игнорировать отказы доступа к шине, вызванные инструкциями чтения и записи. Бит влияет на функционирование обработчиков тяжелых отказов и при эскалации отказов по FAULTMASK:

0 – отказы доступа к шине данных, вызванные инструкциями чтения или записи, приводят к блокировке процессора;

1 – обработчики с уровнем приоритета -1 и -2 игнорируют указанные отказы доступа к шине данных.

Данный бит следует устанавливать лишь в том случае, если обработчик и используемые им данные размещены в абсолютно безопасной области памяти. Как правило, данный бит используется для локализации и исправления проблем доступа к системным устройствам и мостам ввода-вывода.

DIV_0_TRP разрешает процессору формировать отказ или останавливаться в случае деления на ноль при выполнении инструкций SDIV или UDI:

0 – не обрабатывать деление на 0;

1 – обрабатывать.

В случае если бит установлен в 0, при делении на ноль процессор устанавливает частное в 0.

UNALIGN_TRP разрешает процессору формировать отказ при невыровненном доступе к данным:

0 – не обрабатывать невыровненный доступ к словам или полусловам данных;

1 – обрабатывать.

Если бит равен 1, то невыровненный доступ приводит к отказу, вызванному ошибкой программирования (usage fault).

В случае невыровненного доступа по инструкциям LDM, STM, LDRD или STRD отказ формируется всегда, вне зависимости от значения бита UNALIGN_TRP.

USERSETMPEND разрешает доступ к регистру STIR (см. «Таблица 445 – Назначения бит регистра

	31.. 29	28 24	23.. 21	20 16	15 13	12 8	7 .. 5	4 .. 0
E000E400	IP_3	Зарезерв.	IP_2	Зарезерв.	IP_1	Зарезерв.	IP_0	Зарезерв.
E000E404	IP_7		IP_6		IP_5		IP_4	
E000E408	IP_11		IP_10		IP_9		IP_8	
E000E40C	IP_15		IP_14		IP_13		IP_12	
E000E410	IP_19		IP_18		IP_17		IP_16	
E000E414	IP_23		IP_22		IP_21		IP_20	
E000E418	IP_27		IP_26		IP_25		IP_24	
E000E41C	IP_31		IP_30		IP_29		IP_28	

Таблица 446 – Регистр приоритета прерываний IPR[x]

Биты	Поле	Функция
7...5	IP_n	Приоритет прерывания n

NVIC->STIR») из непривилегированного приложения:

0 – доступ запрещен;

1 – разрешен.

NONEBASETHRDENA определяет процедуру перехода процессора в режим приложения (Thread mode): 0 = процессор может перейти в режим приложения только в

случае отсутствия активных исключений, 1 = процессор может перейти в режим приложения из обработчика любого уровня, в соответствии со значением слова EXC_RETURN (см. пункт 31.6.6 «Возврат из обработчика исключения»).

33.1.8 SCB->SHP[x]

Регистры приоритета системных обработчиков

Регистры приоритета системных обработчиков SHPR1-SHPR3 позволяют установить уровень приоритета обработки исключений.

Доступ к регистрам осуществляется побайтно.

Поля PRI_N регистров имеют ширину 8 бит, однако в процессоре реализована поддержка доступа только к старшему полубайту [7...4], при чтении данных из младшего полубайта [3...0] процессор возвращает нули, запись в этот полубайт игнорируется.

Таблица 458 – Поля приоритета обработчиков системных отказов

Обработчик отказа	Поле	Описание регистра
Отказ доступа к памяти	SHP[4]	Регистр №1 приоритета системных обработчиков
Отказ доступа к шине	SHP[5]	
Ошибка программирования (usage fault)	SHP[6]	
Вызов SVCcall	SHP[11]	Регистр №2 приоритета системных обработчиков
Вызов PendSV	SHP[14]	Регистр №3 приоритета системных обработчиков
Вызов SysTick	SHP[15]	

Регистр №1 приоритета системных обработчиков

Таблица 459 – Регистр №1 приоритета системных обработчиков

Номер	31...24	23...16	15...8	7...0
Доступ	R/W	R/W	R/W	R/W
Сброс	0	0	0	0
	PRI_7: Резерв	PRI_6	PRI_5	PRI_4

PRI_7 Резерв.

PRI_6 Приоритет системного обработчика 6, ошибка программирования

PRI_5 Приоритет системного обработчика 5, отказ доступа к шине

PRI_4 Приоритет системного обработчика 4, отказ доступа к памяти

Регистр №2 приоритета системных обработчиков

Таблица 460 – Регистр №2 приоритета системных обработчиков

Номер	31...24	23...0
Доступ	R/W	U
Сброс	0	0
	PRI_11	-

PRI_11 Приоритет системного обработчика 11, вызов SVCcall

Регистр №3 приоритета системных обработчиков

Таблица 461 – Регистр №3 приоритета системных обработчиков

Номер	31...24	23...16	15... 0
Доступ	R/W	R/W	U
Сброс	0	0	0
	PRI_15	PRI_14	.

PRI_15 Приоритет системного обработчика 15, вызов SysTick

PRI_14 Приоритет системного обработчика 14, вызов PendSV

33.1.9 SCB->SHCSR

Регистр управления и состояния системных обработчиков.

Регистр SHCSR позволяет разрешить или запретить работу системных обработчиков, а также содержит сведения:

- о наличии ожидающих обработки отказов доступа к шине, управления памятью, а также вызов SVCall;
- об активных системных обработчиках.

Таблица 462 – Регистр управления и состояния системных обработчиков

Номер	31...19	18	17	16	15	14	13	12	11
Доступ	U	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Сброс	0	0	0	0	0	0	0	0	0
	-	USGF AULT ENA	BUSF AULT ENA	MEMF AULT ENA	SVCA LLPEN DED	BUSFA ULTPE NDED	MEMF AULTP ENDED	USGF AULTP ENDED	SYSTI CKACT

Номер	10	9	8	7	6...4	3	2	1	0
Доступ	R/W	U	R/W	R/W	U	R/W	U	R/W	R/W
Сброс	0	0	0	0	0	0	0	0	0
	PEND SVACT	-	MONI TORACT	CVCA LLAVCT	-	USGF AULT ACT	-	BUSF AULT ACT	MEMF AULT ACT

USGFAULTENA разрешение обработки отказов, вызванных ошибками программирования, 1 – разрешено, 0 – запрещено.

BUSFAULTENA разрешение обработки отказа доступа к шине, 1 – разрешено, 0 – запрещено.

MEMFAULTENA разрешение обработки отказа доступа к памяти, 1 – разрешено, 0 – запрещено.

SVCALLPENDED признак ожидания обработки вызова SVC, возвращает 1, если вызов ожидает обработки.

BUSFAULTPENDED признак ожидания обработки отказа доступа к шине, возвращает 1, если отказ ожидает обработки.

MEMFAULTPENDED признак ожидания обработки отказа доступа к памяти, возвращает 1, если отказ ожидает обработки.

USGFAULTPENDED признак ожидания обработки отказа, вызванного ошибками программирования, возвращает 1, если отказ ожидает обработки.

SYSTICKACT признак активности обработчика исключения SysTick, возвращает 1, если обработчик активен.

PENDSVACT признак активности обработчика исключения PendSV, возвращает 1, если обработчик активен.

MONITORACT признак активности монитора отладчика, возвращает 1, если монитор отладчика активен.

SVCALLACT признак активности обработчика вызова SVC, возвращает 1, если обработчик активен.

USGFAULTACT признак активности обработчика отказа, вызванного ошибкой программирования, возвращает 1, если обработчик активен.

BUSFAULTACT признак активности обработчика отказа доступа к шине, возвращает 1, если обработчик активен.

MEMFAULTACT признак активности обработчика отказа доступа к памяти, возвращает 1, если обработчик активен.

Примечания

1 Установка бита разрешения в 1 разрешает обработку исключения, установка в 0 – запрещает.

2 Чтение 1 из бита-признака активности свидетельствует об активности исключения, 0 – о его неактивности. Существует возможность записи значения в данный бит для принудительного перевода исключения в активное состояние, однако при этом следует предпринять меры предосторожности, описанные далее в разделе;

3 Чтение 1 из бита-признака ожидания свидетельствует о том, что исключение находится в состоянии ожидания обработки. Существует возможность принудительного перевода исключения в состояние ожидания путем записи 1 в данный бит.

Если в системе возникло исключение (отказ), обработчик которого запрещен, процессор формирует запрос на обработку тяжелого отказа.

Существует возможность принудительного перевода того или иного системного исключения в состояние ожидания обработки или активное состояние путем записи в соответствующий разряд регистра SHCSR.

Например, ядро операционной системы может осуществлять запись в биты – признаки активности для того, чтобы осуществить переключение контекста со сменой типа обрабатываемого исключения.

Программа, меняющая значение бит – признаков активности исключения, должна обеспечить необходимую корректировку содержимого стека, в противном случае процессор может сгенерировать отказ. Необходимо убедиться, что программа сохраняет и впоследствии корректно восстанавливает текущее значение признаков активности исключений.

После разрешения системных обработчиков все дальнейшие манипуляции с битами регистра необходимо производить, последовательно выполняя операции чтения, модификации и обратной записи, гарантирующие изменение только необходимых разрядов регистра.

33.1.10 SCB->CFSR

Регистр состояния отказов с конфигурируемым уровнем приоритета.

Регистр CFSR содержит информацию о причине возникновения отказов управления памятью, отказов доступа к шине и ошибок программирования (usage fault).

Таблица 463 – Регистр CFSR

Номер	31...16	15...8	7...0
Доступ	RO	RO	RO
Сброс	0	0	0
	Usage Fault Status Register: UFSR	Bus Fault Status Register: BFSR	Memory Management Fault Status Register: MMFSR

Регистр CFSR доступен побайтно. Возможны следующие варианты доступа к регистру CFSR и его отдельным элементам:

- слово по адресу 0xE000ED28 – полный регистр CFSR;
- байт по адресу 0xE000ED28 – регистр MMFSR;
- полуслово по адресу 0xE000ED28 – регистры MMFSR и BFSR;
- байт по адресу 0xE000ED29 – регистр BFSR;
- полуслово по адресу 0xE000ED2A – регистр UFSR.

33.1.10.1 Поле MMFSR

Регистр состояния отказов доступа к памяти

Регистр MMFSR содержит набор флагов, указывающих на различные причины отказа доступа к памяти.

Таблица 464 – Регистр MMFSR

Номер	7	6, 5	4	3	2	1	0
Доступ	RO	U	RO	RO	U	RO	RO
Сброс	0	0	0	0	0	0	0
	MMARVALID	-	MSTKERR	MUNSTKERR	-	DACCVIOL	IACCVIOL

MMARVALID признак корректности значения в регистре адреса отказа доступа к памяти (MMAR): 0 = значение в MMAR не содержит корректный адрес отказа, 1 = содержит.

В случае если произошла эскалация отказа доступа к памяти, обработчик тяжелого отказа должен установить этот бит в 0. В противном случае после возврата в обработчик отказа доступа к памяти возможна его некорректная работы, так как значение регистра MMAR будет изменено.

MSTKERR признак отказа на этапе сохранения в стеке контекста при передаче управления на обработчик исключения: 0 = отсутствует, 1 = попытка сохранения в стеке контекста при вызове обработчика исключения вызвала одно или несколько нарушений доступа к памяти. В случае, если бит равен 1, значение указателя стека SP по-прежнему корректно, однако содержимое стека может быть неверным. Адрес отказа в регистр MMAR не записывается.

MUNSTKERR признак отказа на этапе восстановления контекста из стека при выходе из обработчика исключения: 0 = отсутствует, 1 = попытка восстановления контекста из стека вызвала одно или несколько нарушений доступа к памяти.

Передача управления на обработчик данного отказа осуществляется без сохранения контекста. Таким образом, в случае, если данный бит равен 1, состояние стека сохраняется, значение указателя стека не меняется, контекст не сохраняется.

Адрес отказа в регистр MMAR не записывается.

DACCVIOL признак нарушения доступа к памяти данных: 0 = отсутствует, 1 = процессор попытался прочесть или записать данные в области, для которой не разрешен такой тип доступа. Если бит равен 1, значение счетчика команд PC, сохраненное в стеке,

указывает на инструкцию, вызвавшую отказ. В регистре MMAR содержится адрес, по которому была осуществлена попытка доступа к памяти.

IACCVIOL признак нарушения доступа к памяти команд: 0 = отсутствует, 1 = процессор попытался считать очередную команду из области памяти, для которой не разрешено выполнение. Этот отказ возникает всякий раз при доступе к области, помеченной как неразрешенная для выполнения (XN), даже в случае, если блок защиты памяти MPU не активен (disabled) или отсутствует. Если бит равен 1, значение счетчика команд PC, сохраненное в стеке, указывает на инструкцию, вызвавшую отказ. Адрес отказа в регистр MMAR не записывается.

33.1.10.2 Поле BFSR

Регистр состояния отказов доступа к шине

Регистр BFSR содержит набор флагов, указывающих на различные причины отказа доступа к шине:

Таблица 465 – Регистр BFSR

Номер	7	6, 5	4	3	2	1	0
Доступ	RO	U	RO	RO	RO	RO	RO
Сброс	0	0	0	0	0	0	0
	BFRVALID		STKERR	UNSTKERR	IMPRECISE	PRECISE	IBUSERR

BFRVALID признак корректности значения в регистре адреса отказа доступа к шине (BFAR): 0 = значение в BFAR не содержит корректный адрес отказа, 1 = содержит.

Процессор устанавливает этот бит в 1 в случае, если известен адрес, при доступе по которому произошел отказ. Возникновение впоследствии других отказов, например, отказов управления памятью, может сбросить этот бит в 0.

В случае если возникла эскалация отказа, обработчик тяжелого отказа должен установить этот бит в 0. В противном случае после возврата в обработчик отказа доступа к шине возможна его некорректная работа, так как значение регистра MMAR будет изменено.

STKERR признак отказа на этапе сохранения в стеке контекста при передаче управления на обработчик исключения: 0 = отсутствует, 1 = попытка сохранения в стеке контекста при вызове обработчика исключения вызвала одно или несколько нарушений доступа к шине. В случае если бит равен 1, значение указателя стека SP по-прежнему корректно, однако содержимое стека может быть неверным. Адрес отказа в регистр BFAR не записывается.

UNSTKERR признак отказа на этапе восстановления контекста из стека при выходе из обработчика исключения: 0 = отсутствует, 1 = попытка восстановления контекста из стека вызвала одно или несколько нарушений доступа к шине.

Передача управления на обработчик данного отказа осуществляется без сохранения контекста. Таким образом, в случае, если данный бит равен 1, состояние стека сохраняется, значение указателя стека не меняется, контекст не сохраняется.

Адрес отказа в регистр BFAR не записывается.

IMPRECISERR признак нелокализованной ошибки доступа к шине данных. 0 = отсутствует, 1 = произошла ошибка доступа к шине данных, однако адрес возврата в стековом фрейме не указывает на инструкцию, вызвавшую ошибку. В случае, если процессор установил этот бит в 1, адрес отказа в регистр BFAR не записывается. Данный отказ является асинхронным, таким образом, если он возник внутри процесса, приоритет которого выше, чем приоритет обработки отказа шины, процессор переводит его в состояние ожидания обслуживания до завершения более приоритетных процессов. В случае, если до передачи управления на обработчик возникла также локализованная ошибка доступа к шине, процессор устанавливает оба соответствующих флага.

PRECISERR признак локализованной ошибки доступа к шине данных. 0 = отсутствует, 1 = произошла ошибка доступа к шине данных, при этом адрес возврата в стековом фрейме указывает на инструкцию, вызвавшую ошибку. В случае, если процессор установил этот бит в 1, он также записывает адрес отказа в регистр BFAR.

IBUSERR признак ошибки доступа к шине инструкций. 0 = отсутствует, 1 = произошла ошибка доступа к шине инструкций. Процессор обнаруживает факт ошибки доступа к шине инструкций на этапе выборки очередной команды, однако признак IBUSERR устанавливается только после попытки выполнения этой инструкции. В случае, если процессор установил этот бит в 1, адрес отказа в регистр BFAR не записывается.

33.1.10.3 Поле UFSR

Регистр состояния отказов, вызванных ошибками программирования

Регистр UFSR содержит набор флагов, указывающих на различные причины отказа.

Таблица 466 – Регистр UFSR

Номер	15...10	9	8	7...4	3	2	1	0
Доступ	U	RO	RO	U	RO	RO	RO	RO
Сброс	0	0	0	0	0	0	0	0
	-	DIV BYZERO	UN ALIGNED	-	NOCPP	INV PC	INV STATE	UNDEF INSTR

DIVBYZERO признак деления на ноль: 0 = деления на ноль не было, либо обработка данного типа ошибки запрещена, 1 = процессор выполнил инструкцию SDIV или UDIV с делителем равным 0. Если бит равен 1, значение счетчика команд PC, сохраненное в стеке, указывает на инструкцию, вызвавшую отказ. Разрешить либо запретить обработку деления на ноль можно путем установки в 1 бита DIV_0_TRP регистра CCR (см. пункт «SCB->CCR»).

UNALIGNED признак доступа к памяти по невыровненному адресу: 0 = не было, либо обработка данного типа ошибки запрещена, 1 = процессор попытался обратиться к памяти по невыровненному адресу. Разрешить либо запретить обработку этой ошибки можно путем установки в 1 бита UNALIGN_TRP регистра CCR (см. пункт «SCB->CCR»).

Инструкции LDM, STM, LDRD, и STRD, пытающиеся обратиться по невыровненному адресу, вызывают исключение всегда, вне зависимости от значения бита UNALIGN_TRP.

NOCP попытка обращения к сопроцессору. Процессор не поддерживает инструкции, требующие наличия сопроцессора. 0 = не было, 1 = была.

INVPC загрузка неверного значения в счетчик команд PC. 0 = не было, 1 = процессор попытался загрузить в счетчик команд PC неверное значение EXC_RETURN, вследствие неправильного восстановления контекста, либо неверного значения EXC_RETURN. Если бит равен 1, значение счетчика команд PC, сохраненное в стеке, указывает на инструкцию, пытавшуюся загрузить неверное значение в PC.

INVSTATE неверное состояние: 0 = не было, 1 = процессор попытался выполнить инструкцию, связанную с неверным использованием регистра EPSR. Если бит равен 1, значение счетчика команд PC, сохраненное в стеке, указывает на инструкцию, попытавшуюся некорректно использовать регистр EPSR.

UNDEFINSTR попытка выполнения неверной инструкции. 0 = не было, 1 = процессор попытался выполнить неверной инструкцию. Если бит равен 1, значение счетчика команд PC, сохраненное в стеке, указывает на инструкцию, вызвавшую отказ. Под неверной понимается инструкция, которую процессор не смог декодировать.

После установки в 1 биты регистра UFSR сохраняют это значение до тех пор, пока не будут принудительно сброшены путем записи в них 1, либо до сброса системы.

33.1.11 SCB->HFSR

Регистр состояния тяжелого отказа

Регистр HFSR содержит сведения о причинах вызова обработчика тяжелого отказа. Особенностью данного регистра является то, что для сброса в 0 его разрядов необходимо записать в них значение 1.

Таблица 467 – Регистр HFSR

Номер	31	30	29...2	1	0
Доступ	R/W	R/W	U	R/W	R/W
Сброс	0	0	0	0	0
	DEBUGEVT	FORCED	-	VECTTBL	Reserved

DEBUGEVT бит зарезервирован для отладки. При записи в регистр данный бит должен быть равен 0, в противном случае поведение процессора непредсказуемо.

FORCED признак тяжелого отказа, возникшего вследствие эскалации отказа с конфигурируемым уровнем приоритета, который не может быть обработан (запрещен или имеет недостаточно высокий приоритет): 0 = нет, 1 = да.

Если этот бит равен 1, то для определения причины отказа обработчику следует прочитать значения остальных разрядов регистров HFSR.

VECTTBL признак возникновения отказа шины при попытке доступа к таблице векторов исключений: 0 = не было, 1 = было. Эта ошибка всегда вызывает передачу управления на обработчик тяжелого отказа. Если бит равен 1, значение счетчика команд PC, сохраненное в стеке, указывает на инструкцию, выполнение которой было прервано для обработки исключения.

После установки в 1 биты регистра HFSR сохраняют это значение до тех пор, пока не будут принудительно сброшены путем записи в них 1, либо до сброса системы.

33.1.12 SCB->MMFAR

Регистр адреса отказа доступа к памяти

Регистр MMFAR содержит адрес, при обращении по которому возникла ошибка управления памятью.

Таблица 468 – Регистр MMFAR

Номер	31...0
Доступ	RO
Сброс	0
	ADDRESS

ADDRESS если бит MMARVALID регистра MMFSR равен 1, это поле содержит адрес, при обращении по которому возникла ошибка управления памятью. В случае ошибки доступа по невыровненному адресу поле содержит фактическое значение адреса, вызвавшего отказ.

Учитывая, что одна единственная операция чтения или записи может быть разбита процессором на несколько операций доступа по выровненному адресу, в регистре адреса отказа может находиться любое значение в диапазоне адресов, по которым осуществлялась попытка доступа.

Флаги регистра MMFSR содержат информацию о причине отказа, а также сообщают, является ли значение MMFAR корректным. Подробнее см. пункт 31.7.3 «Регистры состояния и адреса отказа».

33.1.13 SCB->BFAR

Регистр адреса отказа доступа к шине

Регистр BFAR содержит адрес, при обращении по которому возникла ошибка доступа к шине.

Таблица 469 – Регистр BFAR

Номер	31...0
Доступ	RO
Сброс	0
	ADDRESS

ADDRESS если бит BFARVALID регистра BFSR равен 1, это поле содержит адрес, при обращении по которому возникла ошибка доступа к шине. В случае ошибки доступа по невыровненному адресу поле содержит значение адреса, запрошенного командой процессора, даже если оно не совпадает с адресом, вызвавшим отказ.

Флаги регистра BFSR содержат информацию о причине отказа, а также сообщают, является ли значение BFAR корректным. Подробнее см. пункт 31.7.3 «Регистры состояния и адреса отказа».

33.1.14 Рекомендации по программированию блока управления системой

Необходимо убедиться, что программа использует для обращения к регистрам блока управления системой доступ по корректно выровненным адресам. Обращение ко всем регистрам, за исключением CFSR и SHPR1-SHPR3, должно быть выровнено по границе слова. Регистры CFSR и SHPR1-SHPR3 допускают как побайтный доступ, так и доступ по адресам, выровненным по границе слова или полуслова.

Для того чтобы определить истинный адрес, вызвавший отказ, в обработчике необходимо выполнить следующие действия:

- считать и сохранить значения регистров MMFAR или BFAR;
- проверить значение бита MMARVALID регистра MMFSR, либо бита BFARVALID регистра BFSR. Значения MMFAR или BFAR корректны только в случае, если соответствующие биты равны 1.

Рекомендуется именно такая последовательность операций, так как возникновение исключения с более высоким приоритетом может изменить значения в регистрах MMFAR и BFAR, например, в случае возникновения сбоя в обработчике более высокоприоритетного исключения.

34 Сторожевые таймеры

34.1 Блок сторожевого таймера MDR_IWDG

IWDG – независимый 12-разрядный сторожевой таймер, который считает вниз от основания счета (значения перезагрузки, настраиваемого начального значения) до нуля. При достижении счетчиком нуля сторожевой таймер формирует сигнал сброса, который перезагружает систему. Для предотвращения формирования сброса от IWDG необходимо заблаговременно перезагрузить таймер записью ключевого значения 0хАААА в регистр MDR_IWDG->KR – после чего таймер снова начнет обратный отсчет от установленного значения. Тем самым и обеспечивается контроль над зависанием программы в определённых точках, где такое возможно (не перезагрузив IWDG вовремя, система сбросится). Сброс системы от независимого сторожевого таймера сбрасывает IWDG. Любой другой системный сброс (например, внешний сброс) не влияет на настройки и работу таймера IWDG.

Блок-схема независимого сторожевого таймера приведена на рисунке 138.

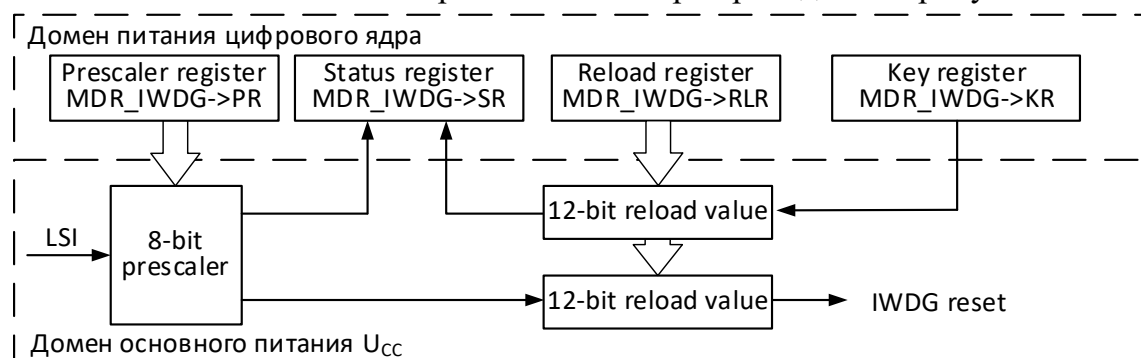


Рисунок 138 – Блок-схема независимого сторожевого таймера

Структурно IWDG состоит из двух частей. Часть с регистрами управления расположена в домене питания цифрового ядра, доступна по APB, тактируется частотой PCLK и сбрасывается по всем типам сброса. Часть непосредственно с независимым сторожевым таймером расположена в домене основного питания микросхемы UCC и тактируется от генератора LSI – поэтому для фактической записи в регистры IWDG в домене основного питания UCC генератор LSI должен находиться в рабочем режиме. Часть сторожевого таймера в домене основного питания UCC сбрасывается только от сброса, формируемого IWDG – после сброса от IWDG сторожевой таймер останавливает свою работу.

Период счета сторожевого таймера от разрешения работы до формирования сброса является функцией от основания счета (значения перезагрузки) и поделенной на значение предделителя частоты LSI.

Обновление значений предделителя IWDG в домене основного питания осуществляется записью регистра MDR_IWDG->PR соответственно. Обновление основания счета (значение, от которого таймер при старте начинает обратный отсчёт) IWDG в домене основного питания осуществляется значением регистра MDR_IWDG->RLR записью ключевого значения 0хАААА в регистр ключа. Для возможности записи в регистры MDR_IWDG->PR и MDR_IWDG->RLR предварительно

в регистр ключа должно быть записано ключевое значение 0x5555 для разрешения доступа по записи в них. При записи в регистр MDR_IWDG->PR значения предделителя и ключа 0xAAAA в регистр MDR_IWDG->KR формируются запросы на обновление соответствующих параметров (предделителя и основания счета), которые передаются IWDG в домене основного питания. Запросы на обновления не захватываются (не сохраняются) частью в домене основного питания, то есть при любом типе сброса запросы на обновление от IWDG в домене питания ядра сбрасываются. Наличие запроса обновления от IWDG в домене питания ядра и/или информация о процессе фактического обновления на стороне IWDG в домене основного питания транслируются флагами PVU (для предделителя) и RVU (для основания счета). Фактическое обновление значений (со сбросом соответствующих флагов после обновления) осуществляется только при разрешенной работе IWDG и наличии частоты LSI. При фактическом обновлении предделителя и основания счета недоступна запись в регистры MDR_IWDG->PR и MDR_IWDG->RLR соответственно. После фактического обновления снимаются запросы обновления со стороны IWDG в домене питания ядра UCC.

Пример работы с блоком IWDG:

- 1 Включение генератора LSI и ожидание его выхода в рабочий режим.
- 2 Разрешение доступа к регистрам MDR_IWDG->PR и MDR_IWDG->RLR записью значения 0x5555 в регистр IWDG_KR.
- 3 Запись значения предделителя в регистр MDR_IWDG->PR.
- 4 Разрешение работы IWDG записью значения 0xCCCC в регистр MDR_IWDG->KR.
- 5 Ожидание обновления значения предделителя (сброса флага PVU в 0).
- 6 Разрешение доступа к регистрам IWDG_PR и IWDG_RLR записью значения 0x5555 в регистр IWDG_KR.
- 7 Проверка флага RVU = 0 перед записью в MDR_IWDG->RLR.
- 8 Запись значения перезагрузки в регистр MDR_IWDG->RLR.
- 9 Периодическая перезагрузка сторожевого таймера в процессе выполнения программы значением регистра MDR_IWDG->RLR с помощью записи ключевого значения 0xAAAA в регистр MDR_IWDG->KR.

34.1.1 Описание регистров блока сторожевых таймеров IWDG

Таблица 470 – Описание регистров блока сторожевого таймера IWDG

Базовый Адрес	Название	Описание
0x4006_8000	MDR_IWDG	Сторожевой таймер IWDG
Смещение		
0x00	MDR_IWDG->KR[15:0]	Регистр ключа
0x04	MDR_IWDG->PR[2:0]	Делитель частоты сторожевого таймера
0x08	MDR_IWDG->RLR[11:0]	Регистр основания счета сторожевого таймера
0x0C	MDR_IWDG->SR[1:0]	Регистр статуса сторожевого таймера

34.1.1.1 MDR_IWDG->KR

Таблица 471 – Регистр KR

Номер	31...16	15...0
Доступ	U	W
Сброс	0	0
	-	KEY[15:0]

Таблица 472 – Описание бит регистра KR

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...16	-	Зарезервировано
15...0	KEY[15:0]	Значение ключа (только запись, читается 0x0000). При записи: – 0xAAAA – перезагрузка значения таймера значением регистра IWDG_RLR. Значение должно периодически программно записываться при разрешенной работе IWDG, в противном случае сторожевой таймер генерирует сброс, если таймер достиг значения нуля. – 0x5555 – разрешение доступа по записи к регистрам MDR_IWDG->PR и MDR_IWDG->RLR. Если после разрешения доступа в регистр ключа записывается другое ключевое значение, доступ к регистрам MDR_IWDG->PR и MDR_IWDG->RLR запрещается. – 0xCCCC – разрешение работы сторожевого таймера (если работа сторожевого таймера не разрешена; работа таймера останавливается аппаратно сбросом от IWDG)

34.1.1.2 MDR_IWDG->PR

Таблица 473 – Регистр PR

Номер	31...3	2...0
Доступ	U	R/W
Сброс	0	0
	-	PR[2:0]

Таблица 474 – Описание регистра PR

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...3	-	Зарезервировано
2...0	PR[2:0]	Делитель частоты сторожевого таймера: 000 – делитель на 4: LSI/4; 001 – делитель на 8: LSI/8; 010 – делитель на 16: LSI/16; 011 – делитель на 32: LSI/32; 100 – делитель на 64: LSI/64;

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
		101 – делитель на 128: LSI/128; 110 – делитель на 256: LSI/256; 111 – делитель на 256: LSI/256. Для возможности записи в регистр должен быть разрешен доступ по записи с помощью ключевого значения 0x5555 в регистре MDR_IWDG->KR. При записи в регистр формируется запрос на обновление значения делителя частоты в часть IWDG в домене основного питания (см. рисунок 138). Запись в регистр MDR_IWDG->PR недоступна при фактической записи предделителя в часть IWDG в домене основного питания U_{CC} (см. рисунок 138). При чтении возвращает записанное в регистр значение, а не значение предделителя IWDG в домене основного питания

34.1.1.3 MDR_IWDG->RLR

Таблица 475 – Регистр RLR

Номер	31...12	11...0
Доступ	U	R/W
Сброс	0	0xFFFF
	-	RLR[11:0]

Таблица 476 – Описание регистра RLR

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...12		Зарезервировано
11...0	RLR[11:0]	Значение перезагрузки (основания счета) сторожевого таймера. Сторожевой таймер декрементируется, начиная с этого значения. Определяет значение, загружаемое в сторожевой таймер при записи значения 0xAAAA в регистр MDR_IWDG->KR. Для возможности записи в регистр должен быть разрешен доступ по записи с помощью ключевого значения 0x5555 в регистре MDR_IWDG->KR. Запись в регистр MDR_IWDG->RLR недоступна при фактической записи основания счета в часть IWDG в домене основного питания U_{CC} (см. рисунок 138). При чтении возвращает записанное в регистр значение, а не значение перезагрузки IWDG в домене основного питания

34.1.1.4 MDR_IWDG->SR

Таблица 477 – Регистр SR

Номер	31...2	1	0
Доступ	U	RO	RO
Сброс	0	0	0
	-	RVU	PVU

Таблица 478 – Описание регистра SR

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...2		Зарезервировано
1	RVU	Флаг обновления значения сторожевого таймера. Устанавливается аппаратно и служит признаком того, что обновляется значение сторожевого таймера IWDG в домене основного питания U_{CC} из регистра перезагрузки MDR_IWDG->RLR (см. рисунок 138). Работа IWDG разрешена и LSI работает: Отображает запрос от регистров управления на обновление значения сторожевого таймера значением регистра MDR_IWDG->RLR и/или фактическое обновление значения сторожевого таймера в домене основного питания U_{CC}. Сбрасывается при завершении фактического обновления значения перезагрузки. Значение регистра перезагрузки MDR_IWDG->RLR может быть обновлено только, если этот бит равен нулю. Работа IWDG не разрешена или LSI не работает: Отображает запрос от регистров управления на обновление значения сторожевого таймера в домене основного питания U_{CC} значением регистра MDR_IWDG->RLR. Сбрасывается любым типом сброса. Значение регистра перезагрузки MDR_IWDG->RLR может быть обновлено независимо от состояния этого бита – после запуска IWDG будет обновлен последним записанным в MDR_IWDG->RLR значением
0	PVU	Флаг обновления делителя частоты сторожевого таймера. Устанавливается аппаратно и служит признаком того, что обновляется значение делителя частоты IWDG в домене основного питания U_{CC} из регистра предделителя MDR_IWDG->PR (см. рисунок 138). Работа IWDG разрешена и LSI работает: Отображает запрос от регистров управления на обновление делителя частоты сторожевого таймера значением регистра MDR_IWDG->PR и/или фактическое обновление значения делителя в домене основного питания U_{CC}. Сбрасывается при завершении фактического обновления делителя. Значение регистра предделителя MDR_IWDG->PR может быть обновлено только, если этот бит равен нулю. Работа IWDG не разрешена или LSI не работает: Отображает запрос от регистров управления на обновление значения сторожевого таймера в домене основного питания U_{CC}

		значением регистра MDR_IWDG->PR. Сбрасывается любым типом сброса. Значение регистра перезагрузки MDR_IWDG->PR может быть обновлено независимо от состояния этого бита – после запуска IWDG будет обновлен последним записанным в MDR_IWDG->PR значением
--	--	---

34.2 Блок сторожевого таймера MDR_WWDG

34.2.1 Описание регистров блока сторожевых таймеров WWDG

Таблица 479 – Описание регистров блока сторожевого таймера WWDG

Базовый Адрес	Название	Описание
0x4006_0000	MDR_WWDG	Сторожевой таймер WWDG
Смещение		
0x00	MDR_WWDG->CR[7:0]	Регистр управления
0x04	MDR_WWDG->CFR[9:0]	Регистр конфигурации
0x08	MDR_WWDG->SR[0]	Регистр статуса

34.2.1.1 MDR_WWDG->CR

Таблица 480 – Регистр CR

Номер	31...8	7	6	5	4	3	2	1	0
Доступ	U	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Сброс		0	1	1	1	1	1	1	1
	-	WDGA	T6	T5	T4	T3	T2	T1	T0

Таблица 481 – Описание бит регистра CR

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...8		Зарезервировано
7	WDGA	Бит активации Этот бит устанавливается программно и очищается только аппаратно при сбросе. Когда WDGA=1, сторожевой таймер может генерировать сброс: 0 – сторожевой таймер отключен; 1 – сторожевой таймер включен
6...0	T[6:0]	Значение семиразрядного счетчика (от старших разрядов к младшим) Эти биты содержат значение сторожевого таймера, который декрементируется каждые 4096×2^{WDGTB} циклов частоты PCLK периферийной шины APB

34.2.1.2 MDR_WWDG->CFR

Таблица 482 – Регистр CFR

Номер	31...10	9	8	7	6	5	4	3	2	1	0
Доступ	U	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Сброс		0	0	0	1	1	1	1	1	1	1
	-	EWI	WDGTB1	WDGTB0	W6	W5	W4	W3	W2	W1	W0

Таблица 483 – Описание бит регистра CFR

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...10		Зарезервировано
9	EWI	Раннее предупреждающее прерывание Если бит установлен, то разрешается генерация прерывания при достижении сторожевым таймером значения 40h. Прерывание запрещается только аппаратным сбросом
8...7	WGTV[1:0]	Делитель частоты сторожевого таймера 00 – частота таймера (PCLK / 4096) /1; 01 – частота таймера (PCLK / 4096) /2; 10 – частота таймера (PCLK / 4096) /4; 11 – частота таймера (PCLK / 4096) /8
6...0	W[6:0]	Значение окна Эти биты содержат значение окна, в пределах которого возможна инициализация бит T[6:0] значением в пределах 40h-7Fh. Если происходит инициализация бит в момент T>W, то формируется сброс на выходе RESET. Если таймер достигнет значения T=3Fh, то также формируется сброс

34.2.1.3 MDR_WWDG->SR

Таблица 484 – Регистр SR

Номер	31...1	0
Доступ	U	R/W
Сброс	0	0
	-	EWIF

Таблица 485 – Описание бит регистра SR

Номер бита	Функциональное имя бита	Расшифровка функционального имени бита, краткое описание назначения и принимаемых значений
31...1		Зарезервировано
0	EWIF	Флаг раннего предупреждающего прерывания Этот бит устанавливается аппаратно, когда сторожевой таймер достигает значения 40h. Бит очищается программно записью нуля. Запись единицы не влияет. Этот бит также устанавливается, если прерывание запрещено EWI=0

35 Электрические параметры микросхемы

Таблица 486 – Электрические параметры микросхемы

Наименование параметра, единица измерения, режим измерения	Буквенное обозначение параметра	Норма параметра		Температура среды, °C
		не менее	не более	
Выходное напряжение низкого уровня, В, на выводах: PA, PB, PC, PD, PE, PF, STANDBY, DN, DP	U _{OL}	–	0,4	25, 85, – 40
Выходное напряжение высокого уровня, В, на выводах: PA, PB, PC, PD, PE, PF, STANDBY, DN, DP при: U _{CC} = 3,0 В	U _{OH}	2,4	–	
Выходное напряжение высокого уровня, В, на выводах: PA, PB, PC, PD, PE, PF, STANDBY при: U _{CC} = 2,2 В		1,6	–	
Уровень напряжения питания для срабатывания схемы генерации сброса, В	U _{POR}	1,8	2,18	
Входной ток утечки высокого уровня, мкА, на выводах: PA, PB, PC, PD, PE, PF, RESET, WAKEUP, DN, DP	I _{ILH}	– 1,0	1,0	
Входной ток утечки высокого уровня, мкА, на выводе: OSC_IN при: BYPASS = 1		– 40,0	40,0	
Входной ток утечки низкого уровня, мкА, на выводах: PA, PB, PC, PD, PE, PF, RESET, WAKEUP, DN, DP	I _{ILL}	– 1,0	1,0	
Входной ток утечки низкого уровня, мкА, на выводе: OSC_IN при: BYPASS = 1		– 1,0	1,0	
Статический ток потребления в режиме покоя (регулятор напряжения выключен), мкА	I _{CCS}	–	10	25, – 40
		–	20	85
Динамический ток потребления, мА	I _{OCC1}	–	120	25, 85, – 40
Выходная частота LSI RC-генератора, кГц	f _{O_LSI}	10	60	
Выходная частота HSI RC-генератора, МГц	f _{O_HSI}	6	10	
Выходная частота PLL, МГц максимальная минимальная	f _{O_PLL}	100 –	– 6	
Параметры АЦП				
Разрядность АЦП	E _{NADC}	12	–	25, 85, – 40
Дифференциальная нелинейность АЦП, единица младшего разряда	E _{DLADC}	– 1	2	
Интегральная нелинейность АЦП, единица младшего разряда	E _{ILADC}	– 3	3	
Ошибка смещения АЦП, единица младшего разряда	E _{OFFADC}	– 6	6	
Ошибка усиления АЦП, %	E _{GAINADC}	– 1	1	

Наименование параметра, единица измерения, режим измерения	Буквенное обозначение параметра	Норма параметра		Температура среды, °C
		не менее	не более	
Параметры ЦАП				
Разрядность ЦАП	E _{NDAC}	12	—	25, 85, — 40
Дифференциальная нелинейность ЦАП, единица младшего разряда	E _{DLDAC}	— 1	2	
Интегральная нелинейность ЦАП, единица младшего разряда	E _{ILDAC}	— 6	6	
Ошибка смещения ЦАП, мВ	E _{OFFDAC}	— 40	40	
Ошибка усиления ЦАП, %	E _{GAINDAC}	— 2	2	
Минимальное выходное напряжение ЦАП, В	U _{O_DAC min}	—	0,08	
Максимальное выходное напряжение ЦАП, В	U _{O_DAC max}	U _{REF(DAC)} - 0,08	—	
Компаратор				
Время включения компаратора, мкс	t _{ON_C} *	—	100	25, 85, — 40
Время задержки переключения компаратора, нс	t _{d_C} *	—	400	
* Значения временных параметров t _{ON_C} , t _{d_C} гарантируются в процессе проведения ФК на максимальной частоте				

Микросхемы устойчивы к воздействию статического электричества с потенциалом не менее 2000 В.

36 Предельно допустимые характеристики микросхемы

Таблица 487 – Предельно допустимые и предельные режимы эксплуатации микросхем

Наименование параметра, единица измерения	Буквенное обозначение параметра	Норма параметра			
		Предельно- допустимый режим		Предельный режим ¹⁾	
		не менее	не более	не менее	не более
Напряжение источника питания, В	U _{CC}	2,2	3,6	–	4,0
Напряжение источника питания при использовании USB, В		3,0	3,6	–	4,0
Напряжение питания аналоговое, В, (АЦП, ЦАП, компаратор)	U _{CCA} ²⁾	2,4	3,6	–	4,0
Напряжение питания батарейного домена, В	U _{CCB}	1,8	3,6	–	4,0
Входное напряжение низкого уровня, В, (при работе в цифровом режиме) на выводах: PA, PB, PC, PD, PE, PF, RESET, WAKEUP, SHDN, JTAG_EN	U _{IL}	0	0,8	– 0,3	–
на выводах: DN, DP		0	0,8	– 0,3	–
на выводе: OSC_IN в режиме генератора		0	0,8	– 0,3	–
Входное напряжение высокого уровня, В, на выводах: PD, PE (0-10)	U _{IH}	2,0	3,6	–	4,0
на выводах: PA, PB, PC, PE (11-15), PF, RESET, WAKEUP, SHDN, JTAG_EN		2,0	5,25	–	5,3
на выводах: DN, DP		2,0	3,6	–	4,0
на выводах: OSC_IN в режиме генератора		2,0	3,6	–	4,0
Выходной ток высокого уровня, мА, (при работе в цифровом режиме) на выводах: PA, PB, PC, PD, PE (0-5, 8-15), PF	I _{OH} ³⁾	– 6	–	– 10	–
на выводах PE (6, 7), STANDBY		– 3	–	– 10	–
на выводах: DN, DP		– 6	–	– 40	–
Выходной ток низкого уровня, мА, на выводах: PA, PB, PC, PD, PE, PF, STANDBY	I _{OL} ⁴⁾	–	6	–	10
на выводах: DN, DP		–	6	–	40
Частота следования импульсов тактовых сигналов, МГц	f _C	–	80	–	–
Частота следования импульсов тактовых сигналов HSE, МГц	f _{C_HSE}	2	16	–	–
в режиме кварцевого генератора		0	80	–	–
Частота следования импульсов тактовых сигналов LSE на входе осциллятора, кГц	f _{C_LSE}	32	33	–	–
в режиме кварцевого генератора		0	1 000	–	–
Частота следования импульсов тактовых сигналов PLL, МГц	f _{C_PLL}	6	16	–	–
Параметры ЦАП					
Напряжение верхней границы опорного напряжения ЦАП, В, на выводах: DAC1_REF, DAC2_REF; при: Cfg_M_REFx = 1	U _{REF(DAC)} ⁵⁾	2,4	U _{CCA}	–	–

Наименование параметра, единица измерения	Буквенное обозначение параметра	Норма параметра			
		Предельно- допустимый режим		Предельный режим ¹⁾	
		не менее	не более	не менее	не более
Резистивная нагрузка ЦАП, кОм	R _{LOAD}	10	–	–	–
Емкостная нагрузка ЦАП, пФ	C _{LOAD}	–	100	–	–
Параметры АЦП					
Напряжение нижней границы внешнего опорного напряжения АЦП, В, на АЦП1 или АЦП2 при ADC1_Cfg_M_REF=1	U _{ADC1_REF-}	0	U _{CCA-2,4}	– 0,3	–
Напряжение верхней границы внешнего опорного напряжения АЦП,В на АЦП1 или АЦП2 при Cfg_M_REF=1	U _{ADC0_REF+}	2,4	U _{CCA}	–	4,0
Диапазон напряжения внешнего опорного источника АЦП, В, U _{REF(ADC)} = U _{ADC0_REF+} – U _{ADC1_REF-}	U _{REF(ADC)}	2,4	U _{CCA}	–	–
Диапазон напряжения на входе АЦП, В	U _{AIN} ⁶⁾	U _{ADC1_REF-}	U _{ADC0_REF+}	– 0,3	4,0
Частота следования импульсов тактовых сигналов АЦП, МГц	f _{C_ADC}	–	14	–	–
Емкость нагрузки, пФ, на выводах: PA, PB, PC, PD, PE, PF, Standby	C _L	–	30	–	–
Число циклов записи/стирания данных, при: T = 85 °С	N _{PR}	10 000	–	–	–
Время хранения информации, лет, при: T = 25 °С	t _{GS}	25	–	–	–
при: T = 85 °С		10	–	–	–
1) Не допускается одновременное воздействие двух и более предельных режимов; 2) Допускается использование отдельного источника для питания аналоговых блоков, при этом его выходное напряжение не должно отличаться от U_{CC} более чем на ± 0,2 В; 3) Суммарный выходной ток высокого уровня I_{OH} не должен превышать 200 мА. Знак минус перед значением тока указывает на направление – вытекающий ток; 4) Суммарный выходной ток низкого уровня I_{OL} не должен превышать 200 мА; 5) При использовании внутреннего опорного напряжения U_{REF(DAC)} = U_{CCA}; 6) При использовании внутреннего источника опорного напряжения U_{ADC1_REF-} = 0 В, U_{ADC0_REF+} = U_{CCA}					

37 Справочные данные

Справочные параметры микросхем приведены в таблице 488.

Таблица 488 – Справочные данные

Наименование параметра, единица измерения, режим измерения	Буквенное обозначение параметра	Норма параметра		Температура среды, °C
		не менее	не более	
Статический ток потребления в режиме покоя (регулятор напряжения включен), мА, при U _{CC} = 3,6 В	I _{CCS1}	—	1,5	25, 85, – 40
Ток потребления батарейного домена, мкА, при U _{CC} = 0 В	I _{CC_B}	—	5	
Динамический ток потребления в режиме ожидания SLEEPDEEP, мА, при U _{CC} = 3,6 В, f _C = f _{O_LSI}	I _{OCC2}	—	2	
Время установления сигналов PVD и PVBD, мкс	t _{SU(PVD)} t _{SU(PVBD)}	—	2	
Выходное напряжение регулятора LDO, В при U _{CC} = 3,6 В, I _{OL} = 80 мА	U _{O_LDO}	1,62	1,98	
Гистерезис портов ввода/вывода, мВ, на выводах: PA-PF при: - ModeRX = 0 - ModeRX = 1	ΔU _{TH(PA-PF)}	100 200	400 500	25, 85, – 40
Длительность фронта переключения выходных сигналов, нс, на выводе: STANDBY, C _I = 30 пФ на выводах: PA – PF при U _{CC} = 2,2 В, - PowerTX = 00, C _I = 50 пФ - PowerTX = 01, C _I = 50 пФ - PowerTX = 10, C _I = 50 пФ - PowerTX = 11, C _I = 50 пФ - PowerTX = 11, C _I = 30 пФ на выводах: DN, DP при U _{CC} = 3,0 В, - Full Speed, C _I = 50 пФ - Low Speed, C _I = 600 пФ	t _{W(STANDBY)}	—	10	
	t _{W(PA-PF)}	—	10	
		—	100	
		—	20	
		—	10	
—		5		
t _{W(DN, DP)}	— —	15 300		
Сопротивление внутренних программируемых резисторов доопределения, кОм	R _{PULL}	20	100	25, 85, – 40
Компаратор				
Напряжение смещения компаратора, мВ, при U _{CC} = 3,6 В	U _{IO_C}	—	±0,5	
Гистерезис компаратора, мВ, при U _{CC} = 3,6 В	ΔU _{TH_C}	8	12	
Напряжение внутреннего опорного источника компаратора, В, при: U _{CC} = 3,6 В	U _{REF_C}	1,17	1,23	– 40

Наименование параметра, единица измерения, режим измерения	Буквенное обозначение параметра	Норма параметра		Температура среды, °C
		не менее	не более	
Тактовые частоты и генераторы				
Время установления сигнала HSIRDY относительно HSION, мкс, при U _{CC} = 2,2 В	t _{SU(HSI)}	—	1	25, 85, – 40
Время установления сигнала LSIRDY относительно LSION, мс, при U _{CC} = 2,2 В	t _{SU(LSI)}	—	80	
Время установления сигнала HSERDY относительно HSEON, мкс, при U _{CC} = 2,2 В	t _{SU(HSE)}	2048/f _{C_HSE}	—	
Время установления сигнала LSERDY относительно LSEON, мкс, при U _{CC} = 2,2 В	t _{SU(LSE)}	4096/ f _{C_LSE}	—	
Время установления сигнала PLLRDY относительно PLLON, мкс, при U _{CC} = 2,2 В	t _{SU(PLL)}	—	100	
Длительность сигнала сброса, мкс, при U _{CC} = 2,2 В	t _{W(СБРОС)}	20	—	
Время запуска после сброса по POR, мс	t _{POR}	—	6	
Минимальная длительность сигнала низкого уровня на входе nRESET для формирования сигнала «Сброс», нс	t _{W(nRESET)min}	200	—	
Максимальная длительность сигнала низкого уровня на входе nRESET, при котором не формируется сигнал «Сброс», нс	t _{W(nRESET)max}	—	10	
АЦП				
Время выборки заряда АЦП, нс, при U _{CC} = 3,6 В	t _{A_ADC}	—	4/f _{C_ADC}	25, 85, – 40
Время преобразования АЦП, нс при U _{CC} = 3,6 В	t _{AO_ADC}	—	28/f _{C_ADC}	
Ток потребления по входу внешней верхней границы опоры АЦП, мкА при - ADC1_Cfg_M_REF = 1 или - ADC2_Cfg_M_REF = 1	I _{ADC0_VREF+}	—	50	
Ток потребления по входу внешней нижней опоры АЦП, мкА при - ADC1_Cfg_M_REF = 1 или - ADC2_Cfg_M_REF = 1	I _{ADC0_VREF-}	– 50	—	
Ток потребления по питанию АЦП, мА при: U _{CCA} = 3,6 В, f _{C_ADC} = 14 МГц	I _{OCCADC}	—	3	
Минимальная частота преобразования АЦП, кГц	f _{C_ADCMIN}	10	—	

Наименование параметра, единица измерения, режим измерения	Буквенное обозначение параметра	Норма параметра		Температура среды, °C
		не менее	не более	
ЦАП				
Время установления сигнала ЦАП, мкс, при U _{CC} = 3,6 В, C _I = 50 пФ, R _I = 10 кОм	t _{SU(DAC)}	—	5,2	25, 85, – 40
Время включения ЦАП, мкс, при U _{CC} = 2,4 В	t _{ON_DAC}	—	10	
Ток потребления по входу опоры, мкА при Cfg_M_REF0 = 1	I _{DAC1_VREF}	—	500	
Ток потребления по входу опоры, мкА при Cfg_M_REF1 = 1	I _{DAC2_VREF}	—	500	
Ток потребления ЦАП, мА при отсутствии нагрузки	I _{OCCDAC}	—	2	
USB				
Сопротивление резистора между линиями DN, DP и питанием, кОм при - D-PULLUP = 1 - D+PULLUP = 1	R _{DN-UCC}	1	2	25, 85, – 40
	R _{DP-UCC}	1	2	
Сопротивление резистора между линиями DN, DP и шиной «Общий», кОм при - D-PULLDOWN = 1 - D+PULLDOWN = 1	R _{DN-GND}	10	20	
	R _{DP-GND}	10	20	
Согласующее сопротивление на линиях DN, DP, Ом	R _{DN} R _{DP}	14	34	

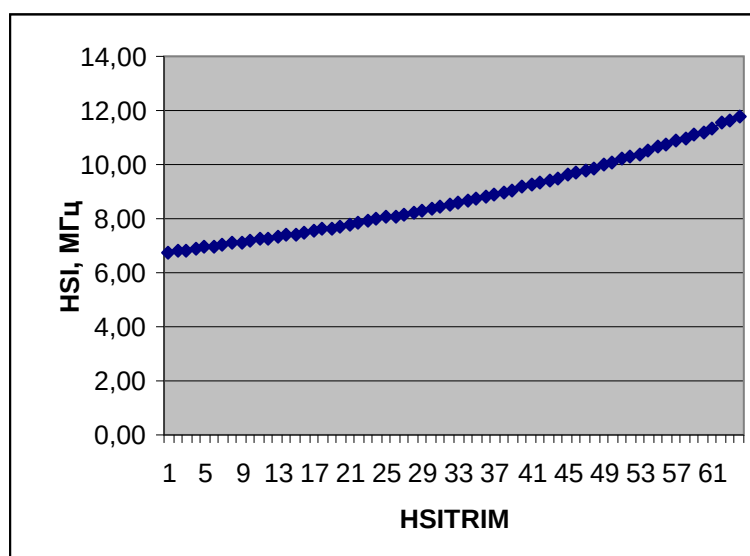


Рисунок 139 – Зависимость частоты HSI от значения HSITRIM

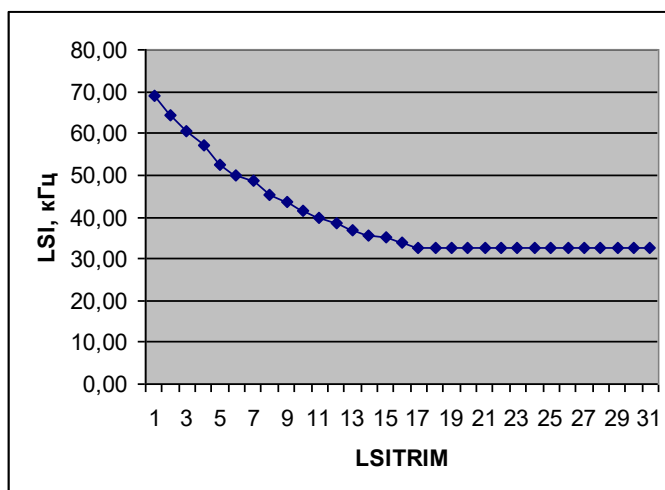


Рисунок 140 – Зависимость частоты LSI от значения LSITRIM

38 Типовые зависимости

Раздел находится в разработке.

39 Габаритный чертеж микросхемы

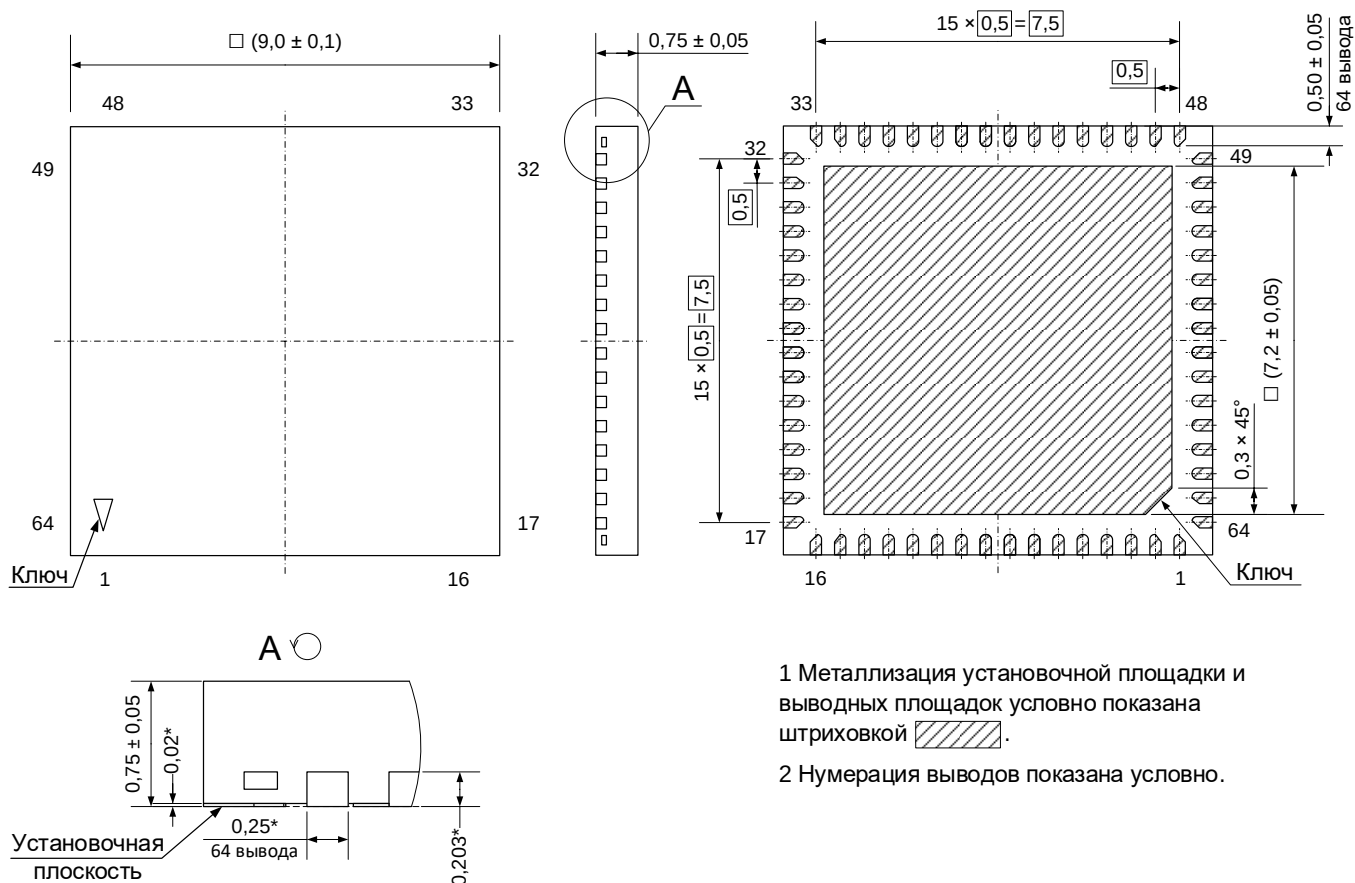


Рисунок 141 – Габаритный чертеж микросхемы K1986BE92FI в корпусе QFN64 9×9×0,75 (0,5)

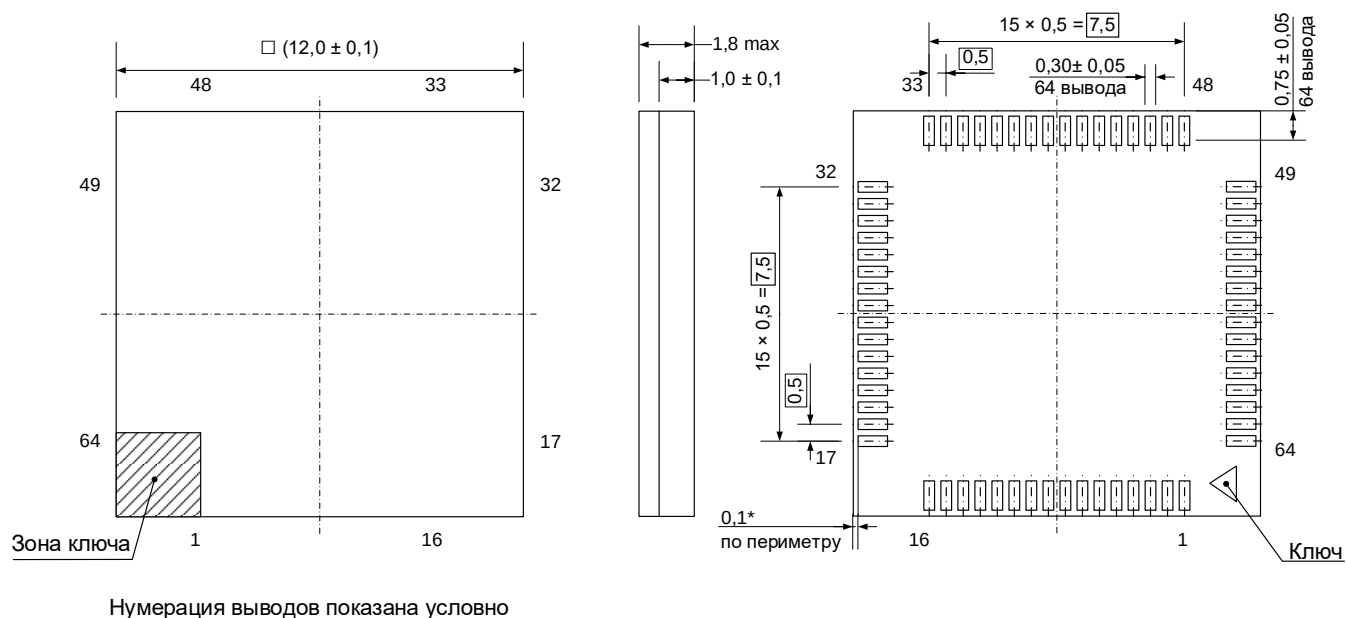


Рисунок 142 – Габаритный чертеж микросхемы K1986BE92FII в корпусе QFN64 12×12×1,6 (0,5)

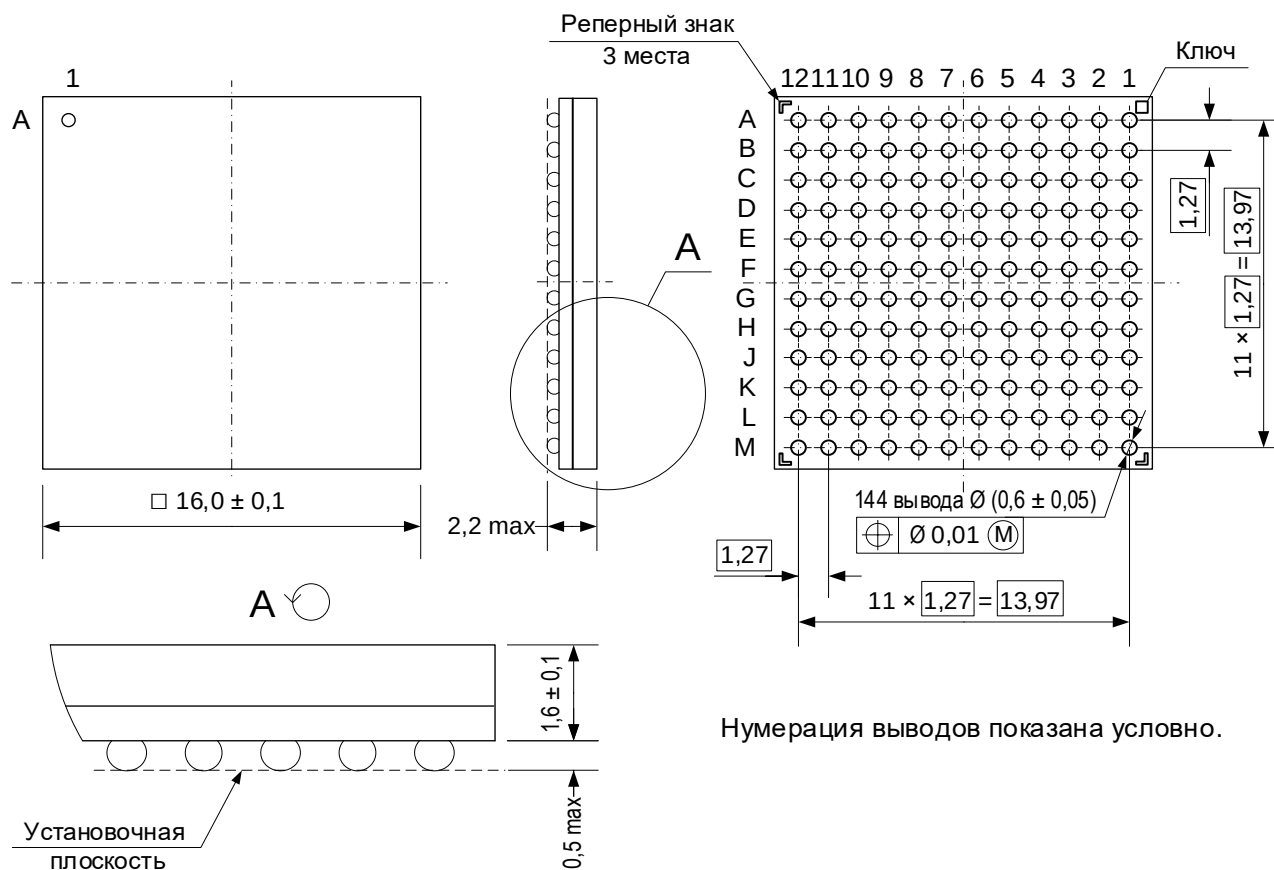


Рисунок 143 – Габаритный чертеж микросхемы K1986BE94GI
в корпусе BGA144 16×16×1,6 (1,27)A

40 Информация для заказа

Обозначение	Маркировка	Тип корпуса	Температурный диапазон, °С
K1986BE92FI	MDR1211FI	QFN64 9×9×0,75 (0,5)	от –40 до 85
K1986BE92FII	MDR1211FII	QFN64 12×12×1,6 (0,5)	от –40 до 85
K1986BE94GI	MDR1209GI	BGA144 16×16×1,6 (1,27)A	от –40 до 85

Условное обозначение микросхем при заказе в договоре на поставку и в конструкторской документации другой продукции должно состоять из:

- наименование изделия – микросхема;
- обозначения типа (типономинала);
- обозначения технических условий ТСКЯ.431000.003ТУ;
- обозначения спецификации ТСКЯ.431296.040СП.

Пример обозначения микросхем:

Микросхема K1986BE92FI – ТСКЯ.431000.003ТУ, ТСКЯ.431296.040СП.

Лист регистрации изменений

№ п/п	Дата	Версия	Краткое содержание изменения	№№ изменяемых листов
1	02.11.2023	1.0.0	Введена впервые	
2	13.11.2023	1.0.1	Добавлена рекомендация по подключению металлизации обратной стороны корпуса микросхем K1986BE92FI	53
3	16.11.2023	1.0.2	Уточнено обозначение корпуса типа BGA	1, 508, 509
4	01.02.2024	1.1.0	Добавлено описание тестового ТАР-контроллера	81, 80
5	08.02.2024	1.1.1	Добавлено обозначение металлизации на ГЧ корпуса QFN64 9×9×0,75 (0,5)	520
6	17.04.2024	1.1.2	На УГО добавлены выводы J6, L6	20
7	29.05.2024	1.2.0	Исправлен ГЧ корпуса BGA144 16×16×1,6 (1,27)A (Рисунок 143)	521
8	26.06.24	1.3.0	Изменена масса м/сх K1986BE92FI, K1986BE92FII. Пункт 8.1 – коррекция формулировок. Рисунок 7 заменен. Пункт 28.8.4 – коррекция формулировок. Подпункт 30.4.5.3 поправлен. Таблица 438 скорректирована. Пункт 32.2.6 скорректирован. Раздел 37 обновлен. Рисунок 138 поправлен. Таблица 486 – исправлена ошибка в обозначении электрического параметра. Раздел 40 – добавлена информация	1 58 58 371 424 474 481 504 - 510 504 515 522
9	07.10.2024	1.3.1	Таблица 3 – исправлена. Таблица 66 – исправлена. Пункт 14.1.4 – добавлена информация. Подпункт 14.1.4.3 – исправлен. Таблица 486 – исправлена верхняя граница параметра U _{POR} с 2,1 на 2,18 В. Раздел 37 – добавлено значение теплового сопротивления кристалл-корпус для микросхем K1986BE92FI	23 162 164 164 511 515
10	05.02.2025	1.3.2	Значение теплового сопротивления кристалл-корпус перенесено в основные параметры; Добавлен уровень чувствительности к влаге MSL; Раздел 7 – дополнен; Таблица 11 – исправлен размер "SYSTEM RAM Bit Band Region" и "PERIPHERAL Bit Band Region"; Пункт 11.2.3 – добавлено примечание; Раздел 16 – дополнен; Таблица 128 – добавлено описание PORT RXTX[15:0]; Таблица 215, таблица 236, таблица 286, таблица 304, таблица 369 – добавлено примечание Таблица 332 – исправлена опечатка;	2 3 54 72 86 177 203 244, 267, 311, 326, 406 341

№ п/п	Дата	Версия	Краткое содержание изменения	№№ изменяемых листов
			Пункт 31.1.2 – скорректирован Подраздел 31.3 – дополнен; Рисунок 142 – исправлен	460 462 520
11	11.02.2025	1.3.3	Рисунок 142 – исправлен	520
12	04.04.2025	1.3.4	Заменено «IWDТ» на «IWDG», «WWDТ» на «WWDG»; Таблица 11, Таблица 99 – заменено «SPIx» на «SSPx»	По тексту 72, 185
13	07.07.2025	1.4.0	Изменен уровень MSL для микросхем K1986BE92FI. Дополнен раздел Указания по применению и эксплуатации	1 54
14	04.09.2025	1.4.1	Рисунок 29 скорректирован; Подраздел 17.1 скорректирован; Таблицы 111, 123 скорректированы; Таблица 114 – скорректировано описание RTC_CAL[7:0], RTC_EN; Таблица 120 – скорректировано описание RTC_PRL[19:0]; Таблица 122 – скорректировано описание RTC_ALRM[31:0]; Таблица 124 – скорректировано описание WEC, ALRF, SECF	190 190 193,198 194 197 197 198
15	13.11.2025	1.4.2	В раздел 7 добавлена информация о теплостойкости; Таблица 157 – исправлены адреса MDR_USB->HRXFDC, MDR_USB->SEP[x].RXFDC; Раздел 27 обновлен	54 229 336 – 342
16	24.04.2026	1.4.3	Исправлено значение теплового сопротивления для микросхемы K1986BE92FI; Добавлен раздел 6; Раздел 7 скорректирован; Таблица 85 – дополнено описание PLL_CPU_RDY, PLL_USB_RDY; Таблица 138 – скорректировано описание PORT SHM[15:0]; Пункты 23.7.1, 23.7.3 дополнены Таблица 419 – скорректирован доступ для всех бит; Таблица 488 – исправлена норма параметра t _{SU} (HSE), t _{SU} (LSE)	2 53 54 180 205 296, 296 454 515